

MySQL大智小技

精选五大主题

图解系列

新特性系列

技术分享

故障系列

DBLE系列

爱可生开源社区

有深度的MySQL社区

MySQL 大智小技

大智小技——大智者，宏观把握，对技术产生的背景以及相应的原理应用了然于胸；小技者，具体细微，不论什么问题一定要剖析入骨，捋清脉络。拥有了大智小技，面对问题轻松自如，一目了然，一锤定音。



扫码获取本书电子版

前言

自爱可生开源社区开始运营以来，已累计发布 3 款开源软件（DBLE/DTLE/TXLE）；举办 11 场各类技术活动；发布 300 余篇技术文章；录制播出 2 套视频教程。未来，社区将持续不断的输出新的技术内容。

《大智小技》由社区团队在所发布的文章中精选出 87 篇编辑成册。内容主要来自爱可生内部团队和社区支援者们的踊跃投稿。书籍内容包括：图解系列、新特性系列、故障分析、技术分享和 DBLE 系列等五大主题。每一篇都是独立的内容，方便翻阅。文章内容从性能优化到事例分析，从版本特性到图解原理等，与 MySQL 有关的方方面面。文章从一个个事件引出一个个知识点的扩展，讲解精简凝练，正所谓“大智小技”。

本书既是社区一年来的工作积累，也是各方友人参与社区发展的记忆留存。希望当您翻看本书时，能回忆起 2019 年的美好时光。

致 谢

2019 年是爱可生开源社区发展的元年。首先是得到爱可生内部团队的大力支持，是你们为社区提供了大量的优质内容并积极参与线下活动，一起为社区的发展献计献策。我们今年与多家数据库行业知名社区的深度合作，共同举办了多场线下活动，增加了各社区间用户的互动和信息共享；当然更少不了社区志愿者们对社区的关注与支持。

此致敬礼！

爱可生开源社区

2019.12.31

爱可生内部团队致谢名单（排名不分先后）

DBLE 团队
DTLE 团队
TXLE 团队
DBA 技术团队
数据库产品研发团队

社区合作致谢名单（排名不分先后）

3306 π 社区
IMG 社区
dbaplus
高效运维

社区作者致谢名单（排名不分先后）

杨奇龙 · 有赞
王航威 · 有赞
钟悦 · 工商银行
蒋乐兴 · 腾讯
高鹏 · 浩鲸科技
苏仕祥 · 浩鲸科技
田帅萌 · 京东金融
任坤 · 金山软件
高鹏 · 易极付科技
网友 · 阿里巴巴
孟维克 · 宝尊电商
王继顺 · 宝尊电商

目 录

第 1 章 图解系列

【缺陷分析】Table cache 导致 MySQL 崩溃·····	001
【缺陷分析】半同步下多从库复制异常·····	005
【原理解析】设置字符集的参数控制了哪些行为·····	010
【原理解析】MySQL 组提交(group commit) ·····	013
【原理解析】MySQL 固定的 server_id 导致数据丢失·····	019
【原理解析】XtraBackup 全量备份还原·····	024
【原理解析】XtraBackup 增量备份还原·····	035
【原理解析】MySQL DDL 为什么成本高? ·····	041
【原理解析】MySQL 为表添加列是怎么"立刻"完成的? ·····	050
【原理解析】XtraBackup 备份恢复时为什么要加 apply-log-only 参数? ·····	057

第 2 章 新特性系列

MySQL 5.7 升级到 MySQL 8.0 的注意事项·····	067
MySQL MGR “一致性读写”特性解读·····	069
MySQL 8.0 通用表达式·····	073
MySQL 8.0 索引特性 1-函数索引·····	078
MySQL 8.0 索引特性 2-索引跳跃扫描·····	085
MySQL 8.0 索引特性 3 -倒序索引·····	089
MySQL 8.0 索引特性 4-不可见索引·····	093
MySQL 8.0 新增 HINT 模式·····	097
MySQL 8.0 直方图·····	103
MySQL 8.0 json 到表的转换·····	109
MySQL 8.0 窗口函数详解·····	114
MySQL 8.0 动态权限·····	122
MySQL 8.0 资源组·····	125
MySQL 8.0 正则替换·····	128
MySQL 8.0 对 count(*)的优化·····	131
MySQL 8.0 错误日志增强特性·····	136
实战 MySQL 8.0.17 Clone Plugin·····	141
MySQL 8.0.16 开始支持 check 完整性·····	149
MySQL 8.0: 字符集从 utf8 转换成 utf8mb4·····	152

第 3 章 故障系列

多从库时半同步复制不工作的 BUG 分析·····	155
JDBC 与 MySQL 临时表空间的分析·····	161
MySQL Insert 加锁与死锁分析·····	167
MySQL 优化：为什么 SQL 走索引还那么慢？·····	171
记一次 MySQL semaphore crash 的分析·····	175
GDB 定位 MySQL5.7 特定版本 hang 死的故障分析#92108·····	179
delete 大表 slave 回放巨慢的问题分析·····	186
event_scheduler 导致复制中断的故障分析·····	192
快速定位令人头疼的全局锁·····	204
FTWRL 一个奇怪的堵塞现象和其堵塞总结·····	208
tcmalloc 解决 mysqld 实例引发的 cpu 过高问题·····	212
MySQL 一次奇怪的故障分析·····	223
产生大量小 relay log 的故障一例·····	237
timestamp 时区转换导致 CPU %sy 高的问题·····	246
delete 语句引发大量 sql 被 kill 问题分析·····	252
MGR 相同 GTID 产生不同 transaction 故障分析·····	258

第 4 章 技术分享

使用 OGG 实现 Oracle 到 MySQL 数据平滑迁移·····	265
基于 Xtrabackup 及可传输表空间实现多源数据恢复·····	276
MySQL 主从复制延时常见场景及分析改善·····	282
HASH_SCAN BUG 之迷惑行为大赏·····	288
我对 MySQL 隔离级别的剖析·····	290
MySQL 碎片问题·····	300
MySQL 默认值选型（是空，还是 NULL）·····	303
MySQL 大对象一例·····	308
死锁分析案例·····	312
innodb-cluster 扫盲-安装篇·····	319
控制 mysqldump 导出的 SQL 文件的事务大小·····	330
MySQL 删库不跑路·····	334
基于 WRITESET 的并行复制方式·····	342
gh-ost 原理剖析·····	352
MySQL 跨实例 copy 大表解决方案·····	360

InnoDB 表空间加密·····	362
slave_relay_log_info 表认知的一些展开·····	372
MySQL 慢查询记录原理和内容解析·····	376
MySQL 的 join_buffer_size 在内连接上的应用·····	388
MySQL 多源复制场景分析·····	393
MySQL show processlist Time 为负数的思考·····	396
如何使用 dbdeployer 快速搭建 MySQL 测试环境·····	402
巧用 binlog Event 发现问题·····	409

第 5 章 DBLE 系列

开源分布式中间件 DBLE 快速入门指南·····	415
DBLE schema.xml 配置解析·····	425
DBLE rule.xml 配置解析·····	430
DBLE server.xml 配置解析·····	432
深度分析 MyCat 与 DBLE 的对比性能调优·····	437
基于分布式中间件的 SQL 改造指南·····	443
开源分布式中间件 DBLE 容器化快速安装部署·····	451
DBLE 如何实现 Prepared Statement 协议·····	454
DBLE 自定义拆分算法·····	459
DBLE 与 MyCat 分片算法对比: hash 分片·····	466
DBLE 与 MyCat 分片算法对比: stringhash 分片·····	469
DBLE 与 MyCat 分片算法对比: enum 分片·····	473
DBLE 与 MyCat 分片算法对比: numberrange 分片·····	475
DBLE 与 MyCat 分片算法对比: patternrange 分片·····	477
DBLE 与 MyCat 分片算法对比: date 分片·····	480
DBLE 与 MyCat 分片算法系列总结·····	483
DBLE 和 Mycat 跨分片查询结果不一致案例分析·····	485
ZooKeeper 快速部署 DBLE 集群环境·····	492
DBLE 沿用 jumpstringhash 移除 Mycat 一致性 hash 原因解析·····	497

图解系列

本系列由爱可生内部团队的图解“三剑客”联合出品。目前累计发文 10 篇，虽然数量上在五个主题中最少，但每篇创作过程都相对复杂。从策划到成文，每个步骤都一丝不苟，内容严谨但不失活泼，篇篇都堪称精品。真正做到了寓教于乐，寓学于趣。

微信扫一扫读原文



【缺陷分析】Table cache 导致 MySQL 崩溃

作者：黄炎 王悦 周海鸣

本文分析的缺陷是 MySQL Bug #89126，其主要现象是：在使用很多表的数据库中，执行 create table 会导致数据库崩溃。

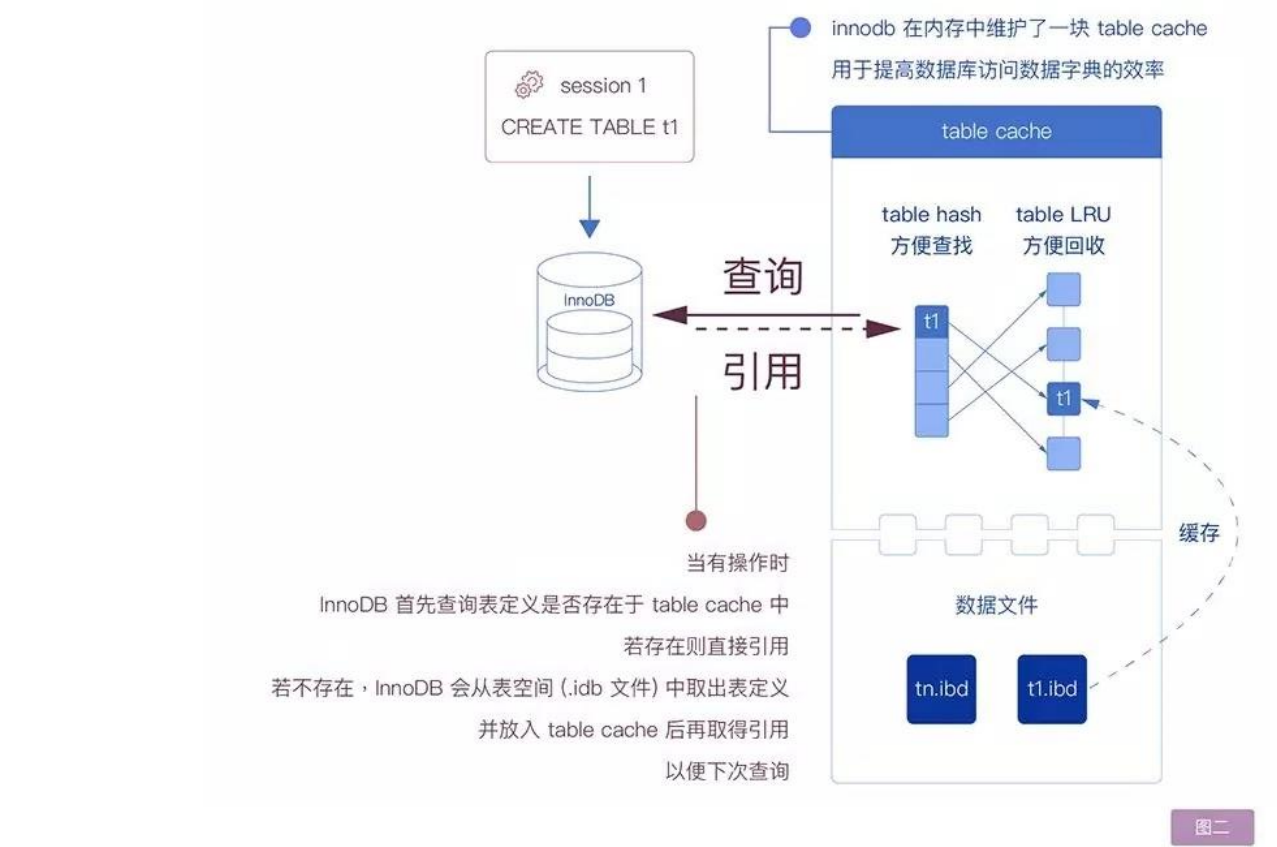
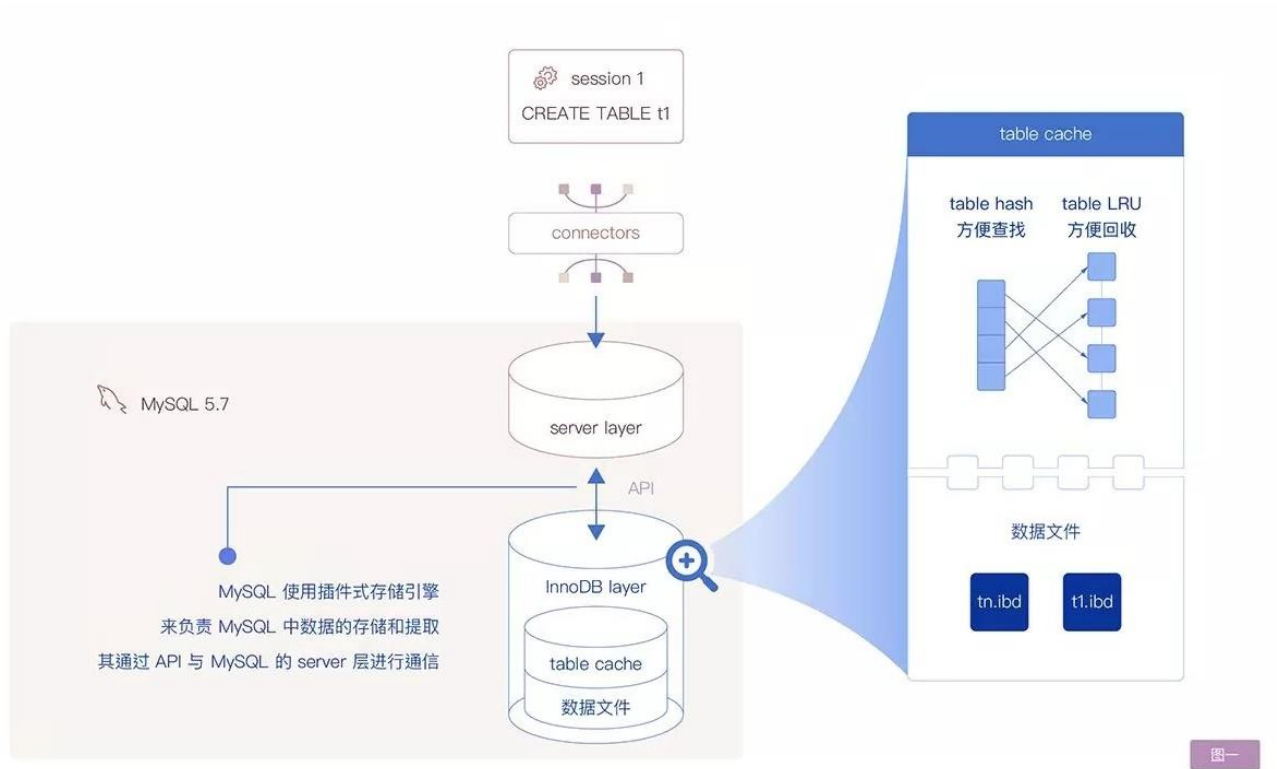
1 缺陷的现象

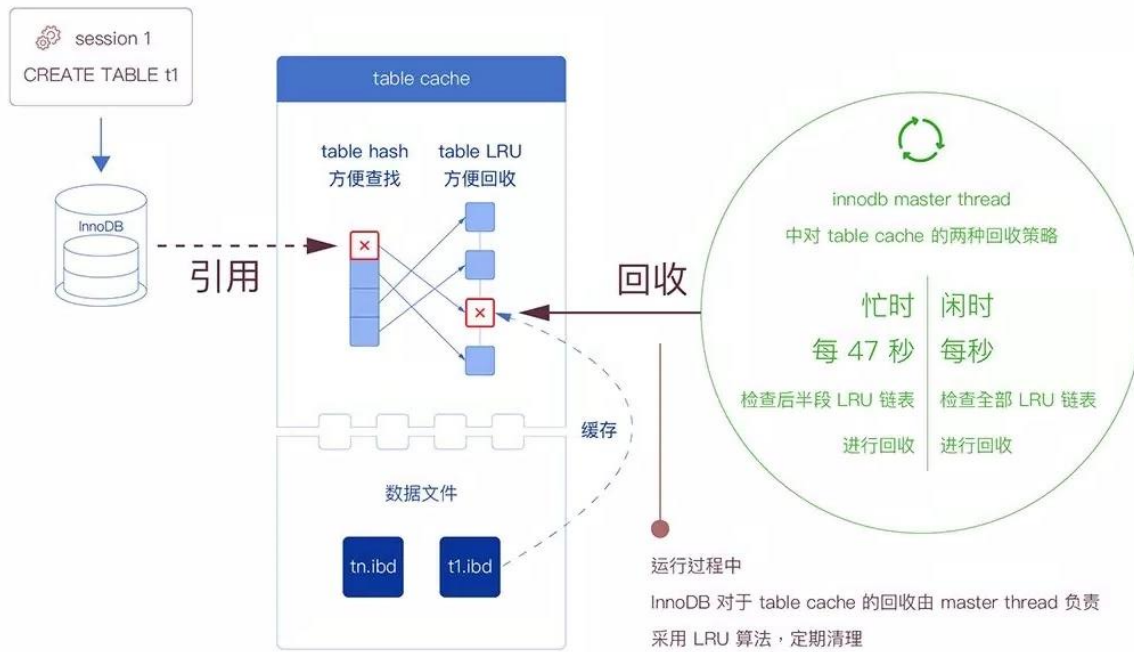
1. 在 MySQL 中创建大量表，填满 InnoDB 的表定义缓存（缓存的大小配置参看后文）。
2. 继续执行 create table ...，有一定概率使得 MySQL 在如下堆栈或类似堆栈崩溃。

```
/data/app/mysql/bin/mysqld(my_print_stacktrace+0x35)[0xf1dff5]
/data/app/mysql/bin/mysqld(handle_fatal_signal+0x4a4)[0x79d3b4]
/lib64/libpthread.so.0(+0xf850)[0x7f8d7b81f850]
/data/app/mysql/bin/mysqld(_Z30dict_index_set_merge_thresholdP12dict_index_tm+0x2ad)[0x11d60bd]
/data/app/mysql/bin/mysqld(_Z32innobase_parse_hint_from_commentP3THDP12dict_table_tPK11TABLE_SHARE+0x6b2)[0xfe8f22]
/data/app/mysql/bin/mysqld(_ZN19create_table_info_t24create_table_update_dictEv+0x1e0)[0xfed290]
/data/app/mysql/bin/mysqld(_ZN11ha_innpart6createEPKcP5TABLEP24st_ha_create_information+0x6cb)[0x1005a0b]
/data/app/mysql/bin/mysqld(_Z15ha_create_tableP3THDPKcS2_S2_P24st_ha_create_informationbb+0x2a3)[0x7e6d63]
/data/app/mysql/bin/mysqld(_Z16rea_create_tableP3THDPKcS2_S2_P24st_ha_create_informationR4ListI12Create_fieldEjP6st_keyP7handlerb+0x11a)
/data/app/mysql/bin/mysqld[0xd58917]
/data/app/mysql/bin/mysqld(_Z26mysql_create_table_no_lockP3THDPKcS2_P24st_ha_create_informationP10Alter_infojPb+0xe7)[0xd591e7]
/data/app/mysql/bin/mysqld(_Z18mysql_create_tableP3THDP10TABLE_LISTP24st_ha_create_informationP10Alter_info+0x9f)[0xd5998f]
/data/app/mysql/bin/mysqld(_Z21mysql_execute_commandP3THDb+0x4ed2)[0xc7622]
/data/app/mysql/bin/mysqld(_Z11mysql_parseP3THDP12Parser_state+0x3a5)[0xc77a15]
/data/app/mysql/bin/mysqld(_Z16dispatch_commandP3THDPK8COM_DATA19enum_server_command+0x116d)[0xc78bed]
/data/app/mysql/bin/mysqld(_Z10do_commandP3THD+0x194)[0xc796e4]
/data/app/mysql/bin/mysqld(handle_connection+0x29c)[0xdc5a0c]
/data/app/mysql/bin/mysqld(pfs_spawn_thread+0x174)[0xf7a804]
/lib64/libpthread.so.0(+0x7806)[0x7f8d7b817806]
/lib64/libc.so.6(clone+0x6d)[0x7f8d7a59064d]
```

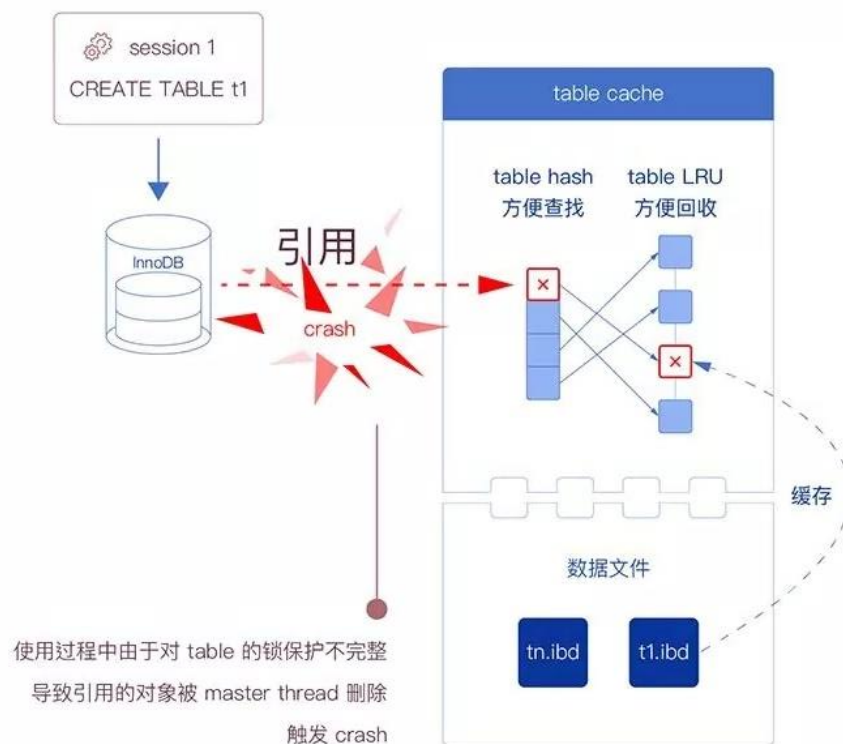
图解MySQL

2 缺陷的原理图解





图三



图四

3 缺陷的原理说明

1. 图 2, `create table ...` 向缓存中插入了新的表定义, 并持有了表定义的引用, 但代码中又将表定义释放了。
2. 图 3, InnoDB 在此时刚好进行缓存回收, 回收掉了已经释放的表定义。
3. 图 4, `create table ...` 继续进行, 使用了表定义的引用, 由于使用了被回收的内存地址, MySQL 继而崩溃。
4. 对缺陷的修复: 第 1 步的行为变更为不释放表定义, 从而在第 2 步中表定义不会回收 (目前这个修复方法尚未被官方接受)。

4 相关的知识点

1. 表定义缓存 (table cache): InnoDB 会对表定义进行缓存, 以减少不停读取表定义文件造成的 IO 压力。
2. 缓存的内部同时维护了两个数据结构:
 - a) Hash 结构, 方便缓存的查找
 - b) LRU (Least Recently Used) 结构, 方便缓存的回收
3. InnoDB 对缓存进行定时回收, 回收策略分为 忙时 和 闲时 两种策略, 用于减少回收动作造成的负担。
4. 缓存的容量可临时超过其限制大小。这个设计与定时回收有关。
5. 缓存的容量上限通过参数 `table_definition_cache` 设置:
 - a) 参数的最小值是 400
 - b) 默认值是通过以下公式计算得出: $\min(400 + \text{table_open_cache}/2, 2000)$

5 配置建议

除非有上万张表, 一般建议设置 `table_definition_cache` 值略大于表数量, 使缓存能够容纳所有的表定义。

扩展阅读

1. 缺陷描述:

<https://bugs.mysql.com/bug.php?id=89126>

2. 描述如何打开和释放表定义的 MySQL 官方文档:

<https://dev.mysql.com/doc/refman/5.7/en/table-cache.html>



【缺陷分析】半同步下多从库复制异常

作者：黄炎 王悦 周海鸣

本文分析的缺陷是 MySQL bug#89370，其主要的现象是：配置半同步(复制)到多个从库，部分从库在一段时间内无法复制数据，但所有复制状态均正常。

1 缺陷的复现

MySQL 版本：5.7.16, 5.7.17, 5.7.21

1. 配置半同步一个 master 两个 slave，设置 master 的
rpl_semi_sync_master_wait_for_slave_count=2，保持一定数据压力
2. 检查 master 的
rpl_semi_sync_master_status 状态为 ON，确保半同步没有退化为异步
3. 设置 master 的
rpl_semi_sync_master_wait_for_slave_count=1
4. 重启一个 slave “stop slave; start slave”

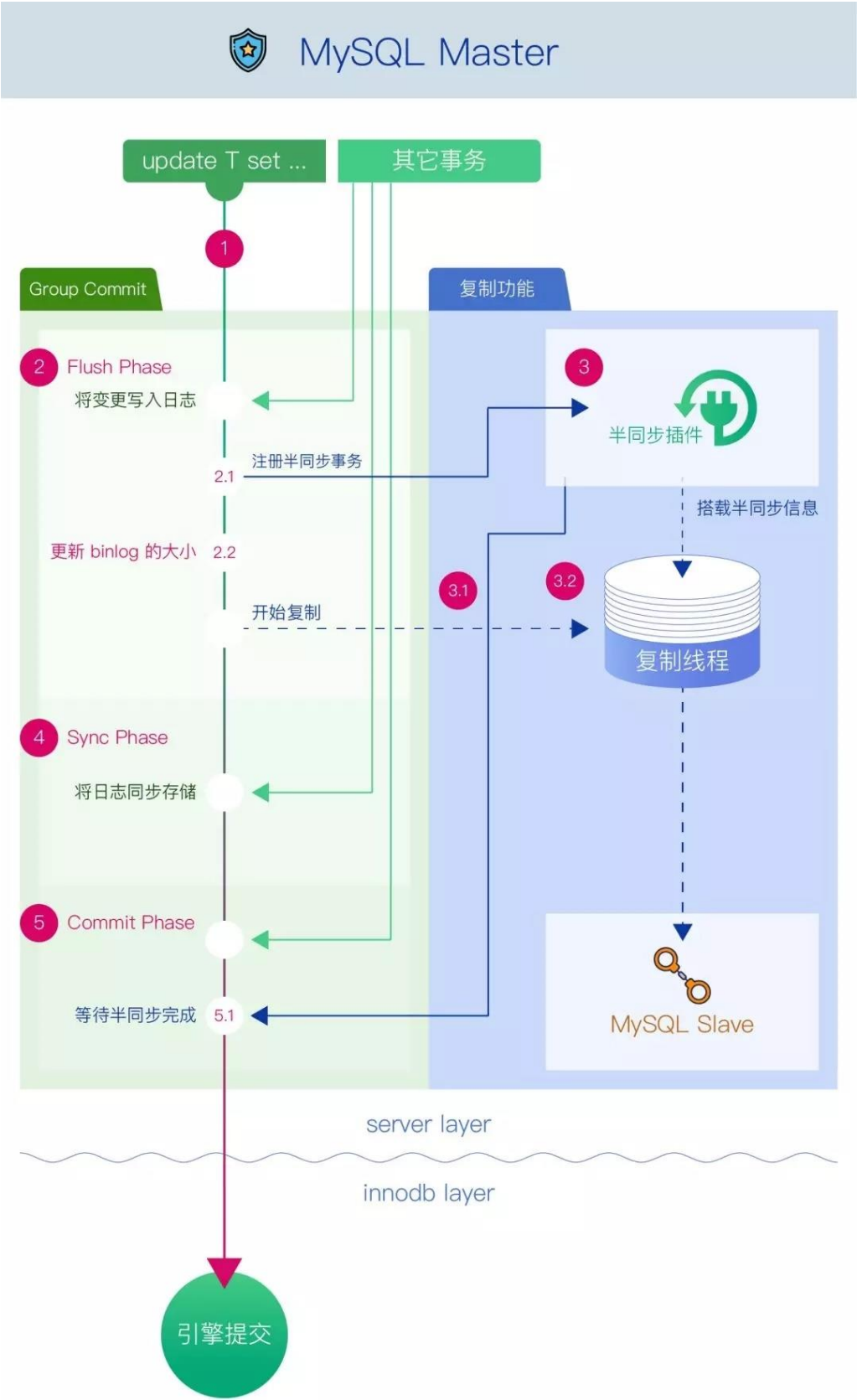
可以观察到步骤 4 中重启的那个 slave 长达数分钟不会有 master 的复制数据流入，但查看复制状态均正常。

2 缺陷的原理图解

下图中按序解析了 MySQL 半同步插件在 binlog group commit 中扮演的角色：

binlog group commit 分为三个阶段：

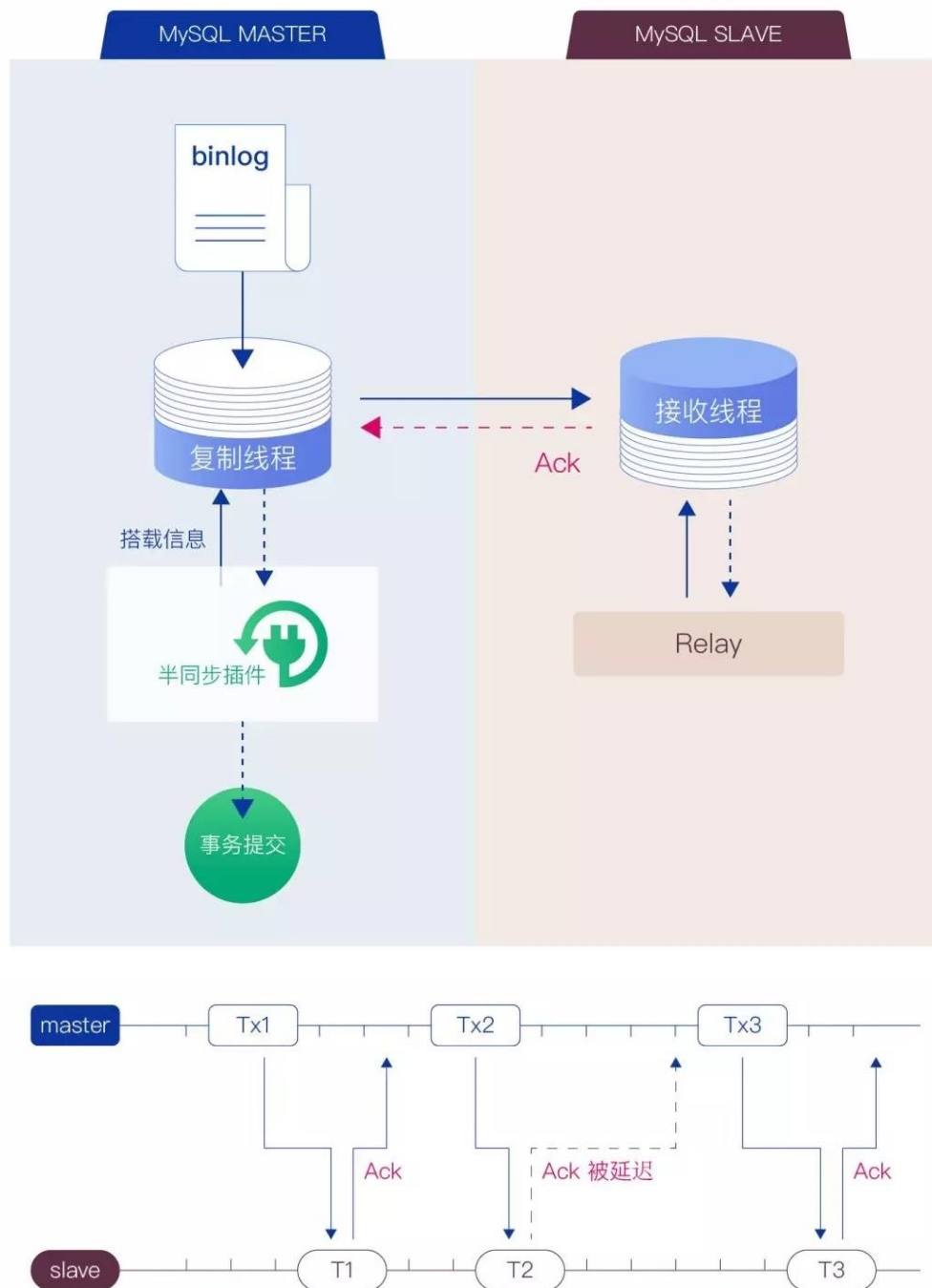
1. Flush Phase(图中序号 2)： 将一组事务写入 binlog 缓存区，向半同步插件注册事务(图中序号 2.1)，更新 binlog 文件位置信息(图中序号 2.2)
2. Sync Phase(图中序号 4)： 对 binlog 做 fsync 操作，将一组事务一起刷入磁盘
3. Commit Phase(图中序号 5)： 等待半同步完成，在引擎层提交事务



图一：描述了半同步复制的大致流程

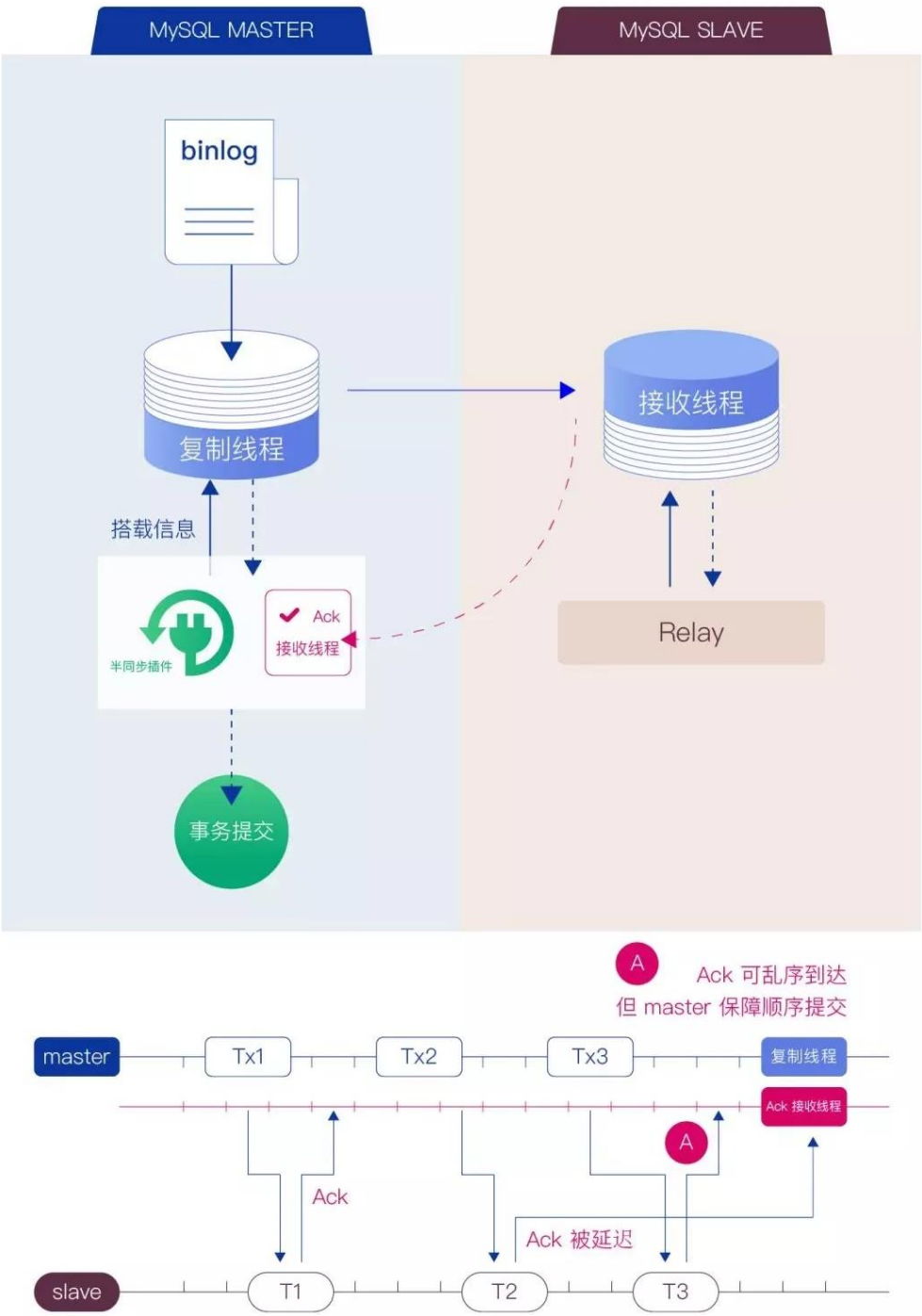
半同步插件：

1. 图中序号 3: binlog 完成位置信息更新后，通过复制线程读取 binlog 文件，将 event 发送给 slave。
 2. 图中序号 5.1: 将事务复制完成信息返回 master，master 根据该信息确定是否可以提交事务
- ☐ MySQL 5.5 版本后引入了半同步插件，解决传统异步复制在 master 节点宕机时可能出现的数据丢失。
 - ☐ 开启半同步能够保证 master 接收到的事务，在得到至少一个 slave 接收确认之后再返回给客户端。
 - ☐ MySQL 5.6 版本之前存在一个著名的 bug#13669，在开启 binlog 时为了保证引擎层与 binlog 的提交顺序一致，使得 group commit 机制失效。从 5.6 版本后才真正引入了 binlog 的 group commit，在保证引擎层和 binlog 事务最终一致的情况下，大幅提高了高并发场景下的处理性能。



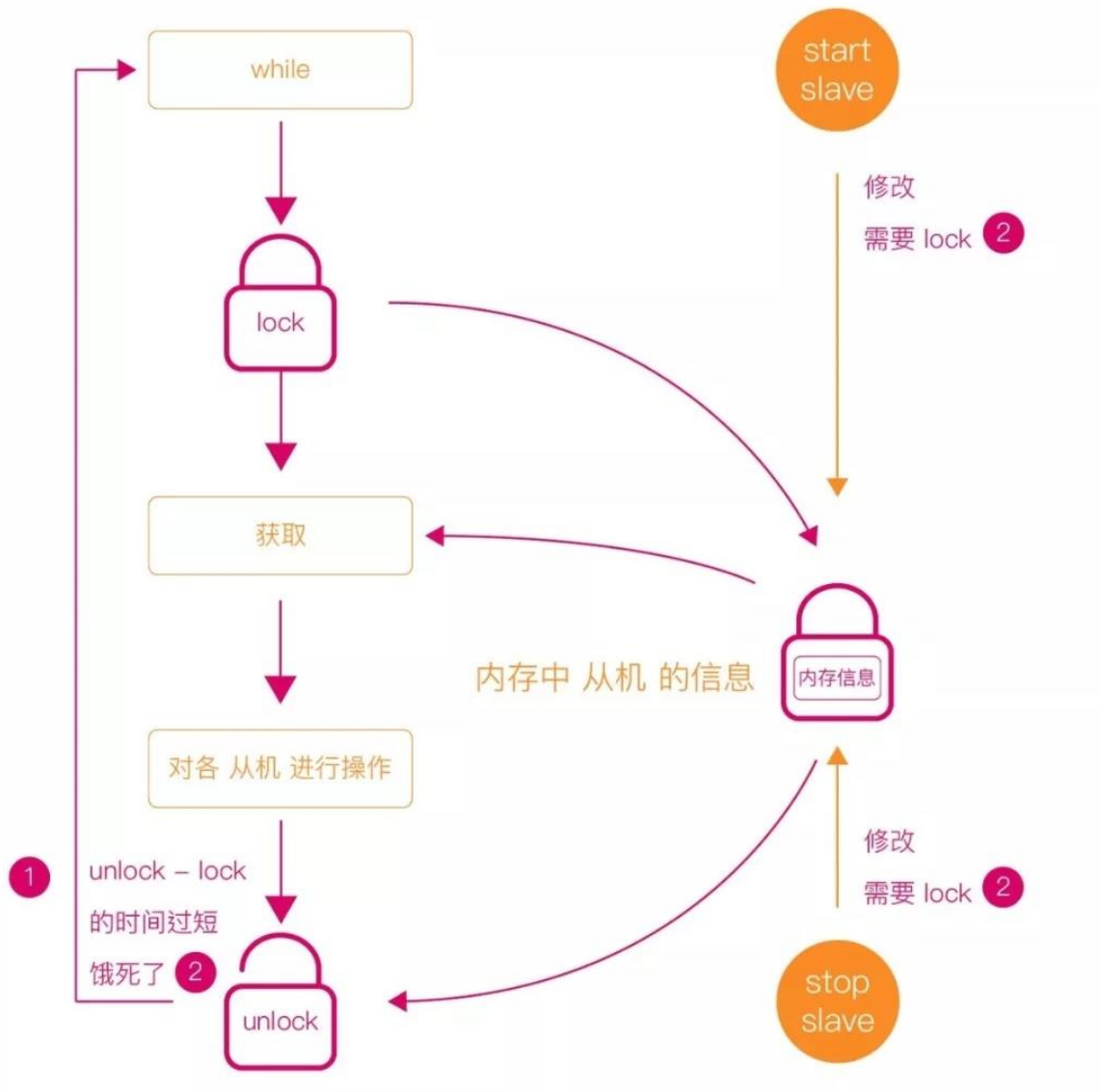
图二：描述了 MySQL 5.6 版本中的 ACK 接收机制

在 MySQL 5.6 版本的实现里，master 的复制线程同时负责发送事务和接收 slave 返回的 ACK 消息，当没有接收到上一个事务的 ACK 消息之前，无法发送下一个事务。如此串行化的机制成为了半同步的性能瓶颈。



图三：描述了 MySQL 5.7 版本中的 ACK 接收机制

在 MySQL 5.7 版本的实现里，将接收 ACK 这部分任务从 master 的复制线程中拆分出来，由半同步插件的 ACK 接收线程单独处理，使得事务发送和接收 ACK 得以并行，极大提高了半同步性能。



图四：描述了本文缺陷的发生原因

本文所要描述的缺陷就出现在 ACK 接收线程中，ACK 线程(图中序号 1)和复制线程(图中序号 2)在抢占互斥锁时产生了竞争。

master 在监听 slave 的 ACK 消息时，无限的 while 循环使得 ACK 线程基本时刻占有互斥锁。当启动另一个 slave 时，master 的新复制线程无法在短时间内抢占该互斥锁，导致复制线程无法启动成功，造成了 slave 的 slave_io_thread 停滞，无法复制数据的现象。

扩展阅读

<https://bugs.mysql.com/bug.php?id=89370>

<https://bugs.mysql.com/bug.php?id=13669>

<https://kristiannielsen.livejournal.com/12254.html>



【原理解析】设置字符集的参数控制了哪些行为

作者：黄炎 王悦

本文以一个字符集设置引起的故障现象入手，介绍 MySQL 字符集的各个参数的作用。

1 故障现象描述

在向 MySQL 导入数据时，先设置 `set names gbk`，然后通过 `source` 导入一个很大的 SQL 文件（文件字符集为 gbk），发现如下行为：

1. 正常情况下，SQL 文件中的 SQL 以分号分割，发往 MySQL 的每一个数据包会带有一个 SQL。
2. 在语句 `INSERT INTO ... VALUES(..., '璿', ...)`；之后，所有数据会打包在一起，通过大数据包协议一起发往 MySQL（若单个数据大于 16M，MySQL 则使用大数据包协议，参考[1]）。

初步猜测，认为是字符集的设置问题。

2 MySQL 的字符集参数

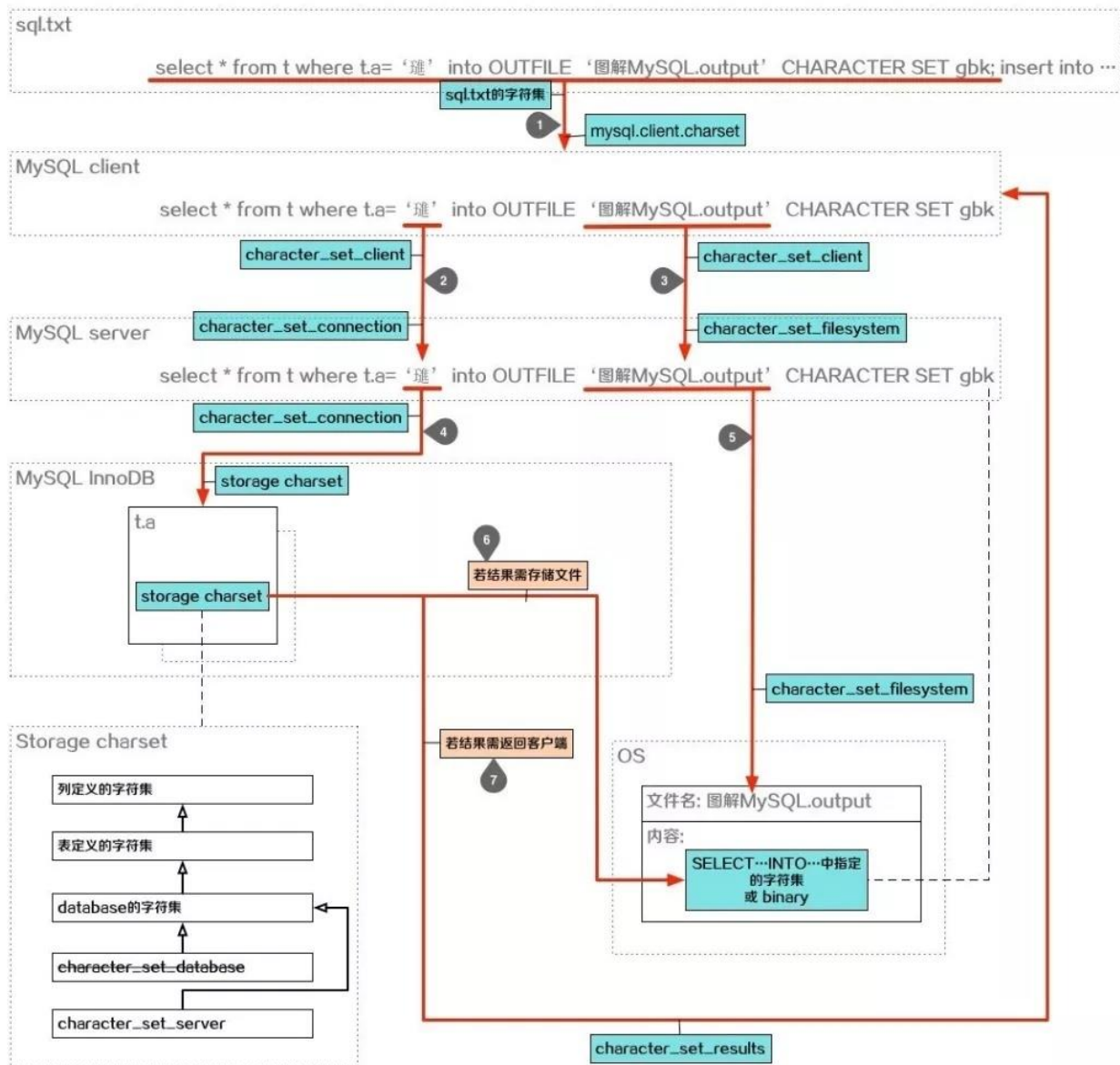
在谈及 MySQL 字符集时，还必须介绍校验集。字符集(character set)表示字符以何种规则进行编码，校验集(collation)表示字符以何种规则进行比较和排序（例如：是否大小写敏感）。

MySQL 有以下字符集相关的设置：

1. 三组设置字符集+校验集的参数：
 - a) `character_set_connection/collation_connection`
 - b) `character_set_server/collation_server`
 - c) `character_set_database/collation_database`，此组参数已被废弃。
2. 四个只能设置字符集的参数：
 - a) `character_set_client`
 - b) `character_set_results`
 - c) `character_set_filesystem`
 - d) `character_set_system`，此参数值固定为 `utf8`，且不可改变。本文不涉及此项。
3. MySQL client 中，有一个内存变量 `charset_info`，本文中称其为 `mysql.client.charset`
4. 存储层：数据库/数据表/数据列 均由单独的字符集+校验集参数，通过 `CREATE` 语句可进行设置。MySQL 文档中有详细记述。

（后文描述字符集+校验集时，以字符集为例，校验集的出现位置与对应的字符集相同）

配置关系如下图，示例为用 `source` 命令导入 `sql.txt`，其中包含一个 `SELECT... INTO OUTFILE` 语句：



	mysql.client.charset	character_set_client	character_set_connection	character_set_results
建立连接(字符集=CS)	CS	CS	CS	CS
SET NAMES CS	-	CS	CS	CS
SET CHARSET CS	CS	CS	character_set_database	CS

图解MySQL

(为描述方便，之后将 character_set_xxx 简写为 cs_xxx)

说明：

- client 从 sql.txt 读取 SQL。SQL 在 sql.txt 中遵循文件的字符集，client 按照 mysql.client.charset 进行读取。
- client 将 SQL 发往 server。SQL 按照 cs_client 字符集进行发送。server 接收 SQL 后，将其中的 **字符串常量** 转换成 cs_connection 字符集。
- server 接收 SQL 后，将其中的 **文件名常量** 转换成 cs_filesystem。
- server 将常量传给存储层 InnoDB 时，需将常量转换成存储层的字符集。
- SQL 中的文件名，以 cs_filesystem 字符集写入文件系统。

6. 若查询结果要存入文件，可在 `SELECT...INTO...` 语句中指定字符集，或默认使用 `binary`。
7. 若查询结果直接返回 `client`，则将结果转换为 `cs_results` 后返回。

关于存储层的字符集：

数据库/数据表/数据列 级别的字符集可分别指定，如图中左下部分所示，子级别可指定字符集或从父级别继承。其中 `cs_database` 已被废弃，但尚未移除。

关于字符集的设置：除直接设置变量，字符集的常见设置方法为：

1. `client` 连接握手时指定字符集，通过 `--default-character-set` 参数启动 `client` 可设置。
2. `SET NAMES` 语句。
3. `SET CHARSET` 语句。

其三种设置方式对参数的影响参看图中最后的列表，可以看出：

1. `SET NAMES` 并不会影响客户端解析 SQL 文件的字符集 `mysql.client.charset`。
2. `SET CHARSET` 会将 `cs_connection` 设置成 `cs_database` 的值，而并非设置的字符集。

3 故障分析

了解 MySQL 的字符集各项参数后，故障的原因就比较明显了：

1. `SET NAMES` 并不影响 `mysql.client.charset`，因此 MySQL 解析 `sql.txt` 时，使用了默认字符集 `utf8`。
2. “璉”字的二进制编码为 `ad5c`，`5c` 的对应字符为“\”。因此“(璉)(单引号)”会被误读为“(之前字符的编码 `xx` + `ad` 对应的字符)(被转义的单引号)”，导致后面的 SQL 因为单引号不封闭而被认为是一个字符串，因此 MySQL `client` 无法正确切分 SQL。

4 回顾

通过之前的说明，可以猜测 MySQL 的字符串参数的意图：

- ☐ 可以 设置字符集+校验集 的机制，都需要进行字符串的比较。比如 `cs_connection` 可用于 `WHERE` 条件中的比较运算，存储层的字符集 可用于 存储内的排序操作。
- ☐ 可以 设置字符集但不能设置校验集 的机制，都跟外部系统相关（相对于 MySQL `server`）。比如 `cs_client` 和 `cs_results` 跟 MySQL `client` 相关，`cs_filesystem` 与服务器的文件系统相关。
- ☐ `mysql.client.charset` 在 MySQL 文档中并未有介绍，但会影响 MySQL `client` 对 SQL 文件的解析。

扩展阅读

<https://dev.mysql.com/doc/refman/5.7/en/charset.html>

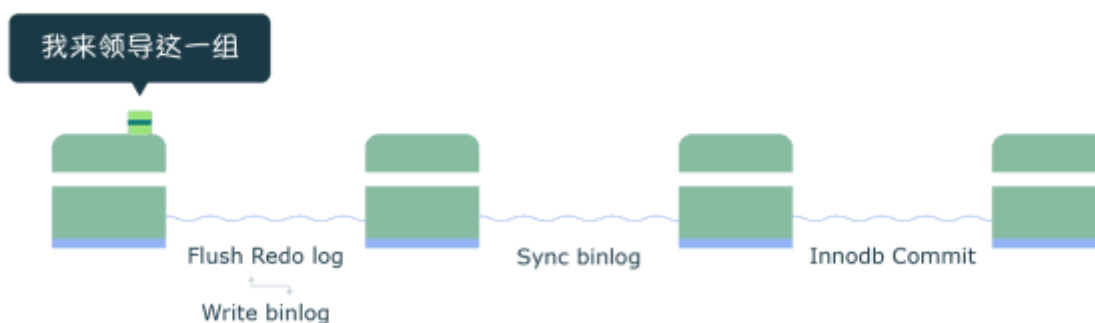
<https://www.cnblogs.com/chyingp/p/mysql-character-set-collation.html>

[1]<https://dev.mysql.com/doc/internals/en/sending-more-than-16mbyte.html>



【原理解析】MySQL 组提交(group commit)

作者：黄炎 王悦 周海鸣



1 前提

□ 以下讨论的前提 是设置 MySQL 的 crash safe 相关参数为双 1:

```
sync_binlog=1
innodb_flush_log_at_trx_commit=1
```

2 背景说明

□ WAL 机制 (Write Ahead Log) 定义:

WAL 指的是对数据文件进行修改前, 必须将修改先记录日志。MySQL 为了保证 ACID 中的一致性和持久性, 使用了 WAL。

□ Redo log 的作用:

Redo log 就是一种 WAL 的应用。当数据库忽然掉电, 再重新启动时, MySQL 可以通过 Redo log 还原数据。也就是说, 每次事务提交时, 不用同步刷新磁盘数据文件, 只需要同步刷新 Redo log 就足够了。相比写数据文件时的随机 IO, 写 Redo log 时的顺序 IO 能够提高事务提交速度。

□ 组提交的作用:

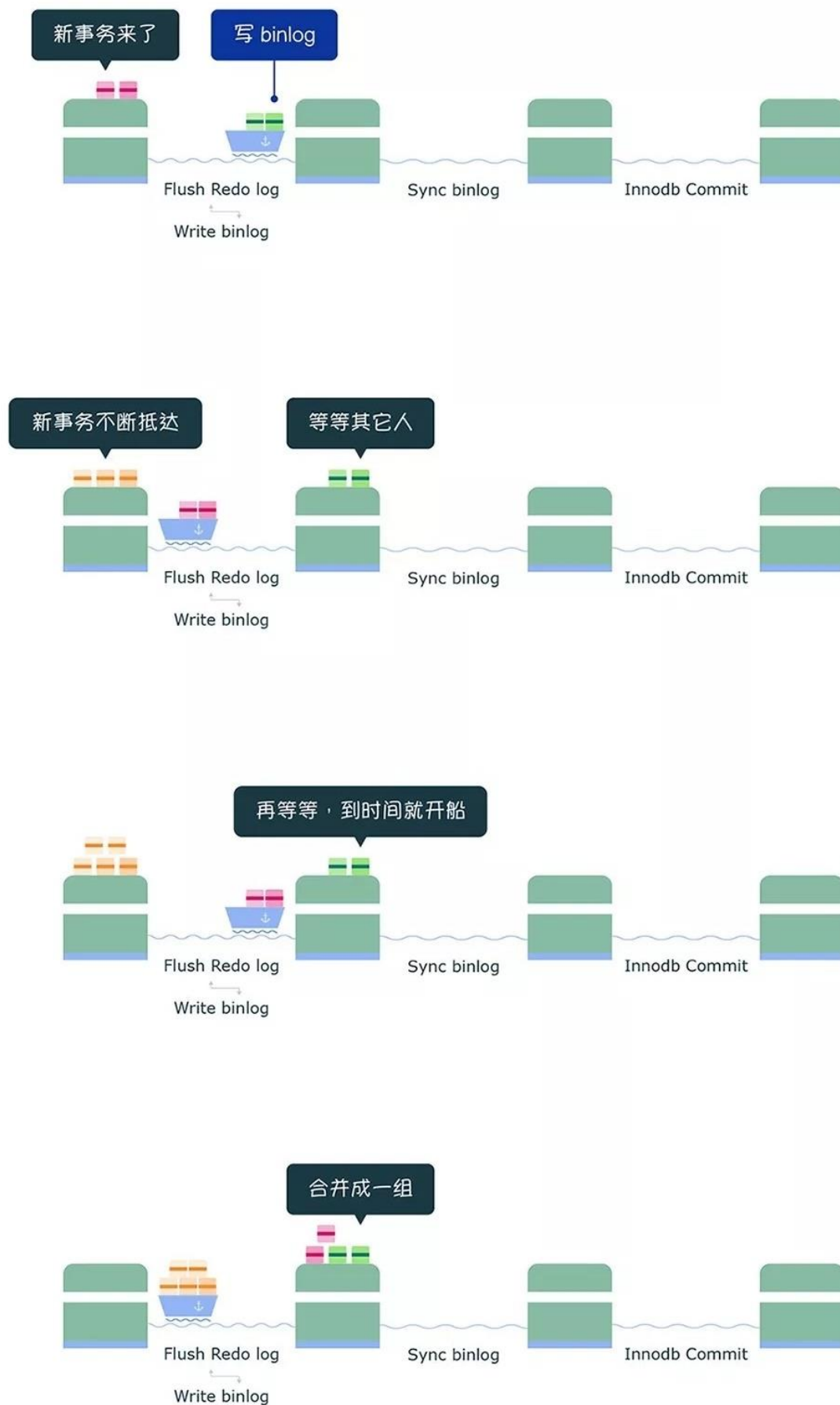
- 在没有开启 binlog 时 Redo log 的刷盘操作将会是最终影响 MySQL TPS 的瓶颈所在。为了缓解这一问题, MySQL 使用了组提交, 将多个刷盘操作合并成一个, 如果说 10 个事务依次排队刷盘的时间成本是 10, 那么将这 10 个事务一次性一起刷盘的时间成本则近似于 1。
- 当开启 binlog 时为了保证 Redo log 和 binlog 的数据一致性, MySQL 使用了二阶段提交, 由 binlog 作为事务的协调者。而引入二阶段提交使得 binlog 又成为了性能瓶颈, 先前的 Redo log 组提交也成了摆设。为了再次缓解这一问题, MySQL 增加了 binlog 的组提交, 目的同样是

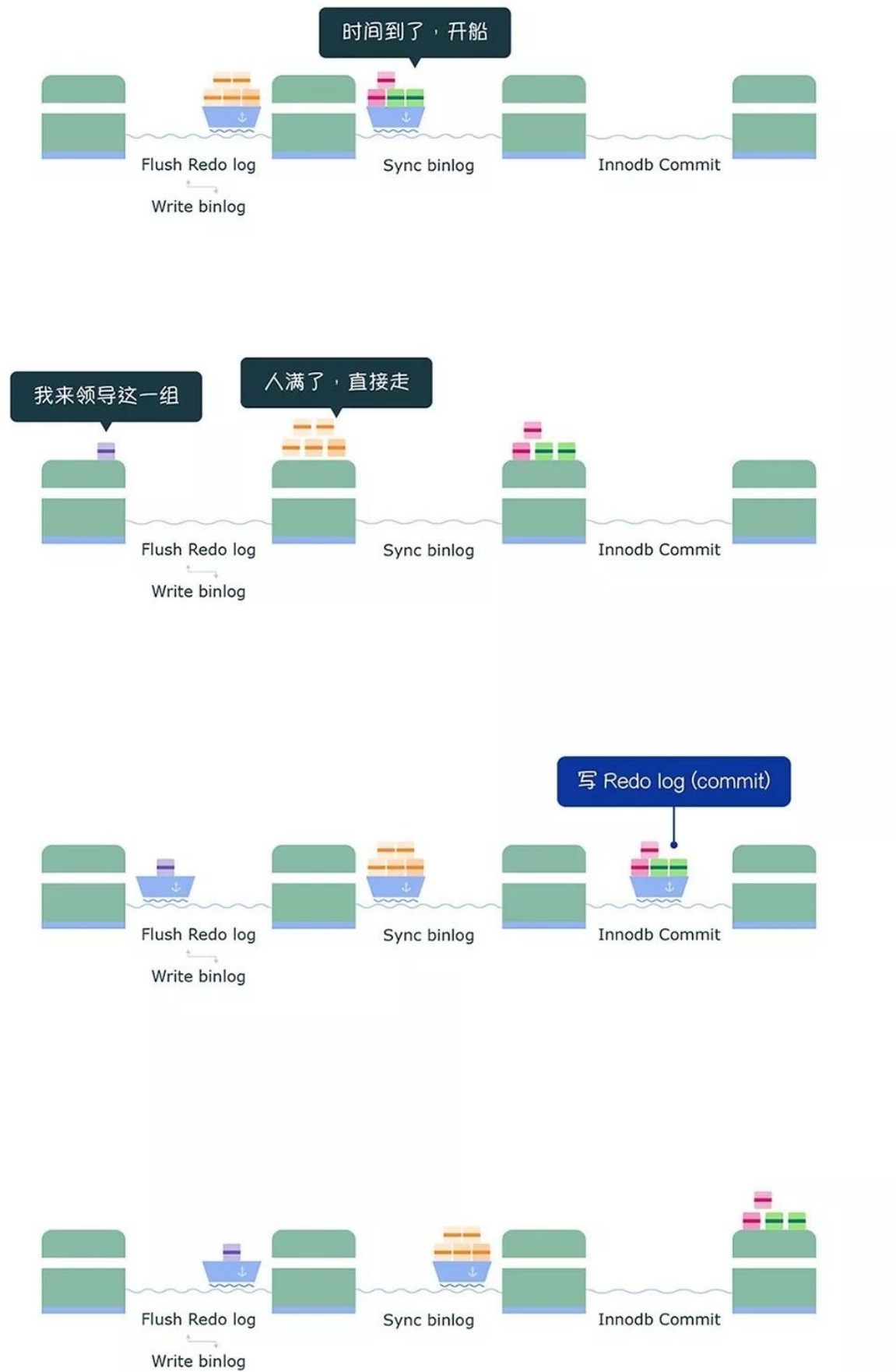
将 binlog 的多个刷盘操作合并成一个，结合 Redo log 本身已经实现的 组提交，分为三个阶段 (Flush 阶段、Sync 阶段、Commit 阶段) 完成 binlog 组提交，最大化每次刷盘的收益，弱化磁盘瓶颈，提高性能。

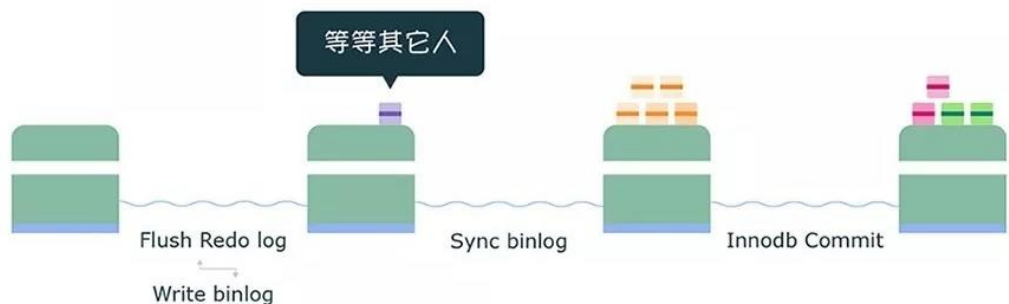
3 图解

下图我们假借“渡口运输”的例子来看看 binlog 组提交三个阶段的流程：









在 MySQL 中每个阶段都有一个队列，每个队列都有一把锁保护，第一个进入队列的事务会成为 leader，leader 领导所在队列的所有事务，全权负责整队的操作，完成后通知队内其他事务操作结束。

Flush 阶段 (图中第一个渡口)

- ☐ 首先获取队列中的事务组
- ☐ 将 Redo log 中 prepare 阶段的数据刷盘(图中 Flush Redo log)
- ☐ 将 binlog 数据写入文件，当然此时只是写入文件系统的缓冲，并不能保证数据库崩溃时 binlog 不丢失 (图中 Write binlog)
- ☐ Flush 阶段队列的作用是提供了 Redo log 的组提交
- ☐ 如果在这一步完成后数据库崩溃，由于协调者 binlog 中不保证有该组事务的记录，所以 MySQL 可能会在重启后回滚该组事务

Sync 阶段 (图中第二个渡口)

- ☐ 这里为了增加一组事务中的事务数量，提高刷盘收益，MySQL 使用两个参数控制获取队列事务组的时机：
 - binlog_group_commit_sync_delay=N: 在等待 N μ s 后，开始事务刷盘(图中 Sync binlog)
 - binlog_group_commit_sync_no_delay_count=N: 如果队列中的事务数达到 N 个，就忽视 binlog_group_commit_sync_delay 的设置，直接开始刷盘(图中 Sync binlog)
- ☐ Sync 阶段队列的作用是支持 binlog 的组提交
- ☐ 如果在这一步完成后数据库崩溃，由于协调者 binlog 中已经有了事务记录，MySQL 会在重启后通过 Flush 阶段中 Redo log 刷盘的数据继续进行事务的提交

Commit 阶段 (图中第三个渡口)

- ☐ 首先获取队列中的事务组
- ☐ 依次将 Redo log 中已经 prepare 的事务在引擎层提交(图中 InnoDB Commit)
- ☐ Commit 阶段不用刷盘，如上所述，Flush 阶段中的 Redo log 刷盘已经足够保证数据库崩溃时的数据安全了

- Commit 阶段队列的作用是承接 Sync 阶段的事务，完成最后的引擎提交，使得 Sync 可以尽早的处理下一组事务，最大化组提交的效率

4 缺陷分析

本文最后要讨论的 bug(可通过阅读原文查看)就是来源于 Sync 阶段中的那个 binlog 参数 `binlog_group_commit_sync_delay`，在 MySQL 5.7.19 中，如果该参数不为 10 的倍数，则会导致事务在 Sync 阶段等待极大的时间，表现出来的现象就是执行的 sql 长时间无法返回。该 bug 已在 MySQL 5.7.24 和 8.0.13 被修复。

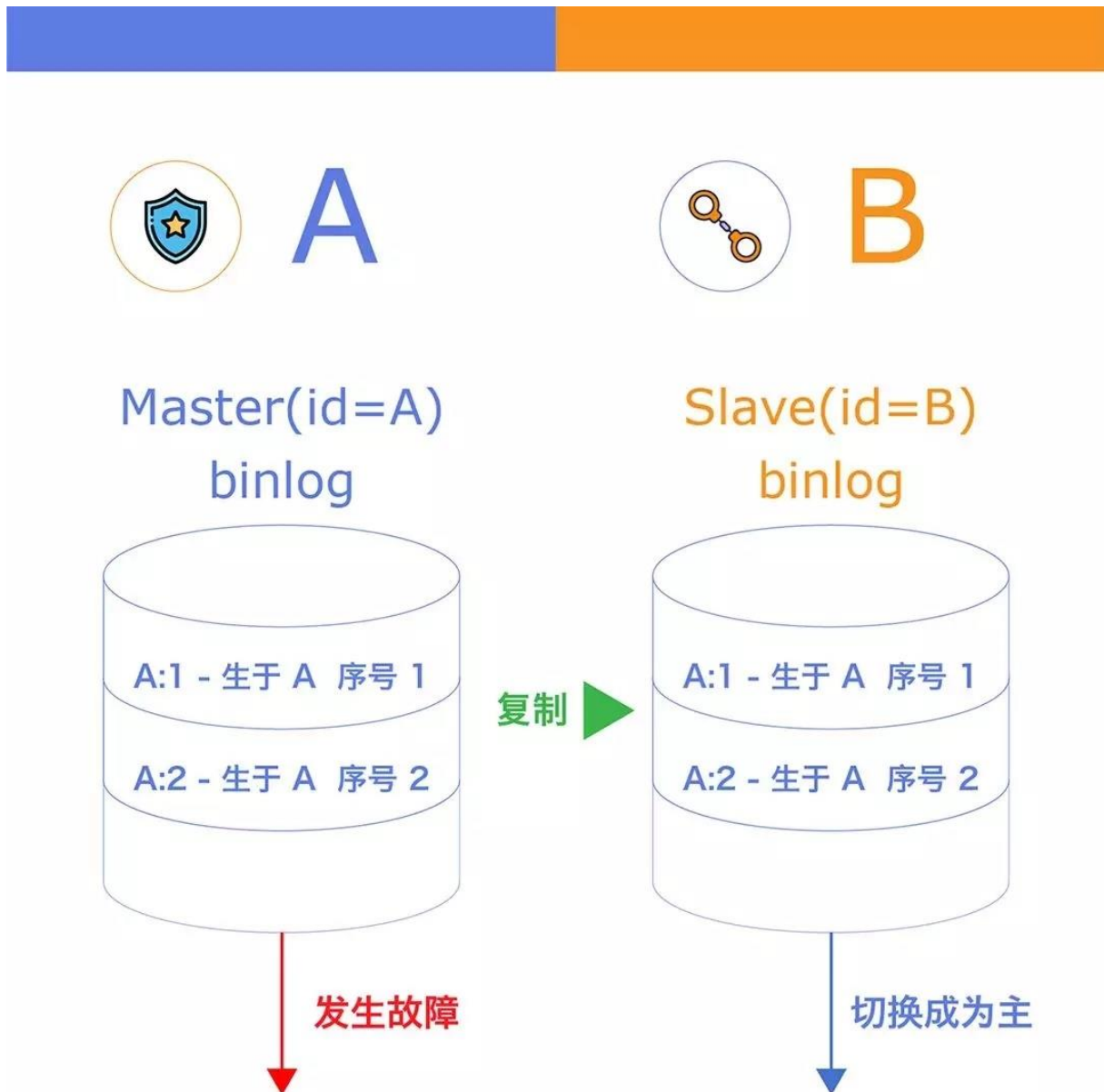


【原理解析】MySQL 固定的 server_id 导致数据丢失

作者：黄炎 王悦 周海鸣

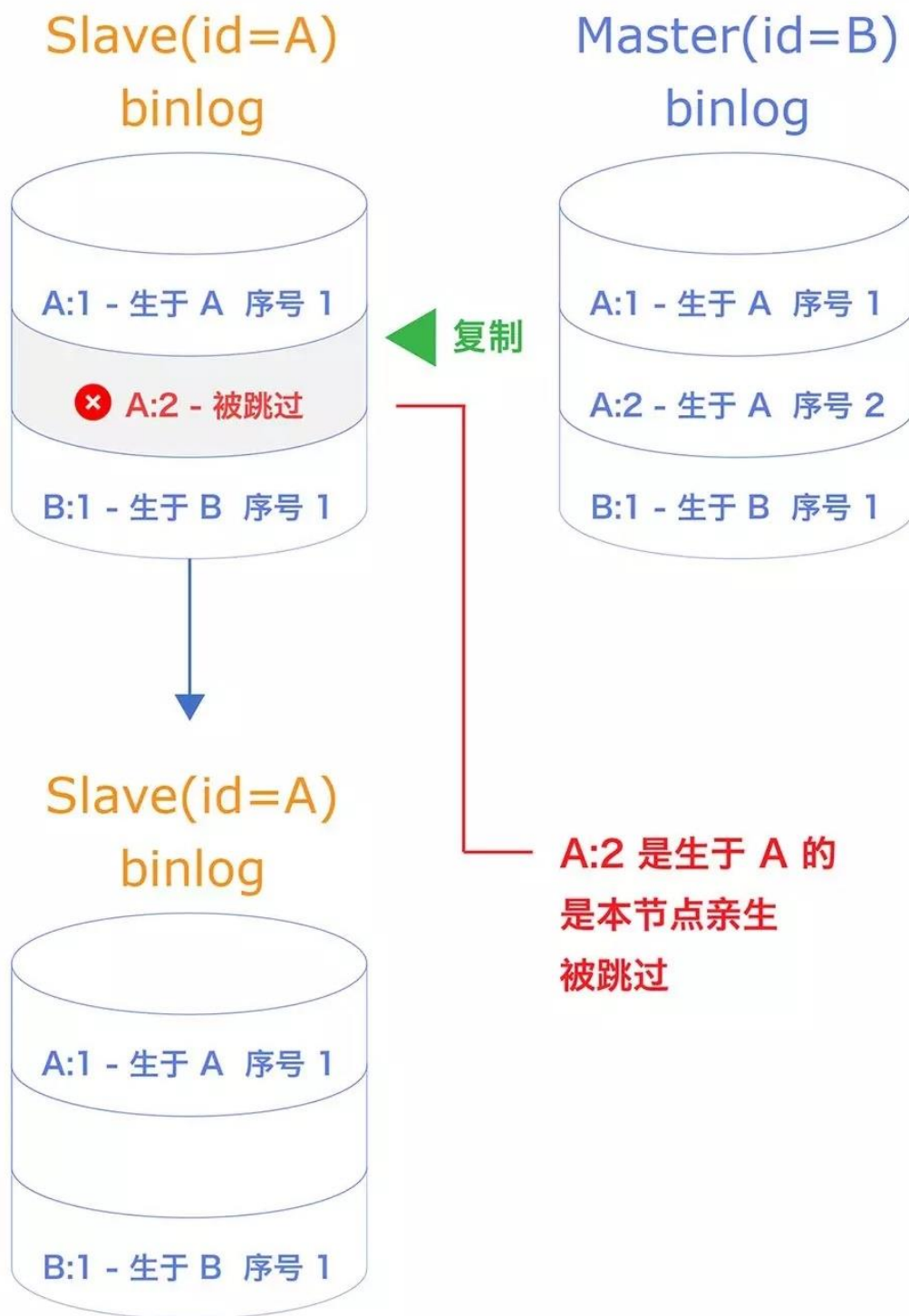
本文我们来看一个场景，两台 MySQL 实例使用主从复制，当 master 故障，触发高可用切换，新 master 上线后，通过备份重建旧 master 并建立复制后，数据发生丢失。

以下我们通过图解还原一遍当时的场景（图中标注的 id 指的是 MySQL 的 server_id）：





第二部分



第三部分

server_id 配置:

1. 默认值 1 或 0
2. 需要配置时通过参数 server-id 指定

Property	Value
Command-Line Format	--server-id=#
System Variable	server_id
Scope	Global
Dynamic	Yes
SET_VAR Hint Applies	No
Type	integer
Default Value (>= 8.0.3)	1
Default Value (<= 8.0.2)	0
Minimum Value	0
Maximum Value	4294967295  图解MySQL

1 背景

- ☐ 当配置 MySQL 复制时，server_id 是必填项，用来区分复制拓扑中的各个实例，例如在循环的级联复制中 (A=>B=>C=>A)，避免重复数据不必要的复制 (C=>A 数据重复，不必要)
- ☐ 当 slave 的 io 线程发现 binlog 中的 server_id 与自身一样时，默认不会将该 binlog 写入自身的 relay log 中，即跳过了该数据的复制，同时也能减少写 relay log 对磁盘的压力
- ☐ 然而这种机制在高可用切换场景下会引入潜在的隐患：
 - 隐患一：

如上图所示，从备份恢复的旧 master 仍沿用了原来的 server_id A，导致 io 线程跳过了 A:2 事务，最终丢失了 A:2 的数据
 - 隐患二：

级联复制中，当不相邻实例的 server_id 相同时，也会出现复制数据丢失
- ☐ 上述两种隐患的存在都是因为在复制拓扑中非直接相连的 MySQL server_id 重复。在普通的一对主从复制中，slave 的 io 线程会检查与自己相连的 master 的 server_id 是否与自身重复，若发现重复会停止复制抛出错误
- ☐ 注：可通过配置--replicate-same-server-id 改变以上默认行为

2 使用建议

- ☐ 配置 MySQL 复制时，为每个实例配置不同的 server_id
- ☐ 通过备份工具还原实例后，为实例配置一个新的 server_id

3 附加题

除了 server_id，MySQL5.6 起引入了 server_uuid

server_uuid 配置

当 MySQL 启动时

1. 尝试从 data_dir/auto.cnf 中读取 uuid
2. 如果 1 尝试失败，则生成一个新的 uuid 并写入 data_dir/auto.cnf

Property	Value
System Variable	server_uuid
Scope	Global
Dynamic	No
Type	string  MySQL

背景

- ☐ 主从复制中，要求 master 和 slave 的 server_uuid 不同，否则在复制初始化时会出现报错
- ☐ GTID 就是使用了 server_uuid 做为全局唯一的标识

使用建议

- ☐ 如果直接拷贝 master 的数据文件来建立 slave，注意要删除 auto.cnf，重启使 MySQL 重新生成一个新的 server_uuid，否则复制将会异常

本文参考

<https://dev.mysql.com/doc/refman/8.0/en/replication-options-slave.html>

微信扫一扫读原文



【原理解析】XtraBackup 全量备份还原

作者：黄炎 王悦 周海鸣

MySQL 备份工具是种类繁多，大体可以分为**物理备份**和**逻辑备份**。物理备份直接包含了数据库的数据文件，适用于大数据量，需要快速恢复的数据库。逻辑备份包含的是一系列文本文件，其中是代表数据库中数据结构和内容的 SQL 语句，适用于较小数据量或是跨版本的数据库备份恢复。

本篇图解的是其中一种备份工具——XtraBackup 的全量备份的工作机制。XtraBackup 是一种物理备份工具，支持热备，在备份时复制所有 MySQL 的数据文件以及一些事务日志信息，在还原时将复制的数据文件放回至 MySQL 数据目录，并应用日志保证数据一致。下面我们来解读其中的过程：

全量备份流程

图 1

1. 复制已有的 redo log，然后监听 redo log 变化并持续复制

图 2

2. 复制事务引擎数据文件

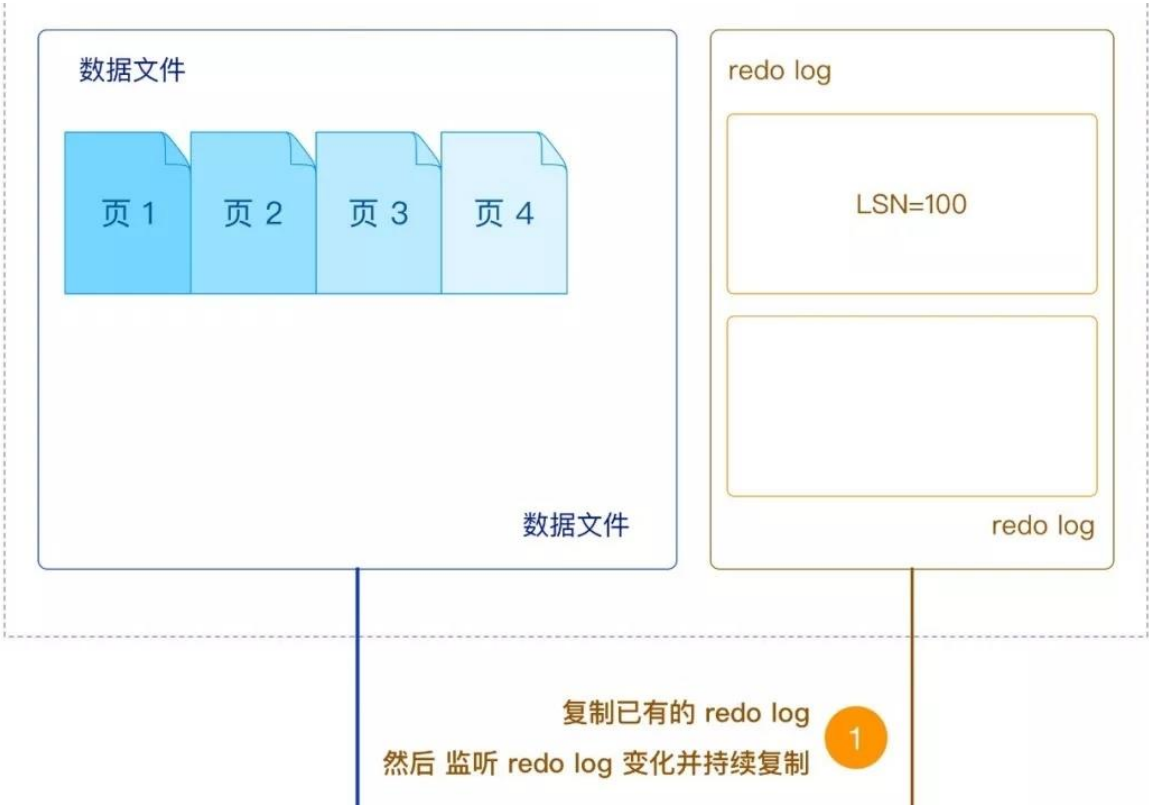


图 1 第 1 部分

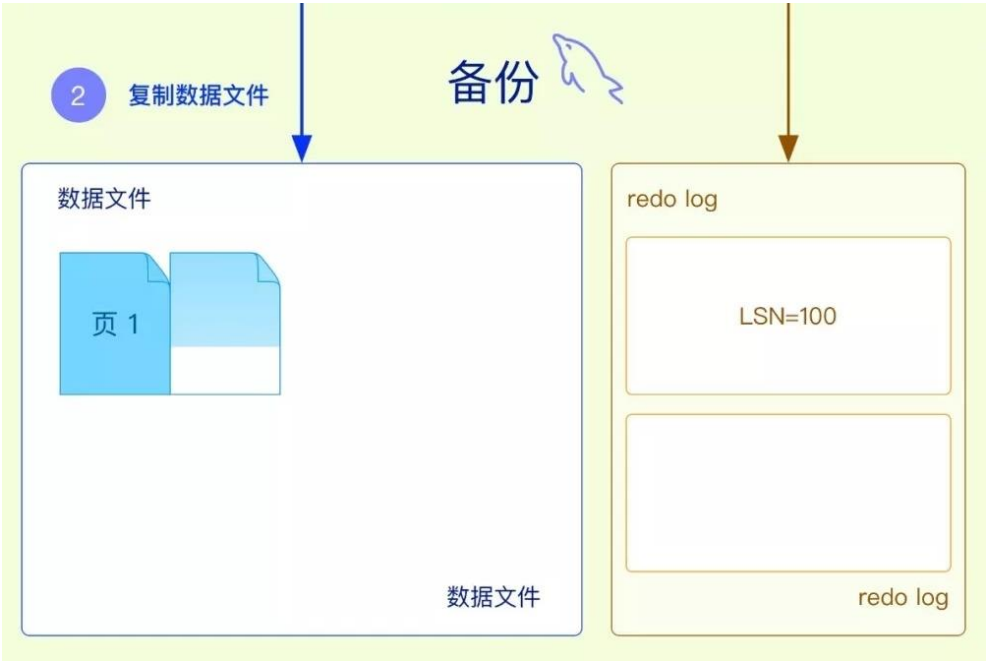


图 1 第 2 部分

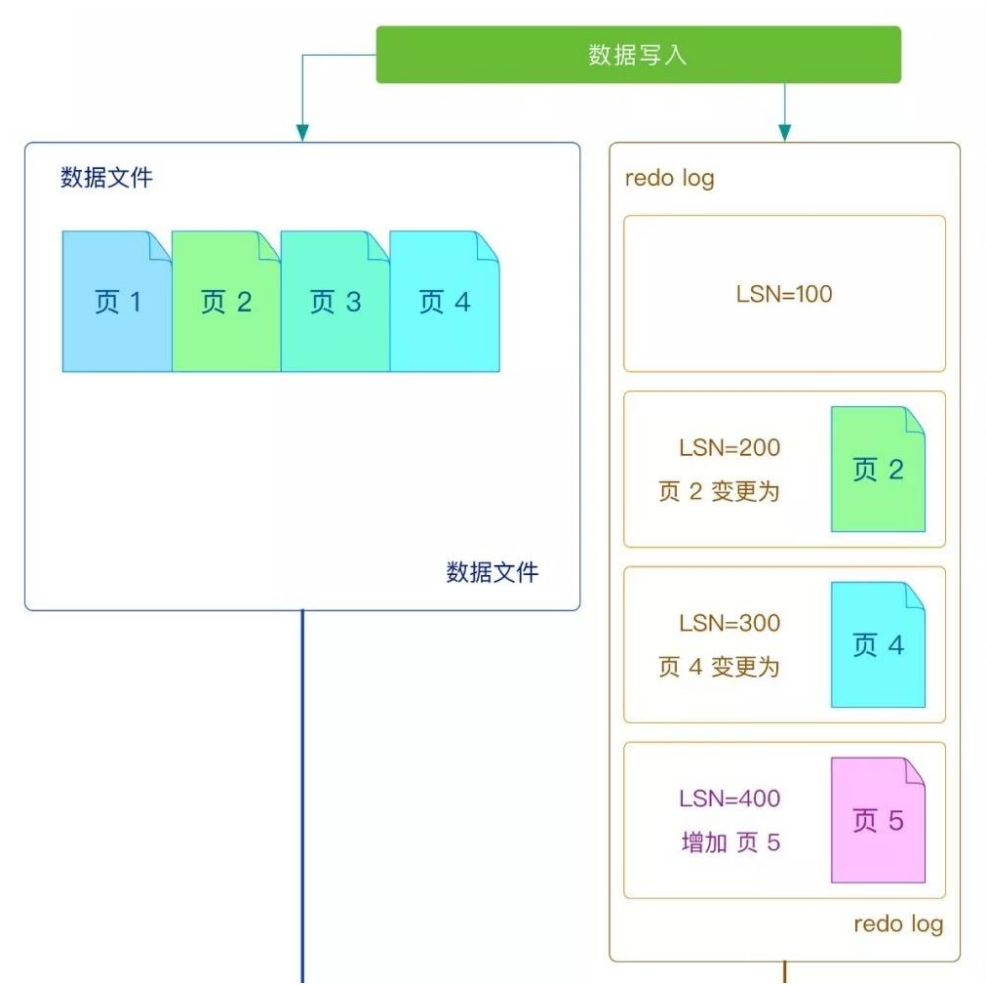


图 2 第 1 部分

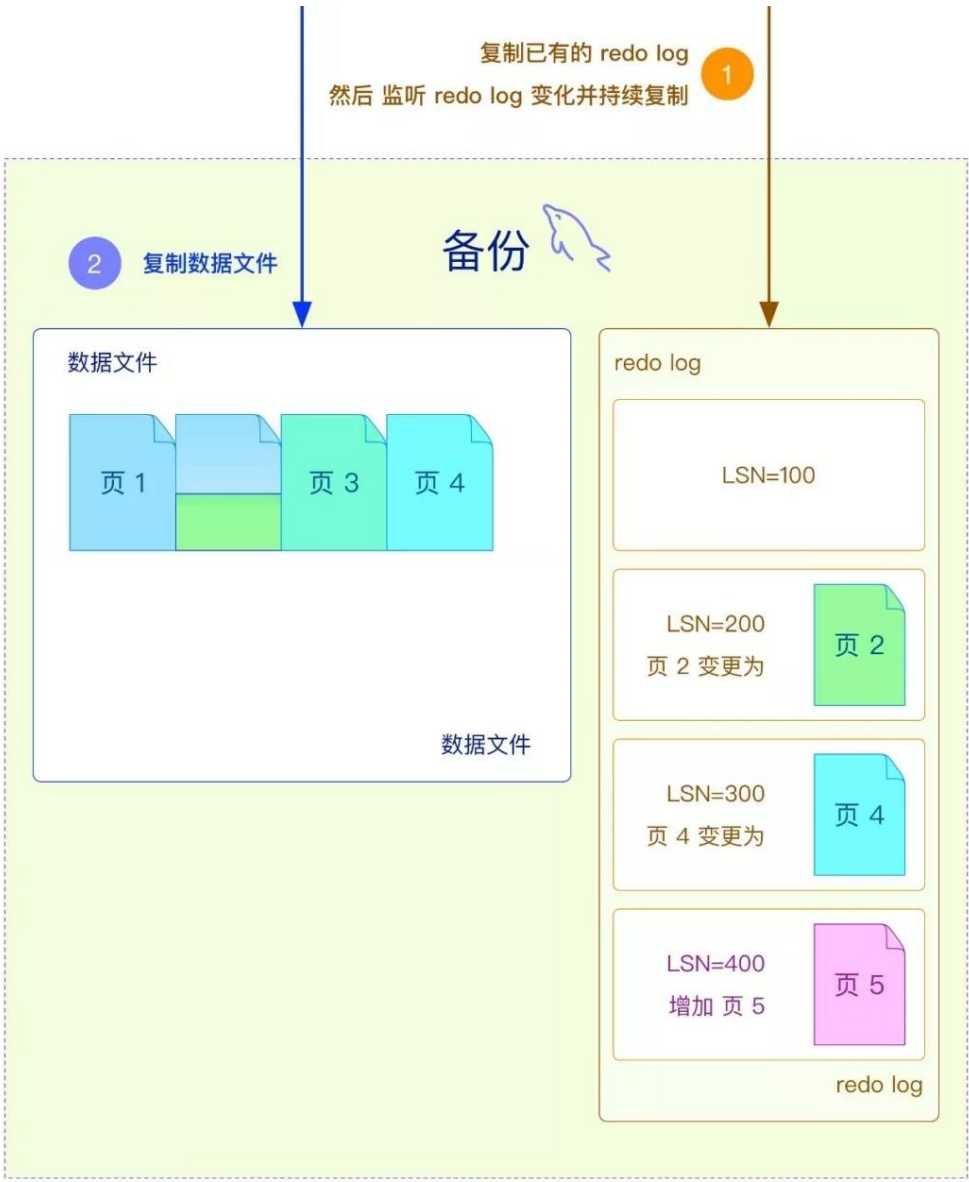


图 2 第 2 部分

讨论 1：为什么要先复制 redo log，而不是直接开始复制数据文件？

因为 XtraBackup 是基于 InnoDB 的 **crash recovery** 机制进行工作的。如上图 2 中的页 2，由于是热备操作，在备份过程中可能有**持续的数据写入**，直接复制出来的数据文件可能有缺失或被修改的页，而 redo log 记录了 InnoDB 引擎的所有事务日志，可以在还原时应用 redo log 来补全数据文件中缺失或修改的页。所以为了确保 redo log 一定包含备份过程中涉及的数据页，需要首先开始复制 redo log。

全量备份流程

图 3

3. 等到数据文件复制完成 4. 加锁：全局读锁

图 4

5. 备份非事务引擎数据文件及其他文件 6. 获取 binlog 点位信息等元数据

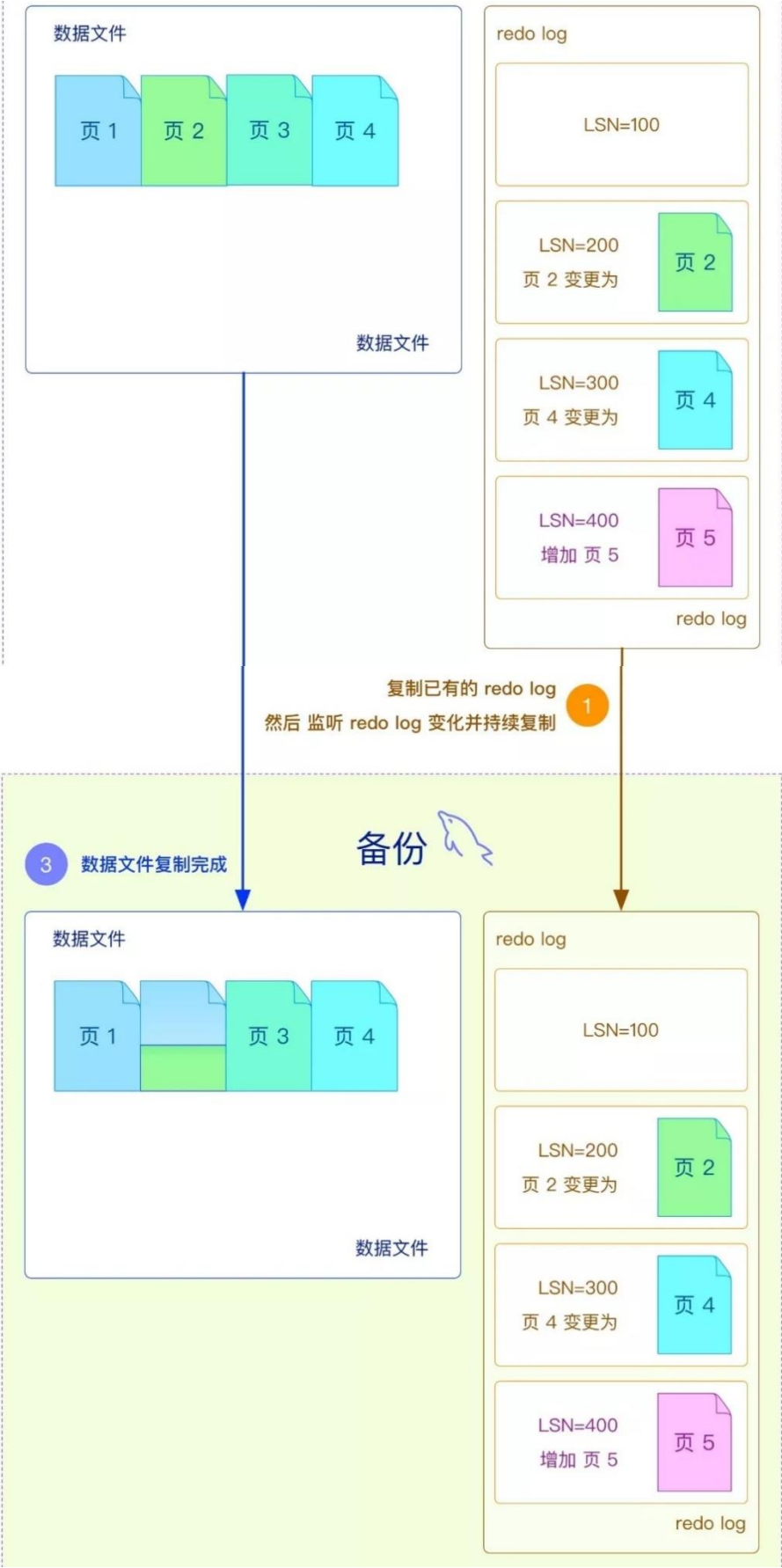


图 3

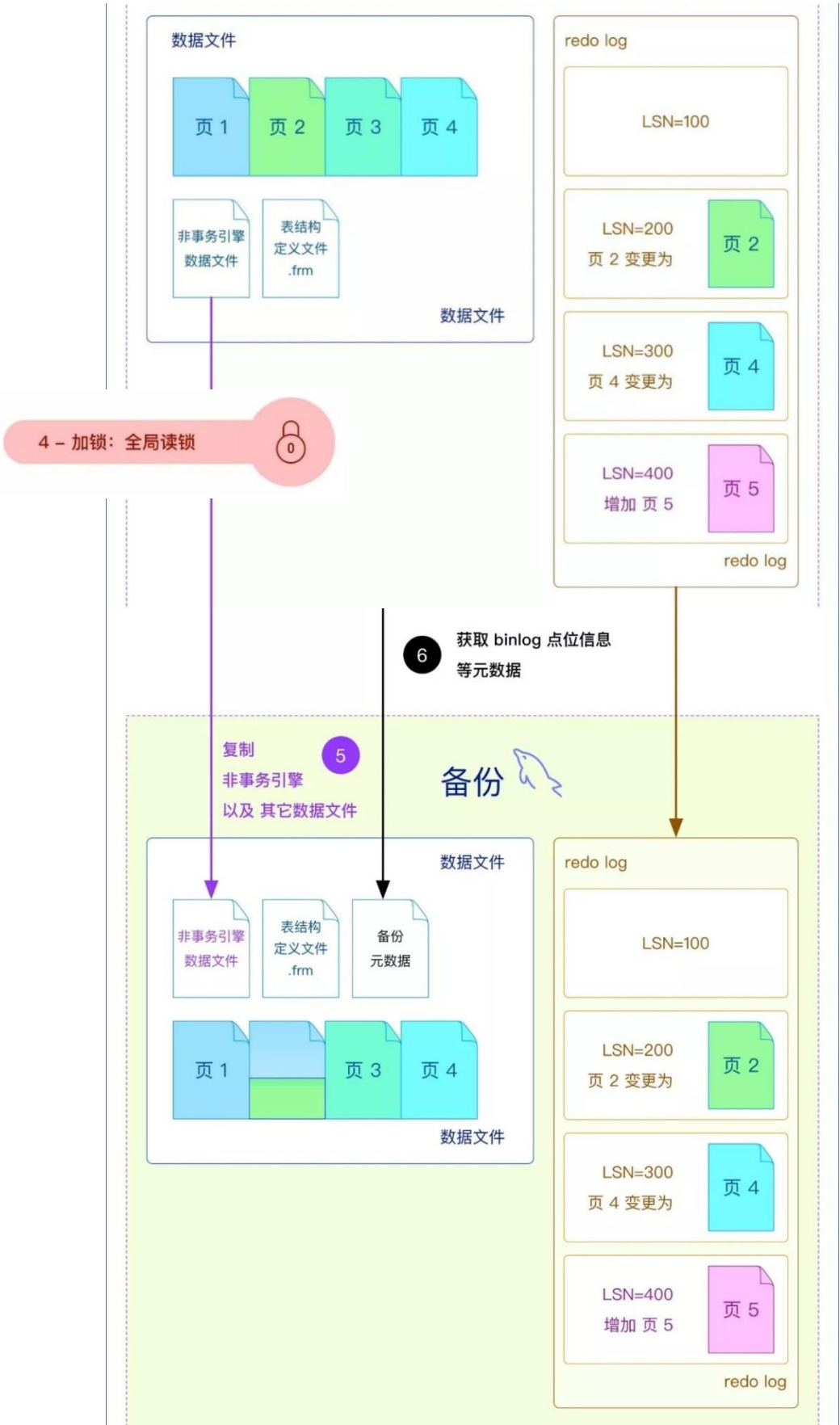


图 4

TIPS:

非事务引擎数据文件较多时，全局读锁的时间会较长。

讨论 2：加全局读锁的作用？

因为要保证”非事务资源 自身的一致性“ 和 ”非事务资源与 事务资源的一致性“。在加锁期间，没有新数据写入，XtraBackup 会复制此时的 binlog 位置信息，frm 表结构，MyISAM 等非事务表。

全量备份流程

图 5

7. 停止复制 redo log
8. 解锁：全局读锁
9. 复制 buffer pool dump
10. 备份完成

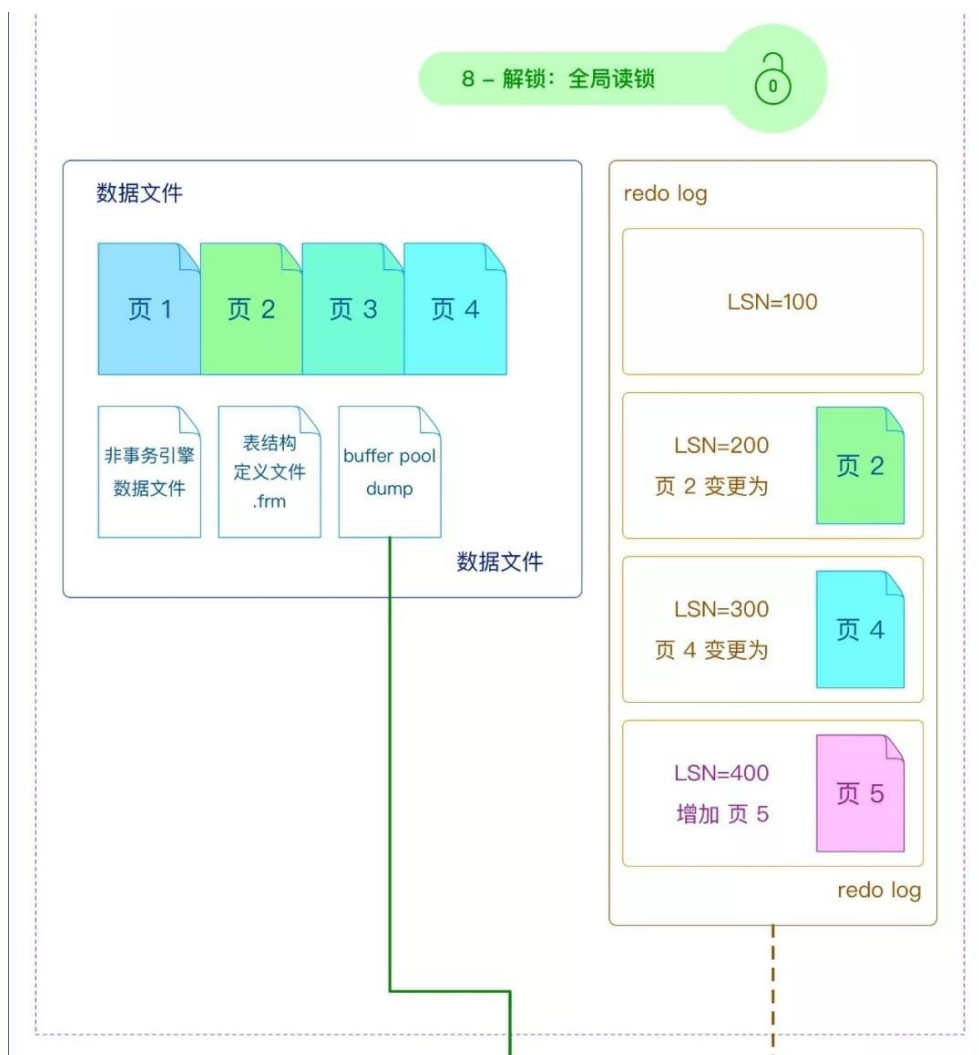


图 5 第 1 部分

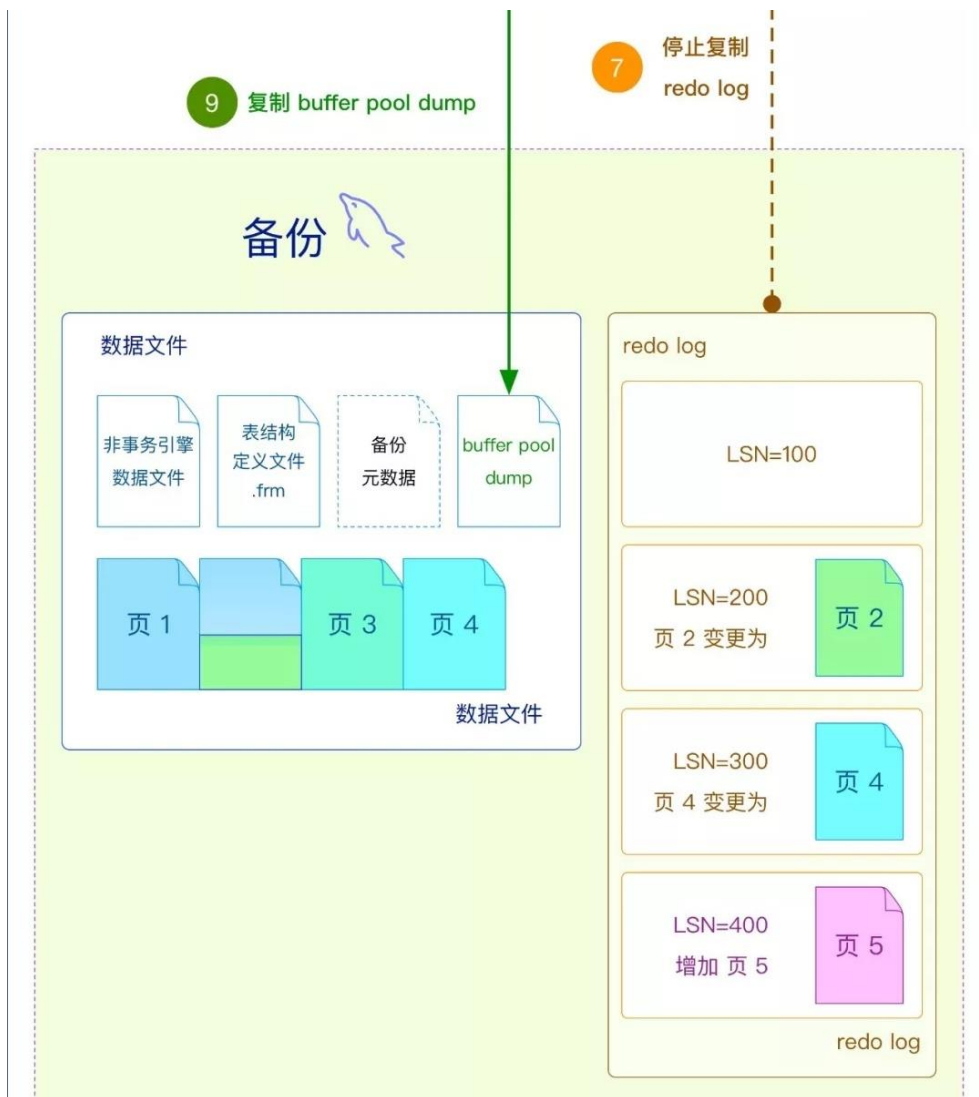


图 5 第 2 部分

讨论 3：为什么要先停止复制 redo log，再解锁全局读锁？

也是为了保证“非事务资源与事务资源的一致性”，保证通过 redo log 回放后的 InnoDB 数据与非 InnoDB 数据都是处于读锁期间取得的位点。

全量备份流程总结

1. 复制已有的 redo log，然后监听 redo log 变化并持续复制
2. 复制事务引擎数据文件
3. 等到数据文件复制完成
4. 加锁：全局读锁
5. 备份非事务引擎数据文件及其他文件
6. 获取 binlog 点位信息等元数据
7. 停止复制 redo log
8. 解锁：全局读锁
9. 复制 buffer pool dump
10. 备份完成

XtraBackup 基于 InnoDB 的 crash recovery 机制，在备份还原时利用 redo log 得到完整的数据文件，并通过全局读锁，保证 InnoDB 数据与非 InnoDB 数据的一致性，最终完成备份还原的功能。

全备还原流程

图 1 (xtrabackup--prepare)

1. 模拟 MySQL 进行 recover，将 redo log 回放到数据文件中
2. 等到 recover 完成
3. 重建 redo log，为启动数据库做准备

图 2 (xtrabackup--copy-back/move-back)

4. 将数据文件复制回 MySQL 数据目录
5. 还原完成



还原图 1 第 1 部分



还原图 1 第 2 部分



还原图 2 第 1 部分



还原图 2 第 2 部分

讨论 4：在 recover 完成后，InnoDB 数据与非 InnoDB 数据是达成一致的么？

InnoDB 数据会被恢复至备份结束时(全局读锁时)的状态，而非 InnoDB 数据本身即是在全局读锁时被复制出来，它们的数据一致。

总结

1. 全量备份开始时，要监听并记录 redo log 的变化。
2. 全量备份拷贝 InnoDB 数据文件时，数据库同时会写入数据，导致数据文件的新旧程度不一致。
3. 拷贝 InnoDB 数据文件后，对数据库施加全局读锁后，才能拷贝非事务类型的信息，比如：binlog 点位、非事务类引擎的数据文件等。
4. 全量恢复时，对 InnoDB 数据文件的不一致，通过将步骤 1 中记录的 redo log 的变化回放，可消除步骤 2 中的数据文件的新旧程度不一致的问题。



【原理解析】XtraBackup 增量备份还原

作者：黄炎 王悦 周海鸣

在讨论增量备份和恢复时，我们需关注以下问题：

1. 增量备份时，如何识别 InnoDB 的哪些数据是增量的。
2. 类似于全量备份，如何解决增量备份的数据文件的新旧不一致问题。
3. 非事务类型的信息，能否产生增量备份。

图解增量备份步骤

由于增量备份是在全量备份的基础上进行备份，先来做一个全量备份：



图 1 第 1 部分



图 1 第 2 部分

讨论 1：现在我们开始做一个增量备份，那么如何识别 InnoDB 的哪些数据是增量的？

从全量备份的示意图里，可以看到数据文件中的数据页都有 LSN 号（在上一篇图解中我们介绍过 LSN 的概念，不再赘述），LSN 可以看做是数据页的变更时间戳。

那么通过这个时间戳，就可以识别数据页在全量备份后是否修改过，即通过 LSN 可以识别数据是否是增量的。

从下图中，可以看到增量备份时，LSN>400 的数据页才会进行备份。

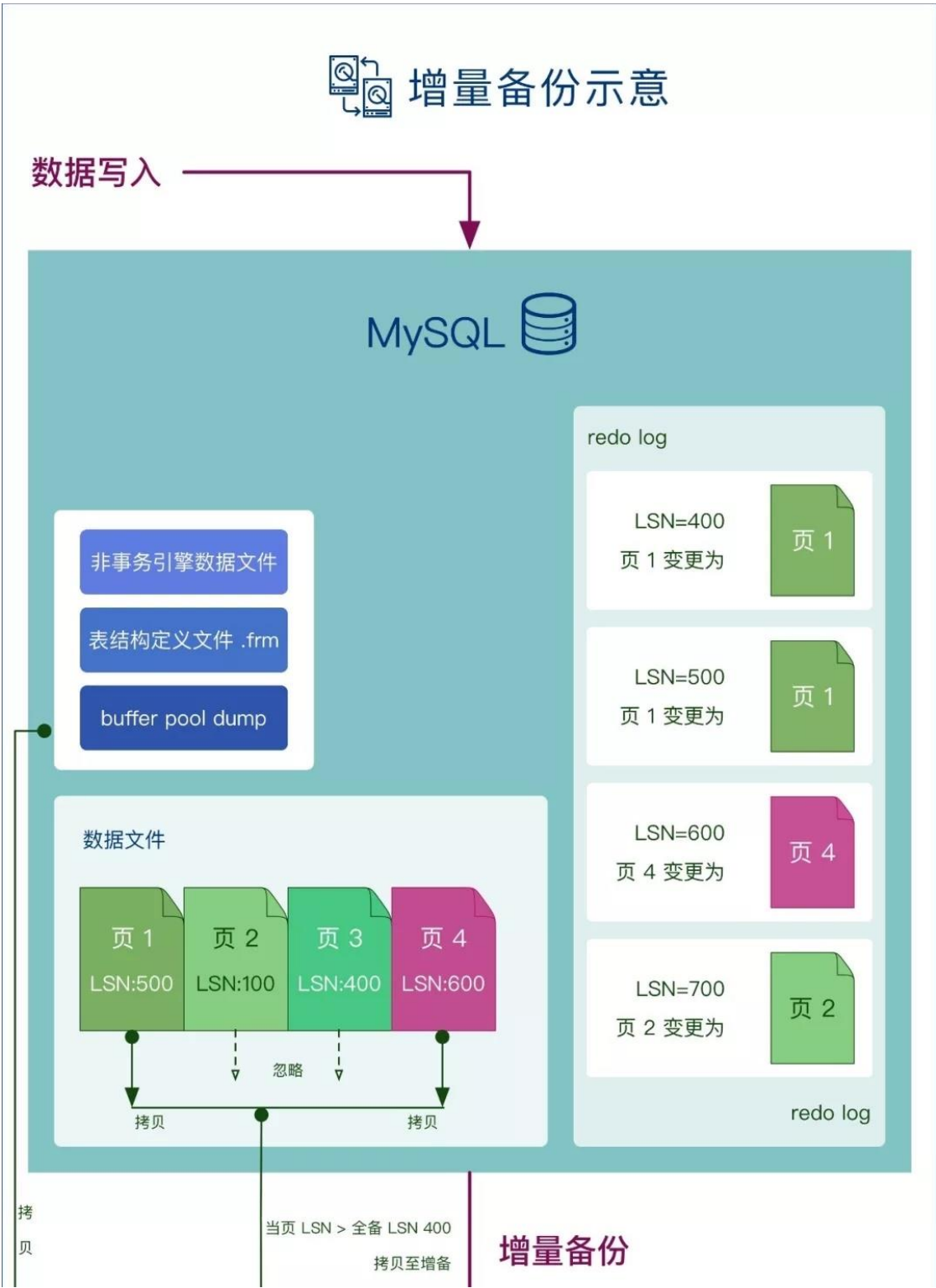


图 2 第 1 部分



图2 第2部分

讨论 2：如果一个数据页原本不是增量范围内的，在增量备份的过程中，数据页更新了，那么增量备份是否会涵盖这个数据页？

这个问题的本质，与全量备份中的数据页新旧不一致的问题 相同，解决方案也相同：通过恢复时回放 redo log，解决数据新旧不一致的问题。

也就是说：增量备份过程中，如果数据页被更新了，那数据文件中的这个数据页有可能被拷贝到备份中，也可能没有被拷贝到备份中，但这个更新信息一定会被 redo log 记录，并被记录在备份中。在恢复过程中，redo log 会被”安全地“回放成功，达成数据的新旧一致。

增量备份恢复的流程

1. 先还原一个全量备份到临时目录（详细步骤参看上一篇图解）。
2. 开始还原增量备份，将增量备份中的增量的数据文件，覆盖到临时目录中。
3. 将增量备份中的 redo log，回放到临时目录中。
4. 将其他文件覆盖到临时目录中。
5. 增备还原完成。

6. 重建 redo log, 为启动数据库做准备。
7. 将临时目录中的文件, 拷贝到 MySQL 的数据目录中。



图 3 第 1 部分



图 3 第 2 部分

讨论 3：从图中可以看到，非事务类的信息是从增量备份直接覆盖到临时目录中的，也就是说：与事务类的信息不同，MySQL 没有为非事务类的信息提供增量的机制，均是全部拷贝和覆盖。

再往前一步：增量备份是否需要全局读锁呢？由讨论 3 的结论可知，虽然示意图上没有标明，但增量备份依然需要全局读锁去保证非事务类的信息的一致性。

回顾

1. 增量备份时，如何识别 InnoDB 的哪些数据是增量的：
数据页上的 LSN 充当了逻辑时间戳的角色，用以识别增量。
2. 类似于全量备份，如何解决增量备份的数据文件的新旧不一致问题：
解决方案与全量备份相同，依靠 redo log 进行回放以保证一致性。
3. 非事务类的信息，能否产生增量备份：
非事务类的信息没有提供识别增量的机制，只能采用全局读锁+全部拷贝+全部回放的机制进行备份。



【原理解析】MySQL DDL 为什么成本高？

作者：黄炎 王悦 周海鸣

本文讨论 MySQL 的 DDL，讨论的背景是 MySQL 8.0+InnoDB。

DDL(Data Definition Language)

众所周知，DDL 定义了数据在数据库中的结构、关系以及权限等。比如 CREATE，ALTER，DROP 等等。本期我们讨论 MySQL 8.0（使用 InnoDB 存储引擎）在修改表结构时，究竟会发生什么？



图 1 第 1 部分

聚簇索引的在文件中的物理排布



当增加一列时

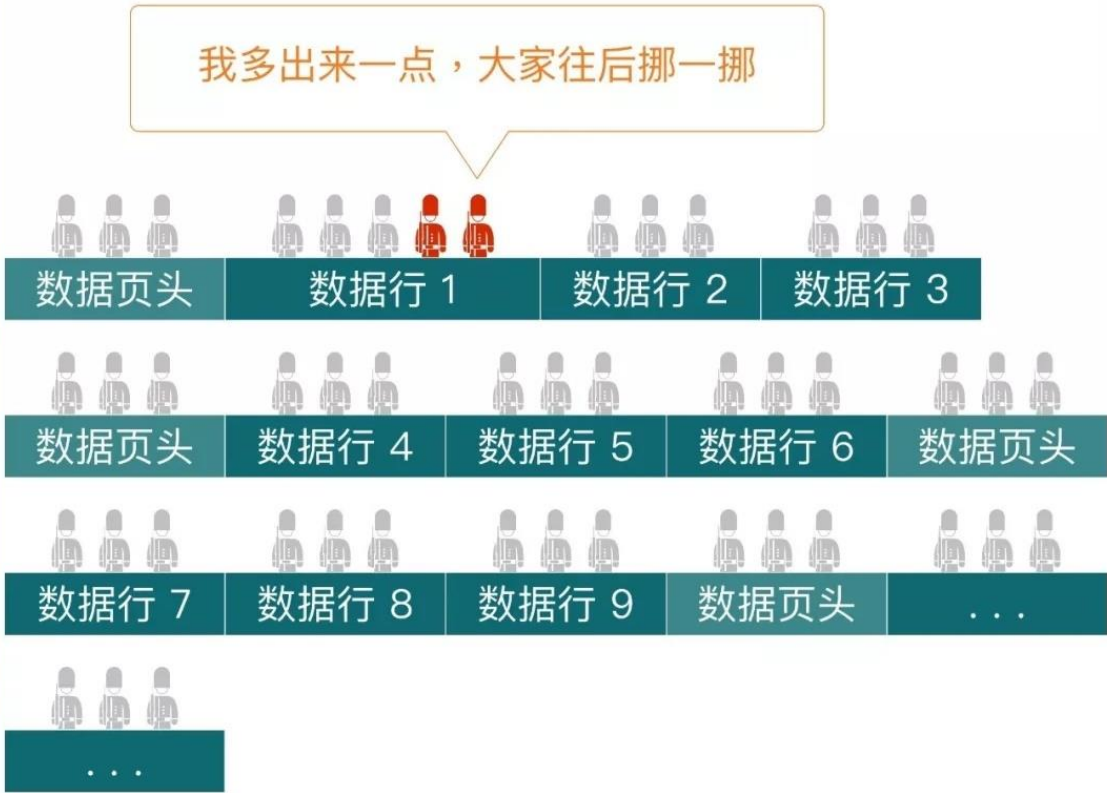


图 1 第 2 部分

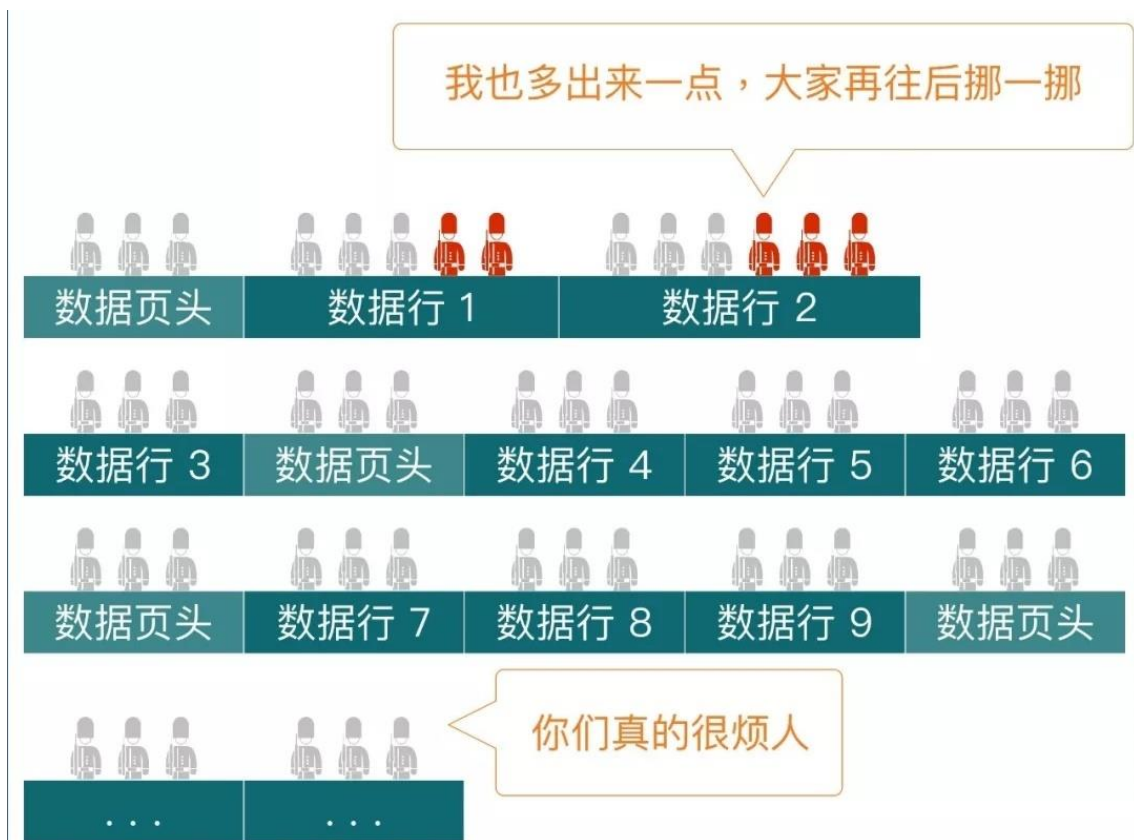


图 1 第 3 部分

DDL 与表结构

既然 DDL 的作用是改变表结构，那表结构在 InnoDB 引擎中是什么样的呢？如上图，逻辑上，InnoDB 表中的数据可以理解成按照主键（聚簇索引）顺序存放的，每一行的数据依次排列（物理上，InnoDB 表中的数据按照 InnoDB 的数据结构 B+树进行排列）。

当需要对表增加一列时，会涉及到每一行数据排列的变动，需要重建整张表的数据，可想而知这种变动的成本是高昂的。

然而并不是每一种 DDL 都要付出这么大的成本，要看具体的分类。

MySQL 8.0 将 DDL 用以下五个维度分类讨论：

- ☐ Instant：此变更可以“立刻”完成
- ☐ In Place：此变更由 InnoDB 引擎独立完成，不需要使用 Redo log 等，可以节省开销
- ☐ Rebuild Table：此变更会重建聚簇索引，一般情况下，涉及到数据变更时才需要重建聚簇索引
- ☐ Permits Concurrent DML：此变更进行时，是否允许其他 DML 变更同一张表。此特性关系到变更是否会长时间阻塞业务。
- ☐ Only Modifies Metadata：此变更是否只变更元信息，不涉及数据变更。

为了容易理解 DDL 的分类，下图中，我们穷举了 MySQL DDL 文档中的分类。

列出了这五个维度的每种组合情况，每种情况中分别挑选一例典型进行讨论。以下分类是按照 DDL 的成本从低到高排序。

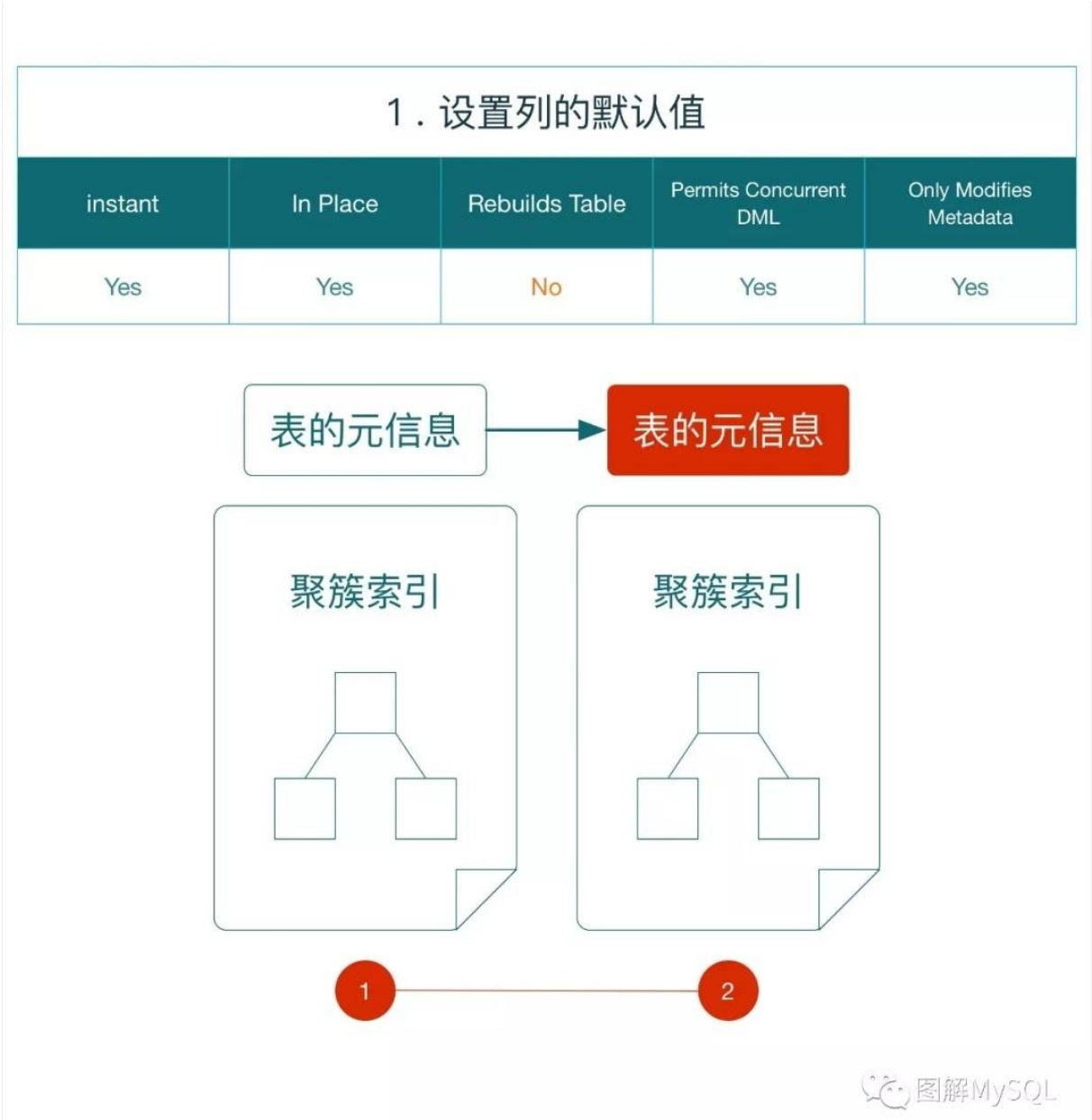


图 2

```
e.g. ALTER TABLE `t1`  
      ALTER COLUMN `c1` SET DEFAULT '1';
```

修改列的默认值不需要变动已有的数据页，仅需要修改表的元信息即可，所以这是成本最低的一种情况，可以"立刻"完成。

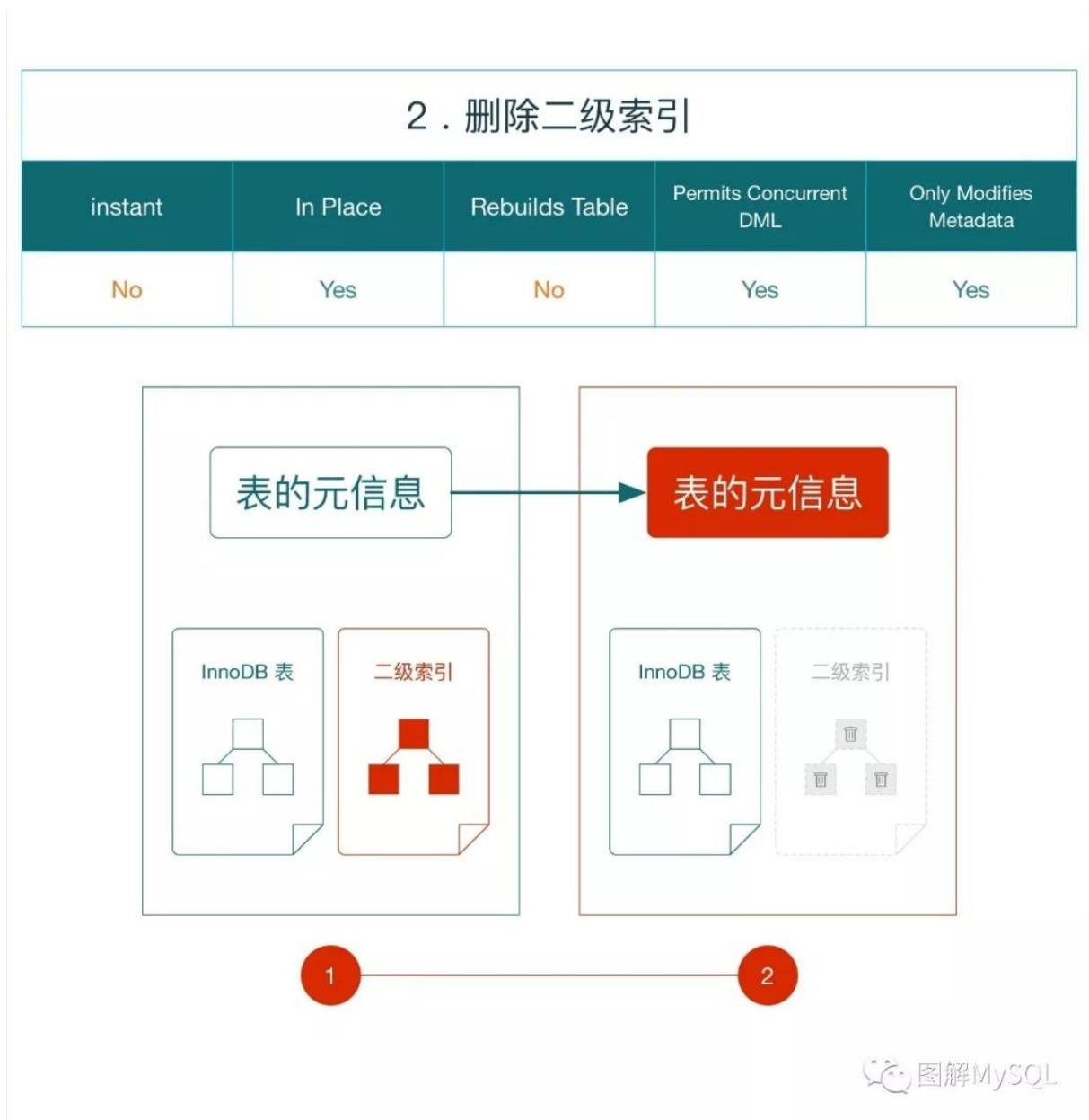


图 3

```
e.g. ALTER TABLE `t1`  
      DROP INDEX `idx1`;
```

删除二级索引除了修改表的元信息之外，需要将对应的二级索引标记为删除状态，因为不需要真的删除，仅仅设置标记量，所以这仍然是一种成本较低的情况。

但由于需要等待所有访问表的事务全部结束后才能成功，所以不算是“立刻”能完成的 DDL。

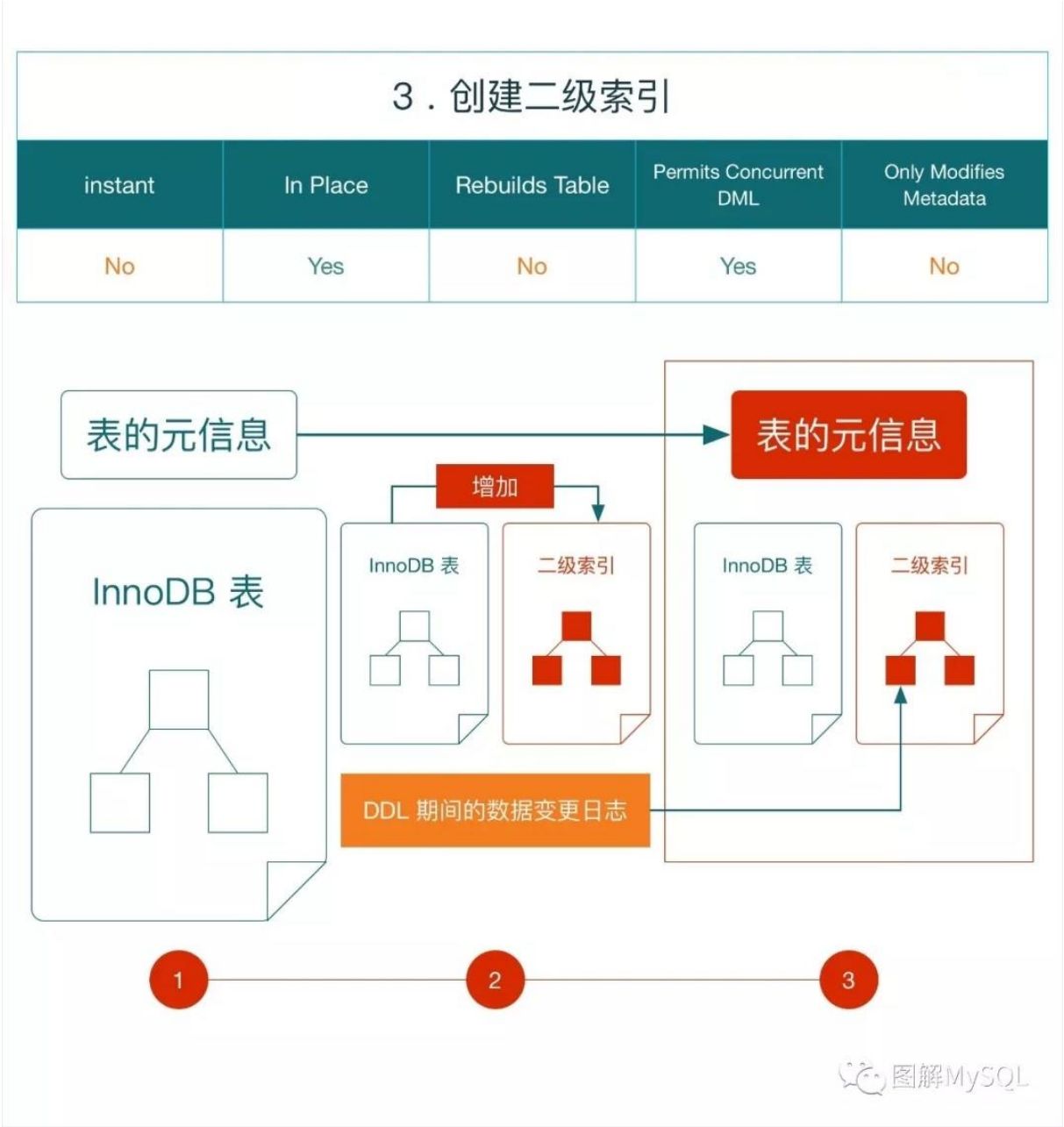
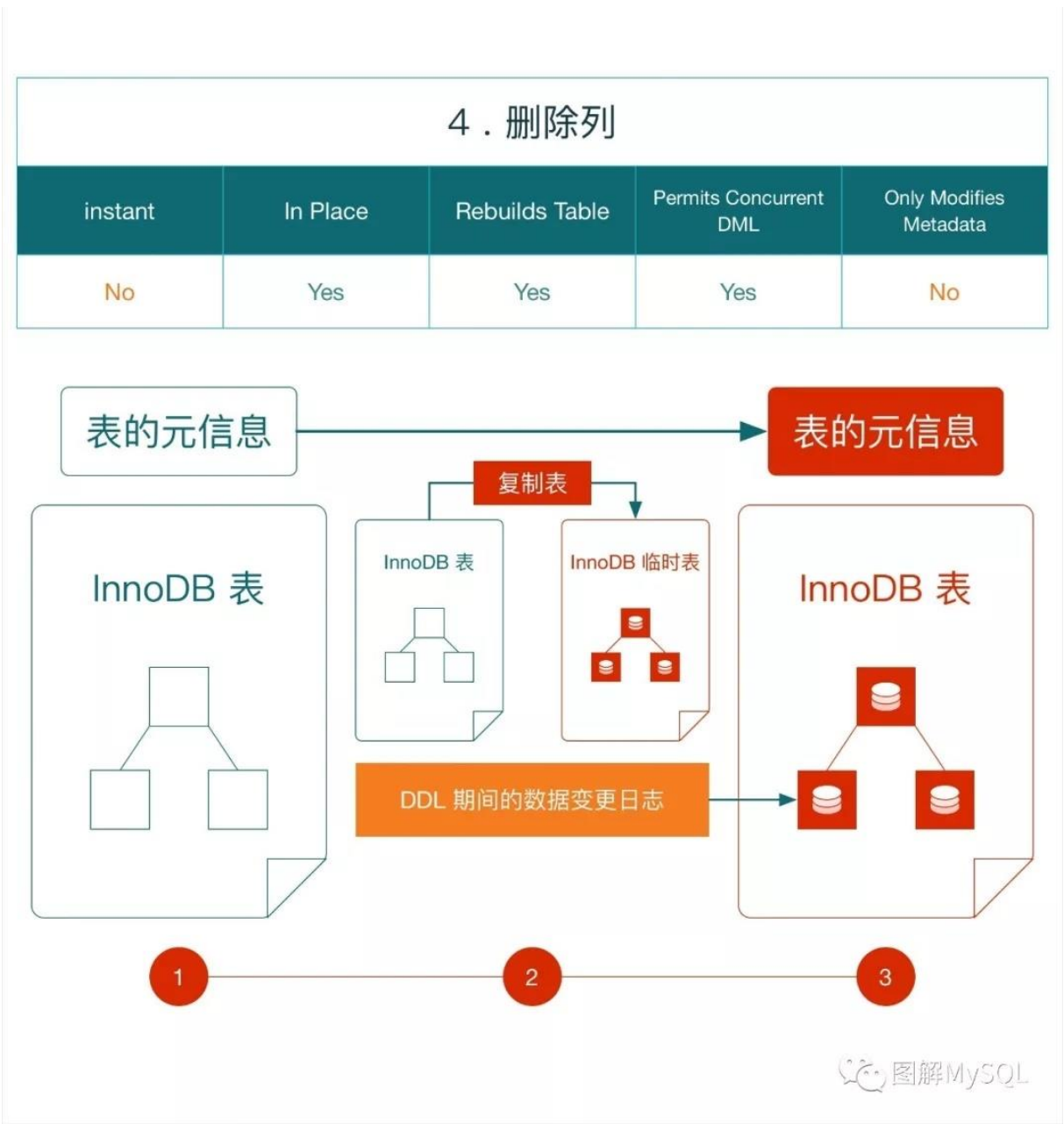


图 4

```
e.g. ALTER TABLE `t1`  
      ADD INDEX `idx1` (`name`(10) ASC);
```

创建二级索引除了修改表元信息之外，还需要在存储引擎层建立相应的二级索引结构。

为了支持并发的 DML 操作，MySQL 还需要额外维护一份 DDL 期间的数据变更日志，在 DDL 操作最后将并发的 DML 操作回放至新建的二级索引。不过由于二级索引是通过聚簇索引构造，不需要包含所有的行数据，所以这还不能算是一种较高成本的操作。



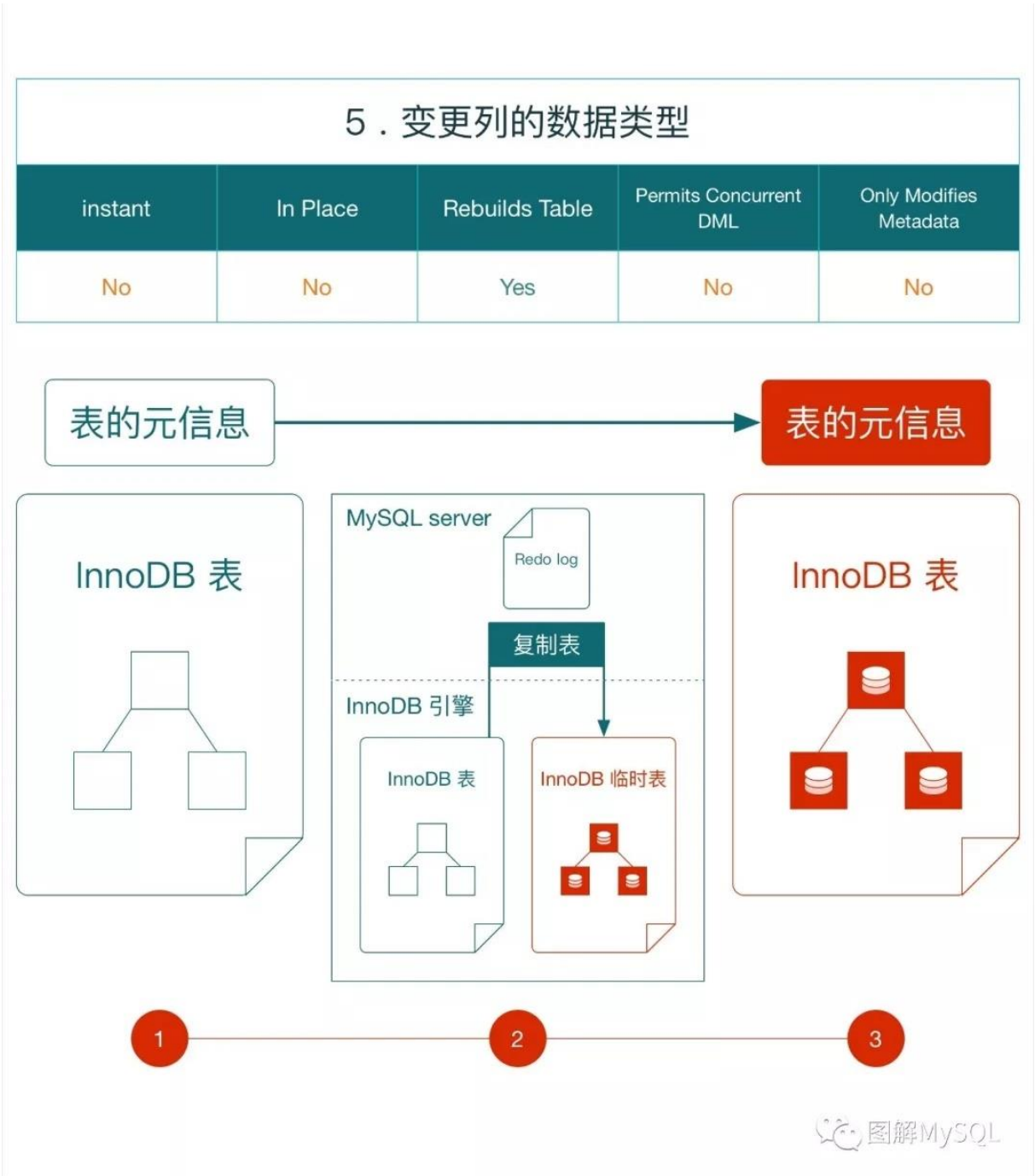


图 6

```
e.g. ALTER TABLE `t1`  
      MODIFY COLUMN `c1` INTEGER;
```

变更数据列类型，按照文档描述这是一种无法 Inplace 的操作，即需要 MySQL 在 server 层完成一次表的复制，相比由 InnoDB 内部完成重建，这种操作需要记录 Redo log，占用更多的 buffer pool。不过由于在执行过程中，无法并发 DML 操作，不需要记录 DDL 期间的变更日志。即便如此，这仍然是一种高成本的操作。

运维建议

- ☐ DDL 应显式指定 ALGORITHM，从低成本(INSTANT)到高成本(COPY)逐一尝试，当不匹配时 MySQL 会报错。以防我们认为的一个低成本的 DDL，因为认为失误而需要重建表，造成运维事故。
- ☐ 在以前版本中，MySQL 的 DDL 都需要重建表，所以会建议将一个表的多个变更写在同一句 DDL 中，用一次重建实施多个变更。
而现在，如果一句 DDL 中的多个变更的算法不同，那么会使用其中最高成本的算法。
运维中，需要仔细甄别情况，使得一部分变更可以更快完成上线。
- ☐ DDL 语句允许我们选择锁类型和 DDL 类型，给予我们更好的自由度。
比如当执行删除列时，MySQL 默认使用的是 Inplace Rebuild 操作，锁级别是 None（允许并发读写）。如果业务可以妥协，那么可以将锁级别设置为 SHARED（允许并发读但阻塞写），这样 DDL 可以更快完成。

思考题

- ☐ 本期我们说添加列需要重建表，而 MySQL 引入的腾讯团队的 Instant 方案 目标就是 让添加列“立刻”完成。那么 Instant 版本的添加列是如何完成的？
- ☐ Instant 是否完全不影响业务，是否是真正的“立刻”完成？
- ☐ 对于 Inplace Rebuild，重建数据的过程与并发的 DML 如何不互相干扰？

微信扫一扫读原文



【原理解析】MySQL 为表添加列是怎么"立刻"完成的

作者：黄炎 王悦 周海鸣

在上一期图解 图解 MySQL | MySQL DDL 为什么成本高？中，我们介绍了：

- 传统情况下，为表添加列需要对表进行重建
- 腾讯团队为 MySQL 引入了 Instant Add Column 的方案（以下称为“立刻加列”功能）可以快速完成 为表添加列 的任务

同时我们留了以下思考题：

- “立刻加列”是如何工作的？
- 所谓“立刻加列”是否完全不影响业务，是否是真正的“立刻”完成？

本期我们针对这几个问题来进行讨论：

传统情况

先回顾一下，在没有“立刻加列”功能时，加列操作是怎么完成的。我们也借此来熟悉一下本期的图例：



图 1 第 1 部分



图 1 第 2 部分

- 当进行 加列操作 时，所有的数据行 都必须 增加一段数据（图中的 列 4 数据）
- 如上一期图解所讲，当改变数据行的长度，就需要 重建表空间（图中灰蓝的部分为发生变更的部分）
- 数据字典中的列定义也会被更新

以上操作的问题在于每次加列操作都需要重建表空间，这就需要大量 IO 以及大量的时间。

立刻加列

“立刻加列”的过程如下图：



图 2

- “立刻加列”时，只会变更数据字典中的内容，包括：
 - 在列定义中增加 新列的定义
 - 增加 新列的默认值
- “立刻加列”后，当要读取表中的数据时：
 - 由于“立刻加列”没有 变更行数据，读取的行数据只有 3 列
 - MySQL 会将 新增的第 4 列的默认值，追加到 读取的数据后

以上过程描述了如何读取 在“立刻加列”之前 写入的数据，其实质是：在读取数据的过程中，“伪造”了一个新列出来。

那么如何读取 在“立刻加列”之后 写入的数据呢？过程如下图：

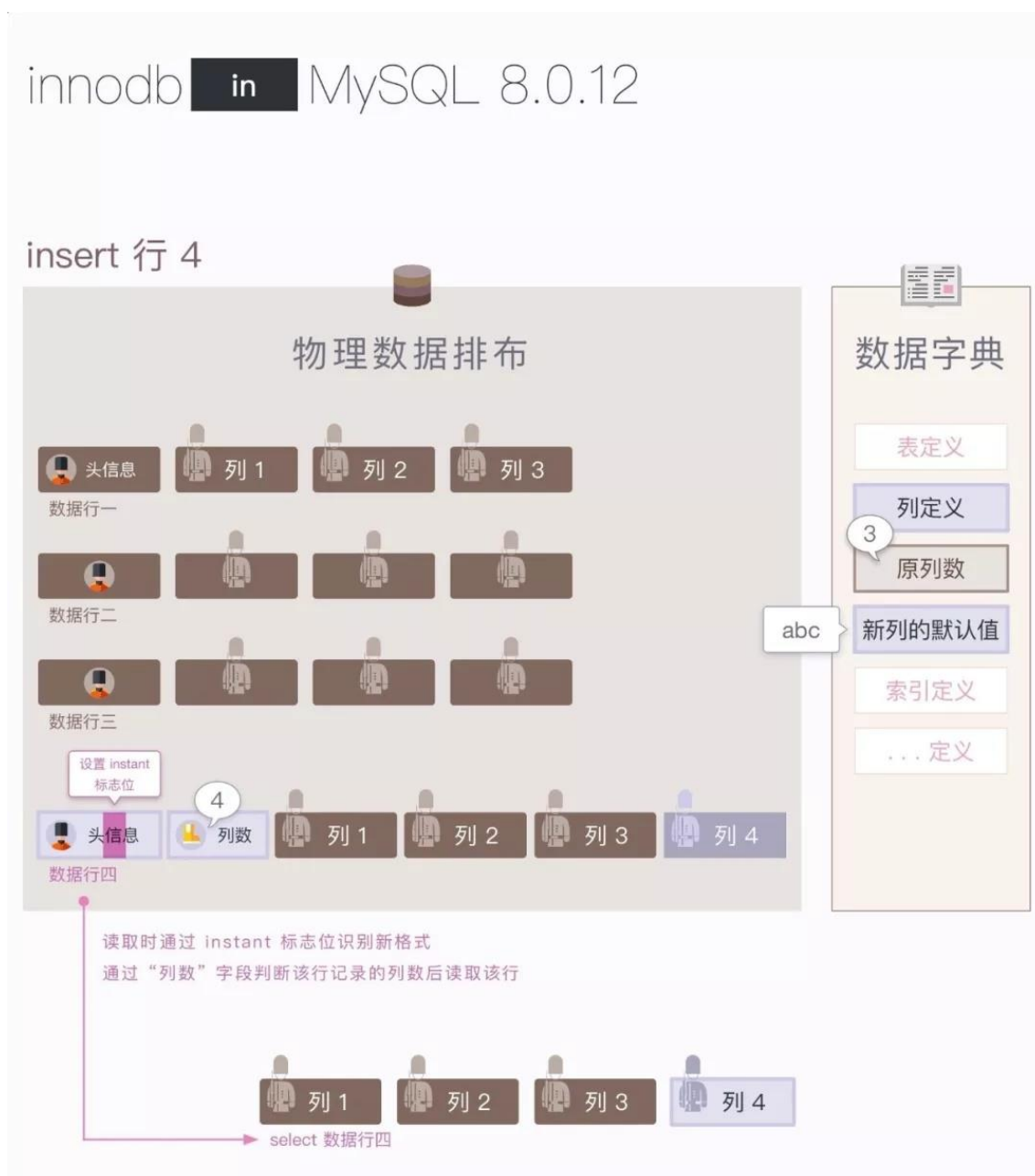


图 3

当读取 行 4 时：

- 通过判断 数据行的头信息中的 `instant` 标志位，可以知道该行的格式是“新格式”：该行头信息后有一个新字段“列数”
- 通过读取 数据行的 `“列数”` 字段，可以知道 该行数据中多少列有“真实”的数据，从而按列数读取数据

通过上图可以看到：读取 在“立刻加列”前/后写入的数据是不同的流程

通过以上的讨论，我们可以总结“立刻加列”之所以高效的原因是：

- 在执行“立刻加列”时，不变更数据行的结构
- 读取“旧”数据时，“伪造”新增的列，使结果正确
- 写入“新”数据时，使用了新的数据格式（增加了 `instant` 标志位 和 `“列数”` 字段），以区分新旧数据
- 读取“新”数据时，可以如实读取数据

那么 我们是否能一直“伪造”下去？“伪造”何时会被拆穿？

考虑以下场景：

1. 用“立刻加列”增加列 A
2. 写入数据行 1
3. 用“立刻加列”增加列 B
4. 写入数据行 2
5. 删除列 B

我们推测一下“删除列 B”的最小代价：需要修改 数据行中的 `instant` 标志位或 `“列数”` 字段，这至少会影响到“立刻加列”之后写入的数据行，成本类似于重建数据

从以上推测可知：当出现 与“立刻加列”操作不兼容 的 DDL 操作时，数据表需要进行**重建**，如下图所示：



图 4

扩展思考题：是否能设计其他的数据格式，取代 `instant` 标志位和“列数”字段，使得 加列/删列 操作都能“立刻完成”？（提示：考虑 加列 - 删列 - 再加列 的情况）

使用限制

在了解原理之后，我们来看看“立刻加列”的使用限制，就很容易能理解其中的前两项：

1. “立刻加列”的加列位置只能在表的最后，而不能加在其他列之间
在元数据中，只记录了 数据行 应有多少列，而没有记录 这些列 应出现的位置。所以无法实现指定列的位置
2. “立刻加列”不能添加主键列
加列 不能涉及聚簇索引的变更，否则就变成了“重建”操作，不是“立刻”完成了
3. “立刻加列”不支持压缩的表格式
按照 WL 的说法：“COMPRESSED is no need to supported”（没必要支持不怎么用的格式）

总结回顾

我们总结一下上面的讨论：

1. “立刻加列”之所以高效的原因是：
 - a) 在执行“立刻加列”时，不变更数据行的结构
 - b) 读取“旧”数据时，“伪造”新增的列，使结果正确
 - c) 写入“新”数据时，使用了新的数据格式（增加了 `instant` 标志位 和“列数”字段），以区分新旧数据
 - d) 读取“新”数据时，可以如实读取数据
2. “立刻加列”的“伪造”手法，不能一直维持下去。当发生 与“立刻加列”操作不兼容 的 DDL 时，表数据就会发生重建

回到之前遗留的两个问题：

- ☐ “立刻加列”是如何工作的？
我们已经解答了这个问题。
- ☐ 所谓“立刻加列”是否完全不影响业务，是否是真正的“立刻”完成？

可以看到：就算是“立刻加列”，也需要变更 数据字典，那么 该上的锁还是逃不掉的。也就是说 这里的“立刻”指的是“不变更数据行的结构”，而并非指“零成本地完成”

本期仍然留下一个思考题：

本文中描述了 在“立刻加列”之后 插入 数据行的情况（数据行会使用新格式）。那么在“立刻加列”之后 更新 数据行会发生什么情况呢？



【原理解析】XtraBackup 备份恢复时为什么要加 apply-log-only 参数？

作者：黄炎 王悦 周海鸣

XtraBackup 在 MySQL 备份场景中被广泛使用，大家一定不陌生。我们也在之前的两篇文章中分享了其备份的原理。（详见 [原理解析] XtraBackup 全量备份还原 & [原理解析] XtraBackup 增量备份还原）

本文想要描述的是 XtraBackup 恢复时参数 apply-log-only 的作用，不知道大家有没有注意到，这个参数如果不设置，可能会产生数据不一致的惨剧。

使用 XtraBackup 对数据库做备份，实际上就是拷贝 MySQL 的数据文件，为了保证备份数据的最终一致，也会同时拷贝备份过程中的 Redo log。



图 1 第 1 部分



图 1 第 2 部分

如图 1 所示，全备开始时，事务 2 尚未提交，Redo log 中仅有事务 2 的一部分数据 (B→F)，XtraBackup 于是将这一部分数据拷贝到全备文件中。

进行一次增备

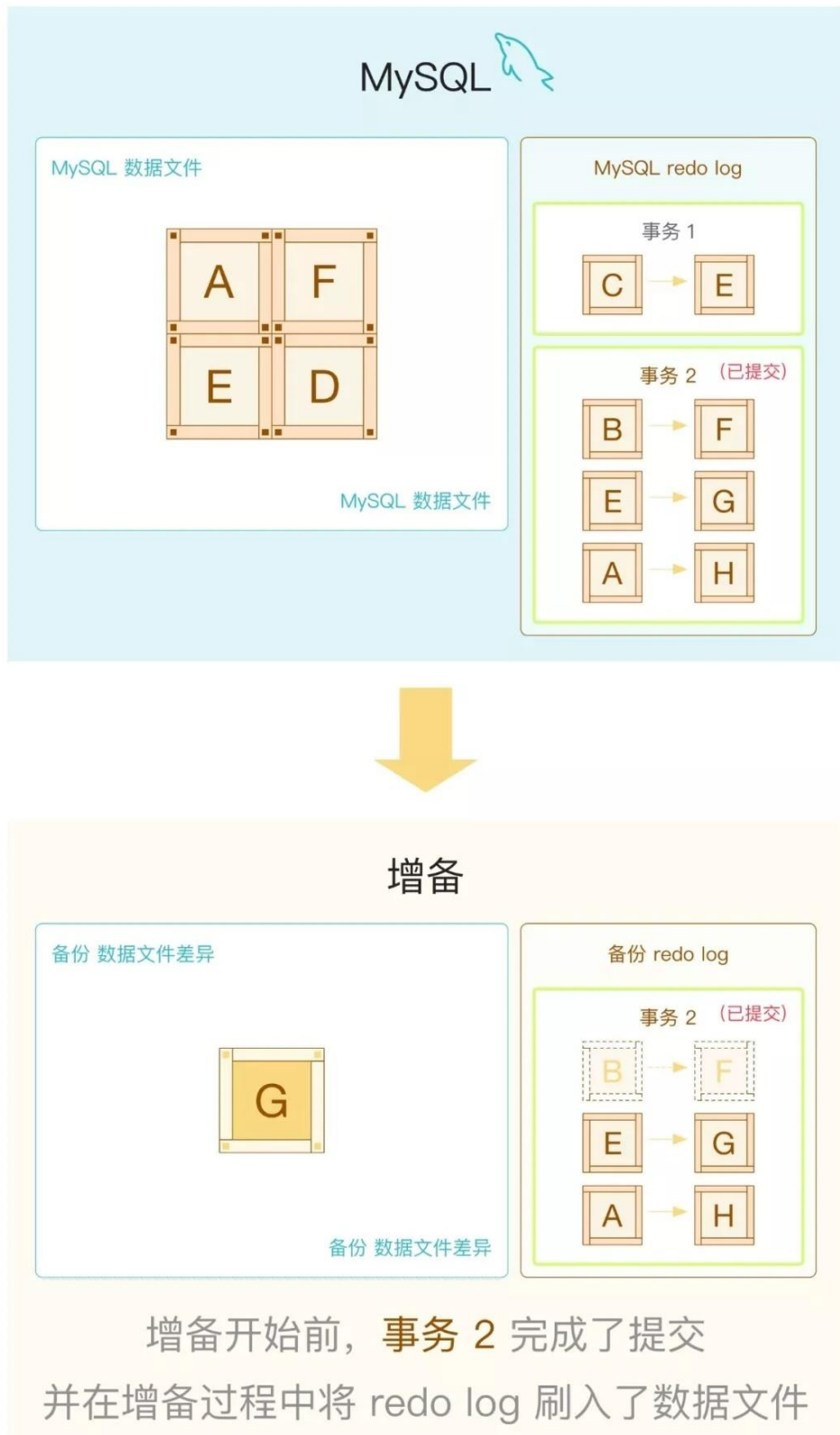


图 2 第 1 部分



图 2 第 2 部分

如图 2 所示，增备开始时，事务 2 已经提交，Redo log 中有了事务 2 的完整数据。XtraBackup 于是将事务 2 的后一部分数据拷贝了下来。

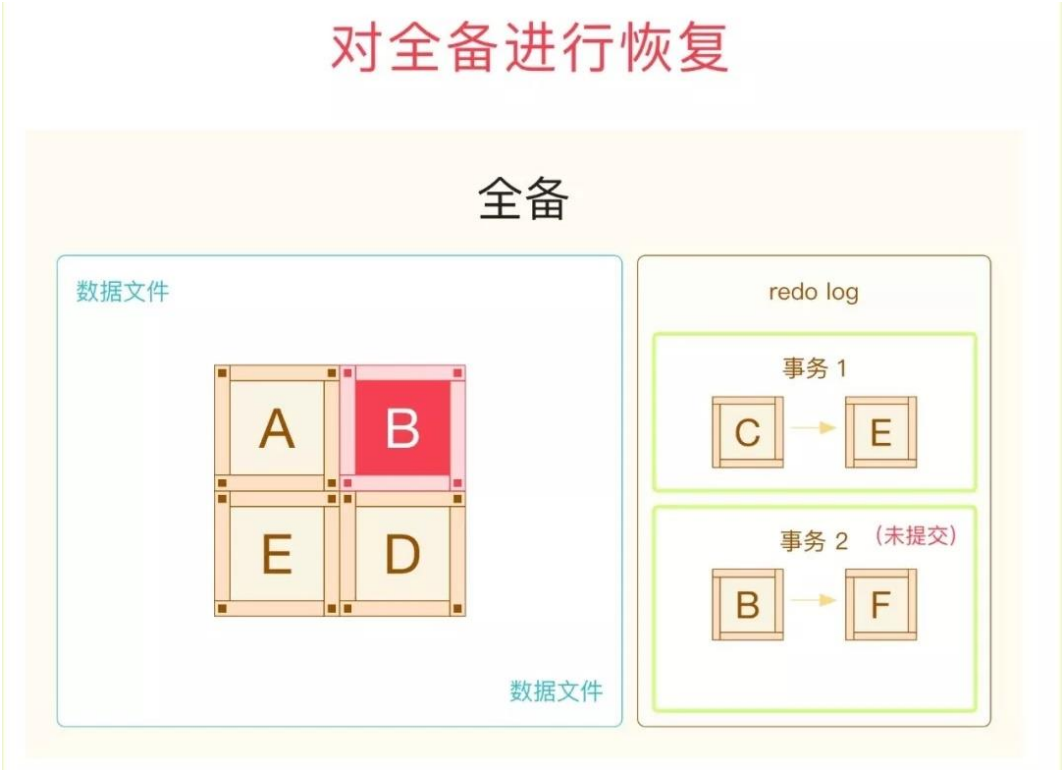


图 3 第 1 部分



图 3 第 2 部分



图 3 第 3 部分

如图 3 所示，恢复时，首先恢复全备中的数据文件，然后回放全备中的 Redo log，回放过程中应用了一部分事务 2(B→F)，如果没有设置 apply-log-only，XtraBackup 会在恢复最后一步应用 undo，将这一部分残缺的事务回滚(F→B)，就此埋下了祸根。



图 4 第 1 部分

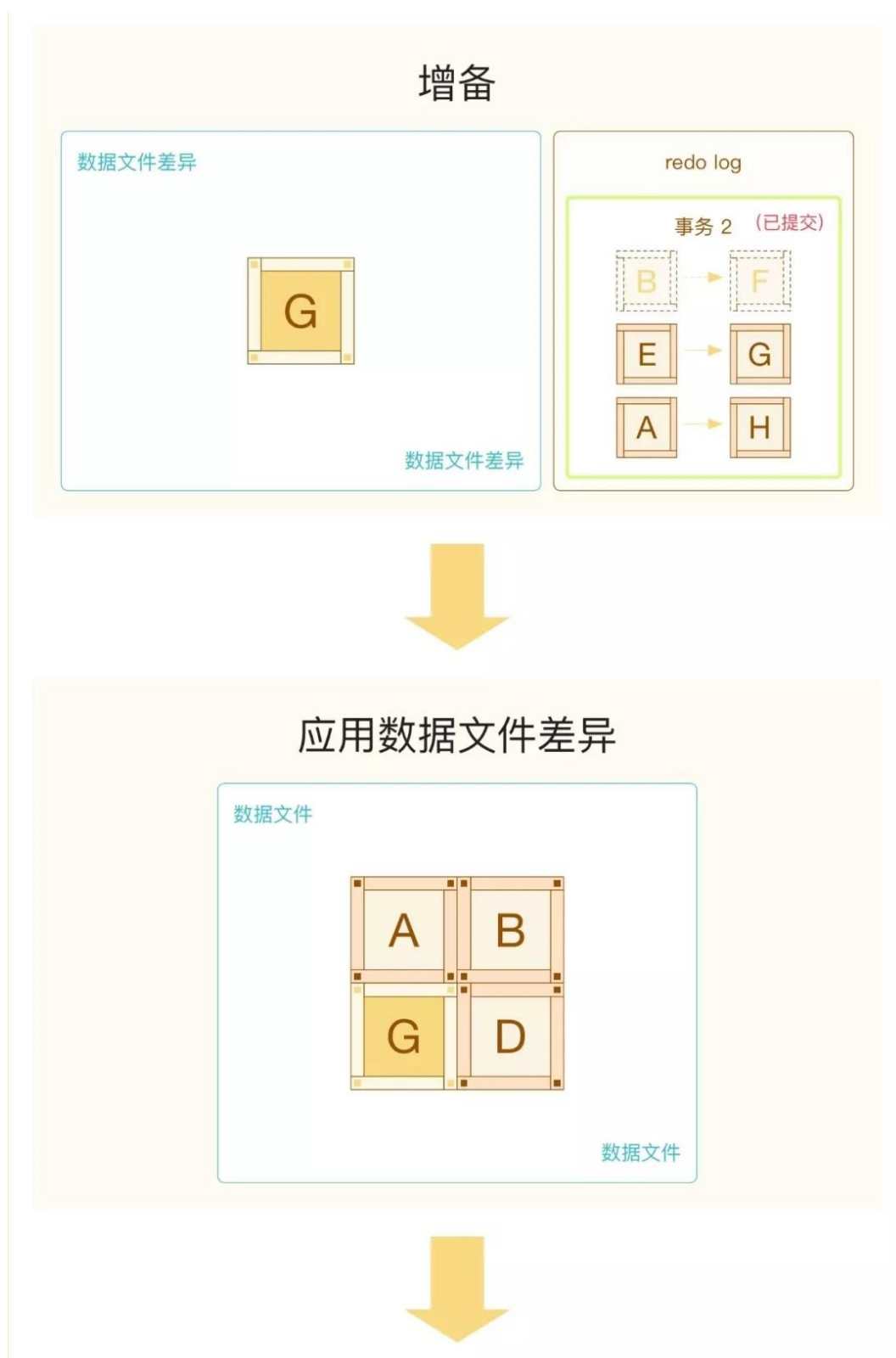


图 4 第 2 部分



图 4 第 3 部分

如图 4 所示，后续恢复增备文件时，继续回放了事务 2 的后一部分 (E→G A→H)，导致最终的数据文件中丢失了事务 2 第一部分的数据 (B→F)，惨剧就发生了。



图 5 第 1 部分



图 5 第 2 部分



图 5 第 3 部分

如图 5 所示，正确的做法应该是除了最后一个增备，所有的备份恢复都应该设置 `apply-log-only` 参数（only 指的就是只回放 redo log 阶段，跳过 undo 阶段），避免未完成事务的回滚。如图所示，此时全备恢复后的数据文件才是完整的（包含了 B→F）。所有增备恢复完成后的数据也是完整的。

参考：

https://www.percona.com/doc/percona-xtrabackup/2.3/xtrabackup_bin/xbk_option_reference.html#cmdoption-xtrabackup-apply-log-only

新特性系列

MySQL 8 官方全年共发版 4 次。除了进一步修复完善已有功能，还发布了很多振奋人心的新功能。本系列共精选 24 篇新特性文章，将今年 8.0 的最主要的新特性，还有在 5.7 与 8.0 过渡时需要注意的技术差异。为您抽丝剥茧，娓娓道来。

微信扫一扫阅读原文



MySQL 5.7 升级到 MySQL 8.0 的注意事项

作者：郭斌斌

1 引言

近期项目进行 MySQL 5.7.21 到 MySQL 8.0.13 的升级测试，采用逻辑升级，配置文件来自于生产环境。在初始化 MySQL 8.0 时，初始化命令秒级完成，而数据目录却是空的，执行初始化操作的 shell 窗口也没有任何的报错提示。

通过翻阅官方手册发现 MySQL 8.0.13 中 NO_AUTO_CREATE_USER 这种 sql_mode 已经废弃，而配置文件的 sql_mode 有这个配置项，最终导致了实例初始化失败。为了减少升级过程中出现类似问题，因此对 MySQL 5.7 到 8.0 的升级进行部分整理，主要包括：**升级对 MySQL 版本的要求、升级都做了哪些内容、数据库升级做了哪些步骤以及注意事项。**

2 升级要求

1. MySQL 5.7 升级到 MySQL 8.0，要求 MySQL 5.7 的版本是 MySQL 5.7.9 或者更高的 GA 版
2. 不支持跨版本升级

3 升级内容

1. 升级数据字典表的版本
2. 升级 MySQL server 版本，主要包括 system 表以及其他 schema 对象

4 升级步骤

1. 升级数据字典，这个阶段主要升级 MySQL 库的数据字典表。数据字典的升级在数据库服务启动过程自动完成。升级 MySQL 库的系统表（剩余的非字典表）；升级
2. Performance schema、information schema、sys schema；升级用户 schema。在 MySQL 8.0.16 版本

之前，需要手动的执行 MySQL_upgrade 来完成该步骤的升级，在 MySQL 8.0.16 版本是由 MySQLd 来完成该步骤的升级。

5 升级注意事项

1. caching_sha2_password 认证插件提供更多的密码加密方式，并且在加密方面具有更好的表现，目前 MySQL 8.0 选用 caching_sha2_password 作为默认的认证插件，MySQL 5.7 的认证插件是 MySQL_native_password。如果客户端版本过低，会造成无法识别 MySQL 8.0 的加密认证方式，最终导致连接问题。
2. MySQL 存储引擎现负责提供自己的分区处理程序，而 MySQL 服务器不再提供通用分区支持，InnoDB 和 NDB 是唯一提供 MySQL 8.0 支持的本地分区处理程序的存储引擎。如果分区表用的是别的存储引擎，存储引擎须进行修改。要么将其转换为 InnoDB 或 NDB，要么删除其分区。通过 mysqldump 从 5.7 获取的备份文件，在导入到 8.0 环境前，确保创建分区表语句中指定的存储引擎必须支持分区，否则会报错。
3. MySQL 8.0 的默认字符集 utf8mb4，可能导致之前数据的字符集跟新建对象的字符集不一致，为避免新旧对象字符集不一致的情况，可以在配置文件将字符集和校验规则设置为旧版本的字符集和校验规则。
4. MySQL 8.0 启动使用的 lower_case_table_names 值必须跟初始化时使用的一致。使用不同的设置重新启动服务器会引入与标识符的排序和比较方式不一致的问题。
<lower_case_table_names> https://dev.mysql.com/doc/refman/8.0/en/server-systemvariables.html#sysvar_lower_case_table_names
5. 要避免 MySQL 8.0 上的启动失败，MySQL 配置文件中的 sql_mode 系统变量不能包含 NO_AUTO_CREATE_USER。
6. 从 MySQL 5.7.24 和 MySQL 8.0.13 开始，mysqldump 从存储程序定义中删除了 NO_AUTO_CREATE_USER。必须手动修改使用早期版本的 mysqldump 创建的转储文件，以删除 NO_AUTO_CREATE_USER。
7. 在 MySQL 8.0.11 中，删除了这些不推荐使用的兼容性 SQL Mode: DB2, MAXDB, MSSQL, MySQL323, MySQL40, ORACLE, POSTGRESQL, NO_FIELD_OPTIONS, NO_KEY_OPTIONS, NO_TABLE_OPTIONS。从 5.7 到 8.0 的复制场景中，如果语句使用到废弃的 SQL Mode 会导致复制异常。
8. 在执行到 MySQL 8.0.3 或更高版本的 in-place 升级时，BACKUP_ADMIN 权限自动授予具有 RELOAD 权限的用户。
9. 本文对 MySQL 5.7 到 MySQL 8.0 的升级过程中出现部分易出现问题进行整理：升级对 MySQL 版本的要求、升级都做了哪些内容、数据库升级做了哪些步骤以及注意事项，希望对大家版本升级有帮助。

参考文档：

<https://dev.mysql.com/doc/refman/8.0/en/upgrade-binary-package.html>

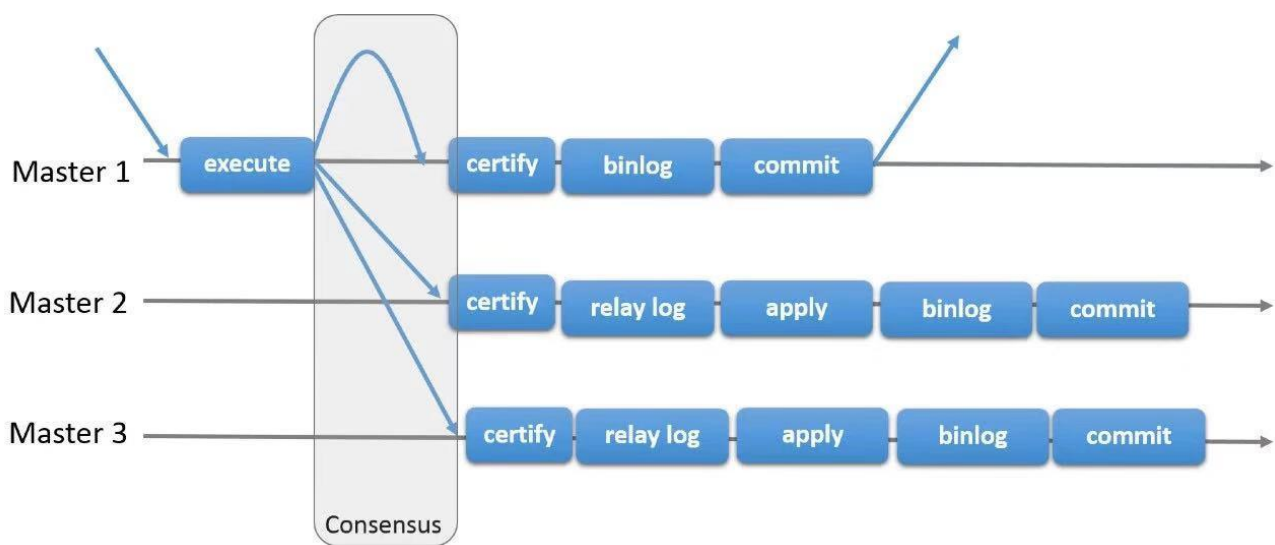


MySQL MGR “一致性读写”特性解读

作者：田帅萌

MySQL 8.0.14 版本增加了一个新特性：MGR 读写一致性；有了此特性，“妈妈”再也不用担心读 MGR 非写节点数据会产生不一致啦。

有同学会疑问：“MGR 不是‘全同步’么，也会产生读写不一致？”，在此肯定的告诉大家 MGR 会产生读写不一致，原因如下：



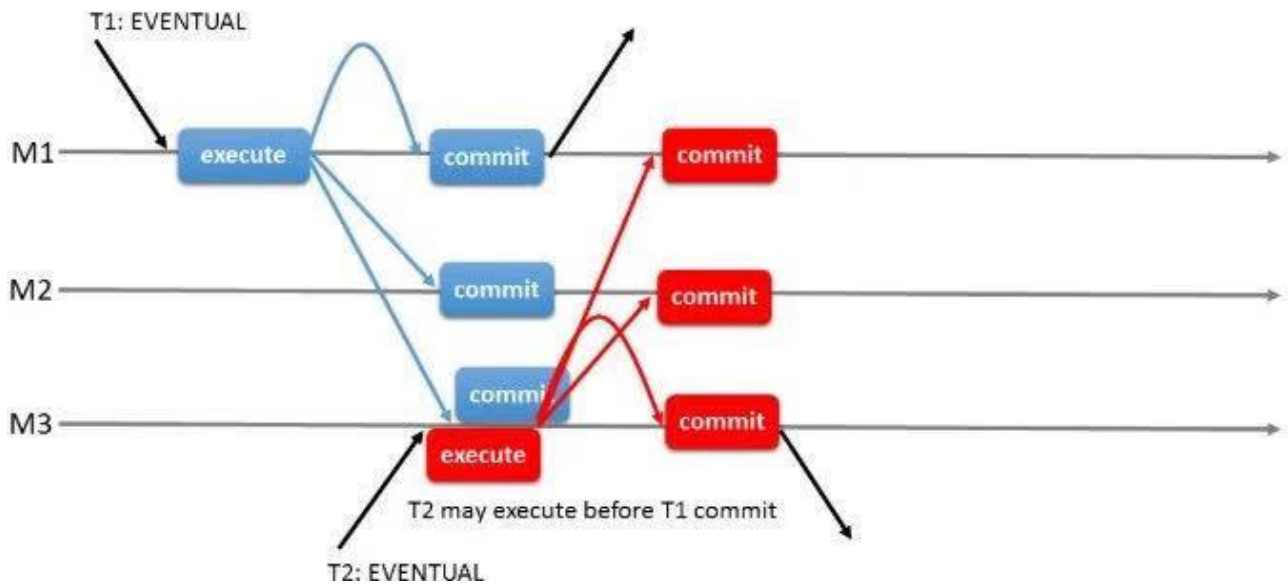
MGR 相对于半同步复制，在 relay log 前增加了冲突检查协调，但是 binlog 回放仍然可能延时，也就是跟我们熟悉的半同步复制存在 io 线程的回放延迟情况类似。当然关于 IO 线程回放慢的原因，跟半同步也类似，比如大事务！！

所以 MGR 并不是全同步方案，关于如何处理一致性读写的问题，MySQL 在 8.0.14 版本中加入了“读写一致性”特性，并引入了参数：`group_replication_consistenc`，下面将对读写一致性的相关参数及不同应用场景进行详细说明。

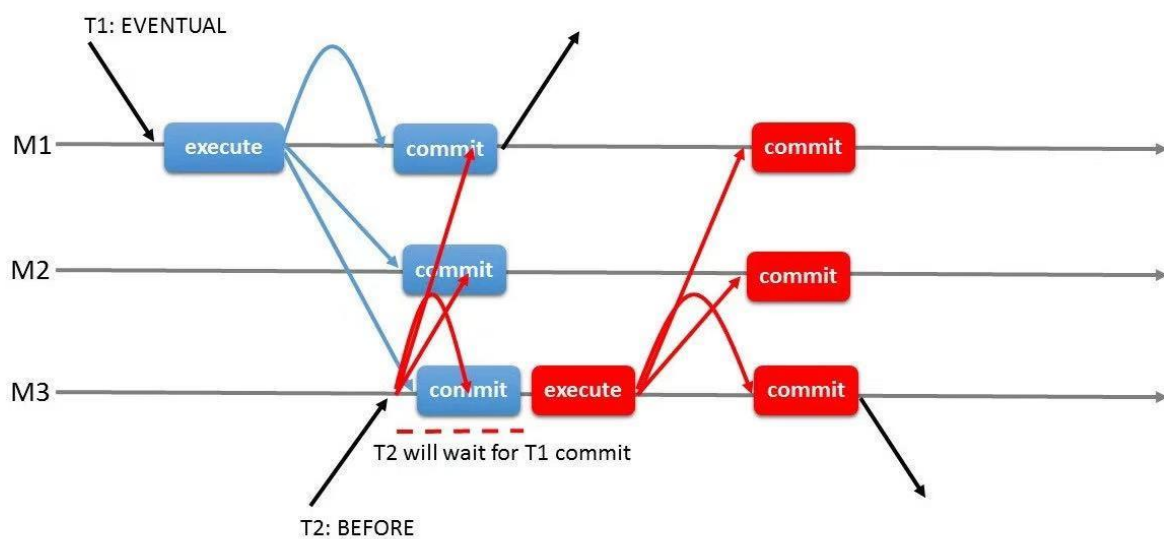
1 参数 `group_replication_consistenc` 的说明

1.1 可选配置值

1. **EVENTUAL** 默认值，开启该级别的事务（T2），事务执行前不会等待先序事务（T1）的回放完成，也不会影响后序事务等待该事务回放完成。



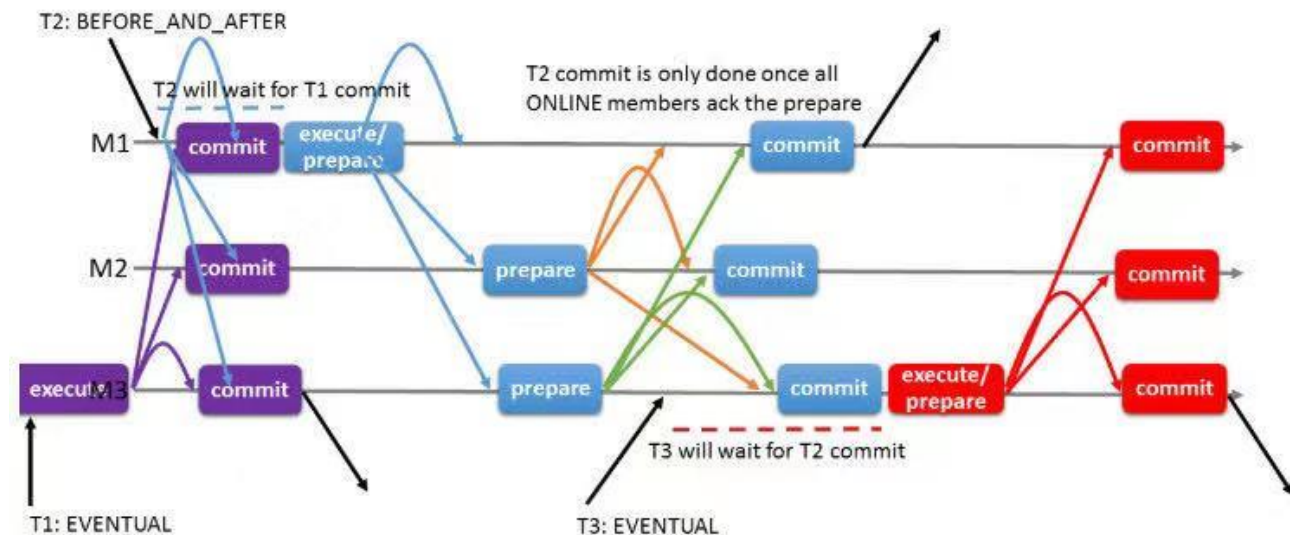
2. **BEFORE** 开启了该级别的事务（T2），在开始前首先要等待先序事务（T1）的回放完成，确保此事务将在最新的数据上执行。



3. **AFTER**，开启该级别的事务（T1），只有等该事务回放完成。其他后序事务（T2）才开始执行，这样所有后序事务都会读取包含其更改的数据库状态，而不管它们在哪个成员上执行。



4. **BEFORE_AND_AFTER** 开启该级别等事务（T2），需要等待前序事务的回放完成（T1）；同时后序事务（T3）等待该事务的回放完成；



5. **BEFORE_ON_PRIMARY_FAILOVER**，在发生切换时，连到新主的事务会被阻塞，等待先序提交的事务回放完成；这样确保在故障切换时客户端都能读取到主服务器上的最新数据，保证了一致性
group_replication_consistency 参数可以用法 SESSION, GLOBAL 去进行更改。

官方说明请参考：

<https://dev.mysql.com/doc/refman/8.0/en/group-replication-options.html>

2 MGR 读写一致性的优缺点

官方引入的 MGR 读写一致性既有它自身的天然优势，也不可避免的存在相应的不足，其优缺点如下：

1. **优点：**MGR 配合中间件，比如 DBLE 这类有读写分离功能的中间件，在 MGR 单主模式下，可以根据业务场景进行读写分离，不用担心会产生延迟，充分利用了 MGR 主节点以外的节点。
2. **缺点：**使用读写一致性会对性能有极大影响，尤其是网络环境不稳定的场景下。

在实际应用中需要大家因地制宜，根据实际情况选择最适配的方案。

3 MGR 读写一致性的方案

针对不同应用场景应当如何选择 MGR 读写一致性的相关方式，官方提供了几个参数以及与其相对应的应用场景：

3.1 AFTER

1. **适用场景 1：**写少读多的场景进行读写分离，担心读取到过期事务，可选择 AFTER。
2. **适用场景 2：**只读为主的集群，有 RW 的事务需要保证提交的事务能被其他后序事务读到最新数据，可选择 AFTER。

3.2 BEFORE

1. **适用场景 1：**应用大量写入数据，偶尔进行读取一致性数据，应当选择 BEFORE。
2. **适用场景 2：**有特定事务需要读写一致性，以便对敏感数据操作时，始终读取最新的数据；应当选择 BEFORE。

3.3 BEFORE_AND_AFTER

适用场景：有一个读为主的集群，有 RW 的事务既要保证读到最新的数据，又要保证这个事务提交后，被其他后序事务读到；在这种情况下可选择 BEFORE_AND_AFTER。

3.4 在特定会话上设置一致性

1. **举例 1：**某一事务语句，需要其他节点的数据强一致性。
可以使用 `SET @@SESSION.group_replication_consistency= 'AFTER'` 进行设置。
2. **举例 2：**跟例 1 相似，在每天执行分析语句事务并且需要获得读取新数据的情况下。
可以使用 `SET @@SESSION.group_replication_consistency= 'BEFORE'` 进行设置

参考文档：

<https://dev.mysql.com/doc/refman/8.0/en/group-replication-configuring-consistency-guarantees.html#group-replication-choose-consistency-level>



MySQL 8.0 通用表达式

作者：杨涛涛

通用表达式在各个商业数据库中比如 ORACLE, SQL SERVER 等早就实现了, MySQL 到了 8.0 才支持这个特性。这里有两个方面来举例说明 WITH 的好处: 第一易用性, 第二效率。

1 举例一 WITH 表达式的易用性

我们第一个例子, 对比视图的检索和 WITH 的检索。我们知道视图在 MySQL 里面的效率一直较差, 虽说 MySQL 5.7 对视图做了相关固化的优化, 不过依然不尽人意。考虑下, 如果多次在同一条 SQL 中访问视图, 那么则会多次固化视图, 势必增加相应的资源消耗。

MySQL 里之前对这种消耗的减少只有一种, 就是动态处理, 不过一直语法较为恶心, 使用不是很广。

MySQL 8.0 后, 又有了一种减少消耗的方式, 就是 WITH 表达式。我们假设以下表结构:

```
| CREATE TABLE `t1` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `rank1` int(11) DEFAULT NULL,
  `rank2` int(11) DEFAULT NULL,
  `log_time` datetime DEFAULT NULL,
  `prefix_uid` varchar(100) DEFAULT NULL,
  `desc1` text,
  PRIMARY KEY (`id`),
  KEY `idx_rank1` (`rank1`),
  KEY `idx_rank2` (`rank2`),
  KEY `idx_log_time` (`log_time`)
) ENGINE=InnoDB AUTO_INCREMENT=2037 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_0900_ai_ci
```

有 1000 行测试记录。

```
mysql> select count(*) from t1;
+-----+
| count(*) |
+-----+
|    1024 |
+-----+
1 row in set (0.00 sec)
```

这里我们建立一个普通的视图：

```
CREATE ALGORITHM=merge SQL SECURITY DEFINER VIEW `v_t1_20` AS
select count(0) AS `cnt`,`t1`.`rank1` AS `rank1`
from `t1`
group by `t1`.`rank1`
order by `t1`.`rank1`
limit 20;
```

1. 检索语句 A:

对视图里的最大和最小值字段 rank1 进行过滤检索出符合条件的记录行数。

```
mysql> use ytt;
mysql> SELECT
    COUNT(*)
FROM
    t1
WHERE
    rank1 = (SELECT
        MAX(rank1)
    FROM
        v_t1_20)
    OR rank1 = (SELECT
        MIN(rank1)
    FROM
        v_t1_20);
+-----+
| count(*) |
+-----+
|      65 |
+-----+
1 row in set (0.00 sec)
```

我们用 WITH 表达式来重写一遍这个查询。

2. 查询语句 B:

```
mysql> use ytt;
mysql> with w_t1_20(cnt,rank1) as (
SELECT
    COUNT(*), rank1
FROM
    t1
GROUP BY rank1
ORDER BY rank1
LIMIT 20
```

```

) SELECT
    COUNT(*)
FROM
    t1
WHERE
    rank1 = (SELECT
        MAX(rank1)
    FROM
        w_t1_20)
    OR rank1 = (SELECT
        MIN(rank1)
    FROM
        w_t1_20);

```

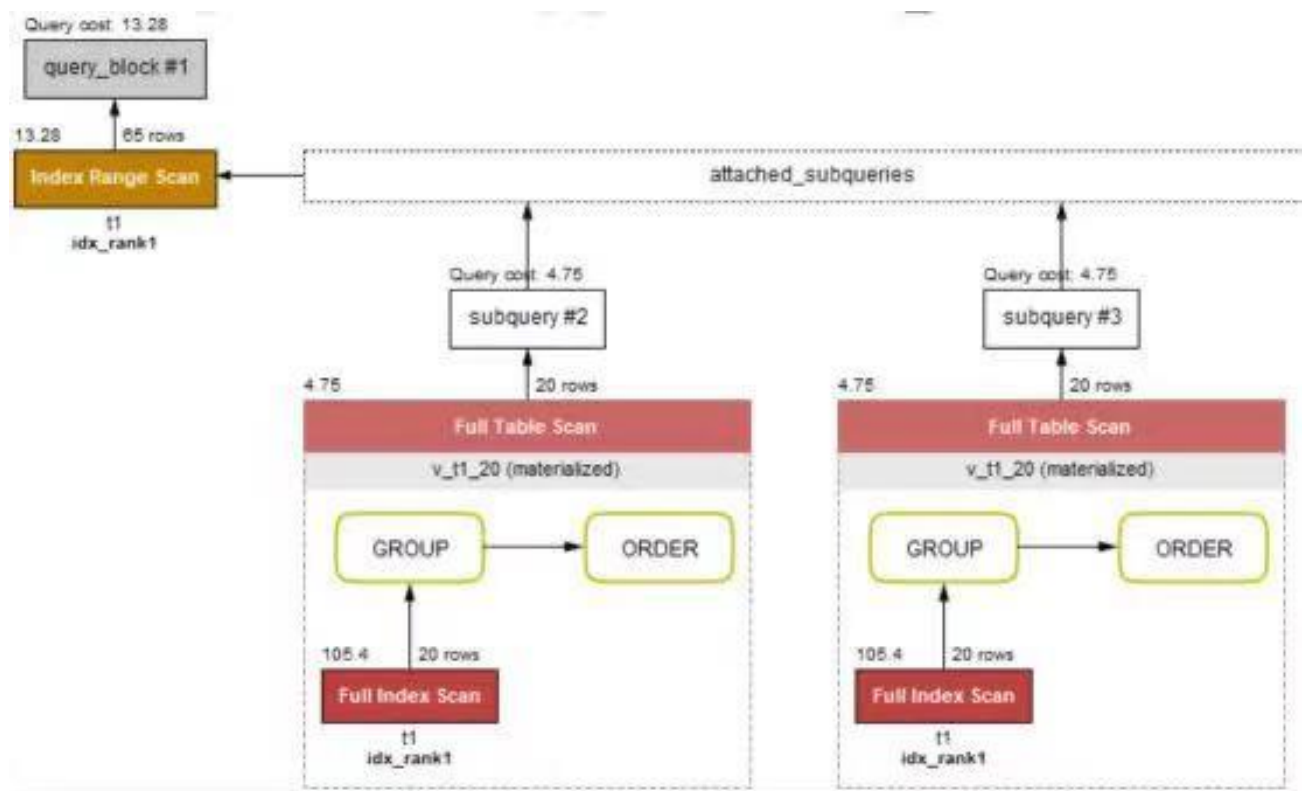
```

+-----+
| count(*) |
+-----+
|      65 |
+-----+
1 row in set (0.00 sec)

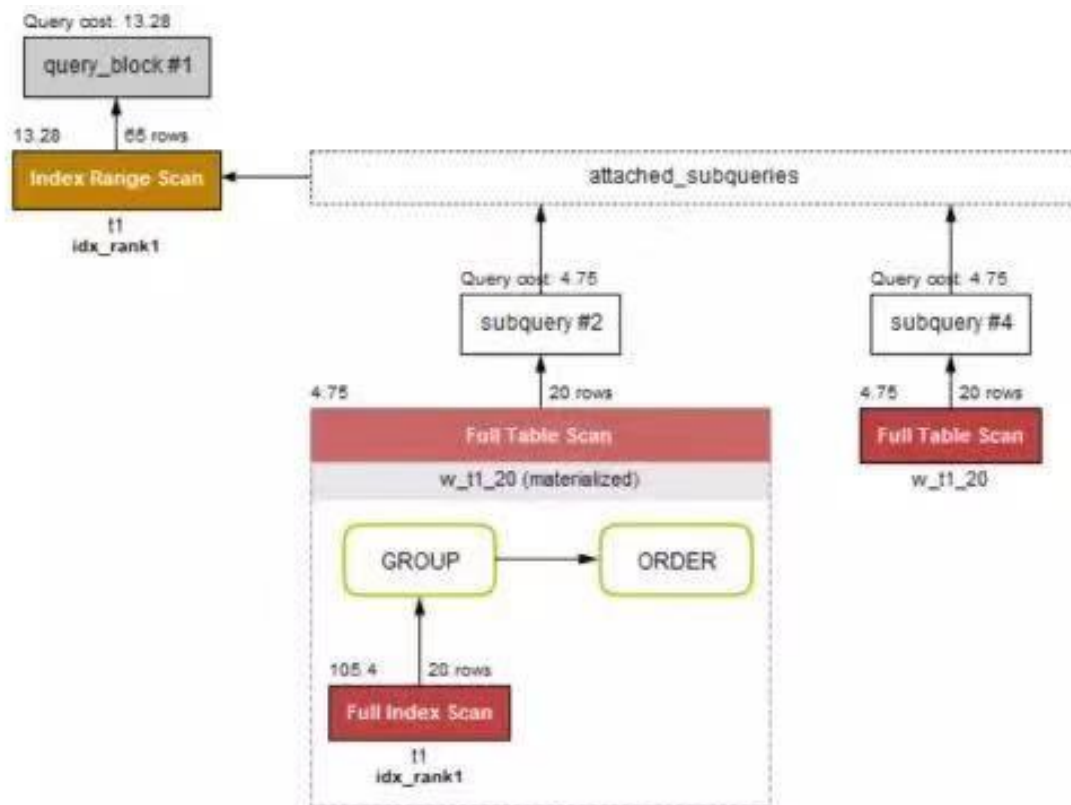
```

我的函数很少，仅作功能性演示，索引表面上看执行时间差不多，我们来对比下两条实现语句的查询计划，

1. A 的计划：



2. B 的计划:



从以上图我们可以看出，B 比 A 少了一次对视图的固化，也就是说，不管我访问 WITH 多少次，仅仅固化一次。有兴趣的可以加大数据量，加大并发测试下性能。

举例二 WITH 表达式的功能性

我们第二个例子，简单说功能性。

比如之前 MySQL 一直存在的一个问题，就是临时表不能打开多次。我们以前只有一种解决办法就是把临时表固化到磁盘，像访问普通表那样访问临时表。现在我们可以用 MySQL 8.0 自带的 WITH 表达式来做这样的业务。

比如以下临时表：

```
create temporary table ytt_tmp1 as
SELECT
    COUNT(*) cnt, rank1
FROM
    t1
GROUP BY rank1
ORDER BY rank1
LIMIT 20
```

我们还是用之前的查询，这里会提示错误。

```
mysql> select count(*) from t1 where rank1 = (select max(rank1) from ytt_tmp1) or
rank1 = (select min(rank1) from ytt_tmp1);
ERROR 1137 (HY000): Can't reopen table: 'ytt_tmp1'
```

现在我们可以用 WITH 来改变这种思路。

```
mysql> use ytt;
mysql> with w_t1_20(cnt,rank1) as (
SELECT
    COUNT(*), rank1
FROM
    t1
GROUP BY rank1
ORDER BY rank1
LIMIT 20
) SELECT
    COUNT(*)
FROM
    t1
WHERE
    rank1 = (SELECT
        MAX(rank1)
    FROM
        w_t1_20)
    OR rank1 = (SELECT
        MIN(rank1)
    FROM
        w_t1_20);
+-----+
| count(*) |
+-----+
|      65 |
+-----+
1 row in set (0.00 sec)
```

当然 WITH 的用法还有很多，感兴趣的可以去看看手册上的更深入的内容。



MySQL 8.0 索引特性 1-函数索引

作者：杨涛涛

函数索引顾名思义就是加给字段加了函数的索引，这里的函数也可以是表达式。所以也叫表达式索引。

MySQL 5.7 推出了虚拟列的功能，MySQL 8.0 的函数索引内部其实也是依据虚拟列来实现的。

我们考虑以下几种场景：

1. 对比日期部分的过滤条件。

```
SELECT ...  
FROM tbl  
WHERE date(time_field1) = current_date;
```

2. 两字段做计算。

```
SELECT ...  
FROM tbl  
WHERE field2 + field3 = 5;
```

3. 求某个字段中间某子串。

```
SELECT ...  
FROM tbl  
WHERE substr(field4, 5, 9) = 'actionsky';
```

4. 求某个字段末尾某子串。

```
SELECT ...  
FROM tbl  
WHERE RIGHT(field4, 9) = 'actionsky';
```

5. 求 JSON 格式的 VALUE。

```
SELECT ...  
FROM tbl  
WHERE CAST(field4 ->> '$.name' AS CHAR(30)) = 'actionsky';
```

以上五个场景如果不用函数索引，改写起来难易不同。不过都要做相关修改，不是过滤条件修正就是表结构变更添加冗余字段加额外索引。

比如第 1 个场景改写为，

```
SELECT ...
FROM tb1
WHERE time_field1 >= concat(current_date, ' 00:00:00')
      AND time_field1 <= concat(current_date, '23:59:59');
```

再比如第 4 个场景的改写，

由于是求最末尾的子串，只能添加一个新的冗余字段，并且做相关的计划任务来一定频率的异步更新或者添加触发器来实时更新此字段值。

```
SELECT ...
FROM tb1
WHERE field4_suffix = 'actionsky';
```

那我们看到，改写也可以实现，不过这样的 SQL 就没有标准化而言，后期不能平滑的迁移了。

MySQL 8.0 推出来了函数索引让这些变得相对容易许多。

不过函数索引也有自己的缺陷，就是写法很固定，必须要严格按照定义的函数来写，不然优化器不知所措。

我们来把上面那些场景实例化。

示例表结构，

```
mysql> desc t_func;
+-----+-----+-----+-----+-----+-----+
| Field | Type                | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| id    | bigint(20) unsigned | NO   | PRI | NULL    | auto_increment |
| rank1 | int(11)              | YES  |     | NULL    |                 |
| str1  | varchar(20)          | YES  |     | NULL    |                 |
| str2  | json                 | YES  |     | NULL    |                 |
| rank2 | int(11)              | YES  |     | NULL    |                 |
| str3  | varchar(20)          | YES  |     | NULL    |                 |
| log_time | datetime             | YES  | MUL | NULL    |                 |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.00 sec)
```

总记录数

```
mysql> SELECT COUNT(*)
FROM t_func;
+-----+
| count(*) |
+-----+
|    16384 |
+-----+
1 row in set (0.01 sec)
```

我们把上面几个场景的索引全加上。

```
mysql > ALTER TABLE t_func ADD INDEX idx_log_time ( ( date( log_time ) ) ),
ADD INDEX idx_u1 ( ( rank1 + rank2 ) ),
ADD INDEX idx_suffix_str3 ( ( RIGHT ( str3, 9 ) ) ),
ADD INDEX idx_substr_str1 ( ( substr( str1, 5, 9 ) ) ),
ADD INDEX idx_str2 ( ( CAST( str2 ->> '$.name' AS CHAR ( 9 ) ) ) );
QUERY OK,
0 rows affected ( 1.13 sec ) Records : 0 Duplicates : 0 WARNINGS : 0
```

我们再看下表结构，发现好几个已经被转换为系统自己的写法了。

```
| t_func | CREATE TABLE `t_func` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `rank1` int(11) DEFAULT NULL,
  `str1` varchar(20) DEFAULT NULL,
  `str2` json DEFAULT NULL,
  `rank2` int(11) DEFAULT NULL,
  `str3` varchar(20) DEFAULT NULL,
  `log_time` datetime DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_log_time_normal` (`log_time`),
  KEY `idx_log_time` ((cast(`log_time` as date))),
  KEY `idx_u1` (((`rank1` + `rank2`))),
  KEY `idx_suffix_str3` ((right(`str3`,9))),
  KEY `idx_substr_str1` ((substr(`str1`,5,9))),
  KEY `idx_str2` ((cast(json_unquote(json_extract(`str2`,_utf8mb4'$.name')) as char(9) charset utf8mb4)))
) ENGINE=InnoDB AUTO_INCREMENT=32754 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci |
```

MySQL 8.0 还有一个特性，就是可以把系统隐藏的列显示出来。

我们用 show extended 列出函数索引创建的虚拟列，

```
mysql> show extended columns from t_func;
```

Field	Type	Null	Key	Default	Extra
id	bigint(20) unsigned	NO	PRI	NULL	auto_increment
rank1	int(11)	YES		NULL	
str1	varchar(20)	YES		NULL	
str2	json	YES		NULL	
rank2	int(11)	YES		NULL	
str3	varchar(20)	YES		NULL	
log_time	datetime	YES	MUL	NULL	
0efe34628f1961d032603cefc6e91e17	date	YES	MUL	NULL	VIRTUAL GENERATED
c6ffeeac1cbba9b52b9da42f5d160a4f	bigint(12)	YES	MUL	NULL	VIRTUAL GENERATED
afe4487f38093a35dfc26c4731b98935	varchar(9)	YES	MUL	NULL	VIRTUAL GENERATED
098af34813a0d04cbb9e5b14183d3d04	varchar(9)	YES	MUL	NULL	VIRTUAL GENERATED
e6853cae11b8bc49921fc9ba30251c7d	varchar(9)	YES	MUL	NULL	VIRTUAL GENERATED
DB_TRX_ID		NO		NULL	
DB_ROLL_PTR		NO		NULL	

```
14 rows in set (0.00 sec)
```

上面 5 个随机字符串列名为函数索引隐式创建的虚拟 COLUMNS。

我们先来看看场景 2，两个整形字段的相加，

```
mysql> SELECT COUNT(*)
FROM t_func
```

```
WHERE rank1 + rank2 = 121;
```

```
+-----+
| count(*) |
+-----+
|      878 |
+-----+
1 row in set (0.00 sec)
```

看下执行计划，用到了 `idx_u1` 函数索引，

```
mysql> explain SELECT COUNT(*) FROM t_func WHERE rank1 + rank2 = 121\G
***** 1. row *****

      id: 1
  select_type: SIMPLE
        table: t_func
  partitions: NULL
         type: ref
possible_keys: idx_u1
          key: idx_u1
        key_len: 9
         ref: const
        rows: 878
   filtered: 100.00
      Extra: NULL
1 row in set, 1 warning (0.00 sec)
```

那如果我们稍微改下这个 SQL 的执行计划，发现此时不能用到函数索引，变为全表扫描了，所以要严格按照函数索引的定义来写 SQL。

```
mysql> explain SELECT COUNT(*) FROM t_func WHERE rank1 = 121 - rank2\G
***** 1. row *****

      id: 1
  select_type: SIMPLE
        table: t_func
  partitions: NULL
         type: ALL
possible_keys: NULL
          key: NULL
        key_len: NULL
         ref: NULL
        rows: 16089
   filtered: 10.00
      Extra: Using where
1 row in set, 1 warning (0.00 sec)
```

再来看看场景 1 的的改写和不改写的性能简单对比，

```
mysql> SELECT *
FROM t_func
WHERE date(log_time) = '2019-04-18' LIMIT 1\G
***** 1. row *****
      id: 2
     rank1: 1
    str1: test-actionsky-test
    str2: {"age": 30, "name": "dell"}
     rank2: 120
    str3: test-actionsky
log_time: 2019-04-18 10:04:53
1 row in set (0.01 sec)
```

我们把普通的索引加上。

```
mysql > ALTER TABLE t_func ADD INDEX idx_log_time_normal ( log_time );
QUERY OK,
0 rows affected ( 0.36 sec ) Records : 0 Duplicates : 0 WARNINGS : 0
```

然后改写下 SQL 看下。

```
mysql> SELECT *
FROM t_func
WHERE date(log_time) >= '2019-04-18 00:00:00'
      AND log_time < '2019-04-19 00:00:00'
***** 1. row *****
      id: 2
     rank1: 1
    str1: test-actionsky-test
    str2: {"age": 30, "name": "dell"}
     rank2: 120
    str3: test-actionsky
log_time: 2019-04-18 10:04:53
1 row in set (0.01 sec)
```

两个看起来没啥差别，我们仔细看下两个的执行计划：

1. 普通索引

```
mysql> explain format=json SELECT *
FROM t_func
WHERE log_time >= '2019-04-18 00:00:00'
      AND log_time < '2019-04-19 00:00:00'
LIMIT 1\G
***** 1. row *****
EXPLAIN: {
```

```
"query_block": {
  "select_id": 1,
  "cost_info": {
    "query_cost": "630.71"
  },
  "table": {
    "table_name": "t_func",
    "access_type": "range",
    "possible_keys": [
      "idx_log_time_normal"
    ],
    "key": "idx_log_time_normal",
    "used_key_parts": [
      "log_time"
    ],
    "key_length": "6",
    "rows_examined_per_scan": 1401,
    "rows_produced_per_join": 1401,
    "filtered": "100.00",
    "index_condition": "((`ytt`.`t_func`.`log_time` >= '2019-04-18 00:00:00') and
d (`ytt`.`t_func`.`log_time` < '2019-04-19 00:00:00'))",
    "cost_info": {
      "read_cost": "490.61",
      "eval_cost": "140.10",
      "prefix_cost": "630.71",
      "data_read_per_join": "437K"
    },
    "used_columns": [
      "id",
      "rank1",
      "str1",
      "str2",
      "rank2",
      "str3",
      "log_time",
      "cast(`log_time` as date)",
      "(`rank1` + `rank2`)",
      "right(`str3`,9)",
      "substr(`str1`,5,9)",
      "cast(json_unquote(json_extract(`str2`,_utf8mb4'$.name')) as char(9) chars
et utf8mb4)"
    ]
  }
}

1 row in set, 1 warning (0.00 sec)
```


2. 函数索引

```
mysql> explain format=json SELECT COUNT(*)
FROM t_func
WHERE date(log_time) = '2019-04-18'
LIMIT 1\G
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "308.85"
    },
    "table": {
      "table_name": "t_func",
      "access_type": "ref",
      "possible_keys": [
        "idx_log_time"
      ],
      "key": "idx_log_time",
      "used_key_parts": [
        "cast(`log_time` as date)"
      ],
      "key_length": "4",
      "ref": [
        "const"
      ],
      "rows_examined_per_scan": 1401,
      "rows_produced_per_join": 1401,
      "filtered": "100.00",
      "cost_info": {
        "read_cost": "168.75",
        "eval_cost": "140.10",
        "prefix_cost": "308.85",
        "data_read_per_join": "437K"
      },
      "used_columns": [
        "log_time",
        "cast(`log_time` as date)"
      ]
    }
  }
}
1 row in set, 1 warning (0.00 sec)
```

从上面的执行计划看起来区别不是很大，唯一不同的是，普通索引在 CPU 的计算上消耗稍微大点，见红色字体。当然，有兴趣的可以大并发的测试下，我这仅仅作作为功能性进行一番演示。



MySQL 8.0 索引特性 2-索引跳跃扫描

作者：杨涛涛

MySQL 8.0 实现了 Index skip scan，翻译过来就是索引跳跃扫描。熟悉 ORACLE 的朋友是不是发现越来越像 ORACLE 了？再者，熟悉 MySQL 5.7 的朋友是不是觉得这个很类似当时优化器的选项 MRR？好了，先具体说下什么 ISS，我后面全部用 ISS 简称。

*考虑以下的场景：

表 t1 有一个联合索引 idx_u1(rank1,rank2)，但是查询的时候却没有 rank1 这列，只有 rank2。
比如，select * from t1 where rank2 = 30。那以前遇到这样的情况，如果没有针对 rank2 这列单独建立普通索引，这条 SQL 怎么着都是走的 FULL TABLE SCAN。

ISS 就是在这样的场景下产生的。ISS 可以在查询过滤组合索引不包括最左列的情况下，走索引扫描，而不必要单独建立额外的索引。因为毕竟额外的索引对写开销很大，能省则省。

还是拿刚才的例子来讲，假设：表 t1 的两个字段 rank1,rank2。有这样的记录，

rank1	rank2
1	100
1	200
1	300
1	400
1	500
1	600
1	700
5	100
5	200
5	300
5	400
5	500

我们给出的 SQL 是，

```
select * from t1 where rank2 >400,
```

那 MySQL 通过 ISS 把这条 SQL 变为，

```
select * from t1 where rank1=1 and rank2 > 400
union all
select * from t1 where rank1 = 5 and rank2 > 400;
```

可以看出来，MySQL 其实内部自己把左边的列做了一次 DISTINCT，完了加进去。

我们拿实际的例子来看下。假设：

还是刚才描述那张表，rank1 字段值的 distinct 值比较少。查询计划的对比，

```
mysql> explain select * from t1 where rank2 =400\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: t1
   partitions: NULL
         type: range
possible_keys: idx_u1
          key: idx_u1
        key_len: 10
         ref: NULL
        rows: 44
   filtered: 100.00
      Extra: Using where; Using index for skip scan
1 row in set, 1 warning (0.00 sec)
```

关闭 ISS，

```
mysql> set @@optimizer_switch='skip_scan=off';
Query OK, 0 rows affected (0.00 sec)

mysql> explain select * from t1 where rank2 =400\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: t1
   partitions: NULL
         type: index
possible_keys: NULL
          key: idx_u1
        key_len: 10
         ref: NULL
        rows: 352
   filtered: 10.00
      Extra: Using where; Using index
1 row in set, 1 warning (0.00 sec)
```

很显然，ISS 扫描的行数要比之前的少很多。

ISS 其实恰好适合在这种左边字段的唯一值较少的情况下，效率来的高。比如性别，状态等等。

那假设：rank1 字段的 distinct 值比较多呢？

我们重新造了点数据，这次，rank1 的唯一值个数有快上万个。

```
mysql> select count(*) from (select rank1,count(*) from t1 group by rank1) t;
+-----+
| count(*) |
+-----+
|      9991 |
+-----+
1 row in set (0.05 sec)
```

我们来再次看一遍这样 SQL 的执行计划，

```
mysql> set @@optimizer_switch='skip_scan=on';
Query OK, 0 rows affected (0.00 sec)

mysql> explain select * from t1 where rank2 = 100\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: t1
   partitions: NULL
         type: index
possible_keys: udx_u1
          key: udx_u1
        key_len: 10
         ref: NULL
        rows: 65685
   filtered: 10.00
      Extra: Using where; Using index
1 row in set, 1 warning (0.00 sec)
```

```
mysql> set @@optimizer_switch='skip_scan=off';
Query OK, 0 rows affected (0.01 sec)

mysql> explain select * from t1 where rank2 = 100\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: t1
   partitions: NULL
         type: index
possible_keys: NULL
          key: udx_u1
        key_len: 10
         ref: NULL
        rows: 65685
   filtered: 10.00
      Extra: Using where; Using index
1 row in set, 1 warning (0.01 sec)
```

这次我们发现，无论如何 MySQL 也不会选择 ISS，而选了 FULL INDEX SCAN。
那这样的场景就必须给 rank2 加一个单独索引了。

```
mysql> alter table t1 add key idx_rank2 (rank2);
Query OK, 0 rows affected (0.75 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> explain select * from t1 where rank2 = 100\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: t1
   partitions: NULL
         type: ref
possible_keys: idx_rank2
          key: idx_rank2
        key_len: 5
         ref: const
        rows: 340
   filtered: 100.00
      Extra: NULL
1 row in set, 1 warning (0.00 sec)
```

那来总结下 ISS 就是一句话：ISS 其实就是 MySQL 8.0 推出的适合联合索引左边列唯一值较少的情况的一种优化策略。



MySQL 8.0 索引特性 3 -倒序索引

作者：杨涛涛

我们今天来介绍下 MySQL 8.0 引入的新特性：倒序索引。

MySQL 长期以来对索引的建立只允许正向 asc 存储，就算建立了 desc，也是忽略掉。

比如对于以下的查询，无法发挥索引的最佳性能。

1. 查询一：

```
select * from tbl where f1 = ... order by id desc;
```

2. 查询二：

```
select * from tbl where f1 = ... order by f1 asc , f2 desc;
```

那对于上面的查询，尤其是数据量和并发到一定峰值的时候，则对 OS 的资源消耗非常大。一般这样的 SQL 在查询计划里面会出现 using filesort 等状态。

比如针对下面的表 t1，针对字段 rank1 有两个索引，一个是正序的，一个是反序的。不过在 MySQL 8.0 之前的版本都是按照正序来存储。

```
mysql> select count(*) from t1;
+-----+
| count(*) |
+-----+
|    10000 |
+-----+
1 row in set (0.08 sec)

mysql> show create table t1\G
***** 1. row *****
        Table: t1
Create Table: CREATE TABLE `t1` (
  `id` int(11) NOT NULL,
  `rank1` int(11) DEFAULT NULL,
  `rank2` int(11) DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_rank1` (`rank1`),
  KEY `idx_rank1_desc` (`rank1`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
1 row in set (0.00 sec)
```



按照 rank1 正向排序的执行计划，

```
mysql> explain select rank1 from t1 group by rank1 order by rank1 asc limit 10\G
***** 1. row *****
      id: 1
    select_type: SIMPLE
        table: t1
    partitions: NULL
           type: range
possible_keys: idx_u1
           key: idx_u1
        key_len: 5
           ref: NULL
          rows: 101
    filtered: 100.00
      Extra: Using index for group-by
1 row in set, 1 warning (0.00 sec)
```



按照 rank1 反向排序的执行计划，

```
mysql> explain select rank1 from t1 group by rank1 order by rank1 desc limit 10\G
***** 1. row *****
      id: 1
    select_type: SIMPLE
        table: t1
    partitions: NULL
           type: range
possible_keys: idx_u1
           key: idx_u1
        key_len: 5
           ref: NULL
          rows: 101
    filtered: 100.00
      Extra: Using index for group-by; Using temporary; Using filesort
1 row in set, 1 warning (0.00 sec)
```



从执行计划来看，反向比正向除了 extra 里多了 **Using temporary; Using filesort** 这两个，其他的一模一样。这两个就代表中间用到了临时表和排序，一般来说，凡是执行计划里用到了这两个的，性能几乎都不咋地。除非我这个临时表不太大，而用于排序的 buffer 也足够大，那性能也不至于太差。那这两个选项到底对性能有多大影响呢？

我们分别执行这两个查询，并且查看 MySQL 的 session 级的 status 就大概能看出些许不同。

正向

```
mysql> show status like '%Handler%';
```

Variable_name	Value
Handler_commit	1
Handler_delete	0
Handler_discover	0
Handler_external_lock	2
Handler_mrr_init	0
Handler_prepare	0
Handler_read_first	1
Handler_read_key	11
Handler_read_last	1
Handler_read_next	0
Handler_read_prev	0
Handler_read_rnd	0
Handler_read_rnd_next	0
Handler_rollback	0
Handler_savepoint	0
Handler_savepoint_rollback	0
Handler_update	0
Handler_write	0

18 rows in set (0.01 sec)

反向

```
mysql> show status like '%Handler%';
```

Variable_name	Value
Handler_commit	1
Handler_delete	0
Handler_discover	0
Handler_external_lock	2
Handler_mrr_init	0
Handler_prepare	0
Handler_read_first	1
Handler_read_key	102
Handler_read_last	1
Handler_read_next	0
Handler_read_prev	0
Handler_read_rnd	10
Handler_read_rnd_next	101
Handler_rollback	0
Handler_savepoint	0
Handler_savepoint_rollback	0
Handler_update	0
Handler_write	100

18 rows in set (0.00 sec)

通过以上两张图，我们发现反向的比正向的多了很多个计数，比如通过扫描的记录数增加了 10 倍，而且还伴有 10 倍的临时表的读和写记录数。那这个开销是非常巨大的。那以上查询是在 MySQL 5.7 上运行的。

MySQL 8.0 给我们带来了倒序索引 (Descending Indexes)，也就是说反向存储的索引。这里不要跟搜索引擎中的倒排索引混淆了，MySQL 这里只是反向排序存储而已。不过这个倒序存储已经解决了很大的问题。我们再看下之前在 MySQL 5.7 上运行的例子。

我们把数据导入到 MySQL 8.0，

```
mysql> load data infile '/var/lib/mysql-files/t1.txt' into table t1;
Query OK, 10000 rows affected (1.82 sec)
Records: 10000 Deleted: 0 Skipped: 0 Warnings: 0
```

再把原来的索引变为倒序索引，

```
***** 1. row *****
Table: t1
Create Table: CREATE TABLE `t1` (
  `id` int(11) NOT NULL,
  `rank1` int(11) DEFAULT NULL,
  `rank2` int(11) DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_rank1_desc` (`rank1` DESC)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.01 sec)
```


再次看下第二个 SQL 的查询计划，

```
mysql> explain select rank1 from t1 group by rank1 order by rank1 desc limit 10\G
***** 1. row *****
      id: 1
    select_type: SIMPLE
        table: t1
    partitions: NULL
           type: range
possible_keys: idx_rank1_desc
           key: idx_rank1_desc
        key_len: 5
           ref: NULL
          rows: 101
     filtered: 100.00
      Extra: Using index for group-by
1 row in set, 1 warning (0.00 sec)
```



很显然，用到了这个倒序索引 `idx_rank1_desc`，而这里的临时表等的信息消失了。

当然了，这里的组合比较多，比如我这张表的字段 `rank1`, `rank2` 两个可以任意组合。

1. 组合一：

```
(rank1 asc,rank2 asc);
```

2. 组合二：

```
(rank1 desc,rank2 desc);
```

3. 组合三：

```
(rank1 asc,rank2 desc);
```

4. 组合四：

```
(rank1 desc,rank2 asc);
```

我把这几个加上，适合的查询比如：

1. 查询一：

```
Select * from t1 where rank1 = 11 order by rank2;
```

2. 查询二：

```
Select * from t1 where 1 order by rank1,rank2;
```

3. 查询三：

```
Select * from t1 where 1 order by rank1 desc,rank2;
```



MySQL 8.0 索引特性 4-不可见索引

作者：杨涛涛

MySQL 8.0 实现了索引的隐藏属性。当然这个特性很多商业数据库早就有了，比如 ORACLE，在 11g 中就实现了。我来介绍下这个小特性。

1 介绍

INVISIBLE INDEX，不可见索引或者叫隐藏索引。就是对优化器不可见，查询的时候优化器不会把她作为备选。

其实以前要想彻底不可见，只能用开销较大的 drop index；现在有了新的方式，可以改变索引的属性，让其不可见，这一操作只更改 metadata，开销非常小。

2 使用场景

我大概描述下有可能使用隐藏索引的场景：

1. 比如，我有张表 t1，本来已经有索引 idxf1，idxf2，idxf3。我通过数据字典检索到 idxf3 基本没有使用过，那我是不是可以判断这个索引直接删掉就好了？那如果删掉后突然有新上的业务要大量使用呢？难道我要频繁的 drop index/add index 吗？这个时候选择开销比较小的隐藏索引就好了。
2. 我的业务只有一个可能每个月固定执行一次的 SQL 用到这个索引，那选择隐藏索引太合适不过了。
3. 又或者是我想要测试下新建索引对我整个业务的影响程度。如果我直接建新索引，那我既有业务涉及到这个字段的有可能会收到很大影响。那这个时候隐藏索引也是非常合适的。

3 举例

下面我来简单举例如何使用隐藏索引：

表结构

```
mysql> create table f1 (id serial primary key, f1 int, f2 int );
Query OK, 0 rows affected (0.11 sec)
```

创建两个索引，默认可见。

索引 1，

```
mysql> alter table f1 add key idx_f1(f1), add key idx_f2(f2);
Query OK, 0 rows affected (0.12 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

索引 2,

```
mysql> show create table f1\G
***** 1. row *****
      Table: f1
Create Table: CREATE TABLE `f1` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `f1` int(11) DEFAULT NULL,
  `f2` int(11) DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `id` (`id`),
  KEY `idx_f1` (`f1`),
  KEY `idx_f2` (`f2`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

简单写个造数据的存储过程。

```
DELIMITER $$
USE `ytt`$$
CREATE PROCEDURE `sp_generate_data_f1` (
  IN f_cnt INT
)
BEGIN
  DECLARE i,j INT DEFAULT 0;

  SET @@autocommit=0;
  WHILE i < f_cnt DO
    SET i = i + 1;
    IF j = 100 THEN
      SET j = 0;
      COMMIT;
    END IF;
    SET j = j + 1;
    INSERT INTO f1 (f1,f2) SELECT CEIL(RAND()*100),CEIL(RAND()*100);
  END WHILE;
  COMMIT;
  SET @@autocommit=1;
END$$
DELIMITER ;
```

生成 1W 条记录

```
mysql> call sp_generate_data_f1(10000);
Query OK, 0 rows affected (5.42 sec)
```

我们把 f2 列上的索引变为不可见，结果瞬间完成。

```
mysql> alter table f1 alter index idx_f2 invisible;
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

在看下表结构，此时索引标记为 Invisible。

```
mysql> show create table f1 \G
***** 1. row *****
      Table: f1
Create Table: CREATE TABLE `f1` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `f1` int(11) DEFAULT NULL,
  `f2` int(11) DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `id` (`id`),
  KEY `idx_f1` (`f1`),
  KEY `idx_f2` (`f2`) /*!80000 INVISIBLE */
) ENGINE=InnoDB AUTO_INCREMENT=10001 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

给一条有 f2 列过滤的 SQL，发现优化器用不到这个索引了。

```
mysql> explain select count(*) from f1 where f2 = 52\G
***** 1. row *****
      id: 1
      select_type: SIMPLE
      table: f1
      partitions: NULL
      type: ALL
possible_keys: NULL
      key: NULL
      key_len: NULL
      ref: NULL
      rows: 9991
      filtered: 1.00
      Extra: Using where
1 row in set, 1 warning (0.00 sec)
```

用 force index 强制使用，直接报错。

```
mysql> explain select count(*) from f1 force index (idx_f2) where f2 = 52\G
ERROR 1176 (42000): Key 'idx_f2' doesn't exist in table 'f1'
```

那 MySQL 8.0 的优化器开关里可以告诉它，有的时候可以用隐藏索引。来打开看看。

```
mysql> set @@optimizer_switch = 'use_invisible_indexes=on';
Query OK, 0 rows affected (0.00 sec)
```

那这条 SQL 现在可以用 idx_f2 了。

```
mysql> explain select count(*) from f1 where f2 = 52\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: f1
   partitions: NULL
         type: ref
possible_keys: idx_f2
          key: idx_f2
        key_len: 5
          ref: const
         rows: 121
   filtered: 100.00
      Extra: Using index
1 row in set, 1 warning (0.00 sec)
```

4 总结

INVISIBLE INDEX 的确是一个很有用的小特性，给索引增加了第三个额外的开关选项。想要了解更多的建议阅读 MySQL 手册。



MySQL 8.0 新增 HINT 模式

作者：杨涛涛

1 概念一，数据的可选择性基数，也就是常说的 cardinality 值。

查询优化器在生成各种执行计划之前，得先从统计信息中取得相关数据，这样才能估算每步操作所涉及到的记录数，而这个相关数据就是 cardinality。简单来说，就是每个值在每个字段中的唯一值分布状态。

比如表 t1 有 100 行记录，其中一列为 f1。f1 中唯一值的个数可以是 100 个，也可以是 1 个，当然也可以是 1 到 100 之间的任何一个数字。这里唯一值越多，就是这个列的可选择基数。

那看到这里我们就明白了，为什么要在基数高的字段上建立索引，而基数低的的字段建立索引反而没有全表扫描来的快。当然这个只是一方面，至于更深入的探讨就不在我这篇探讨的范围了。

2 概念二，关于 HINT 的使用。

这里我来说下 HINT 是什么，在什么时候用。HINT 简单来说就是在某些特定的场景下人工协助 MySQL 优化器的工作，使她生成最优的执行计划。一般来说，优化器的执行计划都是最优化的，不过在某些特定场景下，执行计划可能不是最优化。

比如：表 t1 经过大量的频繁更新操作，(UPDATE, DELETE, INSERT)，cardinality 已经很不准确了，这时候刚好执行了一条 SQL，那么有可能这条 SQL 的执行计划就不是最优的。为什么说有可能呢？

3 来看下具体演示

譬如，以下两条 SQL，

1. A:

```
select * from t1 where f1 = 20;
```

2. B:

```
select * from t1 where f1 = 30;
```

如果 f1 的值刚好频繁更新的值为 30，并且没有达到 MySQL 自动更新 cardinality 值的临界值或者说用户设置了手动更新又或者用户减少了 sample page 等等，那么对这两条语句来说，可能不准确的就是 B 了。

这里顺带说下，MySQL 提供了自动更新和手动更新表 cardinality 值的方法，因篇幅有限，需要的可以查阅手册。

那回到正题上，MySQL 8.0 带来了几个 HINT，我今天就举个 index_merge 的例子。

示例表结构:

```
mysql> desc t1;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | int(11)       | NO   | PRI | NULL    | auto_increment |
| rank1      | int(11)       | YES  | MUL | NULL    |                |
| rank2      | int(11)       | YES  | MUL | NULL    |                |
| log_time   | datetime      | YES  | MUL | NULL    |                |
| prefix_uid | varchar(100)  | YES  |     | NULL    |                |
| desc1      | text          | YES  |     | NULL    |                |
| rank3      | int(11)       | YES  | MUL | NULL    |                |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.00 sec)

mysql> select count(*) from t1;
+-----+
| count(*) |
+-----+
|    32768 |
+-----+
1 row in set (0.01 sec)
```

这里我们两条经典的 SQL:

1. SQL C:

```
select * from t1 where rank1 = 1 or rank2 = 2 or rank3 = 2;
```

2. SQL D:

```
select * from t1 where rank1 =100 and rank2 =100 and rank3 =100;
```

表 t1 实际上在 rank1,rank2,rank3 三列上分别有一个二级索引。

那我们来看 SQL C 的查询计划。

显然，没有用到任何索引，扫描的行数为 32034，cost 为 3243.65。

```
mysql> explain format=json select * from t1 where rank1 =1 or rank2 = 2 or rank3
= 2\G
```

```
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "3243.65"
```

```

    },
    "table": {
      "table_name": "t1",
      "access_type": "ALL",
      "possible_keys": [
        "idx_rank1",
        "idx_rank2",
        "idx_rank3"
      ],
      "rows_examined_per_scan": 32034,
      "rows_produced_per_join": 115,
      "filtered": "0.36",
      "cost_info": {
        "read_cost": "3232.07",
        "eval_cost": "11.58",
        "prefix_cost": "3243.65",
        "data_read_per_join": "49K"
      },
      "used_columns": [
        "id",
        "rank1",
        "rank2",
        "log_time",
        "prefix_uid",
        "desc1",
        "rank3"
      ],
      "attached_condition": "((`ytt`.`t1`.`rank1` = 1) or (`ytt`.`t1`.`rank2` = 2)
or (`ytt`.`t1`.`rank3` = 2))"
    }
  }
}
1 row in set, 1 warning (0.00 sec)

```

我们加上 hint 给相同的查询，再次看看查询计划。这个时候用到了 index_merge, union 了三个列。扫描的行数为 1103, cost 为 441.09, 明显比之前的快了好几倍。

```

mysql> explain format=json select /*+ index_merge(t1) */ * from t1 where rank1 =
1 or rank2 = 2 or rank3 = 2\G
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "441.09"
    },

```



```

"table": {
  "table_name": "t1",
  "access_type": "index_merge",
  "possible_keys": [
    "idx_rank1",
    "idx_rank2",
    "idx_rank3"
  ],
  "key": "union(idx_rank1,idx_rank2,idx_rank3)",
  "key_length": "5,5,5",
  "rows_examined_per_scan": 1103,
  "rows_produced_per_join": 1103,
  "filtered": "100.00",
  "cost_info": {
    "read_cost": "330.79",
    "eval_cost": "110.30",
    "prefix_cost": "441.09",
    "data_read_per_join": "473K"
  },
  "used_columns": [
    "id",
    "rank1",
    "rank2",
    "log_time",
    "prefix_uid",
    "desc1",
    "rank3"
  ],
  "attached_condition": "((`ytt`.`t1`.`rank1` = 1) or (`ytt`.`t1`.`rank2` = 2)
or (`ytt`.`t1`.`rank3` = 2))"
}
}
}
1 row in set, 1 warning (0.00 sec)

```

我们再看下 SQL D 的计划:

1. 不加 HINT,

```

mysql> explain format=json select * from t1 where rank1 =100 and rank2 =100 and ra
nk3 =100\G
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "534.34"
    }
  }
}

```

```
    },
    "table": {
      "table_name": "t1",
      "access_type": "ref",
      "possible_keys": [
        "idx_rank1",
        "idx_rank2",
        "idx_rank3"
      ],
      "key": "idx_rank1",
      "used_key_parts": [
        "rank1"
      ],
      "key_length": "5",
      "ref": [
        "const"
      ],
      "rows_examined_per_scan": 555,
      "rows_produced_per_join": 0,
      "filtered": "0.07",
      "cost_info": {
        "read_cost": "478.84",
        "eval_cost": "0.04",
        "prefix_cost": "534.34",
        "data_read_per_join": "176"
      },
      "used_columns": [
        "id",
        "rank1",
        "rank2",
        "log_time",
        "prefix_uid",
        "desc1",
        "rank3"
      ],
      "attached_condition": "((`ytt`.`t1`.`rank3` = 100) and (`ytt`.`t1`.`rank2` = 100))"
    }
  }
}
1 row in set, 1 warning (0.00 sec)
```

2. 加了 HINT,

```
mysql> explain format=json select /*+ index_merge(t1)*/ * from t1 where rank1 =100
and rank2 =100 and rank3 =100\G
```

```

***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "5.23"
    },
    "table": {
      "table_name": "t1",
      "access_type": "index_merge",
      "possible_keys": [
        "idx_rank1",
        "idx_rank2",
        "idx_rank3"
      ],
      "key": "intersect(idx_rank1,idx_rank2,idx_rank3)",
      "key_length": "5,5,5",
      "rows_examined_per_scan": 1,
      "rows_produced_per_join": 1,
      "filtered": "100.00",
      "cost_info": {
        "read_cost": "5.13",
        "eval_cost": "0.10",
        "prefix_cost": "5.23",
        "data_read_per_join": "440"
      },
      "used_columns": [
        "id",
        "rank1",
        "rank2",
        "log_time",
        "prefix_uid",
        "desc1",
        "rank3"
      ],
      "attached_condition": "((`ytt`.`t1`.`rank3` = 100) and (`ytt`.`t1`.`rank2` = 100) and (`ytt`.`t1`.`rank1` = 100))"
    }
  }
}
1 row in set, 1 warning (0.00 sec)

```

对比下以上两个，加了 HINT 的比不加 HINT 的 cost 小了 100 倍。

总结下，就是说表的 cardinality 值影响这张的查询计划，如果这个值没有正常更新的话，就需要手工加 HINT 了。相信 MySQL 未来的版本会带来更多的 HINT



MySQL 8.0 直方图

作者：杨涛涛

MySQL 8.0 推出了 histogram，也叫柱状图或者直方图。先来解释下什么叫直方图。

1 关于直方图

我们知道，在 DB 中，优化器负责将 SQL 转换为很多个不同的执行计划，完了从中选择一个最优的来实际执行。但是有时候优化器选择的最终计划有可能随着 DB 环境的变化不是最优的，这就导致了查询性能不是很好。比如，优化器无法准确的知道每张表的实际行数以及参与过滤条件的列有多少个不同的值。那其实有时候有人就说了，索引不是可以解决这个问题吗？是的，不同类型的索引可以解决这个问题，但是你不能每个列都建索引吧？如果一张表有 1000 个字段，那全字段索引将会拖死对这张表的写入。而此时，直方图就是相对来说，开销较小的方法。

直方图就是在 MySQL 中为某张表的某些字段提供了一种数值分布的统计信息。比如字段 NULL 的个数，每个不同值出现的百分比、最大值、最小值等等。如果我们用过了 MySQL 的分析型引擎 brighthouse，那对这个概念太熟悉了。

MySQL 的直方图有两种，等宽直方图和等高直方图。等宽直方图每个桶（bucket）保存一个值以及这个值累积频率；等高直方图每个桶需要保存不同值的个数，上下限以及累计频率等。MySQL 会自动分配用哪种类型的直方图，我们无需参与。

MySQL 定义了一张 meta 表 `column_statistics` 来存储直方图的定义，每行记录对应一个字段的直方图，以 json 保存。同时，新增了一个参数 `histogram_generation_max_mem_size` 来配置建立直方图内存大小。

不过直方图有以下限制：

1. 不支持几何类型以及 json。
2. 不支持加密表和临时表。
3. 不支持列值完全唯一。
4. 需要手工的进行键值分布。

那我们来举个简单的例子说明直方图对查询的效果提升。

2 举例

表相关定义以及行数信息等：

```
mysql> show create table t2\G
***** 1. row *****
      Table: t2
Create Table: CREATE TABLE `t2` (
```

```

`id` int(11) NOT NULL AUTO_INCREMENT,
`rank1` int(11) DEFAULT NULL,
`rank2` int(11) DEFAULT NULL,
`rank3` int(11) DEFAULT NULL,
`log_date` date DEFAULT NULL,
PRIMARY KEY (`id`),
KEY `idx_rank1` (`rank1`),
KEY `idx_log_date` (`log_date`)
) ENGINE=InnoDB AUTO_INCREMENT=49140 DEFAULT CHARSET=utf8mb4 \
COLLATE=utf8mb4_0900_ai_ci STATS_PERSISTENT=1 STATS_AUTO_RECALC=0
1 row in set (0.00 sec)

```

```

mysql> select count(*) from t2;
+-----+
| count(*) |
+-----+
|    30940 |
+-----+
1 row in set (0.00 sec)

```

同时对 t2 克隆了一张表 t3

```

mysql> create table t3 like t2;
Query OK, 0 rows affected (0.13 sec)

mysql> insert into t3 select * from t2;
Query OK, 30940 rows affected (1.94 sec)
Records: 30940 Duplicates: 0 Warnings: 0

```

给表 t3 列 rank1 和 log_date 添加 histogram

```

mysql> analyze table t3 update histogram on rank1,log_date;
+-----+-----+-----+-----+
| Table | Op      | Msg_type | Msg_text                                     |
+-----+-----+-----+-----+
| ytt.t3 | histogram | status   | Histogram statistics created for column 'log_date'. |
| ytt.t3 | histogram | status   | Histogram statistics created for column 'rank1'. |
|      |      |      |      |
+-----+-----+-----+-----+
2 rows in set (0.19 sec)

```

我们来看看 histogram 的分布状况

```

mysql> select json_pretty(histogram) result from information_schema.column_statistics
where table_name = 't3' and column_name = 'log_date'\G

```

```
***** 1. row *****
result: {
  "buckets": [
    [
      "2018-04-17",
      "2018-04-20",
      0.01050420168067227,
      4
    ],
    ...
  ],
  "data-type": "date",
  "null-values": 0.0,
  "collation-id": 8,
  "last-updated": "2019-04-17 03:43:01.910185",
  "sampling-rate": 1.0,
  "histogram-type": "equi-height",
  "number-of-buckets-specified": 100
}
1 row in set (0.03 sec)
```

MySQL 自动为这个字段分配了等高直方图，默认为 100 个桶。

1. SQL A:

```
select count(*) from t2/t3 where (rank1 between 1 and 10) and log_date < '2018-09-01';
```

2. SQL A 的执行结果:

```
mysql> select count(*) from t2/t3 where (rank1 between 1 and 10) and log_date < '2018-09-01';
+-----+
| count(*) |
+-----+
|      2269 |
+-----+
1 row in set (0.01 sec)
```

无 histogram 的执行计划

```
mysql> explain format=json select count(*) from t2 where (rank1 between 1 and 10)
and log_date < '2018-09-01'\G
```

```
***** 1. row *****
```

```
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "2796.11"
    },
    "table": {
      "table_name": "t2",
      "access_type": "range",
      "possible_keys": [
        "idx_rank1",
        "idx_log_date"
      ],
      "key": "idx_rank1",
      "used_key_parts": [
        "rank1"
      ],
      "key_length": "5",
      "rows_examined_per_scan": 6213,
      "rows_produced_per_join": 3106,
      "filtered": "50.00",
      "index_condition": "(`ytt`.`t2`.`rank1` between 1 and 10)",
      "cost_info": {
        "read_cost": "2485.46",
        "eval_cost": "310.65",
        "prefix_cost": "2796.11",
        "data_read_per_join": "72K"
      },
      "used_columns": [
        "rank1",
        "log_date"
      ],
      "attached_condition": "(`ytt`.`t2`.`log_date` < '2018-09-01')"
```

有 histogram 的执行计划

```
mysql> explain format=json select count(*) from t3 where (rank1 between 1 and 10)
and log_date < '2018-09-01'\G
```

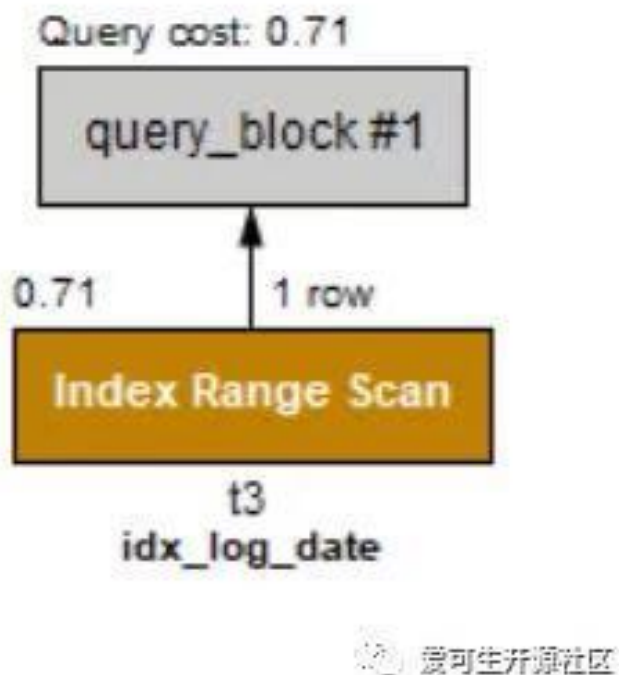
```
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "0.71"
    },
    "table": {
      "table_name": "t3",
      "access_type": "range",
      "possible_keys": [
        "idx_rank1",
        "idx_log_date"
      ],
      "key": "idx_log_date",
      "used_key_parts": [
        "log_date"
      ],
      "key_length": "4",
      "rows_examined_per_scan": 1,
      "rows_produced_per_join": 1,
      "filtered": "100.00",
      "index_condition": "(`ytt`.`t3`.`log_date` < '2018-09-01')",
      "cost_info": {
        "read_cost": "0.61",
        "eval_cost": "0.10",
        "prefix_cost": "0.71",
        "data_read_per_join": "24"
      },
      "used_columns": [
        "rank1",
        "log_date"
      ],
      "attached_condition": "(`ytt`.`t3`.`rank1` between 1 and 10)"
    }
  }
}
1 row in set, 1 warning (0.00 sec)
```

我们看到两个执行计划的对比，有 Histogram 的执行计划 cost 比普通的 sql 快了好多倍。上面文字可以看起来比较晦涩，贴上两张图，看起来就很简单了。

无 histogram



有 histogram



当然，我这里举得例子相对简单，有兴趣的朋友可以更深入学习其他复杂些的例子。



MySQL 8.0 json 到表的转换

作者：杨涛涛

我们知道，JSON 是一种轻量级的数据交互的格式，大部分 NO SQL 数据库的存储都用 JSON。MySQL 从 5.7 开始支持 JSON 格式的数据存储，并且新增了很多 JSON 相关函数。MySQL 8.0 又带来了一个新的把 JSON 转换为 TABLE 的函数 JSON_TABLE，实现了 JSON 到表的转换。

1 举例一

我们看下简单的例子：

简单定义一个两级 JSON 对象

```
mysql> set @ytt='{ "name": [{ "a": "ytt", "b": "action"}, { "a": "dble", "b": "shard"}, { "a": "mysql", "b": "oracle"} ] }';
Query OK, 0 rows affected (0.00 sec)
```

第一级：

```
mysql> select json_keys(@ytt);
+-----+
| json_keys(@ytt) |
+-----+
| ["name"]        |
+-----+
1 row in set (0.00 sec)
```

第二级：

```
mysql> select json_keys(@ytt, '$.name[0]');
+-----+
| json_keys(@ytt, '$.name[0]') |
+-----+
| ["a", "b"]                   |
+-----+
1 row in set (0.00 sec)
```

我们使用 MySQL 8.0 的 JSON_TABLE 来转换 @ytt。

```
mysql> select * from json_table(@ytt, '$.name[*]' columns (f1 varchar(10) path '$.a', f2 varchar(10) path '$.b')) as tt;
```

```

+-----+-----+
| f1    | f2    |
+-----+-----+
| ytt   | action |
| dble  | shard  |
| mysql | oracle |
+-----+-----+
3 rows in set (0.00 sec)

```

2 举例二

再来一个复杂点的例子，用的是 EXPLAIN 的 JSON 结果集。

JSON 串 @json_str1。

```

set @json_str1 = ' {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "1.00"
    },
    "table": {
      "table_name": "bigtable",
      "access_type": "const",
      "possible_keys": [
        "id"
      ],
      "key": "id",
      "used_key_parts": [
        "id"
      ],
      "key_length": "8",
      "ref": [
        "const"
      ],
      "rows_examined_per_scan": 1,
      "rows_produced_per_join": 1,
      "filtered": "100.00",
      "cost_info": {
        "read_cost": "0.00",
        "eval_cost": "0.20",
        "prefix_cost": "0.00",
        "data_read_per_join": "176"
      },
      "used_columns": [
        "id",
        "log_time",

```

```
        "str1",
        "str2"
    ]
}
}
}';
```

第一级:

```
mysql> select json_keys(@json_str1) as 'first_object';
+-----+
| first_object |
+-----+
| ["query_block"] |
+-----+
1 row in set (0.00 sec)
```

第二级:

```
mysql> select json_keys(@json_str1,'$.query_block') as 'second_object';
+-----+
| second_object |
+-----+
| ["table", "cost_info", "select_id"] |
+-----+
1 row in set (0.00 sec)
```

第三级:

```
mysql> select json_keys(@json_str1,'$.query_block.table') as 'third_object'\G
***** 1. row *****
third_object:
[
  "key",
  "ref",
  "filtered",
  "cost_info",
  "key_length",
  "table_name",
  "access_type",
  "used_columns",
  "possible_keys",
  "used_key_parts",
  "rows_examined_per_scan",
  "rows_produced_per_join"
]
111
```

1 row in set (0.01 sec)

第四级:

```
mysql> select json_extract(@json_str1,'$.query_block.table.cost_info') as 'forth
_object'\G
***** 1. row *****
forth_object: {
  "eval_cost":"0.20",
  "read_cost":"0.00",
  "prefix_cost":"0.00",
  "data_read_per_join":"176"
}
1 row in set (0.00 sec)
```

那我们把这个 JSON 串转换为表。

```
SELECT * FROM JSON_TABLE(@json_str1,
  '$.query_block'
  COLUMNS(
    rowid FOR ORDINALITY,
    NESTED PATH '$.table'
    COLUMNS (
      a1_1 varchar(100) PATH '$.key',
      a1_2 varchar(100) PATH '$.ref[0]',
      a1_3 varchar(100) PATH '$.filtered',
      nested path '$.cost_info'
      columns (
        a2_1 varchar(100) PATH '$.eval_cost' ,
        a2_2 varchar(100) PATH '$.read_cost',
        a2_3 varchar(100) PATH '$.prefix_cost',
        a2_4 varchar(100) PATH '$.data_read_per_join'

      ),
      a3 varchar(100) PATH '$.key_length',
      a4 varchar(100) PATH '$.table_name',
      a5 varchar(100) PATH '$.access_type',
      a6 varchar(100) PATH '$.used_key_parts[0]',
      a7 varchar(100) PATH '$.rows_examined_per_scan',
      a8 varchar(100) PATH '$.rows_produced_per_join',
      a9 varchar(100) PATH '$.key'

    ),
    NESTED PATH '$.cost_info'
    columns (
      b1_1 varchar(100) path '$.query_cost'
```

```
),
  c INT path "$.select_id"
)
) AS tt;
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
| rowid | a1_1 | a1_2 | a1_3 | a2_1 | a2_2 | a2_3 | a2_4 | a3   | a4   | a5
| a6   | a7   | a8   | a9   | b1_1 | c     |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
|      1 | id   | const | 100.00 | 0.20 | 0.00 | 0.00 | 176 | 8   | bigtable | const
| id    | 1    | 1     | id     | NULL |      1 |
|      1 | NULL | NULL  | NULL   | NULL | NULL | NULL | NULL | NULL | NULL    | NULL
| NULL  | NULL | NULL  | NULL   | 1.00 |      1 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

当然，JSON_table 函数还有其他的用法，我这里不一一列举了，详细的参考手册。



MySQL 8.0 窗口函数详解

作者：杨涛涛

1 背景

一直以来，MySQL 只有针对聚合函数的汇总类功能，比如 MAX, AVG 等，没有从 SQL 层针对聚合类每组展开处理的功能。不过 MySQL 开放了 UDF 接口，可以用 C 来自己写 UDF，这个就增加了功能行难度。这种针对每组展开处理的功能就叫窗口函数，有的数据库叫分析函数。

在 MySQL 8.0 之前，我们想要得到这样的结果，就得用以下几种方法来实现：

1. session 变量
2. group_concat 函数组合
3. 自己写 store routines

接下来我们用经典的 学生/课程/成绩 来做窗口函数演示

2 准备

学生表

```
mysql> show create table student \G
***** 1. row *****
      Table: student
Create Table: CREATE TABLE student (
  sid int(10) unsigned NOT NULL,
  sname varchar(64) DEFAULT NULL,
  PRIMARY KEY (sid)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

课程表

```
mysql> show create table course\G
***** 1. row *****
      Table: course
Create Table: CREATE TABLE `course` (
  `cid` int(10) unsigned NOT NULL,
  `cname` varchar(64) DEFAULT NULL,
  PRIMARY KEY (`cid`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

成绩表

```
mysql> show create table score\G
***** 1. row *****
      Table: score
Create Table: CREATE TABLE `score` (
  `sid` int(10) unsigned NOT NULL,
  `cid` int(10) unsigned NOT NULL,
  `score` tinyint(3) unsigned DEFAULT NULL,
  PRIMARY KEY (`sid`,`cid`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

测试数据

```
mysql> select * from student;
+-----+-----+
| sid      | sname      |
+-----+-----+
| 201910001 | 张三        |
| 201910002 | 李四        |
| 201910003 | 武松        |
| 201910004 | 潘金莲      |
| 201910005 | 菠菜        |
| 201910006 | 杨发财      |
| 201910007 | 欧阳修      |
| 201910008 | 郭靖        |
| 201910009 | 黄蓉        |
| 201910010 | 东方不败    |
+-----+-----+
10 rows in set (0.00 sec)
```

```
mysql> select * from score;;
+-----+-----+-----+
| sid      | cid      | score |
+-----+-----+-----+
| 201910001 | 20192001 | 50    |
| 201910001 | 20192002 | 88    |
| 201910001 | 20192003 | 54    |
| 201910001 | 20192004 | 43    |
| 201910001 | 20192005 | 89    |
| 201910002 | 20192001 | 79    |
| 201910002 | 20192002 | 97    |
| 201910002 | 20192003 | 82    |
| 201910002 | 20192004 | 85    |
| 201910002 | 20192005 | 80    |
```


201910003 20192001 48
201910003 20192002 98
201910003 20192003 47
201910003 20192004 41
201910003 20192005 34
201910004 20192001 81
201910004 20192002 69
201910004 20192003 67
201910004 20192004 99
201910004 20192005 61
201910005 20192001 40
201910005 20192002 52
201910005 20192003 39
201910005 20192004 74
201910005 20192005 86
201910006 20192001 42
201910006 20192002 52
201910006 20192003 36
201910006 20192004 58
201910006 20192005 84
201910007 20192001 79
201910007 20192002 43
201910007 20192003 79
201910007 20192004 98
201910007 20192005 88
201910008 20192001 45
201910008 20192002 65
201910008 20192003 90
201910008 20192004 89
201910008 20192005 74
201910009 20192001 73
201910009 20192002 42
201910009 20192003 95
201910009 20192004 46
201910009 20192005 45
201910010 20192001 58
201910010 20192002 52
201910010 20192003 55
201910010 20192004 87
201910010 20192005 36

+-----+-----+-----+

50 rows in set (0.00 sec)

```
mysql> select * from course;
+-----+-----+
| cid      | cname      |
+-----+-----+
| 20192001 | mysql      |
| 20192002 | oracle     |
| 20192003 | postgresql |
| 20192004 | mongodb    |
| 20192005 | dble       |
+-----+-----+
5 rows in set (0.00 sec)
```

3 MySQL 8.0 之前

比如求成绩排名前三的学生排名，我来举个用 `session` 变量和 `group_concat` 函数来分别实现的例子：

session 变量方式

每组开始赋一个初始值序号和初始分组字段。

```
SELECT
    b.cname,
    a.sname,
    c.score, c.ranking_score
FROM
    student a,
    course b,
    (
        SELECT
            c.*,
            IF(
                @cid = c.cid,
                @rn := @rn + 1,
                @rn := 1
            ) AS ranking_score,
            @cid := c.cid AS tmpcid
        FROM
            (
                SELECT
                    *
                FROM
                    score
                ORDER BY cid,
                    score DESC
            ) c,
        (
            SELECT
```

```

        @rn := 0 rn,
        @cid := ''
    ) initialize_table
) c
WHERE a.sid = c.sid
AND b.cid = c.cid
AND c.ranking_score <= 3
ORDER BY b.cname,c.ranking_score;

```

```

+-----+-----+-----+-----+
| cname   | sname   | score | ranking_score |
+-----+-----+-----+-----+
| dble    | 张三    | 89    | 1             |
| dble    | 欧阳修  | 88    | 2             |
| dble    | 菠菜    | 86    | 3             |
| mongodb | 潘金莲  | 99    | 1             |
| mongodb | 欧阳修  | 98    | 2             |
| mongodb | 郭靖    | 89    | 3             |
| mysql   | 李四    | 100   | 1             |
| mysql   | 潘金莲  | 81    | 2             |
| mysql   | 欧阳修  | 79    | 3             |
| oracle  | 武松    | 98    | 1             |
| oracle  | 李四    | 97    | 2             |
| oracle  | 张三    | 88    | 3             |
| postgresql | 黄蓉    | 95    | 1             |
| postgresql | 郭靖    | 90    | 2             |
| postgresql | 李四    | 82    | 3             |
+-----+-----+-----+-----+
15 rows in set, 5 warnings (0.01 sec)

```

group_concat 函数方式

利用 findinset 内置函数来返回下标作为序号使用。

```

SELECT
    *
FROM
    (
        SELECT
            b.cname,
            a.sname,
            c.score,
            FIND_IN_SET(c.score, d.gp) score_ranking
        FROM
            student a,
            course b,
            score c,

```

```

(
SELECT
    cid,
    GROUP_CONCAT (
        score
        ORDER BY score DESC SEPARATOR ','
    ) gp
FROM
    score
    GROUP BY cid
    ORDER BY score DESC
) d
WHERE a.sid = c.sid
AND b.cid = c.cid
AND c.cid = d.cid
ORDER BY d.cid,
score_ranking
) ytt
WHERE score_ranking <= 3;

```

```

+-----+-----+-----+-----+
| cname   | sname   | score | score_ranking |
+-----+-----+-----+-----+
| dble    | 张三    | 89    | 1             |
| dble    | 欧阳修  | 88    | 2             |
| dble    | 菠菜    | 86    | 3             |
| mongodb | 潘金莲  | 99    | 1             |
| mongodb | 欧阳修  | 98    | 2             |
| mongodb | 郭靖    | 89    | 3             |
| mysql   | 李四    | 100   | 1             |
| mysql   | 潘金莲  | 81    | 2             |
| mysql   | 欧阳修  | 79    | 3             |
| oracle  | 武松    | 98    | 1             |
| oracle  | 李四    | 97    | 2             |
| oracle  | 张三    | 88    | 3             |
| postgresql | 黄蓉    | 95    | 1             |
| postgresql | 郭靖    | 90    | 2             |
| postgresql | 李四    | 82    | 3             |
+-----+-----+-----+-----+
15 rows in set (0.00 sec)

```

4 MySQL 8.0 窗口函数

MySQL 8.0 后提供了原生的窗口函数支持，语法和大多数数据库一样，比如还是之前的例子：用 `row_number() over ()` 直接来检索排名。

```
mysql>
SELECT
    *
FROM
    (
        SELECT
            b.cname,
            a.sname,
            c.score,
            row_number() over (
                PARTITION BY b.cname
                ORDER BY c.score DESC
            ) score_rank
        FROM
            student AS a,
            course AS b,
            score AS c
        WHERE a.sid = c.sid
        AND b.cid = c.cid
    ) ytt
WHERE score_rank <= 3;
```

```
+-----+-----+-----+-----+
| cname   | sname   | score | score_rank |
+-----+-----+-----+-----+
| dble    | 张三    | 89    | 1          |
| dble    | 欧阳修  | 88    | 2          |
| dble    | 菠菜    | 86    | 3          |
| mongodb | 潘金莲  | 99    | 1          |
| mongodb | 欧阳修  | 98    | 2          |
| mongodb | 郭靖    | 89    | 3          |
| mysql   | 李四    | 100   | 1          |
| mysql   | 潘金莲  | 81    | 2          |
| mysql   | 欧阳修  | 79    | 3          |
| oracle  | 武松    | 98    | 1          |
| oracle  | 李四    | 97    | 2          |
| oracle  | 张三    | 88    | 3          |
| postgresql | 黄蓉    | 95    | 1          |
| postgresql | 郭靖    | 90    | 2          |
| postgresql | 李四    | 82    | 3          |
+-----+-----+-----+-----+
15 rows in set (0.00 sec)
```

那我们再找出课程 MySQL 和 DBLE 里不及格的倒数前两名学生名单。

```
mysql>
SELECT
    *
FROM
    (
        SELECT
            b.cname,
            a.sname,
            c.score,
            row_number () over (
                PARTITION BY b.cid
                ORDER BY c.score ASC
            ) score_ranking
        FROM
            student AS a,
            course AS b,
            score AS c
        WHERE a.sid = c.sid
        AND b.cid = c.cid
        AND b.cid IN (20192005, 20192001)
        AND c.score < 60
    ) ytt
WHERE score_ranking < 3;
```

```
+-----+-----+-----+-----+
| cname | sname      | score | score_ranking |
+-----+-----+-----+-----+
| mysql | 菠菜       | 40    | 1             |
| mysql | 杨发财     | 42    | 2             |
| dble  | 武松       | 34    | 1             |
| dble  | 东方不败   | 36    | 2             |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

我们只演示了 row_number() over() 函数的使用方法，其他的函数大家可以自己体验体验，方法都差不多。

微信扫一扫阅读原文



MySQL 8.0 动态权限

作者：杨涛涛

1 背景

在了解动态权限之前，我们先回顾下 MySQL 的权限列表。权限列表大体分为服务级别和表级别，列级别以及大而广的角色（也是 MySQL 8.0 新增）存储程序等权限。我们看到有一个特殊的 SUPER 权限，可以做好多个操作。比如 SET 变量，在从机重新指定相关主机信息以及清理二进制日志等。那这里可以看到，SUPER 有点太过强大，导致了仅仅想实现子权限变得十分困难，比如用户只能 SET 变量，其他的都不想要。那么 MySQL 8.0 之前没法实现，权限的细分不够明确，容易让非法用户钻空子。

那么 MySQL 8.0 把权限细分为静态权限和动态权限，下面我画了两张详细的区分图，图 1 为动态权限，图 2 为静态权限。

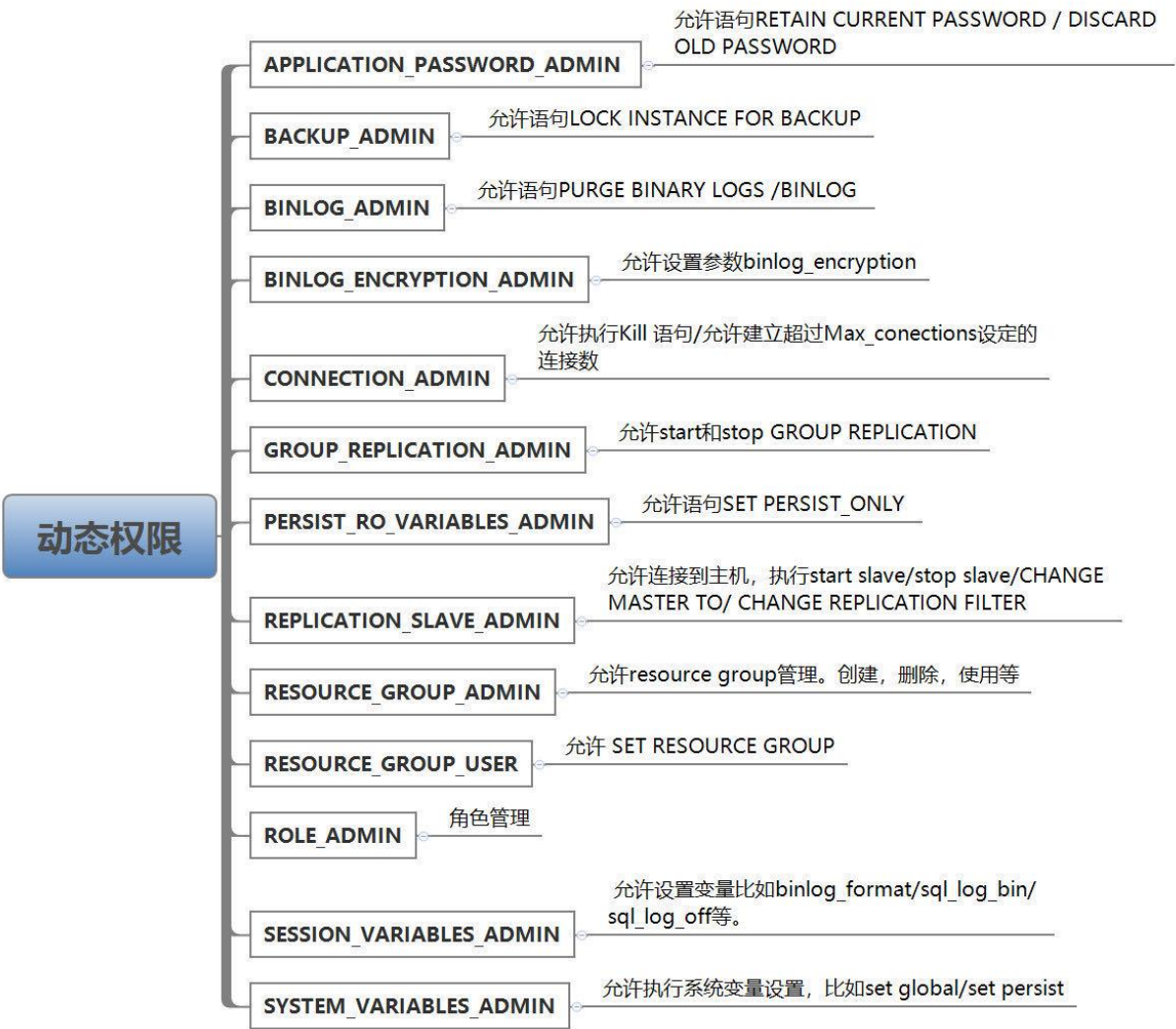


图 1 - 动态权限图

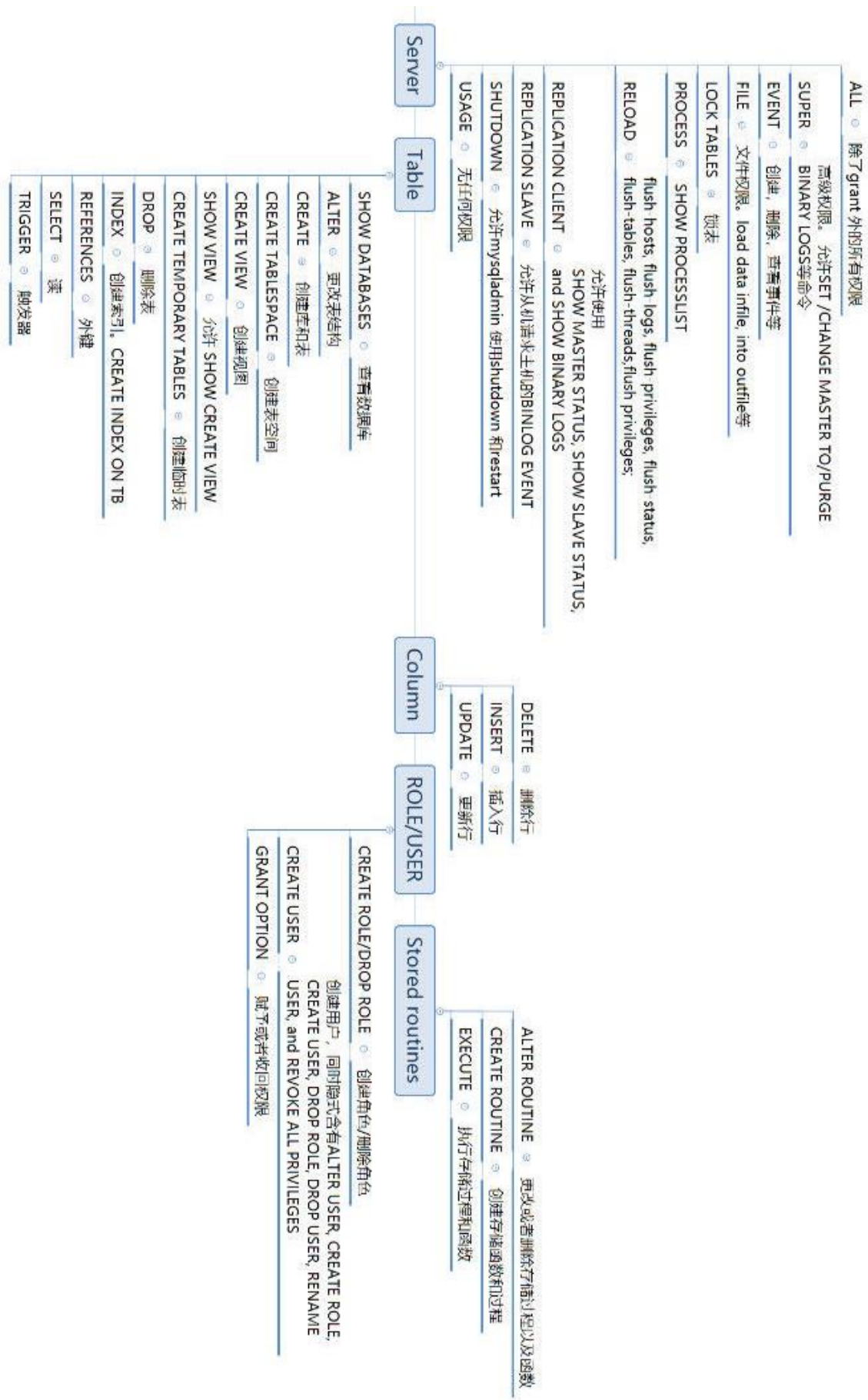


图 2 - MySQL 静态权限的权限管理图

那我们看到其实动态权限就是对 SUPER 权限的细分。SUPER 权限在未来将会被废弃掉。我们来看个简单的例子，比如，用户 'ytt2@localhost'，有 SUPER 权限。

```
mysql> show grants for ytt2@'localhost';
+-----+
| Grants for ytt2@localhost |
+-----+
| GRANT INSERT, UPDATE, DELETE, CREATE, ALTER, SUPER ON *.* TO ytt2@localhost |
+-----+
--+
1 row in set (0.00 sec)
```

但是现在我只想这个用户有 SUPER 的子集，设置变量的权限。那么单独给这个用户赋予两个能设置系统变量的动态权限，完了把 SUPER 给拿掉。

```
mysql> grant session_variables_admin,system_variables_admin on *.* to ytt2@'local
host';
Query OK, 0 rows affected (0.03 sec)
mysql> revoke super on *.* from ytt2@'localhost';
Query OK, 0 rows affected, 1 warning (0.02 sec)
```

我们看到这个 WARNINGS 提示 SUPER 已经废弃了。

```
mysql> show warnings;
+-----+-----+-----+
| Level | Code | Message |
+-----+-----+-----+
| Warning | 1287 | The SUPER privilege identifier is deprecated |
+-----+-----+-----+
1 row in set (0.00 sec)`
```

```
mysql> show grants for ytt2@'localhost';
+-----+
| Grants for ytt2@localhost |
+-----+
| GRANT INSERT, UPDATE, DELETE, CREATE, ALTER ON *.* TO ytt2@localhost |
| GRANT SESSION_VARIABLES_ADMIN,SYSTEM_VARIABLES_ADMIN ON *.* TO ytt2@localhost |
+-----+
2 rows in set (0.00 sec)
```

当然图 2 上还有其它的动态权限，这里就不做特别说明了。



MySQL 8.0 资源组

作者：杨涛涛

在 MySQL 8.0 之前，我们假设一下有一条烂 SQL，

```
select * from t1 order by rand() ;
```

以多个线程在跑，导致 CPU 被跑满了，其他的请求只能被阻塞进不来。那这种情况怎么办？

1 大概有以下几种解决办法：

1. 设置 `max_execution_time` 来阻止太长的读 SQL。那可能存在的问题是会把所有长 SQL 都给 KILL 掉。有些必须要执行很长时间的也会被误杀。
2. 自己写个脚本检测这类语句，比如 `order by rand()`，超过一定时间用 `Kill query thread_id` 给杀掉。

那能不能不要杀掉而让他正常运行，但是又不影响其他的请求呢？

那 mysql 8.0 引入的资源组（resource group，后面简写为 RG）可以基本上解决这类问题。

比如我可以用 RG 来在 SQL 层面给他限制在特定的一个 CPU 核上，这样我就不用管他，让他继续运行，如果有新的此类语句，让他排队好了。

为什么说基本呢？目前只能绑定 CPU 资源，其他的暂时不行。那我来演示下如何使用 RG。

2 创建一个资源组 `user_ytt`. 这里解释下各个参数的含义

1. `type = user` 表示这是一个用户态线程，也就是前台的请求线程。如果 `type=system`，表示后台线程，用来限制 mysql 自己的线程，比如 `Innodb purge thread`, `innodb read thread` 等等。
2. `vcpu` 代表 cpu 的逻辑核数，这里 0-1 代表前两个核被绑定到这个 RG。可以用 `lscpu`, `top` 等列出自己的 CPU 相关信息。
3. `thread_priority` 设置优先级。user 级优先级设置大于 0。

```
mysql
mysql> create resource group user_ytt type = user vcpu = 0-1 thread_priority=19 enable;
Query OK, 0 rows affected (0.03 sec)
```

RG 相关信息可以从 `information_schema.resource_groups` 系统表里检索。

```
mysql> select * from information_schema.resource_groups;
```

```
+-----+-----+-----+-----+
+-----+
| RESOURCE_GROUP_NAME | RESOURCE_GROUP_TYPE | RESOURCE_GROUP_ENABLED | VCPU_IDS |
| THREAD_PRIORITY |
+-----+-----+-----+-----+
+-----+
| USR_default          | USER                | 1 | 0-3          |
| 0 |
| SYS_default          | SYSTEM              | 1 | 0-3          |
| 0 |
| user_ytt             | USER                | 1 | 0-1          |
| 19 |
+-----+-----+-----+-----+
+-----+
3 rows in set (0.00 sec)
```

我们来给语句 `select guid from t1 group by left(guid,8) order by rand()` 赋予 RG `user_ytt`。

```
mysql> show processlist;
```

```
+-----+-----+-----+-----+-----+-----+-----+
| Id | User          | Host      | db  | Command | Time | State                |
| Info |
+-----+-----+-----+-----+-----+-----+
| 4 | event_scheduler | localhost | NULL | Daemon  | 10179 | Waiting on empty queue | NULL
| 240 | root          | localhost | ytt  | Query   | 101 | Creating sort index
| select guid from t1 group by left(guid,8) order by rand() |
| 245 | root          | localhost | ytt  | Query   | 0 | starting
show processlist
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

找到连接 240 对应的 `thread_id`。

```
mysql> select thread_id from performance_schema.threads where processlist_id = 240;
```

```
+-----+
| thread_id |
+-----+
| 278 |
+-----+
```

```
1 row in set (0.00 sec)
```

给这个线程 278 赋予 RG `user_ytt`。没报错就算成功了。

```
mysql> set resource group user_ytt for 278;  
Query OK, 0 rows affected (0.00 sec)
```

当然这个是在运维层面来做的，我们也可以在开发层面结合 MYSQL HINT 来单独给这个语句赋予 RG。

```
mysql> select /*+ resource_group(user_ytt) */guid from t1 group by left(guid,8) o  
rder by rand().  
...  
8388602 rows in set (4 min 46.09 sec)
```

3 RG 的限制:

1. Linux 平台上需要开启 CAPSYSNICE 特性。比如我机器上用 systemd 给 mysql 服务加上
systemctl edit mysql@80
[Service]
AmbientCapabilities=CAP_SYS_NICE
2. mysql 线程池开启后 RG 失效。
3. freebsd, solaris 平台 thread_priority 失效。
4. 目前只能绑定 CPU，不能绑定其他资源。



MySQL 8.0 正则替换

作者：杨涛涛

MySQL 一直以来都支持正则匹配，不过对于正则替换则一直到 MySQL 8.0 才支持。对于这类场景，以前要么在 MySQL 端处理，要么把数据拿出来在应用端处理。

比如我想把表 y1 的列 str1 的出现第 3 个 action 的子串替换成 dble，怎么实现？

1 自己写 SQL 层的存储函数

代码如下写死了 3 个，没有优化，仅仅作为演示，MySQL 里非常不建议写这样的函数。

```
mysql
DELIMITER $$
USE `ytt`$$
DROP FUNCTION IF EXISTS `func_instr_simple_ytt`$$
CREATE DEFINER=`root`@`localhost` FUNCTION `func_instr_simple_ytt`(
    f_str VARCHAR(1000), -- Parameter 1
    f_substr VARCHAR(100), -- Parameter 2
    f_replace_str varchar(100),
    f_times int -- times counter.only support 3.
) RETURNS varchar(1000)
BEGIN
    declare v_result varchar(1000) default 'ytt'; -- result.
    declare v_substr_len int default 0; -- search string length.

    set f_times = 3; -- only support 3.
    set v_substr_len = length(f_substr);
    select instr(f_str,f_substr) into @p1; -- First real position .
    select instr(substr(f_str,@p1+v_substr_len),f_substr) into @p2; Secondary virtual position.
    select instr(substr(f_str,@p2+ @p1 +2*v_substr_len - 1),f_substr) into @p3; -
- Third virtual position.
    if @p1 > 0 && @p2 > 0 && @p3 > 0 then -- Fine.
        select
            concat(substr(f_str,1,@p1 + @p2 + @p3 + (f_times - 1) * v_substr_len - f_times)
            ,f_replace_str,
            substr(f_str,@p1 + @p2 + @p3 + f_times * v_substr_len-2)) into v_result;
    else
        set v_result = f_str; -- Never changed.
    end if;
```

```
-- Purge all session variables.
set @p1 = null;
set @p2 = null;
set @p3 = null;
return v_result;

end;
$$
DELIMITER ;

-- 调用函数来更新:
mysql> update y1 set str1 = func_instr_simple_ytt(str1,'action','db1e',3);
Query OK, 20 rows affected (0.12 sec)
Rows matched: 20  Changed: 20  Warnings: 0
```

2 导出来用 sed 之类的工具替换掉在导入

步骤如下：（推荐使用）

1) 导出表 y1 的记录

```
mysql> select * from y1 into outfile '/var/lib/mysql-files/y1.csv';
Query OK, 20 rows affected (0.00 sec)
```

2) 用 sed 替换导出来的数据

```
root@ytt-Aspire-V5-471G:/var/lib/mysql-files# sed -i 's/action/db1e/3' y1.csv
```

3) 再次导入处理好的数据，完成

```
mysql> truncate y1;
Query OK, 0 rows affected (0.99 sec)

mysql> load data infile '/var/lib/mysql-files/y1.csv' into table y1;
Query OK, 20 rows affected (0.14 sec)
Records: 20  Deleted: 0  Skipped: 0  Warnings: 0
```

以上两种还是推荐导出来处理好了再重新导入，性能来的高些，而且还不用自己费劲写函数代码。那 MySQL 8.0 对于以上的场景实现就非常简单了，一个函数就搞定了。

```
mysql> update y1 set str1 = regexp_replace(str1,'action','db1e',1,3) ;
Query OK, 20 rows affected (0.13 sec)
Rows matched: 20  Changed: 20  Warnings: 0
```

还有一个 `regexp_instr` 也非常有用，特别是这种特指出现第几次的场景。比如定义 SESSION 变量@a。

```
mysql> set @a = 'aa bb cc ee fi lucy 1 1 1 b s 2 3 4 5 2 3 5 561 19 10 10 20 30 10 40';
Query OK, 0 rows affected (0.04 sec)
```

拿到至少两次的数字出现的第二次子串的位置。

```
mysql> select regexp_instr(@a,'[:digit:]{2,}','1,2');
+-----+
| regexp_instr(@a,'[:digit:]{2,}','1,2) |
+-----+
|                                50 |
+-----+
1 row in set (0.00 sec)
```

那我们在看看对多字节字符支持如何。

```
mysql> set @a = '中国 美国 俄罗斯 日本 中国 北京 上海 深圳 广州 北京 上海 武汉 东莞 北京 青岛北京';
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select regexp_instr(@a,'北京',1,1);
+-----+
| regexp_instr(@a,'北京',1,1) |
+-----+
|                17 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> select regexp_instr(@a,'北京',1,2);
+-----+
| regexp_instr(@a,'北京',1,2) |
+-----+
|                29 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> select regexp_instr(@a,'北京',1,3);
+-----+
| regexp_instr(@a,'北京',1,3) |
+-----+
|                41 |
+-----+
1 row in set (0.00 sec)
```

那总结下，这里我提到了 MySQL 8.0 的两个最有用的正则匹配函数 `regexp_replace` 和 `regexp_instr`。针对以前类似的场景算是有一个完美的解决方案。



MySQL 8.0 对 count(*) 的优化

作者：杨涛涛

MySQL 8.0 取消了 `sql_calc_found_rows` 的语法，以后求表 `count(*)` 的写法演进为直接 `select`。我们知道，MySQL 一直依赖对 `count(*)` 的执行很头疼。很早的时候，MyISAM 引擎自带计数器，可以秒回；不过 InnoDB 就需要实时计算，所以很头疼。以前有多方法可以变相解决此类问题，比如：

1 模拟 MyISAM 的计数器

比如表 `ytt1`，要获得总数，我们建立两个触发器分别对 `insert/delete` 来做记录到表 `ytt1_count`，这样只需要查询表 `ytt1_count` 就能拿到总数。`ytt1_count` 这张表足够小，可以长期固化到内存里。不过缺点就是有多余的触发器针对 `ytt1` 的每行操作，写性能降低。这里需要权衡。

2 用 MySQL 自带的 `sql_calc_found_rows` 特性来隐式计算

依然是表 `ytt1`，不过每次查询的时候用 `sql_calc_found_rows` 和 `found_rows()` 来获取总数，比如：

```
mysql> select sql_calc_found_rows * from ytt1 where 1 order by id desc limit 1;
```

```
+-----+-----+
```

```
| id   | r1   |
```

```
+-----+-----+
```

```
| 3072 | 73   |
```

```
+-----+-----+
```

```
1 row in set, 1 warning (0.00 sec)
```

```
mysql> show warnings;
```

```
+-----+-----+-----+-----+-----+-----+
```

```
| Level   | Code | Message
```

```
+-----+-----+-----+-----+-----+-----+
```

```
| Warning | 1287 | SQL_CALC_FOUND_ROWS is deprecated and will be removed in a future release. Consider using two separate queries instead. |
```

```
+-----+-----+-----+-----+-----+-----+
```

```
1 row in set (0.00 sec)
```

```
mysql> select found_rows() as 'count(*)';
```

```
+-----+
```

```
| count(*) |
```

```
+-----+
```

```
| 3072 |
```

```
+-----+
```

```
1 row in set, 1 warning (0.00 sec)
```


这样的好处是写法简单，用的是 MySQL 自己的语法。缺点也有，大概有两点：

1. `sql_calc_found_rows` 是全表扫。
2. `found_rows()` 函数是语句级别的存储，有很大的不确定性，所以在 MySQL 主从架构里，语句级别的行级格式下，从机数据可能会不准确。不过行记录格式改为 ROW 就 OK。所以最大的缺点还是第一点。

从 warnings 信息看，这种是 MySQL 8.0 之后要淘汰的语法。

3 从数据字典里面拿出来粗略的值

```
mysql> select table_rows from information_schema.tables where table_name='ytt1';
+-----+
| TABLE_ROWS |
+-----+
|          3072 |
+-----+
1 row in set (0.12 sec)
```

那这样的适合新闻展示，比如行数非常多，每页显示几行，一般后面的很多大家也都不怎么去看。缺点是数据不是精确值。

4 根据表结构特性特殊的取值

这里假设表 `ytt1` 的主键是连续的，并且没有间隙，那么可以直接

```
mysql> select max(id) as cnt from ytt1;
+-----+
| cnt |
+-----+
| 3072 |
+-----+
1 row in set (0.00 sec)
```

不过这种对表的数据要求比较高。

5 标准推荐取法 (MySQL 8.0.17 建议)

MySQL 8.0 建议用常规的写法来实现。

```
mysql> select * from ytt1 where 1 limit 1;
+----+-----+
| id | r1 |
+----+-----+
| 87 | 1 |
+----+-----+
1 row in set (0.00 sec)
```

```
mysql> select count(*) from ytt1;
+-----+
| count(*) |
+-----+
|      3072 |
+-----+
1 row in set (0.01 sec)
```

第五种写法是 MySQL 8.0.17 推荐的，也就是说以后大部分场景直接实时计算就 OK 了。

MySQL 8.0.17 以及在未来的版本都取消了 sql_calc_found_rows 特性，可以查看第二种方法里的 warnings 信息。相比 MySQL 5.7，8.0 对 count(*) 做了优化，没有必要在用第二种写法了。我们来看看 8.0 比 5.7 在此类查询是否真的有优化？

MySQL 5.7

```
mysql> select version();
+-----+
| version() |
+-----+
| 5.7.27-log |
+-----+
1 row in set (0.00 sec)
```

```
mysql> explain format=json select count(*) from ytt1\G
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "622.40"
    },
  },
  "table": {
    "table_name": "ytt1",
    "access_type": "index",
    "key": "PRIMARY",
    "used_key_parts": [
      "id"
    ],
    "key_length": "4",
    "rows_examined_per_scan": 3072,
    "rows_produced_per_join": 3072,
    "filtered": "100.00",
    "using_index": true,
    "cost_info": {
      "read_cost": "8.00",
```

```

        "eval_cost": "614.40",
        "prefix_cost": "622.40",
        "data_read_per_join": "48K"
    }
}
}
1 row in set, 1 warning (0.00 sec)

```

MySQL 8.0 下执行同样的查询

```

mysql> select version();
+-----+
| version() |
+-----+
| 8.0.17    |
+-----+
1 row in set (0.00 sec)

mysql> explain format=json select count(*) from ytt1\G
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "309.95"
    },
    "table": {
      "table_name": "ytt1",
      "access_type": "index",
      "key": "PRIMARY",
      "used_key_parts": [
        "id"
      ],
      "key_length": "4",
      "rows_examined_per_scan": 3072,
      "rows_produced_per_join": 3072,
      "filtered": "100.00",
      "using_index": true,
      "cost_info": {
        "read_cost": "2.75",
        "eval_cost": "307.20",
        "prefix_cost": "309.95",
        "data_read_per_join": "48K"
      }
    }
  }
}

```

```
    }  
  }  
1 row in set, 1 warning (0.00 sec)
```

从以上结果看出，第二个 SQL 性能 (cost_info) 相对第一个提升了一倍。



MySQL 8.0 错误日志增强特性

作者：郭斌斌

MySQL 8.0 重新定义了错误日志输出和过滤，改善了原来臃肿并且可读性很差的错误日志。

比如增加了 JSON 输出，在原来的日志后面以序号以及 JSON 后缀的方式展示。

比如我机器上的 MySQL 以 JSON 保存的错误日志 `mysqld.log.00.json`：

```
[root@centos-ytt80 mysql80]# jq . mysqld.log.00.json
{
  "log_type": 1,
  "prio": 1,
  "err_code": 12592,
  "subsystem": "InnoDB",
  "msg": "Operating system error number 2 in a file operation.",
  "time": "2019-09-03T08:16:12.111808Z",
  "thread": 8,
  "err_symbol": "ER_IB_MSG_767",
  "SQL_state": "HY000",
  "label": "Error"
}
{
  "log_type": 1,
  "prio": 1,
  "err_code": 12593,
  "subsystem": "InnoDB",
  "msg": "The error means the system cannot find the path specified.",
  "time": "2019-09-03T08:16:12.111915Z",
  "thread": 8,
  "err_symbol": "ER_IB_MSG_768",
  "SQL_state": "HY000",
  "label": "Error"
}
{
  "log_type": 1,
  "prio": 1,
  "err_code": 12216,
  "subsystem": "InnoDB",
  "msg": "Cannot open datafile for read-only: './ytt2/a.ibd' OS error: 71",
  "time": "2019-09-03T08:16:12.111933Z",
  "thread": 8,
  "err_symbol": "ER_IB_MSG_391",
```

```

    "SQL_state": "HY000",
    "label": "Error"
}

```

以 JSON 输出错误日志后可读性和可操作性增强了许多。这里可以用 Linux 命令 jq 或者把这个字符串 COPY 到其他解析 JSON 的工具方便处理。只想非常快速的拿出错误信息，忽略其他信息。

```

[root@centos-ytt80 mysql80]# jq '.msg' mysqld.log.00.json
"Operating system error number 2 in a file operation."
"The error means the system cannot find the path specified."
"Cannot open datafile for read-only: './ytt2/a.ibd' OS error: 71"
"Cannot calculate statistics for table `ytt2`.`a` because the .ibd file is missing.
Please refer to http://dev.mysql.com/doc/refman/8.0/en/innodb-troubleshooting.h
tml for how to resolve the issue."
"Cannot calculate statistics for table `ytt2`.`a` because the .ibd file is missing.
Please refer to http://dev.mysql.com/doc/refman/8.0/en/innodb-troubleshooting.h
tml for how to resolve the issue."

```

使用 JSON 输出的前提是安装 JSON 输出部件。

```

INSTALL COMPONENT 'file://component_log_sink_json';
完了在设置变量 SET GLOBAL log_error_services = 'log_filter_internal; log_sink_json';

```

格式为：过滤规则；日志输出；[过滤规则]日志输出；

查看安装好的部件

```

mysql> select * from mysql.component;
+-----+-----+-----+
| component_id | component_group_id | component_urn |
+-----+-----+-----+
| 2 | 1 | file://component_log_sink_json |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

现在设置 JSON 输出，输出到系统日志的同时输出到 JSON 格式日志。

```

mysql> SET persist log_error_services = 'log_filter_internal; log_sink_internal;
log_sink_json';
Query OK, 0 rows affected (0.00 sec)

```

来测试一把。我之前已经把表 a 物理文件删掉了。

```

mysql> select * from a;
ERROR 1812 (HY000): Tablespace is missing for table `ytt2`.`a`.

```

现在错误日志里有 5 条记录。

```

[root@centos-ytt80 mysql80]# tailf mysqld.log

```

```

2019-09-03T08:16:12.111808Z 8 [ERROR] [MY-012592] [InnoDB] Operating system error
number 2 in a file operation.
2019-09-03T08:16:12.111915Z 8 [ERROR] [MY-012593] [InnoDB] The error means the sy
stem cannot find the path specified.
2019-09-03T08:16:12.111933Z 8 [ERROR] [MY-012216] [InnoDB] Cannot open datafile f
or read-only: './ytt2/a.ibd' OS error: 71
2019-09-03T08:16:12.112227Z 8 [Warning] [MY-012049] [InnoDB] Cannot calculate sta
tistics for table `ytt2`.`a` because the .ibd file is missing. Please refer to htt
p://dev.mysql.com/doc/refman/8.0/en/innodb-troubleshooting.html for how to resol
ve the issue.
2019-09-03T08:16:14.902617Z 8 [Warning] [MY-012049] [InnoDB] Cannot calculate sta
tistics for table `ytt2`.`a` because the .ibd file is missing. Please refer to htt
p://dev.mysql.com/doc/refman/8.0/en/innodb-troubleshooting.html for how to resol
ve the issue.

```

JSON 日志里也有 5 条记录。

```

[root@centos-ytt80 mysql80]# tailf mysqld.log.00.json
{ "log_type" : 1, "prio" : 1, "err_code" : 12592, "subsystem" : "InnoDB", "msg" :
"Operating system error number 2 in a file operation.", "time" : "2019-09-03T08:16:
12.111808Z", "thread" : 8, "err_symbol" : "ER_IB_MSG_767", "SQL_state" : "HY000",
"label" : "Error" }
{ "log_type" : 1, "prio" : 1, "err_code" : 12593, "subsystem" : "InnoDB", "msg" :
"The error means the system cannot find the path specified.", "time" : "2019-09-03
T08:16:12.111915Z", "thread" : 8, "err_symbol" : "ER_IB_MSG_768", "SQL_state" : "
HY000", "label" : "Error" }
{ "log_type" : 1, "prio" : 1, "err_code" : 12216, "subsystem" : "InnoDB", "msg" :
"Cannot open datafile for read-only: './ytt2/a.ibd' OS error: 71", "time" : "2019-
09-03T08:16:12.111933Z", "thread" : 8, "err_symbol" : "ER_IB_MSG_391", "SQL_state
" : "HY000", "label" : "Error" }
{ "log_type" : 1, "prio" : 2, "err_code" : 12049, "subsystem" : "InnoDB", "msg" :
"Cannot calculate statistics for table `ytt2`.`a` because the .ibd file is missing.
Please refer to http://dev.mysql.com/doc/refman/8.0/en/innodb-troubleshooting.h
tml for how to resolve the issue.", "time" : "2019-09-03T08:16:12.112227Z", "thre
ad" : 8, "err_symbol" : "ER_IB_MSG_224", "SQL_state" : "HY000", "label" : "Warning
" }
{ "log_type" : 1, "prio" : 2, "err_code" : 12049, "subsystem" : "InnoDB", "msg" :
"Cannot calculate statistics for table `ytt2`.`a` because the .ibd file is missing.
Please refer to http://dev.mysql.com/doc/refman/8.0/en/innodb-troubleshooting.h
tml for how to resolve the issue.", "time" : "2019-09-03T08:16:14.902617Z", "thre
ad" : 8, "err_symbol" : "ER_IB_MSG_224", "SQL_state" : "HY000", "label" : "Warning
" }

```

那可能有人就问了，这有啥意义呢？只是把格式变了，过滤的规则我看还是没变。

那我们现在给第二条日志输出加过滤规则

先把过滤日志的部件安装起来

```
INSTALL COMPONENT 'file://component_log_filter_dragnet';
```

```
mysql> SET persist log_error_services = 'log_filter_internal; log_sink_internal;  
log_filter_dragnet;log_sink_json';  
Query OK, 0 rows affected (0.00 sec)
```

只保留 error，其余的一律过滤掉。

```
SET GLOBAL dragnet.log_error_filter_rules = 'IF prio>=WARNING THEN drop.';
```

检索一张误删的表

```
mysql> select * from a;  
ERROR 1812 (HY000): Tablespace is missing for table `ytt2`.`a`.
```

查看错误日志和 JSON 错误日志

发现错误日志里有一条 Warning，JSON 错误日志里的被过滤掉了。

```
2019-09-03T08:22:32.978728Z 8 [Warning] [MY-012049] [InnoDB] Cannot calculate sta  
tistics for table `ytt2`.`a` because the .ibd file is missing. Please refer to htt  
p://dev.mysql.com/doc/refman/8.0/en/innodb-troubleshooting.html for how to resol  
ve the issue.
```

再举个例子，每 60 秒只允许记录一个 Warning 事件

```
mysql> SET GLOBAL dragnet.log_error_filter_rules = 'IF prio==WARNING THEN throttl  
e 1/60.';  
Query OK, 0 rows affected (0.00 sec)
```

多次执行

```
mysql> select * from b;  
ERROR 1812 (HY000): Tablespace is missing for table `ytt2`.`b`.  
mysql> select * from b;  
ERROR 1812 (HY000): Tablespace is missing for table `ytt2`.`b`.  
mysql> select * from b;  
ERROR 1812 (HY000): Tablespace is missing for table `ytt2`.`b`.
```

现在错误日志里有三条 warning 信息

```
2019-09-03T08:49:06.820635Z 8 [Warning] [MY-012049] [InnoDB] Cannot calculate sta  
tistics for table `ytt2`.`b` because the .ibd file is missing. Please refer to htt  
p://dev.mysql.com/doc/refman/8.0/en/innodb-troubleshooting.html for how to resol  
ve the issue.
```



```
2019-09-03T08:49:31.455907Z 8 [Warning] [MY-012049] [InnoDB] Cannot calculate statistics for table `ytt2`.`b` because the .ibd file is missing. Please refer to http://dev.mysql.com/doc/refman/8.0/en/innodb-troubleshooting.html for how to resolve the issue.
```

```
2019-09-03T08:50:00.430867Z 8 [Warning] [MY-012049] [InnoDB] Cannot calculate statistics for table `ytt2`.`b` because the .ibd file is missing. Please refer to http://dev.mysql.com/doc/refman/8.0/en/innodb-troubleshooting.html for how to resolve the issue.
```

mysqld.log.00.json 只有一条

```
{ "log_type" : 1, "prio" : 2, "err_code" : 12049, "subsystem" : "InnoDB", "msg" : "Cannot calculate statistics for table `ytt2`.`b` because the .ibd file is missing. Please refer to http://dev.mysql.com/doc/refman/8.0/en/innodb-troubleshooting.html for how to resolve the issue.", "time" : "2019-09-03T08:49:06.820635Z", "thread" : 8, "err_symbol" : "ER_IB_MSG_224", "SQL_state" : "HY000", "and_n_more" : 3, "label" : "Warning" }
```

总结，我这里简单介绍了下 MySQL 8.0 的错误日志过滤以及 JSON 输出。MySQL 8.0 的 `component_log_filter_dragonet` 部件过滤规则非常灵活，可以参考手册，根据它提供的语法写出自己的过滤掉的日志输出。



实战 MySQL 8.0.17 Clone Plugin

作者：陈俊聪

1 背景

很神奇，5.7.17 和 8.0.17，连续两个 17 小版本都让人眼前一亮。前者加入了组复制(Group Replication)功能，后者加入了克隆插件(Clone Plugin)功能。今天我们实战测试一下这个新功能。

2 克隆插件简介

克隆插件允许在本地或从远程 MySQL 实例克隆数据。克隆数据是存储在 InnoDB 其中的数据的物理快照，其中包括库、表、表空间和数据字典元数据。克隆的数据包含一个功能齐全的数据目录，允许使用克隆插件进行 MySQL 服务器配置。

3 克隆插件支持两种克隆方式

- ☐ 本地克隆
- ☐ 远程克隆

4 本地克隆

本地克隆操作将启动克隆操作的 MySQL 服务器实例中的数据克隆到同服务器或同节点上的一个目录里

5 远程克隆

默认情况下，远程克隆操作会删除接受者(recipient)数据目录中的数据，并将其替换为捐赠者(donor)的克隆数据。(可选)您也可以将数据克隆到接受者的其他目录，以避免删除现有数据。

远程克隆操作和本地克隆操作克隆的数据没有区别，数据是相同的。

克隆插件支持复制。除克隆数据外，克隆操作还从捐赠者中提取并传输复制位置信息，并将其应用于接受者，从而可以使用克隆插件来配置组复制或主从复制。使用克隆插件进行配置比复制大量事务要快得多，效率更高。

6 实战部分

6.1 本地克隆

安装克隆插件

启动前

```
[mysql@localhost ~]$  
plugin-load-add=mysql_clone.so
```

或运行中

```
INSTALL PLUGIN clone SONAME 'mysql_clone.so';
```

第二种方法会注册到元数据，所以不用担心重启插件会失效。两种方法都很好用
检查插件有没有启用

```
mysql> SELECT PLUGIN_NAME, PLUGIN_STATUS
        FROM INFORMATION_SCHEMA.PLUGINS
        WHERE PLUGIN_NAME LIKE 'clone';
+-----+-----+
| PLUGIN_NAME | PLUGIN_STATUS |
+-----+-----+
| clone       | ACTIVE        |
+-----+-----+
1 row in set (0.01 sec)
```

设置强制启动失败参数

```
[mysqld]
plugin-load-add=mysql_clone.so
clone=FORCE_PLUS_PERMANENT
```

如果克隆插件对你们很重要，可以设置 clone=FORCE_PLUS_PERMANENT 或 clone=FORCE。作用是：如果插件未成功初始化，就会强制 mysqld 启动失败。

克隆需要的权限

需要有备份锁的权限。备份锁是 MySQL 8.0 的新特性之一，比 5.7 版本的 flush table with read lock 要轻量。

```
mysql> CREATE USER clone_user@'%' IDENTIFIED by 'password';
mysql> GRANT BACKUP_ADMIN ON *.* TO 'clone_user'; # BACKUP_ADMIN 是 MySQL8.0 才有的
备份锁的权限
```

执行本地克隆

```
mysql -uclone_user -ppassword -S /tmp/mysql3008.sock
mysql> CLONE LOCAL DATA DIRECTORY = '/fander/clone_dir';
```

例子中需要 MySQL 的运行用户拥有 fander 目录的 rwx 权限，要求 clone_dir 目录不存在

本地克隆的步骤如下

- ☐ DROP DATA
- ☐ FILE COPY
- ☐ PAGE COPY
- ☐ REDO COPY
- ☐ FILE SYNC

观察方法如下

```
mysql> SELECT STAGE, STATE, END_TIME FROM performance_schema.clone_progress;
+-----+-----+-----+
| STAGE      | STATE      | END_TIME                |
+-----+-----+-----+
| DROP DATA | Completed   | 2019-07-25 21:00:18.858471 |
| FILE COPY  | Completed   | 2019-07-25 21:00:19.071174 |
| PAGE COPY  | Completed   | 2019-07-25 21:00:19.075325 |
| REDO COPY  | Completed   | 2019-07-25 21:00:19.076661 |
| FILE SYNC  | Completed   | 2019-07-25 21:00:19.168961 |
| RESTART    | Not Started | NULL                     |
| RECOVERY    | Not Started | NULL                     |
+-----+-----+-----+
7 rows in set (0.00 sec)
```

当然还有另外一个观察方法

```
mysql> set global log_error_verbosity=3;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> CLONE LOCAL DATA DIRECTORY = '/fander/clone6_dir';
Query OK, 0 rows affected (0.24 sec)
```

```
[root@192-168-199-101 data]# tailf mysql-error.err
2019-07-25T22:22:58.261861+08:00 8 [Note] [MY-013457] [InnoDB] Clone Begin Master Task by root@localhost
2019-07-25T22:22:58.262422+08:00 8 [Note] [MY-013457] [InnoDB] Clone Apply Master Loop Back
2019-07-25T22:22:58.262523+08:00 8 [Note] [MY-013457] [InnoDB] Clone Apply Begin Master Task
...
2019-07-25T22:22:58.471108+08:00 8 [Note] [MY-013458] [InnoDB] Clone Apply State FLUSH DATA:
2019-07-25T22:22:58.472178+08:00 8 [Note] [MY-013458] [InnoDB] Clone Apply State FLUSH REDO:
2019-07-25T22:22:58.506488+08:00 8 [Note] [MY-013458] [InnoDB] Clone Apply State DONE
2019-07-25T22:22:58.506676+08:00 8 [Note] [MY-013457] [InnoDB] Clone Apply End Master Task ID: 0 Passed, code: 0:
2019-07-25T22:22:58.506707+08:00 8 [Note] [MY-013457] [InnoDB] Clone End Master Task ID: 0 Passed, code: 0:
```

测试，起一个克隆的实例

```
/opt/mysql8.0/bin/mysqld --datadir=/fander/clone_dir --port=3333 --socket=/tmp/mysql3333.sock --user=mysql3008user --lower-case-table-names=1 --mysqlx=OFF
```

#解释，因为我没有使用 my.cnf，所以加了比较多参数

#--datadir 指定启动的数据目录

#--port 指定启动的 MySQL 监听端口

#--socket 指定 socket 路径

#--user 捐赠者`的目录权限是 mysql3008user:mysql3008user, 用户也是 mysql3008user, 我没有修改

#--lower-case-table-names=1 同`捐赠者`

#--mysqlx=OFF 不关闭的话，默认 mysqlx 端口会合`捐赠者`的 33060 重复冲突

#登录检查

```
mysql -uroot -proot -S /tmp/mysql3333.sock
```

```
mysql> show master status\G # 可见 GTID 是`捐赠者`的子集，所以说明`接受者`直接和`捐赠者`建主从复制是很简单的
```

6.2 远程克隆

远程克隆的前提条件和限制

- ☐ 捐赠者和接受者都需要安装克隆插件
- ☐ 捐赠者和接受者分别需要有至少 BACKUP_ADMIN/CLONE_ADMIN 权限的账号
 - 暗示了接受者必须先启动一个数据库实例(空或有数据的实例均可，因为都会被删除)
- ☐ 克隆目标目录必须有写入权限
- ☐ 克隆操作期间不允许使用 DDL，允许并发 DML。要求相同版本号，您无法 MySQL 5.7 和 MySQL 8.0 之间进行克隆，而且要求版本>=8.0.17
- ☐ 同一平台同一架构，例如 linux to windows、x64 to x32 是不支持。
- ☐ 足够的磁盘空间
- ☐ 可以克隆操作一般表空间，但必须要有目录权限，不支持克隆使用绝对路径创建的一般空间。与源表空间文件具有相同路径的克隆表空间文件将导致冲突
- ☐ 远程克隆时不支持 CLONE INSTANCE FROM 中通过使用 mysqlx 的端口
- ☐ 克隆插件不支持克隆 MySQL 服务器配置 my.cnf 等
- ☐ 克隆插件不支持克隆二进制日志。
- ☐ 克隆插件仅克隆存储的数据 InnoDB。不克隆其他存储引擎数据。MyISAM 并且 CSV 存储在包括 sys 模式的任何模式中的表都被克隆为空表。
- ☐ 不支持通过 MySQL router 连接到捐赠者实例。
- ☐ 一些参数是必须一致的，例如 innodb_page_size、innodb_data_file_path、--lower_case_table_names
- ☐ 如果克隆加密或页面压缩数据，则捐赠者和接收者必须具有相同的文件系统块大小
- ☐ 如果要克隆加密数据，则需要安全连接
- ☐ clone_valid_donor_list 在接受者的设置必须包含捐赠者 MySQL 服务器实例的主机地址。
- ☐ 必须没有其他克隆操作正在运行。一次只允许一次克隆操作。要确定克隆操作是否正在运行，请查

询该 clone_status 表。

- 默认情况下，克隆数据后会自动重新启动**接受者** MySQL 实例。要自动重新启动，必须在接收方上提供监视进程以检测服务器是否已关闭。否则，在克隆数据后，克隆操作将停止并出现以下错误，并且关闭**接受者** MySQL 服务器实例：

```
ERROR 3707 (HY000): Restart server failed (mysqld is not managed by supervisor process).
```

此错误不表示克隆失败。这意味着必须在克隆数据后手动重新启动接受者的 MySQL 实例。

远程克隆实战

和本地克隆一样，远程克隆需要插件安装和用户授权。**捐赠者**、**接受者**的授权略有不同。

假设前提条件都满足，步骤如下：

1. 确保**捐赠者**和**接受者**都安装了克隆插件

```
INSTALL PLUGIN clone SONAME 'mysql_clone.so';
```

2. 用户账号授权

捐赠者授权

```
mysql> CREATE USER clone_user@'192.168.199.101' IDENTIFIED by 'password1';
mysql> GRANT BACKUP_ADMIN ON *.* TO 'clone_user'@'192.168.199.101'; # BACKUP_ADMIN 是 MySQL8.0 才有的备份锁的权限
```

接受者授权

```
mysql> CREATE USER clone_user@'192.168.199.102' IDENTIFIED by 'password2';
mysql> GRANT CLONE_ADMIN ON *.* TO 'clone_user'@'192.168.199.102';
```

CLONE_ADMIN 权限 = BACKUP_ADMIN 权限 + SHUTDOWN 权限。SHUTDOWN 仅限允许用户 shutdown 和 restart mysqld。授权不同是因为，接受者需要 restart mysqld。

3. **接受者**设置**捐赠者**列表清单

```
mysql -uclone_user -ppassword2 -h192.168.199.102 -P3008
mysql> SET GLOBAL clone_valid_donor_list = '192.168.199.101:3008';
```

这看起来是一个安全相关参数。多个实例用逗号分隔，例如“HOST1:PORT1,HOST2:PORT2,HOST3:PORT3”
接受者开始拉取克隆**捐赠者**数据

```
CLONE INSTANCE FROM clone_user@'192.168.199.101':3008
IDENTIFIED BY 'password1';
```

远程克隆步骤如下

```
mysql> SELECT STAGE, STATE, END_TIME FROM performance_schema.clone_progress;
+-----+-----+-----+
| STAGE      | STATE      | END_TIME                |
+-----+-----+-----+
| DROP DATA | Completed  | 2019-07-25 21:56:01.725783 |
| FILE COPY  | Completed  | 2019-07-25 21:56:02.228686 |
| PAGE COPY  | Completed  | 2019-07-25 21:56:02.331409 |
| REDO COPY  | Completed  | 2019-07-25 21:56:02.432468 |
| FILE SYNC  | Completed  | 2019-07-25 21:56:02.576936 |
| RESTART    | Completed  | 2019-07-25 21:56:06.564090 |
| RECOVERY   | Completed  | 2019-07-25 21:56:06.892049 |
+-----+-----+-----+
7 rows in set (0.01 sec)
```

接受者如果非 `mysqld_safe` 启动的，会报错误，但不影响克隆，需要您人手启动 `mysqld` 即可。

```
ERROR 3707 (HY000): Restart server failed (mysqld is not managed by supervisor process).
```

实操后小结：克隆与 xtrabackup 的对比

- ☐ 克隆和 xtrabackup 备份都属于物理热备，备份恢复原理也近似。
- ☐ 克隆在恢复实例时需要先启动一个实例并授权，而 xtrabackup 不需要。
- ☐ xtrabackup 在 backup 后需要 apply log，克隆是类似 mysqlbackup 提供的 backup-and-apply-log，合并在一起做。
- ☐ xtrabackup 备份文件的权限等于执行命令的人的权限，恢复实例时需要人手 chown 回实例权限，克隆备份后权限和原数据权限一致，无需再人手 chown，方便恢复。
- ☐ xtrabackup 恢复时需要在 mysql 中执行 reset master；然后 set global gtid_purged="UUID:NUMBER"，具体的 UUID:NUMBER 的值为备份文件中的 xtrabackup_info 文件的内容；克隆不需要这个操作步骤，默认克隆完就可以建立复制了。
- ☐ xtrabackup 备份完一般是 scp 拷贝到另外一台机器恢复，走的是 22 端口；克隆走的是 MySQL 的监听端口。所以在目录权限正确的情况下，甚至根本不需要登录 Linux 服务器的权限。如下：

```
[root@192-168-199-103 ~]# mysql -uroot -ppassword2 -h192.168.199.102 -P3008 -e "SET GLOBAL clone_valid_donor_list = '192.168.199.101:3008';"
mysql: [Warning] Using a password on the command line interface can be insecure.
[root@192-168-199-103 ~]# mysql -uroot -ppassword2 -h192.168.199.102 -P3008 -e "CLONE INSTANCE FROM root@'192.168.199.101':3008 IDENTIFIED BY 'password1';"
mysql: [Warning] Using a password on the command line interface can be insecure.
```

6.3 利用克隆建立主从复制

克隆出来的接受者实例，可以与捐赠者建立主从复制。当然建立组复制也是可以的，鉴于篇幅的原因，不做示例有兴趣可以阅读官方文档 18.4.3.1 节“克隆分布式恢复”。

<https://dev.mysql.com/doc/refman/8.0/en/group-replication-cloning.html>

传统复制时，通过以下命令查看 position 位置

```
mysql> SELECT BINLOG_FILE, BINLOG_POSITION FROM performance_schema.clone_status;
+-----+
| BINLOG_FILE      | BINLOG_POSITION |
+-----+
| mysql-bin.000014 |          2179 |
+-----+
1 row in set (0.01 sec)
```

GTID 复制时，通过以下命令查看 GTID 位置

```
mysql> SELECT @@GLOBAL.GTID_EXECUTED;
+-----+
| @@GLOBAL.GTID_EXECUTED          |
+-----+
| 990fa9a4-7aca-11e9-89fa-000c29abbade:1-11 |
+-----+
1 row in set (0.00 sec)
```

假设已经有如下授权，我们就可以建立复制啦

```
create user repl@'%' identified WITH 'mysql_native_password' by 'password';
GRANT REPLICATION SLAVE, REPLICATION CLIENT ON *.* TO 'repl'@'%';
```

建立 GTID 主从复制关系(接受者上)

```
CHANGE MASTER TO
  MASTER_HOST='192.168.199.101',
  MASTER_USER='repl',
  MASTER_PASSWORD='password',
  MASTER_PORT=3008,
  MASTER_AUTO_POSITION=1;
```

7 监控克隆操作

检查克隆进度

```
mysql> SELECT STATE FROM performance_schema.clone_status;
+-----+
| STATE      |
+-----+
| Completed |
+-----+
1 row in set (0.00 sec)
```



```
SELECT STATE, ERROR_NO, ERROR_MESSAGE FROM performance_schema.clone_status;
```

检查克隆是否出错

```
mysql> SELECT STATE, ERROR_NO, ERROR_MESSAGE FROM performance_schema.clone_status;
+-----+-----+-----+
| STATE      | ERROR_NO | ERROR_MESSAGE |
+-----+-----+-----+
| Completed |      0 |               |
+-----+-----+-----+
1 row in set (0.00 sec)
```

检查克隆次数

```
show global status like 'Com_clone'; # `捐赠者` 每次+1, `接受者` 0
```

随时可以 kill 掉克隆

使用

```
SELECT * FROM performance_schema.clone_status\G
```

或

```
show processlist
```

查看线程 ID, 然后 kill ID

8 总结

克隆功能非常有趣。他操作简单, 可以用于快速搭建、恢复主从复制或组复制, 可以部分取代开源热备软件 xtrabackup, 期待未来官方会把他做得越来越好, 功能越来越丰富。



MySQL 8.0.16 开始支持 check 完整性

作者：蒋乐兴

一直以来 MySQL 都只实现了实体完整性、域完整性、引用完整性、唯一键约束。唯独 **check 完整性** 迟迟没有到来，MySQL 8.0.16 (2019-04-25 GA) 版本为这个画上了句号。

1 没有 check 完整性约束会怎样

1. 没有 check 完整性约束可能会影响到数据的质量。

```
-- 创建一个 person 表来保存名字，年龄这两个基本信息
CREATE TABLE person ( NAME VARCHAR ( 16 ), age INT );

-- 这种情况下可以插入一个年龄为 -32 的行，负的年龄明显是没有意义的
INSERT INTO person ( NAME, age )
VALUE ( '张三岁', - 32 );

select * from person;
+-----+-----+
| name   | age   |
+-----+-----+
| 张三岁 | -32   |
+-----+-----+
1 row in set (0.00 sec)
```

2. 如果单单只是不能容忍负值，我们可以换一种非负整数类型来克服一下。

```
-- 先删除之前的 person 定义
drop table if exists person;
Query OK, 0 rows affected (0.01 sec)

-- 创建一个新的 person 表
create table if not exists person(name varchar(16),age int unsigned);
Query OK, 0 rows affected (0.01 sec)

-- 声明的时候说明了 age 只能是正数，所以插入负数就会报错
insert into person(name,age) value('张三岁',-32);
ERROR 1264 (22003): Out of range value for column 'age' at row 1
```

上面的这种解决方案其实就是通过域完整性来实现的数据验证，域完整性的能力还是有边界的。

比如说，要求 age 一定要 18 岁之上它是做不到的，而这种需求正是 check 完整性大显身手的地方。

2 看用 check 如何解决

1. MySQL 8.0.16+版本，check 完整性解决 age 要大于 18 的问题。

```
select @@version;
```

```
+-----+
```

```
| @@version |
```

```
+-----+
```

```
| 8.0.16    |
```

```
+-----+
```

```
1 row in set (0.00 sec)
```

```
-- 先删除之前定义的表
```

```
drop table if exists person;
```

```
Query OK, 0 rows affected (0.01 sec)
```

```
-- 创建新的表，并对 age 列进行验证，要求它一定要大于 18
```

```
create table if not exists person(
```

```
    name varchar(16),
```

```
    age int,
```

```
    constraint ck_person_001 check (age > 18) -- 加一个 check 约束条件
```

```
);
```

```
Query OK, 0 rows affected (0.02 sec)
```

```
-- 查看表的定义
```

```
show create table person;
```

```
+-----+-----+
```

```
| Table | Create Table |
```

```
+-----+-----+
```

```
| person | CREATE TABLE `person` (
```

```
`name` varchar(16) DEFAULT NULL,
```

```
`age` int(11) DEFAULT NULL,
```

```
CONSTRAINT `ck_person_001` CHECK ((`age` > 18))
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci |
```

```
+-----+-----+
```

```
1 row in set (0.00 sec)
```

```
-- 插入的数据如果不能通过 check 约束就会报错
```

```
insert into person(name,age) value('张三岁',-32);
```

```
ERROR 3819 (HY000): Check constraint 'ck_person_001' is violated.
```

```
insert into person(name,age) values('张三岁',17);
```

```
ERROR 3819 (HY000): Check constraint 'ck_person_001' is violated.
```

```
insert into person(name,age) values('张三岁',19);
Query OK, 1 row affected (0.01 sec)
```

2. MySQL 8.0.16 以下版本的 MySQL 会怎样?

```
select @@version;
+-----+
| @@version |
+-----+
| 8.0.15    |
+-----+
1 row in set (0.00 sec)
```

```
drop table if exists person;
Query OK, 0 rows affected (0.00 sec)
```

```
create table if not exists person(
    name varchar(16),
    age int,
    constraint ck_person_001 check (age > 18) -- 加一个 check 约束条件
);
Query OK, 0 rows affected (0.01 sec)
```

-- 可以看到在低版本下 check 约束被直接无视了，也就是说低版本是没有 check 约束的

```
show create table person;
```

```
+-----+-----+
| Table | Create Table |
+-----+-----+
| person | CREATE TABLE `person` (
`name` varchar(16) COLLATE utf8mb4_general_ci DEFAULT NULL,
`age` int(11) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci |
+-----+-----+
1 row in set (0.00 sec)
```

```
--
insert into person(name,age) value('张三岁',-32);
Query OK, 1 row affected (0.00 sec)
```

可以看到在 MySQL 8.0.16 以下的版本中，check 直接被忽略。

新版本在 CHECK Constraints 功能上的完善提高了对非法或不合理数据写入的控制能力。
除了以上示例中的列约束之外，CHECK Constraints 还支持表约束。

想要了解更多关于 CHECK Constraints 的详细语法规则和注意事项，请参考 MySQL 官网文档
<https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html>



MySQL 8.0: 字符集从 utf8 转换成 utf8mb4

作者：胡呈清

整理 MySQL 8.0 文档时发现一个变更：

默认字符集由 latin1 变为 utf8mb4。想起以前整理过字符集转换文档，升级到 MySQL 8.0 后大概率会有字符集转换的需求，在此正好分享一下。

当时的需求背景是：

部分系统使用的字符集是 utf8，但 utf8 最多只能存 3 字节长度的字符，不能存放 4 字节的生僻字或者表情符号，因此打算迁移到 utf8mb4。

1 迁移方案一

1. 准备新的数据库实例，修改以下参数：

```
[mysqld]
## Character Settings
init_connect='SET NAMES utf8mb4'
#连接建立时执行设置的语句，对 super 权限用户无效
character-set-server = utf8mb4
collation-server = utf8mb4_general_ci
#设置服务端校验规则，如果字符串需要区分大小写，设置为 utf8mb4_bin
skip-character-set-client-handshake
#忽略应用连接自己设置的字符编码，保持与全局设置一致
## InnoDB Settings
innodb_file_format = Barracuda
innodb_file_format_max = Barracuda
innodb_file_per_table = 1
innodb_large_prefix = ON
#允许索引的最大字节数为 3072（不开启则最大为 767 字节，对于类似 varchar(255)字段的索引会有问题，因为 255*4 大于 767）
```

2. 停止应用，观察，确认不再有数据写入

可通过 show master status 观察 GTID 或者 binlog position，没有变化则没有写入。

3. 导出数据

先导出表结构：

```
mysqldump -u -p --no-data --default-character-set=utf8mb4 --single-transaction --
set-gtid-purged=OFF --databases testdb > /backup/testdb.sql
```

后导出数据:

```
mysqldump -u -p --no-create-info --master-data=2 --flush-logs --routines --events  
--triggers --default-character-set=utf8mb4 --single-transaction --set-gtid-purg  
ed=OFF --database testdb > /backup/testdata.sql
```

4. 修改建表语句

修改导出的表结构文件, 将表、列定义中的 utf8 改为 utf8mb4

5. 导入数据

先导入表结构:

```
mysql -u -p testdb < /backup/testdb.sql
```

后导入数据:

```
mysql -u -p testdb < /backup/testdata.sql
```

6. 建用户

查出旧环境的数据库用户, 在新数据库中创建

7. 修改新数据库端口, 启动应用进行测试

关闭旧数据库, 修改新数据库端口重启, 启动应用

2 迁移方案二

1. 修改表的字符编码会锁表, 建议先停止应用
2. 停止 mysql, 备份数据目录 (也可以其他方式进行全备)
3. 修改配置文件, 重启数据库

```
[mysqld]  
## Character Settings  
init_connect='SET NAMES utf8mb4'  
#连接建立时执行设置的语句, 对 super 权限用户无效  
character-set-server = utf8mb4  
collation-server = utf8mb4_general_ci  
#设置服务端校验规则, 如果字符串需要区分大小写, 设置为 utf8mb4_bin  
skip-character-set-client-handshake  
#忽略应用连接自己设置的字符编码, 保持与全局设置一致  
## InnoDB Settings  
innodb_file_format = Barracuda  
innodb_file_format_max = Barracuda  
innodb_file_per_table = 1  
innodb_large_prefix = ON  
#允许索引的最大字节数为 3072 (不开启则最大为 767 字节, 对于类似 varchar(255) 字段的索引会有问题, 因为 255*4 大于 767)
```

4. 查看所有表结构，包括字段、修改库和表结构，如果字段有定义字符编码，也需要修改字段属性，sql 语句如下：

修改表的字符集：

```
alter table t convert to character set utf8mb4;
```

影响：拷贝全表，速度慢，会加锁，阻塞写操作

修改字段的字符集（utf8mb4 每字符占 4 字节，注意字段类型的最大字节数与字符长度关系）：

```
alter table t modify a char CHARACTER SET utf8mb4;
```

影响：拷贝全表，速度慢，会加锁，阻塞写操作

修改 database 的字符集：

```
alter database sbtest CHARACTER SET utf8mb4;
```

影响：只需修改元数据，速度很快

5. 修改 JDBC url `characterEncoding=utf-8`

故障系列

顾名思义，本系列分享内容都源于用户的现场故障。为您分析故障原因和解决定位故障的过程，将问题的相关知识点加以融合说明。为您呈现一线 DBA 最真实的工作写照。希望在看过作者们的故障分析后，有助于读者快速定位解决现场故障。

微信扫一扫读原文



多从库时半同步复制不工作的 BUG 分析

作者：胡呈清 黄炎

1 问题描述

MySQL 版本：5.7.16, 5.7.17, 5.7.21

存在多个半同步从库时，如果参数 `rpl_semi_sync_master_wait_for_slave_count=1`，启动第 1 个半同步从库时可以正常启动，启动第 2 个半同步从库后有很大概率 `slave_io_thread` 停滞，（复制状态正常，`Slave_IO_Running: Yes`, `Slave_SQL_Running: Yes`，但是完全不同步主库 binlog）

2 复现步骤

1. 主库配置参数如下：

```
rpl_semi_sync_master_wait_for_slave_count = 1
rpl_semi_sync_master_wait_no_slave = OFF
rpl_semi_sync_master_enabled = ON
rpl_semi_sync_master_wait_point = AFTER_SYNC
```

2. 启动从库 A 的半同步复制 `start slave`，查看从库 A 复制正常

3. 启动从库 B 的半同步复制 `start slave`，查看从库 B，复制线程正常，但是不同步主库 binlog

3 分析过程

首先弄清楚这个问题，需要先结合 MySQL 其他的一些状态信息，尤其是主库的 `dump` 线程状态来进行分析

3.1 从库 A 启动复制后，主库的半同步状态已启动：

```
show global status like '%semi%';
+-----+
| Variable_name | Value |
+-----+
| Rpl_semi_sync_master_clients | 1
....
| Rpl_semi_sync_master_status | ON
```

再看主库的 dump 线程，也已经启动：

```
select * from performance_schema.threads where PROCESSLIST_COMMAND='Binlog Dump
GTID'\G
***** 1. row *****
THREAD_ID: 21872
NAME: thread/sql/one_connection
TYPE: FOREGROUND
PROCESSLIST_ID: 21824
PROCESSLIST_USER: universe_op
PROCESSLIST_HOST: 172.16.21.5
PROCESSLIST_DB: NULL
PROCESSLIST_COMMAND: Binlog Dump GTID
PROCESSLIST_TIME: 300
PROCESSLIST_STATE: Master has sent all binlog to slave; waiting for more updates
PROCESSLIST_INFO: NULL
PARENT_THREAD_ID: NULL
ROLE: NULL
INSTRUMENTED: YES
HISTORY: YES
CONNECTION_TYPE: TCP/IP
THREAD_OS_ID: 24093
```

再看主库的 error log，也显示 dump 线程（21824）启动成功，其启动的半同步复制：

```
2018-05-25T11:21:58.385227+08:00 21824 [Note] Start binlog_dump to
master_thread_id(21824) slave_server(1045850818), pos(, 4)
2018-05-25T11:21:58.385267+08:00 21824 [Note] Start semi-sync binlog_dump to
slave (server_id: 1045850818), pos(, 4)
2018-05-25T11:21:59.045568+08:00 0 [Note] Semi-sync replication switched ON at
(mysql-bin.000005, 81892061)
```

1. 从库 B 启动复制后，主库的半同步状态，还是只有 1 个半同步从库

```
Rpl_semi_sync_master_clients=1:
show global status like '%semi%';
```

```
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Rpl_semi_sync_master_clients | 1
...
| Rpl_semi_sync_master_status | ON
...
```

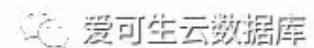
再看主库的 dump 线程，这时有 3 个 dump 线程，但是新起的那两个一直为 starting 状态：

```
select * from performance_schema.threads where PROCESSLIST_COMMAND='Binlog Dump GTID\G
***** 1. row *****
THREAD_ID: 21872
NAME: thread/sql/one_connection
TYPE: FOREGROUND
PROCESSLIST_ID: 21824
PROCESSLIST_USER: universe_op
PROCESSLIST_HOST: 172.16.21.5
PROCESSLIST_DB: NULL
PROCESSLIST_COMMAND: Binlog Dump GTID
PROCESSLIST_TIME: 695
PROCESSLIST_STATE: Master has sent all binlog to slave; waiting for more updates
PROCESSLIST_INFO: NULL
PARENT_THREAD_ID: NULL
ROLE: NULL
INSTRUMENTED: YES
HISTORY: YES
CONNECTION_TYPE: TCP/IP
THREAD_OS_ID: 24093
***** 2. row *****
THREAD_ID: 21895
NAME: thread/sql/one_connection
TYPE: FOREGROUND
PROCESSLIST_ID: 21847
PROCESSLIST_USER: universe_op
PROCESSLIST_HOST: 172.16.21.6
PROCESSLIST_DB: NULL
PROCESSLIST_COMMAND: Binlog Dump GTID
PROCESSLIST_TIME: 94
PROCESSLIST_STATE: starting
PROCESSLIST_INFO: NULL
PARENT_THREAD_ID: NULL
ROLE: NULL
INSTRUMENTED: YES
HISTORY: YES
CONNECTION_TYPE: TCP/IP
THREAD_OS_ID: 24102
```

```

***** 3. row *****
THREAD_ID: 21898
NAME: thread/sql/one_connection
TYPE: FOREGROUND
PROCESSLIST_ID: 21850
PROCESSLIST_USER: universe_op
PROCESSLIST_HOST: 172.16.21.6
PROCESSLIST_DB: NULL
PROCESSLIST_COMMAND: Binlog Dump GTID
PROCESSLIST_TIME: 34
PROCESSLIST_STATE: starting
PROCESSLIST_INFO: NULL
PARENT_THREAD_ID: NULL
ROLE: NULL
INSTRUMENTED: YES
HISTORY: YES
CONNECTION_TYPE: TCP/IP
THREAD_OS_ID: 24966
3 rows in set (0.00 sec)

```



再看主库的 error log，21847 这个新的 dump 线程一直没起来，直到 1 分钟之后从库 retry (Connect_Retry 和 Master_Retry_Count 相关)，主库上又启动了 1 个 dump 线程 21850，还是起不来，并且 21847 这个僵尸线程还停不掉：

```

2018-05-25T11:31:59.586214+08:00 21847 [Note] Start binlog_dump to
master_thread_id(21847) slave_server(873074711), pos(, 4)
2018-05-25T11:32:59.642278+08:00 21850 [Note] While initializing dump thread for
slave with UUID <f4958715-5ef3-11e8-9271-0242ac101506>, found a zombie dump
thread with the same UUID. Master is killing the zombie dump thread(21847).
2018-05-25T11:32:59.642452+08:00 21850 [Note] Start binlog_dump to
master_thread_id(21850) slave_server(873074711), pos(, 4)

```

3.3 到这里我们可以知道，从库 B slave_io_thread 停滞的根本原因是因为主库上对应的 dump 线程

启动不了。如何进一步分析线程调用情况？推荐使用 gstack 或者 pstack（实为 gstack 软链）来查看线程调用栈，其用法很简单：gstack <process-id>

3.4 看主库的 gstack, 可以看到 24102 线程 (旧的复制 dump 线程) 堆栈:

```
Thread 4 (Thread 0x7f0be0549700 (LWP 24102)):  
#0 0x00007f0c03c0f1bd in __lll_lock_wait () from /lib64/libpthread.so.0  
#1 0x00007f0c03c0ad38 in _L_lock_975 () from /lib64/libpthread.so.0  
#2 0x00007f0c03c0ace1 in pthread_mutex_lock () from /lib64/libpthread.so.0  
#3 0x00007f0be7588d84 in native_mutex_lock (mutex=0x7f0be778daf0 <ack_receiver+16>)  
at /export/home/pb2/build/sb_0-19016729-1464157976.67/mysql-5.7.13/include/thr_mutex.h:84  
#4 my_mutex_lock (mp=0x7f0be778daf0 <ack_receiver+16>) at /export/home/pb2/build/sb_0-19016729-  
1464157976.67/mysql-5.7.13/include/thr_mutex.h:182  
#5 inline_mysql_mutex_lock (src_line=160, that=<optimized out>, src_file=<optimized out>)  
at /export/home/pb2/build/sb_0-19016729-1464157976.67/mysql-5.7.13/include/mysql/psi/mysql_thread.h:730  
#6 Ack_receiver::remove_slave (this=0x7f0be778dae0 <ack_receiver>, thd=0x7f0b8800fa20)
```

可以看到 24966 线程 (新的复制 dump 线程) 堆栈:

```
Thread 12 (Thread 0x7f0be0751700 (LWP 24966)):  
#0 0x00007f0c03c0f1bd in __lll_lock_wait () from /lib64/libpthread.so.0  
#1 0x00007f0c03c0ad38 in _L_lock_975 () from /lib64/libpthread.so.0  
#2 0x00007f0c03c0ace1 in pthread_mutex_lock () from /lib64/libpthread.so.0  
#3 0x00007f0be75895cc in native_mutex_lock (mutex=0x7f0be778daf0 <ack_receiver+16>)  
at /export/home/pb2/build/sb_0-19016729-1464157976.67/mysql-5.7.13/include/thr_mutex.h:84  
#4 my_mutex_lock (mp=0x7f0be778daf0 <ack_receiver+16>) at /export/home/pb2/build/sb_0-19016729-  
1464157976.67/mysql-5.7.13/include/thr_mutex.h:182  
#5 inline_mysql_mutex_lock (src_line=138, that=0x7f0be778daf0 <ack_receiver+16>, src_file=<optimized out>)  
at /export/home/pb2/build/sb_0-19016729-1464157976.67/mysql-5.7.13/include/mysql/psi/mysql_thread.h:730  
#6 Ack_receiver::add_slave (this=0x7f0be778dae0 <ack_receiver>, thd=<optimized out>)  
at /export/home/pb2/build/sb_0-19016729-  
1464157976.67/mysql-5.7.13/plugin/semisync/semisync_master_ack_receiver.cc:138
```

两线程都在等待 Ack_Receiver 的锁, 而线程 21875 在持有锁, 等待 select:

```
Thread 15 (Thread 0x7f0bce7fc700 (LWP 21875)):  
#0 0x00007f0c028c9bd3 in select () from /lib64/libc.so.6  
#1 0x00007f0be7589070 in Ack_receiver::run (this=0x7f0be778dae0 <ack_receiver>)  
at /export/home/pb2/build/sb_0-19016729-1464157976.67/mysql-  
5.7.13/plugin/semisync/semisync_master_ack_receiver.cc:261  
#2 0x00007f0be75893f9 in ack_receive_handler (arg=0x7f0be778dae0 <ack_receiver>)  
at /export/home/pb2/build/sb_0-19016729-1464157976.67/mysql-  
5.7.13/plugin/semisync/semisync_master_ack_receiver.cc:34  
#3 0x00000000011cf5f4 in pfs_spawn_thread (arg=0x2d68f00) at  
/export/home/pb2/build/sb_0-19016729-1464157976.67/mysql-  
5.7.13/storage/perfschema/pfs.cc:2188  
#4 0x00007f0c03c08dc5 in start_thread () from /lib64/libpthread.so.0  
#5 0x00007f0c028d276d in clone () from /lib64/libc.so.6
```

理论上 select 不应 hang, Ack_receiver 中的逻辑也不会死循环, 请教公司大神黄炎进行一波源码分析。

3.5 semisync_master_ack_receiver.cc 的以下代码形成了对互斥锁的抢占, 饿死了其他竞争者:

```
void Ack_receiver::run()
{
    ...
    while(1)
    {
        mysql_mutex_lock(&m_mutex);
        ...
        select(...);
        ...
        mysql_mutex_unlock(&m_mutex);
    }
    ...
}
```

在 mysql_mutex_unlock 调用后, 应适当增加其他线程的调度机会。

试验:在 mysql_mutex_unlock 调用后增加 sched_yield();, 可验证问题现象消失。

4 结论

1. 从库 slave_io_thread 停滞的根本原因是主库对应的 dump thread 启动不了;
2. rpl_semi_sync_master_wait_for_slave_count=1 时, 启动第一个半同步后, 主库 ack_receiver 线程会不断的循环判断收到的 ack 数量是否 >=
3. rpl_semi_sync_master_wait_for_slave_count, 此时判断为 true, ack_receiver 基本不会空闲一直持有锁。此时启动第 2 个半同步, 对应主库要启动第 2 个 dump thread, 启动 dump thread 要等待 ack_receiver 锁释放, 一直等不到, 所以第 2 个 dump thread 启动不了。

相信各位 DBA 同学看了后会很震惊, “什么? 居然还有这样的 bug...”, 这里要说明一点, 这个 bug 触发是有几率的, 但是几率又很大。这个问题已经由我司大神提交了 bug 和 patch:

<https://bugs.mysql.com/bug.php?id=89370>, 加上本人提交 SR 后时不时的催一催, 官方终于确认修复在 5.7.23 (官方最终另有修复方法, 没采纳这个 patch)。



JDBC 与 MySQL 临时表空间的分析

作者：秦沛 胡呈清

1 背景

应用 JDBC 连接参数采用 `useCursorFetch=true`，查询结果集存放在 `mysqld` 临时表空间中，导致 `ibtmp1` 文件大小暴增到 90 多 G，耗尽服务器磁盘空间。为了限制临时表空间的大小，设置了：

```
innodb_temp_data_file_path = ibtmp1:12M:autoextend:max:2G
```

2 问题描述

在限制了临时表空间后，当应用仍按以前的方式访问时，`ibtmp1` 文件达到 2G 后，程序一直等待直到超时断开连接。`SHOW PROCESSLIST` 显示程序的连接线程为 `sleep` 状态，`state` 和 `info` 信息为空。这个对应用开发来说不太友好，程序等待超时之后要分析原因也缺少提示信息。

3 问题分析过程

为了分析问题，我们进行了以下测试

测试环境：

1. mysql: 5.7.16
2. java: 1.8u162
3. jdbc 驱动: 5.1.36
4. OS: Red Hat 6.4

3.1 手工模拟临时表超过最大限制的场景

模拟以下环境：

1. `ibtmp1:12M:autoextend:max:30M`
2. 将一张 500 万行的 `sbtest` 表的 `k` 字段索引删除

运行一条 `group by` 的查询，产生的临时表大小超过限制后，会直接报错：

```
select sum(k) from sbtest1 group by k;
ERROR 1114 (HY000): The table '/tmp/#sql_60f1_0' is full
```

3.2 查驱动对 MySQL 的设置

我们上一步看到，SQL 手工执行会返回错误，但是 `jdbc` 不返回错误，导致连接一直 `sleep`，怀疑是 `mysql` 驱动做了特殊设置，驱动连接 `mysql`，通过 `general_log` 查看做了哪些设置。未发现做特殊设置。

3.3 测试 JDBC 连接

问题的背景中有对 JDBC 做特殊配置：useCursorFetch=true，不知道是否与隐藏报错有关，接下来进行测试：

```
→engine.execute(props,"jdbc:mysql://10.186.24.31:3336/hucq?useSSL=false");
//→engine.execute(props,"jdbc:mysql://10.186.24.31:3336/hucq?useSSL=false&useCursorFetch=true");
```

发现以下现象：

1. 加参数 useCursorFetch=true 时，做同样的查询确实不会报错

这个参数是为了防止返回结果集过大而采用分段读取的方式。即程序下发一个 sql 给 mysql 后，会等 mysql 可以读结果的反馈，由于 mysql 在执行 sql 时，返回结果达到 ibtmp 上限后报错，但没有关闭该线程，该线程处理 sleep 状态，程序得不到反馈，会一直等，没有报错。如果 kill 这个线程，程序则会报错。

2. 不加参数 useCursorFetch=true 时，做同样的查询则会报错

```
java.sql.SQLException: The table '/tmp/#sql_60f1_0' is full
    at com.mysql.jdbc.SQLException.createSQLException(SQLException.java:998)
    at com.mysql.jdbc.MysqlIO.checkErrorPacket(MysqlIO.java:3847)
    at com.mysql.jdbc.MysqlIO.nextRowFast(MysqlIO.java:2059)
    at com.mysql.jdbc.MysqlIO.nextRow(MysqlIO.java:1933)
    at com.mysql.jdbc.MysqlIO.readSingleRowSet(MysqlIO.java:3275)
    at com.mysql.jdbc.MysqlIO.getResultSet(MysqlIO.java:463)
    at com.mysql.jdbc.MysqlIO.readResultsForQueryOrUpdate(MysqlIO.java:3009)
    at com.mysql.jdbc.MysqlIO.readAllResults(MysqlIO.java:2257)
    at com.mysql.jdbc.MysqlIO.sqlQueryDirect(MysqlIO.java:2650)
    at com.mysql.jdbc.ConnectionImpl.execSQL(ConnectionImpl.java:2541)
    at com.mysql.jdbc.ConnectionImpl.execSQL(ConnectionImpl.java:2499)
    at com.mysql.jdbc.StatementImpl.executeQuery(StatementImpl.java:1432)
```

4 结论

正常情况下，sql 执行过程中临时表大小达到 ibtmp 上限后会报错；

当 JDBC 设置 useCursorFetch=true，sql 执行过程中临时表大小达到 ibtmp 上限后不会报错。

5 解决方案

1. 进一步了解到使用 useCursorFetch=true 是为了防止查询结果集过大撑爆 jvm；
2. 但是使用 useCursorFetch=true 又会导致普通查询也生成临时表，造成临时表空间过大的问题；
3. 临时表空间过大的解决方案是限制 ibtmp1 的大小，然而 useCursorFetch=true 又导致 JDBC 不返回错误。
4. 所以需要使用其它方法来达到相同的效果，且 sql 报错后程序也要相应的报错。除了 useCursorFetch=true 这种段读取的方式外，还可以使用流读取的方式。流读取程序详见附件部分。

报错对比

1. 段读取方式，sql 报错后，程序不报错
2. 流读取方式，sql 报错后，程序会报错

内存占用对比

这里对比了普通读取、段读取、流读取三种方式，初始内存占用 28M 左右：

1. 普通读取后，内存占用 100M 多
2. 段读取后，内存占用 60M 左右
3. 流读取后，内存占用 60M 左右

6 知识补充点

MySQL 共享临时表空间知识点

MySQL 5.7 在 temporary tablespace 上做了改进，已经实现将 temporary tablespace 从 ibdata（共享表空间文件）中分离。并且可以重启重置大小，避免出现像以前 ibdata 过大难以释放的问题。

其参数为：innodb_temp_data_file_path

6.1 表现

MySQL 启动时 datadir 下会创建一个 ibtmp1 文件，初始大小为 12M，默认值下会无限扩展：

通常来说，查询导致的临时表（如 group by）如果超出 tmp_table_size、max_heap_table_size 大小限制则创建 innodb 磁盘临时表（MySQL5.7 默认临时表引擎为 innodb），存放在共享临时表空间；

如果某个操作创建了一个大小为 100 M 的临时表，则临时表空间数据文件会扩展到 100M 大小以满足临时表的需要。当删除临时表时，释放的空间可以重新用于新的临时表，但 ibtmp1 文件保持扩展大小。

6.2 查询视图

可查询共享临时表空间的使用情况：

```
SELECT FILE_NAME, TABLESPACE_NAME, ENGINE, INITIAL_SIZE,
TOTAL_EXTENTS*EXTENT_SIZE AS TotalSizeBytes, DATA_FREE, MAXIMUM_SIZE FROM
INFORMATION_SCHEMA.FILES WHERE TABLESPACE_NAME = 'innodb_temporary'\G
```

```
***** 1. row *****
      FILE_NAME:      /data/mysql5722/data/ibtmp1
TABLESPACE_NAME:      innodb_temporary
      ENGINE:         InnoDB
      INITIAL_SIZE:    12582912
      TotalSizeBytes:   31457280
      DATA_FREE:       27262976
      MAXIMUM_SIZE:    31457280
1 row in set (0.00 sec)
```

6.3 回收方式

重启 MySQL 才能回收

6.4 限制大小

为防止临时数据文件变得过大，可以配置该 `innodb_temp_data_file_path`（需重启生效）选项以指定最大文件大小，当数据文件达到最大大小时，查询将返回错误：

```
innodb_temp_data_file_path=ibtmp1:12M:autoextend:max:2G
```

6.5 临时表空间与 tmpdir 对比

共享临时表空间用于存储非压缩 InnoDB 临时表 (non-compressed InnoDB temporary tables)、关系对象 (related objects)、回滚段 (rollback segment) 等数据；

`tmpdir` 用于存放指定临时文件 (temporary files) 和临时表 (temporary tables)，与共享临时表空间不同的是，`tmpdir` 存储的是 compressed InnoDB temporary tables。

可通过如下语句测试：

```
CREATE TEMPORARY TABLE compress_table (id int, name char(255))
ROW_FORMAT=COMPRESSED;
CREATE TEMPORARY TABLE uncompress_table (id int, name char(255)) ;
```

7 附件

SimpleExample.java

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Properties;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.atomic.AtomicLong;
public class SimpleExample {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.jdbc.Driver");
        Properties props = new Properties();
        props.setProperty("user", "root");
        props.setProperty("password", "root");
        SimpleExample engine = new SimpleExample();
        // engine.execute(props,"jdbc:mysql://10.186.24.31:3336/hucq?useSSL=false");
        engine.execute(props,"jdbc:mysql://10.186.24.31:3336/hucq?useSSL=false&useCursorFetch=true");
    }
    final AtomicLong tmAl = new AtomicLong();
    final String tableName="test";
```

```
public void execute(Properties props,String url) {
    CountdownLatch cdl = new CountdownLatch(1);
    long start = System.currentTimeMillis();
    for (int i = 0; i < 1; i++) {
        TestThread insertThread = new TestThread(props,cdl, url);
        Thread t = new Thread(insertThread);
        t.start();
        System.out.println("Test start");
    }
    try {
        cdl.await();
        long end = System.currentTimeMillis();
        System.out.println("Test end,total cost:" + (end-start) + "ms");
    } catch (Exception e) {
    }
}
```

```
class TestThread implements Runnable {
    Properties props;
    private CountdownLatch countDownLatch;
    String url;
    public TestThread(Properties props,CountDownLatch cdl,String url) {
        this.props = props;
        this.countDownLatch = cdl;
        this.url = url;
    }
    public void run() {
        Connection connection = null;
        PreparedStatement ps = null;
        Statement st = null;
        long start = System.currentTimeMillis();
        try {
            connection = DriverManager.getConnection(url,props);
            connection.setAutoCommit(false);
            st = connection.createStatement();

            //st.setFetchSize(500);
            st.setFetchSize(Integer.MIN_VALUE);    //仅修改此处即可

            ResultSet rstmp;

            st.executeQuery("select sum(k) from sbtest1 group by k");
            rstmp = st.getResultSet();
            while(rstmp.next()){

            }

        }
    }
}
```

```
    } catch (Exception e) {
        System.out.println(System.currentTimeMillis() - start);
        System.out.println(new java.util.Date().toString());
        e.printStackTrace();
    } finally {
        if (ps != null)
            try {
                ps.close();
            } catch (SQLException e1) {
                e1.printStackTrace();
            }
        if (connection != null)
            try {
                connection.close();
            } catch (SQLException e1) {
                e1.printStackTrace();
            }
        this.countDownLatch.countDown();
    }
}
}
```



MySQL Insert 加锁与死锁分析

作者：胡呈清

s session 1	s session 2	s session 2
commit;		
begin;delete from t3 where c2=15		
	begin;insert into t3(c2) values(15);	begin;insert into t3(c2) values(15);

在我们尝试回答这个问题前，一定要注意前提条件，如果你看过登博的《MySQL 加锁处理》，一定知道前提不同答案也就不同，如果还没看过建议你去看一下，链接：<http://hedengcheng.com/?p=771>

那么这个问题缺少哪些前提条件？

1. c2 字段建有唯一索引
2. 隔离级别为：READ-COMMITTED

其实网络上有类似的案例分析，其中丁奇老师在《MySQL 实战 45 讲》中的第 40 篇《insert 语句的锁为什么这么多？》中有一样的例子和分析，但是我的理解有些微差异，所以来说说我的看法，如果有不对的地方请大家指正。首先我会分析一下这个场景的加锁情况和死锁原因，然后对于差异的点进行展开，最后总结 insert 的加锁情况（关于 insert 的加锁行为，其实不像 delete 那样简单清晰，里面有一些需要注意的点）。

1 加锁情况与死锁原因分析

为方便大家复现，完整表结构和数据如下：

```
CREATE TABLE `t3` (  
  `c1` int(11) NOT NULL AUTO_INCREMENT,  
  `c2` int(11) DEFAULT NULL,  
  PRIMARY KEY (`c1`),  
  UNIQUE KEY `c2` (`c2`)  
) ENGINE=InnoDB  
insert into t3 values(1,1),(15,15),(20,20);
```

在 session1 执行 commit 的瞬间，我们会看到 session2、session3 的其中一个报死锁。这个死锁是这样产生的：

1. session1 执行 delete 会在唯一索引 c2 的 c2 = 15 这一记录上加 X lock (也就是在 MySQL 内部观测到的: X Lock but not gap);
2. session2 和 session3 在执行 insert 的时候, 由于唯一约束检测发生唯一冲突, 会加 S Next-Key Lock, 即对 (1,15] 这个区间加锁包括间隙, 并且被 session1 的 X Lock 阻塞, 进入等待;
3. session1 在执行 commit 后, 会释放 X Lock, session2 和 session3 都获得 S Next-Key Lock;
4. session2 和 session3 继续执行插入操作, 这个时候 INSERT INTENTION LOCK (插入意向锁) 出现了, 并且由于插入意向锁会被 gap 锁阻塞, 所以 session2 和 session3 互相等待, 造成死锁。

死锁日志如下:

```

2019-03-06 14:55:59 0x7f2ab4d76700
*** (1) TRANSACTION:
TRANSACTION 6136, ACTIVE 3 sec inserting
mysql tables in use 1, locked 1
LOCK WAIT 3 lock struct(s), heap size 1136, 3 row lock(s), undo log entries 1
MySQL thread id 58, OS thread handle 139821399099136, query id 20130 localhost root update
insert into t3(c2) values(15)
*** (1) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 38 page no 4 n bits 72 index c2 of table `hucq`.`t3` trx id 6136 lock_mode X locks gap before rec insert intention waiting
Record lock, heap no 4 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 80000014; asc ;;
 1: len 4; hex 80000014; asc ;;

*** (2) TRANSACTION:
TRANSACTION 6135, ACTIVE 7 sec inserting
mysql tables in use 1, locked 1
3 lock struct(s), heap size 1136, 3 row lock(s), undo log entries 1
MySQL thread id 57, OS thread handle 139821399369472, query id 20129 localhost root update
insert into t3(c2) values(15)
*** (2) HOLDS THE LOCK(S):
RECORD LOCKS space id 38 page no 4 n bits 72 index c2 of table `hucq`.`t3` trx id 6135 lock mode S
Record lock, heap no 4 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 80000014; asc ;;
 1: len 4; hex 80000014; asc ;;

Record lock, heap no 5 PHYSICAL RECORD: n_fields 2; compact format; info bits 32
 0: len 4; hex 8000000f; asc ;;
 1: len 4; hex 8000000f; asc ;;

*** (2) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 38 page no 4 n bits 72 index c2 of table `hucq`.`t3` trx id 6135 lock_mode X locks gap before rec insert intention waiting
Record lock, heap no 4 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 4; hex 80000014; asc ;;
 1: len 4; hex 80000014; asc ;;

```

2 INSERT INTENTION LOCK

在之前的死锁分析第四点, 如果不分析插入意向锁, 也是会造成死锁的, 因为插入最终还是要对记录加 X Lock 的, session2 和 session3 还是会互相阻塞互相等待。

但是插入意向锁是客观存在的, 我们可以在官方手册中查到, 不可忽略:

Prior to inserting the row, a type of gap lock called an insert intention gap lock is set. This lock signals the intent to insert in such a way that multiple transactions inserting into the same index gap need not wait for each other if they are not inserting at the same position within the gap.

插入意向锁其实是一种特殊的 gap lock, 但是它不会阻塞其他锁。假设存在值为 4 和 7 的索引记录, 尝试插入值 5 和 6 的两个事务在获取插入行上的排它锁之前使用插入意向锁锁定间隙, 即在 (4, 7) 上加 gap lock, 但是这两个事务不会互相冲突等待。

当插入一条记录时，会去检查当前插入位置的下一条记录上是否存在锁对象，如果下一条记录上存在锁对象，就需要判断该锁对象是否锁住了 gap。如果 gap 被锁住了，则插入意向锁与之冲突，进入等待状态（插入意向锁之间并不互斥）。总结一下这把锁的属性：

1. 它不会阻塞其他任何锁；
2. 它本身仅会被 gap lock 阻塞。

在学习 MySQL 过程中，一般只有在它被阻塞的时候才能观察到，所以这也是它常常被忽略的原因吧...

3 GAP LOCK

在此例中，另外一个重要的点就是 gap lock，通常情况下我们说到 gap lock 都只会联想到 REPEATABLE-READ 隔离级别利用其解决幻读。但实际上在 READ-COMMITTED 隔离级别，也会存在 gap lock，只发生在：唯一约束检查到有唯一冲突的时候，会加 S Next-key Lock，即对记录以及和上一条记录之间的间隙加共享锁。

通过下面这个例子就能验证：

session 1	session 2
begin;	
insert into t3(c2) values(20);ERROR 1062 (23000): Duplicate entry '20' for key 'c2'	
	begin;
	insert into t3(c2) values(18);block

这里 session1 插入数据遇到唯一冲突，虽然报错，但是对 (15, 20] 加的 S Next-Key Lock 并不会马上释放，所以 session2 被阻塞。另外一种情况就是本文开始的例子，当 session2 插入遇到唯一冲突但是是因为被 X Lock 阻塞，并不会立刻报错 “Duplicate key”，但是依然要等待获取 S Next-Key Lock。

有个困惑很久的疑问：出现唯一冲突需要加 S Next-Key Lock 是事实，但是加锁的意义是什么？还是说是通过 S Next-Key Lock 来实现的唯一约束检查，但是这样意味着在插入没有遇到唯一冲突的时候，这个锁会立刻释放，这不符合二阶段锁原则。这点希望能与大家一起讨论得到好的解释。

如果是在 REPEATABLE-READ，除以上所说的唯一约束冲突外，gap lock 的存在是这样的：

普通索引（非唯一索引）的 S/X Lock，都带 gap 属性，会锁住记录以及前 1 条记录到后 1 条记录之间的间隙，比如有 [4, 6, 8] 记录，delete 6 时除了会对 6 记录加 X lock，也会锁住 (4, 6), (6, 8) 这个两个间隙。

对于 gap lock，相信 DBA 们的心情是一样一样的，所以我的建议是：

1. 在绝大部分的业务场景下，都可以把 MySQL 的隔离级别设置为 READ-COMMITTED；
2. 在业务方便控制字段值唯一的情况下，尽量减少表中唯一索引的数量。

4 锁冲突矩阵

前面我们说的 GAP LOCK 其实是锁的属性，另外我们知道 InnoDB 常规锁模式有：S 和 X，即共享锁和排他锁。锁模式和锁属性是可以随意组合的，组合之后的冲突矩阵如下，这对我们分析死锁很有帮助：

行：待加锁 列：存在锁	S(not gap)	S(gap)	S(next-key)	X(not gap)	X(gap)	X(next-key)	insert intention
X(not gap)	冲突		冲突	冲突		冲突	
X(next-key)	冲突		冲突	冲突		冲突	冲突
S(not gap)				冲突		冲突	
S(gap)							冲突
S(next-key)				冲突		冲突	冲突
X(gap)							冲突
insert intention							

5 INSERT 加锁总结

无 Unique Key: X Lock but not gap

有 Unique Key:

1. 唯一性约束检查发生冲突时，会加 S Lock，带 gap 属性，会锁住记录以及与前 1 条记录之前的间隙；
2. 如果插入的位置有带 gap 属性的 S/X Lock，则插入意向锁（LOCK_INSERT_INTENTION）被阻塞，进入等待状态；
3. 如果新数据顺利插入，最后对记录加 X Lock but not gap。

select、delete 加锁行为是很简单的，刚我们看了 insert 的加锁稍有点复杂，那么 update 是怎么加锁的呢？或者更复杂一点的 replace into 呢？欢迎大家一起讨论。



MySQL 优化：为什么 SQL 走索引还那么慢？

作者：王航威

1 背景

这个问题是一个朋友遇到的@风云，并且这位朋友已经得出了近乎正确的判断，下面进行一些描述。
2019-01-11 9:00-10:00 一个 MySQL 数据库把 CPU 打满了。

硬件配置：256G 内存，48core

2 分析过程

接手这个问题时现场已经不在，信息有限，所以我们先从监控系统中查看一下当时的状态。从 PMM 监控来看，这个 MySQL 实例每天上午九点 CPU 都会升高到 10%-20%，只有 1 月 2 号 和 1 月 11 号 CPU 达到 100%，也就是今天的故障。怀疑是业务在九点会有压力下，排查方向是慢查询。

2.1 按执行次数统计 slow log 发现次数最多的一条 sql:

```
mysqldumpslow -s c slow.log>/tmp/slow_report.txt
Count: 3276 Time=21.75s (71261s) Lock=0.00s (1s) Rows=0.9 (2785), xxx
SELECT T.TASK_ID,
T.xx,
T.xx,
...
FROM T_xx_TASK T
WHERE N=N
AND T.STATUS IN (N,N,N)
AND IFNULL(T.MAX_OPEN_TIMES,N) > IFNULL(T.OPEN_TIMES,N)
AND (T.CLOSE_DATE IS NULL OR T.CLOSE_DATE >= SUBDATE(NOW(),INTERVAL 'S' MINUTE))
AND T.REL_DEVTYPE = N
AND T.REL_DEVID = N
AND T.TASK_DATE >= 'S'
AND T.TASK_DATE <= 'S'
ORDER BY TASK_ID DESC
LIMIT N,N
```

2.2 在 slow log 中找到这条查询记录扫描行数：“Rows_examined: 1161559”，看起来是全表扫描，

CPU 升高通常原因就是同时执行大量慢 sql，所以接下来分析这个 sql3.

2.3 因为 T_xxx_TASK 表在现场应急时清理过数据（从 110 万删至 4 万行），所以需要备份恢复该表到故障前。恢复备份后，查看执行计划与执行时间：

```
explain SELECT T.TASK_ID,
T.xx,
...
FROM T_xxx_TASK T
WHERE 1=1
AND T.STATUS IN (1,2,3)
AND IFNULL(T.MAX_OPEN_TIMES,0) > IFNULL(T.OPEN_TIMES,0)
AND (T.CLOSE_DATE IS NULL OR T.CLOSE_DATE >= SUBDATE(NOW(),INTERVAL '10' MINUTE))
AND T.REL_DEVTYPE = 1
AND T.REL_DEVID = 000000025xxx
AND T.TASK_DATE >= '2019-01-11'
AND T.TASK_DATE <= '2019-01-11'
ORDER BY TASK_ID DESC
LIMIT 0,20;
1 row in set (10.37 sec)
```

type	possible_keys	key	key_len	ref	rows	filtered	Extra
index	INDX_BIOM_ELOCK_TASK	INDX_BIOM_ELOCK_TASK3	53	NULL	644	0.01	Using where

执行时间 10s+，查看表索引信息：

```
show index from T_xxx_TASK;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality
t_bioma_elock_task	0	PRIMARY	1	SEQ	A	1161486
t_bioma_elock_task	0	INDX_BIOM_ELOCK_TASK3	1	TASK_ID	A	1162508
t_bioma_elock_task	1	INDX_BIOM_ELOCK_TASK	1	TASK_DATE	A	236
t_bioma_elock_task	1	INDX_BIOM_ELOCK_TASK	2	STATUS	A	904
t_bioma_elock_task	1	INDX_BIOM_ELOCK_TASK2	1	BRANCHID	A	4054
t_bioma_elock_task	1	INDX_BIOM_ELOCK_TASK2	2	TASK_DATE	A	117554

6 rows in set (0.00 sec)

看到这里其实已经可以基本确定是这个 SQL 引起的了，因为执行一次就要 10s+，而且那个时间点会并发下发大量的这个 SQL。但是有一点陷阱藏在这里：

- 1. 执行计划中明明有使用到索引，为什么执行还是这么慢？
- 2. 执行计划中显示扫描行数为 644，为什么 slow log 中显示 100 多万行？
 - a. 我们先看执行计划，选择的索引 “INDX_BIOM_ELOCK_TASK3(TASK_ID)”。结合 sql 来看，因为有 “ORDER BY TASK_ID DESC” 子句，排序通常很慢，如果使用了文件排序性能会更差，优化器选择这个索引避免了排序。

那为什么不选 possible_keys:INDX_BIOM_ELOCK_TASK 呢？原因也很简单，TASK_DATE 字段区分度太低了，走这个索引需要扫描的行数很大，而且还要进行额外的排序，优化器综合判断代价更大，所以就不

选这个索引了。不过如果我们强制选择这个索引（用 `force index` 语法），会看到 SQL 执行速度更快少于 10s，那是因为优化器基于代价的原则并不等价于执行速度的快慢；

b. 再看执行计划中的 `type:index`，“index”代表“全索引扫描”，其实和全表扫描差不多，只是扫描的时候是按照索引次序进行而不是行，主要优点就是避免了排序，但是开销仍然非常大。

Extra:Using where 也意味着扫描完索引后还需要回表进行筛选。一般来说，得保证 `type` 至少达到 `range` 级别，最好能达到 `ref`。在第 2 点中提到的“慢日志记录 `Rows_examined: 1161559`，看起来是全表扫描”，这里更正为“全索引扫描”，扫描行数确实等于表的行数；c. 关于执行计划中：“`rows: 644`”，其实这个只是估算值，并不准确，我们分析慢 SQL 时判断准确的扫描行数应该以 `slow log` 中的 `Rows_examined` 为准。

2.4 优化建议：添加组合索引 `IDX_REL_DEVID_TASK_ID(REL_DEVID,TASK_ID)`

3 优化过程

`TASK_DATE` 字段存在索引，但是选择度很低，优化器不会走这个索引，建议后续可以删除这个索引：

```
select count(*),count(distinct TASK_DATE) from T_BIOMA_ELOCK_TASK;
+-----+-----+
| count(*) | count(distinct TASK_DATE) |
+-----+-----+
| 1161559 | 223 |
+-----+-----+
```

在这个 sql 中 `REL_DEVID` 字段从命名上看选择度较高，通过下面 sql 来检验确实如此：

```
select count(*),count(distinct REL_DEVID) from T_BIOMA_ELOCK_TASK;
+-----+-----+
| count(*) | count(distinct REL_DEVID) |
+-----+-----+
| 1161559 | 62235 |
+-----+-----+
```

由于有排序，所以得把 `task_id` 也加入到新建的索引中，`REL_DEVID,task_id` 组合选择度 100%：

```
select count(*),count(distinct REL_DEVID,task_id) from T_BIOMA_ELOCK_TASK;
+-----+-----+
| count(*) | count(distinct REL_DEVID,task_id) |
+-----+-----+
| 1161559 | 1161559 |
+-----+-----+
```

在测试环境添加 `REL_DEVID, TASK_ID` 组合索引，测试 sql 性能：`alter table T_BIOMA_ELOCK_TASK add index idx_REL_DEVID_TASK_ID(REL_DEVID,TASK_ID);`

添加索引后执行计划：

这里还要注意一点“隐式转换”：REL_DEVID 字段数据类型为 varchar，需要在 sql 中加引号：AND T.REL_DEVID = 000000025xxx >> AND T.REL_DEVID = '000000025xxx'

type	possible_keys	key	key_len	ref	rows	filtered	Extra
ref	INDX_BIOM_ELOCK_TASK,idx_REL_DEVID_TASK_ID	idx_REL_DEVID_TASK_ID	48	const	27	0.19	Using where

执行时间从 10s+ 降到毫秒级别：

1 row in set (0.00 sec)

4 结论

一个典型的 order by 查询的优化，添加更合适的索引可以避免性能问题：执行计划使用索引并不意味着就能执行快。



记一次 MySQL semaphore crash 的分析

作者：洪斌

DBA 应该对 InnoDB: Semaphore wait has lasted > 600 seconds. We intentionally crash the server because it appears to be hung. 一点都不陌生，MySQL 后台线程 `srv_error_monitor_thread` 发现存在阻塞超过 600s 的 latch 锁时，如果连续 10 次检测该锁仍没有释放，就会触发 panic 避免服务持续 hang 下去。

1 发生了什么

版本号：MySQL 5.5.40

1. 日志中持续输出线程等待数据字典锁，位置是 `dict0dict.c line 305`，等待时间超过了 900s。
2. 持有锁的线程是 139998697924352，其十六进制是 7f53fca8a700。

```
--Thread 139998393616128 has waited at dict0dict.c line 305 for 934.00 seconds
the semaphore:
```

```
X-lock on RW-latch at 0x105alb8 created in file dict0dict.c line 748
a writer (thread id 139998697924352) has reserved it in mode exclusive
number of readers 0, waiters flag 1, lock_word: 0
Last time read locked in file dict0dict.c line 302
Last time write locked in file /pb2/build/sb_0-13157587-
1410170252.03/rpm/BUILD/mysql-5.5.40/mysql-
5.5.40/storage/innobase/dict/dict0dict.c line 305
```

上锁线程 139998697924352 的事务信息，查询数据字典表的操作。

```
---TRANSACTION 0, not started updating table statistics:
```

```
MySQL thread id 14075, OS thread handle 0x7f53fca8a700, query id 110414021
21.14.5.139 jzjkusr
SELECT ROUND(SUM(DATA_LENGTH+INDEX_LENGTH+DATA_FREE)/1024/1024/1024) AS
MY_DB_TOTAL_SIZE FROM information_schema.TABLES
```

检查下持锁线程 139998697924352 是否存在其他锁等待。

发现线程 139998697924352，self-lock 在 `btr0sea.c line 1134`，该锁结构和 AHI 相关。

```
--Thread 139998697924352 has waited at btr0sea.c line 1134 for 934.00 seconds
the semaphore:
```

```

X-lock (wait_ex) on RW-latch at 0x1eb06448 created in file btr0sea.c line 178
a writer (thread id 139998697924352) has reserved it in mode wait exclusive
number of readers 1, waiters flag 1, lock_word: ffffffffffffffff
Last time read locked in file btr0sea.c line 1057
Last time write locked in file /pb2/build/sb_0-13157587-
1410170252.03/rpm/BUILD/mysql-5.5.40/mysql-5.5.40/storage/innobase/btr/btr0sea.c
line 1134

```

接下来看下两处锁结构分别在哪个函数内：

1. dict0dict.c line 305 在 dict_table_stats_lock 函数内
2. btr0sea.c line 1134 在 btr_search_drop_page_hash_index 函数内

什么情况会调用这些函数？

1. 启用 innodb_table_monitor，输出日志时调用 dict_table_stats_lock 上 X 锁，本案例未开启。启用 innodb_stats_on_metadata 时，查询数据字典表会触发统计信息的更新操作，会调用 dict_table_stats_lock 上 X 锁。这与持锁线程的事务信息匹配。
2. Adaptive hash index (AHI) 是 InnoDB 用来加速索引页查找的 hash 表结构。当页面访问次数满足一定条件后，这个页面的地址将存入一个 hash 表中，减少 B 树查询的开销。MySQL 5.5 版本 AHI 是由全局锁 btr_search_latch 维护 hash 表修改的一致性。
3. InnoDB buffer pool 状态显示 free buffer 基本保持 0 空闲。InnoDB buffer pool 驱逐数据页时，会调用 btr_search_drop_page_hash_index 函数，从 AHI 中清理该数据页。

----- BUFFER POOL AND MEMORY -----

```

Total memory allocated 17582522368; in additional pool allocated 0
Dictionary memory allocated 4289681
Buffer pool size      1048576
Free buffers          0
Database pages        1040831
Old database pages    384193
Modified db pages     0

```

2 小结

AHI 的全局锁 btr_search_latch 经常会是竞争热点影响性能，5.7 版本后有所改善与 InnoDB buffer 一样做了多实例拆分。本案例在开启 Innodb_stats_on_metadata 参数，查询元数据信息时触发统计信息更新，上锁数据字典，阻塞了大量业务操作，又由于 buffer pool 空间不足，导致表驱逐旧页触发 AHI 的 btr_search_latch 锁竞争，最终导致信号量超时 crash。

3 彩蛋

在动辄几兆的日志中分析 Semaphore crash，寻找锁、线程、事务之间的关系，相当令人抓狂的。借助 sed、awk、grep 三大法宝，虽有效率提升，但仍不够高效。为了偷懒写了一个小程序，帮助 DBA 快速梳理出这些关系。它的用法是这样的：

```
hongbin@MBP ~> mysqldb doctor -f /Users/hongbin/workbench/mysqld_safe.log
```

目标版本，查代码时找对应版本 5.7.16

日志中出现的 semaphore crash 次数和 mysql 启动次数，如果启动次数大于 crash 次数说明可能是正常启动或其他 crash 造成：

```
***** MySQL service start count *****
```

```
MySQL Semaphore crash -> 3 times ["2018-08-13 23:12:18" "2018-08-14 12:13:43"
"2018-08-16 13:42:36"]
```

```
MySQL Service start -> 3 times ["2018-08-13 23:12:59" "2018-08-14 12:15:20"
"2018-08-16 13:46:37"]
```

线程主要在等待哪些 RW-latch，内容包括：锁位置、出现次数、线程 id（出现次数），重点关注出现次数较多的：

```
***** Which thread waited lock *****
```

```
row0purge.cc:861 -> 58 140477266503424:(57) 140617703745280:(1)
gi.cc:14791 -> 1 140477035656960:(1)
trx0undo.ic:171 -> 1 140617682765568:(1)
ha_innodb.cc:14791 -> 620 140617389913856:(58) 140202719565568:(58)
140202716903168:(57) 140477029533440:(56) 140617407219456:(55)
140477035656960:(52) 140477035124480:(29) 140477108467456:(29)
140477025539840:(26) 140477031130880:(25) 140477027669760:(22)
140617634944768:(21) 140617634146048:(21) 140477019948800:(21)
140477026604800:(20) 140477022078720:(18) 140477018883840:(16)
140477038585600:(15) 140477028734720:(10) 140477022877440:(9) 140477034325760:(1)
140477031663360:(1)
srv0srv.cc:1968 -> 208 140477276993280:(185) 140617714235136:(23)
ha_innodb.cc:5510 -> 601 140617398167296:(57) 140617409615616:(55)
140617392043776:(53) 140477110597376:(52) 140617395771136:(50)
140617636275968:(45) 140617632548608:(40) 140617634146048:(33)
140617639675648:(32) 140617397102336:(28) 140617639409408:(23)
140617635743488:(21) 140617637811968:(18) 140617638610688:(16)
140617399232256:(12) 140617638344448:(10) 140617638078208:(10)
140477033793280:(10) 140477029267200:(10) 140617397368576:(9) 140617635211008:(6)
140617393641216:(5) 140617637545728:(3) 140617402693376:(2) 140477037254400:(1)
dict0dict.cc:1239 -> 136 140477122623232:(50) 140617392842496:(35)
```

```
140202726487808:(26) 140477123688192:(12) 140477038851840:(5) 140477030065920:(4)
140617634412288:(4)
row0trunc.cc:1835 -> 1 140477109798656:(1)
```

上述锁被哪些写线程持有 X 锁，重点关注出现次数较多的：

```
***** Which writer threads block at *****

140616681907968 -> 1 trx0undo.ic:171:(1)
140477173069568 -> 243 srv0srv.cc:1968:(185) row0purge.cc:861:(57)
row0trunc.cc:1835:(1)
140617682765568 -> 29 srv0srv.cc:1968:(23) ha_innodb.cc:5510:(5)
row0purge.cc:861:(1)
```

写线程对应的事务信息，也可能存在日志记录没有输出事务信息：

```
***** These writer threads trx state *****

MySQL thread id 83874, OS thread handle 140477173069568, query id 13139674
10.0.1.146 aml deleting from reference tables
```

统计写线程持有 S 锁情况：

```
****These writer threads at last time reads locked ****

140477173069568 -> 243 row0purge.cc:861:(243)
140617682765568 -> 24 row0purge.cc:861:(24)
140616681907968 -> 1 trx0undo.ic:190:(1)
```

统计写线程持有 X 锁情况：

```
****These writer threads at last time write locked ****

140477173069568 -> 243 dict0stats.cc:2366:(243)
140617682765568 -> 24 dict0stats.cc:2366:(24)
140616681907968 -> 1 buf0flu.cc:1198:(1)
```

通过事后日志分析，有可能出现线程的事务信息没有输出到日志中的情况，无法获知事务具体执行了什么操作。应对这种情况，小程序加入了事中采集事务信息。用法是这样的：

```
hongbin@MBP ~> mysqldb -uxxx -pxxx doctor -w
```

它会监视目标 mysql 的错误日志，一旦出现 “a writer (thread id 140616681907968) has reserved it in mode” 关键字就查询 ps 中的事务信息，并保存下来。



GDB 定位 MySQL5.7 特定版本 hang 死的故障分析

#92108

作者：洪斌

1 问题背景

某环境上有一组 Percona MySQL 5.7.23-23 的半同步主从, 我们采用 Prometheus 监控框架, 按其接口规范自研了独立的 exporter 用于监控数据采集, 类似于 mysqld_exporter, 工作方式大致为:

1. 开启一个 http 端口
2. 采集本机 MySQL 状态 (简单的 show 语句)
3. 将状态发布到 http 端口

exporter 工作模式比较简单, 理论上不会对数据库造成严重影响。但在数据库节点上部署启动 exporter 后, 发现数据库的连接状态异常, 对业务产生了一定影响。

1. 检查连接可用性
 - 1.1 通过 TCP 检查连接可用性 -- 新连接无法建立!!
 - 1.2 通过 Socket 检查连接可用性 -- 仍然无法建立连接
2. 已连接的业务报错查询超时
3. 检查 mysql-error.log -- 没有任何内容
4. 检查系统日志 -- 没有任何相关信息
5. 观察系统资源占用, 无异常占用

为何再正常不过的状态采集会导致如此大的问题? 由于日志中已经无法继续判断原因, 所以迅速采集调用栈信息:

```
pstack $(pgrep -xn mysqld)
```

Tips:

如何采集 mysql 调用栈信息:

<https://www.ibm.com/developerworks/cn/linux/l-cn-deadlock/index.html>

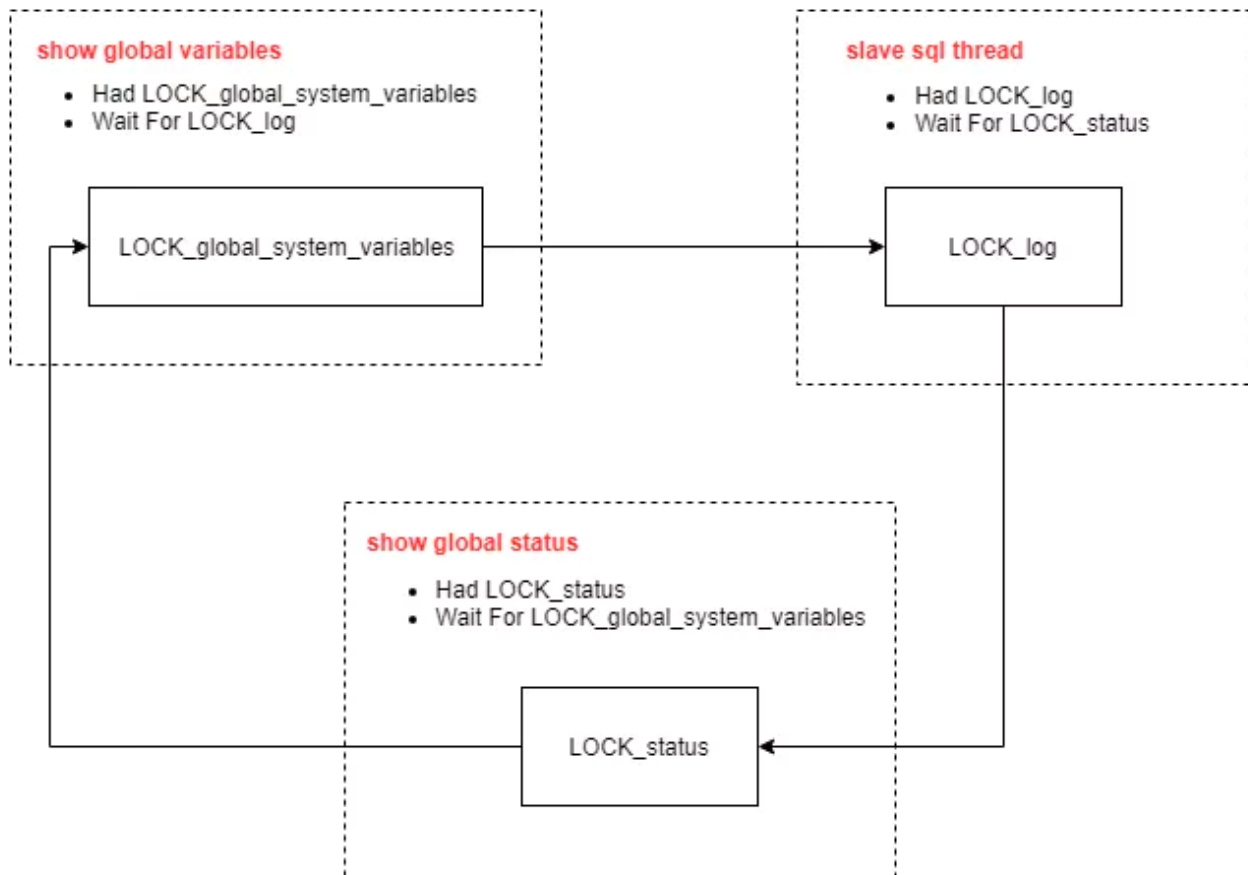
2 问题分析

梳理 exporter 对数据库发起的 SQL 语句:

1. show global variables
2. show global status

3. show master status
4. show slave status

通过 SQL 语句列表和调用栈信息, 大致可以观测出 MySQL 内部出现了 Mutex 死锁, 死锁关系如下:



1. SHOW GLOBAL VARIABLES
持有 LOCK_global_system_variables, 等待 LOCK_log
2. SLAVE SQL THREAD
持有 LOCK_log, 等待 LOCK_status
3. SHOW GLOBAL STATUS
持有 LOCK_status, 等待 LOCK_global_system_variables

我们来看下每部分的调用栈信息:

源码部分基于 Percona MySQL 5.7.23-23

2.1 SHOW GLOBAL VARIABLES

```

Thread 18 (Thread 0x7fad69737700 (LWP 3588)):
#0  0x00007fdd3527642d in __lll_lock_wait () from /lib64/libpthread.so.0
#1  0x00007fdd35271dcb in _L_lock_812 () from /lib64/libpthread.so.0
  
```

```
#2 0x00007fdd35271c98 in pthread_mutex_lock () from /lib64/libpthread.so.0
#3 0x0000000000d60a5e in PolyLock_lock_log::rdlock() ()
#4 0x0000000000c3e4d5 in sys_var::value_ptr(THD*, THD*, enum_var_type,
st_mysql_lex_string*) ()
#5 0x0000000000d1ce70 in get_one_variable_ext(THD*, THD*, st_mysql_show_var
const*, enum_var_type, enum_mysql_show_type, system_status_var*, charset_info_st
const**, char*, unsigned long*) ()
#6 0x0000000000d1cee1 in get_one_variable(THD*, st_mysql_show_var const*,
enum_var_type, enum_mysql_show_type, system_status_var*, charset_info_st const**,
char*, unsigned long*) ()
#7 0x0000000000d2234c in show_status_array(THD*, char const*,
st_mysql_show_var*, enum_var_type, system_status_var*, char const*, TABLE_LIST*,
bool, Item*) ()
#8 0x0000000000d27ec6 in fill_variables(THD*, TABLE_LIST*, Item*) ()
#9 0x0000000000d1550c in do_fill_table(THD*, TABLE_LIST*, QEP_TAB*) ()
#10 0x0000000000d286ac in get_schema_tables_result(JOIN*,
enum_schema_table_state) ()
#11 0x0000000000d0afed in JOIN::prepare_result() ()
#12 0x0000000000c9a2cf in JOIN::exec() ()
#13 0x0000000000d0b93d in handle_query(THD*, LEX*, Query_result*, unsigned long
long, unsigned long long) ()
#14 0x00000000007625d8 in execute_sqlcom_select(THD*, TABLE_LIST*) ()
#15 0x0000000000ccd191 in mysql_execute_command(THD*, bool) ()
#16 0x0000000000cd06f5 in mysql_parse(THD*, Parser_state*) ()
#17 0x0000000000cd12bd in dispatch_command(THD*, COM_DATA const*,
enum_server_command) ()
#18 0x0000000000cd2cff in do_command(THD*) ()
#19 0x0000000000d9c8f8 in handle_connection ()
#20 0x0000000000f18904 in pfs_spawn_thread ()
#21 0x00007fdd3526fe25 in start_thread () from /lib64/libpthread.so.0
#22 0x00007fdd3344434d in clone () from /lib64/libc.so.6
```

关键代码

```
/**sql\set_var.cc:267**/
uchar *sys_var::value_ptr(THD *running_thd, ...)
{
    mysql_mutex_assert_owner(&LOCK_global_system_variables);
}
```

```
/**sql\sys_vars.cc:3788**/
void PolyLock_lock_log::rdlock()
{
    mysql_mutex_lock(mysql_bin_log.get_log_lock());
}
```

2.2 SLAVE SQL THREAD

```
Thread 25 (Thread 0x7fad7c0f5700 (LWP 47842)):
#0  0x00007fdd3527642d in __lll_lock_wait () from /lib64/libpthread.so.0
#1  0x00007fdd35271e01 in _L_lock_1022 () from /lib64/libpthread.so.0
#2  0x00007fdd35271da2 in pthread_mutex_lock () from /lib64/libpthread.so.0
#3  0x00000000000e92d37 in MYSQL_BIN_LOG::publish_coordinates_for_global_status()
const ()
#4  0x00000000000e99791 in MYSQL_BIN_LOG::ordered_commit(THD*, bool, bool) ()
#5  0x00000000000e9ccbfb in MYSQL_BIN_LOG::commit(THD*, bool) ()
#6  0x000000000008084d1 in ha_commit_trans(THD*, bool, bool) ()
#7  0x00000000000d7c9c9 in trans_commit(THD*) ()
#8  0x00000000000e65efb in Xid_log_event::do_commit(THD*) ()
#9  0x00000000000e69b45 in Xid_apply_log_event::do_apply_event(Relay_log_info
const*) ()
#10 0x00000000000e6f5dd in Log_event::apply_event(Relay_log_info*) ()
#11 0x00000000000eb87f6 in apply_event_and_update_pos(Log_event**, THD*,
Relay_log_info*) ()
#12 0x00000000000ec4e90 in handle_slave_sql ()
#13 0x00000000000f18904 in pfs_spawn_thread ()
#14 0x00007fdd3526fe25 in start_thread () from /lib64/libpthread.so.0
#15 0x00007fdd3344434d in clone () from /lib64/libc.so.6
```

关键代码

```
/**sql\binlog.cc:10424*/
void MYSQL_BIN_LOG::publish_coordinates_for_global_status(void) const
{
    mysql_mutex_assert_owner(&LOCK_log);

    mysql_mutex_lock(&LOCK_status);
}
```

2.3 SHOW GLOBAL STATUS

```
Thread 14 (Thread 0x7fad69633700 (LWP 3613)):
#0  0x00007fdd3527642d in __lll_lock_wait () from /lib64/libpthread.so.0
#1  0x00007fdd35271e01 in _L_lock_1022 () from /lib64/libpthread.so.0
#2  0x00007fdd35271da2 in pthread_mutex_lock () from /lib64/libpthread.so.0
#3  0x00000000000d222eb in show_status_array(THD*, char const*,
st_mysql_show_var*, enum_var_type, system_status_var*, char const*, TABLE_LIST*,
bool, Item*) ()
#4  0x00000000000d29860 in fill_status(THD*, TABLE_LIST*, Item*) ()
#5  0x00000000000d1550c in do_fill_table(THD*, TABLE_LIST*, QEP_TAB*) ()
```

```
#6 0x0000000000d286ac in get_schema_tables_result(JOIN*,
enum_schema_table_state) ()
#7 0x0000000000d0afed in JOIN::prepare_result() ()
#8 0x0000000000c9a2cf in JOIN::exec() ()
#9 0x0000000000d0b93d in handle_query(THD*, LEX*, Query_result*, unsigned long
long, unsigned long long) ()
#10 0x00000000007625d8 in execute_sqlcom_select(THD*, TABLE_LIST*) ()
#11 0x0000000000cca8a6 in mysql_execute_command(THD*, bool) ()
#12 0x0000000000cd06f5 in mysql_parse(THD*, Parser_state*) ()
#13 0x0000000000cd12bd in dispatch_command(THD*, COM_DATA const*,
enum_server_command) ()
#14 0x0000000000cd2cff in do_command(THD*) ()
#15 0x0000000000d9c8f8 in handle_connection ()
#16 0x0000000000f18904 in pfs_spawn_thread ()
#17 0x00007fdd3526fe25 in start_thread () from /lib64/libpthread.so.0
#18 0x00007fdd3344434d in clone () from /lib64/libc.so.6
```

关键代码

```
/**sql\sql_show.cc:8020*/
int fill_status(THD *thd, TABLE_LIST *tables, Item *cond)
{
    ...
    mysql_mutex_lock(&LOCK_status);
}

/**sql\sql_show.cc:3061*/
static bool show_status_array(THD *thd, const char *wild, SHOW_VAR
*variables, ...)
{
    ...
    mysql_mutex_lock(&LOCK_global_system_variables);
}
```

2.4 问题 1: 新连接为什么无法建立?

连接请求时的调用栈

```
Thread 6 (Thread 0x7fad6942b700 (LWP 4343)):
#0 0x00007fdd3527642d in __lll_lock_wait () from /lib64/libpthread.so.0
#1 0x00007fdd35271e01 in _L_lock_1022 () from /lib64/libpthread.so.0
#2 0x00007fdd35271da2 in pthread_mutex_lock () from /lib64/libpthread.so.0
#3 0x0000000000c88ff7 in THD::init() ()
#4 0x0000000000c8be9f in THD::THD(bool) ()
#5 0x0000000000e54a1a in Channel_info::create_thd() ()
#6 0x0000000000d9fedc in Channel_info_tcpip_socket::create_thd() ()
#7 0x0000000000d9c7c2 in handle_connection ()
```

```
#8 0x0000000000f18904 in pfs_spawn_thread ()
#9 0x00007fdd3526fe25 in start_thread () from /lib64/libpthread.so.0
#10 0x00007fdd3344434d in clone () from /lib64/libc.so.6
```

连接的初始化需要 LOCK_global_system_variables, 此 mutex 被占用, 导致新连接无法初始化

```
/**sql\sql_class.cc:1629**/
void THD::init(void)
{
    mysql_mutex_lock(&LOCK_global_system_variables);
    ...
    mysql_mutex_unlock(&LOCK_global_system_variables);

    /*
```

新连接请求发起时一直被挂起, 没有任何响应, 现象如下:

```
[root@localhost /]# mysql -uroot -p -S /mysql/data/mysql.sock
Enter password:

^C^C^C^C^C
```

2.5 问题 2:查询操作为什么超时?

查询操作的调用栈

```
Thread 16 (Thread 0x7fad696b5700 (LWP 3597)):
#0 0x00007fdd3527642d in __lll_lock_wait () from /lib64/libpthread.so.0
#1 0x00007fdd35271e01 in _L_lock_1022 () from /lib64/libpthread.so.0
#2 0x00007fdd35271da2 in pthread_mutex_lock () from /lib64/libpthread.so.0
#3 0x0000000000876508 in Item_func_get_system_var::fix_length_and_dec() ()
#4 0x000000000087953d in Item_func::fix_fields(THD*, Item**) ()
#5 0x0000000000c72485 in setup_fields(THD*, Bounds_checked_array<Item*>,
List<Item>&, unsigned long, List<Item>*, bool, bool) ()
#6 0x0000000000d072d5 in st_select_lex::prepare(THD*) ()
#7 0x0000000000d0ba79 in handle_query(THD*, LEX*, Query_result*, unsigned long
long, unsigned long long) ()
#8 0x00000000007625d8 in execute_sqlcom_select(THD*, TABLE_LIST*) ()
#9 0x0000000000ccd191 in mysql_execute_command(THD*, bool) ()
#10 0x0000000000cd06f5 in mysql_parse(THD*, Parser_state*) ()
#11 0x0000000000cd12bd in dispatch_command(THD*, COM_DATA const*,
enum_server_command) ()
#12 0x0000000000cd2cff in do_command(THD*) ()
#13 0x0000000000d9c8f8 in handle_connection ()
#14 0x0000000000f18904 in pfs_spawn_thread ()
```

```
#15 0x00007fdd3526fe25 in start_thread () from /lib64/libpthread.so.0
#16 0x00007fdd3344434d in clone () from /lib64/libc.so.6
```

SQL 查询时也需要 LOCK_global_system_variables, 此 mutex 被占用, 导致查询被阻塞

```
/**sql\item_func.cc:7275**/
void Item_func_get_system_var::fix_length_and_dec()
{
    ...
    mysql_mutex_lock(&LOCK_global_system_variables);
}
```

3 问题结论

经过一轮分析, 我们基本找到问题原因. 但心中还存在疑问:

1. LOCK_global_system_variables 是热点的 mutex, 大概率会出现争用, 但不至于死锁。
2. show global variables 操作为何需要 LOCK_log?
3. 这样常用的查询真的会稳定导致数据库死锁么?

进一步搜索, 发现社区已有类似的缺陷反馈:

<https://bugs.mysql.com/bug.php?id=92108>

该缺陷可理解为:

MySQL 5.7.22 引入了如下两个参数:

1. binlog_transaction_dependency_tracking

https://dev.mysql.com/doc/refman/5.7/en/replication-options-binary-log.html#sysvar_binlog_transaction_dependency_tracking

2. binlog_transaction_dependency_history_size

https://dev.mysql.com/doc/refman/5.7/en/replication-options-binary-log.html#sysvar_binlog_transaction_dependency_history_size

在 5.7.23 版本中, 读取这两个参数需要 LOCK_log. 从而会导致各种各样的死锁(bug 正文中提到了另外一种场景)。

在 5.7.25 版本以后, 这两个参数的读取使用了另外一种影响更低的 lock, 从而避免了本文以及 bug 中提到的死锁。

4 问题处理

对于环境中出现死锁的这一组 Percona MySQL 5.7.23-23 半同步主从, 我们使用 5.7.25-28 版本进行了升级后此问题消失, 至此可以基本确定导致问题的根本原因。

微信扫一扫读原文



delete 大表 slave 回放巨慢的问题分析

作者：洪斌

1 问题

在 master 上执行了一个无 where 条件 delete 操作，该表 50 多万记录。binlog_format 是 mixed 模式，但 transaction_isolation 是 RC 模式，所以 dml 语句会以 row 模式记录。此表没有主键有非唯一索引。在 slave 重放时超过 10 小时没有执行完成。

2 分析

首先了解下 slave 在 row 模式下是如何重放 relay log 的。在 row 模式下，binlog 中会记录 DML 变更操作的事件描述信息、BEFORE IMAGE、AFTER IMAGE。

Header: table id
column information etc.
BEFORE IMAGE AFTER IMAGE
BEFORE IMAGE AFTER IMAGE

DML 事件类型与 image 的关系矩阵

EVENT TYPE	BEFORE IMAGE	AFTER IMAGE
WRITE_ROWS_EVENT	No	Yes
DELETE_ROWS_EVENT	Yes	No
UPDATE_ROWS_EVENT	Yes	Yes

delete 和 update 包含了查找操作，基于 BI 内容搜索找到对应的记录执行相应操作。

基于 row 模式 binlog 的重放主要在此函数中进行 Rows_log_event::do_apply_event，它根据事件类型调用相应的 do_before_row_operations 以 delete 操作为例

Delete_rows_log_event::do_before_row_operations, 此函数会更新 sql command 计数器(com_delete)

接下来调用 Rows_log_event::row_operations_scan_and_key_setup 分配需要的内存空间

Prepare memory structures for search operations. If search is performed:

- 1.using hash search => initialize the hash
- 2.using key => decide on key to use and allocate mem structures
- 3.using table scan => do nothing

选择何种搜索策略取决于 Rows_log_event::decide_row_lookup_algorithm_and_key 的结果, 其决策矩阵依赖表的索引信息和 slave_rows_search_algorithms 参数的设置。

Decision table:

I --> Index scan / search
 T --> Table scan
 H --> Hash scan
 Hi --> Hash over index
 Ht --> Hash over the entire table

Index\Option	I , T , H	I, T	I, H	T, H
PK / UK	I	I	I	Hi
K	Hi	I	Hi	Hi
No Index	Ht	T	Ht	Ht

默认 slave_rows_search_algorithms 是 TABLE_SCAN, INDEX_SCAN, 对应函数

Rows_log_event::do_index_scan_and_update

如果是 INDEX_SCAN, HASH_SCAN, 对应函数 Rows_log_event::do_hash_scan_and_update

在没有主键的情况下, 会遍历 binlog 每行事件, 再用该事件的 BI 去查找对应的记录, 然后变更成对应 AI 信息。

```
for each row in the event do
{
search for the correct row to be modified using BI
replace the row in the table with the corresponding AI
}
```


如果是 HASH SCAN over table, 会先对 binlog 事件中的记录执行 hash, 放到 hash 表中, 再对表中每行记录进行 hash, 与 hash 表中的记录对比, 条件匹配回放 AI 部分。

```
for each row in the event do
{
hash the row.
}
for each row in the table do
{
key= hash the row;
If (key is present in the hash)
{
apply the AI to the row.
}
}
```

如果是 HASH SCAN over index, 在有非唯一索引的情况下, 对 binlog 事件中的记录执行 hash 时, 也会将该记录的 key 保存在一个去重的 key 列表集合中, 然后根据该索引集合去查找记录, 对找到的记录执行 hash 操作并与 hash 表中的记录对比, 如果匹配则回放 AI 部分。

```
for each row in the event do
{
hash the row.
store the key in a list of distinct key.
}
for each row corresponding key values in the key list do
{
key= hash the row;
if (key is present in the hash)
{
apply the AI to the row.
}
}
```

从上述分析可以推测在没有主键的情况下 Hi 的扫描方式会快于 Ht 和 Index scan。

3 测试

对比 slave_rows_search_algorithms 在 TABLE_SCAN, INDEX_SCAN 和 INDEX_SCAN, HASH_SCAN 两种参数设置下, delete 大表哪个效率更高。

```
CREATE TABLE `ants_bnzbw_temp` (  
  `accrued_status` varchar(1) COLLATE utf8_bin DEFAULT NULL,  
  `contract_no` varchar(32) COLLATE utf8_bin DEFAULT NULL,  
  `business_date` date DEFAULT NULL,  
  `prin_bal` int(11) DEFAULT NULL COMMENT,  
  `ovd_prin_bal` int(11) DEFAULT NULL COMMENT ,  
  `ovd_int_bal` int(11) DEFAULT NULL COMMENT ,  
  `int_amt` int(11) DEFAULT NULL COMMENT ,  
  `ovd_prin_pnl_amt` int(11) DEFAULT NULL COMMENT ,  
  `ovd_int_pnl_amt` int(11) DEFAULT NULL COMMENT,  
  KEY `accrued_status` (`accrued_status`) USING BTREE,  
  KEY `contract_no` (`contract_no`) USING BTREE,  
  KEY `business_date` (`business_date`) USING BTREE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin  
  
master [localhost] {msandbox} (test) > select count(*) from ants_bnzbw_temp;  
+-----+  
| count(*) |  
+-----+  
| 522490 |  
+-----+  
1 row in set (0.15 sec)  
  
master [localhost] {msandbox} (test) > delete from ants_bnzbw_temp;  
Query OK, 522490 rows affected (25.86 sec)
```

主机 slave1

slave_rows_search_algorithms='INDEX_SCAN,HASH_SCAN'

事务执行大约 2000s（没有实时追踪事务执行时间）

```
SET @@SESSION.GTID_NEXT= '00020594-1111-1111-1111-111111111111:237'/*!*/;  
# at 221356832  
#180102 14:04:48 server id 1 end_log_pos 221356895 CRC32 0xafdd018f Query  
thread_id=20 exec_time=25 error_code=0  
  
---TRANSACTION 5582, ACTIVE 1447 sec  
mysql tables in use 1, locked 1  
2581 lock struct(s), heap size 319696, 799680 row lock(s), undo log entries  
399840
```

调用栈采样

```
frame #0: 0x000000010f94f331 mysqld`page_rec_get_next_low(unsigned char const*,  
unsigned long) + 81  
frame #1: 0x000000010f94bc28 mysqld`row_search_mvcc(unsigned char*,  
page_cur_mode_t, row_prebuilt_t*, unsigned long, unsigned long) + 9192  
189
```

```

frame #2: 0x0000000010f86d27e mysqld`ha_innobase::index_read(unsigned char*,
unsigned char const*, unsigned int, ha_rkey_function) + 734
frame #3: 0x0000000010f036b6c mysqld`handler::ha_index_read_map(unsigned char*,
unsigned char const*, unsigned long, ha_rkey_function) + 140
frame #4: 0x0000000010f6d5a94 mysqld`Rows_log_event::next_record_scan(bool) + 324
frame #5: 0x0000000010f6d66cf
mysqld`Rows_log_event::do_scan_and_update(Relay_log_info const*) + 159
frame #6: 0x0000000010f6d7198
mysqld`Rows_log_event::do_apply_event(Relay_log_info const*) + 1064
frame #7: 0x0000000010f718d42 mysqld`apply_event_and_update_pos(Log_event**, THD*,
Relay_log_info*) + 530
frame #8: 0x0000000010f711f46 mysqld`handle_slave_sql + 4438

```

主机 slave2

```
slave_rows_search_algorithms='TABLE_SCAN,INDEX_SCAN'
```

事务执行超过 11145s, 还没执行完成

```

---TRANSACTION 4520, ACTIVE 11145 sec
mysql tables in use 1, locked 1
622 lock struct(s), heap size 90320, 191792 row lock(s), undo log entries 95896

```

调用栈采样

```

* frame #0: 0x00000000109fd9c3a mysqld`btr_search_s_lock(dict_index_t const*)
+ 58
  frame #1: 0x00000000109fdb37f
mysqld`btr_search_guess_on_hash(dict_index_t*, btr_search_t*, dtuple_t
const*, unsigned long, unsigned long, btr_cur_t*, unsigned long, mtr_t*) +
479
  frame #2: 0x00000000109fc84a9
mysqld`btr_cur_search_to_nth_level(dict_index_t*, unsigned long, dtuple_t
const*, page_cur_mode_t, unsigned long, btr_cur_t*, unsigned long, char
const*, unsigned long, mtr_t*) + 649
  frame #3: 0x0000000010a177324 mysqld`row_search_on_row_ref(btr_pcur_t*,
unsigned long, dict_table_t const*, dtuple_t const*, mtr_t*) + 164
  frame #4: 0x0000000010a17746f mysqld`row_get_clust_rec(unsigned long,
unsigned char const*, dict_index_t*, dict_index_t**, mtr_t*) + 175
  frame #5: 0x0000000010a1988e5 mysqld`row_vers_impl_x_locked(unsigned char
const*, dict_index_t*, unsigned long const*) + 293
  frame #6: 0x0000000010a0f39db
mysqld`lock_rec_convert_impl_to_expl(buf_block_t const*, unsigned char
const*, dict_index_t*, unsigned long const*) + 603
  frame #7: 0x0000000010a0f4914
mysqld`lock_sec_rec_read_check_and_lock(unsigned long, buf_block_t const*,

```

```
unsigned char const*, dict_index_t*, unsigned long const*, lock_mode,
unsigned long, que_thr_t*) + 596
    frame #8: 0x000000010a1802f1 mysqld`sel_set_rec_lock(btr_pcur_t*,
unsigned char const*, dict_index_t*, unsigned long const*, unsigned long,
unsigned long, que_thr_t*, mtr_t*) + 193
    frame #9: 0x000000010a17e280 mysqld`row_search_mvcc(unsigned char*,
page_cur_mode_t, row_prebuilt_t*, unsigned long, unsigned long) + 6720
    frame #10: 0x000000010a0a027e mysqld`ha_innobase::index_read(unsigned
char*, unsigned char const*, unsigned int, ha_rkey_function) + 734
    frame #11: 0x0000000109869b6c mysqld`handler::ha_index_read_map(unsigned
char*, unsigned char const*, unsigned long, ha_rkey_function) + 140
    frame #12: 0x0000000109f09065
mysqld`Rows_log_event::do_index_scan_and_update(Relay_log_info const*) +
821
    frame #13: 0x0000000109f0a198
mysqld`Rows_log_event::do_apply_event(Relay_log_info const*) + 1064
    frame #14: 0x0000000109f4bd42
mysqld`apply_event_and_update_pos(Log_event**, THD*, Relay_log_info*) + 530
    frame #15: 0x0000000109f44f46 mysqld`handle_slave_sql + 4438
```

4 结论

通过测试发现使用 `slave_rows_search_algorithms=INDEX_SCAN,HASH_SCAN` 配置在此场景下回放 binlog 会有大幅性能改善，这种方式会有一定内存开销，所以要保障内存足够创建 hash 表，才会看到性能提升。

对于此问题的改进建议：

1. 避免无 where 条件的 delete 或 update 操作大表，如果需要全表 delete 请使用 truncate 操作
2. 在 binlog row 模式下表结构最好能有主键
3. 将 `slave_rows_search_algorithms` 设置为 `INDEX_SCAN,HASH_SCAN`，会有一定性能改善。



event_scheduler 导致复制中断的故障分析

作者：洪斌

1 问题背景

在 5.6.29 和 5.7.11 版本之前，binlog 格式为 mixed，event 中包含 sysdate 函数时，会导致复制中断。

测试版本：mysql 5.6.23 重现方法：分别设置 statement、mixed、row 格式执行以下语句，mixed 格式时会导致复制中断，其他两种格式不会造成复制中断。

```
DELIMITER $$
DROP EVENT IF EXISTS `MIS_CUMULATIVE_REV_EVENT`
$$
CREATE EVENT IF NOT EXISTS `MIS_CUMULATIVE_REV_EVENT`
ON SCHEDULE EVERY 3 second STARTS sysdate()
ON COMPLETION PRESERVE
DO BEGIN
DECLARE EXIT HANDLER FOR SQLEXCEPTION
SELECT CONCAT('SQL EXCEPTION - TERMINATING EVENT FOR STORED PROCEDURE CALL');
END
$$
```

1. statement 模式

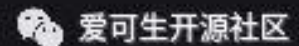
set global binlog_format=statement;

```
master [localhost] {msandbox} (test) > show events\G
***** 1. row *****
      Db: test
      Name: MIS_CUMULATIVE_REV_EVENT
      Definer: msandbox@localhost
      Time zone: SYSTEM
      Type: RECURRING
      Execute at: NULL
      Interval value: 30
      Interval field: MINUTE
      Starts: 2016-05-08 10:43:51
      Ends: NULL
      Status: ENABLED
      Originator: 1
character_set_client: utf8
collation_connection: utf8_general_ci
      Database Collation: latin1_swedish_ci
1 row in set (0.00 sec)
```

设置 5s 延迟观察下 starts 字段时间，复制正常，主从有差异

```
slave2 [localhost] {msandbox} (test) > change master to MASTER_DELAY=5;
```

```
slave2 [localhost] {msandbox} (test) > show events\G
***** 1. row *****
      Db: test
      Name: MIS_CUMULATIVE_REV_EVENT
      Definer: msandbox@localhost
      Time zone: SYSTEM
      Type: RECURRING
      Execute at: NULL
      Interval value: 30
      Interval field: MINUTE
      Starts: 2016-05-08 10:43:56
      Ends: NULL
      Status: SLAVESIDE_DISABLED
      Originator: 1
character_set_client: utf8
collation_connection: utf8_general_ci
      Database Collation: latin1_swedish_ci
1 row in set (0.01 sec)
```



2. Row 模式

```
set global binlog_format=row;
```

```
master [localhost] {msandbox} (test) > show events\G
***** 1. row *****
      Db: test
      Name: MIS_CUMULATIVE_REV_EVENT
      Definer: msandbox@localhost
      Time zone: SYSTEM
      Type: RECURRING
      Execute at: NULL
      Interval value: 30
      Interval field: MINUTE
      Starts: 2016-05-08 10:47:00
      Ends: NULL
      Status: ENABLED
      Originator: 1
character_set_client: utf8
```



复制正常，主从有差异

```

slave2 [localhost] {msandbox} (test) > show events\G
***** 1. row *****
      Db: test
      Name: MIS_CUMULATIVE_REV_EVENT
      Definer: msandbox@localhost
      Time zone: SYSTEM
      Type: RECURRING
      Execute at: NULL
      Interval value: 30
      Interval field: MINUTE
      Starts: 2016-05-08 10:47:05
      Ends: NULL
      Status: SLAVESIDE_DISABLED
      Originator: 1
character_set_client: utf8
collation_connection: utf8_general_ci
      Database Collation: latin1_swedish_ci
1 row in set (0.01 sec)

```



start 字段同样有差异，因为 sysdate 函数是 sql 实际执行时间；如果是 now 函数，start 字段时间会是一样的。

3. mixed 模式

set global binlog_format=mixed; 关闭延迟

```

slave2 [localhost] {msandbox} (test) > show slave status\G
***** 1. row *****
      Slave_IO_State: Waiting for master to send event
      Master_Host: 127.0.0.1
      Master_User: rsandbox
      Master_Port: 20886
      Connect_Retry: 60
      Master_Log_File: mysql-bin.000001
      Read_Master_Log_Pos: 6367
      Relay_Log_File: mysql_sandbox20888-relay-bin.000002
      Relay_Log_Pos: 314
      Relay_Master_Log_File: mysql-bin.000001
      Slave_IO_Running: Yes
      Slave_SQL_Running: No
      Replicate_Do_DB:
      Replicate_Ignore_DB:
      Replicate_Do_Table:
      Replicate_Ignore_Table:
      Replicate_Wild_Do_Table:
      Replicate_Wild_Ignore_Table:
      Last_Errno: 1778
      Last_Error: Error 'Cannot execute statements with implicit commit inside a transaction when @@SES
fault database: 'test'. Query: 'CREATE DEFINER='msandbox'@'localhost' EVENT IF NOT EXISTS `MIS_CUMULATIVE_REV_EVENT`
ON SCHEDULE EVERY 30 MINUTE STARTS sysdate()
ON COMPLETION PRESERVE
DO BEGIN
DECLARE EXIT HANDLER FOR SQLEXCEPTION
SELECT CONCAT('SQL EXCEPTION - TERMINATING EVENT FOR STORED PROCEDURE CALL');
END'

```


195

正常的 statement 和 row 模式

```
# at 151
#160508 9:38:10 server id 1 end_log_pos 199 CRC32 0x9ec9df6d GTID last_committed=0 sequence_number=0
SET @@SESSION.GTID_NEXT= '3109f376-7181-11e4-ac56-40323a8b0ea3:1'/*!*/;
# at 199
#160508 9:38:10 server id 1 end_log_pos 326 CRC32 0x4c447a56 Query thread_id=8 exec_time=0 error_code=0
use `test`/*!*/;
SET TIMESTAMP=1462671490/*!*/;
SET @@session.pseudo_thread_id=8/*!*/;
SET @@session.foreign_key_checks=1, @@session.sql_auto_is_null=0, @@session.unique_checks=1, @@session.autocommit=1/*!*/;
SET @@session.sql_mode=1073741824/*!*/;
SET @@session.auto_increment_increment=1, @@session.auto_increment_offset=1/*!*/;
/*!\\C utf8 *//*!*/;
SET @@session.character_set_client=33,@@session.collation_connection=33,@@session.collation_server=8/*!*/;
SET @@session.lc_time_names=0/*!*/;
SET @@session.collation_database=DEFAULT/*!*/;
DROP EVENT IF EXISTS `MIS_CUMULATIVE_REV_EVENT`
/*!*/;
# at 326
#160508 9:38:13 server id 1 end_log_pos 374 CRC32 0x73db9a8c GTID last_committed=0 sequence_number=0
SET @@SESSION.GTID_NEXT= '3109f376-7181-11e4-ac56-40323a8b0ea3:2'/*!*/; ←
# at 374
#160508 9:38:13 server id 1 end_log_pos 764 CRC32 0x1c5bcb3f Query thread_id=8 exec_time=0 error_code=0
SET TIMESTAMP=1462671493/*!*/;
SET @@session.time_zone='SYSTEM'/*!*/;
CREATE DEFINER=`msandbox`@`localhost` EVENT IF NOT EXISTS `MIS_CUMULATIVE_REV_EVENT`
ON SCHEDULE EVERY 30 MINUTE STARTS sysdate()
ON COMPLETION PRESERVE
DO BEGIN
DECLARE EXIT HANDLER FOR SQLEXCEPTION
SELECT CONCAT('SQL EXCEPTION - TERMINATING EVENT FOR STORED PROCEDURE CALL');
END
```

 爱可生开源社区

从这些截图可知，在 mixed 模式下创建 event 时，在同一个事务内即产生了 DDL statement，也产生了额外写入 mysql.event 表的 row 事件。

对于 DDL 语句在任何 binlog 格式下，都会以 statement 格式记录。

mixed 模式下遇到非复制安全函数会转换成 row 模式。例如：sysdate() 返回实际执行时间而非调用时间与 now 函数有差异。

```
mysql> SELECT NOW(), SLEEP(2), NOW();
+-----+-----+-----+
| NOW()          | SLEEP(2) | NOW()          |
+-----+-----+-----+
| 2006-04-12 13:47:36 |          0 | 2006-04-12 13:47:36 |
+-----+-----+-----+
mysql> SELECT SYSDATE(), SLEEP(2), SYSDATE();
+-----+-----+-----+
| SYSDATE()      | SLEEP(2) | SYSDATE()      |
+-----+-----+-----+
| 2006-04-12 13:47:44 |          0 | 2006-04-12 13:47:46 |
+-----+-----+-----+
```

3 问题分析

利用 systemtap 观察下，create event 时函数调用，定位为何产生了 row 事件。查看源码中有哪些 create_event 函数

```
[root@10-186-21-66 ~]# stap -L
'process("/usr/local/mysql/bin/mysqld").function("create_event") '
process("/usr/local/mysql-advanced-5.6.23-linux-glibc2.5-
x86_64/bin/mysqld").function("create_event@/export/home/pb2/build/sb_0-14249461-
1422537824.58/mysqlcom-pro-5.6.23/sql/event_db_repository.cc:660") $this:class
Event_db_repository* const $thd:struct THD* $parse_data:struct Event_parse_data*
$create_if_not:bool $event_already_exists:bool* $table:struct TABLE* $sp:struct
sp_head* $saved_mode:sql_mode_t $mdl_savepoint:class MDL_savepoint
process("/usr/local/mysql-advanced-5.6.23-linux-glibc2.5-
x86_64/bin/mysqld").function("create_event@/export/home/pb2/build/sb_0-14249461-
1422537824.58/mysqlcom-pro-5.6.23/sql/event_queue.cc:200") $this:class
Event_queue* const $thd:struct THD* $new_element:struct Event_queue_element*
$created:bool* $__FUNCTION__:char[] const
process("/usr/local/mysql-advanced-5.6.23-linux-glibc2.5-
x86_64/bin/mysqld").function("create_event@/export/home/pb2/build/sb_0-14249461-
1422537824.58/mysqlcom-pro-5.6.23/sql/events.cc:307")          $thd:struct          THD*
$parse_data:struct Event_parse_data* $if_not_exists:bool
```

共有以下 3 处

第 1 处:

```
/**
 * Create a new event.
 * @param[in,out] thd          THD
 * @param[in]     parse_data   Event's data from parsing stage
 * @param[in]     if_not_exists Whether IF NOT EXISTS was
 *                             specified
 *
 * In case there is an event with the same name (db) and
 * IF NOT EXISTS is specified, an warning is put into the stack.
 * @sa Events::drop_event for the notes about locking, pre-locking
 * and Events DDL.
 * @retval FALSE OK
 * @retval TRUE  Error (reported)
 */
bool
Events::create_event(THD *thd, Event_parse_data *parse_data,
                    bool if_not_exists)
```

第 2 处:

```
/**
 * Creates an event record in mysql.event table.
 * Creates an event. Relies on mysql_event_fill_row which is shared with
 * ::update_event.
```

```

@pre All semantic checks must be performed outside. This function
only creates a record on disk.
@pre The thread handle has no open tables.
@param[in,out] thd          THD
@param[in]      parse_data   Parsed event definition
@param[in]      create_if_not TRUE if IF NOT EXISTS clause was provided
                                to CREATE EVENT statement
@param[out]     event_already_exists When method is completed successfully
                                set to true if event already exists else
                                set to false

@retval FALSE success
@retval TRUE  error
*/
bool
Event_db_repository::create_event(THD *thd, Event_parse_data *parse_data,
                                bool create_if_not,
                                bool *event_already_exists)

```

第3处:

```

/**
Adds an event to the queue.
Compute the next execution time for an event, and if it is still
active, add it to the queue. Otherwise delete it.
The object is left intact in case of an error. Otherwise
the queue container assumes ownership of it.
@param[in] thd      thread handle
@param[in] new_element a new element to add to the queue
@param[out] created set to TRUE if no error and the element is
                    added to the queue, FALSE otherwise
@retval TRUE an error occurred. The value of created is undefined,
            the element was not deleted.
@retval FALSE success
*/
bool
Event_queue::create_event(THD *thd, Event_queue_element *new_element,
                        bool *created)

```

利用 systemtap 观察以下两个函数调用情况

```

probe process("/usr/local/mysql/bin/mysqld").function("create_event") {
    printf(">>>>>create_event\n");
    print_ubacktrace();
}
probe
process("/usr/local/mysql/bin/mysqld").function("set_current_stmt_binlog_format_

```

```
row") {  
    printf("+++++set_current_stmt_binlog_format_row\n");  
    print_ubacktrace();  
}
```

运行 `stap -v event.stp`, 在另一个 mysql 终端执行创建 event 语句

```
>>>>>create_event  
0x7d36b1 : _ZN6Events12create_eventEP3THDP16Event_parse_datab+0x21/0x340  
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]  
0x727984 : _Z21mysql_execute_commandP3THD+0x5804/0x6ce0 [...local/mysql-  
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]  
0x729178 : _Z11mysql_parseP3THDPcjP12Parser_state+0x318/0x420 [...local/mysql-  
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]  
0x72a4cb : _Z16dispatch_command19enum_server_commandP3THDPcj+0xbcb/0x28f0  
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]  
0x72c2c7 : _Z10do_commandP3THD+0xd7/0x1c0 [...local/mysql-advanced-5.6.23-  
linux-glibc2.5-x86_64/bin/mysqld]  
0x6f3ee6 : _Z24do_handle_one_connectionP3THD+0x116/0x1b0 [...local/mysql-  
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]  
0x6f3fc5 : handle_one_connection+0x45/0x60 [...local/mysql-advanced-5.6.23-  
linux-glibc2.5-x86_64/bin/mysqld]  
0x9b22f6 : pfs_spawn_thread+0x126/0x140 [...local/mysql-advanced-5.6.23-linux-  
glibc2.5-x86_64/bin/mysqld]  
0x7ff7b42859d1 [/lib64/libpthread-2.12.so+0x79d1/0x219000]  
>>>>>create_event  
0x8bae27 :  
_ZN19Event_db_repository12create_eventEP3THDP16Event_parse_databPb+0x17/0x2b0  
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]  
0x7d377f : _ZN6Events12create_eventEP3THDP16Event_parse_datab+0xef/0x340  
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]  
0x727984 : _Z21mysql_execute_commandP3THD+0x5804/0x6ce0 [...local/mysql-  
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]  
0x729178 : _Z11mysql_parseP3THDPcjP12Parser_state+0x318/0x420 [...local/mysql-  
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]  
0x72a4cb : _Z16dispatch_command19enum_server_commandP3THDPcj+0xbcb/0x28f0  
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]  
0x72c2c7 : _Z10do_commandP3THD+0xd7/0x1c0 [...local/mysql-advanced-5.6.23-  
linux-glibc2.5-x86_64/bin/mysqld]  
0x6f3ee6 : _Z24do_handle_one_connectionP3THD+0x116/0x1b0 [...local/mysql-  
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]  
0x6f3fc5 : handle_one_connection+0x45/0x60 [...local/mysql-advanced-5.6.23-  
linux-glibc2.5-x86_64/bin/mysqld]  
0x9b22f6 : pfs_spawn_thread+0x126/0x140 [...local/mysql-advanced-5.6.23-linux-  
glibc2.5-x86_64/bin/mysqld]  
0x7ff7b42859d1 [/lib64/libpthread-2.12.so+0x79d1/0x219000]
```

```

+++++set_current_stmt_binlog_format_row
0x8e6556 : _ZN3THD21decide_logging_formatEP10TABLE_LIST+0x606/0xa60
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x6d4e68 : _Z11lock_tablesP3THDP10TABLE_LISTjj+0xe8/0x860 [...local/mysql-
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x6de9d2 :
_Z20open_and_lock_tablesP3THDP10TABLE_LISTbjP19Prelocking_strategy+0xa2/0xe0
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x8ba6a1 :
_ZN19Event_db_repository16open_event_tableEP3THD13thr_lock_typePP5TABLE+0x101/0x
1d0 [...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x8bae94 :
_ZN19Event_db_repository12create_eventEP3THDP16Event_parse_databPb+0x84/0x2b0
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x7d377f : _ZN6Events12create_eventEP3THDP16Event_parse_datab+0xef/0x340
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x727984 : _Z21mysql_execute_commandP3THD+0x5804/0x6ce0 [...local/mysql-
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x729178 : _Z11mysql_parseP3THDPcjP12Parser_state+0x318/0x420 [...local/mysql-
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x72a4cb : _Z16dispatch_command19enum_server_commandP3THDPcj+0xbcb/0x28f0
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x72c2c7 : _Z10do_commandP3THD+0xd7/0x1c0 [...local/mysql-advanced-5.6.23-
linux-glibc2.5-x86_64/bin/mysqld]
0x6f3ee6 : _Z24do_handle_one_connectionP3THD+0x116/0x1b0 [...local/mysql-
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x6f3fc5 : handle_one_connection+0x45/0x60 [...local/mysql-advanced-5.6.23-
linux-glibc2.5-x86_64/bin/mysqld]
0x9b22f6 : pfs_spawn_thread+0x126/0x140 [...local/mysql-advanced-5.6.23-linux-
glibc2.5-x86_64/bin/mysqld]
0x7ff7b42859d1 [/lib64/libpthread-2.12.so+0x79d1/0x219000]
>>>>>create_event
0x8bc64e :
_ZN11Event_queue12create_eventEP3THDP19Event_queue_elementPb+0x1e/0xe0
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x7d3916 : _ZN6Events12create_eventEP3THDP16Event_parse_datab+0x286/0x340
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x727984 : _Z21mysql_execute_commandP3THD+0x5804/0x6ce0 [...local/mysql-
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x729178 : _Z11mysql_parseP3THDPcjP12Parser_state+0x318/0x420 [...local/mysql-
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x72a4cb : _Z16dispatch_command19enum_server_commandP3THDPcj+0xbcb/0x28f0
[...local/mysql-advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x72c2c7 : _Z10do_commandP3THD+0xd7/0x1c0 [...local/mysql-advanced-5.6.23-
linux-glibc2.5-x86_64/bin/mysqld]
0x6f3ee6 : _Z24do_handle_one_connectionP3THD+0x116/0x1b0 [...local/mysql-

```

```
advanced-5.6.23-linux-glibc2.5-x86_64/bin/mysqld]
0x6f3fc5 : handle_one_connection+0x45/0x60 [...local/mysql-advanced-5.6.23-
linux-glibc2.5-x86_64/bin/mysqld]
0x9b22f6 : pfs_spawn_thread+0x126/0x140 [...local/mysql-advanced-5.6.23-linux-
glibc2.5-x86_64/bin/mysqld]
```

基本可知函数调用顺序 Event::create_event -> Event_db_repository::create_event -> decide_logging_format -> set_current_stmt_binlog_format_row_if_mixed -> Event_queue::create_event

binlog.cc decide_logging_format()

```
if (lex->is_stmt_unsafe() || lex->is_stmt_row_injection()
    || (flags_write_all_set & HA_BINLOG_STMT_CAPABLE) == 0)
{
    /* log in row format! */
    set_current_stmt_binlog_format_row_if_mixed();
}
```

因为 sysdate() 是非安全函数，所以调用了 set_current_stmt_binlog_format_row_if_mixed()。问题应该出在 set_current_stmt_binlog_format_row_if_mixed

```
[root@10-186-21-66 ~]# addr2line -e /usr/local/mysql/bin/mysqld 0x8e6556
/export/home/pb2/build/sb_0-14249461-1422537824.58/mysqlcom-pro-
5.6.23/sql/sql_class.h:3669
```

sql_class.h

```
inline void set_current_stmt_binlog_format_row_if_mixed()
{
    DEBUG_ENTER("set_current_stmt_binlog_format_row_if_mixed");
    /*
        This should only be called from decide_logging_format.
        @todo Once we have ensured this, uncomment the following
        statement, remove the big comment below that, and remove the
        in_sub_stmt==0 condition from the following 'if'.
    */
    /* DEBUG_ASSERT(in_sub_stmt == 0); */
    /*
        If in a stored/function trigger, the caller should already have done the
        change. We test in_sub_stmt to prevent introducing bugs where people
        wouldn't ensure that, and would switch to row-based mode in the middle
        of executing a stored function/trigger (which is too late, see also
        reset_current_stmt_binlog_format_row()); this condition will make their
        tests fail and so force them to propagate the
        lex->binlog_row_based_if_mixed upwards to the caller.
    */
}
```

```

*/
if ((variables.binlog_format == BINLOG_FORMAT_MIXED) &&
    (in_sub_stmt == 0))
    set_current_stmt_binlog_format_row();
DEBUG_VOID_RETURN;
}

```

省略部分代码

```

inline void set_current_stmt_binlog_format_row()
{
    DEBUG_ENTER("set_current_stmt_binlog_format_row");
    current_stmt_binlog_format= BINLOG_FORMAT_ROW;
    DEBUG_VOID_RETURN;
}

```

从上述代码可以看出，此函数判断 binlog 格式是 mixed 时则调用 set_current_stmt_binlog_format_row

修复补丁中，在 create_event 和 update_event 函数中，在调用 Event_db_repository::create_event 前设置 binlog 格式为 statement，再执行 Event_db_repository::create_event 时就不会产生 row 事件了。

```

@@ -308,6 +308,7 @@ Events::create_event(THD *thd, Event_parse_data *parse_data,
{
    bool ret;
    bool save_binlog_row_based, event_already_exists;
+   ulong save_binlog_format= thd->variables.binlog_format;
    DEBUG_ENTER("Events::create_event");
    if (check_if_system_tables_error())
@@ -341,10 +342,15 @@ Events::create_event(THD *thd, Event_parse_data
*parse_data,
    */
    if ((save_binlog_row_based= thd->is_current_stmt_binlog_format_row()))
        thd->clear_current_stmt_binlog_format_row();
+   thd->variables.binlog_format= BINLOG_FORMAT_STMT;
+
    if (lock_object_name(thd, MDL_key::EVENT,
                        parse_data->dbname.str, parse_data->name.str))
-   DEBUG_RETURN(TRUE);
+   {
+       ret= true;
+       goto err;
+   }
    /* On error conditions my_error() is called so no need to handle here */
    if (!(ret= db_repository->create_event(thd, parse_data, if_not_exists,
@@ -399,10 +405,12 @@ Events::create_event(THD *thd, Event_parse_data
*parse_data,
    }

```

```
    }  
  }  
+err:  
    /* Restore the state of binlog format */  
    DEBUG_ASSERT(!thd->is_current_stmt_binlog_format_row());  
    if (save_binlog_row_based)  
        thd->set_current_stmt_binlog_format_row();  
+ thd->variables.binlog_format= save_binlog_format;
```

完整补丁见: `git log -p 112899f406d2f1f838a180669781a8973ef3e343`

4 结论

1. 升级到 5.6.29 版本或改用 row 模式
2. 如果 event 的 dml 语句中出现类似 `sysdate()`，在 mixed 模式下并不会造成数据不一致，会自动转成 row 模式
3. 在主从环境下创建 event_scheduler，slave 默认是禁用状态 `SLAVESIDE_DISABLED`。应考虑主从切换后，原主从的 event_scheduler 状态，避免产生意外行为，比如 new slave 写入数据。



快速定位令人头疼的全局锁

作者：洪斌

1 背景

在用 xtrabackup 等备份工具做备份时会有全局锁，正常情况锁占用时间很短，但偶尔会遇到锁长时间占用导致系统写入阻塞，现象是 show processlist 看到众多会话显示 wait global read lock，那可能对业务影响会很大。而且 show processlist 是无法看到哪个会话持有了全局锁，如果直接杀掉备份进程有可能进程杀掉了，但锁依然没释放，数据库还是无法写入。这时我们需要有快速定位持有全局锁会话的方法，杀掉对应会话数据库就恢复正常了。

通常这种紧急情况发生，需要 DBA 有能力快速恢复业务，如果平时没有储备，现找方法肯定是来不及的，所以我整理了几种方法，在实际故障中帮助我快速的定位到锁会话恢复了业务，非常有效，与大家分享。

2 方法

方法 1：利用 metadata_locks 视图

此方法仅适用于 MySQL 5.7 以上版本，该版本 performance_schema 新增了 metadata_locks，如果上锁前启用了元数据锁的探针（默认是未启用的），可以比较容易的定位全局锁会话。过程如下：

开启元数据锁对应的探针

```
mysql> UPDATE performance_schema.setup_instruments SET ENABLED = 'YES' WHERE
NAME = 'wait/lock/metadata/sql/mdl';
Query OK, 1 row affected (0.04 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

模拟上锁

```
mysql> flush tables with read lock;
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> select * from performance_schema.metadata_locks;
```

```
+-----+-----+-----+-----+-----+-----+
| OBJECT_TYPE | OBJECT_SCHEMA | OBJECT_NAME | OBJECT_INSTANCE_BEGIN |
LOCK_TYPE | LOCK_DURATION | LOCK_STATUS | SOURCE |
OWNER_THREAD_ID | OWNER_EVENT_ID |
+-----+-----+-----+-----+-----+-----+
| GLOBAL | NULL | NULL | 140613033070288 | SHARED
```

```

| EXPLICIT      | GRANTED      | lock.cc:1110      |          268969 |          80 |
| COMMIT       | NULL         | NULL             | 140612979226448 | SHARED
| EXPLICIT      | GRANTED      | lock.cc:1194      |          268969 |          80 |
| GLOBAL        | NULL         | NULL             | 140612981185856 |
INTENTION_EXCLUSIVE | STATEMENT    | PENDING          | sql_base.cc:3189 |
303901 |          665 |
| TABLE        | performance_schema | metadata_locks | 140612983552320 |
SHARED_READ      | TRANSACTION   | GRANTED          | sql_parse.cc:6030 |
268969 |          81 |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+
4 rows in set (0.01 sec)
OBJECT_TYPE=GLOBAL LOCK_TYPE=SHARED 表示全局锁
mysql> select t.processlist_id from performance_schema.threads t join
performance_schema.metadata_locks ml on ml.owner_thread_id = t.thread_id where
ml.object_type='GLOBAL' and ml.lock_type='SHARED';
+-----+
| processlist_id |
+-----+
|          268944 |
+-----+
1 row in set (0.00 sec)

```

定位到锁会话 ID 直接 kill 该会话即可。

方法 2：利用 events_statements_history 视图

此方法适用于 MySQL 5.6 以上版本，启用 performance_schema.eventsstatements_history (5.6 默认未启用，5.7 默认启用)，该表会 SQL 历史记录执行，如果请求太多，会自动清理早期的信息，有可能将上锁会话的信息清理掉。过程如下：

```

mysql> update performance_schema.setup_consumers set enabled = 'YES' where NAME
= 'events_statements_history'
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1 Changed: 0 Warnings: 0
mysql> flush tables with read lock;
Query OK, 0 rows affected (0.00 sec)
mysql> select * from performance_schema.events_statements_history where sql_text
like 'flush tables%'\G
***** 1. row *****
      THREAD_ID: 39
      EVENT_ID: 21
    END_EVENT_ID: 21
    EVENT_NAME: statement/sql/flush
      SOURCE: socket_connection.cc:95

```

```

TIMER_START: 94449505549959000
TIMER_END: 94449505807116000
TIMER_WAIT: 257157000
LOCK_TIME: 0
SQL_TEXT: flush tables with read lock
DIGEST: 03682cc3e0eaed3d95d665c976628d02
DIGEST_TEXT: FLUSH TABLES WITH READ LOCK
...
    NESTING_EVENT_LEVEL: 0
1 row in set (0.00 sec)
mysql> select t.processlist_id from performance_schema.threads t join
performance_schema.events_statements_history h on h.thread_id = t.thread_id
where h.digest_text like 'FLUSH TABLES%';
+-----+
| processlist_id |
+-----+
|          12   |
+-----+
1 row in set (0.01 sec)

```

方法 3: 利用 gdb 工具

如果上述两种都用不了或者没来得及启用, 可以尝试第三种方法。利用 gdb 找到所有线程信息, 查看每个线程中持有全局锁对象, 输出对应的会话 ID, 为了便于快速定位, 我写成了脚本形式。也可以使用 gdb 交互模式, 但 attach mysql 进程后 mysql 会完全 hang 住, 读请求也会受到影响, 不建议使用交互模式。

```

#!/bin/bash
set -v
threads=$(gdb -p $1 -q -batch -ex 'info threads'| awk '/mysql/{print $1}'|grep -
v '*'|sort -nkl)
for i in $threads; do
    echo "##### thread $i #####"
    lock=`gdb -p $1 -q -batch -ex "thread $i" -ex 'p do_command::thd->thread_id' -
ex 'p do_command::thd->global_read_lock'|grep -B3
GRL_ACQUIRED_AND_BLOCKS_COMMIT`
    if [[ $lock =~ 'GRL_ACQUIRED_AND_BLOCKS_COMMIT' ]]; then
        echo "$lock"
        break
    fi
done
# thread_id 变量, 5.6 和 5.7 版本有所不同, 5.6 版本是 thd->thread_id, 5.7 版本是 thd-
>m_thread_id, 这里需要留意下

```

脚本输出

```
##### thread 2 #####  
[Switching to thread 2 (Thread 0x7f610812b700 (LWP 10702))]  
#0 0x00007f6129685f0d in poll () from /lib64/libc.so.6  
$1 = 9 此处就是mysql中的会话ID  
$2 = {static m_active_requests = 1, m_state =  
Global_read_lock::GRL_ACQUIRED_AND_BLOCKS_COMMIT, m_md1_global_shared_lock =  
0x7f60e800cb10, m_md1_blocks_commits_lock = 0x7f60e801c900}
```

但实际环境可能会比较复杂，用 gdb 可能也无法获得你想要的信息，是不是就没辙了。

方法 4: show processlist

如果备份程序使用的特定用户执行备份，如果是 root 用户备份，那 time 值越大的是持锁会话的概率越大，如果业务也用 root 访问，重点是 state 和 info 为空的，这里有个小技巧可以快速筛选，筛选后尝试 kill 对应 ID，再观察是否还有 wait global read lock 状态的会话。

```
mysql> pager awk '/username/{if (length($7) == 4) {print $0}}'|sort -rk6  
mysql> show processlist
```

如果以上方法全部无效，最后释放终极大招...

方法 5: 重启试试!

如果你有更好的方法，可以留言分享。



FTWRL 一个奇怪的堵塞现象和其堵塞总结

作者：任坤

1 背景

MySQL 版本：5.6.29，普通主从
OS：CentOS 6.8

最近一段时间线上某实例频繁报警 CPU 飙高，每次都捕获到同一种 SQL，结构如下：

```
select uid from test_history where cat_id = '99999' and create_time >= '2019-07-12 19:00:00.000' and uid in (.....)
```

其中 uid 一次性会传入上百个。表结构为：

```
Create Table: CREATE TABLE `test_history` (  
  `id` int(11) NOT NULL AUTO_INCREMENT COMMENT '主键 ID',  
  `cat_id` varchar(64) NOT NULL,  
  `uid` varchar(128) NOT NULL,  
  `msg` varchar(64) NOT NULL,  
  `create_time` datetime NOT NULL COMMENT '创建时间',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `idx_cat_uid` (`cat_id`,`uid`,`create_time`),  
  KEY `idx__time` (`create_time`)  
) ENGINE=InnoDB AUTO_INCREMENT=***** DEFAULT CHARSET=utf8
```

SQL 使用到了索引 idx_msg_uid_time，单条执行可以秒级完成，但是并发执行会遭遇执行时间过长（超过 1 个小时）且 CPU 过高的问题。

2 诊断思路

mpstat -P ALL 1，查看 cpu 使用情况，主要消耗在 sys 即 os 系统调用上

03:16:32	PM	CPU	%usr	%nice	%sys	%iwait	%irq	%soft	%steal	%guest	%idle
03:16:33	PM	all	6.26	0.00	31.69	0.00	0.00	0.08	44.87	0.00	17.10
03:16:33	PM	0	7.00	0.00	34.00	0.00	0.00	0.00	39.00	0.00	20.00
03:16:33	PM	1	5.94	0.00	35.64	0.00	0.00	0.00	46.53	0.00	11.88
03:16:33	PM	2	7.92	0.00	32.67	0.00	0.00	0.00	48.51	0.00	10.89
03:16:33	PM	3	3.96	0.00	27.72	0.00	0.00	0.00	43.56	0.00	24.75
03:16:33	PM	4	5.94	0.00	31.68	0.00	0.00	0.00	50.50	0.00	11.88
03:16:33	PM	5	6.06	0.00	34.34	0.00	0.00	0.00	48.48	0.00	11.11
03:16:33	PM	6	5.88	0.00	34.31	0.00	0.00	0.00	47.06	0.00	12.75
03:16:33	PM	7	14.71	0.00	27.45	0.00	0.00	0.00	35.29	0.00	22.55
03:16:33	PM	8	9.00	0.00	36.00	0.00	0.00	0.00	44.00	0.00	11.00
03:16:33	PM	9	3.92	0.00	32.35	0.00	0.00	0.00	48.04	0.00	15.69
03:16:33	PM	10	7.92	0.00	27.72	0.00	0.00	0.00	31.68	0.00	32.67
03:16:33	PM	11	7.00	0.00	31.00	0.00	0.00	0.00	49.00	0.00	13.00
03:16:33	PM	12	3.92	0.00	33.33	0.00	0.00	0.00	49.02	0.00	13.73
03:16:33	PM	13	3.96	0.00	33.66	0.00	0.00	0.00	48.51	0.00	13.86
03:16:33	PM	14	5.94	0.00	38.61	0.00	0.00	0.00	43.56	0.00	11.88
03:16:33	PM	15	4.95	0.00	31.68	0.00	0.00	0.00	52.48	0.00	10.89
03:16:33	PM	16	6.06	0.00	31.31	0.00	0.00	1.01	49.49	0.00	12.12
03:16:33	PM	17	5.88	0.00	19.61	0.00	0.00	0.98	35.29	0.00	38.24
03:16:33	PM	18	5.88	0.00	37.25	0.00	0.00	0.00	42.16	0.00	14.71
03:16:33	PM	19	5.00	0.00	34.00	0.00	0.00	0.00	49.00	0.00	12.00
03:16:33	PM	20	2.02	0.00	19.19	0.00	0.00	0.00	29.29	0.00	49.49
03:16:33	PM	21	9.00	0.00	33.00	0.00	0.00	0.00	29.00	0.00	9.00
03:16:33	PM	22	4.90	0.00	32.35	0.00	0.00	0.00	50.00	0.00	5.00
03:16:33	PM	23	5.94	0.00	33.66	0.00	0.00	0.00	48.51	0.00	11.88

perf top, cpu 主要消耗在 spin lock

Samples: 48M of event 'cpu-clock', Event count (approx.): 295983469369	
Overhead	shared object Symbol
73.34%	[kernel] [k] _spin_lock
2.93%	[kernel] [k] _spin_unlock_irqrestore
1.92%	mysqld [.] rec_get_offsets_func
1.63%	[kernel] [k] finish_task_switch
1.62%	[kernel] [k] __do_softirq
1.53%	mysqld [.] buf_page_get_gen
1.44%	mysqld [.] cmp_dtuple_rec_with_match_low
1.03%	mysqld [.] page_cur_search_with_match
0.72%	mysqld [.] btr_cur_search_to_nth_level
0.67%	mysqld [.] row_search_for_
0.64%	libc-2.12.so [.] _int_malloc
0.58%	mysqld [.] page_check_dir

生成 perf report 查看详细情况

Samples: 8M of event	cpu-clock	Event count (approx.): 2232364000000
children	Self	Command Shared object Symbol
- 81.48%	81.48%	mysqld [kernel.kallsyms] [k] _spin_lock
- _spin_lock		
57.75%	__lll_unlock_wake_private	
42.25%	__lll_lock_wait_private	
- 52.83%	0.08% mysqld libc-2.12.so	[.] __lll_unlock_wake_private
__lll_unlock_wake_private		
- 37.04%	0.15% mysqld libc-2.12.so	[.] __lll_lock_wait_private
__lll_lock_wait_private		
- 3.97%	3.97% mysqld [kernel.kallsyms]	[k] _spin_unlock_irqrestore
- _spin_unlock_irqrestore		
__lll_unlock_wake_private		
- 1.13%	1.12% mysqld	[.] rec_get_offsets_func
rec_get_offsets_func		
- 1.10%	1.09% mysqld mysqld	[.] buf_page_get_gen
buf_page_get_gen		
- 0.89%	0.89% mysqld [kernel.kallsyms]	[k] finish_task_switch
- finish_task_switch		
98.63%	__lll_lock_wait_private	
- 0.67%	pthread_cond_timedwait@@GLIBC_2.3.2	
- srv_purge_coordinator_thread		
start_thread		

CPU 主要消耗在 mutex 争用上，说明有锁热点。

采用 pt-pmp 跟踪 mysqld 执行情况，热点主要集中在 mem heap alloc 和 mem heap free 上。

Pstack 提供更详细的 API 调用栈

```
#0 0x00000003e0caf80cf in __lll_unlock_wake_private () from /lib64/libc.so.6
#1 0x00000003e0ca7cf6a in _L_unlock_5936 () from /lib64/libc.so.6
#2 0x00000003e0ca78bbc in _int_free () from /lib64/libc.so.6
#3 0x000000000097dcb3 in mem_area_free(void*, mem_pool_t*) ()
#4 0x000000000097d2d2 in mem_heap_block_free(mem_block_info_t*,
mem_block_info_t*) ()
#5 0x00000000009e6474 in row_vers_build_for_consistent_read(unsigned char const*,
mtr_t*, dict_index_t*, unsigned long**, read_view_t*, mem_block_info_t**,
mem_block_info_t*, unsigned char**) ()
#6 0x00000000009dce75 in row_search_for_mysql(unsigned char*, unsigned long,
row_prebuilt_t*, unsigned long, unsigned long) ()
#7 0x0000000000939c95 in ha_innobase::index_read(unsigned char*, unsigned char
const*, unsigned int, ha_rkey_function) ()
```

Innodb 在读取数据记录时的 API 路径为

```
row_search_for_mysql --》
row_vers_build_for_consistent_read --》
mem_heap_create_block_func --》
mem_area_alloc --》
malloc --》
_L_unlock_10151 --》
__lll_unlock_wait_private
```

row_vers_build_for_consistent_read 会陷入一个死循环，跳出条件是该条记录不需要快照读或者已经从 undo 中找出对应的快照版本，每次循环都会调用 mem_heap_alloc/free。

而该表的记录更改很频繁，导致其 undo history list 比较长，搜索快照版本的代价更大，就会频繁的申请和释放堆内存。

Linux 原生的内存库函数为 ptmalloc，malloc/free 调用过多时很容易产生锁热点。

当多条 SQL 并发执行时，会最终触发 os 层面的 spinlock，导致上述情形。

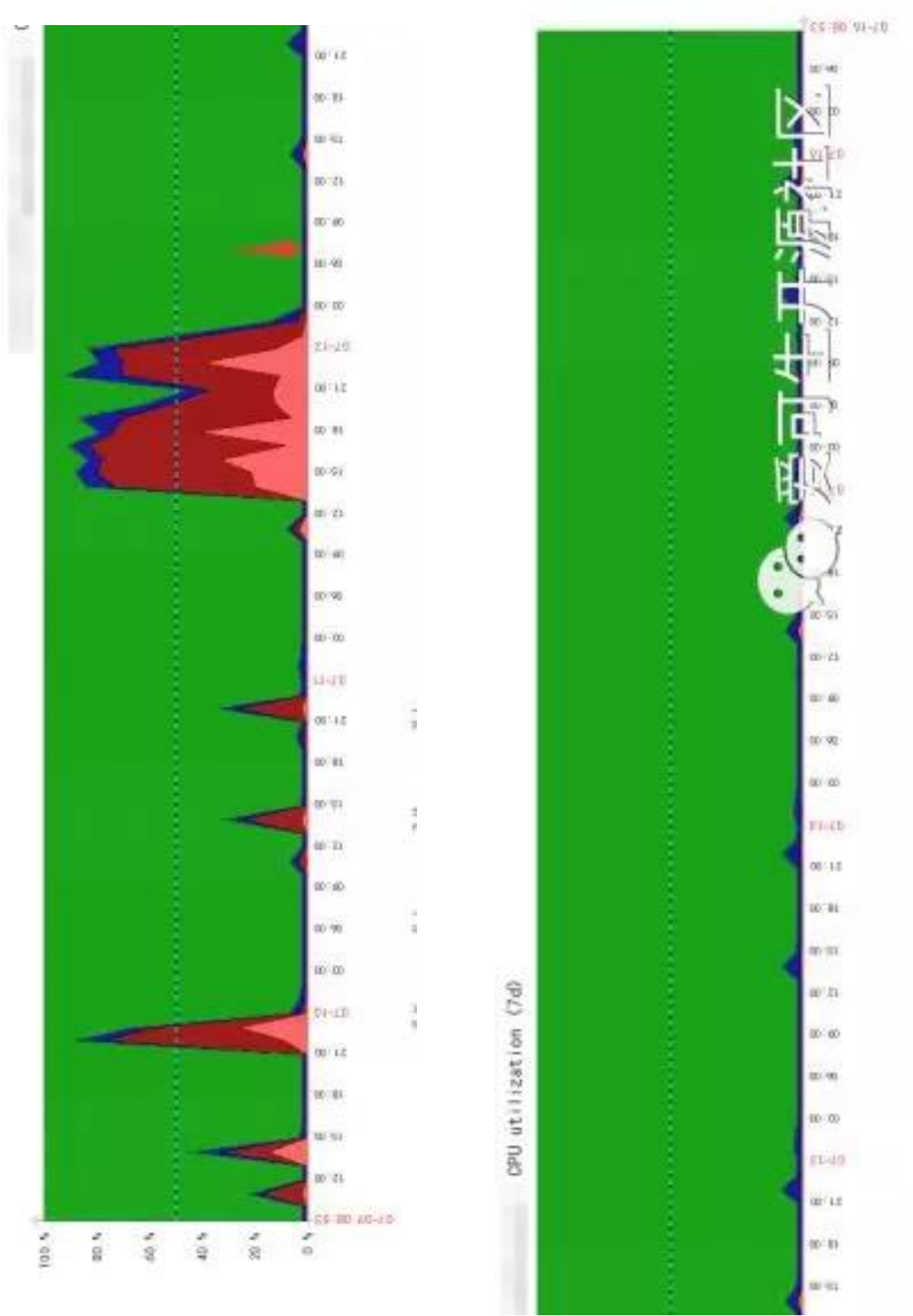
3 解决方案

将 mysqld 的内存库函数替换成 tcmalloc，相比 ptmalloc，tcmalloc 可以更好的支持高并发调用。

修改 my.cnf，添加如下参数并重启

```
[mysqld_safe]
malloc-lib=tcmalloc
```

上周五早上 7 点执行的操作，到现在超过 72 小时，期间该实例没有再出现 cpu 长期飙高的情形。以下是修改前后 cpu 使用率对比：



微信扫一扫读原文



tcmalloc 解决 mysqld 实例引发的 cpu 过高问题

作者：高鹏（八怪）

1 两种不同的现象

首先建立一张有几条数据的表就可以了，我这里是 baguait1 表了。

案例一

SESSION1	SESSION2	SESSION3
步骤1: select sleep(1000) from baguait1 for update;		
	步骤2: flush table with read lock; 堵塞	
		步骤3: kill session2
		步骤4: select * from baguait1 limit 1; 成功

爱可生开源社区

步骤 2 “flush table with read lock;” 操作等待状态为 “Waiting for global read lock”，如下：

```
mysql> select Id,State,Info  from information_schema.processlist where
command<>'sleep';
+----+-----+-----+
| Id | State                | Info                                |
+----+-----+-----+
| 1  | Waiting on empty queue | NULL                               |
| 18 | Waiting for global read lock | flush table with read lock      |
| 3  | User sleep              | select sleep(1000) from baguait1 for update |
| 6  | executing               | select Id,State,Info  from
information_schema.processlist where command<>'sleep' |
+----+-----+-----+
```

案例 2

这里比较奇怪了，实际上我很久以前就遇到过和测试过但是没有仔细研究过，这次刚好详细看看。

SESSION1	SESSION2	SESSION3
步骤1: select sleep(1000) from baguait1		
	步骤2: flush table with read lock;堵塞	
		步骤3: kill session2
		步骤4: select * from baguait1 limit 1;堵塞

 爱可生开源社区

步骤 2 “flush table with read lock;” 操作等待状态为 “Waiting for table flush”，状态如下：

```
mysql> select Id,State,Info from information_schema.processlist where
command<>'sleep';
```

```
+----+-----+-----+
| Id | State                | Info                                |
+----+-----+-----+
| 1  | Waiting on empty queue | NULL                               |
| 26 | User sleep             | select sleep(1000) from baguait1  |
| 23 | Waiting for table flush | flush table with read lock        |
| 6  | executing              | select Id,State,Info from
information_schema.processlist where command<>'sleep' |
+----+-----+-----+
```

步骤 4 “select * from testmts.baguait1 limit 1” 操作等待状态为 “Waiting for table flush”，这个现象看起来非常奇怪没有任何特殊的其他操作，select 居然堵塞了。

```
mysql> select Id,State,Info from information_schema.processlist where
command<>'sleep';
```

```
+----+-----+-----+
| Id | State                | Info                                |
+----+-----+-----+
```

```

-----+
| 1 | Waiting on empty queue | NULL
|
| 26 | User sleep           | select sleep(1000) from baguait1
|
| 27 | executing              | select Id,State,Info  from
information_schema.processlist where command<>'sleep' |
| 6 | Waiting for table flush | select * from testmts.baguait1 limit 1
|
+-----+-----+-----+-----+
-----+

```

如果仔细对比两个案例实际上区别仅仅在于 步骤 1 中的 select 语句是否加了 for update，案例 2 中我们发现即便我们将 “flush table with read lock;” 会话 KILL 掉也会堵塞随后的关于本表上全部操作（包括 select），这个等待实际上会持续到步骤 1 的 sleep 操作完成过后。

对于线上数据库的话，如果在长时间的 select 大表期间执行 “flush table with read lock;” 就会出现这种情况，这种情况会造成全部关于本表的操作等待，即便你发现后杀掉了 FTWRL 会话也无济于事，等待会持续到 select 操作完成后，除非你 KILL 掉长时间的 select 操作。为什么会出现这种情况呢？我们接下来慢慢分析。

2 sleep 函数生效点

关于本案例中我使用 sleep 函数来代替 select 大表操作做为测试，在这里这个代替是成立的。为什么成立呢我们来看一下 sleep 函数的生效点如下：

```

T@3: | | | | | | | | >evaluate_join_record
T@3: | | | | | | | | | enter: join: 0x7ffee0007350 join_tab index: 0 table: tii
cond: 0x0
T@3: | | | | | | | | | counts: evaluate_join_record join->examined_rows++: 1
T@3: | | | | | | | | | >end_send
T@3: | | | | | | | | | | >Query_result_send::send_data
T@3: | | | | | | | | | | | >send_result_set_row
T@3: | | | | | | | | | | | | >THD::enter_cond
T@3: | | | | | | | | | | | | THD::enter_stage: 'User sleep'
/mysqldata/percona-server-locks-detail-5.7.22/sql/item_func.cc:6057
T@3: | | | | | | | | | | | | | >PROFILING::status_change
T@3: | | | | | | | | | | | | | <PROFILING::status_change 384
T@3: | | | | | | | | | | | | | <THD::enter_cond 3405

```

这里看出 sleep 的生效点实际上每次 Innodb 层返回一行数据经过 where 条件判断后，再触发 sleep 函数，也就是每行经过 where 条件过滤的数据在发送给客户端之前都会进行一次 sleep 操作。这个时候实际上该打开表的和该上 MDL LOCK 的都已经完成了，因此使用 sleep 函数来模拟大表 select 操作导致的 FTWRL 堵塞是可以的。

3 FTWRL 做了什么工作

实际上这部分我们可以在函数 `mysql_execute_command` 寻找 case `SQLCOM_FLUSH` 的部分，实际上主要调用函数为 `reload_acl_and_cache`，其中核心部分为：

```
if (thd->global_read_lock.lock_global_read_lock(thd))//加 MDL GLOBAL 级别 S 锁
    return 1;                                // Killed
    if (close_cached_tables(thd, tables, //关闭表操作释放 share 和 cache
        ((options & REFRESH_FAST) ? FALSE : TRUE),
        thd->variables.lock_wait_timeout)) //等待时间受 lock_wait_timeout 影响
    {
        /*
         NOTE: my_error() has been already called by reopen_tables() within
         close_cached_tables().
        */
        result= 1;
    }
    if (thd->global_read_lock.make_global_read_lock_block_commit(thd)) // MDL
    COMMIT 锁
    {
        /* Don't leave things in a half-locked state */
        thd->global_read_lock.unlock_global_read_lock(thd);
        return 1;
    }
```

更具体的关闭表的操作和释放 table 缓存的部分包含在函数 `close_cached_tables` 中，我就不详细写了。但是我们需要明白 table 缓存实际上包含两个部分：

1. table cache define: 每一个表第一次打开的时候都会建立一个静态的表定义结构内存，当多个会话同时访问同一个表的时候，从这里拷贝成相应的 instance 供会话自己使用。由参数 `table_definition_cache` 定义大小，由状态值 `Open_table_definitions` 查看当前使用的个数。对应函数 `get_table_share`。
2. table cache instance: 同上所述，这是会话实际使用的表定义结构是一 instance。由参数 `table_open_cache` 定义大小，由状态值 `Open_tables` 查看当前使用的个数。对应函数 `open_table_from_share`。

这里我统称为 table 缓存，好了下面是我总结的 FTWRL 的大概步骤：

第一步：加 MDL LOCK 类型为 GLOBAL 级别为 S。如果出现等待状态为 `Waiting for global read lock`。注意 select 语句不会上 GLOBAL 级别上锁，但是 DML/DDI/FOR UPDATE 语句会上 GLOBAL 级别的 IX 锁，IX 锁和 S 锁不兼容会出现这种等待。下面是这个兼容矩阵：

	Type of active				
Request	scoped lock				
type	IS(*)	IX	S	X	
-----+-----+-----+-----+-----+					
IS	+	+ +	+ +		
IX	+	+ -	- -		
S	+	- +	- +		
X	+	- -	- -		

- 第二步：推进全局表缓存版本。源码中就是一个全局变量 refresh_version++。
- 第三步：释放没有使用的 table 缓存。可自行参考函数 close_cached_tables 函数。
- 第四步：判断是否有正在占用的 table 缓存，如果有则等待，等待占用者释放。等待状态为 Waiting for table flush。这一步会去判断 table 缓存的版本和全局表缓存版本是否匹配，如果不匹配则等待如下：

```
for (uint idx=0 ; idx < table_def_cache.records ; idx++)
{
    share= (TABLE_SHARE*) my_hash_element(&table_def_cache, idx); //寻找整个
table cache shared hash 结构
    if (share->has_old_version()) //如果版本 和 当前 的 refresh_version 版本不一致
    {
        found= TRUE;
        break; //跳出第一层查找 是否有老版本 存在
    }
}
...
if (found) //如果找到老版本，需要等待
{
    /*
    The method below temporarily unlocks LOCK_open and frees
    share's memory.
    */
    if (share->wait_for_old_version(thd, &abstime,
                                MDL_wait_for_subgraph::DEADLOCK_WEIGHT_DDL))
    {
        mysql_mutex_unlock(&LOCK_open);
        result= TRUE;
        goto err_with_reopen;
    }
}
```

而等待的结束就是占用的 table 缓存的占用者释放，这个释放操作存在于函数 close_thread_table 中，如下：

```
if (table->s->has_old_version() || table->needs_reopen() ||
```

```
    table_def_shutdown_in_progress)
{
    tc->remove_table(table); //关闭 table cache instance
    mysql_mutex_lock(&LOCK_open);
    intern_close_table(table); //去掉 table cache define
    mysql_mutex_unlock(&LOCK_open);
}
```

最终会调用函数 MDL_wait::set_status 将 FTWRL 唤醒，也就是说对于正在占用的 table 缓存而言，释放者不是 FTWRL 会话线程而是占用者自己的会话线程。不管怎么样最终整个 table 缓存将会被清空，如果经过 FTWRL 后去查看 Open_table_definitions 和 Open_tables 将会发现重新计数了。下面是唤醒函数的代码，也很明显：

```
bool MDL_wait::set_status(enum_wait_status status_arg) open_table
{
    bool was_occupied= TRUE;
    mysql_mutex_lock(&m_LOCK_wait_status);
    if (m_wait_status == EMPTY)
    {
        was_occupied= FALSE;
        m_wait_status= status_arg;
        mysql_cond_signal(&m_COND_wait_status); //唤醒
    }
    mysql_mutex_unlock(&m_LOCK_wait_status); //解锁
    return was_occupied;
}
```

第五步：加 MDL LOCK 类型 COMMIT 级别为 S。如果出现等待状态为 `Waiting for commit lock`。如果有大事务的提交很可能出现这种等待。

4 案例 1 解析

步骤 1：我们使用 select for update 语句，这个语句会加 GLOBAL 级别的 IX 锁，持续到语句结束（注意实际上还会加对象级别的 MDL_SHARED_WRITE(SW)锁持续到事务结束，和 FTWRL 无关不做描述）。

步骤 2：我们使用 FTWRL 语句，根据上面的分析需要获取 GLOBAL 级别的 S 锁，不兼容，因此出现了等待 Waiting for global read lock。

步骤 3：我们 KILL 掉了 FTWRL 会话，这种情况下会话退出，FTWRL 就像没有执行过一样不会有任何影响，因为它在第一步就堵塞了。

步骤 4：我们的 select 操作不会受到任何影响，因为在 GLOBAL 级别 select 不会加 MDL LOCK，对象级别 MDL LOCK select/select for update 是兼容的（即 MDL_SHARED_READ(SR)和 MDL_SHARED_WRITE(SW)兼容），且 FTWRL 还没有执行实际的操作。

5 案例 2 解析

步骤 1: 我们使用 select 语句, 这个语句不会在 GLOBAL 级别上任何的锁 (注意实际上还会加对象级别的 MDL_SHARED_READ(SR) 锁持续到事务结束, 和 FTWRL 无关不做描述)。

步骤 2: 我们使用 FTWRL 语句, 根据上面的分析我们发现 FTWRL 语句可以获取了 GLOBAL 级别的 S 锁, 因为单纯的 select 语句不会在 GLOBAL 级别上任何锁。同时会将全局表缓存版本推进然后释放掉没有使用的 table 缓存。但是在第三节的第四步中我们发现因为 baguait1 的表缓存正在被占用, 因此出现了等待, 等待状态为 `Waiting for table flush`。

步骤 3: 我们 KILL 掉了 FTWRL 会话, 这种情况下虽然 GLOBAL 级别的 S 锁会释放, 但是全局表缓存版本已经推进了, 同时没有使用的 table 缓存已经释放掉了。

步骤 4: 再次执行一个 baguait1 表上的 select 查询操作, 这个时候在打开表的时候会去判断是否 table 缓存的版本和全局表缓存版本匹配如果不匹配进入等待, 等待为 `Waiting for table flush`, 下面是这个判断:

```
if (share->has_old_version())
{
    /*
     * We already have an MDL lock. But we have encountered an old
     * version of table in the table definition cache which is possible
     * when someone changes the table version directly in the cache
     * without acquiring a metadata lock (e.g. this can happen during
     * "rolling" FLUSH TABLE(S)).
     * Release our reference to share, wait until old version of
     * share goes away and then try to get new version of table share.
     */
    release_table_share(share);
    ...
    wait_result= tdc_wait_for_old_version(thd, table_list->db,
                                         table_list->table_name,
                                         ot_ctx->get_timeout(),
                                         deadlock_weight);
}
```

整个等待操作和 FTWRL 一样, 会等待占用者释放 table 缓存后才会醒来继续。因此后续本表的所有 select/DML/DDL 都会堵塞, 代价极高, 即便 KILL 掉 FTWR 会话也无用。

6 FTWRL 堵塞和被堵塞的简单总结

6.1 被什么堵塞

1. 长时间的 DDL\DML\FOR UPDATE 堵塞 FTWRL, 因为 FTWRL 需要获取 GLOBAL 的 S 锁, 而这些语句都会对 GLOBAL 持有 IX (MDL_INTENTION_EXCLUSIVE) 锁, 根据兼容矩阵不兼容。等待为:

Waiting for global read lock 。本文的案例 1 就是这种情况。

2. 长时间的 select 堵塞 FTWRL ， 因为 FTWRL 会释放所有空闲的 table 缓存，如果有占用者占用某些 table 缓存，则会等待占用者自己释放这些 table 缓存。等待为：Waiting for table flush 。本文的案例 2 就是这种情况，会堵塞随后关于本表的任何语句，即便 KILL FTWRL 会话也不行，除非 KILL 掉长时间的 select 操作才行。实际上 flush table 也会存在这种堵塞情况。
3. 长时间的 commit(如大事务提交)也会堵塞 FTWRL ， 因为 FTWRL 需要获取 COMMIT 的 S 锁，而 commit 语句会对 commit 持有 IX (MDL_INTENTION_EXCLUSIVE) 锁，根据兼容矩阵不兼容。等待为 Waiting for commit lock 。

6.2 堵塞什么

1. FTWRL 会堵塞 DDL\DDL\FOR UPDATE 操作，堵塞点为 GLOBAL 级别的 S 锁，等待为：Waiting for global read lock 。
2. FTWRL 会堵塞 commit 操作，堵塞点为 COMMIT 的 S 锁，等待为 Waiting for commit lock 。
3. FTWRL 不会堵塞 select 操作，因为 select 不会在 GLOBAL 级别上锁。

最后提醒一下很多备份工具都要执行 FTWRL 操作，一定要注意它的堵塞/被堵塞场景和特殊场景。

6.3 备注栈帧和断点：

(1) 使用的断点

- ☐ MDL_context::acquire_lock 获取 DML LOCK
- ☐ open_table_from_share 获取 table cache instance
- ☐ alloc_table_share 分配 table define (share)
- ☐ get_table_share 获取 table define (share)
- ☐ close_cached_tables flush table 关闭全部 table cache instance 和 table define
- ☐ reload_acl_and_cache flush with read lock 进行 MDL LOCK 加锁为 GLOBAL TYPE:S ,同时调用 close_cached_tables 同时获取 COMMIT 级别 TYPE S
- ☐ MDL_wait::set_status 唤醒操作
- ☐ close_thread_table 占用者判断释放
- ☐ my_hash_delete hash 删除操作，从 table cache instance 和 table define 中释放 table 缓存都是需要调用这个删除操作的。

(2) FTWRL 堵塞栈帧由 select 堵塞栈帧：

```
(gdb) bt
#0  0x00007ffff7bd3a5e in pthread_cond_timedwait@@GLIBC_2.3.2 () from
/lib64/libpthread.so.0
#1  0x000000000192027b in native_cond_timedwait (cond=0x7ffedc007c78,
mutex=0x7ffedc007c30, abstime=0x7fffec5bbb90)
    at /mysqldata/percona-server-locks-detail-5.7.22/include/thr_cond.h:129
#2  0x00000000019205ea in safe_cond_timedwait (cond=0x7ffedc007c78,
```



```

mp=0x7ffedc007c08, abstime=0x7fffec5bbb90,
    file=0x204cdd0 "/mysqldata/percona-server-locks-detail-5.7.22/sql/mdl.cc",
line=1899) at /mysqldata/percona-server-locks-detail-5.7.22/mysys/thr_cond.c:88
#3 0x00000000014b9f21 in my_cond_timedwait (cond=0x7ffedc007c78,
mp=0x7ffedc007c08, abstime=0x7fffec5bbb90,
    file=0x204cdd0 "/mysqldata/percona-server-locks-detail-5.7.22/sql/mdl.cc",
line=1899) at /mysqldata/percona-server-locks-detail-
5.7.22/include/thr_cond.h:180
#4 0x00000000014ba484 in inline_mysql_cond_timedwait (that=0x7ffedc007c78,
mutex=0x7ffedc007c08, abstime=0x7fffec5bbb90,
    src_file=0x204cdd0 "/mysqldata/percona-server-locks-detail-
5.7.22/sql/mdl.cc", src_line=1899)
    at /mysqldata/percona-server-locks-detail-
5.7.22/include/mysql/psi/mysql_thread.h:1229
#5 0x00000000014bb702 in MDL_wait::timed_wait (this=0x7ffedc007c08,
owner=0x7ffedc007b70, abs_timeout=0x7fffec5bbb90, set_status_on_timeout=true,
    wait_state_name=0x2d897b0) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/mdl.cc:1899
#6 0x00000000016cdb30 in TABLE_SHARE::wait_for_old_version (this=0x7ffee0a4fc30,
thd=0x7ffedc007b70, abstime=0x7fffec5bbb90, deadlock_weight=100)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/table.cc:4717
#7 0x000000000153829b in close_cached_tables (thd=0x7ffedc007b70, tables=0x0,
wait_for_refresh=true, timeout=31536000)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_base.cc:1291
#8 0x00000000016123ec in reload_acl_and_cache (thd=0x7ffedc007b70,
options=16388, tables=0x0, write_to_binlog=0x7fffec5bc9dc)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_reload.cc:224
#9 0x00000000015cee9c in mysql_execute_command (thd=0x7ffedc007b70,
first_level=true) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_parse.cc:4433
#10 0x00000000015d2fde in mysql_parse (thd=0x7ffedc007b70,
parser_state=0x7fffec5bd600) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_parse.cc:5901
#11 0x00000000015c6b72 in dispatch_command (thd=0x7ffedc007b70,
com_data=0x7fffec5bdd70, command=COM_QUERY)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_parse.cc:1490

```

(3) 杀点 FTWRL 会话后其他 select 操作等待栈帧:

```

#0 MDL_wait::timed_wait (this=0x7ffee8008298, owner=0x7ffee8008200,
abs_timeout=0x7fffec58a600, set_status_on_timeout=true,
wait_state_name=0x2d897b0)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/mdl.cc:1888
#1 0x00000000016cdb30 in TABLE_SHARE::wait_for_old_version (this=0x7ffee0011620,
thd=0x7ffee8008200, abstime=0x7fffec58a600, deadlock_weight=0)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/table.cc:4717

```

```
#2 0x000000000153b6ba in tdc_wait_for_old_version (thd=0x7ffee8008200,
db=0x7ffee80014a0 "testmts", table_name=0x7ffee80014a8 "tii",
wait_timeout=31536000,
    deadlock_weight=0) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_base.cc:2957
#3 0x000000000153ca97 in open_table (thd=0x7ffee8008200,
table_list=0x7ffee8001708, ot_ctx=0x7fffec58aab0)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_base.cc:3548
#4 0x000000000153f904 in open_and_process_table (thd=0x7ffee8008200,
lex=0x7ffee800a830, tables=0x7ffee8001708, counter=0x7ffee800a8f0, flags=0,
    prelocking_strategy=0x7fffec58abe0, has_prelocking_list=false,
ot_ctx=0x7fffec58aab0) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_base.cc:5213
#5 0x0000000001540a58 in open_tables (thd=0x7ffee8008200, start=0x7fffec58aba0,
counter=0x7ffee800a8f0, flags=0, prelocking_strategy=0x7fffec58abe0)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_base.cc:5831
#6 0x0000000001541e93 in open_tables_for_query (thd=0x7ffee8008200,
tables=0x7ffee8001708, flags=0)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_base.cc:6606
#7 0x00000000015d1dca in execute_sqlcom_select (thd=0x7ffee8008200,
all_tables=0x7ffee8001708) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_parse.cc:5416
#8 0x00000000015ca380 in mysql_execute_command (thd=0x7ffee8008200,
first_level=true) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_parse.cc:2939
#9 0x00000000015d2fde in mysql_parse (thd=0x7ffee8008200,
parser_state=0x7fffec58c600) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_parse.cc:5901
#10 0x00000000015c6b72 in dispatch_command (thd=0x7ffee8008200,
com_data=0x7fffec58cd70, command=COM_QUERY)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_parse.cc:1490
```

(4) 占用者释放唤醒 FTWRL 栈帧:

```
Breakpoint 3, MDL_wait::set_status (this=0x7ffedc000c78,
status_arg=MDL_wait::GRANTED) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/mdl.cc:1832
1832     bool was_occupied= TRUE;
(gdb) bt
#0 MDL_wait::set_status (this=0x7ffedc000c78, status_arg=MDL_wait::GRANTED) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/mdl.cc:1832
#1 0x00000000016c2483 in free_table_share (share=0x7ffee0011620) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/table.cc:607
#2 0x0000000001536a22 in table_def_free_entry (share=0x7ffee0011620) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/sql_base.cc:524
#3 0x00000000018fd7aa in my_hash_delete (hash=0x2e4cfe0, record=0x7ffee0011620
```

```
"\002") at /mysqldata/percona-server-locks-detail-5.7.22/mysys/hash.c:625
#4 0x0000000001537673 in release_table_share (share=0x7ffee0011620) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/sql_base.cc:949
#5 0x00000000016cad10 in closefrm (table=0x7ffee000f280, free_share=true) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/table.cc:3597
#6 0x0000000001537d0e in intern_close_table (table=0x7ffee000f280) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/sql_base.cc:1109
#7 0x0000000001539054 in close_thread_table (thd=0x7ffee0000c00,
table_ptr=0x7ffee0000c68) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_base.cc:1780
#8 0x00000000015385fe in close_open_tables (thd=0x7ffee0000c00) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/sql_base.cc:1443
#9 0x0000000001538d4a in close_thread_tables (thd=0x7ffee0000c00) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/sql_base.cc:1722
#10 0x00000000015d19bc in mysql_execute_command (thd=0x7ffee0000c00,
first_level=true) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_parse.cc:5307
#11 0x00000000015d2fde in mysql_parse (thd=0x7ffee0000c00,
parser_state=0x7fffec5ee600) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_parse.cc:5901
#12 0x00000000015c6b72 in dispatch_command (thd=0x7ffee0000c00,
com_data=0x7fffec5eed70, command=COM_QUERY)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_parse.cc:1490
```



MySQL 一次奇怪的故障分析

作者：高鹏（八怪）

1 问题来源

这是一个朋友问我的典型案例。整个故障现象表现为，MySQL 数据库频繁的出现大量的请求不能响应。下面是一些他提供的证据：

1.1 show processlist

从状态信息来看出现如下情况：

1. insert 操作：状态为 update
2. update/delete 操作：状态为 updating
3. select 操作：状态为 sending data

因此可以推断应该是语句执行期间出现了问题，由于篇幅原因只给出一部分，并且我将语句部分也做了相应截断：

```
show processlist-----
.....
11827639    root    dbmis    Execute 9    updating    UPDATE
17224594    root    dbmis    Execute 8    Sending data    SELECT sum(exchange_coin)
as exchange_coin FROM
17224595    root    dbmis    Execute 8    update    INSERT INTO
17224596    root    dg      Execute 8    update    INSERT INTO
17224597    root    dbmis    Execute 8    update    INSERT INTO
17224598    root    dbmis    Execute 7    update    INSERT INTO
17224599    root    dbmis    Execute 7    Sending data    SELECT COUNT(*) AS tp_count
FROM
17224600    root    dg      Execute 7    update    INSERT INTO
17224601    root    dbmis    Execute 6    update    INSERT INTO
17224602    root    dbmis    Execute 6    Sending data    SELECT sum(exchange_coin)
as exchange_coin FROM
17224606    root    dbmis    Execute 5    update    INSERT INTO
17224619    root    dbmis    Execute 2    update    INSERT INTO
17224620    root    dbmis    Execute 2    update    INSERT INTO
17224621    root    dbmis    Execute 2    Sending data    SELECT sum(exchange_coin)
as exchange_coin
17224622    root    dg      Execute 2    update    INSERT INTO
17224623    root    dbmis    Execute 1    update    INSERT INTO
17224624    root    dbmis    Execute 1    update    INSERT INTO
```

```
17224625    root    dg    Execute 1    update    INSERT INTO
17224626    root    dbmis  Execute 0    update    INSERT INTO
```

1.2 系统 IO/CPU

从 vmstat 来看，CPU 使用不大，而 IO 也在可以接受的范围内（vmstat wa%不高且 b 列为 0）如下：

```
vmstat-----
procs -----memory----- --swap-- -----io---- -system-- -----cpu-----
 r b  swpd  free  buff  cache  si  so  bi  bo  in  cs us sy id wa st
 2 0  927300 3057100      0 53487316    0  0   5 192    0  0 3 1 96 0 0
iostat-----
Linux 3.10.0-693.el7.x86_64 (fang-data1)      09/23/2019  _x86_64_      (32 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           2.72    0.00    0.52    0.45    0.00   96.31

Device:            rrqm/s   wrqm/s     r/s     w/s    rkB/s    wkB/s avgrq-sz avgqu-sz
await r_await w_await  svctm  %util
sdb                9.73    11.28     3.93  264.54   415.23   2624.20    22.64    0.25
0.93    3.25    0.90    0.80  21.61
sda                10.13    11.59     6.34  264.22   450.68   2624.20    22.73    0.01
0.05    2.55    1.00    0.93  25.19
sdc                11.60    11.36     5.03  263.12   453.02   2592.44    22.71    0.17
0.62    5.08    0.53    0.81  21.60
sde                 0.01     0.10     0.11  160.45     6.69    920.23    11.55    0.16
1.01    1.80    1.01    0.83  13.32
sdd                11.26    11.30     2.23  263.18   412.90   2592.44    22.65    0.17
0.65   10.37    0.56    0.82  21.78
md126              0.00     0.00    11.30  468.80   164.79   5216.64    22.42    0.00
0.00    0.00    0.00    0.00    0.00
dm-0               0.00     0.00     0.11   58.80     6.69    920.23    31.47    0.15
2.56    1.96    2.56    2.16  12.74
dm-1               0.00     0.00     0.06    0.08     0.24     0.31     8.00    0.01
41.80    1.20   72.78    0.83    0.01
dm-2               0.00     0.00    11.24  408.66   164.55   5216.33    25.63    0.14
0.32    1.02    0.30    0.46  19.29
```

这就比较奇怪了，一般来说数据库不能及时响应请求很大可能是由于系统负载过高。如果说 DML 还可能是 Innodb 锁造成的堵塞，但是大量 sending data 状态下的 select 操作一般可能都和系统负载过高有联系，但是这里系统负载还在可以接受的范围内。

2 pstack 分析

借助 pstack 查看线程的栈帧，查看 pstack 发现如下（由于篇幅限制只给出部分说明问题的部分）：

2.1 insert 线程:

```
Thread 85 (Thread 0x7fbb0d42b700 (LWP 20174)):  
#0 0x00007fbfae164c73 in select () from /lib64/libc.so.6  
#1 0x0000000000987c0f in os_thread_sleep (tm=<optimized out>) at  
/home/install/lnmp1.5/src/mysql-5.6.40/storage/innobase/os/os0thread.cc:287  
#2 0x00000000009e4dea in srv_conc_enter_innodb_with_atomics  
(trx=trx@entry=0x7fba4802f9c8) at /home/install/lnmp1.5/src/mysql-  
5.6.40/storage/innobase/srv/srv0conc.cc:276  
#3 srv_conc_enter_innodb (trx=trx@entry=0x7fba4802f9c8) at  
/home/install/lnmp1.5/src/mysql-5.6.40/storage/innobase/srv/srv0conc.cc:511  
#4 0x000000000093b948 in innobase_srv_conc_enter_innodb (trx=0x7fba4802f9c8) at  
/home/install/lnmp1.5/src/mysql-  
5.6.40/storage/innobase/handler/ha_innodb.cc:1280  
#5 ha_innodb::write_row (this=0x7fb8440ab260, record=0x7fb8440ab650 "") at  
/home/install/lnmp1.5/src/mysql-  
5.6.40/storage/innobase/handler/ha_innodb.cc:6793  
#6 0x00000000005b440f in handler::ha_write_row (this=0x7fb8440ab260,  
buf=0x7fb8440ab650 "") at /home/install/lnmp1.5/src/mysql-  
5.6.40/sql/handler.cc:7351  
#7 0x00000000006dd3a8 in write_record (thd=thd@entry=0x1d396c90,  
table=table@entry=0x7fb8440aa970, info=info@entry=0x7fbb0d429400,  
update=update@entry=0x7fbb0d429480) at /home/install/lnmp1.5/src/mysql-  
5.6.40/sql/sql_insert.cc:1667  
#8 0x00000000006e2541 in mysql_insert (thd=thd@entry=0x1d396c90,  
table_list=<optimized out>, fields=..., values_list=..., update_fields=...,  
update_values=..., duplic=DUP_REPLACE, ignore=false) at  
/home/install/lnmp1.5/src/mysql-5.6.40/sql/sql_insert.cc:1072  
#9 0x00000000006fa90a in mysql_execute_command (thd=thd@entry=0x1d396c90) at  
/home/install/lnmp1.5/src/mysql-5.6.40/sql/sql_parse.cc:3500
```

2.2 update 线程

```
Thread 81 (Thread 0x7fbb24b67700 (LWP 27490)):  
#0 0x00007fbfae164c73 in select () from /lib64/libc.so.6  
#1 0x0000000000987c0f in os_thread_sleep (tm=<optimized out>) at  
/home/install/lnmp1.5/src/mysql-5.6.40/storage/innobase/os/os0thread.cc:287  
#2 0x00000000009e4dea in srv_conc_enter_innodb_with_atomics  
(trx=trx@entry=0x7fb94003c608) at /home/install/lnmp1.5/src/mysql-  
5.6.40/storage/innobase/srv/srv0conc.cc:276  
#3 srv_conc_enter_innodb (trx=trx@entry=0x7fb94003c608) at  
/home/install/lnmp1.5/src/mysql-5.6.40/storage/innobase/srv/srv0conc.cc:511  
#4 0x000000000093ae4e in innobase_srv_conc_enter_innodb (trx=0x7fb94003c608) at  
/home/install/lnmp1.5/src/mysql-  
5.6.40/storage/innobase/handler/ha_innodb.cc:1280
```

```

#5  ha_innodb::index_read (this=0x7fb95c05b540, buf=0x7fb95c2ae4f0
"\377\377\377", key_ptr=<optimized out>, key_len=<optimized out>,
find_flag=<optimized out>) at /home/install/lnmp1.5/src/mysql-
5.6.40/storage/innodb/handler/ha_innodb.cc:7675
#6  0x00000000005ab6e0 in ha_index_read_map (find_flag=HA_READ_KEY_EXACT,
keypart_map=3, key=0x7fb940017048 "7\307\017e\257h", buf=<optimized out>,
this=0x7fb95c05b540) at /home/install/lnmp1.5/src/mysql-
5.6.40/sql/handler.cc:2753
#7  handler::read_range_first (this=0x7fb95c05b540, start_key=<optimized out>,
end_key=<optimized out>, eq_range_arg=<optimized out>, sorted=<optimized out>)
at /home/install/lnmp1.5/src/mysql-5.6.40/sql/handler.cc:6717
#8  0x00000000005aa206 in handler::multi_range_read_next (this=0x7fb95c05b540,
range_info=0x7fbb24b65240) at /home/install/lnmp1.5/src/mysql-
5.6.40/sql/handler.cc:5871
#9  0x0000000000804acb in QUICK_RANGE_SELECT::get_next (this=0x7fb94000f720) at
/home/install/lnmp1.5/src/mysql-5.6.40/sql/opt_range.cc:10644
#10 0x000000000082ae2d in rr_quick (info=0x7fbb24b65410) at
/home/install/lnmp1.5/src/mysql-5.6.40/sql/records.cc:369
#11 0x0000000000766e1b in mysql_update (thd=thd@entry=0x1d1f2250,
table_list=<optimized out>, fields=..., values=..., conds=0x7fb9400009c8,
order_num=<optimized out>, order=<optimized out>, limit=18446744073709551615,
handle_duplicates=DUP_ERROR, ignore=false,
found_return=found_return@entry=0x7fbb24b65800,
updated_return=updated_return@entry=0x7fbb24b65d60) at
/home/install/lnmp1.5/src/mysql-5.6.40/sql/sql_update.cc:744

```

2.3 select 线程

```

Thread 66 (Thread 0x7fbb3c355700 (LWP 16028)):
#0  0x00007fbfae164c73 in select () from /lib64/libc.so.6
#1  0x0000000000987c0f in os_thread_sleep (tm=<optimized out>) at
/home/install/lnmp1.5/src/mysql-5.6.40/storage/innodb/os/os0thread.cc:287
#2  0x00000000009e4dea in srv_conc_enter_innodb_with_atomics
(trx=trx@entry=0x7fb988354858) at /home/install/lnmp1.5/src/mysql-
5.6.40/storage/innodb/srv/srv0conc.cc:276
#3  srv_conc_enter_innodb (trx=trx@entry=0x7fb988354858) at
/home/install/lnmp1.5/src/mysql-5.6.40/storage/innodb/srv/srv0conc.cc:511
#4  0x000000000093ae4e in innodb_srv_conc_enter_innodb (trx=0x7fb988354858) at
/home/install/lnmp1.5/src/mysql-
5.6.40/storage/innodb/handler/ha_innodb.cc:1280
#5  ha_innodb::index_read (this=0x7fb9880e33a0, buf=0x7fb988351b50
"\377\377\377\377", key_ptr=<optimized out>, key_len=<optimized out>,
find_flag=<optimized out>) at /home/install/lnmp1.5/src/mysql-
5.6.40/storage/innodb/handler/ha_innodb.cc:7675
#6  0x00000000005ab6e0 in ha_index_read_map (find_flag=HA_READ_AFTER_KEY,
keypart_map=7, key=0x7fb988134a48 "", buf=<optimized out>, this=0x7fb9880e33a0)

```

```
at /home/install/lnmp1.5/src/mysql-5.6.40/sql/handler.cc:2753
#7 handler::read_range_first (this=0x7fb9880e33a0, start_key=<optimized out>,
end_key=<optimized out>, eq_range_arg=<optimized out>, sorted=<optimized out>)
at /home/install/lnmp1.5/src/mysql-5.6.40/sql/handler.cc:6717
#8 0x00000000005aa206 in handler::multi_range_read_next (this=0x7fb9880e33a0,
range_info=0x7fbb3c353400) at /home/install/lnmp1.5/src/mysql-
5.6.40/sql/handler.cc:5871
#9 0x00000000000804acb in QUICK_RANGE_SELECT::get_next (this=0x7fb988002050) at
/home/install/lnmp1.5/src/mysql-5.6.40/sql/opt_range.cc:10644
#10 0x0000000000082ae2d in rr_quick (info=0x7fb98809c210) at
/home/install/lnmp1.5/src/mysql-5.6.40/sql/records.cc:369
#11 0x000000000006d44fd in sub_select (join=0x7fb98809a728,
join_tab=0x7fb98809c180, end_of_records=<optimized out>) at
/home/install/lnmp1.5/src/mysql-5.6.40/sql/sql_executor.cc:1259
#12 0x000000000006d2823 in do_select (join=0x7fb98809a728) at
/home/install/lnmp1.5/src/mysql-5.6.40/sql/sql_executor.cc:936
#13 JOIN::exec (this=0x7fb98809a728) at /home/install/lnmp1.5/src/mysql-
5.6.40/sql/sql_executor.cc:194
```

好了有了这些栈帧视乎发现一些共同点他们都处于 `innobase_srv_conc_enter_innodb` 函数下，本函数正是下面参数实现的方式：

1. `innodb_thread_concurrency`
2. `innodb_concurrency_tickets`

所以我随即告诉他检查这两个参数，如果设置了可以尝试取消。过后数据库故障得到解决。

3 参数和相关说明

实际上涉及到的参数主要是 `innodb_thread_concurrency` 和 `innodb_concurrency_tickets`。将高压线下线程之间抢占 CPU 而造成线程上下文切换的情况尽量阻塞在 `Innodb` 层之外，这就需要 `innodb_thread_concurrency` 参数了。同时又要保证对于那些（长时间处理线程）不会长时间的堵塞（短时间处理线程），比如某些 `select` 操作需要查询很久，而某些 `select` 操作查询量很小，如果等待（长时间的 `select` 操作）结束后（短时间 `select` 操作）才执行，那么显然会出现（短时间 `select` 操作）饥饿问题，换句话说对（短时间 `select` 操作）是不公平的，因此就引入了 `innodb_concurrency_tickets` 参数。

3.1 `innodb_thread_concurrency`

同一时刻能够进入 `Innodb` 层的会话（线程）数。如果在 `Innodb` 层干活的会话（线程）数量超过这个参数的设置，新会话（线程）将不能从 `MySQL` 层进入到 `Innodb` 层，它们将进入一个短暂的睡眠状态。休眠多久则通过参数 `innodb_thread_sleep_delay` 参数指定，如果还设置了参数 `innodb_adaptive_max_sleep_delay` 那么 `Innodb` 将会自动调整休眠时间，具体的算法实际上就在 `srv_conc_enter_innodb_with_atomics` 函数中，感兴趣的可以执行查看。其次这种休眠实际上是一个定时醒来的时钟，通过 `::nanosleep` 或者 `select`（多路 I/O 转接函数）进行实现，定时唤醒后会话（线程）重新判断是否可以进入 `Innodb` 层。函数 `os_thread_sleep` 部分如下：


```
#elif defined(HAVE_NANOSLEEP)
    struct timespec t;
    t.tv_sec = tm / 1000000;
    t.tv_nsec = (tm % 1000000) * 1000;
    ::nanosleep(&t, NULL);
#else
    struct timeval t;
    t.tv_sec = tm / 1000000;
    t.tv_usec = tm % 1000000;
    select(0, NULL, NULL, NULL, &t);
```

关于到底如何设置这个值，官方文档有如下建议：

Use the following guidelines to help find and maintain an appropriate setting:

- If the number of concurrent user threads for a workload is less than 64, set
innodb_thread_concurrency=0.
- If your workload is consistently heavy or occasionally spikes, start by setting
innodb_thread_concurrency=128 and then lowering the value to 96, 80, 64, and so on, until
you find the number of threads that provides the best performance. For example, suppose your
system typically has 40 to 50 users, but periodically the number increases to 60, 70, or even 200.
You find that performance is stable at 80 concurrent users but starts to show a regression above
this number. In this case, you would set innodb_thread_concurrency=80 to avoid impacting
performance.
- If you do not want InnoDB to use more than a certain number of virtual CPUs for user threads
(20 virtual CPUs, for example), set innodb_thread_concurrency to this number (or possibly
lower, depending on performance results). If your goal is to isolate MySQL from other applications,
you may consider binding the mysqld process exclusively to the virtual CPUs. Be aware,
however, that exclusive binding could result in non-optimal hardware usage if the mysqld process
is not consistently busy. In this case, you might bind the mysqld process to the virtual CPUs but
also allow other applications to use some or all of the virtual CPUs.
- innodb_thread_concurrency values that are too high can cause performance regression due

to increased contention on system internals and resources.

- In some cases, the optimal `innodb_thread_concurrency` setting can be smaller than the number of virtual CPUs.
- Monitor and analyze your system regularly. Changes to workload, number of users, or computing environment may require that you adjust the `innodb_thread_concurrency` setting

可以发现要合理的设置这个值并不那么容易并且要求较高。

3.2 innodb_concurrency_tickets

实际上这里的 tickets 可以理解为 MySQL 层和 InnoDB 层交互的次数，比如一个 select 一条数据就是需要 InnoDB 层返回一条数据然后 MySQL 层进行 where 条件的过滤然后返回给客户端，抛开 where 条件过滤的情况，如果我们一条语句需要查询 100 条数据，那么实际上需要进入 InnoDB 层 100 次，那么实际上消耗的 tickets 就是 100。当然对于 insert select 这种操作，需要的 tickets 是普通 select 的两倍，因为查询需要进入 InnoDB 层一次，insert 需要再次进入 InnoDB 层一次，后面我们就使用 insert select 的方式来模拟堵塞的情况，最后还会给出说明。

这样我们也就理解为什么 `innodb_concurrency_tickets` 可以避免（长时间处理线程）长时间堵塞（短时间处理线程）的原因了。假设 `innodb_concurrency_tickets` 为 5000（默认值），有一个需要查询 100W 行数据的大 select 操作和一个需要查询 100 行数据的小 select 操作，大 select 操作先进行，但是当查询了 5000 行数据后将丢失 CPU 使用权，小 select 操作将会进行并且一次性完成。

最后关于这里涉及的参数可以继续参考官方文档中的说明，我们线上并没有设置这些参数，因为感觉很难设置合适，如果设置不当反而会遇到问题，就如本案例一样。

3.3 事务操作状态

实际上如果是处于这种堵塞情况，我们完全可以在 `information_schema.innodb_trx` 和 `show engine innodb status` 中看到如下：

```
---TRANSACTION 162307, ACTIVE 133 sec sleeping before entering InnoDB (这里)
mysql tables in use 2, locked 2
767 lock struct(s), heap size 106968, 212591 row lock(s), undo log entries 15451
MySQL thread id 14, OS thread handle 140736751912704, query id 1077 localhost
root Sending data
insert into testui select * from testui
---TRANSACTION 162302, ACTIVE 320 sec, thread declared inside InnoDB 1
mysql tables in use 2, locked 2
2477 lock struct(s), heap size 336344, 609049 row lock(s), undo log entries
83582
MySQL thread id 13, OS thread handle 140737153779456, query id 1050 localhost
root Sending data
insert into testti3 select * from testti3
```

```
mysql> select
trx_id,trx_state,trx_query,trx_operation_state,trx_concurrency_tickets from
information_schema.innodb_trx \G
***** 1. row *****
      trx_id: 84325
      trx_state: RUNNING
      trx_query: insert into baguait4 select * from testgp
      trx_operation_state: sleeping before entering InnoDB (这里)
trx_concurrency_tickets: 0
***** 2. row *****
      trx_id: 84319
      trx_state: RUNNING
      trx_query: insert into baguait3 select * from testgp
      trx_operation_state: sleeping before entering InnoDB
trx_concurrency_tickets: 0
```

我们可以看到事务操作状态被标记为 `sleeping before entering InnoDB`。但是需要注意一点的是对于只读事务比如 `select` 操作而言，`show engine innodb status` 可能看不到。但是遗憾的是案例中朋友并没有采集 `trx_operation_state` 的值。

4 模拟测试

这里我们简单模拟，我们一共启用 3 个事务，其中两个 `insert select` 操作，一个单纯的 `select` 操作，当然这里的都是耗时操作，涉及的表每个表都有大概 100W 的数据。同时为了方便观察我们需要设置参数：

- 1. `innodb_thread_concurrency=1`
- 2. `innodb_concurrency_tickets=10`

操作步骤如下：

S1	S2	S3
insert into baguait4 select * from testgp		
	insert into baguait3 select * from testgp	
		select * from baguait1

如果多观察几次你可以看到如下的现象：

```
mysql> select
trx_id,trx_state,trx_query,trx_operation_state,trx_concurrency_tickets from
information_schema.innodb_trx \G show processlist;
```

```
***** 1. row *****
      trx_id: 84529
      trx_state: RUNNING
      trx_query: insert into baguait4 select * from testgp
      trx_operation_state: sleeping before entering InnoDB
      trx_concurrency_tickets: 0
***** 2. row *****
      trx_id: 84524
      trx_state: RUNNING
      trx_query: insert into baguait3 select * from testgp
      trx_operation_state: inserting
      trx_concurrency_tickets: 1
***** 3. row *****
      trx_id: 422211785606640
      trx_state: RUNNING
      trx_query: select * from baguait1
      trx_operation_state: sleeping before entering InnoDB
      trx_concurrency_tickets: 0
3 rows in set (0.00 sec)
```

```
+---+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+
| Id | User          | Host      | db      | Command | Time | State
| Info                                | Rows_sent | Rows_examined |
+---+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+
| 1 | event_scheduler | localhost | NULL    | Daemon  | 3173 | Waiting on empty
queue | NULL                                | 0 | 0 |
| 6 | root           | localhost | testmts | Query   | 70  | Sending data
| insert into baguait3 select * from testgp | 0 | 0 |
| 7 | root           | localhost | testmts | Query   | 68  | Sending data
| insert into baguait4 select * from testgp | 0 | 0 |
| 8 | root           | localhost | testmts | Query   | 66  | Sending data
| select * from baguait1                    | 120835 | 0 |
| 9 | root           | localhost | NULL    | Query   | 0   | starting
| show processlist                         | 0 | 0 |
+---+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+
5 rows in set (0.00 sec)
mysql> select
      trx_id, trx_state, trx_query, trx_operation_state, trx_concurrency_tickets from
      information_schema.innodb_trx \G show processlist;
```

```
***** 1. row *****
```

```

        trx_id: 84529
        trx_state: RUNNING
        trx_query: insert into baguait4 select * from testgp
        trx_operation_state: sleeping before entering InnoDB
trx_concurrency_tickets: 0
***** 2. row *****
        trx_id: 84524
        trx_state: RUNNING
        trx_query: insert into baguait3 select * from testgp
        trx_operation_state: sleeping before entering InnoDB
trx_concurrency_tickets: 0
***** 3. row *****
        trx_id: 422211785606640
        trx_state: RUNNING
        trx_query: select * from baguait1
        trx_operation_state: fetching rows
trx_concurrency_tickets: 3
3 rows in set (0.00 sec)
+----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
| Id | User          | Host      | db      | Command | Time | State
| Info                                | Rows_sent | Rows_examined |
+----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
| 1 | event_scheduler | localhost | NULL    | Daemon  | 3177 | Waiting on empty
queue | NULL                                |          |          |
| 6 | root           | localhost | testmts | Query   | 74  | Sending data
| insert into baguait3 select * from testgp |          |          |
| 7 | root           | localhost | testmts | Query   | 72  | Sending data
| insert into baguait4 select * from testgp |          |          |
| 8 | root           | localhost | testmts | Query   | 70  | Sending data
| select * from baguait1                    | 128718 |          |
| 9 | root           | localhost | NULL    | Query   | 0   | starting
| show processlist                          |          |          |
+----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

```

我们可以观察到 `trx_operation_state` 的状态 3 个操作都在交替的变化，但是总有 2 个处于 `sleeping before entering InnoDB` 状态。并且我们可以观察到 `trx_concurrency_tickets` 总是不会大于 10 的。因此我们有理由相信在同一时刻只有一个操作进入了 Innodb 层。但是需要注意的是在 `show engine innodb status` 中观察不到 `select` 的操作如下：

```

-----
TRANSACTIONS
-----

```

```
Trx id counter 84538
Purge done for trx's n:o < 84526 undo n:o < 0 state: running but idle
History list length 356
Total number of lock structs in row lock hash table 0
LIST OF TRANSACTIONS FOR EACH SESSION:
---TRANSACTION 422211785609424, not started
0 lock struct(s), heap size 1160, 0 row lock(s)
---TRANSACTION 422211785608032, not started
0 lock struct(s), heap size 1160, 0 row lock(s)
---TRANSACTION 84529, ACTIVE 103 sec inserting, thread declared inside InnoDB 6
mysql tables in use 2, locked 1
1 lock struct(s), heap size 1160, 0 row lock(s), undo log entries 111866
MySQL thread id 7, OS thread handle 140737158833920, query id 80 localhost root
Sending data
insert into baguait4 select * from testgp
Trx read view will not see trx with id >= 84529, sees < 84524
---TRANSACTION 84524, ACTIVE 105 sec sleeping before entering InnoDB
mysql tables in use 2, locked 1
1 lock struct(s), heap size 1160, 0 row lock(s), undo log entries 105605
MySQL thread id 6, OS thread handle 140737159034624, query id 79 localhost root
Sending data
insert into baguait3 select * from testgp
Trx read view will not see trx with id >= 84524, sees < 84524
```

但是我们还需要注意 show engine innodb status 有如下输出第一行说明了有 2 个会话（线程）堵塞在 InnoDB 层以外。

```
-----
ROW OPERATIONS
-----
1 queries inside InnoDB, 2 queries in queue
3 read views open inside InnoDB
2 RW transactions active inside InnoDB
```

5 实现方法

前面我们已经描述了每次 MySQL 层和 InnoDB 层的交互都会进行一次这样的判断，它用来决定会话（线程）是否能够进入 InnoDB 层，下面就是大概的逻辑，由函数 innobase_srv_conc_enter_innodb 调入。

```
->是否设置了参数 innodb_thread_concurrency
->是
    ->是否 tickets 大于 0
        ->是、直接进入 InnoDB 层并且 tickets 减 1
        ->否、调入函数 srv_conc_enter_innodb
            ->调入函数 srv_conc_enter_innodb_with_atomics
                ->开启死循环
```

->是否活跃线程数小于 innodb_thread_concurrency 设置

->是、增加活跃线程数，并且自动调整 delay 参数，退出死循环，满 tickets 进入

Innodb 层

->否、自动调整 delay 参数后设置事务操作状态为"sleeping before entering

InnoDB", 然后进入休眠状态直到时间达到后重新醒来继续循环

->否、直接进入 Innodb 层

我们可以看到这个实现方式，在 Innodb 以外的会话（线程）会一直等待直到 Innodb 层内活跃的线程数小于 innodb_thread_concurrency 为止，并且每次进入 Innodb 层都会将 tickets 减 1。

6 关于 insert select 操作消耗 tickets 的说明

这里额外说明一下，因为我在测试的时候看了一下，对于一行数据而言首先需要 select 查询出来然后再 insert 插入到表中，这里实际上一行数据涉及到进入 Innodb 层两次，那么就需要消耗 2 个 tickets，下面留下两个栈帧供自己后面参考：

6.1 insert select 查询数据进入 Innodb 层

```
#0 innobase_srv_conc_enter_innodb (prebuilt=0x7ffedcb98d10) at
/mysqldata/percona-server-locks-detail-
5.7.22/storage/innobase/handler/ha_innodb.cc:1740
#1 0x0000000001a53f7c in ha_innobase::general_fetch (this=0x7ffedcb9d760,
buf=0x7ffedc9469b0 "\375\n", direction=1, match_mode=0)
    at /mysqldata/percona-server-locks-detail-
5.7.22/storage/innobase/handler/ha_innodb.cc:9846
#2 0x0000000001a545ee in ha_innobase::rnd_next (this=0x7ffedcb9d760,
buf=0x7ffedc9469b0 "\375\n")
    at /mysqldata/percona-server-locks-detail-
5.7.22/storage/innobase/handler/ha_innodb.cc:10083
#3 0x0000000000f836d6 in handler::ha_rnd_next (this=0x7ffedcb9d760,
buf=0x7ffedc9469b0 "\375\n") at /mysqldata/percona-server-locks-detail-
5.7.22/sql/handler.cc:3146
#4 0x00000000014e2a55 in rr_sequential (info=0x7ffedcb4f120) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/records.cc:521
#5 0x0000000001581277 in sub_select (join=0x7ffedcb4ea20,
qep_tab=0x7ffedcb4f0d0, end_of_records=false)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_executor.cc:1280
#6 0x0000000001580be6 in do_select (join=0x7ffedcb4ea20) at /mysqldata/percona-
server-locks-detail-5.7.22/sql/sql_executor.cc:950
#7 0x000000000157eaa2 in JOIN::exec (this=0x7ffedcb4ea20) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/sql_executor.cc:199
#8 0x0000000001620327 in handle_query (thd=0x7ffedc012960, lex=0x7ffedc014f90,
result=0x7ffedcc46680, added_options=1342177280, removed_options=0)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_select.cc:185
#9 0x000000000180466d in Sql_cmd_insert_select::execute (this=0x7ffedcc46608,
```

```
thd=0x7ffedc012960)
```

6.2 insert select 插入数据进入 Innodb 层

```
#0 innobase_srv_conc_enter_innodb (prebuilt=0x7ffedcb9c6f0) at
/mysqldata/percona-server-locks-detail-
5.7.22/storage/innobase/handler/ha_innodb.cc:1740
#1 0x0000000001a50587 in ha_innobase::write_row (this=0x7ffedc946470,
record=0x7ffedcb78d00 "\375\n")
    at /mysqldata/percona-server-locks-detail-
5.7.22/storage/innobase/handler/ha_innodb.cc:8341
#2 0x0000000000f9041d in handler::ha_write_row (this=0x7ffedc946470,
buf=0x7ffedcb78d00 "\375\n") at /mysqldata/percona-server-locks-detail-
5.7.22/sql/handler.cc:8466
#3 0x00000000018004b9 in write_record (thd=0x7ffedc012960, table=0x7ffedcb8f940,
info=0x7ffedcc466c8, update=0x7ffedcc46740)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_insert.cc:1881
#4 0x00000000018019b9 in Query_result_insert::send_data (this=0x7ffedcc46680,
values=...) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_insert.cc:2279
#5 0x00000000015853a8 in end_send (join=0x7ffedcb4ea20, qep_tab=0x7ffedcb4f248,
end_of_records=false)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_executor.cc:2925
#6 0x0000000001581f71 in evaluate_join_record (join=0x7ffedcb4ea20,
qep_tab=0x7ffedcb4f0d0) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_executor.cc:1645
#7 0x0000000001581372 in sub_select (join=0x7ffedcb4ea20,
qep_tab=0x7ffedcb4f0d0, end_of_records=false)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_executor.cc:1297
#8 0x0000000001580be6 in do_select (join=0x7ffedcb4ea20) at /mysqldata/percona-
server-locks-detail-5.7.22/sql/sql_executor.cc:950
#9 0x000000000157eaa2 in JOIN::exec (this=0x7ffedcb4ea20) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/sql_executor.cc:199
#10 0x0000000001620327 in handle_query (thd=0x7ffedc012960, lex=0x7ffedc014f90,
result=0x7ffedcc46680, added_options=1342177280, removed_options=0)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_select.cc:185
#11 0x000000000180466d in Sql_cmd_insert_select::execute (this=0x7ffedcc46608,
thd=0x7ffedc012960)
```

实际上插入数据正是在查询完数据后调用函数 `evaluate_join_record` 的时候，通过回调了函数 `Query_result_insert::send_data` 来实现，这点和单纯的 `select` 不一样单纯的 `select` 这里调入是函数 `Query_result_send::send_data` 如下：

```
#0 Query_result_send::send_data (this=0x7ffedcc465f8, items=...) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/sql_class.cc:2915
```



```
#1 0x00000000015853a8 in end_send (join=0x7ffedcb4e930, qep_tab=0x7ffedcb4f4b0,
end_of_records=false)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_executor.cc:2925
#2 0x0000000001581f71 in evaluate_join_record (join=0x7ffedcb4e930,
qep_tab=0x7ffedcb4f338) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_executor.cc:1645
#3 0x0000000001581372 in sub_select (join=0x7ffedcb4e930,
qep_tab=0x7ffedcb4f338, end_of_records=false)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_executor.cc:1297
#4 0x0000000001580be6 in do_select (join=0x7ffedcb4e930) at /mysqldata/percona-
server-locks-detail-5.7.22/sql/sql_executor.cc:950
#5 0x000000000157eaa2 in JOIN::exec (this=0x7ffedcb4e930) at
/mysqldata/percona-server-locks-detail-5.7.22/sql/sql_executor.cc:199
#6 0x0000000001620327 in handle_query (thd=0x7ffedc012960, lex=0x7ffedc014f90,
result=0x7ffedcc465f8, added_options=0, removed_options=0)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_select.cc:185
#7 0x00000000015d1f77 in execute_sqlcom_select (thd=0x7ffedc012960,
all_tables=0x7ffedcc45cf0) at /mysqldata/percona-server-locks-detail-
5.7.22/sql/sql_parse.cc:5445
```



MySQL: 产生大量小 relay log 的故障一例

作者: 高鹏 (八怪)

1 案例来源和现象

这个案例是朋友 @peaceful 遇到的线上问题, 最终线索也是他自己找到的。现象如下:

1. 出现了大量很小的 relay log 如下, 堆积量大约 2600 个:

```
...
-rw-r----- 1 mysql dba      12827 Oct 11 12:28 mysql-relay-bin.036615
-rw-r----- 1 mysql dba       4908 Oct 11 12:28 mysql-relay-bin.036616
-rw-r----- 1 mysql dba       1188 Oct 11 12:28 mysql-relay-bin.036617
-rw-r----- 1 mysql dba       5823 Oct 11 12:29 mysql-relay-bin.036618
-rw-r----- 1 mysql dba        507 Oct 11 12:29 mysql-relay-bin.036619
-rw-r----- 1 mysql dba       1188 Oct 11 12:29 mysql-relay-bin.036620
-rw-r----- 1 mysql dba       3203 Oct 11 12:29 mysql-relay-bin.036621
-rw-r----- 1 mysql dba      37916 Oct 11 12:30 mysql-relay-bin.036622
-rw-r----- 1 mysql dba        507 Oct 11 12:30 mysql-relay-bin.036623
-rw-r----- 1 mysql dba       1188 Oct 11 12:31 mysql-relay-bin.036624
-rw-r----- 1 mysql dba       4909 Oct 11 12:31 mysql-relay-bin.036625
-rw-r----- 1 mysql dba       1188 Oct 11 12:31 mysql-relay-bin.036626
-rw-r----- 1 mysql dba        507 Oct 11 12:31 mysql-relay-bin.036627
-rw-r----- 1 mysql dba        507 Oct 11 12:32 mysql-relay-bin.036628
-rw-r----- 1 mysql dba       1188 Oct 11 12:32 mysql-relay-bin.036629
-rw-r----- 1 mysql dba        454 Oct 11 12:32 mysql-relay-bin.036630
-rw-r----- 1 mysql dba       6223 Oct 11 12:32 mysql-relay-bin.index
```

2. 主库错误日志有如下错误

```
2019-10-11T12:31:26.517309+08:00 61303425 [Note] While initializing dump thread
for slave with UUID <eade0d03-ad91-11e7-8559-c81f66be1379>, found a zombie dump
thread with the same UUID. Master is killing the zombie dump thread(61303421).
2019-10-11T12:31:26.517489+08:00 61303425 [Note] Start binlog_dump to
master_thread_id(61303425) slave_server(19304313), pos(, 4)
2019-10-11T12:31:44.203747+08:00 61303449 [Note] While initializing dump thread
for slave with UUID <eade0d03-ad91-11e7-8559-c81f66be1379>, found a zombie dump
thread with the same UUID. Master is killing the zombie dump thread(61303425).
2019-10-11T12:31:44.203896+08:00 61303449 [Note] Start binlog_dump to
master_thread_id(61303449) slave_server(19304313), pos(, 4)
```

2 slave_net_timeout 参数分析

实际上第一眼看这个案例我也觉得很奇怪，因为很少有人会去设置 `slave_net_timeout` 参数，同样我们也没有设置过，因此关注较少。但是@peaceful 自己找到了可能出现问题的设置就是当前从库 `slave_net_timeout` 参数设置为 10。我就顺着这个线索往下分析，我们先来看看 `slave_net_timeout` 参数的功能。当前看来从库的 `slave_net_timeout` 有如下两个功能：

1. 设置 IO 线程在空闲情况下（没有 Event 接收的情况下）的连接超时时间。
这个参数 5.7.7 过后是 60 秒，以前是 3600 秒，修改后需要重启主从才会生效。
2. 如果 `change master` 没有指定 `MASTER_HEARTBEAT_PERIOD` 的情况下会设置为 `slave_net_timeout/2`

一般我们配置主从都没有去指定这个心跳周期，因此就是 `slave_net_timeout/2`，它控制的是如果在主库没有 Event 产生的情况下，多久发送一个心跳 Event 给从库的 IO 线程，用于保持连接。但是一旦我们配置了主从（`change master`）这个值就定下来了，不会随着 `slave_net_timeout` 参数的更改而更改，我们可以在 `slave_master_info` 表中找到相应的设置如下：

```
mysql> select Heartbeat from slave_master_info \G
***** 1. row *****
Heartbeat: 30
1 row in set (0.01 sec)
```

如果我们要更改这个值只能重新 `change master` 才行。

3 原因总结

如果满足下面三个条件，将会出现案例中的故障：

1. 主从中的 `MASTER_HEARTBEAT_PERIOD` 的值大于从库 `slave_net_timeout`
2. 主库当前压力很小持续 `slave_net_timeout` 设置时间没有产生新的 Event
3. 之前主从有一定的延迟

那么这种情况下在主库心跳 Event 发送给从库的 IO 线程之前，IO 线程已经断开了。断开后 IO 线程会进行重连，每次重连将会生成新的 relay log，但是这些 relay log 由于延迟问题不能清理就出现了案例中的情况。

下面是官方文档中关于这部分说明：

```
If you are logging master connection information to tables,
MASTER_HEARTBEAT_PERIOD can be seen
as the value of the Heartbeat column of the mysql.slave_master_info table.
Setting interval to 0 disables heartbeats altogether. The default value for
interval is equal to the
value of slave_net_timeout divided by 2.
Setting @@global.slave_net_timeout to a value less than that of the current
heartbeat interval
results in a warning being issued. The effect of issuing RESET SLAVE on the
heartbeat interval is to
reset it to the default value.
```

4 案例模拟

有了理论基础就很好了模拟了，延迟这一点我模拟的时候关闭了从库的 SQL 线程来模拟 relay log 积压的情况，因为这个案例和 SQL 线程没有太多的关系。提前配置好主从，查看当前的心跳周期和 slave_net_timeout 参数如下：

```
mysql> show variables like '%slave_net_timeout%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| slave_net_timeout | 60    |
+-----+-----+
1 row in set (0.01 sec)

mysql> select Heartbeat from slave_master_info \G
***** 1. row *****
Heartbeat: 30
1 row in set (0.00 sec)
```

4.1 停止从库的 SQL 线程

```
stop slave sql_thread;
```

4.2 设置 slave_net_timeout 为 10

```
mysql> set global slave_net_timeout=10;
Query OK, 0 rows affected, 1 warning (0.00 sec)
mysql> show warnings;
+-----+-----+-----+
| Level | Code | Message |
+-----+-----+-----+
| Warning | 1704 | The requested value for the heartbeat period exceeds the value of `slave_net_timeout' seconds. A sensible value for the period should be less than the timeout. |
+-----+-----+-----+
1 row in set (0.00 sec)
```

可以看到这里实际上已经有一个警告了。

4.3 重启 IO 线程这样才会让 slave_net_timeout 参数生效

```
mysql> stop slave ;
Query OK, 0 rows affected (0.01 sec)
mysql> start slave io_thread;
Query OK, 0 rows affected (0.01 sec)
```

4.4 观察现象

大概每 10 秒会生成一个 relay log 文件如下：

```
-rw-r----- 1 mysql mysql      500 2019-09-27 23:48:32.655001361 +0800
relay.000142
-rw-r----- 1 mysql mysql      500 2019-09-27 23:48:42.943001355 +0800
relay.000143
-rw-r----- 1 mysql mysql      500 2019-09-27 23:48:53.293001363 +0800
relay.000144
-rw-r----- 1 mysql mysql      500 2019-09-27 23:49:03.502000598 +0800
relay.000145
-rw-r----- 1 mysql mysql      500 2019-09-27 23:49:13.799001357 +0800
relay.000146
-rw-r----- 1 mysql mysql      500 2019-09-27 23:49:24.055001354 +0800
relay.000147
-rw-r----- 1 mysql mysql      500 2019-09-27 23:49:34.280001827 +0800
relay.000148
-rw-r----- 1 mysql mysql      500 2019-09-27 23:49:44.496001365 +0800
relay.000149
-rw-r----- 1 mysql mysql      500 2019-09-27 23:49:54.789001353 +0800
relay.000150
-rw-r----- 1 mysql mysql      500 2019-09-27 23:50:05.485001371 +0800
relay.000151
-rw-r----- 1 mysql mysql      500 2019-09-27 23:50:15.910001430 +0800
relay.000152
```

大概每 10 秒主库的日志会输出如下日志：

```
2019-10-08T02:27:24.996827+08:00 217 [Note] While initializing dump thread for
slave with UUID <010fde77-2075-11e9-ba07-5254009862c0>, found a zombie dump
thread with the same UUID. Master is killing the zombie dump thread(216).
2019-10-08T02:27:24.998297+08:00 217 [Note] Start binlog_dump to
master_thread_id(217) slave_server(953340), pos(, 4)
2019-10-08T02:27:35.265961+08:00 218 [Note] While initializing dump thread for
slave with UUID <010fde77-2075-11e9-ba07-5254009862c0>, found a zombie dump
thread with the same UUID. Master is killing the zombie dump thread(217).
2019-10-08T02:27:35.266653+08:00 218 [Note] Start binlog_dump to
master_thread_id(218) slave_server(953340), pos(, 4)
2019-10-08T02:27:45.588074+08:00 219 [Note] While initializing dump thread for
slave with UUID <010fde77-2075-11e9-ba07-5254009862c0>, found a zombie dump
thread with the same UUID. Master is killing the zombie dump thread(218).
2019-10-08T02:27:45.589814+08:00 219 [Note] Start binlog_dump to
master_thread_id(219) slave_server(953340), pos(, 4)
2019-10-08T02:27:55.848558+08:00 220 [Note] While initializing dump thread for
slave with UUID <010fde77-2075-11e9-ba07-5254009862c0>, found a zombie dump
```

```
thread with the same UUID. Master is killing the zombie dump thread(219).
2019-10-08T02:27:55.849442+08:00 220 [Note] Start binlog_dump to
master_thread_id(220) slave_server(953340), pos(, 4)
这个日志就和案例中的一模一样了。
```

4.5 解决问题

知道原因后解决也就很简单了我们只需设置 `slave_net_timeout` 参数为 `MASTER_HEARTBEAT_PERIOD` 的 2 倍就可以了，设置后重启主从即可。

5 实现方式

这里我们将通过简单的源码调用分析来看看到底 `slave_net_timeout` 参数和 `MASTER_HEARTBEAT_PERIOD` 对主从的影响。

5.1 从库使用参数 `slave_net_timeout`

从库 IO 线程启动时候会通过参数 `slave_net_timeout` 设置超时：

```
->connect_to_master
->mysql_options
case MYSQL_OPT_CONNECT_TIMEOUT: //MYSQL_OPT_CONNECT_TIMEOUT
    mysql->options.connect_timeout= *(uint*) arg;
    break;
```

而在建立和主库的连接时候会使用这个值

```
connect_to_master
->mysql_real_connect
->get_vio_connect_timeout
timeout_sec= mysql->options.connect_timeout;
```

因此我们也看到了 `slave_net_timeout` 参数只有在 IO 线程重启的时候才会生效

5.2 从库设置 `MASTER_HEARTBEAT_PERIOD` 值

在每次使用从库 `change master` 时候会设置这个值如下，默认为 `slave_net_timeout/2`：

```
->change_master
->change_receive_options
mi->heartbeat_period= min<float>(SLAVE_MAX_HEARTBEAT_PERIOD,
                                (slave_net_timeout/2.0f));
```

因此我们看到只有 `change master` 才会重新设置这个值，重启主从是不会重新设置的。

5.3 使用 MASTER_HEARTBEAT_PERIOD 值

每次 IO 线程启动时候会将这个值传递给主库的 DUMP 线程，方式应该是通过构建语句 SET @masterheartbeatperiod 来完成的。如下：

```
->handle_slave_io
->get_master_version_and_clock
if (mi->heartbeat_period != 0.0) {
    char llbuf[22];
    const char query_format[] = "SET @master_heartbeat_period= %s";
    char query[sizeof(query_format) - 2 + sizeof(llbuf)];
```

主库启动 DUMP 线程的时候会通过搜索的方式找到这个值如下

```
->Binlog_sender::init
->Binlog_sender::init_heartbeat_period
user_var_entry *entry=
    (user_var_entry*) my_hash_search(&m_thd->user_vars, (uchar*) name.str,
                                    name.length);
m_heartbeat_period= entry ? entry->val_int(&null_value) : 0;
```

5.4 DUMP 线程使用 MASTER_HEARTBEAT_PERIOD 发送心跳 Event

这里主要是通过一个超时等待来完成，如下：

```
->Binlog_sender::wait_new_events
->Binlog_sender::wait_with_heartbeat
set_timespec_nsec(&ts, m_heartbeat_period); //心跳超时
ret= mysql_bin_log.wait_for_update_bin_log(m_thd, &ts); //等待
if (ret != ETIMEDOUT && ret != ETIME) //如果是正常收到则收到信号，说明有新的 Event 到来，否则如果是超时则发送心跳 Event
    break; //正常返回 0 是超时返回 ETIMEDOUT 继续循环
if (send_heartbeat_event(log_pos)) //发送心跳 Event
    return 1;
```

5.5 重连会杀掉可能的存在的 DUMP 线程

根据 UUID 进行比对如下：

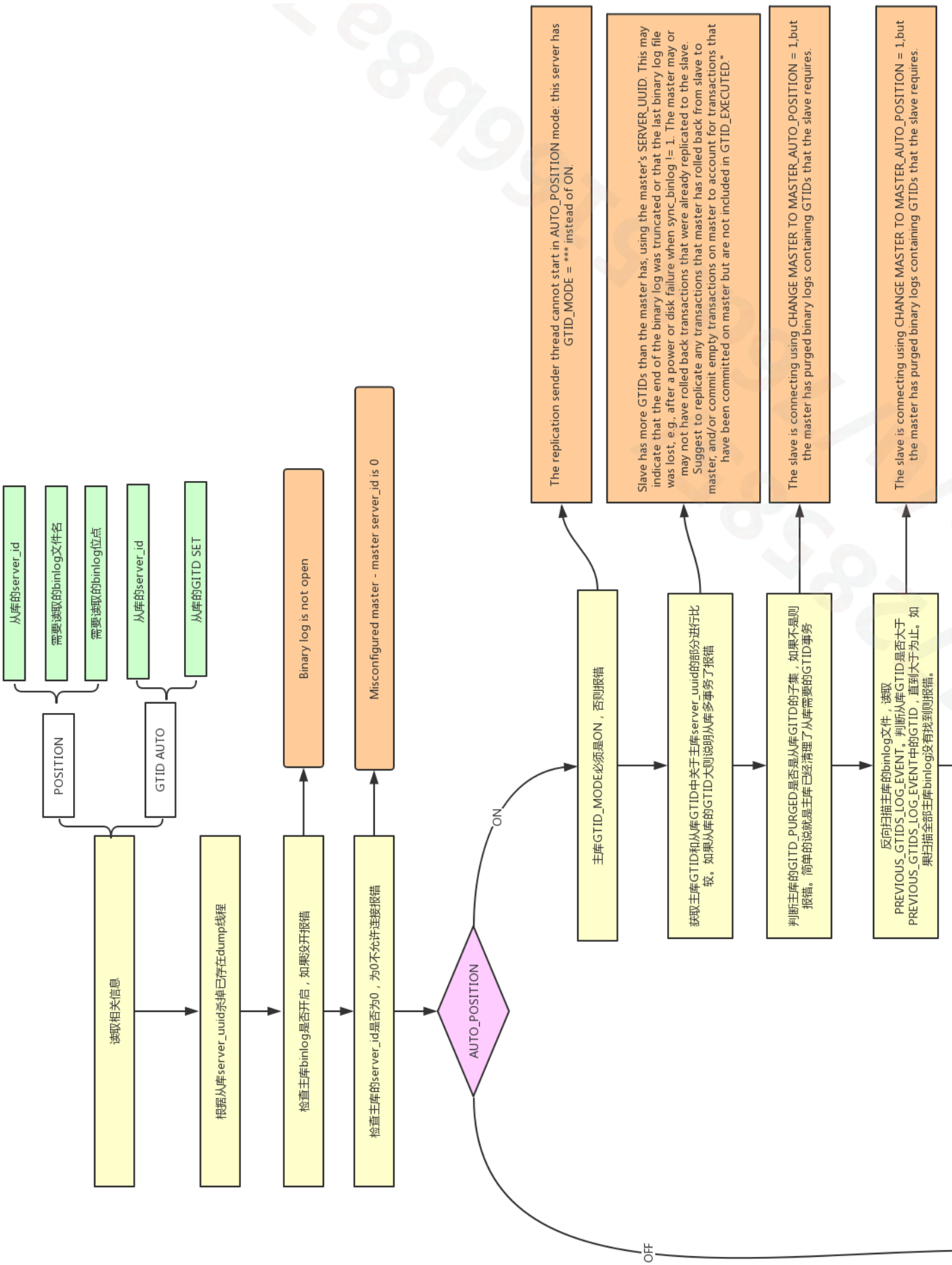
```
->kill_zombie_dump_threads
Find_zombie_dump_thread find_zombie_dump_thread(slave_uuid);
THD *tmp= Global_THD_manager::get_instance()->
    find_thd(&find_zombie_dump_thread);

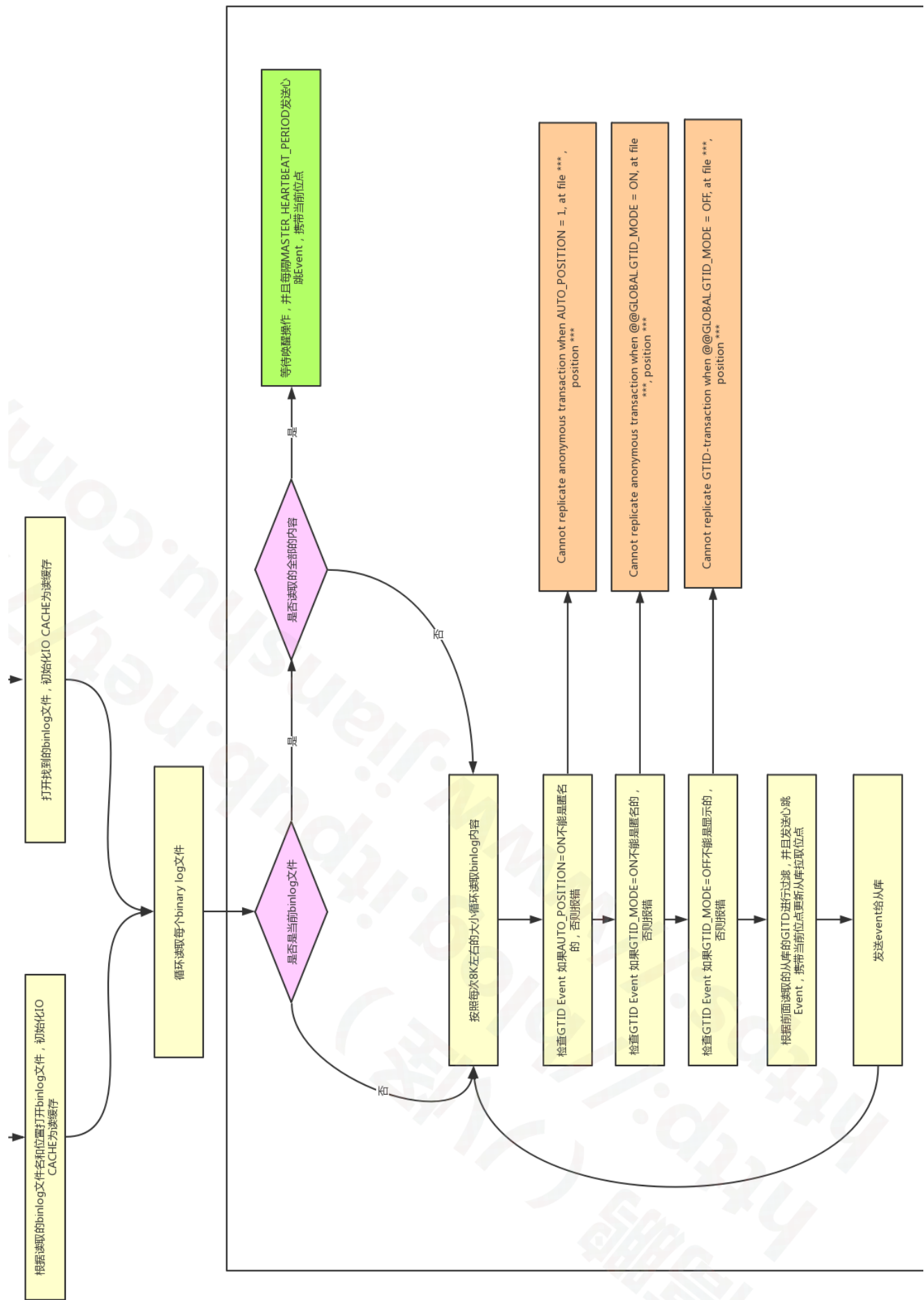
if (tmp)
{
    /*
    Here we do not call kill_one_thread() as
```

```
    it will be slow because it will iterate through the list
    again. We just to do kill the thread ourselves.
*/
if (log_warnings > 1)
{
    if (slave_uuid.length())
    {
        sql_print_information("While initializing dump thread for slave with "
                              "UUID <%s>, found a zombie dump thread with the "
                              "same UUID. Master is killing the zombie dump "
                              "thread(%u).", slave_uuid.c_ptr(),
                              tmp->thread_id());
    } //这里就是本案例中的日志了
.....
```

这里我们看到了案例中的日志。

5.6 关于 DUMP 线程流程图







timestamp 时区转换导致 CPU %sy 高的问题

作者：高鹏（八怪）

这个问题是一个朋友遇到的@风云，并且这位朋友已经得出了近乎正确的判断，下面进行一些描述。

1 问题展示

```
top - 10:29:06 up 1:45, 2 users, load average: 2.09, 8.36, 14.41
Tasks: 1095 total, 1 running, 1094 sleeping, 0 stopped, 0 zombie
Cpu(s): 7.3%us, 40.4%sy, 0.0%ni, 52.2%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 148699128k total, 12785808k used, 135913320k free, 207060k buffers
Swap: 32767992k total, 0k used, 32767992k free, 1309084k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
9587	mysql	20	0	50.5g	3.9g	8892	S	2957.4	2.7	1776:34	mysqld

我们可以看到 40.4%sy 正是系统调用负载较高的表现，随即朋友采集了 perf 如下：

```
Samples: 44K of event 'cycles', Event count (approx.): 21326871443
79.48% [kernel] [k] _spin_lock
1.59% mysql [.] btr_search_guess_on_hash(dict_index_t*, btr_search_t*, dtuple_t const*, unsigned long, unsigned long, btr_cur_t*, unsigned long, mtr_t*
1.45% mysql [.] row_search_for_mysql(unsigned char*, unsigned long, row_prebuilt_t*, unsigned long, unsigned long)
1.38% mysql [.] rec_get_offsets_func(unsigned char const*, dict_index_t const*, unsigned long*, unsigned long, mem_block_info_t**)
0.74% libc-2.12.so [.] _IO_vfscanf
0.66% libc-2.12.so [.] memcpy
0.61% mysql [.] btr_cur_search_to_nth_level(dict_index_t*, unsigned long, dtuple_t const*, unsigned long, unsigned long, btr_cur_t*, unsigned long, cha
0.38% mysql [.] cmp_dtuple_rec_with_match_low(dtuple_t const*, unsigned char const*, unsigned long const*, unsigned long, unsigned long, unsigned long
0.37% mysql [.] row_sel_field_store_in_mysql_format_func(unsigned char*, mysql_row_tmpl_t const*, unsigned char const*, unsigned long)
0.30% mysql [.] my_strncollsp_utf8
0.26% [kernel] [k] native_write_msr_safe
```

接下来朋友采集了 pstack 给我，我发现大量的线程处于如下状态下：

```
Thread 38 (Thread 0x7fe57a86f700 (LWP 67268)):
#0 0x0000003dee4f82ce in __lll_lock_wait_private () from /lib64/libc.so.6
#1 0x0000003dee49df8d in _L_lock_2163 () from /lib64/libc.so.6
#2 0x0000003dee49dd47 in __tz_convert () from /lib64/libc.so.6
#3 0x000000000007c02e7 in Time_zone_system::gmt_sec_to_TIME(st_mysql_time*, long)
const ()
#4 0x00000000000811df6 in Field_timestampf::get_date_internal(st_mysql_time*) ()
#5 0x00000000000809ea9 in Field_temporal_with_date::val_date_temporal() ()
#6 0x000000000005f43cc in get_datetime_value(THD*, Item***, Item**, Item*, bool*)
()
#7 0x000000000005e7ba7 in Arg_comparator::compare_datetime() ()
#8 0x000000000005eef4e in Item_func_gt::val_int() ()
#9 0x000000000006fc6ab in evaluate_join_record(JOIN*, st_join_table*) ()
#10 0x00000000000700e7e in sub_select(JOIN*, st_join_table*, bool) ()
```

```
#11 0x000000000006fecc1 in JOIN::exec() ()
```

我们可以注意一下 `__tz_convert` 这正是时区转换的证据。

2 关于 timestamp 简要说明

timestamp: 占用 4 字节, 内部实现是新纪元时间 (1970-01-01 00:00:00) 以来的秒, 那么这种格式在展示给用户的时候就需要做必要的时区转换才能得到正确数据。下面我们通过访问 ibd 文件来查看一下内部表示方法, 使用到了我的两个工具 innodb 和 bcview。

详细参考: <https://www.jianshu.com/p/719f1bbb21e8>。

timestamp 的内部表示

建立一个测试表

```
mysql> show variables like '%time_zone%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| system_time_zone | CST |
| time_zone | +08:00 |
+-----+-----+
mysql> create table tmm(dt timestamp);
Query OK, 0 rows affected (0.04 sec)
mysql> insert into tmm values('2019-01-01 01:01:01');
Query OK, 1 row affected (0.00 sec)
```

我们来查看一下内部表示如下:

```
[root@gp1 test]# ./bcview tmm.ibd 16 125 25|grep 00000003
current block:00000003--Offset:00125--cnt bytes:25--data
is:000001ac3502000000070d52c80000002f01105c2a4b4d0000
```

整理一下如下:

1. 000001ac3502: rowid
2. 000000070d52: trx id
3. c80000002f0110: roll ptr
4. 5c2a4b4d: timestamp 类型的实际数据十进制为 1546275661

我们使用 Linux 命令如下:

```
[root@gp1 ~]# date -d @1546275661
Tue Jan 1 01:01:01 CST 2019
```

因为我的 Linux 也是 CST +8 时区这里数据也和 MySQL 中显示一样。下面我们调整一下时区再来看看取值如下：

```
mysql> set time_zone='+06:00';
Query OK, 0 rows affected (0.00 sec)
mysql> select * from tmm;
+-----+
| dt                |
+-----+
| 2018-12-31 23:01:01 |
+-----+
1 row in set (0.01 sec)
```

这里可以看到减去了 2 个小时，因为我的时区从+8 变为了+6。

3 timestamp 转换

在进行新纪元时间（1970-01-01 00:00:00）以来的秒到实际时间之间转换的时候 MySQL 根据参数 `time_zone` 的设置有两种选择：

1. `time_zone` 设置为 SYSTEM 的话：使用 `sys_time_zone` 获取的 OS 会话时区，同时使用 OS API 进行转换。对应转换函数 `Time_zone_system::gmt_sec_to_TIME`
2. `time_zone` 设置为实际的时区的话：比如 ‘+08:00’，那么使用 MySQL 自己的方法进行转换。对应转换函数 `Time_zone_offset::gmt_sec_to_TIME`

实际上 `Time_zone_system` 和 `Time_zone_offset` 均继承于 `Time_zone` 类，并且实现了 `Time_zone` 类的虚函数进行了重写，因此上层调用都是 `Time_zone::gmt_sec_to_TIME`。

注意这种转换操作是每行符合条件的数据都需要转换的。

4 问题修复方案

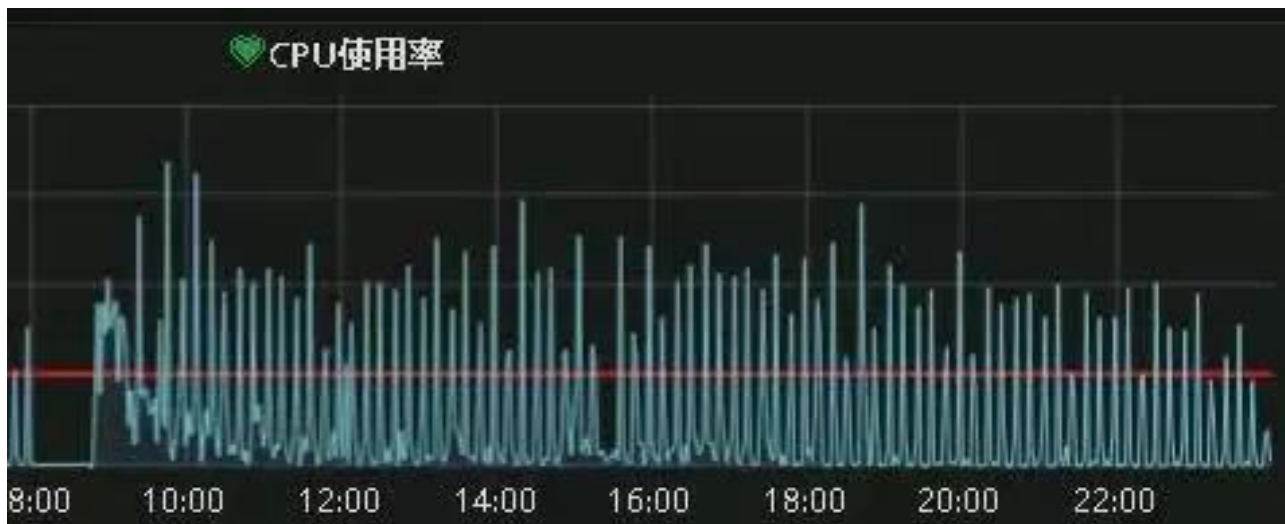
我们从问题栈帧来看这个故障使用的是 `Time_zone_system::gmt_sec_to_TIME` 函数进行转换的，因此可以考虑如下：

1. `time_zone`：设置为指定的时区，比如 ‘+08:00’。这样就不会使用 OS API 进行转换了，而转为 MySQL 自己的内部实现 调用 `Time_zone_offset::gmt_sec_to_TIME` 函数。但是需要注意的是，如果使用 MySQL 自己的实现那么 `us%` 会加剧。
2. 使用 `datetime` 代替 `timestamp`，新版本 `datetime` 为 5 个字节，只比 `timestamp` 多一个字节。

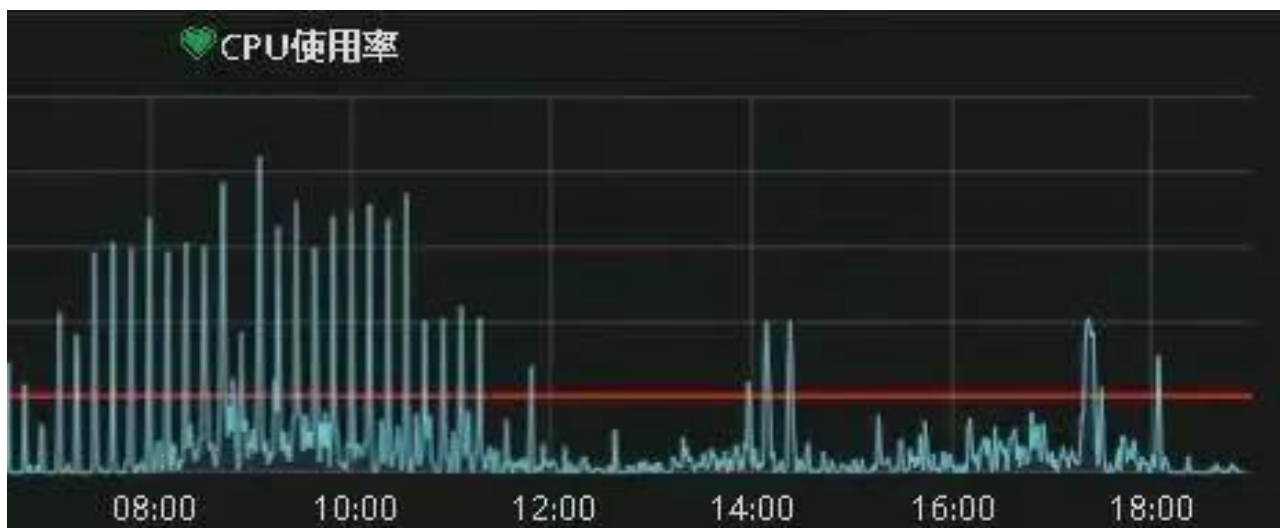
5 修复前后

`sy%` 使用量对比 据朋友说他大概在上午 11 点多完成了修改，做的方式是将 `time_zone` 修改为 ‘+08:00’，下面展示修改前后 CPU 使用率的对比：

修复前:



修复后:



6 备用栈帧

1. time_zone= 'SYSTEM' 转换栈帧

```
#0 Time_zone_system::gmt_sec_to_TIME (this=0x2e76948, tmp=0x7fffec0f3ff0,
t=1546275661) at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/tztime.cc:1092
#1 0x0000000000f6b65c in Time_zone::gmt_sec_to_TIME (this=0x2e76948,
tmp=0x7fffec0f3ff0, tv=...) at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/tztime.h:60
#2 0x0000000000f51643 in Field_timestampf::get_date_internal
(this=0x7ffe7ca66540, ltime=0x7fffec0f3ff0)
    at /root/mysqlall/percona-server-locks-detail-5.7.22/sql/field.cc:6014
#3 0x0000000000f4ff49 in Field_temporal_with_date::val_str (this=0x7ffe7ca66540,
val_buffer=0x7fffec0f4370, val_ptr=0x7fffec0f4370)
    at /root/mysqlall/percona-server-locks-detail-5.7.22/sql/field.cc:5429
```

```

#4 0x0000000000f11d7b in Field::val_str (this=0x7ffe7ca66540,
str=0x7fffec0f4370) at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/field.h:866

#5 0x0000000000f4549d in Field::send_text (this=0x7ffe7ca66540,
protocol=0x7ffe7c001e88) at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/field.cc:1725
#6 0x000000000014dfb82 in Protocol_text::store (this=0x7ffe7c001e88,
field=0x7ffe7ca66540)
    at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/protocol_classic.cc:1415
#7 0x0000000000fb06c0 in Item_field::send (this=0x7ffe7c006ec0,
protocol=0x7ffe7c001e88, buffer=0x7fffec0f4760)
    at /root/mysqlall/percona-server-locks-detail-5.7.22/sql/item.cc:7801
#8 0x0000000000156b15c in THD::send_result_set_row (this=0x7ffe7c000b70,
row_items=0x7ffe7c005d58)
    at /root/mysqlall/percona-server-locks-detail-5.7.22/sql/sql_class.cc:5026
#9 0x00000000001565758 in Query_result_send::send_data (this=0x7ffe7c006e98,
items=...) at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/sql_class.cc:2932
#10 0x00000000001585490 in end_send (join=0x7ffe7c007078, qep_tab=0x7ffe7c0078d0,
end_of_records=false)
    at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/sql_executor.cc:2925
#11 0x00000000001582059 in evaluate_join_record (join=0x7ffe7c007078,
qep_tab=0x7ffe7c007758)

```

2. time_zone= '+08:00' 转换栈帧

```

#0 Time_zone_offset::gmt_sec_to_TIME (this=0x6723d90, tmp=0x7fffec0f3ff0,
t=1546275661) at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/tztime.cc:1418
#1 0x0000000000f6b65c in Time_zone::gmt_sec_to_TIME (this=0x6723d90,
tmp=0x7fffec0f3ff0, tv=...) at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/tztime.h:60
#2 0x0000000000f51643 in Field_timestampf::get_date_internal
(this=0x7ffe7ca66540, ltime=0x7fffec0f3ff0)
    at /root/mysqlall/percona-server-locks-detail-5.7.22/sql/field.cc:6014
#3 0x0000000000f4ff49 in Field_temporal_with_date::val_str (this=0x7ffe7ca66540,
val_buffer=0x7fffec0f4370, val_ptr=0x7fffec0f4370)
    at /root/mysqlall/percona-server-locks-detail-5.7.22/sql/field.cc:5429
#4 0x0000000000f11d7b in Field::val_str (this=0x7ffe7ca66540,
str=0x7fffec0f4370) at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/field.h:866
#5 0x0000000000f4549d in Field::send_text (this=0x7ffe7ca66540,
protocol=0x7ffe7c001e88) at /root/mysqlall/percona-server-locks-detail-

```

```
5.7.22/sql/field.cc:1725
#6 0x00000000014dfb82 in Protocol_text::store (this=0x7ffe7c001e88,
field=0x7ffe7ca66540)
    at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/protocol_classic.cc:1415
#7 0x000000000fb06c0 in Item_field::send (this=0x7ffe7c006ec0,
protocol=0x7ffe7c001e88, buffer=0x7ffffec0f4760)
    at /root/mysqlall/percona-server-locks-detail-5.7.22/sql/item.cc:7801
#8 0x000000000156b15c in THD::send_result_set_row (this=0x7ffe7c000b70,
row_items=0x7ffe7c005d58)
    at /root/mysqlall/percona-server-locks-detail-5.7.22/sql/sql_class.cc:5026
#9 0x0000000001565758 in Query_result_send::send_data (this=0x7ffe7c006e98,
items=...) at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/sql_class.cc:2932
#10 0x0000000001585490 in end_send (join=0x7ffe7c007078, qep_tab=0x7ffe7c0078d0,
end_of_records=false)
    at /root/mysqlall/percona-server-locks-detail-
5.7.22/sql/sql_executor.cc:2925
#11 0x0000000001582059 in evaluate_join_record (join=0x7ffe7c007078,
qep_tab=0x7ffe7c007758)
```




delete 语句引发大量 sql 被 kill 问题分析

作者：王航威

1 现象

某个数据库经常在某个时间点，比如凌晨 2 点或者白天某些时间段发出如下报警：

```
[Critical][prod][mysql] - 超 200 kill SQL/分钟
[P0][PROBLEM][all(#2) db_data.Com_kill db=XXXX[m]:3306 10.53333>=3.3]
[O1 2019-11-01 03:40:00]
```

报警的意思是每分钟超过 200 个 SQL 被 kill，是一个严重告警级别，会打电话给 DBA。大半夜报警的确令人不爽，那么如何解决这个问题呢？通过检查日志，我们发现被 kill 的 SQL 都是 delete 语句。业务方其实会定时的跑删除任务，这个任务涉及到 N 多个表，删除任务持续时间比较长，所以白天和晚上都有一定概率会触发 sql-killer，然后报警。

在有赞的数据库运维体系中，每个实例都会配置一个 sql-killer 的实时工具，用于 kill query 超过指定阈值的 SQL 请求（类似 pt-killer）。

2 初步分析

在之前的案例分析过程中，碰到过因为长事务导致特定表上面的查询耗时增加的问题。经分析发现，这次被 kill 的 SQL 是分布在各个表上面，而且查询发现并不存在长事务。分析问题发生时候的数据库快照信息，QPS 都很低，除了差不多 10TPS 的 delete 和几十的 select，没有发现有问题的 SQL。

分析当时的 `show engine innodb status` 的信息，发现每次出问题的时候都会出现一些 latch 的等待，如下图所示：

```
OS WAIT ARRAY INFO: reservation count 37278184010
--Thread 139693697992448 has waited at ibuf0ibuf.cc line 4568 for 0.00 seconds the semaphore:
S-lock on RW-latch at 0x7f118d2c1590 created in file buf0buf.cc line 1456
a writer (thread id 139693937428224) has reserved it in mode wait exclusive
number of readers 1, waiters flag 1, lock_word: ffffffff
Last time read locked in file ibuf0ibuf.cc line 3454
Last time write locked in file /usr/.../BUILD/percona-server-5.7.22-22/storage/innobase/btr/btr0btr.cc line 177
--Thread 139693614065408 has waited at ibuf0ibuf.cc line 4568 for 0.00 seconds the semaphore:
S-lock on RW-latch at 0x7f118d2c1590 created in file buf0buf.cc line 1456
a writer (thread id 139693937428224) has reserved it in mode wait exclusive
number of readers 1, waiters flag 1, lock_word: ffffffff
Last time read locked in file ibuf0ibuf.cc line 3454
Last time write locked in file /usr/.../BUILD/percona-server-5.7.22-22/storage/innobase/btr/btr0btr.cc line 177
--Thread 139693664421632 has waited at btr0cur.cc line 990 for 0.00 seconds the semaphore:
SX-lock on RW-latch at 0x7f0cd7aea4a8 created in file dict0dict.cc line 2744
a writer (thread id 139693689599744) has reserved it in mode SX
number of readers 6, waiters flag 1, lock_word: ffffffff
Last time read locked in file btr0cur.cc line 1022
Last time write locked in file /data/user/.../BUILD/percona-server-5.7.22-22/storage/innobase/btr/btr0cur.cc line 990
--Thread 139693798704896 has waited at ibuf0ibuf.cc line 4568 for 0.00 seconds the semaphore:
S-lock on RW-latch at 0x7f118d2c1590 created in file buf0buf.cc line 1456
a writer (thread id 139693937428224) has reserved it in mode wait exclusive
number of readers 1, waiters flag 1, lock_word: ffffffff
Last time read locked in file ibuf0ibuf.cc line 3454
```

找到对应的代码行数

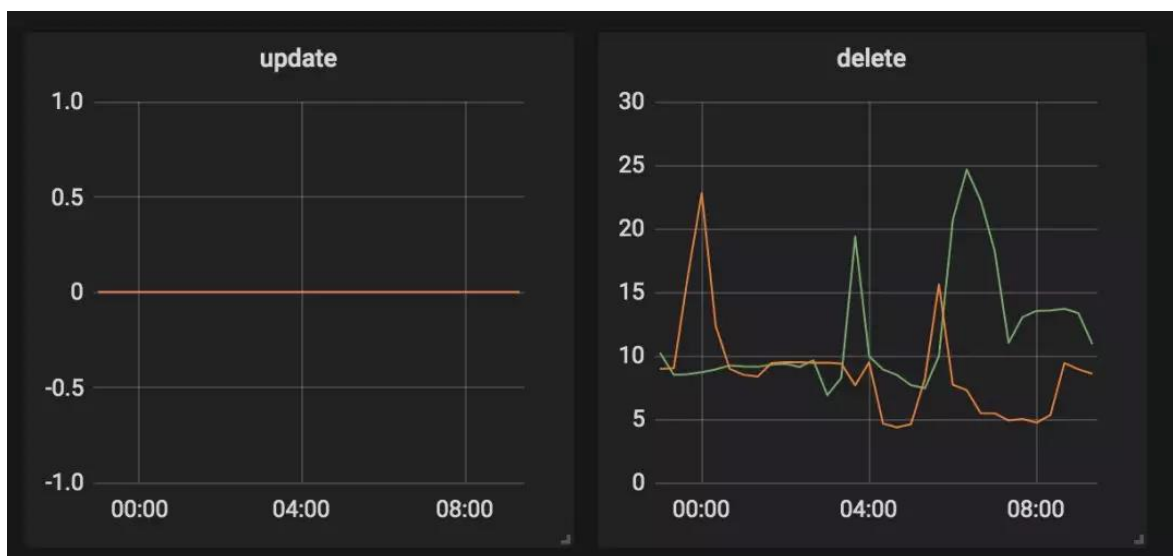
```
160
161  /*****
162  Gets the root node of a tree and x- or s-latches it.
163  @return root page, x- or s-latched */
164  buf_block_t*
165  btr_root_block_get(
166  /*=====*/
167      const dict_index_t* index, /*!< in: index tree */
168      ulint                mode, /*!< in: either RW_S_LATCH
169                                or RW_X_LATCH */
170      mtr_t*               mtr) /*!< in: mtr */
171  {
172      const ulint    space = dict_index_get_space(index);
173      const page_id_t page_id(space, dict_index_get_page(index));
174      const page_size_t page_size(dict_table_page_size(index->table));
175
176      buf_block_t* block = btr_block_get(page_id, page_size, mode,
177                                       index, mtr);
178
179      SRV_CORRUPT_TABLE_CHECK(block, return(0));
180
181      btr_assert_not_corrupted(block, index);
182  #ifdef UNIV_BTR_DEBUG
183      if (!dict_index_is_ibuf(index)) {
184          const page_t* root = buf_block_get_frame(block);
```

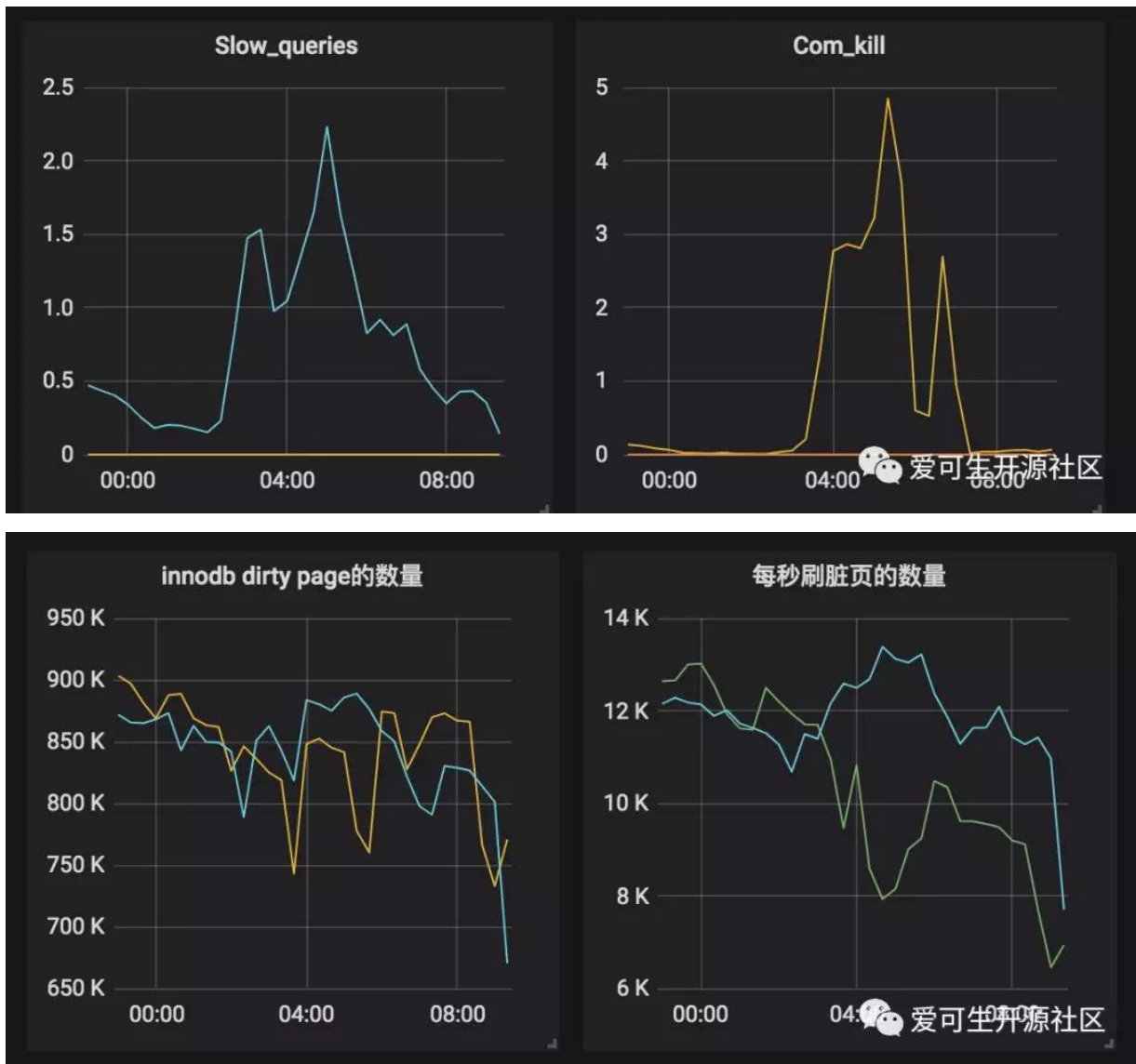
爱可生开源社区

看代码锁位置像是在等待各种 Buffer Pool 的各种 latch。为啥会等待在这里呢，又没有 DDL 相关的 SQL，于是百思不得其解。问题诊断一时间陷入困境。

3 抽丝剥茧

由于等待和 Buffer Pool 的各种 latch 相关，而且 delete 操作本身会产生大量脏数据，那会不会跟刷脏页操作相关呢？我们看下 SQL 被 kill 的量和刷脏页的量之间的关系：





发现每秒刷脏页的量和 SQL 被 kill 的量的曲线有点相近，看着刷脏页的量挺大的，但是每秒 delete 的 TPS 又不是很高，为啥这么低的 TPS 会让刷脏页频率抖动以及 SQL 执行变慢呢？



曾经换过不同批次的机器，发现问题依旧，并没有改善，说明并不是机器本身的问题。继续浏览 buffer pool 相关的监控指标，像是发现新大陆一样的发现了一个异常指标：脏页比例达到了快 90%!!!

为啥脏页比例会达到 90% 呢，无非就是刷脏页的速度跟不上产生的速度，要么就是 IO 能力不行，要么就是产生脏页的速度过快，要么就是内存池太小，导致 Buffer Pool 被脏页占满。

那么这个脏页比例达到快 90%会有什么问题呢？MySQL 有两个关于脏页的参数

```
# yzsql    3306 param dirty
Variable_name  Value
innodb_max_dirty_pages_pct      75.000000
innodb_max_dirty_pages_pct_lwm  50.000000
```

我们查看下官方定义 innodb_max_dirty_pages_pct:

InnoDB tries to flush data from the buffer pool so that the percentage of dirty pages does not exceed this value. The default value is 75.

innodb_max_dirty_pages_pct 是为了避免脏页比例大于 75%，改变该参数不会影响刷脏页的速度。
innodb_max_dirty_pages_pct_lwm:

Defines a low water mark representing the percentage of dirty pages at which preflushing is enabled to control the dirty page ratio. The default of 0 disables the pre-flushing behavior entirely.

innodb_max_dirty_pages_pct_lwm 表示的是当脏页比例达到该参数表示的低水位时候，刷脏线程就开始预刷脏来控制脏页比例，避免达到 innodb_max_dirty_pages_pct。刷脏页的最大 IO 能力是受 innodb_io_capacity 和 innodb_io_capacity_max 控制。

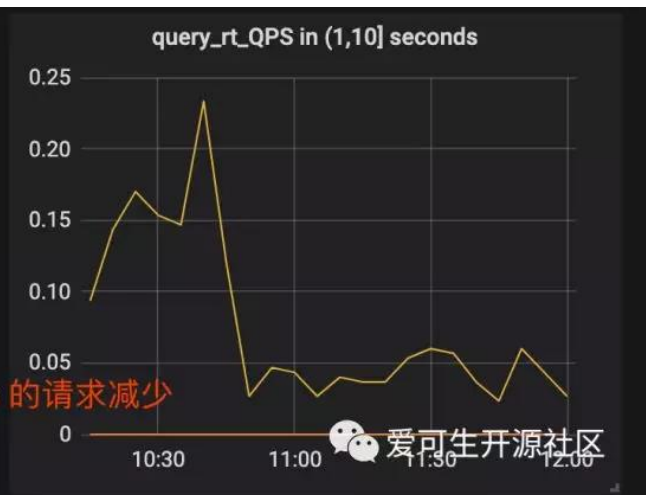
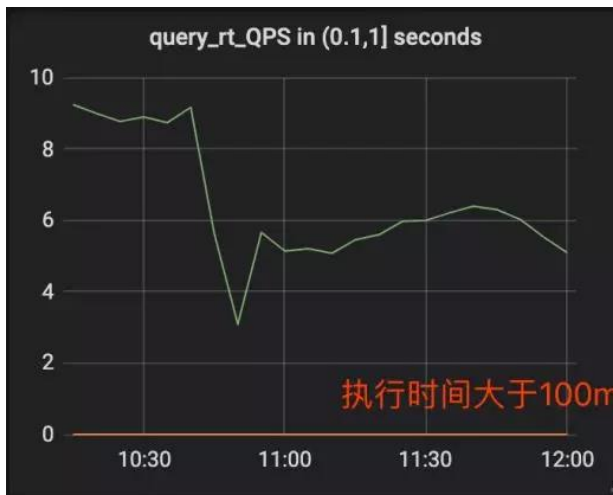
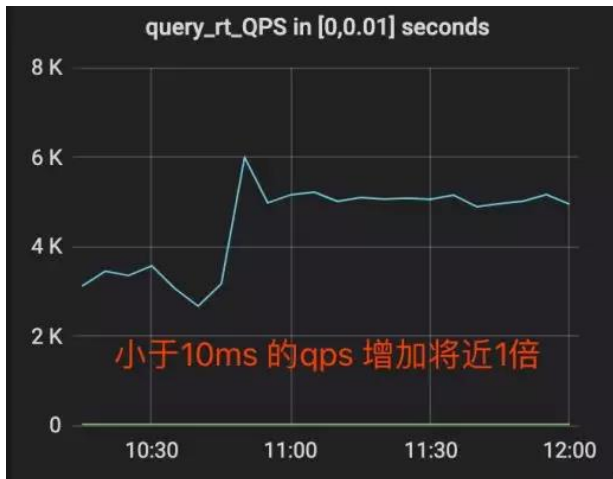
生产上我们将 innodb_max_dirty_pages_pct_lwm 设置成了 5

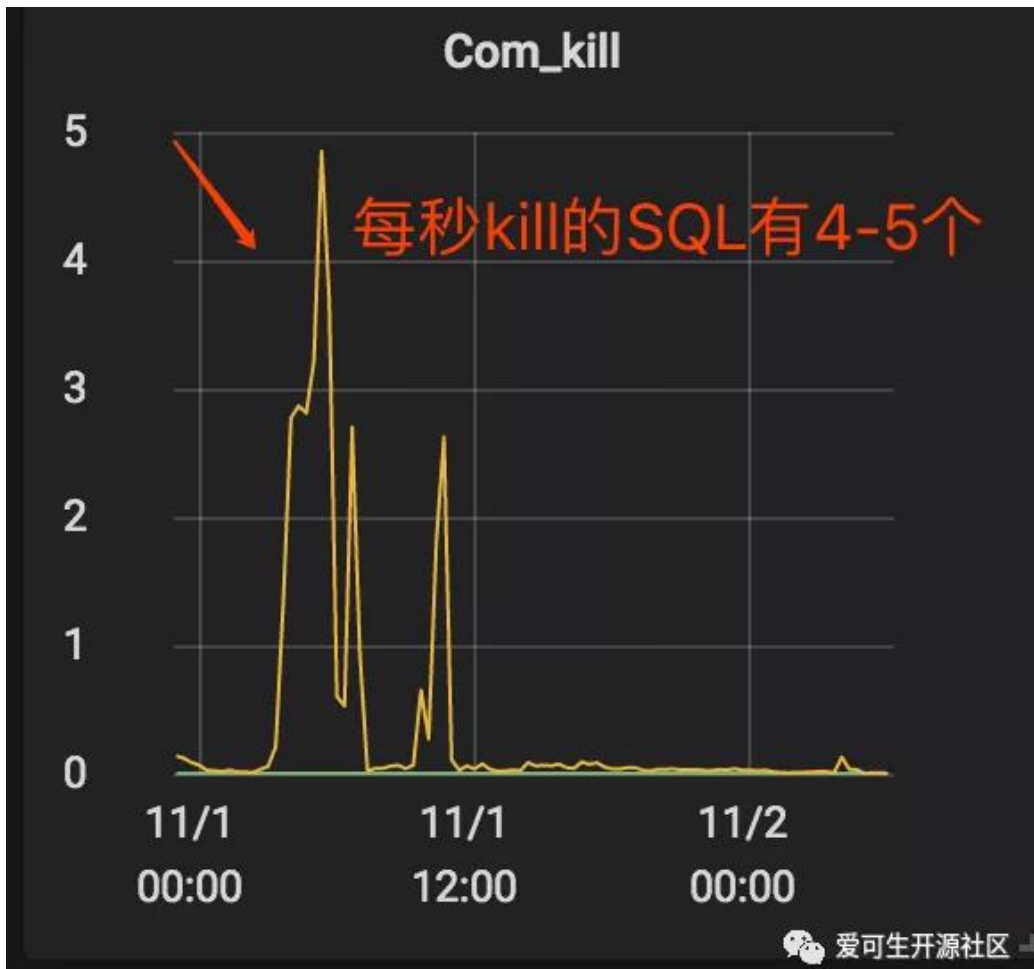
当脏页比例大于 innodb_max_dirty_pages_pct 时候，InnoDB 会进行非常激烈的刷脏页操作，但是由于 DELETE 操作还是在进行，脏页产生的速度还是非常快，刷脏页的速度还是跟不上脏页产生的速度。为了避免脏页比例进一步扩大，更新将会被堵塞，从而导致 DELETE 执行变慢，直至被 KILL。

发现问题之后，根据我们之前的假设，有三种解决方案：

1. 调大 io_capacity，但是由于主机是多实例部署，IO 占用已经比较高，PASS。
2. 降低脏页产生速度，也就是调低 DELETE 速度，因为数据产生的速度很快，为了避免删除跟不上插入的速度，也被 PASS。
3. 调大 Buffer Pool，可以容纳更多的脏页。

说干就干，得益于 MySQL 5.7 的在线调整 Buffer Pool，立马将 Buffer Pool Size 扩了一倍，效果非常显著。





脏页比例立马下降，被 kill 的 SQL 也下降了。平均 SQL rt 下降很多。

4 总结

得益于 MySQL 的开源，很多错误都可以直接确认到对应的代码，大致定位到问题发生的地方，给问题排查带来了很大方便。同时对 MySQL buffer pool 的命中率以及脏页比例也要多多关注，对 SQL 的性能都有很大的影响。



MGR 相同 GTID 产生不同 transaction 故障分析

作者：王福祥

MGR 作为 MySQL 原生的高可用方案，它的基于共识协议的同步和决策机制，看起来也更为先进。吸引了一票用户积极尝试，希望通过 MGR 架构解决 RPO=0 的高可用切换。在实际使用中经常会遇到因为网络抖动的问题造成集群故障，最近我们某客户就遇到了这类问题，导致数据不一致。

1 问题现象

这是在生产环境中一组 MGR 集群，单主架构，我们可以看到在相同的 GTID86afb16f-1b8c-11e8-812f-0050568912a4:57305280 下，本应执行相同的事务，但 binlog 日志显示不同事务信息。

Primary 节点 binlog:

```
SET @@SESSION.GTID_NEXT= '86afb16f-1b8c-11e8-812f-0050568912a4:57305280'/*!*/;
# at 637087357
#190125 15:02:55 server id 3136842491 end_log_pos 637087441 Query
thread_id=19132957 exec_time=0 error_code=0
SET TIMESTAMP=1548399775/*!*/;
BEGIN
/*!*/;
# at 637087441
#190125 15:02:55 server id 3136842491 end_log_pos 637087514 Table_map:
`world`.`IC_WB_RELEASE` mapped to number 398
# at 637087514
#190125 15:02:55 server id 3136842491 end_log_pos 637087597 Write_rows: table id
398
flags: STMT_END_F
BINLOG '
n7RKXBP7avi6SQAABov+SUAAl4BAAAAAEAB2ljZW50ZXIAFUlDX1FVRVJZX1VTRVJDQVJEX0xP
'/*!*/;
### INSERT INTO `world`.`IC_WB_RELEASE`
### SET
```

Secondary 节点 binlog:

```
SET @@SESSION.GTID_NEXT= '86afb16f-1b8c-11e8-812f-0050568912a4:57305280'/*!*/;
# at 543772830
#190125 15:02:52 server id 3136842491 end_log_pos 543772894 Query
thread_id=19162514 exec_time=318 error_code=0
SET TIMESTAMP=1548399772/*!*/;
```

```
BEGIN
/*!*/;
# at 543772894
#190125 15:02:52 server id 3136842491 end_log_pos 543772979 Table_map:
`world`.`IC_QUERY_USERCARD_LOG` mapped to number 113
# at 543772979
#190125 15:02:52 server id 3136842491 end_log_pos 543773612 Delete_rows: table
id
113 flags: STMT_END_F
BINLOG '
nLRKXBP7avi6VQAAADNRaSAAAHEAAAAAAAEAB2ljZW50ZXIADUlDX1dCX1JFTEVBUEUACw8PEg8
'/*!*/;
### DELETE FROM `world`.`IC_QUERY_USERCARD_LOG`
### WHERE
```

从以上信息可以推测，primary 节点在这个 GTID 下对 world.IC_WB_RELEASE 表执行了 insert 操作事件没有同步到 secondary 节点，secondary 节点收到主节点的其他事件，造成了数据是不一致的。当在表 IC_WB_RELEASE 发生 delete 操作时，引发了下面的故障，导致从节点脱离集群。

```
2019-01-28T11:59:30.919880Z 6 [ERROR] Slave SQL for channel 'group_replication_ap
plier': Could not execute Delete_rows event on table `world`.`IC_WB_RELEASE`; Can
't find record in 'IC_WB_RELEASE', Error_code: 1032; handler error HA_ERR_KEY_NOT
_FOUND, Error_code: 1032
2019-01-28T11:59:30.919926Z 6 [Warning] Slave: Can't find record in 'IC_WB_RELEAS
E' Error_code: 1032
2019-01-28T11:59:30.920095Z 6 [ERROR] Plugin group_replication reported: 'The app
lier thread execution was aborted. Unable to process more transactions, this membe
r will now leave the group.'
2019-01-28T11:59:30.920165Z 6 [ERROR] Error running query, slave SQL thread abort
ed. Fix the problem, and restart the slave SQL thread with "SLAVE START". We stopp
ed at log 'FIRST' position 271.
2019-01-28T11:59:30.920220Z 3 [ERROR] Plugin group_replication reported: 'Fatal e
rror during execution on the Applier process of Group Replication. The server will
now leave the group.'
2019-01-28T11:59:30.920298Z 3 [ERROR] Plugin group_replication reported: 'The ser
ver was automatically set into read only mode after an error was detected.'
```

2 问题分析

1. 主节点在向从节点同步事务时，至少有一个 GTID 为 86afb16f-1b8c-11e8-812f-0050568912a4:57305280（执行的是 insert 操作）的事务没有同步到从节点，此时从实例还不存在这个 GTID；于是主实例 GTID 高于从实例。数据就已经不一致了。
2. 集群业务正常进行，GTID 持续上涨，新上涨的 GTID 同步到了从实例，占用了 86afb16f-1b8c-11e8-812f-0050568912a4:57305280 这个 GTID，所以从实例没有执行 insert 操作，少了一部分数据。
3. 主节点对 GTID 为 86afb16f-1b8c-11e8-812f-0050568912a4:57305280 执行的 insert 数据进行 delete，而从节点由于没有同步到原本的 insert 操作；没有这部分数据就不能 delete，于是脱离了

集群。

对于该故障的分析，我们要从主从实例 GTID 相同，但是事务不同的原因入手，该问题猜测与 bug (<https://bugs.mysql.com/bug.php?id=92690>) 相关，我们针对 MGR 同步事务的时序做如下分析。

3 相关知识背景

MGR 全组同步数据的 Xcom 组件是基于 paxos 算法的实现；每当提交新生事务时，主实例会将新生事务发送到从实例进行协商，组内协商通过后全组成员一起提交该事务；每一个节点都以同样的顺序，接收到了同样的事务日志，所以每一个节点以同样的顺序回放了这些事务，最终组内保持了一致的状态。

3.1 paxos 包括两个角色：

1. 提议者 (Proposer)：主动发起投票选主，允许发起提议。
2. 接受者 (Acceptor)：被动接收提议者 (Proposer) 的提议，并记录和反馈，或学习达成共识的提议。

3.2 paxos 达成共识的过程包括两个阶段：

第一阶段 (prepare)

a: 提议者 (Proposer) 发送 prepare 请求，附带自己的提案标识 (ballot，由数值编号加节点编号组成) 以及 value 值。组成员接收 prepare 请求；

b: 如果自身已经有了确认的值，则将该值以 ack_prepare 形式反馈；在这个阶段中，Proposer 收到 ack 回复后会对比 ballot 值，数值大的 ballot 会取代数值小的 ballot。如果收到大多数应答之后进入下一个阶段。

第二阶段 (accept)

a: 提议者 (Proposer) 发送 accept 请求

b: 接收者 (Acceptor) 收到请求后对比自身之前的 ballot 是否相同以及是否接收过 value 值。如果未接受过 value 值 以及 ballot 相同，则返回 ack_accept，如果接收过 value 值，则选取最大的 ballot 返回 ack_accept。

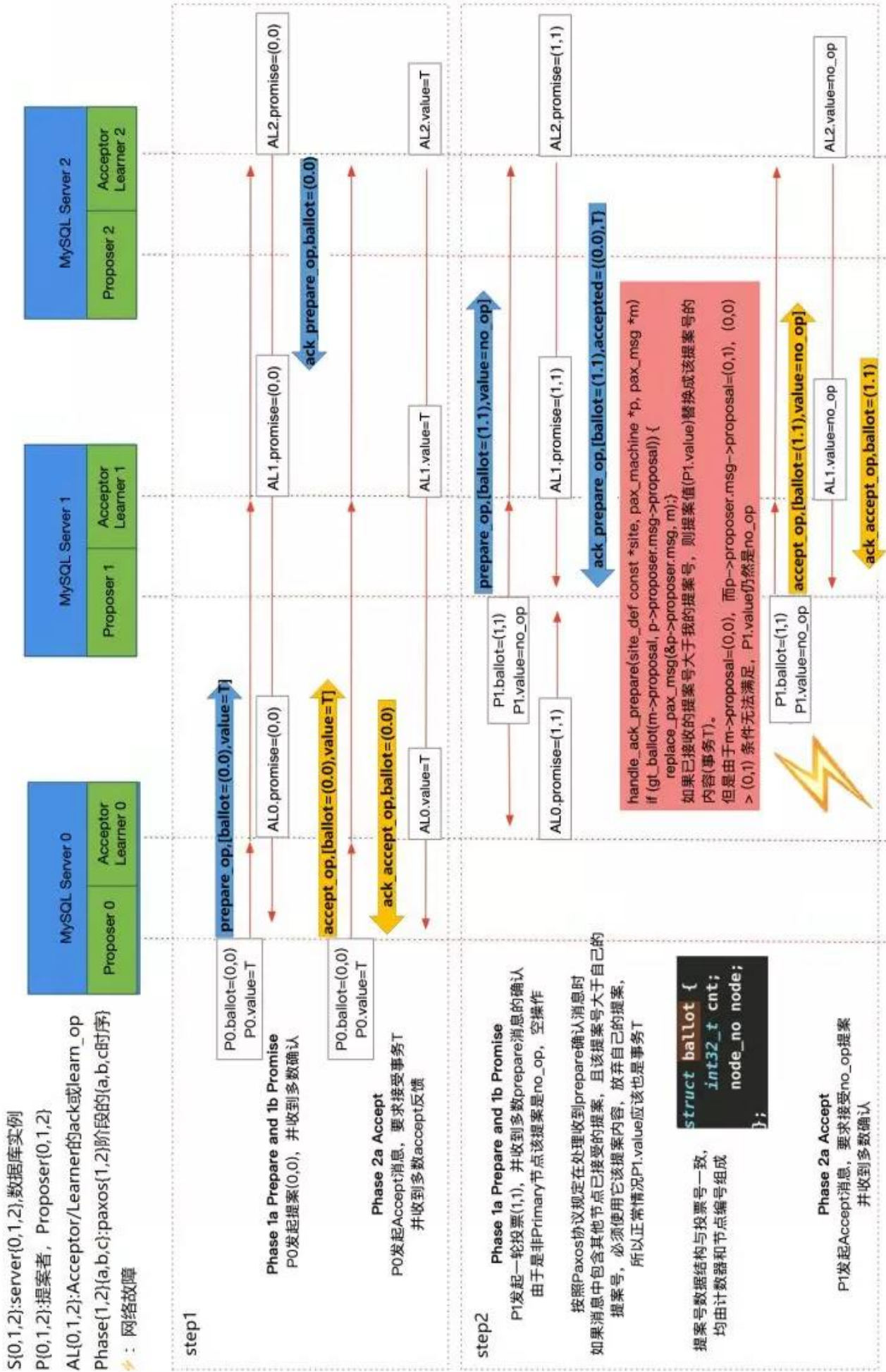
c: 之后接受相同 value 值的 Proposer 节点发送 learn_op，收到 learn_op 节点的实例表示确认了数据修改，传递给上层应用。

针对本文案例我们需要强调几个关键点：

1. 该案例最根本的异常对比发生在第二次提案的 prepare 阶段。
2. prepare 阶段的提案标识由数值编号和节点编号两部分组成；其中数值编号类似自增长数值，而节点编号不变。

4 分析过程

结合 paxos 时序，我们对案例过程进行推测：



Tips: 以下分析过程请结合时序图操作步骤观看

1. **【step1】** primary 节点要执行对表 world.IC_WB_RELEASE 的 insert 操作，向组内发送假设将 ballot 设置为 (0.0) 以及将 value 值 world.IC_WB_RELEASE 的 prepare 请求，并收到大多数成员的 ack_prepare 返回，于是开始发送 accept 请求。primary 节点将 ballot (0.0) 的提案信息发送至组内，仍收到了大多数成员 ack_accept (ballot=0.0value=world.IC_WB_RELEASE) 返回。然后发送 learn_op 信息 **【step3】**。
2. 同时其他从节点由于网络原因没有收到主实例的 learn_op 信息 **【step3】**，而其中一台从实例开始新的 prepare 请求 **【step2】**，请求 value 值为 no_op (空操作) ballot=1.1 (此编号中节点编号是关键，该 secondary 节点编号大于 primary 节点编号，导致了后续的问题，数值编号无论谁大谁小都要被初始化)。

其他的从实例由于收到过主节点的 value 值；所以将主节点的 (ballot=0.0, value=world.IC_WB_RELEASE) 返回；而收到的 ack_prepare 的 ballot 值的数值符号全组内被初始化为 0，整个 ballot 的大小完全由节点编号决定，于是从节点选取了 ballot 较大的该实例 value 值作为新的提案，覆盖了主实例的 value 值并收到大多数成员的 ack_accept **【step2】**。并在组成员之间发送了 learn_op 信息 **【step3】**，跳过了主实例提议的事务。

从源码中可以看到关于 handle_ack_prepare 的逻辑。

handle_ack_prepare has the following code:

```
if (gt_ballot(m->proposal,p->proposer.msg->proposal))
{
    replace_pax_msg(&p->proposer.msg, m);
    ...
}
```

3. 此时，主节点在 accept 阶段收到了组内大多数成员的 ack_accept 并收到了自己所发送的 learn_op 信息，于是把自己的提案（也就是 binlog 中对表的 insert 操作）提交 **【step3】**，该事务 GTID 为 86afb16f-1b8c-11e8-812f-0050568912a4:57305280。而其他节点的提案为 no_op **【step3】**，所以不会提交任何事务。此时主实例 GTID 大于其他从实例的。
4. 主节点新生 GTID 继续上涨；同步到从实例，占用了从实例的 86afb16f-1b8c-11e8-812f-0050568912a4:57305280 这个 GTID，于是产生了主节点与从节点 binlog 中 GTID 相同但是事务不同的现象。
5. 当业务执行到对表 world.IC_WB_RELEASE 的 delete 操作时，主实例能进行操作，而其他实例由于没有 insert 过数据，不能操作，就脱离了集群。

5 过程总结:

1. 旧主发送 prepare 请求，收到大多数 ack，进入 accept 阶段，收到大多数 ack。
2. 从实例由于网络原因没有收到 learn_op 信息。
3. 其中一台从实例发送新的 prepare 请求，value 为 no_op。
4. 新一轮的 prepare 请求中，提案标识的数值编号被初始化，新的提案者大于主实例，从实例选取新

提案，执行空操作，不写入 relay-log。代替了主实例的事务，使主实例 GTID 大于从实例。

5. 集群网络状态恢复，新生事物正常同步到从实例，占用了本该正常同步的 GTID，使集群中主节点与从节点相同 GTID 对应的事务时不同的。

6 结论

针对此问题我们也向官方提交 SR，官方已经反馈社区版 MySQL 5.7.26 和 MySQL 8.0.16 中会修复，如果是企业版客户可以申请最新的 hotfix 版本。

在未升级 MySQL 版本前，若再发生此类故障，在修复时需要人工检查，检查切换时 binlog 中 GTID 信息与新主节点对应 GTID 的信息是否一致。如果不一致需要人工修复至一致状态，如果一致才可以将被踢出的原主节点加回集群。

所以正在使用了 MGR 5.7.26 之前社区版的 DBA 同学请注意避坑。

技术分享

本系列的文章是全书最多的一个部分，共收录 23 篇，作者 13 位。就像一个百宝箱一样，内容涉及到优化、数据恢复、事务锁等基础概念、Oracle 迁移、复制功能、数据库设计、好用的 MySQL 工具等等等等。给读者一种百花齐放，百家争鸣的感觉。每一篇文章都围绕这一个知识点展开，有大量的代码、操作命令和配图说明，展示出了不同的分享者的不同写作风格。

微信扫一扫阅读原文



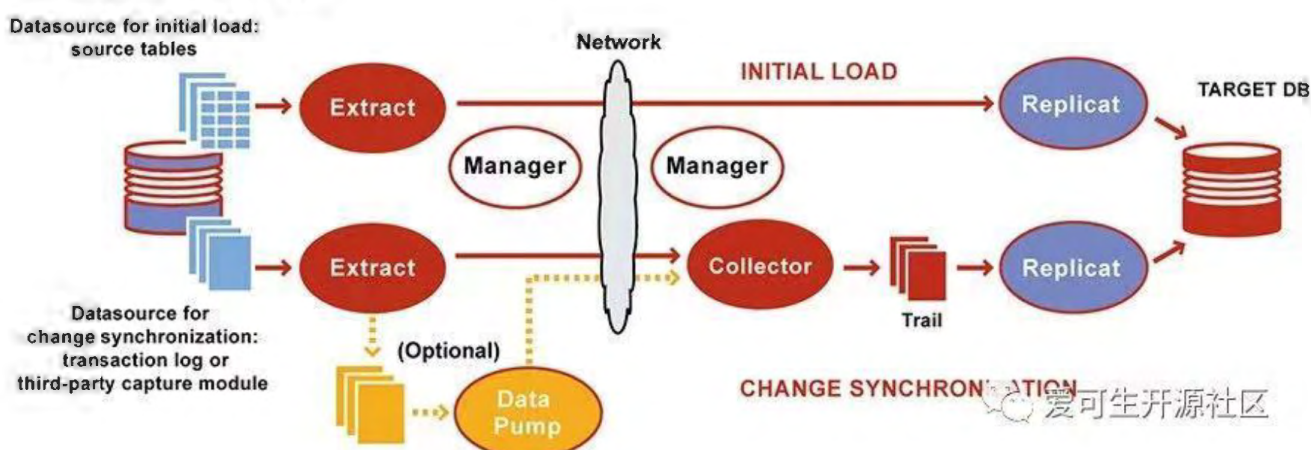
使用 OGG 实现 Oracle 到 MySQL 数据平滑迁移

作者：杜开生

1 OGG 概述

OGG 全称为 Oracle GoldenGate, 是由 Oracle 官方提供的用于解决异构数据环境中数据复制的一个商业工具。相比于其它迁移工具 OGG 的优势在于可以直接解析源端 Oracle 的 redo log, 因此能够实现在不需要对原表结构做太多调整的前提下完成数据增量部分的迁移。本篇文章将重点介绍如何使用 OGG 实现 Oracle 到 MySQL 数据的平滑迁移, 以及讲述个人在迁移过程中所碰到问题的解决方案。

1.1 OGG 逻辑架构



参照上图简单给大家了解下 OGG 逻辑架构, 让大家对 OGG 数据同步过程有个简单了解, 后面章节会详细演示相关进程的配置方式, 在 OGG 使用过程中主要涉及以下进程及文件:

1. Manager 进程: 需要源端跟目标端同时运行, 主要作用是监控管理其它进程, 报告错误, 分配及清理数据存储空间, 发布阈值报告等

2. Extract 进程：运行在数据库源端，主要用于捕获数据的变化，负责全量、增量数据的抽取
3. Trails 文件：临时存放在磁盘上的数据文件
4. Data Pump 进程：运行在数据库源端，属于 Extract 进程的一个辅助进程，如果不配置 Data Pump，Extract 进程会将抽取的数据直接发送到目标端的 Trail 文件，如果配置了 Data Pump，Extract 进程会将数据抽取到本地 Trail 文件，然后通过 Data Pump 进程发送到目标端，配置 Data Pump 进程的主要好处是即使源端到目标端发生网络中断，Extract 进程依然不会终止
5. Collector 进程：接收源端传输过来的数据变化，并写入本地 Trail 文件中
6. Replicat 进程：读取 Trail 文件中记录的数据变化，创建对应的 DML 语句并在目标端回放

2 迁移方案

2.1 环境信息

软件名称	源端	目标端
OGG版本	OGG 12.2.0.2.2 For Oracle	OGG 12.2.0.2.2 For MySQL
数据库版本	Oracle 11.2.0.4	MySQL 5.7.27
OGG_HOME	/home/oracle/ogg	/opt/ogg
IP地址	17X.1X.84.124	17X.1X.84.121
数据库	oms	oms

2.2 表结构迁移

表结构迁移属于难度不高但内容比较繁琐的一步。我们在迁移表结构时使用了一个叫 sqlines 的开源工具，对于 sqlines 工具在 MySQL 端创建失败及不符合预期的表结构再进行特殊处理，以此来提高表结构转换的效率。

注意：OGG 在 Oracle 迁移 MySQL 的场景下不支持 DDL 语句同步，因此表结构迁移完成后到数据库切换前尽量不要再修改表结构。

2.3 数据迁移

数据同步的操作均采用 OGG 工具进行，考虑数据全量和增量的衔接，OGG 需要先将增量同步的抽取进程启动，抓取数据库的 redo log，待全量抽取结束后开启增量数据回放，应用全量和增量这期间产生的日志数据，OGG 可基于参数配置进行重复数据处理，所以使用 OGG 时优先将增量进行配置并启用。此外，为了避免本章节篇幅过长，OGG 参数将不再解释，有需要的朋友可以查看官方提供的 Reference 文档查询任何你不理解的参数。

2.3.1 源端 OGG 配置

(1) Oracle 数据库配置

针对 Oracle 数据库，OGG 需要数据库开启归档模式及增加辅助补充日志、强制记录日志等来保障 OGG 可抓取到完整的日志信息

查看当前环境是否满足要求，输出结果如下图所示：

```
SQL> SELECT NAME,LOG_MODE,OPEN_MODE,PLATFORM_NAME,FORCE_LOGGING,SUPPLEMENTAL_LOG
_DATA_MIN FROM V$DATABASE;
```

Row 1	Fields	Comments
NAME	APPDB	
LOG_MODE	NOARCHIVELOG	
OPEN_MODE	READ WRITE	
PLATFORM_NAME	Linux x86 64-bit	
FORCE_LOGGING	YES	
SUPPLEMENTAL_LOG_DATA_MIN	YES	爱可生开源社区

如果条件不满足则实行该部分，

开启归档(开启归档需要重启数据库)

```
SQL> shutdown immediate;
SQL> startup mount;
SQL> alter database archivelog;
SQL> alter database open;
SQL> archive log list;
```

开启附加日志和强制日志

```
SQL> alter database add supplemental log data;
SQL> alter database force logging;
SQL> alter system switch logfile;
```

启用 OGG 支持

```
SQL> show parameter enable_goldengate_replication
SQL> alter system set enable_goldengate_replication=true;
```

再次查看当前环境是否满足要求

```
SQL> SELECT
NAME,LOG_MODE,OPEN_MODE,PLATFORM_NAME,FORCE_LOGGING,SUPPLEMENTAL_LOG_DATA_MIN
FROM V$DATABASE;
```

(2) Oracle 数据库 OGG 用户创建

OGG 需要有一个用户有权限对数据库的相关对象做操作，以下为涉及的权限，该示例将创建一个用户名和密码均为 ogg 的 Oracle 数据库用户并授予以下权限

查看当前数据库已存在的表空间，可使用已有表空间或新建单独的表空间

```
SQL> SELECT TABLESPACE_NAME, CONTENTS FROM DBA_TABLESPACES;
```

查看当前表空间文件数据目录

```
SQL> SELECT NAME FROM V$DATAFILE;
```

创建一个新的 OGG 用户的表空间

```
SQL> CREATE TABLESPACE OGG_DATA DATAFILE
'/u01/app/oracle/oradata/cms/ogg_data01.dbf' SIZE 2G;
```

创建 ogg 用户且指定对应表空间并授权

```
SQL> CREATE USER ogg IDENTIFIED BY ogg DEFAULT TABLESPACE OGG_DATA;
```

```
SQL> grant connect,resource,unlimited tablespace to ogg;
```

```
SQL> grant create session,alter session to ogg;
```

```
SQL> grant execute on utl_file to ogg;
```

```
SQL> grant select any dictionary, select any table to ogg;
```

```
SQL> grant alter any table to ogg;
```

```
SQL> grant flashback any table to ogg;
```

```
SQL> grant select any transaction to ogg;
```

```
SQL> grant sysdba to ogg;
```

```
SQL> grant execute on dbms_streams_adm to ogg;
```

```
SQL> grant execute on dbms_flashback to ogg;
```

```
SQL> exec dbms_goldengate_auth.grant_admin_privilege('OGG');
```

(3) 源端 OGG 管理进程(MGR)配置

切换至 ogg 软件目录并执行 ggsci 进入命令行终端

```
shell> cd $OGG_HOME
```

```
shell> ggsci
```

编辑/创建 mgr 配置文件

```
ggsci> edit params mgr
```

```
PORT 7809
```

```
DYNAMICPORTLIST 8000-8050
```

```
-- AUTOSTART extract
```

```
-- AUTORESTART extract,retries 4,waitminutes 4
```

```
STARTUPVALIDATIONDELAY 5
```

```
ACCESSRULE, PROG *, IPADDR 17X.1X.*, ALLOW
```

```
ACCESSRULE, PROG SERVER, ALLOW
```

```
PURGEOLDEXTRACTS /home/oracle/ogg/dirdat/*, USECHECKPOINTS,MINKEEPFILES 3
```

启动并查看 mgr 状态

```
ggsci> start mgr
```

```
ggsci> info all
```

```
ggsci> view report mgr
```

(4) 源端 OGG 表级补全日志(trandata)配置

表级补全日志需要在最小补全日志打开的情况下才起作用,之前只在数据库级开启了最小补全日志(alter database add supplemental log data;),redo log 记录的信息还不够全面,必须再使用 add trandata 开启表级的补全日志以获得必要的信息。

```
#### 使用 ogg 用户登录 appdb 实例 (TNS 配置) 对 cms 所有表增加表级补全日志
shell> cd $OGG_HOME
shell> ggsci
ggsci> dblogin userid ogg@appdb,password ogg
ggsci> add trandata cms.*
```

(5) 源端 OGG 抽取进程(extract)配置

Extract 进程运行在数据库源端,负责从源端数据表或日志中捕获数据。Extract 进程利用其内在的 checkpoint 机制,周期性地检查并记录其读写的位置,通常是写入到本地的 trail 文件。这种机制是为了保证如果 Extract 进程终止或者操作系统宕机,我们重启 Extract 进程后,GoldenGate 能够恢复到以前的状态,从上一个断点处继续往下运行,而不会有任何数据损失。

```
#### 切换至 ogg 软件目录并执行 ggsci 进入命令行终端
shell> cd $OGG_HOME
shell> ggsci
```

```
#### 创建一个新的增量抽取进程,从 redo 日志中抽取数据
ggsci> add extract e_cms,tranlog,begin now
```

```
#### 创建一个抽取进程抽取的数据保存路径并与新建的抽取进程进行关联
ggsci> add exttrail /home/oracle/ogg/dirdat/ms,extract e_cms,megabytes 1024
```

```
#### 创建抽取进程配置文件
ggsci> edit params e_cms
extract e_cms
setenv (NLS_LANG = "AMERICAN_AMERICA.AL32UTF8")
setenv (ORACLE_HOME = "/data/oracle/11.2/db_1")
setenv (ORACLE_SID = "cms")
userid ogg@appdb,password ogg
discardfile /home/oracle/ogg/dirrpt/e_cms.dsc,append,megabytes 1024
exttrail /home/oracle/ogg/dirdat/ms
statoptions reportfetch
reportcount every 1 minutes,rate
warnlongtrans 1H,checkinterval 5m
table cms.*;
```

```
#### 启动源端抽取进程
ggsci> start e_cms
ggsci> info all
ggsci> view report e_cms
```

(6) 源端 OGG 传输进程(pump)配置

pump 进程运行在数据库源端，其作用非常简单。如果源端的 Extract 抽取进程使用了本地 trail 文件，那么 pump 进程就会把 trail 文件以数据块的形式通过 TCP/IP 协议发送到目标端，Pump 进程本质上是 Extract 进程的一种特殊形式，如果不使用 trail 文件，那么 Extract 进程在抽取完数据后，直接投递到目标端。

补充：pump 进程启动时需要与目标端的 mgr 进程进行连接，所以需要优先将目标端的 mgr 提前配置好，否则会报错连接被拒绝，无法传输抽取的日志文件到目标端对应目录下

切换至 ogg 软件目录并执行 ggsci 进入命令行终端

```
shell> cd $OGG_HOME
```

```
shell> ggsci
```

增加一个传输进程与抽取进程抽取的文件进行关联

```
shell> add extract p_cms,exttrailsource /home/oracle/ogg/dirdat/ms
```

增加配置将抽取进程抽取的文件数据传输到远程对应目录下

注意 rmttrail 参数指定的是目标端的存放目录，需要目标端存在该目录路径

```
shell> add rmttrail /opt/ogg/dirdat/ms,extract p_cms
```

创建传输进程配置文件

```
shell>edit params p_cms
```

```
extract p_cms
```

```
setenv (NLS_LANG = "AMERICAN_AMERICA.AL32UTF8")
```

```
setenv (ORACLE_HOME = "/data/oracle/11.2/db_1")
```

```
setenv (ORACLE_SID = "cms")
```

```
userid ogg@appdb,password ogg
```

```
RMTHOST 17X.1X.84.121,MGRPORT 7809
```

```
RMTTRAIL /opt/ogg/dirdat/ms
```

```
discardfile /home/oracle/ogg/dirrpt/p_cms.dsc,append,megabytes 1024
```

```
table cms.*;
```

启动源端抽取进程

启动前确保目标端 mgr 进程已开启

```
ggsci> start p_cms
```

```
ggsci> info all
```

```
ggsci> view report p_cms
```

(7) 源端 OGG 异构 mapping 文件(defgen)生成

该文件记录了源库需要复制的表的表结构定义信息，在源库生成该文件后需要拷贝到目标库的 dirdef 目录，当目标库的 replica 进程将传输过来的数据 apply 到目标库时需要读写该文件，同构的数据库不需要进行该操作。

创建 mapping 文件配置

```
shell> cd $OGG_HOME
```

```
shell> vim ./dirprm/mapping_cms.prm
```

```
defsfile ./dirdef/cms.def,purge
userid ogg@appdb,password ogg
table cms.*;
```

基于配置生成 cms 库的 mapping 文件

默认生成的文件保存在\$OGG_HOME 目录的 dirdef 目录下

```
shell> ./defgen paramfile ./dirprm/mapping_cms.prm
```

将该文件拷贝至目标端对应的目录

```
shell> scp ./dirdef/cms.def root@17X.1X.84.121:/opt/ogg/dirdef
至此源端环境配置完成
```

2.3.2 目标端 OGG 配置

(1) 目标端 MySQL 数据库配置

- 确认 MySQL 端表结构已经存在
- MySQL 数据库 OGG 用户创建

```
mysql> create user 'ogg'@'%' identified by 'ogg';
mysql> grant all on *.* to 'ogg'@'%';
```

提前创建好 ogg 存放 checkpoint 表的数据库

```
mysql> create database ogg;
```

(2) 目标端 OGG 管理进程(MGR)配置

目标端的 MGR 进程和源端配置一样，可直接将源端配置方式在目标端重复执行一次即可，该部分不在赘述

(3) 目标端 OGG 检查点日志表(checkpoint)配置

checkpoint 表用来保障一个事务执行完成后，在 MySQL 数据库从有一张表记录当前的日志回放点，与 MySQL 复制记录 binlog 的 GTID 或 position 点类似。

切换至 ogg 软件目录并执行 ggsci 进入命令行终端

```
shell> cd $OGG_HOME
shell> ggsci
```

```
ggsci> edit param ./GLOBALS
checkpointtable ogg.ggs_checkpoint
```

```
ggsci> dblogin sourcedb ogg@17X.1X.84.121:3306 userid ogg
ggsci> add checkpointtable ogg.ggs_checkpoint
```

(4) 目标端 OGG 回放线程(replicat)配置

Replicat 进程运行在目标端，是数据投递的最后一站，负责读取目标端 Trail 文件中的内容，并将解析其解析为 DML 语句，然后应用到目标数据库中。

切换至 ogg 软件目录并执行 ggsci 进入命令行终端

```
shell> cd $OGG_HOME
```

```
shell> ggsci
```

添加一个回放线程并与源端 pump 进程传输过来的 trail 文件关联, 并使用 checkpoint 表确保数据不丢失

```
ggsci> add replicat r_cms,exttrail /opt/ogg/dirdat/ms,checkpointtable
ogg.ggs_checkpoint
```

增加/编辑回放进程配置文件

```
ggsci> edit params r_cms
```

```
replicat r_cms
```

```
targetdb cms@17X.1X.84.121:3306,userid ogg,password ogg
```

```
sourcedefs /opt/ogg/dirdef/cms.def
```

```
discardfile /opt/ogg/dirrpt/r_cms.dsc,append,megabytes 1024
```

```
HANDLECOLLISIONS
```

```
MAP cms.*,target cms.*;
```

注意: replicat 进程只需配置完成, 无需启动, 待全量抽取完成后再启动。

至此源端环境配置完成。

待全量数据抽取完毕后启动目标端回放进程即可完成数据准实时同步。

2.3.3 全量同步配置

全量数据同步为一次性操作, 当 OGG 软件部署完成及增量抽取进程配置并启动后, 可配置 1 个特殊的 extract 进程从表中抽取数据, 将抽取的数据保存到目标端生成文件, 目标端同时启动一个单次运行的 replicat 回放进程将数据解析并回放至目标数据库中。

(1) 源端 OGG 全量抽取进程(extract)配置

切换至 ogg 软件目录并执行 ggsci 进入命令行终端

```
shell> cd $OGG_HOME
```

```
shell> ggsci
```

增加/编辑全量抽取进程配置文件

其中 RMTFILE 指定抽取的数据直接传送到远端对应目录下

注意: RMTFILE 参数指定的文件只支持 2 位字符, 如果超过 replicat 则无法识别

```
ggsci> edit params ei_cms
```

```
SOURCEISTABLE
```

```
SETENV (NLS_LANG = "AMERICAN_AMERICA.AL32UTF8")
```

```
SETENV (ORACLE_SID=cms)
```

```
SETENV (ORACLE_HOME=/data/oracle/11.2/db_1)
```

```
USERID ogg@appdb,PASSWORD ogg
```

```
RMTHOST 17X.1X.84.121,MGRPORT 7809
```

```
RMTFILE /opt/ogg/dirdat/ms,maxfiles 100,megabytes 1024,purge
```

```
TABLE cms.*;
```

启动并查看抽取进程正常

```
shell>          nohup          ./extract          paramfile          ./dirprm/ei_cms.prm
reportfile ./dirrpt/ei_cms.rpt &
```

查看日志是否正常进行全量抽取

```
shell> tail -f ./dirrpt/ei_cms.rpt
```

(2) 目标端 OGG 全量回放进程(replicat)配置

切换至 ogg 软件目录并执行 ggsci 进入命令行终端

```
shell> cd $OGG_HOME
```

```
shell> ggsci
```

```
ggsci> edit params ri_cms
```

```
SPECIALRUN
```

```
END RUNTIME
```

```
TARGETDB cms@17X.1X.84.121:3306,USERID ogg,PASSWORD ogg
```

```
EXTFILE /opt/ogg/dirdat/ms
```

```
DISCARDFILE ./dirrpt/ri_cms.dsc,purge
```

```
MAP cms.*,TARGET cms.*;
```

启动并查看回放进程正常

```
shell>          nohup          ./replicat          paramfile          ./dirprm/ri_cms.prm
reportfile ./dirrpt/ri_cms.rpt &
```

查看日志是否正常进行全量回放

```
shell> tail -f ./dirrpt/ri_cms.rpt
```

3 数据校验

数据校验是数据迁移过程中必不可少的环节，本章节提供给几个数据校验的思路共大家参考，校验方式可以由以下几个角度去实现：

1. 通过 OGG 日志查看全量、增量过程中 discards 记录是否为 0 来判断是否丢失数据；
2. 通过对源端、目标端的表执行 count 判断数据量是否一致；
3. 编写类似于 pt-table-checksum 校验原理的程序，实现行级别一致性校验，这种方式优缺点特别明显，优点是能够完全准确对数据内容进行校验，缺点是需要遍历每一行数据，校验成本较高；
4. 相对折中的数据校验方式是通过业务角度，提前编写好数十个返回结果较快的 SQL，从业务角度抽样校验。

4 迁移问题处理

本章节将讲述迁移过程中碰到的一些问题及相应的解决方式。

4.1 MySQL 限制

在 Oracle 到 MySQL 的表结构迁移过程中主要碰到以下两个限制：

1. Oracle 端的表结构因为最初设计不严谨，存在大量的列使用 varchar(4000)数据类型，导致迁移到 MySQL 后超出行限制，表结构无法创建。由于 MySQL 本身数据结构的限制，一个 16K 的数据页最少要存储两行数据，因此单行数据不能超过 65,535 bytes，因此针对这种情况有两种解决方式：
 - 根据实际存储数据的长度，对超长的 varchar 列进行收缩；
 - 对于无法收缩的列转换数据类型为 text，但这在使用过程中可能导致一些性能问题；
2. 与第一点类似，在 Innodb 存储引擎中，索引前缀长度限制是 767 bytes，若使用 DYNAMIC、COMPRESSED 行格式且开启 innodb_large_prefix 的场景下，这个限制是 3072 bytes，即使用 utf8mb4 字符集时，最多只能对 varchar(768)的列创建索引；
3. 使用 ogg 全量初始化同步时，若存在外键约束，批量导入时由于各表的插入顺序不唯一，可能子表先插入数据而主表还未插入，导致报错子表依赖的记录不存在，因此建议数据迁移阶段禁用主外键约束，待迁移结束后再打开。

```
mysql>set global foreign_key_checks=off;
```

4.2 全量与增量衔接

HANDLECOLLISIONS 参数是实现 OGG 全量数据与增量数据衔接的关键，其实现原理是在全量抽取前先开启增量抽取进程，抓去全量应用期间产生的 redo log，当全量应用完成后，开启增量回放进程，应用全量期间的增量数据。使用该参数后增量回放 DML 语句时主要有以下场景及处理逻辑：

- 目标端不存在 delete 语句的记录，忽略该问题并不记录到 discardfile
- 目标端丢失 update 记录
 - 更新的是主键值，update 转换成 insert
 - 更新的键值是非主键，忽略该问题并不记录到 discardfile
- 目标端重复 insert 已存在的主键值，这将被 replicat 进程转换为 UPDATE 现有主键值的行

4.3 OGG 版本选择

在 OGG 版本选择上我们也根据用户的场景多次更换了 OGG 版本，最初因为客户的 Oracle 数据库版本为 11.2.0.4，因此我们在选择 OGG 版本时优先选择使用了 11 版本，但是使用过程中发现，每次数据抽取生成的 trail 文件达到 2G 左右时，OGG 报错连接中断，查看 RMTFILE 参数详细说明了解到 trail 文件默认限制为 2G，后来我们替换 OGG 版本为 12.3，使用 MAXFILES 参数控制生成多个指定大小的 trail 文件，回放时 Replicat 进程也能自动轮转读取 Trail 文件，最终解决该问题。但是如果不幸 Oracle 环境使用了 Linux 5 版本的系统，那么你的 OGG 需要再降一个小版本，最高只能使用 OGG 12.2。

4.4 无主键表处理

在迁移过程中还碰到一个比较难搞的问题就是当前 Oracle 端存在大量表没有主键。在 MySQL 中的表没有主键这几乎是不被允许的，因为很容易导致性能问题和主从延迟。同时在 OGG 迁移过程中表没有主键也会产生一些隐患，比如对于没有主键的表，OGG 默认是将这个一行数据中所有的列拼凑起来作为唯一键，但实际还是可能存在重复数据导致数据同步异常，Oracle 官方对此也提供了一个解决方案，通过对无主键表添加 GUID 列来作为行唯一标示，具体操作方式可以搜索 MOS 文档 ID 1271578.1 进行查看。

4.5 OGG 安全规则

- 报错信息

2019-03-08 06:15:22 ERROR OGG-01201 Error reported by MGR : Access denied.
错误信息含义源端报错表示为该抽取进程需要和目标端的 mgr 进程通讯，但是被拒绝，具体操作为：源端的 extract 进程需要与目标端 mgr 进行沟通，远程将目标的 replicat 进行启动，由于安全性现在而被拒绝连接。

■ 报错原因

在 Oracle OGG 11 版本后，增加了新特性安全性要求，如果需要远程启动目标端的 replicat 进程，需要在 mgr 节点增加访问控制参数允许远程调用

■ 解决办法

在源端和目标端的 mgr 节点上分别增加访问控制规则并重启

表示该 mgr 节点允许 (ALLOW) 10.186 网段 (IPADDR) 的所有类型程序 (PROG *) 进行连接访问
ACCESSRULE, PROG *, IPADDR 10.186.*.*, ALLOW

4.6 数据抽取方式

□ 报错信息

2019-03-15 14:49:04 ERROR OGG-01192 Trying to use RMTTASK on data types
which may be written as LOB chunks (Table: 'UNIONPAYCMS.CMS_OT_CONTENT_RTF').

□ 报错原因

根据官方文档说明，当前直接通过 Oracle 数据库抽取数据写到 MySQL 这种 initial-load 方式，不支持 LOBs 数据类型，而表 UNIONPAYCMS.CMSOTCONTENT_RTF 则包含了 CLOB 字段，无法进行传输，并且该方式不支持超过 4k 的字段数据类型

□ 解决方法

将抽取进程中的 RMTTASK 改为 RMTFILE 参数官方建议将数据先抽取成文件，再基于文件数据解析进行初始化导入

5 OGG 参考资料

阅读 OGG 官方文档，查看 support.oracle.com 上 Oracle 官方给予的问题解决方案是学习 OGG 非常有效的方式，作者这里也打包了一些文档供大家下载查看，其中个人感觉比较重要并且查看比较多的文档包括：How to Replicate Data Between Oracle and MySQL Database? (文档 ID 1605674.1).pdf 这个文档的作用相当于 OGG 快速开始，能够帮助用户快速的配置并跑通 OGG 全量、增量数据同步的流程，非常适合初学者上手 Administering Oracle GoldenGate for Windows and UNIX.pdf Administering 文档内容较多，其中详细介绍了各种 OGG 的使用配置场景，大家可以根据自己需要选择重点章节浏览 Reference for Oracle GoldenGate for Windows and UNIX.pdf Reference 文档算是我查看最多的一个文档，其中包含了 OGG 所有参数的描述、语法及使用示例，非常的详细，这也是前面未对参数进行展开讲解的原因。

文档资料百度网盘下载链接：

<https://pan.baidu.com/s/13fwguorcVrboeBH8DYTpZA> 密码:e8rh



基于 Xtrabackup 及可传输表空间实现多源数据恢复

作者：余振兴

Keywords: MySQL、xtrabackup、Transportable、Tablespaces、MySQLdump、multi-source、replication

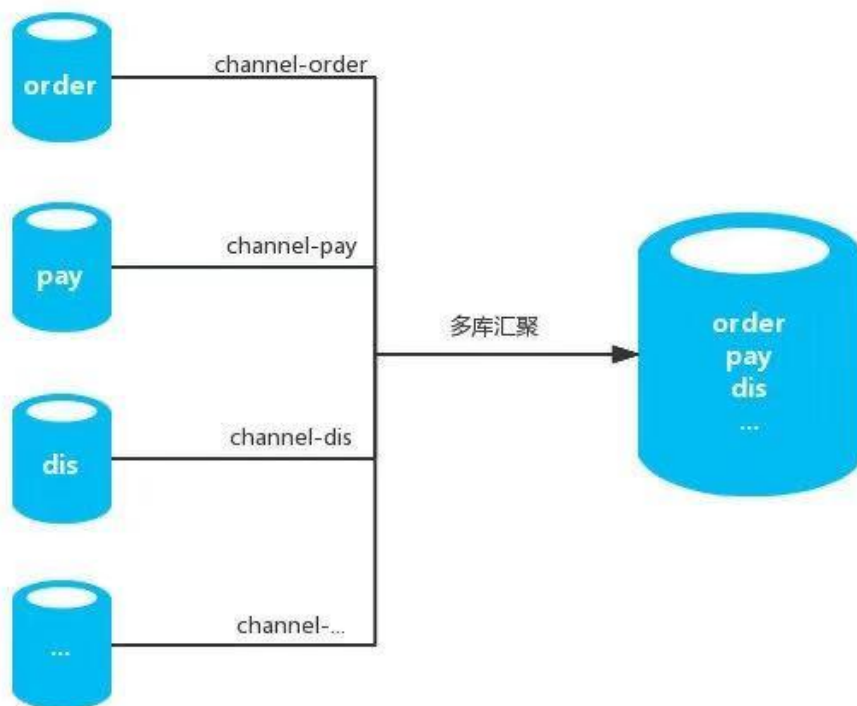
1 使用背景

MySQL 在 5.7 以后引入了**多源复制 (Multi-Source)** 功能，可用于将多个单独的数据库汇聚到一个数据库实例下，方便用户进行数据分析、汇总或推送至其他数据库平台。早期针对汇聚场景初始化各源端数据到汇聚库，为了提升效率通常会使用使用开源并行逻辑导入导出工具 **myloader/mydumper** 进行数据导入/导出，当源端多实例多库数据量较大(100G 以上) 情况下，使用 **myloader/mydumper** 花费的时间则可能无法满足时间要求，需要考虑是否有更高效的方式进行数据初始化同步操作，以及当复制通道异常时更便捷快速修复。

ps：多源复制使用物理备份(xtrabackup)做数据初始化时，常规方式只有第一个通道可做覆盖还原，后续通道需逻辑还原或用其他方式还原，如该方案所描述的方式。

MySQL 在 5.6 以后支持了可传输表空间，以及 Xtrabackup 针对该功能在备份时提供了一个对应参数 **export**，该参数支持对 InnoDB 存储引擎表的备份数据 **export** 转换，且与常规备份操作一样可生成备份时间点的 binlog 信息 (GTID 信息)，结合这两特性可以更高效和快速的方式实现多库数据汇聚的初始导入操作。

场景架构图如下



1.1 可传输表空间基本流程

先简单梳理一下可传输表空间基本流程：

- 在源端将 InnoDB 表进行 ibd 数据文件导出处理(`export tablespace`)
- 在目标端创建与源端相同表结构的表
- 将目标端的表数据文件舍弃(`discard tablespace`)
- 用源端的相对应 ibd 文件覆盖到目标端
- 在目标端执行表空间文件的导入/置换(`import tablespace`)

单纯使用可传输表空间功能无法记录对应表的事务点，也就是如果需要进行汇聚复制同步，无法知道从 binlog 哪个文件的哪个 position 点进行数据同步，这也就是需要引入 xtrabackup 及 export 功能的原因。备份元数据信息本身会记录复制同步点信息。

1.2 使用前提条件/限制

- MySQL 须为 5.6 以上版本
- 表存储引擎须为 InnoDB 存储引擎(支持可传输表空间)
- 导入完成后建议执行 `ANALYZE TABLE` 更新统计信息

2 技术要点

- sysbench 源端造测试数据并模拟压力
- mysqldump 备份源端表结构
- concat() 拼接批量可传输表空间 SQL
- xtrabackup 备份源端单库(部分库)数据
- Transportable Tablespace 可传输表空间功能
- Multi-source Replication 多源复制
- mysql-error.log 错误信息日志校验

■ ANALYZE TABLE 更新统计信息

3 实施步骤

3.1 实验环境

角色	IP	数据库	MySQL Version
源端	10.186.60.16	sbtest	5.7.24 Community
目标端	10.186.60.18	sbtest	5.7.24 Community

3.2 源端操作

■ 1) 造测试数据并模拟小压力

使用 sysbench 创建 4 张各 100W 记录的测试表，并使用 2 个并发持续模拟业务压力：

造数据

```
shell> /opt/sysbench-0.9/sysbench/sysbench --test=/opt/sysbench-0.9/sysbench/tests/db/oltp.lua --oltp-table-size=1000000 --oltp-tables-count=4 --mysql-user=sysbench --mysql-password=sysbench --mysql-host=10.186.60.16 --mysql-port=3333 prepare
```

模拟小压力

```
shell> /opt/sysbench-0.9/sysbench/sysbench --test=/opt/sysbench-0.9/sysbench/tests/db/oltp.lua --oltp-table-size=1000000 --oltp-tables-count=4 --mysql-user=sysbench --mysql-password=sysbench --mysql-host=10.186.60.16 --mysql-port=3333 --num-threads=2 --max-requests=0 --max-time=0 --report-interval=1 run
```

■ 2) 备份单库数据

使用 xtrabackup 备份工具对源端 sbtest 库进行单库备份，保存至 /data/mysql/backup/ 目录下：

```
shell> innobackupex --databases=sbtest /data/mysql/backup/
```

■ 3) 备份表结构

为可传输表空间做准备，将源端表结构备份并后续在目标端导入：

```
shell> cd /data/mysql/backup
shell> mysqldump --no-data --set-gtid-purged=off sbtest>sbtest_schema.sql
```

■ 4) 批量生成可传输表空间命令

□ discard

使用 concat 函数拼接出批量 DISCARD TABLESPACE 的 SQL:

```
shell> cd /data/mysql/backup
shell> mysql -ssre "select concat('ALTER TABLE
',TABLE_SCHEMA, '.',TABLE_NAME,' DISCARD TABLESPACE;') from
information_schema.tables where TABLE_SCHEMA='sbtest';" >discard_tbs.sql
```

输出文件如下所示

```
shell> cat discard_tbs.sql
ALTER TABLE sbtest.sbtest1 DISCARD TABLESPACE;
ALTER TABLE sbtest.sbtest2 DISCARD TABLESPACE;
ALTER TABLE sbtest.sbtest3 DISCARD TABLESPACE;
ALTER TABLE sbtest.sbtest4 DISCARD TABLESPACE;
```

□ import

使用 concat 函数拼接出批量 IMPORT TABLESPACE 的 SQL:

```
shell> cd /data/mysql/backup
shell> mysql -ssre "select concat('ALTER TABLE
',TABLE_SCHEMA, '.',TABLE_NAME,' IMPORT TABLESPACE;') from
information_schema.tables where TABLE_SCHEMA='sbtest';">import_tbs.sql
```

输出文件如下所示

```
shell> cat import_tbs.sql
ALTER TABLE sbtest.sbtest1 IMPORT TABLESPACE;
ALTER TABLE sbtest.sbtest2 IMPORT TABLESPACE;
ALTER TABLE sbtest.sbtest3 IMPORT TABLESPACE;
ALTER TABLE sbtest.sbtest4 IMPORT TABLESPACE;
```

拷贝源端/data/mysql/backup 目录下生成的所有相关文件到目标端/data/mysql/backup 下

3.3 目标端操作

■ 1) 备份数据恢复准备

对备份数据进行 apply-log 日志应用及将数据进行 export 转换生成配置文件:

```
shell> innobackupex --apply-log --export /data/mysql/backup/2019-02-22_15-26-20/
```

export 执行完后 sbtest 库下备份文件如下所示

exp 结尾的文件为 Percona 针对 Percona XtraDB 做 export 的配置文件

cfg 结尾的文件为 Percona 针对 MySQL 可传输表空间 export 的配置文件

```
shell> ll /data/mysql/backup/2019-02-22_15-26-20/sbtest/
总用量 999556
-rw-r----- 1 root root      67 2月 22 15:40 db.opt
```

```

-rw-r--r-- 1 root root      569 2月 22 15:53 sbtest1.cfg
-rw-r----- 1 root root    16384 2月 22 15:53 sbtest1.exp
-rw-r----- 1 root root     8632 2月 22 15:40 sbtest1.frm
-rw-r----- 1 root root 255852544 2月 22 15:53 sbtest1.ibd
-rw-r--r-- 1 root root      569 2月 22 15:53 sbtest2.cfg
-rw-r----- 1 root root    16384 2月 22 15:53 sbtest2.exp
-rw-r----- 1 root root     8632 2月 22 15:40 sbtest2.frm
-rw-r----- 1 root root 255852544 2月 22 15:53 sbtest2.ibd
-rw-r--r-- 1 root root      569 2月 22 15:53 sbtest3.cfg
-rw-r----- 1 root root    16384 2月 22 15:53 sbtest3.exp
-rw-r----- 1 root root     8632 2月 22 15:40 sbtest3.frm
-rw-r----- 1 root root 255852544 2月 22 15:53 sbtest3.ibd
-rw-r--r-- 1 root root      569 2月 22 15:53 sbtest4.cfg
-rw-r----- 1 root root    16384 2月 22 15:53 sbtest4.exp
-rw-r----- 1 root root     8632 2月 22 15:40 sbtest4.frm
-rw-r----- 1 root root 255852544 2月 22 15:53 sbtest4.ibd

```

■ 2) 恢复表结构至目标端

将源端表结构在目标端数据库创建:

```
shell> cd /data/mysql/backup
```

手工创建 sbtest 库

```
shell> mysql -e "create database sbtest;"
```

导入源端对应的表结构

```
shell> mysql sbtest< sbtest_schema.sql
```

验证

```
shell> mysql -e "show tables from sbtest;"
```

■ 3) 舍弃目标端对应表空间文件

```
shell> cd /data/mysql/backup
```

```
shell> mysql <discard_tbs.sql
```

执行后效果如下所示, ibd 文件已被舍弃

```
shell> ll /data/mysql/data/sbtest/
```

```

-rw-r----- 1 mysql mysql   67 2月 22 15:58 db.opt
-rw-r----- 1 mysql mysql 8632 2月 22 15:58 sbtest1.frm
-rw-r----- 1 mysql mysql 8632 2月 22 15:58 sbtest2.frm
-rw-r----- 1 mysql mysql 8632 2月 22 15:58 sbtest3.frm
-rw-r----- 1 mysql mysql 8632 2月 22 15:58 sbtest4.frm

```

■ 4) 拷贝表数据及配置文件到目标端 sbtest 库数据目录

拷贝 ibd 文件

```
shell> cp /data/mysql/backup/2019-02-22_15-26-20/sbtest/*.ibd
/data/mysql/data/sbtest/
```

拷贝 cfg 文件

```
shell> cp /data/mysql/backup/2019-02-22_15-26-20/sbtest/*.cfg
/data/mysql/data/sbtest/
```

修改文件权限为 mysql 用户

```
shell> chown -R mysql:mysql /data/mysql/data/sbtest
```

■ 5) 执行导入表空间文件操作

```
shell> cd /data/mysql/backup
```

出现 Warning 为可传输表空间正常输出, 可忽略, 详情可参考下图所示 note 说明

```
shell> mysql <import_tbs.sql
```

```
Warning (Code 1814): InnoDB: Tablespace has been discarded for table
'sbtest1'
```

```
Warning (Code 1814): InnoDB: Tablespace has been discarded for table
'sbtest2'
```

```
Warning (Code 1814): InnoDB: Tablespace has been discarded for table
'sbtest3'
```

```
Warning (Code 1814): InnoDB: Tablespace has been discarded for table
'sbtest4'
```

可通过 MySQL 错误日志查看 import 期间是否存在导入异常

```
shell> less /data/mysql/data/mysql-error.log
```

Note

You may also receive a warning that a tablespace is discarded (if you discarded the tablespace for the destination table) and a message stating that statistics could not be calculated due to a missing .ibd file:

```
1 2013-07-18 15:14:38 34960 [Warning] InnoDB: Table "test"."t" tablespace is set as discarded.
2 2013-07-18 15:14:38 7f34d9a37700 InnoDB: cannot calculate statistics for table "test"."t"
3 because the .ibd file is missing. For help, please refer to
4 http://dev.mysql.com/doc/refman/8.0/en/innodb-troubleshooting.html
```

3.4 建立复制(多源复制)

```
mysql> CHANGE MASTER TO
MASTER_HOST='10.186.60.16',
MASTER_USER='repl',
MASTER_PORT=3333,
MASTER_PASSWORD='repl',
MASTER_LOG_FILE='mysql-bin.000004',
```

```
MASTER_LOG_POS=149327998 FOR CHANNEL '10-186-60-16';
```

```
mysql> CHANGE REPLICATION FILTER REPLICATE_WILD_DO_TABLE=('sbtest.%');
```

```
mysql> START SLAVE FOR CHANNEL '10-186-60-16';
```

```
mysql> SHOW SLAVE STATUS FOR CHANNEL '10-186-60-16'\G;
```

参考链接

Percona xtrabackup export 功能介绍

https://www.percona.com/doc/percona-xtrabackup/2.4/xtrabackup_bin/restoring_individual_tables.html

MySQL 可传输表空间功能说明

<https://dev.mysql.com/doc/refman/5.7/en/tablespace-copying.html>

场景示例思路来源博客

<http://www.cnblogs.com/xuanzhi201111/p/6609867.html>



MySQL 主从复制延时常见场景及分析改善

作者：杨奇龙

1 序言

在运维 MySQL 数据库时, DBA 会接收到比较多关于主备延时的报警:

```
check_ins_slave_lag (err_cnt:1)critical-slavelag on ins:3306=39438
```

相信 slave 延迟是 MySQL DBA 遇到的一个老生常谈的问题了。我们先来分析一下 slave 延迟带来的风险:

- 异常情况下, 主从 HA 无法切换, HA 软件需要检查数据的一致性, 延迟时, 主备不一致
- 备库复制 hang 会导致备份失败(flush tables with read lock 会 900s 超时)
- 以 slave 为基准进行的备份, 数据不是最新的, 而是延迟

本文主要探讨如何解决, 如何规避 slave 延迟的问题, 接下来我们要分析一下导致备库延迟的几种原因。

2 slave 延迟的场景以及解决方法

2.1 无主键、无索引或索引区分度不高

有如下特征:

- a. show slave status 显示 position 一直没有变
- b. show open tables 显示某个表一直是 in_use 为 1
- c. show create table 查看表结构可以看到无主键, 或者无任何索引, 或者索引区分度很差。

解决方法:

- a. 找到表区分度比较高的几个字段, 可以使用这个方法判断:

```
select count(*) from xx;  
select count(*) from (select distinct xx from xxx) t;
```

如果 2 个查询 count(*) 的结果差不多, 说明可以对这些字段加索引

- b. 备库 stop slave;
可能会执行比较久, 因为需要回滚事务。

- c. 备库
set global slave_rows_search_algorithms='TABLE_SCAN,INDEX_SCAN,HASH_SCAN';
或者
set sql_log_bin=0;
alter table xx add key xx(xx);

d. 备库 start slave

如果是 innodb, 可以通过 show innodb status 来查

看 rows_inserted, updated, deleted, selected 这几个指标来判断。

如果每秒修改的记录数比较多, 说明复制正在以比较快的速度执行。

其实针对无索引的表, 可以直接调整从库上的参数 slave_rows_search_algorithms。

2.2 主库上有大事务, 导致从库延时

现象解析 binlog 发现类似于下图的情况看:

```

BEGIN
/* */;
# at 3579
# at 3669
# at 4655
# at 5650
# at 6666
# at 7700
# at 8743
# at 9732
# at 10719
# at 11718
# at 12732
# at 13744
# at 14769
# at 15809
#131015 8:35:21 server id 1963306001 end_log_pos 3669      Table_map: [redacted] mapped to number 4594487
#131015 8:35:21 server id 1963306001 end_log_pos 4655      Write_rows: table id 4594487
#131015 8:35:21 server id 1963306001 end_log_pos 5650      Write_rows: table id 4594487
#131015 8:35:21 server id 1963306001 end_log_pos 6666      Write_rows: table id 4594487
#131015 8:35:21 server id 1963306001 end_log_pos 7700      Write_rows: table id 4594487
#131015 8:35:21 server id 1963306001 end_log_pos 8743      Write_rows: table id 4594487
#131015 8:35:21 server id 1963306001 end_log_pos 9732      Write_rows: table id 4594487
#131015 8:35:21 server id 1963306001 end_log_pos 10719     Write_rows: table id 4594487
#131015 8:35:21 server id 1963306001 end_log_pos 11718    Write_rows: table id 4594487
#131015 8:35:21 server id 1963306001 end_log_pos 12732    Write_rows: table id 4594487
#131015 8:35:21 server id 1963306001 end_log_pos 13744    Write_rows: table id 4594487
#131015 8:35:21 server id 1963306001 end_log_pos 14769    Write_rows: table id 4594487

```

解决方法:

事前防范, 与开发沟通, 增加缓存, 异步写入数据库, 减少业务直接对 db 的大事务写入。

事中补救, 调整数据库中 io 相关的参数比如 innodb_flush_log_at_trx_commit 和 sync_binlog 或者打开并行复制功能。

2.3 主库写入频繁, 从库压力跟不上导致延时

此类原因的主要现象是数据库的 IUD 操作非常多, slave 由于 sql_thread 单线程的原因追不上主库。

解决方法:

a 升级从库的硬件配置, 比如 ssd, fio。

b 使用@丁奇的预热工具-relay fetch

在备库 sql 线程执行更新之前, 预先将相应的数据加载到内存中, 并不能提高 sql_thread 线程执行 sql 的能力,

也不能加快 io_thread 线程读取日志的速度。

c 使用多线程复制 阿里 MySQL 团队实现的方案--基于行的并行复制。

该方案允许对同一张表进行修改的两个事务并行执行, 只要这两个事务修改了表中的不同的行。

这个方案可以达到事务间更高的并发度, 但是局限是必须使用 Row 格式的 binlog。

因为只有使用 Row 格式的 binlog 才可以知道一个事务所修改的行的范围, 而使用 Statement 格式的 binlog 只能知道修改的表对象。

2.4 大量 myisam 表，在备份时导致 slave 延迟

由于 xtrabackup 工具备份到最后会执行 flash tables with read lock，对数据库进行锁表以便进行一致性备份，然后对于 myisam 表锁，会阻碍 slave_sql_thread 停滞运行进而导致 hang。

该问题目前的比较好的解决方式是修改表结构为 innodb 存储引擎的表。

3 MySQL 的改进

为了解决复制延迟的问题，MySQL 也在不遗余力的解决主从复制的性能瓶颈，研发高效的复制算法。

3.1 基于组提交的并行复制

MySQL 的复制机制大致原理是：slave 通过 io_thread 将主库的 binlog 拉到从库并写入 relay log，由 SQL thread 读出来 relay log 并进行重放。当主库写入并发写入压力很大，也即 N:1 的情形，slave 就可能会出现延迟。MySQL 5.6 版本提供并行复制功能，slave 复制相关的线程由 io_thread, coordinator_thread, worker 构成，其中：

- **coordinator_thread** 负责读取 relay log，将读取的 binlog event 以事务为单位分发到各个 worker thread 进行执行，并在必要时执行 binlog event，比如是 DDL 或者跨库事务的时候。
- **worker_thread** 执行分配到的 binlog event，各个线程之间互不影响，具体 worker_thread 的个数由 slave_parallel_workers 决定。

需要注意的是 dbname 与 worker 线程的绑定信息在一个 hash 表中进行维护，hash 表以 entry 为单位，entry 中记录当前 entry 所代表的数据库名，有多少个事务相关的已被分发，执行这些事务的 worker thread 等信息。

分配线程是以数据库名进行分发的，当一个实例中只有一个数据库的时候，不会对性能有提高，相反，由于增加额外的操作，性能还会有一点回退。

MySQL 5.7 版本提供基于组提交的并行复制，通过设置如下参数来启用并行复制。

```
slave_parallel_workers>0
global.slave_parallel_type='LOGICAL_CLOCK'
```

即主库在 ordered_commit 中的第二阶段，**将同一批 commit 的 binlog 打上一个相同的 last_committed 标签，同一 last_committed 的事务在备库是可以同时执行的，因此大大简化了并行复制的逻辑，并打破了相同 DB 不能并行执行的限制。**备库在执行时，具有同一 last_committed 的事务在备库可以并行的执行，互不干扰，也不需要绑定信息，后一批 last_committed 的事务需要等待前一批相同 last_committed 的事务执行完后才可以执行。这样的实现方式大大提高了 slave 应用 relaylog 的速度。

#默认是 DATABASE 模式的，需要调整或者在 my.cnf 中配置

```
mysql> show variables like 'slave_parallel%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
```

```

| slave_parallel_type | DATABASE |
| slave_parallel_workers | 4 |
+-----+
2 rows in set (0.00 sec)
mysql> STOP SLAVE SQL_THREAD;
Query OK, 0 rows affected (0.00 sec)
mysql> set global slave_parallel_type='LOGICAL_CLOCK';
Query OK, 0 rows affected (0.00 sec)
mysql> START SLAVE SQL_THREAD;
Query OK, 0 rows affected (0.01 sec)

```

启用并行复制之后查看 processlist，系统多了四个线程 Waiting for an event from Coordinator

```

mysql> show processlist;
+----+-----+-----+-----+-----+-----+-----+-----+
| Id | User | Host | db | Command | Time | State | Info |
+----+-----+-----+-----+-----+-----+-----+
| 9 | root | localhost:40270 | NULL | Query | 0 | starting | show processlist |
| 10 | system user | | NULL | Connect | 1697 | Waiting for master to send event | NULL |
| 31 | system user | | NULL | Connect | 5 | Slave has read all relay log; waiting for more updates | NULL |
| 32 | system user | | NULL | Connect | 5 | Waiting for an event from Coordinator | NULL |
| 33 | system user | | NULL | Connect | 5 | Waiting for an event from Coordinator | NULL |
| 34 | system user | | NULL | Connect | 5 | Waiting for an event from Coordinator | NULL |
| 35 | system user | | NULL | Connect | 5 | Waiting for an event from Coordinator | NULL |
+----+-----+-----+-----+-----+-----+-----+
7 rows in set (0.00 sec)

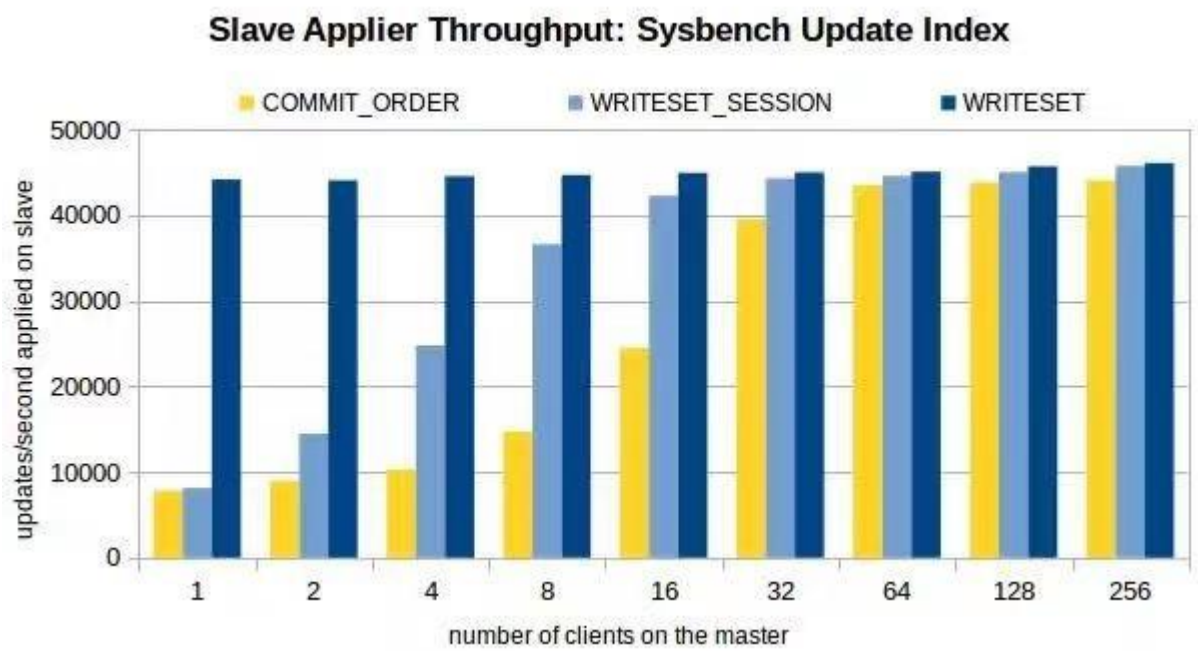
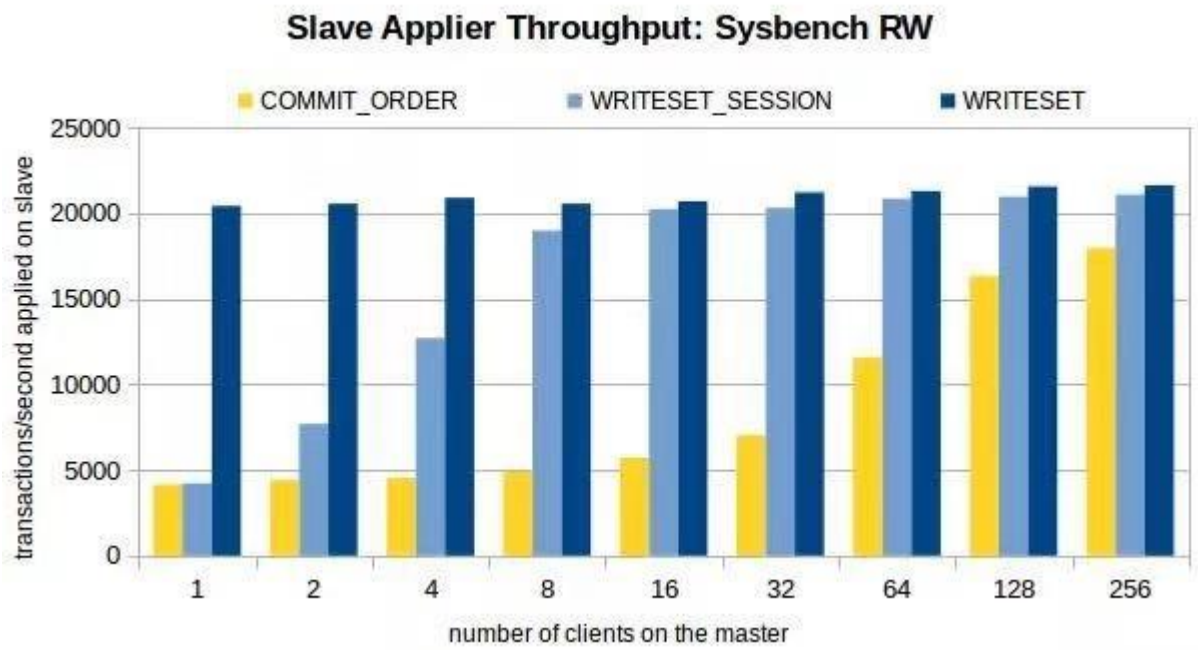
```

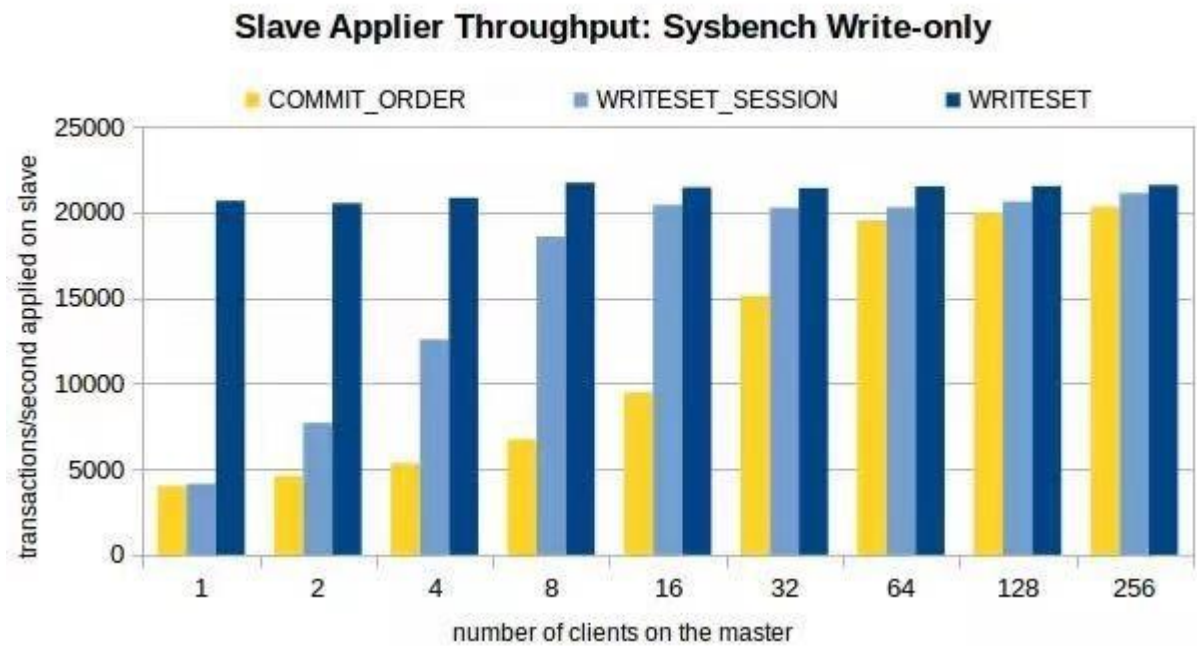
核心参数:

binlog_group_commit_sync_no_delay_count:一组里面有多少事物才提交,单位 (ms)
binlog_group_commit_sync_delay:等待多少时间后才进行组提交

3.2 基于写集合的并行复制

其实从 MySQL 5.7.22 就提供基于 `write set` 的复制优化策略。WriteSet 并行复制的思想是：不同事务的不同记录不重叠，则都可在从机上并行回放，可以看到并行的力度从组提交细化为记录级。
废话不多说，直接上性能压测图：





4 总结

slave 延迟的原因可以归结为 slave apply binlog 的速度跟不上主库写入的速度，如何解决复制延迟呢？其实也是如何提高 MySQL 写速度的问题。从目前的硬件和软件的发展来看，硬件存储由之前的 HDD 机械硬盘发展到现在的 SSD, PCI-E SSD, 再到 NVMe Express (NVMe), IO 性能一直在提升。MySQL 的主从复制也从单线程复制到不同算法的并行复制(基于库，事务，行)，应用 binlog 的速度也越来越快。本文归纳从几个常见的复制延迟场景，有可能还不完整，也欢迎大家留言讨论。

拓展阅读

[1] 一种 MySQL 主从同步加速方案—改进

<https://dinglin.iteye.com/blog/1187154>

[2] MySQL 多线程同步 MySQL-Transfer 介绍(已经不在维护)

<https://dinglin.iteye.com/blog/1581877>

[3] MySQL 并行复制演进及 MySQL 8.0 中基于 WriteSet 的优化

<https://www.cnblogs.com/DataArt/p/10240093.html>

[4] 速度提升 5~10 倍，基于 WRITESSET 的 MySQL 并行复制

[5] <https://mysqlhighavailability.com/improving-the-parallel-applier-with-writeset-based-dependency-tracking/>



HASH_SCAN BUG 之迷惑行为大赏

作者：胡呈清

1 前言

我们知道，MySQL 有一个老问题，当表上无主键时，那么对于在该表上做的 DML，如果是以 ROW 模式复制，则每一个行记录前镜像在备库都可能产生一次全表扫描（或者二级索引扫描），大多数情况下，这种开销都是非常不可接受的，并且产生大量的延迟。

在 MySQL 5.6 中提供了一个新的参数：`slave_rows_search_algorithms`，可以部分解决无主键表导致的复制延迟问题，其基本思路是对于在一个 ROWS EVENT 中的所有前镜像收集起来，然后在一次扫描全表时，判断 HASH 中的每一条记录进行更新。

2 HASH_SCAN BUG 的迷惑行为

本文不是要继续讨论 `slave_rows_search_algorithms` 的原理，而是在使用 `slave_rows_search_algorithms` 参数时遇到一个坑，扑朔迷离的地方在于这不像一个 bug，这又是一个 bug，最后官方的操作更是让人看不懂。

2.1 问题描述

row-based replication，主从数据一致情况下，slave sql 线程报错 Can't find record。

2.2 如何复现

配置：

```
slave_rows_search_algorithms = 'INDEX_SCAN,HASH_SCAN'
binlog_format = ROW
```

在主库上：

1. CREATE TABLE t1 (A INT UNIQUE KEY, B INT); insert into t1 values (1,2);
 2. replace into t1 values (1,3), (1,4);
 3. 然后从库就会出现报错了。
 4. 在从库上：set global slave_rows_search_algorithms='INDEX_SCAN, TABLE_SCAN'; start slave;
- 问题解决

2.3 或者用如下方法避免：

1. 将 UNIQUE KEY 调整为 PRIMARY KEY;
2. 将 replace into tt.t1 values (1,3), (1,4); 调整为 replace into tt.t1 values (1,3); replace into tt.t1 values (1,4);

2.4 分析过程：

查看 `slaverowssearch_algorithms` 参数定义：

The default value is `INDEX_SCAN, TABLE_SCAN`, which means that all searches that can use indexes do use them, and searches without any indexes use table scans.

To use hashing for any searches that do not use a primary or unique key, set `INDEX_SCAN, HASH_SCAN`. Specifying `INDEX_SCAN, HASH_SCAN` has the same effect as specifying `INDEX_SCAN, TABLE_SCAN, HASH_SCAN`, which is allowed.

Do not use the combination `TABLE_SCAN, HASH_SCAN`. This setting forces hashing for all searches. It has no advantage over `INDEX_SCAN, HASH_SCAN`, and it can lead to “record not found” errors or duplicate key errors in the case of a single event containing multiple updates to the same row, or updates that are order-dependent.

（1）根据手册描述，可以理解为：

从库定位数据可选项有 `INDEX_SCAN`、`HASH_SCAN`、`TABLE_SCAN`，优先级是依次递减的。也就是说如果有主键，走 `INDEX_SCAN`，没主键则根据设置走 `HASH_SCAN` 还是 `TABLE_SCAN`（而且如果两者都配了，则优先 `HASH_SCAN`）；无主键设置 `INDEX_SCAN, HASH_SCAN`，可以提升从库回访效率，降低延迟；如果走了 `HASH_SCAN`，当对同一行数据同时更新多次时，会导致无法找到行记录。看到这里，手册已经说明原因了。但是矛盾的地方在于手册有推荐使用 `INDEX_SCAN, HASH_SCAN` 的意思，并且 8.0.2 版本开始的默认值已经修改为：`INDEX_SCAN, HASH_SCAN`

（2）针对上面的疑问提交 SR

Oracle 工程师回应这是 `HASH_SCAN` 的 bug，修复到了 MySQL 8.0.17（还没有正式发布）：

It happens when one row is updated twice within an `Update_rows_log_event`. `HASH_SCAN` will put both rows in a hash map. Then it iterates over the rows of the table, looks up each row in the hash. If any row is found, it applies the update. Since it only makes one lookup per row, it will miss the second update. In the end, it checks that all rows in the hash were applied and generates an error otherwise. This is what is seen in the bug report.

等等，为什么只修复到 8.0 而没在 5.7 修复？再根据之前就有的疑问，我来脑补一波：

1. 文档写了在某些情况 `HASH_SCAN` 有这个问题（吐槽：`HASH_SCAN` 就是优化无主键情况从库复制效率的，但是无主键且对同一行数据同时更新多次时 `HASH_SCAN` 又会导致从库无法找到记录，那我用还是不用呢？黑人问号.gif），所以暂时我先假设“这不是个 bug”；
2. Oracle 工程师反手甩给我一个 bug 报告，啪啪打脸，官方承认这就是一个 bug；
3. 官方好像也认为有点不对劲，一拍脑袋，咱们只在 8.0 修复，把锅甩给“8.0.2 的默认值修改成了 `HASH_SCAN`”。

这个迷惑行为可以用一句经典总结：it's not a bug, it's a feature!

2.5 解决方案

1. 给表添加主键（规范必须有主键才是王道）；
2. 修改参数 `slave_rows_search_algorithms='INDEX_SCAN, TABLE_SCAN'`；
3. 最符合初衷的做法是升级到 8.0.17，可惜这步对于绝大多数生产环境来说都太大了。



我对 MySQL 隔离级别的剖析

作者：姚嵩

术式之后皆为逻辑，一切皆为需求和实现。希望此文能从需求、现状和解决方式的角度帮大家理解隔离级别。

1 隔离级别产生

在串型执行的条件下，数据修改的顺序是固定的、可预期的结果，但是并发执行的情况下，数据的修改是不可预期的，也不固定，为了实现数据修改在并发执行的情况下得到一个固定、可预期的结果，由此产生了隔离级别。

所以隔离级别的作用是用来平衡数据库并发访问与数据一致性的方法。

2 事务的 4 种隔离级别

READ UNCOMMITTED	未提交读，可以读取未提交的数据。
READ COMMITTED	已提交读，对于锁定读(select with for update 或者 for share)、update 和 delete 语句，InnoDB 仅锁定索引记录，而不锁定它们之间的间隙，因此允许在锁定的记录旁边自由插入新记录。
	Gap locking 仅用于外键约束检查和重复键检查。
REPEATABLE READ	可重复读，事务中的一致性读取读取的是事务第一次读取所建立的快照。
SERIALIZABLE	序列化

在了解了 4 种隔离级别的需求后，在采用锁控制隔离级别的基础上，我们需要了解加锁的对象(数据本身&间隙)，以及了解整个数据范围的全集组成。

3 数据范围全集组成

SQL 语句根据条件判断不需要扫描的数据范围（不加锁）；

SQL 语句根据条件扫描到的可能需要加锁的数据范围；

以单个数据范围为例，数据范围全集包含：（数据范围不一定是连续的值，也可能是间隔的值组成）

1. 数据已经填充了整个数据范围：（被完全填充的数据范围，不存在数据间隙）

- 整形，对值具有唯一约束条件的数据范围 1~5，
已有数据 1、2、3、4、5，此时数据范围已被完全填充；
- 整形，对值具有唯一约束条件的数据范围 1 和 5，
已有数据 1、5，此时数据范围已被完全填充；

2. 数据填充了部分数据范围：（未被完全填充的数据范围，是存在数据间隙）

- 整形的数据范围 1~5，已有数据 1、2、3、4、5，但是因为没有任何唯一约束，所以数据范围可以继续被 1~5 的数据重复填充；

- 整形，具有唯一约束条件的数据范围 1~5，已有数据 2，5，此时数据范围未被完全填充，还可以填充 1、3、4；
- 3. 数据范围内没有任何数据（存在间隙）
 - 如下：整形的数据范围 1~5，数据范围内当前没有任何数据。

在了解了数据全集的组成后，我们再来看看事务并发时，会带来什么问题。

4 无控制的并发所带来的问题

并发事务如果不加以控制的话会带来一些问题，主要包括以下几种情况。

1. 范围内已有数据更改导致的：
 - **更新丢失**：当多个事务选择了同一行，然后基于最初选定的值更新该行时，
 - 由于每个事物不知道其他事务的存在，最后的更新就会覆盖其他事务所做的更新；
 - **脏读**：一个事务正在对一条记录做修改，这个事务完成并提交前，这条记录就处于不一致状态。
 - 这时，另外一个事务也来读取同一条记录，如果不加控制，
 - 第二个事务读取了这些“脏”数据，并据此做了进一步的处理，就会产生提交的数据依赖关系。
 - 这种现象就叫“脏读”。
2. 范围内数据量发生了变化导致：
 - **不可重复读**：一个事务在读取某些数据后的某个时间，再次读取以前读过的数据，
 - 却发现其读出的数据已经发生了改变，或者某些记录已经被删除了。
 - 这种现象就叫“不可重复读”。
 - **幻读**：一个事务按相同的查询条件重新读取以前检索过的数据，
 - 却发现其他事务插入了满足其查询条件的新数据，这种现象称为“幻读”。
 - 可以简单的认为满足条件的数据量变化了。

因为无控制的并发会带来一系列的问题，这些问题会导致无法满足我们所需要的结果。

因此我们需要控制并发，以实现我们所期望的结果(隔离级别)。

5 MySQL 隔离级别的实现

InnoDB 通过加锁的策略来支持这些隔离级别。

行锁包含：

- **Record Locks**
索引记录锁，索引记录锁始终锁定索引记录，即使表中未定义索引，这种情况下，InnoDB 创建一个隐藏的聚簇索引，并使用该索引进行记录锁定。
- **Gap Locks**
间隙锁是索引记录之间的间隙上的锁，或者对第一条记录之前或者最后一条记录之后的锁。间隙锁是性能和并发之间权衡的一部分。对于无间隙的数据范围不需要间隙锁，因为没有间隙。
- **Next-Key Locks**
索引记录上的记录锁和索引记录之前的 gap lock 的组合。假设索引包含 10、11、13 和 20。

可能的 next-key locks 包括以下间隔，其中圆括号表示不包含间隔端点，方括号表示包含端点：

- (负无穷大, 10]
- (10, 11]
- (11, 13]
- (13, 20]
- (20, 正无穷大)

对于最后一个间隔，next-key 将会锁定索引中最大值的上方，

“上确界”伪记录的值高于索引中任何实际值。

上确界不是一个真正的索引记录，因此，实际上，这个 next-key 只锁定最大索引值之后的间隔。基于此，当获取的数据范围中，数据已填充了所有的数据范围，那么此时是不存在间隔的，也就不需要 gap lock。

对于数据范围内存在间隔的，需要根据隔离级别确认是否对间隔加锁。

默认的 REPEATABLE READ 隔离级别，为了保证可重复读，除了对数据本身加锁以外，还需要对数据间隔加锁。

READ COMMITTED 已提交读，不匹配行的记录锁在 MySQL 评估了 where 条件后释放。

对于 update 语句，InnoDB 执行“semi-consistent”读取，这样它会将最新提交的版本返回到 MySQL，以便 MySQL 可以确定该行是否与 update 的 where 条件相匹配。

现在我们来验证以下 MySQL 对于隔离级别的实现是否符合预期。

5.1 场景演示

下面整理几种场景，确定其所加锁：

数据准备：

```
CREATE TABLE `t` (  
  `a` int(11) NOT NULL,  
  `b` int(11) DEFAULT NULL,  
  `c` int(11) DEFAULT NULL,  
  `d` int(11) DEFAULT NULL,  
  `e` int(11) DEFAULT NULL,  
  PRIMARY KEY (`a`),  
  UNIQUE KEY `c` (`c`,`d`,`e`),  
  KEY `b` (`b`)  
);  
  
insert into t values  
(1,2,1,2,3),(2,2,1,2,4),(3,3,1,2,5),(4,4,2,3,4),(5,5,3,3,5),(6,7,4,5,5),(7,10,5,  
6,7),(8,13,5,3,1),(9,20,6,9,10),(10,23,7,7,7) ;
```

说明:

```
session A
SQL1
SQL2
....
session B
SQLN
SQLN+1
...
```

上面的语句都是按照时间顺序排序。

所有的测试数据都是基于数据准备好的原始数据，每次测试完成，请自我修复现场。

场景一

rr 模式+唯一索引筛选:

数据:

数据范围已被数据完全填充 (1)

已有数据的更改

```
session A
start transaction ;
update t set a=11 where a=3 ;      # 持有 Record Locks (a=3)
session B
update t set a=12 where a=3 ;      # 等待 a=3 的 Record Locks,
                                  直到 session A 将其释放/等锁超时
```

数据范围有数据，但未被完全填充 (2)

已有数据的更改

```
session A
start transaction ;
update t set b=11 where a>=10 ;    # 持有 Record Locks (a=10),
                                  有 Next-Key Locks (a>10)
session B
update t set b=12 where a=10 ;     # 等待 a=10 的 Record Locks,
                                  直到 session A 将其释放/等锁超时
```

间隙的填充

```
session A
start transaction ;
update t set b=11 where a>=10 ;    # 持有 Record Locks (a=10),
                                  有 Next-Key Locks (a>10)
session B
insert into t values(11,1,21,1,1); # 插入等待,
                                  因为存在 a>10 的 Next-Key Locks
```

数据范围内没有数据 (3)

间隙的填充

```
session A
start transaction ;
update t set b=11 where a>=100 and a<=200 ;      # 存在 Gap Locks
session B
insert into t values(150,1,21,1,1);    # 插入等待,
                                     因为存在(10,无穷大)的 Gap Locks,
                                     a 最大的一个值是 10
insert into t values(11,1,1,2,2) ;    # 插入等待,
                                     因为存在(10,无穷大)的 Gap Locks,
                                     a 最大的一个值是 10
update t set b=8 where a=10 ;          # 更改成功,
                                     因为 session A 并未持有 Record Locks
update t set b=23 where a=10 ;        # 恢复现场
```

场景二

rc 模式+唯一索引筛选:

数据:

数据范围已被数据完全填充 (4)

已有数据的更改

```
session A
start transaction ;
update t set a=11 where a=3 ;    # 持有 Record Locks (a=3)
session B
update t set a=12 where a=3 ;    # 等待 a=3 的 Record Locks,
                                直到 session A 将其释放/等锁超时
```

数据范围有数据,但未被完全填充 (5)

已有数据的更改

```
session A
start transaction ;
update t set b=11 where a>=10 ;    # 持有 Record Locks (a=10),
                                无 Next-Key Locks (a>10)
session B
update t set b=12 where a=10 ;    # 等待 a=10 的 Record Locks,
                                直到 session A 将其释放/等锁超时
```

间隙的填充

```
session A
start transaction ;
update t set b=11 where a>=10 ;    # 持有 Record Locks (a=10),
                                无 Next-Key Locks (a>10)
session B
```

```
insert into t values(11,1,21,1,1); # 插入成功
delete from t where a=11 ;          # 恢复现场
```

数据范围内没有数据 (6)

间隙的填充

```
session A
start transaction ;
update t set b=11 where a>=100 and a<=200 ; # 无对应的索引,
                                              所以无 Record Locks,
                                              无 Next-Key Locks

session B
insert into t values(150,1,21,1,1);          # 插入成功
delete from t where a=150 ;                  # 恢复现场
```

场景三

rr 模式+非唯一索引筛选: (非唯一索引筛选的情况下, 不存在数据完全填充的场景)

数据:

数据范围有数据, 但未被完全填充 (7)

已有数据的更改

```
session A
start transaction ;
update t set e=7 where b=2 and e=4 ; # 获取非唯一索引, 命中 b=2 的索引值,
                                      b=2 的索引值对应多条记录。
                                      此时有 Record Locks, 加在非唯一索引上
```

```
session B
update t set e=7 where b=2 and e=10 ; # session A 已获取了 b=2 的 Record
```

Locks ,

session B 等待 session A 将其释放/等锁超时
即使该条件命中的是空记录

间隙的填充

```
session A
start transaction ;
update t set d=6 where b>=5 and b<=7 ; # 获取了 b=5 和 b=7 的 Record
```

Locks,

和 (5,7) 的 Gap Locks

```
session B
insert into t values(11,6,11,12,13) ; # 插入 b=6 的记录, 插入等待,
                                      存在 b 的数据范围 (5,7) 的 Gap Locks。
```

数据范围内没有数据 (8)

间隙的填充

```
session A
start transaction ;
update t set b=100 where b>=120 ; # b>=120 命中了空的数据范围,
```

所以无 Record Locks,

但存在 (23, 正无穷) 的 Gap Locks

session B

insert into t values (200,200,200,200,200) ; # 插入等待,

等待 (23, 正无穷) 的 Gap Locks 的释放

insert into t values (100,24,3,4,60) ; # 插入等待,

等待 (23, 正无穷) 的 Gap locks 的释放

update t set b=12 where b=23 ; # 更新成功,

因为 session A 并未获取 Record Locks,

所以不会阻止已有行的更新

update t set b=23 where b=12 ; # 恢复现场

场景四

rc 模式+非唯一索引筛选: (非唯一索引筛选的情况下, 不存在数据完全填充的场景)

数据:

数据范围有数据, 但未被完全填充 (9)

已有数据的更改

session A

start transaction ;

update t set e=7 where b=2 and e=4 ; # 获取非唯一索引, 命中 b=2 的索引值,
b=2 的索引值对应多条记录。

此时有 Record Locks, 加在非唯一索引上

session B

update t set e=7 where b=2 and e=10 ; # session A 已获取了 b=2 的 Record
Locks ,

session B 等待 session A 将其释放/等锁超时,
即使该条件命中的是空记录

间隙的填充

session A

start transaction ;

update t set d=6 where b>=5 and b<=7 ; # 获取了 b=5 和 b=7 的 Record

Locks,

无 Next-Key Locks

session B

insert into t values (11,6,11,12,13) ; # 插入成功, 因为无 Next-Key Locks

数据范围内没有数据 (10)

间隙的填充

session A

start transaction ;

update t set b=100 where b>=120 ; # b>=120 命中了空的数据范围,
所以无 Record Locks,

无 Next-Key Locks

session B

```
insert into t values(200,200,200,200,200) ;    # 插入成功，因为无 Next-Key Locks
```

场景五

rr 模式+无索引筛选：（无索引筛选的情况下，不存在数据完全填充的场景）

数据：

数据范围有数据，但未被完全填充（11）

已有数据的更改

```
session A
start transaction ;
update t set c=100 where d=2 ;    # d=2 对应 a=1、a=2、a=3 的记录，
                                   因为 where 条件未使用索引，故只能全表扫描，
                                   并对所有行加 Record Locks，
                                   并且在间隙中加上 Gap Locks

session B
update t set b=100 where a=1 ;    # 等待 session A 获取的 a=1 的 X 锁
```

间隙的填充

因为 where 条件并未使用索引，所以最终加锁都回归到了对基表的聚簇索引加锁，
但是 where 条件未使用索引，自然更无唯一约束，

所以逻辑上可以认为除 where 命中的行以外的其他范围都是间隙。

而实际上因为通过聚簇索引加锁，所以在每两个聚簇索引之间才会加上 Gap Locks。

```
session A
start transaction ;
update t set c=100 where d=2 ;    # d=2 对应 a=1、a=2、a=3 的记录，
                                   因为 where 条件未使用索引，故只能全表扫描，
                                   并对所有聚簇索引加 Record Locks，
                                   并且加上 Next-Key Locks

session B
update t set b=50 where a=8 ;    # 等待 session A 获取的 a=8 的 X 锁
insert into t values(110,30,30,40,500);    # 等待 Next-Key Locks
```

数据范围内没有数据（12）

间隙的填充

因为 where 条件并未使用索引，所以最终加锁都回归到了对基表的聚簇索引加锁，
但是 where 条件未使用索引，自然更无唯一约束，

所以逻辑上可以认为除 where 命中的行以外的其他范围都是间隙。

而实际上因为通过聚簇索引加锁，所以在每两个聚簇索引之间才会加上 Gap Locks。

```
session A
start transaction ;
update t set c=100 where d=20 ;    # 因为 where 条件未使用索引，
                                   故只能全表扫描，
                                   并对所有聚簇索引加 Record Locks，
                                   并且加上 Next-Key Locks
```

```
session B
update t set b=50 where a=8 ;           # 等待 session A 获取的 a=8 的 X 锁
insert into t values (100,3,100,20,4) ; # 等待 Next-Key Locks
```

场景六

rc 模式+无索引筛选：（无索引筛选的情况下，不存在数据完全填充的场景）

数据：

数据范围有数据，但未被完全填充（13）

已有数据的更改

```
session A
start transaction ;
update t set c=100 where d=2 ;           # 对应 a=1、a=2、a=3 的记录，
                                           因为 where 条件未使用索引，故只能全表扫描，
                                           并对 a=1、a=2、a=3 聚簇索引加 Record Locks，
                                           但是无 Next-Key Locks
```

```
session B
update t set b=100 where a=1 ;           # 等待 session A 获取的 a=1 的 Record
```

Locks

间隙的填充

因为 where 条件并未使用索引，所以最终加锁都回归到了对基表的聚簇索引加锁，但是 where 条件未使用索引，自然更无唯一约束，

所以逻辑上可以认为除 where 命中的行以外的其他范围都是间隙。

而实际上因为通过聚簇索引加锁，所以在每两个聚簇索引之间才会加上 Gap Locks。

```
session A
start transaction ;
update t set c=100 where d=2 ;           # 对应 a=1、a=2、a=3 的记录，
                                           因为 where 条件未使用索引，故只能全表扫描，
                                           并对 a=1、a=2、a=3 聚簇索引加 Record Locks，
                                           但是无 Next-Key Locks
```

```
session B
update t set b=50 where a=8 ;           # 成功，因为 a=8 并未被 session A 加锁
insert into t values (110,30,30,2,500); # 成功，因为不存在 Gap Locks &
                                           next-Key Locks
```

数据范围内没有数据（14）

间隙的填充*

```
session A
start transaction ;
update t set c=100 where d=20 ;           # 无 Record Locks，
                                           也无 Next-Key Locks
```

```
session B
```



```
insert into t values (100,3,100,20,4) ;    # 成功
delete from t where a=100 ;                # 恢复现场
```

6 总结&延展：

唯一索引存在唯一约束，所以变更后的数据若违反了唯一约束的原则，则会失败。

当 where 条件使用二级索引筛选数据时，会对二级索引命中的条目和对应的聚簇索引都加锁；所以其他事务变更命中加锁的聚簇索引时，都会等待锁。

行锁的增加是一行一行增加的，所以可能导致并发情况下死锁的发生。

例如，

在 session A 对符合条件的某聚簇索引加锁时，可能 session B 已持有该聚簇索引的 Record Locks，而 session B 正在等待 session A 已持有的某聚簇索引的 Record Locks。session A 和 session B 是通过两个不相干的二级索引定位到的聚簇索引。session A 通过索引 idA，session B 通过索引 idB。

当 where 条件获取的数据无间隙时，无论隔离级别为 rc 或 rr，都不会存在间隙锁。比如通过唯一索引获取到了已完全填充的数据范围，此时不需要间隙锁。

间隙锁的目的在于阻止数据插入间隙，所以无论是通过 insert 或 update 变更导致的间隙内数据的存在，都会被阻止。

rc 隔离级别模式下，查询和索引扫描将禁用 gap locking，此时 gap locking 仅用于外键约束检查和重复键检查（主要是唯一性检查）。

rr 模式下，为了防止幻读，会加上 Gap Locks。事务中，SQL 开始则加锁，事务结束才释放锁。就锁类型而言，应该有优化锁，锁升级等，例如 rr 模式未使用索引查询的情况下，是否可以直接升级为表锁。就锁的应用场景而言，在回放场景中，如果确定事务可并发，则可以考虑不加锁，加快回放速度。锁只是并发控制的一种粒度，只是一个很小的部分：

从不同场景下是否需要控制并发，（已知无交集且有序的数据的变更，MySQL 的 MTS 相同前置事务的多事务并发回放）

并发控制的粒度，（锁是一种逻辑粒度，可能还存在物理层和其他逻辑粒度或方式）

相同粒度下的优化，（锁本身存在优化，如 IX、IS 类型的优化锁）

粒度加载的安全&性能（如获取行锁前，先获取页锁，页锁在执行获取行锁操作后即释放，无论是否获取成功）等多个层次去思考并发这玩意。



MySQL 碎片问题

作者：宗扬

MySQL 的碎片是 MySQL 运维过程中比较常见的问题，碎片的存在十分影响数据库的性能，本文将对 MySQL 碎片进行一次讲解。

1 判断方法：

MySQL 的碎片是否产生，通过查看

```
show table status from table_name\G;
```

这个命令中 Data_free 字段，如果该字段不为 0，则产生了数据碎片。

2 产生的原因：

2.1 经常进行 delete 操作

经常进行 delete 操作，产生空白空间，如果进行新的插入操作，MySQL 将尝试利用这些留空的区域，但仍然无法将其彻底占用，久而久之就产生了碎片；

演示：

创建一张表，往里面插入数据，进行一个带有 where 条件或者 limit 的 delete 操作，删除前后对比一下 Data_free 的变化。

删除前：

```
Data_free: 0
```

删除后：

```
root@5.6.37-log test 08:52:42>delete from t3 limit 100000;
Query OK, 100000 rows affected, 1 warning (0.30 sec)

root@5.6.37-log test 08:53:10>show table status like 't3'\G;
***** 1. row *****
      Name: t3
     Engine: InnoDB
    Version: 10
   Row_format: Compact
        Rows: 418740
 Avg_row_length: 51
  Data_length: 21544960
Max_data_length: 0
   Index_length: 0
  Data_free: 4194304
```

爱可生开源社区

Data_free 不为 0，说明有碎片；

2.2 update 更新

update 更新可变长度的字段(例如 varchar 类型)，将长的字符串更新成短的。之前存储的内容长，后来存储是短的，即使后来插入新数据，那么有一些空白区域还是没能有效利用的。

演示：

创建一张表，往里面插入一条数据，进行一个 update 操作，前后对比一下 Data_free 的变化。

```
CREATE TABLE `t1` ( `k` varchar(3000) DEFAULT NULL ) ENGINE=MyISAM DEFAULT
CHARSET=utf8;
```

更新语句: update t1 set k='aaa' ;

更新前长度: 223 Data_free: 0

更新后长度: 3 Data_free: 204

Data_free 不为 0，说明有碎片；

3 产生影响：

1. 由于碎片空间是不连续的，导致这些空间不能充分被利用；
2. 由于碎片的存在，导致数据库的磁盘 I/O 操作变成离散随机读写，加重了磁盘 I/O 的负担。

4 清理办法：

1. MyISAM: optimize table 表名；(OPTIMIZE 可以整理数据文件, 并重排索引)
2. InnoDB:
 - 1) ALTER TABLE tablename ENGINE=InnoDB；(重建表存储引擎，重新组织数据)
 - 2) 进行一次数据的导入导出

碎片清理的性能对比：

引用我之前一个生产库的数据，对比一下清理前后的差异。

5 空间对比：

库名	清理前大小	清理后大小
facebook	2.2G	1.1G
instagram	40G	22G
linkedin	555M	208M
googleplus	19G	8.4G
twitter	107G	44G

SQL 执行速度:

```
select count(*) from test.twitter_11;
```

修改前: 1 row in set (7.37 sec)

修改后: 1 row in set (1.28 sec)

6 结论:

通过对比, 可以看到碎片清理前后, 节省了很多空间, SQL 执行效率更快。所以, 在日常运维工作中, 应对碎片进行定期清理, 保证数据库有稳定的性能。



MySQL 默认值选型（是空，还是 NULL）

作者：周启超

如果对一个字段没有过多要求，是使用 “ ” 还是使用 NULL，一直是个让人困惑的问题。即使有前人留下的开发规范，但是能说清原因的也没有几个。NULL 是 “ ” 吗？在辨别 NULL 是不是空的这个问题上，感觉就像是在证明 1 + 1 是不是等于 2。

在 MySQL 中的 NULL 是一种特殊的数据。一个字段是否允许为 NULL，字段默认值是否为 NULL。

主要有如下几种情况：

字段类型	表定义中设置方式	字段值
数值类型 (INT/BIGINT)	Default NULL / Default 0	NULL / NUM
字符类型 (CHAR/VARCHAR)	Default NULL / Default " " / Default 'a b'	NULL / " " / String

1 NULL 与空字符存储上的区别

表中如果允许字段为 NULL，会为每行记录分配 NULL 标志位。NULL 除了在该行的行首存有 NULL 标志位，实际存储不占有任何空间。如果表中所有字段都是非 NULL，就不存在这个标示位了。网上有一些验证 MySQL 中 NULL 存储方式的文章，可以参考下。

2 NULL 使用上的一些问题

数值类型，对一个允许 NULL 的字段进行 min、max、sum、加减、order by、group by、distinct 等操作的时候。字段值为非 NULL 值时，操作很明确。如果使用 NULL，需要清楚的知道如下规则：

数值类型，以 INT 列为例

1) 在 min/max/sum /avg 中 NULL 值会被直接忽略掉，如下是测试结果，可能 min/max/sum 还比较可以理解，但 avg 真的是你想要的结果吗？

```
CREATE TABLE `t1` (  
  `id` int(16) NOT NULL AUTO_INCREMENT,  
  `name` varchar(20) DEFAULT NULL,  
  `number` int(11) DEFAULT NULL,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB AUTO_INCREMENT=5 DEFAULT CHARSET=utf8;
```

```

select * from t1;
+-----+-----+-----+
| id   | name   | number |
+-----+-----+-----+
| 1    | zhangsan | NULL   |
| 2    | lisi    | NULL   |
| 3    | wangwu  | 0      |
| 4    | zhangliu | 4      |
+-----+-----+-----+

select max(number) from t1;
+-----+
| max(number) |
+-----+
| 4           |
+-----+

select min(number) from t1;
+-----+
| min(number) |
+-----+
| 0           |
+-----+

select sum(number) from t1;
+-----+
| sum(number) |
+-----+
| 4           |
+-----+

select avg(number) from t1;
+-----+
| avg(number) |
+-----+
| 2.0000      |
+-----+

```

2) 对 NULL 做加减操作, 如 1 + NULL，结果仍是 NULL

```

select 1+NULL;
+-----+
| 1+NULL |
+-----+
| NULL   |
+-----+

```

3) order by 以升序检索字段的时候 NULL 会排在最前面（倒序相反）

```
select * from t1 order by number;
+----+-----+-----+
| id | name   | number |
+----+-----+-----+
| 1  | zhangsan | NULL   |
| 2  | lisi    | NULL   |
| 3  | wangwu  | 0      |
| 4  | zhangliu | 4      |
+----+-----+-----+
select * from t1 order by number desc;
+----+-----+-----+
| id | name   | number |
+----+-----+-----+
| 4  | zhangliu | 4      |
| 3  | wangwu  | 0      |
| 1  | zhangsan | NULL   |
| 2  | lisi    | NULL   |
+----+-----+-----+
```

4) group by/distinct 时，NULL 值被视为相同的值

```
select distinct(number) from t1;
+-----+
| number |
+-----+
| NULL   |
| 0      |
| 4      |
+-----+
select number,count(*) from t1 group by number;
+-----+-----+
| number | count(*) |
+-----+-----+
| NULL   | 2        |
| 0      | 1        |
| 4      | 1        |
+-----+-----+
```

字符类型，在使用 NULL 值的时候，也需要格外注意

1) 字段是字符时，你无法一目了然的区分这个值到底是 NULL，还是字符串 'NULL'

```
insert into t1 (name,number) values ('NULL',5);
insert into t1 (number) values (6);
select * from t1 where number in (5,6);
```

```
+-----+-----+-----+
| id | name | number |
+-----+-----+-----+
| 5 | NULL | 5 |
| 6 | NULL | 6 |
+-----+-----+-----+
```

```
select name is NULL from t1 where number=5;
```

```
+-----+
| name is NULL |
+-----+
|          0 |
+-----+
```

```
select name is NULL from t1 where number=6;
```

```
+-----+
| name is NULL |
+-----+
|          1 |
+-----+
```

2) 统计包含 NULL 字段的值，NULL 值不包括在里面

```
select count(*) from t1;
```

```
+-----+
| count(*) |
+-----+
|          6 |
+-----+
```

```
select count(name) from t1;
```

```
+-----+
| count(name) |
+-----+
|          5 |
+-----+
```

```
select * from t1 where name is null;
```

```
+-----+-----+-----+
| id | name | number |
+-----+-----+-----+
| 6 | NULL | 6 |
+-----+-----+-----+
```


3) 如果你用 `length` 去统计一个 `VARCHAR` 的长度时, `NULL` 返回的将不是数字

```
select length(name) from t1 where name is null;
+-----+
| length(name) |
+-----+
|           NULL          |
+-----+
```

3 总结

`NULL` 本身是一个特殊值, MySQL 采用特殊的方法来处理 `NULL` 值。从理解肉眼判断, 操作符运算等操作上, 可能和我们预期的效果不一致。可能会给我们项目上的操作不符合预期。

你必须要使用 `IS NULL/IS NOT NULL` 这种与普通 SQL 大相径庭的方式去处理 `NULL`。

尽管在存储空间上, 在索引性能上可能并不比空值差, 但是为了避免其身上特殊性, 给项目带来不确定因素, 因此建议默认值不要使用 `NULL`。



MySQL 大对象一例

作者：杨涛涛

1 背景

MySQL 一直以来都有 TEXT、BLOB 等类型用来存储图片、视频等大对象信息。比如一张图片，随便一张都 5M 以上。视频也是，随便一部视频就是 2G 以上。

假设用 MySQL 来存放电影视频等信息，一部是 2G，那么存储 1000 部就是 2TB，2TB 也就是 1000 条记录而已，但是对数据库性能来说，不仅仅是看记录数量，更主要的还得看占用磁盘空间大小。空间大了，所有以前的经验啥的都失效了。

所以一般来说存放这类信息，也就是存储他们的存放路径，至于文件本身存放在哪里，那这就不是数据库考虑的范畴了。数据库只关心怎么来的快，怎么来的小。

2 举例

虽然不推荐 MySQL 这样做，但是也得知道 MySQL 该怎么做才行，做到心里有数。比如下面一张微信图片，大概 5M 的样子。

```
root@ytt:/var/lib/mysql-files# ls -sihl 微信图片_20190711095019.jpg
274501 5.4M -rw-r--r-- 1 root root 5.4M Jul 11 07:17 微信图片_20190711095019.jpg
```

拷贝 100 份这样的图片来测试

```
root@ytt:/var/lib/mysql-files# for i in `seq 1 100`; do cp 微信图片_20190711095019.jpg "$i".jpg;done;
root@ytt:/var/lib/mysql-files# ls
100.jpg  17.jpg  25.jpg  33.jpg  41.jpg  4.jpg   58.jpg  66.jpg  74.jpg  82.jpg
90.jpg  99.jpg  f8.tsv
10.jpg   18.jpg  26.jpg  34.jpg  42.jpg  50.jpg  59.jpg  67.jpg  75.jpg  83.jpg
91.jpg   9.jpg   微信图片_20190711095019.jpg
1111.jpg 19.jpg  27.jpg  35.jpg  43.jpg  51.jpg  5.jpg   68.jpg  76.jpg  84.jpg
92.jpg   f1.tsv
11.jpg   1.jpg   28.jpg  36.jpg  44.jpg  52.jpg  60.jpg  69.jpg  77.jpg  85.jpg
93.jpg   f2.tsv
12.jpg   20.jpg  29.jpg  37.jpg  45.jpg  53.jpg  61.jpg  6.jpg   78.jpg  86.jpg
94.jpg   f3.tsv
13.jpg   21.jpg  2.jpg   38.jpg  46.jpg  54.jpg  62.jpg  70.jpg  79.jpg  87.jpg
95.jpg   f4.tsv
```

```

14.jpg 22.jpg 30.jpg 39.jpg 47.jpg 55.jpg 63.jpg 71.jpg 7.jpg 88.jpg
96.jpg f5.tsv
15.jpg 23.jpg 31.jpg 3.jpg 48.jpg 56.jpg 64.jpg 72.jpg 80.jpg 89.jpg
97.jpg f6.tsv
16.jpg 24.jpg 32.jpg 40.jpg 49.jpg 57.jpg 65.jpg 73.jpg 81.jpg 8.jpg
98.jpg f7.tsv

```

我们建三张表，分别用 LONGBLOB、LONGTEXT 和 VARCHAR 来存储这些图片信息

```

mysql> show create table tt_image1\G
***** 1. row *****
      Table: tt_image1
Create Table: CREATE TABLE `tt_image1` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `image_file` longblob,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
mysql> show create table tt_image2\G
***** 1. row *****
      Table: tt_image2
Create Table: CREATE TABLE `tt_image2` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `image_file` longtext,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
mysql> show create table tt_image3\G
***** 1. row *****
      Table: tt_image3
Create Table: CREATE TABLE `tt_image3` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `image_file` varchar(100) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)

```

我们来给三张表插入 100 张图片（插入前，建议把 max_allowed_packet 设置到最大）

```

tt_image1
root@ytt:/var/lib/mysql-files# for i in `seq 1 100`; \
do mysql -S /var/run/mysqld/mysqld.sock -e "insert into ytt.tt_image1(image_file)
\
values (load_file('/var/lib/mysql-files/${i}.jpg'))";done;
tt_image2
root@ytt:/var/lib/mysql-files# for i in `seq 1 100`; \

```

```
do mysql -S /var/run/mysqld/mysqld.sock -e "insert into ytt.tt_image2(image_file) \
values (hex(load_file('/var/lib/mysql-files/$i.jpg')));"done;
tt_image3
root@ytt:/var/lib/mysql-files# aa='begin;';for i in `seq 1 100`; \
do aa=$aa"insert into ytt.tt_image3(image_file) values \
('/var/lib/mysql-files/$i.jpg');" \
done;aa=$aa'commit;';mysql -S /var/run/mysqld/mysqld.sock -e "`echo $aa`";
```

检查下三张表记录数

```
mysql> select 'tt_image1' as name ,count(*) from tt_image1 union all \
select 'tt_image2',count(*) from tt_image2 union all select 'tt_image3', count(*)
from tt_image3;
+-----+-----+
| name      | count(*) |
+-----+-----+
| tt_image1 |      100 |
| tt_image2 |      100 |
| tt_image3 |      100 |
+-----+-----+
3 rows in set (0.00 sec)
```

看下文件大小，可以看到实际大小排名，LONGTEXT 字段存储的最大，LONGBLOB 字段缩小到一半，最小的是存储图片路径的表 tt_image3。所以这里从存储空间来看，存放路径最占优势。

```
root@ytt:/var/lib/mysql/ytt# ls -silhS tt_image*
274603 1.1G -rw-r----- 1 mysql mysql 1.1G Jul 11 07:27 tt_image2.ibd
274602 545M -rw-r----- 1 mysql mysql 544M Jul 11 07:26 tt_image1.ibd
274605 80K -rw-r----- 1 mysql mysql 112K Jul 11 07:27 tt_image3.ibd
```

那么怎么把图片取出来呢？

tt_image3 肯定是最容易的

```
mysql> select * from tt_image3;
+----+-----+
| id | image_file      |
+----+-----+
| 1  | /var/lib/mysql-files/1.jpg |
+----+-----+
...
100 rows in set (0.00 sec)
```

tt_image1 直接导出来二进制文件即可，下面我写了个存储过程，导出所有图片。

```
mysql> DELIMITER $$
mysql> USE `ytt`$$
mysql> DROP PROCEDURE IF EXISTS `sp_get_image`$$
mysql> CREATE DEFINER=`ytt`@`localhost` PROCEDURE `sp_get_image`()
mysql> BEGIN
    DECLARE i,cnt INT DEFAULT 0;
    SELECT COUNT(*) FROM tt_image1 WHERE 1 INTO cnt;
    WHILE i < cnt DO
        SET @stmt = CONCAT('select image_file from tt_image1 limit ',i,',1 into
dumpfile ''/var/lib/mysql-files/image',i,'.jpg'');
        PREPARE s1 FROM @stmt;
        EXECUTE s1;
        DROP PREPARE s1;
        SET i = i + 1;
    END WHILE;
END$$
mysql> DELIMITER ;
mysql> call sp_get_image;
```

tt_image2 类似，把 select 语句里 image_file 变为 unhex(image_file) 即可。

3 总结

这里我举了个用 MySQL 来存放图片的例子，总的来说有以下三点：

1. 占用磁盘空间大（这样会带来各种各样的功能与性能问题，比如备份，写入，读取操作等）；
2. 使用不易；
3. 还是推荐用文件路径来代替实际的文件内容存放。



死锁分析案例

作者：高鹏（八怪）

1 问题由来

这是我同事问我的一个问题，在网上看到了如下案例，本案例 RC RR 都可以出现，其实这个死锁原因也比较简单，我们来具体看看：

构造数据

```
CREATE database deadlock_test;
use deadlock_test;
CREATE TABLE `push_token` (
  `id` bigint(20) NOT NULL AUTO_INCREMENT,
  `token` varchar(128) NOT NULL COMMENT 'push token',
  `app_id` varchar(128) DEFAULT NULL COMMENT 'appid',
  `deleted` tinyint(1) NOT NULL COMMENT '是否已删除 0: 否 1: 是',
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_token_appid` (`token`,`app_id`)
) ENGINE=InnoDB AUTO_INCREMENT=3384 DEFAULT CHARSET=utf8 COMMENT='pushtoken 表';
insert into push_token (id, token, app_id, deleted) values(1,"token1",1,0);
```

操作数据

s1 (TRX_ID367661)	s2 (TRX_ID367662)	s3 (TRX_ID367663)
begin; UPDATE push_token SET deleted = 1 WHERE token = 'token1' AND app_id = '1';		
	begin; DELETE FROM push_token WHERE id IN (1);	
		begin; UPDATE push_token SET deleted = 1 WHERE token = 'token1' AND app_id = '1';
commit;		
Query OK, 0 rows affected (0.00 sec)	Query OK, 1 row affected (17.32 sec)	ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction

2 分析方法

我使用的分析方法是把整个加锁的日志打印出来，当然需要用到我自己做了输出修改的一个版本，如下：

<https://github.com/gaopengcarl/percona-server-locks-detail-5.7.22>

这个版本我打开了的日志记录参数如下：

```
mysql> show variables like '%gaopeng%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| gaopeng_mdl_detail | OFF |
| innodb_gaopeng_row_lock_detail | ON |
+-----+-----+
2 rows in set (0.01 sec)
```

这样大部分的 InnoDB 加锁记录都会记录到 errlog 日志了。好了下面我详细分析一下日志：

3 分析过程

初始化的情况整个表只有 1 条记录，本表包含一个主键和一个唯一键。

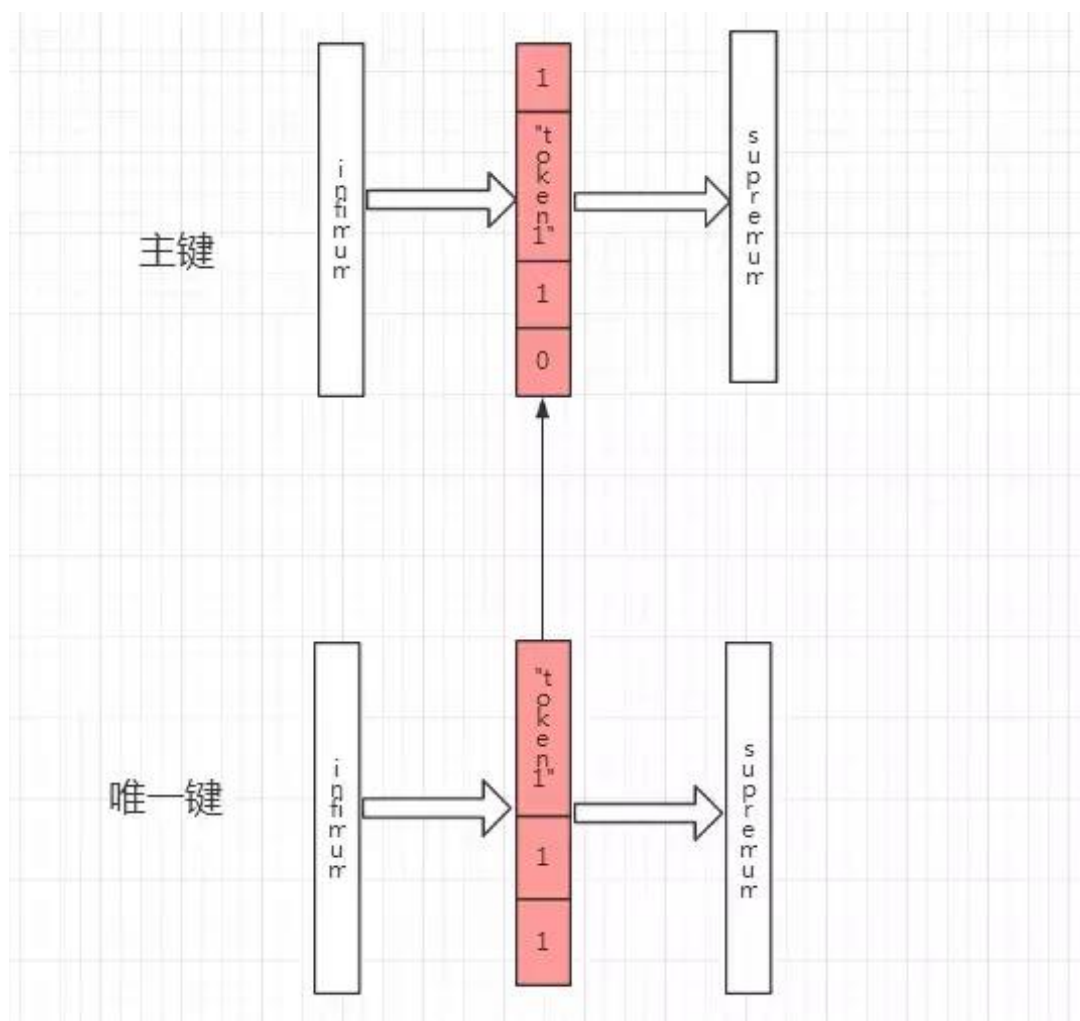
1. s1 (TRX_ID367661) 执行语句

```
begin;
UPDATE push_token SET deleted = 1 WHERE token = 'token1' AND app_id = '1';
```

日志输出：

```
2019-08-18T19:10:05.117317+08:00 6 [Note] InnoDB: TRX ID:(367661)
table:deadlock_test/push_token index:uk_token_appid space_id: 449 page_id:4
heap_no:2 row lock mode:LOCK_X|LOCK_NOT_GAP|
PHYSICAL RECORD: n_fields 3; compact format; info bits 0
0: len 6; hex 746f6b6556e31; asc token1;;
1: len 1; hex 31; asc 1;;
2: len 8; hex 800000000000000001; asc ;;
2019-08-18T19:10:05.117714+08:00 6 [Note] InnoDB: TRX ID:(367661)
table:deadlock_test/push_token index:PRIMARY space_id: 449 page_id:3 heap_no:2
row lock mode:LOCK_X|LOCK_NOT_GAP|
PHYSICAL RECORD: n_fields 6; compact format; info bits 0
0: len 8; hex 800000000000000001; asc ;;
1: len 6; hex 0000000059c2c; asc ,;;
2: len 7; hex bf0000000420110; asc B ;;
3: len 6; hex 746f6b6556e31; asc token1;;
4: len 1; hex 31; asc 1;;
5: len 1; hex 80; asc ;;
```

我们看到主键和唯一键都加锁了，模式为 LOCKX|LOCKNOT_GAP|如下图：



并且这个时候数据实际上是标记删除状态。

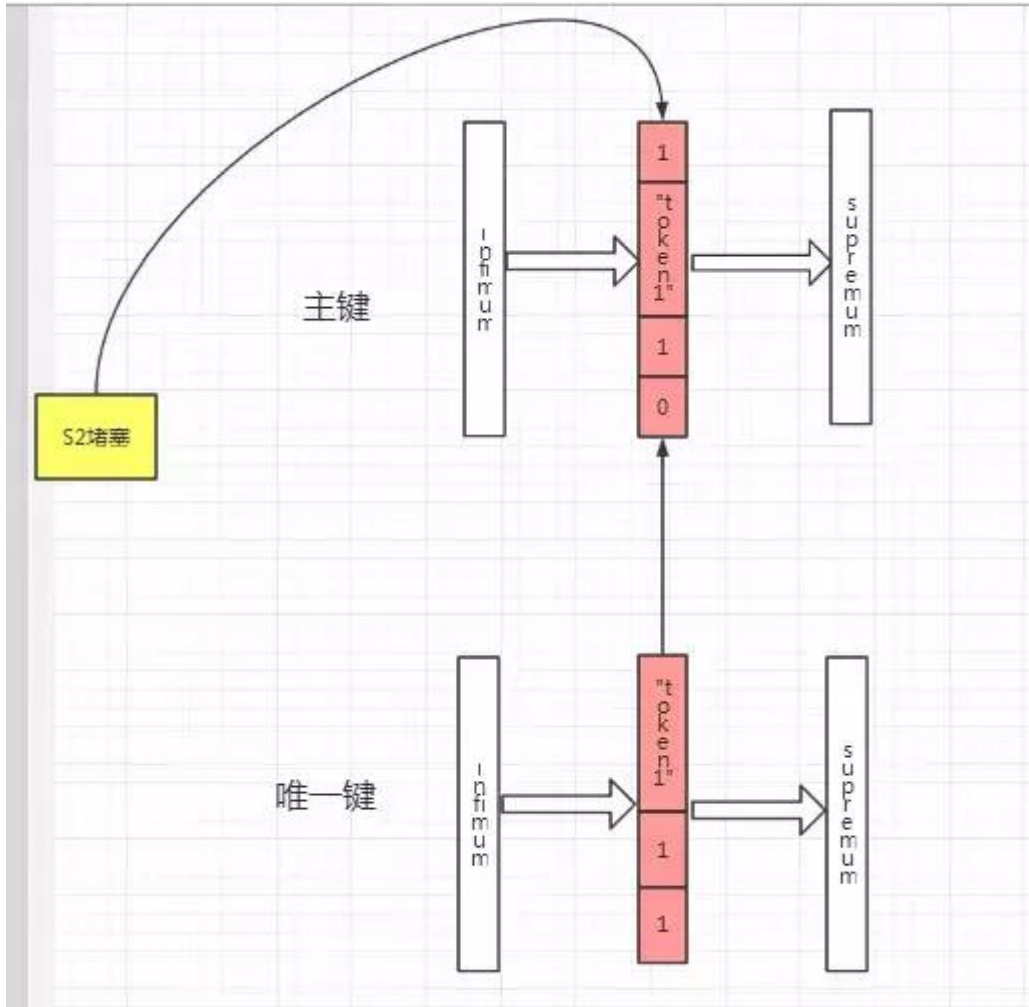
2. s2 (TRX_ID367662) 执行语句

```
begin;DELETE FROM push_token WHERE id IN (1);
```

日志输出:

```
2019-08-18T19:10:22.751467+08:00 9 [Note] InnoDB: TRX ID:(367662)
table:deadlock_test/push_token index:PRIMARY space_id: 449 page_id:3 heap_no:2
row lock mode:LOCK_X|LOCK_NOT_GAP|
PHYSICAL RECORD: n_fields 6; compact format; info bits 0
0: len 8; hex 8000000000000001; asc ;;
1: len 6; hex 000000059c2d; asc -;;
2: len 7; hex 400000002a1dc8; asc @ * ;;
3: len 6; hex 746f6b656e31; asc token1;;
4: len 1; hex 31; asc 1;;
5: len 1; hex 81; asc ;;
2019-08-18T19:10:22.752753+08:00 9 [Note] InnoDB: Trx(367662) is blocked!!!!
```


这个时候 S2 需要获取主键上的:LOCKX|LOCKNOT_GAP| 锁，因此被堵塞了如下图：



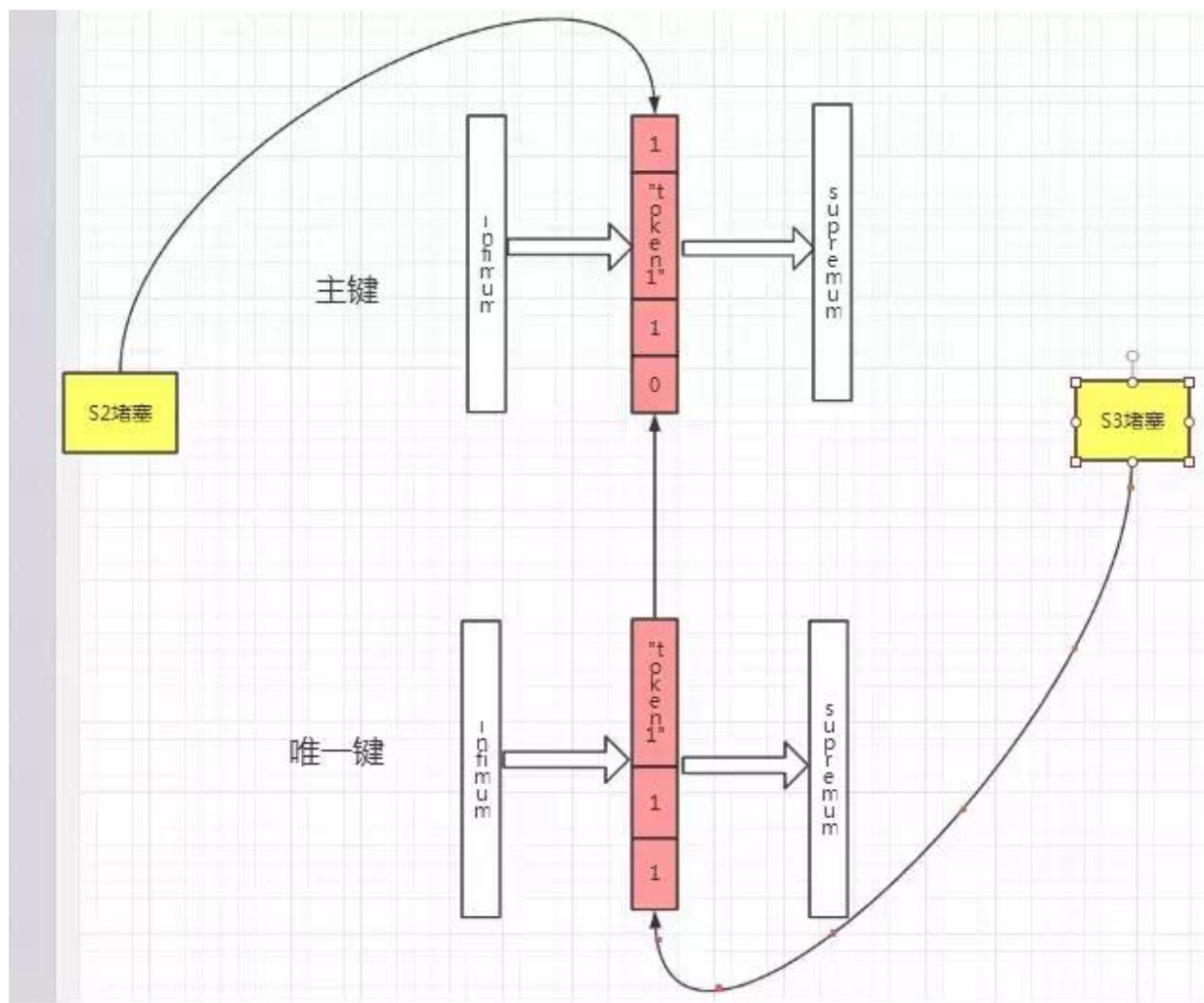
3. s3 (TRX_ID367663) 执行语句

```
begin; UPDATE push_token SET deleted = 1 WHERE token = 'token1' AND app_id = '1';
```

日志输出：

```
2019-08-18T19:10:30.822111+08:00 8 [Note] InnoDB: TRX ID:(367663)
table:deadlock_test/push_token index:uk_token_appid space_id: 449 page_id:4
heap_no:2 row lock mode:LOCK_X|LOCK_NOT_GAP|
PHYSICAL RECORD: n_fields 3; compact format; info bits 0
0: len 6; hex 746f6b6556e31; asc token1;;
1: len 1; hex 31; asc 1;;
2: len 8; hex 8000000000000001; asc ;;
2019-08-18T19:10:30.918248+08:00 8 [Note] InnoDB: Trx(367663) is blocked!!!!
```

这个时候 S3 需要获取唯一键上的 LOCKX|LOCKNOT_GAP 锁，因此被堵塞了如下图：



4. s1 (TRX_ID367661) 执行语句

这一步完成后死锁出现。

```
commit;
```

日志输出如下：

367663 和 367662 各自获取需要的锁

2019-08-18T19:10:36.566733+08:00 8 [Note] InnoDB: TRX ID:(367663)

table:deadlock_test/push_token index:uk_token_appid space_id: 449 page_id:4

heap_no:2 row lock mode:LOCK_X|LOCK_NOT_GAP|

PHYSICAL RECORD: n_fields 3; compact format; info bits 0

0: len 6; hex 746f6b656e31; asc token1;;

1: len 1; hex 31; asc 1;;

2: len 8; hex 8000000000000001; asc ;;

2019-08-18T19:10:36.568711+08:00 9 [Note] InnoDB: TRX ID:(367662)

table:deadlock_test/push_token index:PRIMARY space_id: 449 page_id:3 heap_no:2

row lock mode:LOCK_X|LOCK_NOT_GAP|

PHYSICAL RECORD: n_fields 6; compact format; info bits 0

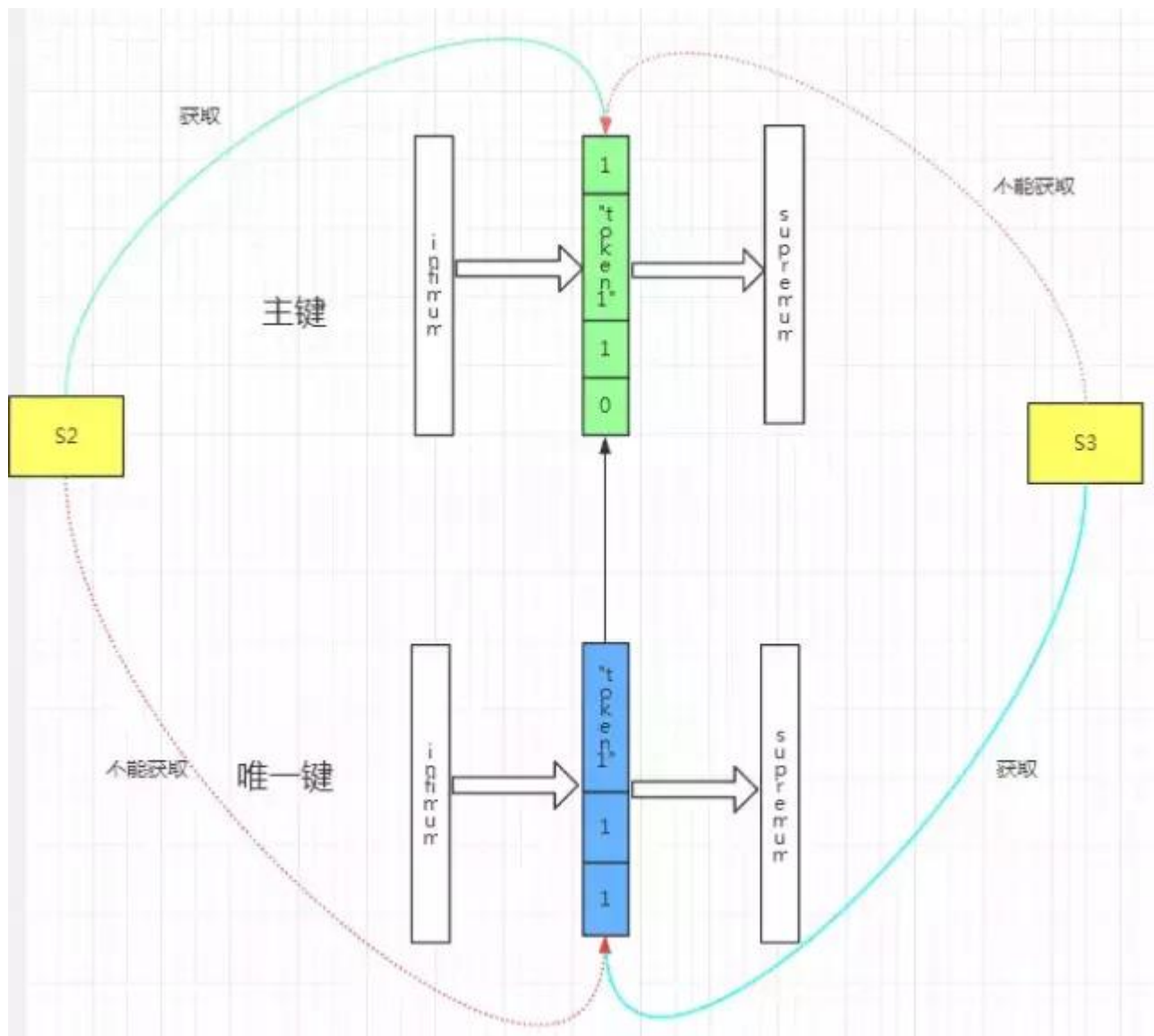
```

0: len 8; hex 800000000000000001; asc ;;
1: len 6; hex 0000000059c2d; asc -;;
2: len 7; hex 4000000002a1dc8; asc @ * ;;
3: len 6; hex 746f6b656e31; asc token1;;
4: len 1; hex 31; asc 1;;
5: len 1; hex 81; asc ;;
367663 获取主键锁堵塞、367662 获取唯一键锁堵塞，死锁形成
2019-08-18T19:10:36.570313+08:00 8 [Note] InnoDB: TRX ID:(367663)
table:deadlock_test/push_token index:PRIMARY space_id: 449 page_id:3 heap_no:2
row lock mode:LOCK_X|LOCK_NOT_GAP|
PHYSICAL RECORD: n_fields 6; compact format; info bits 0
0: len 8; hex 800000000000000001; asc ;;
1: len 6; hex 0000000059c2d; asc -;;
2: len 7; hex 4000000002a1dc8; asc @ * ;;
3: len 6; hex 746f6b656e31; asc token1;;
4: len 1; hex 31; asc 1;;
5: len 1; hex 81; asc ;;
2019-08-18T19:10:36.571199+08:00 8 [Note] InnoDB: Trx(367663) is blocked!!!!
2019-08-18T19:10:36.572481+08:00 9 [Note] InnoDB: TRX ID:(367662)
table:deadlock_test/push_token index:uk_token_appid space_id: 449 page_id:4
heap_no:2 row lock mode:LOCK_X|LOCK_NOT_GAP|
PHYSICAL RECORD: n_fields 3; compact format; info bits 0
0: len 6; hex 746f6b656e31; asc token1;;
1: len 1; hex 31; asc 1;;
2: len 8; hex 800000000000000001; asc ;;
2019-08-18T19:10:36.573073+08:00 9 [Note] InnoDB: Transactions deadlock detected,
dumping detailed information.

```

死锁分析案例

完成这一步 s1 实际上释放了锁，然后我们首先看到 s2 获取了主键上的 LOCKX|LOCKNOTGAP 锁，s3 获取了唯一键上的 LOCKX|LOCKNOTGAP 锁。但是随后 s3 获取主键上的 LOCKX|LOCKNOTGAP 锁堵塞，s2 获取唯一键上的 LOCKX|LOCKNOTGAP 锁堵塞。因此死锁形成，如下图：



好了我们看到了死锁就这样出现。整个分析过程我们只要看到加锁的日志实际上很容易就分析得出来。



innodb-cluster 扫盲-安装篇

作者：杨涛涛

本文介绍用 MySQL Shell 搭建 MGR 的详细过程。

1 使用前，关掉防火墙

包括 selinux, firewalld, 或者 MySQL 企业版的 firewall (如果用了企业版的话)

2 两台机器：(4 台 MySQL 实例)

```
192.168.2.219 centos-ytt57-1 3311/3312
192.168.2.229 centos-ytt57-2 3311/3312
```

3 安装

安装 MySQL (两台都装), MySQL Shell (任意一台), mysqlrouter (任意一台, 官方建议和应用程序装在一个服务器上)

```
yum install mysql-community-server mysql-shell mysql-router-community
```

4 搭建 InnoDB-Cluster (两台都装)

1. 配置文件如下:

```
shell>vi /etc/my.cnf
master-info-repository=table
relay-log-info-repository=table
gtid_mode=ON
enforce_gtid_consistency=ON
binlog_checksum=NONE
log_slave_updates=ON
binlog_format=ROW
transaction_write_set_extraction=XXHASH64
```

2. 系统 HOSTS 配置 (两台都配)

```
shell>vi /etc/hosts
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
```

192.168.2.219 centos-ytt57-2

192.168.2.229 centos-ytt57-3

用 MySQL coalesce 函数确认下 report-host 是否设置正确

```
(root@localhost) : [(none)] >SELECT coalesce(@@report_host, @@hostname) as r;
+-----+
| r      |
+-----+
| centos-ytt57-1 |
+-----+
1 row in set (0.00 sec)
```

3. 创建管理员用户搭建 GROUP REPLICATION （四个节点都要）

```
create user root identified by 'Root@123';
grant all on *.* to root with grant option;
flush privileges;
```

4. MySQLsh 连接其中一台节点:

```
[root@centos-ytt57-1 mysql]# mysqlsh root@localhost:3321
```

5. 检查配置正确性:

如果这里不显示 OK, 把对应的参数加到配置文件重启 MySQL 再次检查

```
dba.checkInstanceConfiguration("root@centos-ytt57-2:3311");
dba.checkInstanceConfiguration("root@centos-ytt57-2:3312");
dba.checkInstanceConfiguration("root@centos-ytt57-3:3311");
dba.checkInstanceConfiguration("root@centos-ytt57-3:3312");
mysqlsh 执行检测
```

```
[root@centos-ytt57-1 mysql]# mysqlsh --log-level=8 root@localhost:3311
MySQL localhost:3311 ssl JS > dba.checkInstanceConfiguration("root@centos-
ytt57-2:3311")
{
  "status": "ok"
}
```

6. 创建集群, 节点 1 上创建。(结果显示 Cluster successfully created 表示成功)

```
MySQL localhost:3311 ssl JS > var cluster = dba.createCluster('ytt_mgr');
Cluster successfully created. Use Cluster.addInstance() to add MySQL instances.
At least 3 instances are needed for the cluster to be able to withstand up to
one server failure.
```

7. 添加节点 2, 3, 4 (全部显示 OK, 表示添加成功)

```
MySQL localhost:3311 ssl JS > cluster.addInstance('root@centos-ytt57-2:3312');
MySQL localhost:3311 ssl JS > cluster.addInstance('root@centos-ytt57-3:3311');
MySQL localhost:3311 ssl JS > cluster.addInstance('root@centos-ytt57-3:3312');
```

8. 查看拓扑图: (describe 简单信息, status 详细描述)

```
MySQL localhost:3311 ssl JS > cluster.describe()
{
  "clusterName": "ytt_mgr",
  "defaultReplicaSet": {
    "name": "default",
    "topology": [
      {
        "address": "centos-ytt57-2:3311",
        "label": "centos-ytt57-2:3311",
        "role": "HA",
        "version": "8.0.17"
      },
      {
        "address": "centos-ytt57-2:3312",
        "label": "centos-ytt57-2:3312",
        "role": "HA",
        "version": "8.0.17"
      },
      {
        "address": "centos-ytt57-3:3311",
        "label": "centos-ytt57-3:3311",
        "role": "HA",
        "version": "8.0.17"
      },
      {
        "address": "centos-ytt57-3:3312",
        "label": "centos-ytt57-3:3312",
        "role": "HA",
        "version": "8.0.17"
      }
    ],
    "topologyMode": "Single-Primary"
  }
}
```

```
MySQL localhost:3311 ssl JS > cluster.status()
"clusterName": "ytt_mgr",
"defaultReplicaSet": {
```

```
"name": "default",
"primary": "centos-ytt57-2:3311",
"ssl": "REQUIRED",
"status": "OK",
"statusText": "Cluster is ONLINE and can tolerate up to ONE failure.",
"topology": {
  "centos-ytt57-2:3311": {
    "address": "centos-ytt57-2:3311",
    "mode": "R/W",
    "readReplicas": {},
    "role": "HA",
    "status": "ONLINE",
    "version": "8.0.17"
  },
  "centos-ytt57-2:3312": {
    "address": "centos-ytt57-2:3312",
    "mode": "R/O",
    "readReplicas": {},
    "role": "HA",
    "status": "ONLINE",
    "version": "8.0.17"
  },
  "centos-ytt57-3:3311": {
    "address": "centos-ytt57-3:3311",
    "mode": "R/O",
    "readReplicas": {},
    "role": "HA",
    "status": "ONLINE",
    "version": "8.0.17"
  },
  "centos-ytt57-3:3312": {
    "address": "centos-ytt57-3:3312",
    "mode": "R/O",
    "readReplicas": {},
    "role": "HA",
    "status": "ONLINE",
    "version": "8.0.17"
  }
},
"topologyMode": "Single-Primary"
},
"groupInformationSourceMember": "centos-ytt57-2:3311"
```

9. 简单测试下数据是否同步

```
(root@localhost) : [(none)] >create database ytt;
```



```

Query OK, 1 row affected (0.03 sec)
(root@localhost) : [(none)] >use ytt;
Database changed
(root@localhost) : [ytt] >create table p1(id int primary key, log_time datetime);
Query OK, 0 rows affected (0.08 sec)
(root@localhost) : [ytt] >insert into p1 values (1,now());
Query OK, 1 row affected (0.04 sec)
(root@localhost) : [ytt] >show master status;
+-----+-----+-----+-----+-----+
| File           | Position | Binlog_Do_DB | Binlog_Ignore_DB | Executed_Gtid_Set |
+-----+-----+-----+-----+-----+
| mysql0.000001 | 25496   |              |                  | 6c7bb9db-b759-11e9-
a9c0-0800276cf0fc:1-41 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

```

查看其他三个节点

```

(root@localhost) : [ytt] >show tables;
+-----+
| Tables_in_ytt |
+-----+
| p1             |
+-----+
1 row in set (0.00 sec)
(root@localhost) : [ytt] >select * from p1;
+----+-----+
| id | log_time           |
+----+-----+
| 1  | 2019-08-05 16:44:20 |
+----+-----+
1 row in set (0.00 sec)

```

停掉主节点:

```
[root@centos-ytt57-2 mysql0]# systemctl stop mysqld@0
```

现在查看, 主节点已经变为本机 3312 节点

```

"centos-ytt57-2:3312": {
  "address": "centos-ytt57-2:3312",
  "mode": "R/W",

```

```
"readReplicas": {},  
"role": "HA",  
"status": "ONLINE"  
}
```

10. 报错处理

错误日志里显示

```
2019-08-05T09:01:35.125591Z 0 [ERROR] Plugin group_replication reported: 'The  
group name option is mandatory'  
2019-08-05T09:01:35.125622Z 0 [ERROR] Plugin group_replication reported: 'Unable  
to start Group Replication on boot'
```

同时用 `cluster.rescan()` 扫描, 发现

```
The instance 'centos-ytt57-2:3311' is no longer part of the ReplicaSet.
```

重新加入此节点到集群:

```
cluster.rejoinInstance('centos-ytt57-2:3311')  
再次执行 cluster.status() 查看集群状态: "status": "OK",
```

11. 移除和加入

```
cluster.removeInstance("centos-ytt57-3:3312");  
The instance 'centos-ytt57-3:3312' was successfully removed from the cluster.  
cluster.addInstance("centos-ytt57-3:3312");  
The instance 'centos-ytt57-3:3312' was successfully added to the cluster.
```

12. 用 mysqlrouter 生成连接 MGR 相关信息

涉及到两个用户:

`--user=mysqlrouter` 是使用 `mysqlrouter` 的系统用户
自动创建的 MySQL 用户是用来与 MGR 通信的用户。

如果想查看这个用户的用户名以及密码, 就加上 `--force-password-validation`, 不过一般也没有必要查看。

```
[root@centos-ytt57-2 ytt]# mysqlrouter --bootstrap root@centos-ytt57-2:3311 --  
user=mysqlrouter --force-password-validation --report-host centos-ytt57-2  
Please enter MySQL password for root:  
# Reconfiguring system MySQL Router instance...  
- Checking for old Router accounts  
- Found old Router accounts, removing  
- Creating mysql account mysql_router1_rdr89tx20r0a@'%' for cluster management
```

```

- Storing account in keyring
- Adjusting permissions of generated files
- Creating configuration /etc/mysqlrouter/mysqlrouter.conf
# MySQL Router configured for the InnoDB cluster 'ytt_mgr'
After this MySQL Router has been started with the generated configuration
$ /etc/init.d/mysqlrouter restart
or
$ systemctl start mysqlrouter
or
$ mysqlrouter -c /etc/mysqlrouter/mysqlrouter.conf
the cluster 'ytt_mgr' can be reached by connecting to:
## MySQL Classic protocol
- Read/Write Connections: centos-ytt57-2:6446
- Read/Only Connections: centos-ytt57-2:6447
## MySQL X protocol
- Read/Write Connections: centos-ytt57-2:64460
- Read/Only Connections: centos-ytt57-2:64470

```

13. 启动 mysqlrouter, 生效对应服务。

```
[root@centos-ytt57-2 mysqlrouter]# systemctl start mysqlrouter
```

14. 使用 ROUTER 连接 MySQL

创建一个普通用户使用 ROUTER (四个节点都建立, 具体权限看个人)

```

[root@centos-ytt57-2 mysqlrouter]# /home/ytt/enter_mysql 80
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1779
Server version: 8.0.17 MySQL Community Server - GPL
Copyright (c) 2000, 2019, Oracle and/or its affiliates. All rights reserved.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
(root@centos-ytt57-2) : [(none)] >create user ytt_mysqlrouter;
Query OK, 0 rows affected (0.12 sec)
(root@centos-ytt57-2) : [(none)] >alter user ytt_mysqlrouter identified by
'mysql_router';
Query OK, 0 rows affected (0.09 sec)
(root@centos-ytt57-2) : [(none)] >grant all on ytt.* to ytt_mysqlrouter;
Query OK, 0 rows affected (0.05 sec)
(root@centos-ytt57-2) : [(none)] >flush privileges;
Query OK, 0 rows affected (0.07 sec)

```

14.1 写入端口连接:

```
[root@centos-ytt57-2 mysqlrouter]# mysql -uytt_mysqlrouter -pmysql_router -
hcentos-ytt57-2 -P6446
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 2030
Server version: 8.0.17 MySQL Community Server - GPL
Copyright (c) 2000, 2019, Oracle and/or its affiliates. All rights reserved.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
(ytt_mysqlrouter@centos-ytt57-2) : [(none)] >use ytt
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
Database changed
(ytt_mysqlrouter@centos-ytt57-2) : [ytt] >insert into p1 values (200,now());
Query OK, 1 row affected (0.04 sec)
(ytt_mysqlrouter@centos-ytt57-2) : [ytt] >select * from p1;
+-----+-----+
| id | log_time          |
+-----+-----+
| 1 | 2019-08-05 23:15:38 |
| 2 | 2019-08-05 23:15:42 |
| 100 | 2019-08-05 23:52:58 |
| 200 | 2019-08-05 23:56:23 |
+-----+-----+
4 rows in set (0.00 sec)
(ytt_mysqlrouter@centos-ytt57-2) : [ytt] >\q
Bye
[root@centos-ytt57-2 mysqlrouter]#
```

14.2 读取端口连接: (默认循环选择读服务节点)

```
[root@centos-ytt57-2 mysqlrouter]# mysql -uytt_mysqlrouter -pmysql_router -
hcentos-ytt57-2 -P6447
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 472
Server version: 8.0.17 MySQL Community Server - GPL
Copyright (c) 2000, 2019, Oracle and/or its affiliates. All rights reserved.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```

(ytt_mysqlrouter@centos-ytt57-2) : [(none)] >use ytt
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
Database changed
(ytt_mysqlrouter@centos-ytt57-2) : [ytt] >insert into p1 values (300,now());
ERROR 1290 (HY000): The MySQL server is running with the --read-only option so
it cannot execute this statement
(ytt_mysqlrouter@centos-ytt57-2) : [ytt] >select * from p1;
+-----+-----+
| id | log_time          |
+-----+-----+
|  1 | 2019-08-05 23:15:38 |
|  2 | 2019-08-05 23:15:42 |
| 100 | 2019-08-05 23:52:58 |
| 200 | 2019-08-05 23:56:23 |
+-----+-----+
4 rows in set (0.00 sec)
## (ytt_mysqlrouter@centos-ytt57-2) : [ytt] >\s
mysql Ver 8.0.17 for Linux on x86_64 (MySQL Community Server - GPL)
Connection id:          472
Current database:       ytt
Current user:           ytt_mysqlrouter@centos-ytt57-2
SSL:                    Cipher in use is DHE-RSA-AES128-GCM-SHA256
Current pager:          stdout
Using outfile:          ''
Using delimiter:        ;
Server version:         8.0.17 MySQL Community Server - GPL
Protocol version:       10
Connection:             centos-ytt57-2 via TCP/IP
Server characterset:    utf8mb4
Db      characterset:    utf8mb4
Client characterset:    utf8mb4
Conn.  characterset:    utf8mb4
TCP port:               6447
Uptime:                 45 min 10 sec
## Threads: 5 Questions: 188 Slow queries: 0 Opens: 257 Flush tables: 3 Open
tables: 161 Queries per second avg: 0.069
(ytt_mysqlrouter@centos-ytt57-2) : [ytt] >select @@server_id;
+-----+
| @@server_id |
+-----+
|          2 |
+-----+
1 row in set (0.01 sec)
(ytt_mysqlrouter@centos-ytt57-2) : [ytt] >\q
Bye

```

15. 机器全挂了，重启 MGR

```
MySQL localhost:3311 ssl JS > dba.rebootClusterFromCompleteOutage()
Reconfiguring the default cluster from complete outage...
The instance 'centos-ytt57-2:3312' was part of the cluster configuration.
Would you like to rejoin it to the cluster? [y/N]: y
The instance 'centos-ytt57-3:3311' was part of the cluster configuration.
Would you like to rejoin it to the cluster? [y/N]: y
The instance 'centos-ytt57-3:3312' was part of the cluster configuration.
Would you like to rejoin it to the cluster? [y/N]: y
The safest and most convenient way to provision a new instance is through
automatic clone provisioning, which will completely overwrite the state of
'localhost:3311' with a physical snapshot from an existing cluster member. To
use this method by default, set the 'recoveryMethod' option to 'clone'.
The incremental distributed state recovery may be safely used if you are sure
all updates ever executed in the cluster were done with GTIDs enabled, there
are no purged transactions and the new instance contains the same GTID set as
the cluster or a subset of it. To use this method by default, set the
'recoveryMethod' option to 'incremental'.
Incremental distributed state recovery was selected because it seems to be
safely usable.
^C
The cluster was successfully rebooted.
Script execution interrupted by user.
MySQL localhost:3311 ssl JS > var cluster = dba.getCluster('ytt_mgr');
MySQL localhost:3311 ssl JS > cluste.status();
ReferenceError: cluste is not defined
MySQL localhost:3311 ssl JS > cluster.status();
{
  "clusterName": "ytt_mgr",
  "defaultReplicaSet": {
    "name": "default",
    "primary": "centos-ytt57-2:3311",
    "ssl": "REQUIRED",
    "status": "OK",
    "statusText": "Cluster is ONLINE and can tolerate up to ONE failure.",
    "topology": {
      "centos-ytt57-2:3311": {
        "address": "centos-ytt57-2:3311",
        "mode": "R/W",
        "readReplicas": {},
        "role": "HA",
        "status": "ONLINE",
        "version": "8.0.17"
      },

```

```
"centos-ytt57-2:3312": {
  "address": "centos-ytt57-2:3312",
  "mode": "R/O",
  "readReplicas": {},
  "role": "HA",
  "status": "ONLINE",
  "version": "8.0.17"
},
"centos-ytt57-3:3311": {
  "address": "centos-ytt57-3:3311",
  "mode": "R/O",
  "readReplicas": {},
  "role": "HA",
  "status": "ONLINE",
  "version": "8.0.17"
},
"centos-ytt57-3:3312": {
  "address": "centos-ytt57-3:3312",
  "mode": "R/O",
  "readReplicas": {},
  "role": "HA",
  "status": "ONLINE",
  "version": "8.0.17"
}
},
"topologyMode": "Single-Primary"
},
"groupInformationSourceMember": "centos-ytt57-2:3311"
}
```



控制 mysqldump 导出的 SQL 文件的事务大小

作者：陈俊聪

1 背景

有人问 mysqldump 出来的 insert 语句，是否可以按每 10row 一条 insert 语句的形式组织。

2 思考 1：参数 --extended-insert

回忆过去所学, 我只知道有一对参数：

--extended-insert (默认值)

表示使用长 INSERT，多 row 在合并一起批量 INSERT，提高导入效率

--skip-extended-insert 一行一个的短 INSERT

均不满足群友需求，无法控制按每 10 row 一条 insert 语句的形式组织。

3 思考 2：“避免大事务”

之前一直没有考虑过这个问题。这个问题的提出，相信主要是为了“避免大事务”。所以满足 insert 均为小事务即可。

下面，我们来探讨一下以下问题：

1. 什么是大事务？
2. 那么 mysqldump 出来的 insert 语句可能是大事务吗？

3.1 什么是大事务？

1. 定义：运行时间比较长，操作的数据比较多的事务我们称之为大事务。
2. 大事务风险：
 - 锁定太多的数据，造成大量的阻塞和锁超时，回滚所需要的时间比较长。
 - 执行时间长，容易造成主从延迟。
 - undo log 膨胀
3. 避免大事务：我这里按公司实际场景，规定了，每次操作/获取数据量应该少于 5000 条，结果集应该小于 2M

3.2 mysqldump 出来的 SQL 文件有大事务吗？

前提，MySQL 默认是自提交的，所以如果没有明确地开启事务，一条 SQL 语句就是一条事务。在 mysqldump 里，就是一条 SQL 语句为一条事务。

按照我的“避免大事务”自定义规定，答案是没有的。原来，mysqldump 会按照参数 `--net-buffer-length`，来自动切分 SQL 语句。默认值是 1M。按照我们前面定义的标准，没有达到我们的 2M 的大事务标准。`--net-buffer-length` 最大可设置为 16777216，人手设置大于这个值，会自动调整为 16777216，即 16M。设置 16M，可以提升导出导入性能。如果为了避免大事务，那就不建议调整这个参数，使用默认值即可。

```
[root@192-168-199-198 ~]# mysqldump --net-buffer-length=104652800 -uroot -proot
-P3306 -h192.168.199.198 test t >16M.sql
mysqldump: [Warning] option 'net_buffer_length': unsigned value 104652800
adjusted to 16777216
#设置大于 16M，参数被自动调整为 16M
```

注意，指的是 mysqldump 的参数，而不是 mysqld 的参数。官方文档提到：If you increase this variable, ensure that the MySQL server `net_buffer_length` system variable has a value at least this large.

意思是 mysqldump 增大这个值，mysqld 也得增大这个值，测试结论是不需要的。怀疑官方文档有误。

不过，在导入的时候，受到服务器参数 `max_allowed_packet` 影响，它控制了服务器能接受的数据包的最大大小，默认值是 4194304，即 4M。所以导入数据库时需要调整参数 `max_allowed_packet` 的值。

```
set global max_allowed_packet=16*1024*1024*1024;
```

不调整的话，会出现以下报错：

```
[root@192-168-199-198 ~]# mysql -uroot -proot -P3306 -h192.168.199.198 test
<16M.sql
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 2006 (HY000) at line 46: MySQL server has gone away
```

相关测试最后，我放出我的相关测试步骤

```
mysql> select version();
+-----+
| version() |
+-----+
| 5.7.26-log |
+-----+
1 row in set (0.00 sec)
```

造 100 万行数据

```
create database test;
use test;
CREATE TABLE `t` (
  `a` int(11) DEFAULT NULL,
  `b` int(11) DEFAULT NULL,
  `c` varchar(255) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
insert into t values
(1,1,'abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz');
```

```
insert into t select * from t; #重复执行 20 次
```

```
# 直到出现 Records: 524288 Duplicates: 0 Warnings: 0
```

```
# 说明数据量达到 100 多万条了。
```

```
mysql> select count(*) from t;
```

```
+-----+
```

```
| count(*) |
```

```
+-----+
```

```
| 1048576 |
```

```
+-----+
```

```
1 row in set (1.04 sec)
```

数据大小如下，有 284MB

```
[root@192-168-199-198 test]# pwd
```

```
/data/mysql/mysql3306/data/test
```

```
[root@192-168-199-198 test]# du -sh t.ibd
```

```
284M    t.ibd
```

--net-buffer-length=1M

```
[root@192-168-199-198 ~]# mysqldump -uroot -proot -S /tmp/mysql3306.sock test
```

```
t >1M.sql
```

```
[root@192-168-199-198 ~]# du -sh 1M.sql
```

```
225M    1M.sql
```

```
[root@192-168-199-198 ~]# cat 1M.sql |grep -i insert |wc -l
```

```
226
```

默认 --net-buffer-length=1M 的情况下，225M 的 SQL 文件里有 226 条 insert，平均下来确实就是每条 insert 的 SQL 大小为 1M。

--net-buffer-length=16M

```
[root@192-168-199-198 ~]# mysqldump --net-buffer-length=16M -uroot -proot -S
/tmp/mysql3306.sock test t >16M.sql
[root@192-168-199-198 ~]# du -sh 16M.sql
225M    16M.sql
[root@192-168-199-198 ~]# cat 16M.sql |grep -i insert |wc -l
15
```

默认--net-buffer-length=16M 的情况下，225M 的 SQL 文件里有 15 条 insert，平均下来确实就是每条 insert 的 SQL 大小为 16M。

所以，这里证明了 --net-buffer-length 确实可用于拆分 mysqldump 备份文件的 SQL 大小的。

4 性能测试

insert 次数越多，交互次数就越多，性能越低。但鉴于上面例子的 insert 数量差距不大，只有 16 倍，性能差距不会很大(实际测试也是如此)。我们直接对比--net-buffer-length=16K 和--net-buffer-length=16M 的情况，他们 insert 次数相差了 1024 倍。

```
[root@192-168-199-198 ~]# time mysql -uroot -proot -S /tmp/mysql3306.sock test
<16K.sql
mysql: [Warning] Using a password on the command line interface can be insecure.
real    0m10.911s  #11 秒
user    0m1.273s
sys     0m0.677s
[root@192-168-199-198 ~]# mysql -uroot -proot -S /tmp/mysql3306.sock -e "reset
master";
mysql: [Warning] Using a password on the command line interface can be insecure.
[root@192-168-199-198 ~]# time mysql -uroot -proot -S /tmp/mysql3306.sock test
<16M.sql
mysql: [Warning] Using a password on the command line interface can be insecure.
real    0m8.083s  #8 秒
user    0m1.669s
sys     0m0.066s
```

结果明显。--net-buffer-length 设置越大，客户端与数据库交互次数越少，导入越快。

5 结论

mysqldump 默认设置下导出的备份文件，符合导入需求，不会造成大事务。性能方面也符合要求，不需要调整参数。

参考链接：https://dev.mysql.com/doc/refman/5.7/en/mysqldump.html#option_mysqldump_net-buffer-length



MySQL 删库不跑路

作者：洪斌

每个 DBA 是不是都有过删库的经历？删库了没有备份怎么办？备份恢复后无法启动服务什么情况？表定义损坏数据无法读取怎么办？

我曾遇到某初创互联网企业，因维护人员不规范的备份恢复操作，导致系统表空间文件被初始化，上万张表无法读取，花了数小时才抢救回来。当你发现数据无法读取时，也许并非数据丢失了，可能是 DBMS 找不到描述数据的信息。

1 背景

先来了解下几张关键的 InnoDB 数据字典表，它们保存了部分表定义信息，在我们恢复表结构时需要用到。

SYS_TABLES 描述 InnoDB 表信息

```
CREATE TABLE `SYS_TABLES` (  
  `NAME` varchar(255) NOT NULL DEFAULT '', 表名  
  `ID` bigint(20) unsigned NOT NULL DEFAULT '0', 表 id  
  `N_COLS` int(10) DEFAULT NULL,  
  `TYPE` int(10) unsigned DEFAULT NULL,  
  `MIX_ID` bigint(20) unsigned DEFAULT NULL,  
  `MIX_LEN` int(10) unsigned DEFAULT NULL,  
  `CLUSTER_NAME` varchar(255) DEFAULT NULL,  
  `SPACE` int(10) unsigned DEFAULT NULL, 表空间 id  
  PRIMARY KEY (`NAME`)  
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

SYS_INDEXES 描述 InnoDB 索引信息

```
CREATE TABLE `SYS_INDEXES` (  
  `TABLE_ID` bigint(20) unsigned NOT NULL DEFAULT '0', 与 sys_tables 的 id 对应  
  `ID` bigint(20) unsigned NOT NULL DEFAULT '0', 索引 id  
  `NAME` varchar(120) DEFAULT NULL, 索引名称  
  `N_FIELDS` int(10) unsigned DEFAULT NULL, 索引包含字段的个数  
  `TYPE` int(10) unsigned DEFAULT NULL,  
  `SPACE` int(10) unsigned DEFAULT NULL, 存储索引的表空间 id  
  `PAGE_NO` int(10) unsigned DEFAULT NULL, 索引的 root page id  
  PRIMARY KEY (`TABLE_ID`, `ID`)  
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

SYS_COLUMNS 描述 InnoDB 表的字段信息

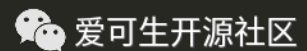
```
CREATE TABLE `SYS_COLUMNS` (
  `TABLE_ID` bigint(20) unsigned NOT NULL, 与 sys_tables 的 id 对应
  `POS` int(10) unsigned NOT NULL,      字段相对位置
  `NAME` varchar(255) DEFAULT NULL,      字段名称
  `MTYPE` int(10) unsigned DEFAULT NULL,  字段编码
  `PRTYPE` int(10) unsigned DEFAULT NULL,  字段校验类型
  `LEN` int(10) unsigned DEFAULT NULL,    字段字节长度
  `PREC` int(10) unsigned DEFAULT NULL,   字段精度
  PRIMARY KEY (`TABLE_ID`, `POS`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

SYS_FIELDS 描述全部索引的字段列

```
CREATE TABLE `SYS_FIELDS` (
  `INDEX_ID` bigint(20) unsigned NOT NULL,
  `POS` int(10) unsigned NOT NULL,
  `COL_NAME` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`INDEX_ID`, `POS`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

./storage/innobase/include/dict0boot.h 文件定义了每个字典表的 index id，对应 id 的 page 中存储着字典表的数据。

```
/* The ids for the basic system tables and their indexes */
#define DICT_TABLES_ID      1
#define DICT_COLUMNS_ID    2
#define DICT_INDEXES_ID    3
#define DICT_FIELDS_ID     4
```



这里我们需要借助 undrop-for-innodb 工具恢复数据，它能读取表空间信息得到 page，将数据从 page 中提取出来。

```
# wget https://github.com/chhabhaiya/undrop-for-innodb/archive/master.zip
# yum install -y gcc flex bison
# make
# make sys_parser
```

./sys_parser 读取表结构信息

sys_parser [-h] [-u] [-p] [-d] databases/table

stream_parser 读取 InnoDB page 从 ibdata1 或 ibd 或分区表

```
# ./stream_parser
You must specify file with -f option
Usage: ./stream_parser -f <innodb_datafile> [-T N:M] [-s size] [-t size] [-V|-g]
Where:
    -h          - Print this help
    -V or -g    - Print debug information
    -s size     - Amount of memory used for disk cache (allowed examples 1G 10M).
Default 100M
    -T          - retrieves only pages with index id = NM (N - high word, M - low
word of id)
    -t size     - Size of InnoDB tablespace to scan. Use it only if the parser
can't determine it by himself.
```

c_parser 从 innodb page 中读取记录保存到文件

```
# ./c_parser
Error: Usage: ./c_parser -4|-5|-6 [-dDV] -f <InnoDB page or dir> -t table.sql [-
T N:M] [-b <external pages directory>]
Where
    -f <InnoDB page(s)> -- InnoDB page or directory with pages(all pages should
have same index_id)
    -t <table.sql> -- CREATE statement of a table
    -o <file> -- Save dump in this file. Otherwise print to stdout
    -l <file> -- Save SQL statements in this file. Otherwise print to stderr
    -h -- Print this help
    -d -- Process only those pages which potentially could have deleted records
(default = NO)
    -D -- Recover deleted rows only (default = NO)
    -U -- Recover UNdeleted rows only (default = YES)
    -V -- Verbose mode (lots of debug information)
    -4 -- innodb_datafile is in REDUNDANT format
    -5 -- innodb_datafile is in COMPACT format
    -6 -- innodb_datafile is in MySQL 5.6 format
    -T -- retrieves only pages with index id = NM (N - high word, M - low word
of id)
    -b <dir> -- Directory where external pages can be found. Usually it is
pages-XXX/FIL_PAGE_TYPE_BLOB/
    -i <file> -- Read external pages at their offsets from <file>.
    -p prefix -- Use prefix for a directory name in LOAD DATA INFILE command
```

接下来，我们演示场景的几种数据恢复场景。

场景 1：drop table

是否启用了 `innodb_file_per_table` 其恢复方法有所差异，当发生误删表时，应尽快停止 MySQL 服务，不要启动。若 `innodb_file_per_table=ON`，最好只读方式重新挂载文件系统，防止其他进程写入数据覆盖之前块设备的数据。

如果评估记录是否被覆盖，可以表中某些记录的作为关键字看是否能从 ibdata1 中筛选出。

```
# grep WOODYHOFFMAN ibdata1
```

Binary file ibdata1 matches

也可以使用 bvi（适用于较小文件）或 hexdump -C（适用于较大文件）工具

以表 sakila.actor 为例

```
CREATE TABLE `actor` (
  `actor_id` smallint(5) unsigned NOT NULL AUTO_INCREMENT,
  `first_name` varchar(45) NOT NULL,
  `last_name` varchar(45) NOT NULL,
  `last_update` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
  CURRENT_TIMESTAMP,
  PRIMARY KEY (`actor_id`),
  KEY `idx_actor_last_name` (`last_name`)
) ENGINE=InnoDB AUTO_INCREMENT=201 DEFAULT CHARSET=utf8
```

首先恢复表结构信息

1. 解析系统表空间获取 page 信息

```
./stream_parser -f /var/lib/mysql/ibdata1
```

2. 新建一个 schema，把系统字典表的 DDL 导入

```
cat dictionary/SYS_* | mysql recovered
```

3. 创建恢复目录

```
mkdir -p dumps/default
```

4. 解析系统表空间包含的字典表信息

```
./c_parser -4f pages-ibdata1/FIL_PAGE_INDEX/0000000000000001.page -t
dictionary/SYS_TABLES.sql > dumps/default/SYS_TABLES 2>
dumps/default/SYS_TABLES.sql
./c_parser -4f pages-ibdata1/FIL_PAGE_INDEX/0000000000000002.page -t
dictionary/SYS_COLUMNS.sql > dumps/default/SYS_COLUMNS 2>
dumps/default/SYS_COLUMNS.sql
./c_parser -4f pages-ibdata1/FIL_PAGE_INDEX/0000000000000003.page -t
dictionary/SYS_INDEXES.sql > dumps/default/SYS_INDEXES 2>
dumps/default/SYS_INDEXES.sql
./c_parser -4f pages-ibdata1/FIL_PAGE_INDEX/0000000000000004.page -t
dictionary/SYS_FIELDS.sql > dumps/default/SYS_FIELDS 2>
dumps/default/SYS_FIELDS.sql
```

5. 导入恢复的数据字典

```
cat dumps/default/*.sql | mysql recovered
```

6. 读取恢复后的表结构信息

```
./sys_parser -pmsandbox -d recovered sakila/actor
```

由于 5.x 版本 innodb 引擎并非完整记录表结构信息，会丢失 AUTO_INCREMENT 属性、二级索引和外键约束，DECIMAL 精度等信息。

若是 mysql 5.5 版本 frm 文件被从系统删除，在原目录下 touch 与原表名相同的 frm 文件，还能读取表结构信息和数据。若只有 frm 文件，想要获得表结构信息，可使用 `mysqlfrm --diagnostic /path/to/xxx.frm`，连接 mysql 会显示字符集信息。

■ `innodb_file_per_table=OFF`

因为是共享表空间模式，数据页都存储在 ibdata1，可以从 ibdata1 文件中提取数据。

1. 获取表的 table id，sys_table 存有表的 table id，sys_table 表 index id 是 1，所以从 0000000000000001.page 获取表 id

```
./c_parser -4Df pages-ibdata1/FIL_PAGE_INDEX/0000000000000001.page -t
dictionary/SYS_TABLES.sql | grep sakila/actor
000000000B28 2A000001430D4D SYS_TABLES "sakila/actor" 158 4 1 0 0 "" 0
000000000B28 2A000001430D4D SYS_TABLES "sakila/actor" 158 4 1 0 0 "" 0
```

2. 利用 table id 获取表的主键 id，sys_indexes 存有表索引信息，innodb 索引组织表，找到主键 id 即找到数据，sys_indexes 的 index id 是 3，所以从 0000000000000003.page 获取主键 id

```
./c_parser -4Df pages-ibdata1/FIL_PAGE_INDEX/0000000000000003.page -t
dictionary/SYS_INDEXES.sql | grep 158
000000000B28 2A000001430BCA SYS_INDEXES 158 376 "PRIMARY" 1
3 0 4294967295
000000000B28 2A000001430C3C SYS_INDEXES 158 377
"idx\_actor\_last\_name" 1 0 0 4294967295
000000000B28 2A000001430BCA SYS_INDEXES 158 376 "PRIMARY" 1
3 0 4294967295
000000000B28 2A000001430C3C SYS_INDEXES 158 377
"idx\_actor\_last\_name" 1 0 0 4294967295
```

3. 知道了主键 id，就可以从对应 page 中提取表数据，并生成 sql 文件。

```
./c_parser -4f pages-ibdata1/FIL_PAGE_INDEX/00000000000000376.page -t
sakila/actor.sql > dumps/default/actor 2> dumps/default/actor_load.sql
```


4. 最后导入恢复的数据

```
cat dumps/default/*.sql | mysql sakila
```

■ innodb_file_per_table=ON

这种情况恢复步骤与上述基本一致，但由于是独立表空间模式，数据页存储在各自的 ibd 文件，ibd 文件删除了，无法通过 ibdata1 提取数据页，所以 pages-ibdata1 目录找不到数据页，streamparser 要从块设备中读取数据页信息。扫描完成后，在 pages-sda1 目录下提取数据。

```
./stream_parser -f /dev/sda1 -t 1000000k
```

场景 2: Corrupted InnoDB table

在 InnoDB 表发生损坏，即使 innodb_force_recovery=6 也无法启动 MySQL 日志中可能会出现类似报错

```
InnoDB: Database page corruption on disk or a failed
InnoDB: file read of page 4.
```

此时的恢复策略需要将数据页从独立表空间中提取出，再删除表空间，重新创建表导入数据。

1. 先获得故障表的主键 index id
2. 通过 index id page 获取到数据记录

```
select t.name, t.table_id, i.index_id, i.page_no from INNODB_SYS_TABLES t join
INNODB_SYS_INDEXES i on t.table_id=i.table_id and t.name='test/sbtest1';
```

3. 由于数据页可能有部分记录损坏，需要过滤掉“坏”的数据，保留好的数据 例如：前两行记录实际是“坏”数据，需要过滤掉。

```
root@test:~/recovery/undrop-for-innodb# ./c_parser -6f pages-
actor.ibd/FIL_PAGE_INDEX/0000000000000015.page -t sakila/actor.sql >
dumps/default/actor 2> dumps/default/actor_load.sql
root@test:~/recovery/undrop-for-innodb# cat dumps/default/actor
-- Page id: 3, Format: COMPACT, Records list: Invalid, Expected records: (0 200)
72656D756D07 08000010002900 actor 30064 "\0\0\0\0" "" "1972-09-20
23:07:44"
1050454E454C 4F50454755494E actor 19713 "ESSC" "" "2100-08-09
07:52:36"
000000000051E 9F0000014D011A actor 2 "NICK" "WAHLBERG" "2006-02-15
04:34:33"
000000000051E 9F0000014D0124 actor 3 "ED" "CHASE" "2006-02-15
04:34:33"
000000000051E 9F0000014D012E actor 4 "JENNIFER" "DAVIS" "2006-02-15
```

```
04:34:33"
000000000051E    9F0000014D0138 actor    5      "JOHNNY"      "LOLLOBRIGIDA"
"2006-02-15 04:34:33"
000000000051E    9F000001414141 actor    6      "AAAAA" "AAAAAAAAAA"      "2004-09-10
01:53:05"
000000000051E    9F0000014D016A actor   10      "CHRISTIAN"    "GABLE" "2006-02-15
04:34:33"
...
```

可以在 sql 文件中加上筛选条件，比如：通过 actor_id 做范围筛选，再用新的 sql 文件读数据页。

```
CREATE TABLE `actor` (
  `actor_id` smallint(5) unsigned NOT NULL AUTO_INCREMENT
    /*!FILTER
      int_min_val: 1
      int_max_val: 300 */
  `first_name` varchar(45) NOT NULL,
  `last_name` varchar(45) NOT NULL,
  `last_update` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  PRIMARY KEY (`actor_id`),
  KEY `idx_actor_last_name` (`last_name`)
) ENGINE=InnoDB AUTO_INCREMENT=201 DEFAULT CHARSET=utf8;
```

4. 删除故障表文件，innodb_force_recovery=6 启动 MySQL，启动后删除元数据
5. 创建新表导入恢复好的数据疑问：如何知道丢失了多少记录？读取数据页时开头会显示期望的记录数，最后会显示实际恢复的记录数，差值便是丢失记录数

```
-- Page id: 3, Format: COMPACT, Records list: Invalid, Expected records: (0 200)
-- Page id: 3, Found records: 197, Lost records: YES, Leaf page: YES
```

场景 3：磁盘或文件系统损坏如何恢复数据

这种情况下尽快保护损坏的块设备不要再写入，并用 dd 工具读取镜像数据用作恢复

本地方式

```
dd if=/dev/sdb of=/path/to/faulty_disk.img conv=noerror
```

远程方式

```
remote server> nc -l 1234 > faulty_disk.img
local server> dd if=/dev/sdb of=/dev/stdout conv=noerror | nc a.b.c.d 1234
```

保存好磁盘镜像后，后续恢复操作参考场景 2。

总结

1. 千万不要在服务运行时把 copy 数据文件作为备份方式，看似备份了数据，但实际数据是不一致的。
2. 正确的使用物理备份工具 xtrabackup/meb 或逻辑备份方式。
3. 对备份数据要定期进行恢复验证测试。

希望你永远不会用到这些方法，做好备份，勤验证！

参考链接：

<https://twindb.com/how-to-recover-innodb-dictionary/> <https://twindb.com/recover-corrupt-mysql-database/> <https://twindb.com/take-image-from-corrupted-hard-drive/>
<https://twindb.com/data-loss-after-mysql-restart/>
<https://twindb.com/repair-corrupted-innodb-table-with-corruption-in-secondary-index/>
<https://twindb.com/resolving-page-corruption-in-compressed-innodb-tables/>
<https://dev.mysql.com/doc/refman/5.7/en/innodb-troubleshooting-datadict.html>



基于 WRITESET 的并行复制方式

作者：高鹏（八怪）

背景

基于 COMMIT_ORDER 的并行复制只有在有压力的情况下才可能会形成一组，压力不大的情况下在从库的并行度并不会高。但是基于 WRITESET 的并行复制目标就是在 ORDER_COMMIT 的基础上再尽可能的降低 last commit，这样在从库获得更好的并行度（即便在主库串行执行的事务在从库也能并行应用）。它使用的方式就是通过扫描 Writerset 中的每一个元素（行数据的 hash 值）在一个叫做 Writerset 的历史 MAP（行数据的 hash 值和 seq number 的一个 MAP）中进行比对，寻找是否有冲突的行，然后做相应的处理，后面我们会详细描述这种行为。如果要使用这种方式我们需要在主库设置如下两个参数：

1. transaction_write_set_extraction=XXHASH64
2. binlog_transaction_dependency_tracking=WRITESET

它们是在 5.7.22 才引入的。

1 奇怪的 last commit

我们先来看一个截图，仔细观察其中的 last commit：

GTID	last_committed	sequence_number
GTID	last_committed=830	sequence_number=897
GTID	last_committed=160	sequence_number=898
GTID	last_committed=719	sequence_number=899
GTID	last_committed=511	sequence_number=900
GTID	last_committed=160	sequence_number=901
GTID	last_committed=160	sequence_number=902
GTID	last_committed=829	sequence_number=903
GTID	last_committed=160	sequence_number=904
GTID	last_committed=715	sequence_number=905
GTID	last_committed=638	sequence_number=906
GTID	last_committed=160	sequence_number=907
GTID	last_committed=836	sequence_number=908
GTID	last_committed=255	sequence_number=909
GTID	last_committed=498	sequence_number=910
GTID	last_committed=836	sequence_number=911
GTID	last_committed=903	sequence_number=912
GTID	last_committed=772	sequence_number=913
GTID	last_committed=383	sequence_number=914
GTID	last_committed=678	sequence_number=915
GTID	last_committed=693	sequence_number=916
GTID	last_committed=726	sequence_number=917
GTID	last_committed=819	sequence_number=918

我们可以看到其中的 last commit 看起来是乱序的，这种情况在基于 COMMIT_ORDER 的并行复制方式下是不可能出现的。实际上它就是我们前面说的基于 WRITESET 的并行复制再尽可能降低的 last commit 的结果。这种情况会在 MTS 从库获得更好的并行回放效果，第 19 节将会详细解释并行判定的标准。

2 Writese 是什么

实际上 Writese 是一个集合，使用的是 C++ STL 中的 set 容器，在类 Rpl_transaction_write_set_ctx 中包含了如下定义：

```
std::set<uint64> write_set_unique;
```

集合中的每一个元素都是 hash 值，这个 hash 值和我们的 transaction_write_set_extraction 参数指定的算法有关，其来源就是行数据的主键和唯一键。每行数据包含了两种格式：

1. 字段值为二进制格式
2. 字段值为字符串格式

每行数据的具体格式为：

主键/ 唯一键名称	分隔符	库名	分隔符	库名 长度	表名	分隔符	表名 长度	键字段 1	分隔符	长度	键字段 2	分隔符	长度	其他字段 ...
--------------	-----	----	-----	----------	----	-----	----------	----------	-----	----	----------	-----	----	-------------

在 InnoDB 层修改一行数据之后会将这上面的格式的数据进行 hash 后写入到 Writese 中。可以参考函数 add_pke，后面我也会以伪代码的方式给出部分流程。

但是需要注意一个事务的所有的行数据的 hash 值都要写入到一个 Writese。如果修改的行比较多那么可能需要更多内存来存储这些 hash 值。虽然 8 字节比较小，但是如果一个事务修改的行很多，那么还是需要消耗较多的内存资源的。

为了更直观的观察这种数据格式，可以使用 debug 的方式获取。下面我们来看一下。

3 Writese 的生成

我们使用如下表：

```
mysql> use test
Database changed
mysql> show create table jj10 \G
***** 1. row *****
      Table: jj10
Create Table: CREATE TABLE `jj10` (
  `id1` int(11) DEFAULT NULL,
  `id2` int(11) DEFAULT NULL,
  `id3` int(11) NOT NULL,
  PRIMARY KEY (`id3`),
  UNIQUE KEY `id1` (`id1`),
  KEY `id2` (`id2`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1
1 row in set (0.00 sec)
```

我们写入一行数据：

```
insert into jj10 values(36,36,36);
```

这一行数据一共会生成 4 个元素分别为：

注意：这里显示的?是分隔符

1. 主键二进制格式

```
$1 = "PRIMARY?test?4jj10?4\200\000\000$?4"
```

**注意：\200\000\000\$：为 3 个八进制字节和 ASCII 字符 \$，
其转换为 16 进制就是"0x80 00 00 24 "***

分解为：

主键名称	分隔符	库名	分隔符	库名长度	表名	分隔符	表名长度	主键字段1	分隔符	长度
PRIMARY	?	test	?	4	jj10	?	4	0x80 00 00 24	?	4

2. 主键字符串格式：

```
$2 = "PRIMARY?test?4jj10?436?2"
```

分解为：

主键名称	分隔符	库名	分隔符	库名长度	表名	分隔符	表名长度	主键字段1	分隔符	长度
PRIMARY	?	test	?	4	jj10	?	4	36	?	2

3. 唯一键二进制格式

```
$3 = "id1?test?4jj10?4\200\000\000$?4"
```

解析同上

4. 唯一键字符串格式：

```
$4 = "id1?test?4jj10?436?2"
```

解析同上

最终这些数据会通过 hash 算法后写入到 Writerset 中。

4 函数 add_pke 的大概流程

下面是一段伪代码，用来描述这种生成过程：

如果表中存在索引：

 将数据库名，表名信息写入临时变量

 循环扫描表中每个索引：

 如果不是唯一索引：

退出本次循环继续循环。

循环两种生成数据的方式(二进制格式和字符串格式):

将索引名字写入到 pke 中。

将临时变量信息写入到 pke 中。

循环扫描索引中的每一个字段:

将每一个字段的信息写入到 pke 中。

如果字段扫描完成:

将 pke 生成 hash 值并且写入到写集合中。

如果没有找到主键或者唯一键记录一个标记, 后面通过这个标记来判定是否使用 Writese 的并行复制方式

5 Writese 设置对 last commit 的处理方式

前一节我们讨论了基于 ORDER_COMMIT 的并行复制是如何生成 last_commit 和 seq number 的。实际上基于 WRITASET 的并行复制方式只是在 ORDER_COMMIT 的基础上对 last_commit 做更进一步处理, 并不影响原有的 ORDER_COMMIT 逻辑, 因此如果要回退到 ORDER_COMMIT 逻辑非常方便。可以参考 MYSQL_BIN_LOG::write_gtid 函数。

根据 binlog_transaction_dependency_tracking 取值的不同会做进一步的处理, 如下:

1. ORDER_COMMIT: 调用 m_commit_order.get_dependency 函数。这是前面我们讨论的方式。
2. WRITASET: 调用 m_commit_order.get_dependency 函数, 然后调用 m_writese.get_dependency。可以看到 m_writese.get_dependency 函数会对原有的 last commit 做处理。
3. WRITASET_SESSION: 调用 m_commit_order.get_dependency 函数, 然后调用 m_writese.get_dependency 再调用 m_writese_session.get_dependency。m_writese_session.get_dependency 会对 last commit 再次做处理。

这段描述的代码对应:

```
case DEPENDENCY_TRACKING_COMMIT_ORDER:
    m_commit_order.get_dependency(thd, sequence_number, commit_parent);
    break;
case DEPENDENCY_TRACKING_WRITASET:
    m_commit_order.get_dependency(thd, sequence_number, commit_parent);
    m_writese.get_dependency(thd, sequence_number, commit_parent);
    break;
case DEPENDENCY_TRACKING_WRITASET_SESSION:
    m_commit_order.get_dependency(thd, sequence_number, commit_parent);
    m_writese.get_dependency(thd, sequence_number, commit_parent);
    m_writese_session.get_dependency(thd, sequence_number, commit_parent);
    break;
```

6 Writeseet 的历史 MAP

我们到这里已经讨论了 Writeseet 是什么，也已经说过如果要降低 last commit 的值我们需要通过对事务的 Writeseet 和 Writeseet 的历史 MAP 进行比对，看是否冲突才能决定降低为什么值。那么必须在内存中保存一份这样的一个历史 MAP 才行。在源码中使用如下方式定义：

```
/*
Track the last transaction sequence number that changed each row
in the database, using row hashes from the writeseet as the index.
*/
typedef std::map<uint64,int64> Writeseet_history; //map 实现
Writeseet_history m_writeseet_history;
```

我们可以看到这是 C++ STL 中的 map 容器，它包含两个元素：

1. Writeseet 的 hash 值
2. 最新一次本行数据修改事务的 seq number

它是按照 Writeseet 的 hash 值进行排序的。

其次内存中还维护一个叫做 m_writeseet_history_start 的值，用于记录 Writeseet 的历史 MAP 中最早事务的 seq number。如果 Writeseet 的历史 MAP 满了就会清理这个历史 MAP 然后将本事务的 seq number 写入 m_writeseet_history_start，作为最早的 seq number。后面会看到对于事务 last commit 的值的修改总是从这个值开始然后进行比较判断修改的，如果在 Writeseet 的历史 MAP 中没有找到冲突那么直接设置 last commit 为这个 m_writeseet_history_start 值即可。

下面是清理 Writeseet 历史 MAP 的代码：

```
if (exceeds_capacity || !can_use_writesets)
//Writeseet 的历史 MAP 已满
{
    m_writeseet_history_start= sequence_number;
//如果超过最大设置，清空 writeseet history。从当前 seq number 重新记录， 也就是最小的那个事务 seq number
    m_writeseet_history.clear();
//清空历史 MAP
}
```

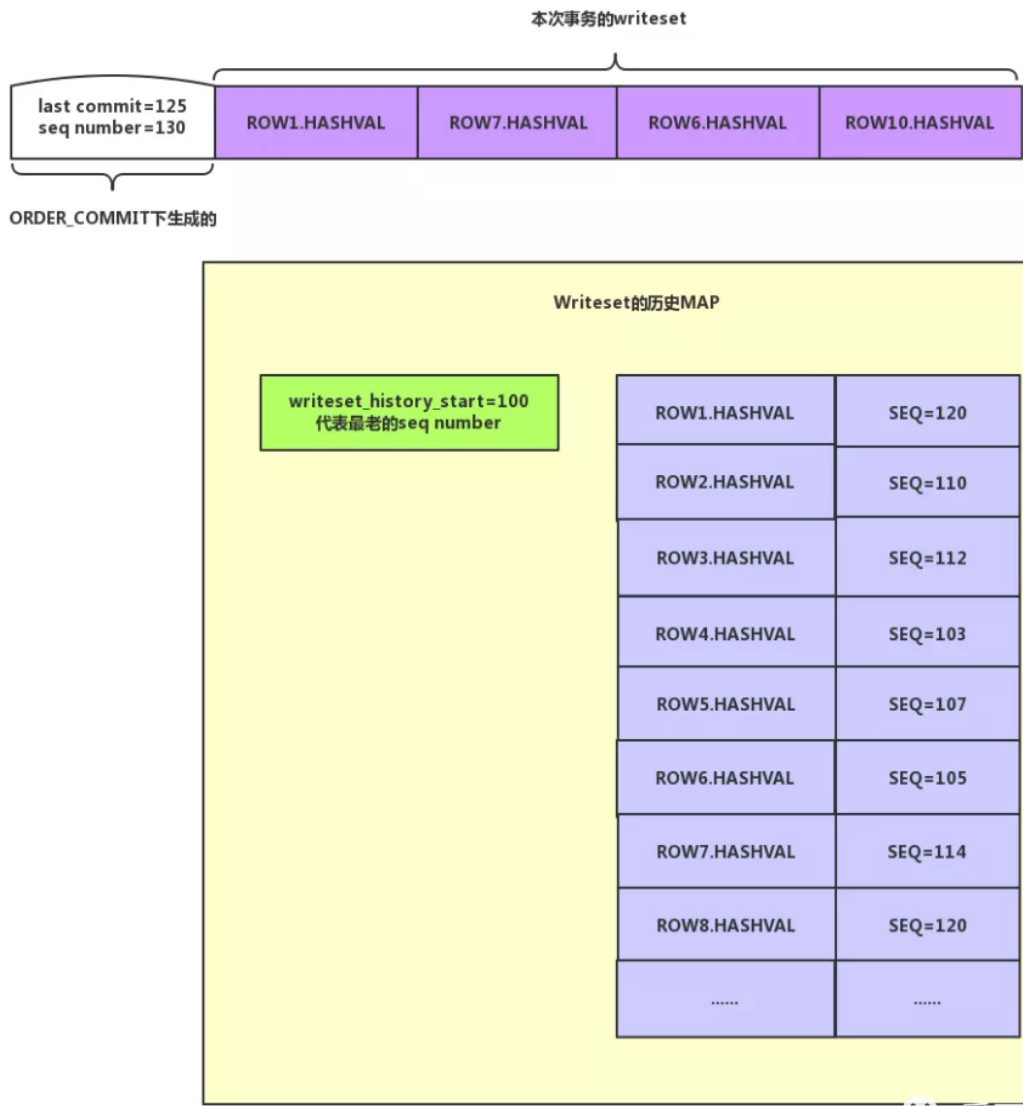
7 Writeseet 的并行复制对 last commit 的处理流程

这里介绍一下整个处理的过程，假设如下：

1. 当前通过基于 ORDER_COMMIT 的并行复制方式后，构造出来的是 (last commit=125, seq number=130)。
2. 本事务修改了 4 条数据，我分别使用 ROW1/ROW7/ROW6/ROW10 代表。
3. 表只包含主键没有唯一键，并且我的图中只保留行数据的二进制格式的 hash 值，而没有包含数据的

字符串格式的 hash 值。

初始化情况如下图：



爱可生开源社区

第一步 设置 last commit 为 writeset_history_start 的值也就是 100。

第二步 ROW1.HASHVAL 在 Writeset 历史 MAP 中查找，找到冲突的行 ROW1.HASHVAL 将历史 MAP 中这行数据的 seq number 更改为 130。同时设置 last commit 为 120。

第三步 ROW7.HASHVAL 在 Writeset 历史 MAP 中查找，找到冲突的行 ROW7.HASHVAL 将 Writeset 历史 MAP 中这行数据的 seq number 更改为 130。由于历史 MAP 中对应的 seq number 为 114，小于 120 不做更改。last commit 依旧为 120。

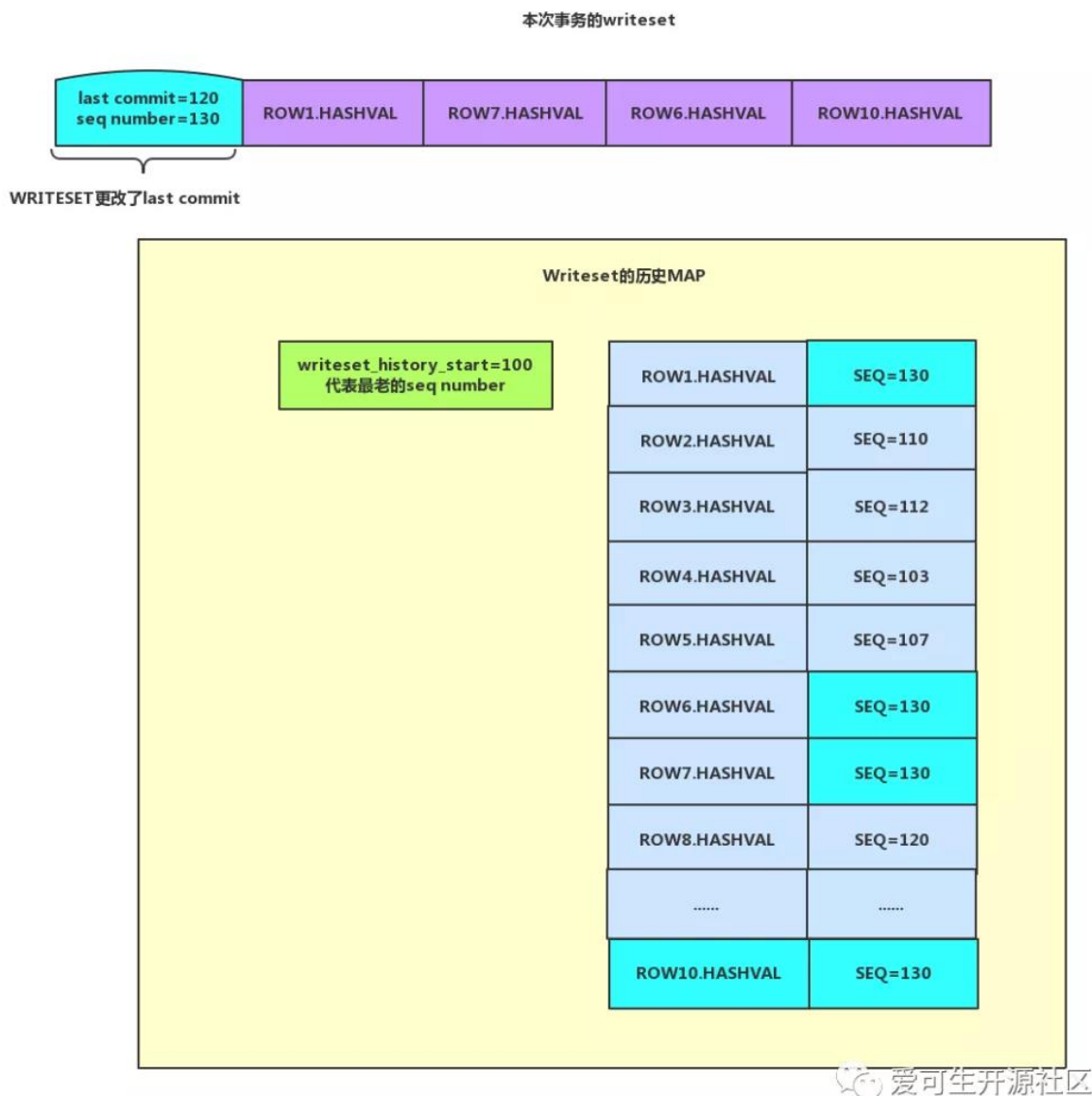
第四步 ROW6.HASHVAL 在 Writeset 历史 MAP 中查找，找到冲突的行 ROW6.HASHVAL 将 Writeset 历史 MAP 中这行数据的 seq number 更改为 130。由于历史 MAP 中对应的 seq number 为 105，小于 120 不做更改。last commit 依旧为 120。

第五步 ROW10.HASHVAL 在 Writeset 历史 MAP 中查找，没有找到冲突的行，因此需要将这一行插入到 Writeset 历史 MAP 中查找（需要判断是否导致历史 MAP 占满，如果占满则不需要插入，后面随即将清理掉）。即要将 ROW10.HASHVAL 和 seq number=130 插入到 Writeset 历史 MAP 中。

整个过程结束。last commit 由以前的 125 降低为 120，目的达到了。实际上我们可以看出 Writeset 历史 MAP 就相当于保存了一段时间以来修改行的快照，如果保证本次事务修改的数据在这段时间内没有冲

突，那么显然是可以在从库并行执行的。

last commit 降低后如下图：



整个逻辑就在函数 `Writeset_trx_dependency_tracker::get_dependency` 中，下面是一些关键代码，代码稍多：

```
if (can_use_writesets) //如果能够使用 writese 方式
{
    /*
    Check if adding this transaction exceeds the capacity of the writese
    history. If that happens, m_writeset_history will be cleared only after 而
    add_pke
    using its information for current transaction.
    */
    exceeds_capacity=
        m_writeset_history.size() + writese->size() > m_opt_max_history_size;
```

```

//如果大于参数 binlog_transaction_dependency_history_size 设置清理标记
/*
    Compute the greatest sequence_number among all conflicts and add the
    transaction's row hashes to the history.
*/
int64 last_parent= m_writeset_history_start;
//临时变量，首先设置为最小的一个 seq number
for (std::set<uint64>::iterator it= writeset->begin(); it != writeset->end();
++it)
//循环每一个 Writeset 中的每一个元素
{
    Writeset_history::iterator hst= m_writeset_history.find(*it);
//是否在 writeset history 中 已经存在了。map 中的元素是 key 是 writeset 值是 sequence
number
    if (hst != m_writeset_history.end()) //如果存在
    {
        if (hst->second > last_parent && hst->second < sequence_number)
            last_parent= hst->second;
//如果已经大于了不需要设置
        hst->second= sequence_number;
//更改这行记录的 sequence_number
    }
    else
    {
        if (!exceeds_capacity)
            m_writeset_history.insert(std::pair<uint64, int64>(*it,
sequence_number));
//没有冲突则插入。
    }
}
.....
if (!write_set_ctx->get_has_missing_keys())
//如果没有主键和唯一键那么不更改 last commit
{
    /*
        The WRITESET commit_parent then becomes the minimum of largest parent
        found using the hashes of the row touched by the transaction and the
        commit parent calculated with COMMIT_ORDER.
    */;
    commit_parent= std::min(last_parent, commit_parent);
//这里对 last commit 做更改了。降低他的 last commit
}
}
}
}
if (exceeds_capacity || !can_use_writesets)

```

```
{
    m_writeset_history_start= sequence_number;
//如果超过最大设置 清空 writeset history。从当前 sequence 重新记录 也就是最小的那个事务
sequence number
    m_writeset_history.clear();//清空真个 MAP
}
```

8 WRITESSET_SESSION 的方式

前面说过这种方式就是在 WRITESSET 的基础上继续处理，实际上它的含义就是同一个 session 的事务不允许在从库并行回放。代码很简单，如下：

```
int64 session_parent= thd->rpl_thd_ctx.dependency_tracker_ctx().
    get_last_session_sequence_number();
//取本 session 的上一次事务的 seq number
if (session_parent != 0 && session_parent < sequence_number)
//如果本 session 已经做过事务并且本次当前的 seq number 大于上一次的 seq number
    commit_parent= std::max(commit_parent, session_parent);
//说明这个 session 做过多次事务不允许并发，修改为 order_commit 生成的 last commit
thd->rpl_thd_ctx.dependency_tracker_ctx().
    set_last_session_sequence_number(sequence_number);
//设置 session_parent 的值为本次 seq number 的值
```

经过这个操作后，我们发现这种情况最后 last commit 恢复成 ORDER_COMMIT 的方式。

9 关于 binlog_transaction_dependency_history_size 参数说明

本参数默认值为 25000。代表的是我们说的 Writeset 历史 MAP 中元素的个数。如前面分析的 Writeset 生成过程中修改一行数据可能会生成多个 HASH 值，因此这个值还不能完全等待于修改的行数，可以理解为如下：

■ $\text{binlog_transaction_dependency_history_size}/2 = \text{修改的行数} * (1 + \text{唯一键个数})$

我们通过前面的分析可以发现如果这个值越大那么在 Writeset 历史 MAP 中能容下的元素也就越多，生成的 last commit 就可能更加精确（更加小），从库并发的效率也就可能越高。但是我们需要注意设置越大相应的内存需求也就越高了。

10 没有主键的情况

实际上在函数 add_pke 中就会判断是否有主键或者唯一键，如果存在唯一键也是可以。Writeset 中存储了唯一键的行数据 hash 值。参考函数 add_pke，下面是判断：

```
if (!(table->key_info[key_number].flags & (HA_NOSAME )) == HA_NOSAME))
//跳过非唯一的 KEY
    continue;
```

如果没有主键或者唯一键那么下面语句将被触发：

```
if (writeset_hashes_added == 0)
    ws_ctx->set_has_missing_keys();
```

然后我们在生成 last commit 会判断这个设置如下：

```
if (!write_set_ctx->get_has_missing_keys())
//如果没有主键和唯一键那么不更改 last commit
{
    /*
    The WRITESET commit_parent then becomes the minimum of largest parent
    found using the hashes of the row touched by the transaction and the
    commit parent calculated with COMMIT_ORDER.
    */;
    commit_parent= std::min(last_parent, commit_parent);//这里对 last commit 做更
改了。降低他的 last commit
}
}
```

因此没有主键可以使用唯一键，如果都没有的话 WRITESET 设置就不会生效回退到老的 ORDER_COMMIT 方式。

11 为什么同一个 session 执行的事务也能生成同样的 last commit

有了前面的基础，我们就很容易解释这种现象了。其主要原因就是 Writeset 的历史 MAP 的存在，只要这些事务修改的行没有冲突，也就是主键/唯一键不相同，那么在基于 WRITESET 的并行复制方式中就可以存在这种现象，但是如果 binlog_transaction_dependency_tracking 设置为 WRITESET_SESSION 则不会出现这种现象。

写在最后

好了到这里我们明白了基于 WRITESET 的并行复制方式的优点，但是它也有明显的缺点如下：

1. Writeset 中每个 hash 值都需要和 Writeset 的历史 MAP 进行比较。
2. Writeset 需要额外的内存空间。
3. Writeset 的历史 MAP 需要额外的内存空间。

如果从库没有延迟，则不需要考虑这种方式，即便有延迟我们也应该先考虑其他方案。



gh-ost 原理剖析

作者：杨奇龙

1 简介

上一篇文章（[gh-ost 在线 ddl 变更工具](#)，请扫码查看原文链接）介绍 gh-ost 参数和具体的使用方法、核心特性（可动态调整暂停）、动态修改参数等等。本文分几部分从源码方面解释 gh-ost 的执行过程、数据迁移、切换细节设计。

2 原理

2.1 执行过程

本例基于在主库上执行 DDL 记录的核心过程。核心代码在 [github.com/github/gh-ost/go/logic/migrator.go](https://github.com/github/gh-ost/blob/master/go/logic/migrator.go) 的 Migrate()

```
func (this *Migrator) Migrate() //Migrate executes the complete migration logic.
This is the major gh-ost function.
```

1. 检查数据库实例的基础信息

- a 测试 db 是否可连通,
- b 权限验证

```
show grants for current_user()
```

- c 获取 binlog 相关信息,包括 row 格式和修改 binlog 格式后的重启 replicate

```
select @@global.log_bin, @@global.binlog_format
select @@global.binlog_row_image
```

- d 原表存储引擎是否是 innodb,检查表相关的外键,是否有触发器,行数预估等操作,需要注意的是行数预估有两种方式 一个是通过 explain 读执行计划 另外一个 select count(*) from table ,遇到几百 G 的大表,后者一定非常慢。

```
explain select /* gh-ost */ * from `test`.`b` where 1=1
```

2. 模拟 slave, 获取当前的位点信息, 创建 binlog streamer 监听 binlog

```
2019-09-08T22:01:20.944172+08:00 17760 Query show /* gh-ost readCurrentBinlogC
ordinates */ master status
2019-09-08T22:01:20.947238+08:00 17762 Connect root@127.0.0.1 on using TCP/I
P
2019-09-08T22:01:20.947349+08:00 17762 Query SHOW GLOBAL VARIABLES LIKE 'BINLO
G_CHECKSUM'
2019-09-08T22:01:20.947909+08:00 17762 Query SET @master_binlog_checksum='NONE
```

```
,
2019-09-08T22:01:20.948065+08:00    17762 Binlog Dump    Log: 'mysql-bin.000005'  P
os: 795282
```

3. 创建 日志记录表 `xx_ghc` 和影子表 `xx_gho` 并且执行 `alter` 语句将影子表变更为目标表结构。如下日志记录了该过程, `gh-ost` 会将核心步骤记录到 `_b_ghc` 中。

```
2019-09-08T22:01:20.954866+08:00    17760 Query create /* gh-ost */ table `test`.`_
_b_ghc` (
    id bigint auto_increment,
    last_update timestamp not null DEFAULT CURRENT_TIMESTAMP ON UPDATE CURR
ENT_TIMESTAMP,
    hint varchar(64) charset ascii not null,
    value varchar(4096) charset ascii not null,
    primary key(id),
    unique key hint_uidx(hint)
) auto_increment=256
2019-09-08T22:01:20.957550+08:00    17760 Query create /* gh-ost */ table `test`.`_
_b_gho` like `test`.`b`
2019-09-08T22:01:20.960110+08:00    17760 Query alter /* gh-ost */ table `test`.`_
_b_gho` engine=innodb
2019-09-08T22:01:20.966740+08:00    17760 Query
    insert /* gh-ost */ into `test`.`_b_ghc`(id, hint, value) values (NULLIF(2, 0),
'state', 'GhostTableMigrated') on duplicate key update last_update=NOW(),value=VA
LUES(value)
```

4. insert into xx_gho select * from xx 拷贝数据

获取当前的最大主键和最小主键, 然后根据命令行传参 `chunk` 获取数据 insert 到影子表里面。

获取最小主键 `select `id` from `test`.`b` order by `id` asc limit 1;`

获取最大主键 `select `id` from `test`.`b` order by `id` desc limit 1;`

获取第一个 chunk:

```
select /* gh-ost `test`.`b` iteration:0 */ `id` from `test`.`b` where (((`id` >
_binary'1') or ((`id` = _binary'1')))) and (((`id` < _binary'21') or ((`id` =
_binary'21')))) order by `id` asc limit 1 offset 999;
```

循环插入到目标表:

```
insert /* gh-ost `test`.`b` */ ignore into `test`.`_b_gho` (`id`, `sid`, `name`,
`score`, `x`) (select `id`, `sid`, `name`, `score`, `x` from `test`.`b` force
index (`PRIMARY`) where (((`id` > _binary'1') or ((`id` = _binary'1')))) and
((`id` < _binary'21') or ((`id` = _binary'21')))) lock in share mode;
```

循环到最大的 id, 之后依赖 binlog 增量同步

需要注意的是 rowcopy 过程中是对原表加上 `lock in share mode`, 防止数据在 copy 的过程中被修改。

这点对后续理解整体的数据迁移非常重要。因为 gh-ost 在 copy 的过程中不会修改这部分数据记录。对于解析 binlog 获得的 INSERT, UPDATE, DELETE 事件我们只需要分析 copy 数据之前 log before copy 和 copy 数据之后 log after copy。整体的数据迁移会在后面做详细分析。

5. 增量应用 binlog 迁移数据

核心代码在 gh-ost/go/sql/builder.go 中，这里主要做 DML 转换的解释，当然还有其他函数做辅助工作，比如数据库，表名校验 以及语法完整性校验。解析到 delete 语句对应转换为 delete 语句

```
func BuildDMLDeleteQuery(databaseName, tableName string, tableColumns, uniqueKeyColumns *ColumnList, args []interface{}) (result string, uniqueKeyArgs []interface{}, err error) {
    ....省略代码...
    result = fmt.Sprintf(`
        delete /* gh-ost %s.%s */
            from
                %s.%s
            where
                %s
    `, databaseName, tableName,
        databaseName, tableName,
        equalsComparison,
    )
    return result, uniqueKeyArgs, nil
}
```

解析到 insert 语句对应转换为 replace into 语句

```
func BuildDMLInsertQuery(databaseName, tableName string, tableColumns, sharedColumns, mappedSharedColumns *ColumnList, args []interface{}) (result string, sharedArgs []interface{}, err error) {
    ....省略代码...
    result = fmt.Sprintf(`
        replace /* gh-ost %s.%s */ into
            %s.%s
            (%s)
        values
            (%s)
    `, databaseName, tableName,
        databaseName, tableName,
        strings.Join(mappedSharedColumnNames, ", "),
        strings.Join(preparedValues, ", "),
    )
    return result, sharedArgs, nil
}
```


解析到 update 语句 对应转换为语句

```
func BuildDMLUpdateQuery(databaseName, tableName string, tableColumns, sharedColumns, mappedSharedColumns, uniqueKeyColumns *ColumnList, valueArgs, whereArgs []interface{}) (result string, sharedArgs, uniqueKeyArgs []interface{}, err error) {
    ....省略代码...
    result = fmt.Sprintf(`
        update /* gh-ost %s.%s */
            %s.%s
        set
            %s
        where
            %s
    `, databaseName, tableName,
        databaseName, tableName,
        setClause,
        equalsComparison,
    )
    return result, sharedArgs, uniqueKeyArgs, nil
}
```

数据迁移的数据一致性分析

gh-ost 做 DDL 变更期间对原表和影子表的操作有三种：对原表的 row copy（我们用 A 操作代替），业务对原表的 DML 操作(B)，对影子表的 apply binlog(C)。而且 binlog 是基于 DML 操作产生的，因此对影子表的 apply binlog 一定在 对原表的 DML 之后，共有如下几种顺序：

	row copy (A)	DML(B)	apply binlog (C)
A-B-C 模式,数据先被拷贝到影子表	insert ignore into b select id>0 and id<3; 将数据同步到影子表	insert into b(id,val) vaues(2,'b');	replace into b(id,val) vaues(2,'b');
		update b set val='b' where id=1;	update b set val='b' where id=1;
		delete from b where id=1 ;	delete from b where id=1 ;
B-C-A 模式 , 数据还未copy到影子表, 先应用dml 的binlog	DML(B)	apply binlog	row copy (A)
	insert into b(id,val) vaues(2,'a');	replace into b(id,val) vaues(2,'a');	因为id=2的binlog比rowcopy先到达, 影子表已有id=2的数据, insert ignore。
	update b set val='b' where id=1;	update b set val='b' where id=1; id=1的记录还未拷贝到影子表, 所以update 空记录	row copy 将id=1的记录copy过来
	delete from b where id=1 ;	delete from b where id=1 ; id=1的记录还未拷贝到影子表, 所以update 空记录	row copy的时候查询不到 id=1的数据, 故也不会同步到影子表
B-A-C 模式 , 数据还未copy到影子表, 先应用dml 的binlog	DML(B)	row copy (A)	apply binlog (C)
	insert into b(id,val) vaues(2,'a');	insert ignore into b(id,val) vaues(1,'a'), (2,'b');	因为id=2的rowcopy比binlog先到达, 影子表已有id=2的数据, apply binlog 会replace into 覆盖id=2。
	update b set val='b' where id=1;	insert ignore into b(id,val) vaues(1,'b'); 执行update 之后, rowcopy将最新的数据拷贝到影子表	影子表已经有id=1的最新记录, binlog apply 执行 update b set val='b' where id=1;
	delete from b where id=1 ;	因为id=1的记录已经被删除, 故row copy 拷贝空行	apply binlog的时候 空操作

通过上面的几种组合操作的分析，我们可以看到数据最终是一致的。尤其是当 copy 结束之后，只剩下 apply binlog，情况更简单。

6. copy 完数据之后进行原始表和影子表 cut-over 切换

gh-ost 的切换是原子性切换，基本是通过两个会话的操作来完成。作者写了三篇文章解释 cut-over 操作的思路和切换算法。详细的思路请移步到下面的链接。

<http://code.openark.org/blog/mysql/solving-the-non-atomic-table-swap-take-iii-making-it-atomic>

<http://code.openark.org/blog/mysql/solving-the-non-atomic-table-swap-take-ii>

<http://code.openark.org/blog/mysql/solving-the-facebook-osc-non-atomic-table-swap-problem>

这里将第三篇文章描述核心切换逻辑摘录出来。其原理是基于 MySQL 内部机制：被 lock table 阻塞之后，执行 rename 的优先级高于 DML，也即先执行 rename table，然后执行 DML。假设 gh-ost 操作的会话是 c10 和 c20，其他业务的 DML 请求的会话是 c1-c9, c11-c19, c21-c29。

1 会话 c1..c9：对 b 表正常执行 DML 操作。

2 会话 c10：创建 _b_del 防止提前 rename 表，导致数据丢失。

```
create /* gh-ost */ table `test`.`_b_del` (
    id int auto_increment primary key
) engine=InnoDB comment='ghost-cut-over-sentry'
```

3 会话 c10 执行 LOCK TABLES b WRITE, `\_b\_del` WRITE。

4 会话 c11-c19 新进来的 dml 或者 select 请求，但是会因为表 b 上有锁而等待。

5 会话 c20：设置锁等待时间并执行 rename

```
set session lock_wait_timeout:=1
rename /* gh-ost */ table `test`.`b` to `test`.`_b_20190908220120_del`,
`test`.`_b_gho` to `test`.`b`
c20 的操作因为 c10 锁表而等待。
```

6 c21-c29 对于表 b 新进来的请求因为 lock table 和 rename table 而等待。

7 会话 c10 通过 sql 检查会话 c20 在执行 rename 操作并且在等待 dml 锁。

```
select id
from information_schema.processlist
where
    id != connection_id()
    and 17765 in (0, id)
    and state like concat('%', 'metadata lock', '%')
    and info like concat('%', 'rename', '%')
```

8 c10 基于步骤 7 执行 drop table `\_b\_del`，删除命令执行完，b 表依然不能写。所有的 dml 请求都被阻塞。

9 c10 执行 UNLOCK TABLES；此时 c20 的 rename 命令第一个被执行。而其他会话 c1-c9, c11-c19, c21-c29 的请求可以操作新的表 b。

划重点（敲黑板）

1. 创建 `_b_del` 表是为了防止 `cut-over` 提前执行，导致数据丢失。
2. 同一个会话先执行 `write lock` 之后还是可以 `drop` 表的。
3. 无论 `rename table` 和 `DML` 操作谁先执行，被阻塞后 `rename table` 总是优先于 `DML` 被执行。

大家可以一边自己执行 `gh-ost`，一边开启 `general log` 查看具体的操作过程。

```
2019-09-08T22:01:24.086734 17765 create /* gh-ost */ table `test`.`_b_20190908220120_del` (  
    id int auto_increment primary key  
    ) engine=InnoDB comment='ghost-cut-over-sentry'  
2019-09-08T22:01:24.091869 17760 Query lock /* gh-ost */ tables `test`.`b` write  
e, `test`.`_b_20190908220120_del` write  
2019-09-08T22:01:24.188687 17765 START TRANSACTION  
2019-09-08T22:01:24.188817 17765 select connection_id()  
2019-09-08T22:01:24.188931 17765 set session lock_wait_timeout:=1  
2019-09-08T22:01:24.189046 17765 rename /* gh-ost */ table `test`.`b` to `tes  
t`.`_b_20190908220120_del`, `test`.`_b_gho` to `test`.`b`  
2019-09-08T22:01:24.192293+08:00 17766 Connect root@127.0.0.1 on test using T  
CP/IP  
2019-09-08T22:01:24.192409 17766 SELECT @@max_allowed_packet  
2019-09-08T22:01:24.192487 17766 SET autocommit=true  
2019-09-08T22:01:24.192578 17766 SET NAMES utf8mb4  
2019-09-08T22:01:24.192693 17766 select id  
    from information_schema.processlist  
    where  
        id != connection_id()  
        and 17765 in (0, id)  
        and state like concat('%', 'metadata lock', '%')  
        and info like concat('%', 'rename', '%')  
2019-09-08T22:01:24.193050 17766 Query select is_used_lock('gh-ost.17760.lock')  
2019-09-08T22:01:24.193194 17760 Query drop /* gh-ost */ table if exists `test`.  
_b_20190908220120_del`  
2019-09-08T22:01:24.194858 17760 Query unlock tables  
2019-09-08T22:01:24.194965 17760 Query ROLLBACK  
2019-09-08T22:01:24.197563 17765 Query ROLLBACK  
2019-09-08T22:01:24.197594 17766 Query show /* gh-ost */ table status from `tes  
t` like '_b_20190908220120_del'  
2019-09-08T22:01:24.198082 17766 Quit  
2019-09-08T22:01:24.298382 17760 Query drop /* gh-ost */ table if exists `test`.  
_b_ghc`
```

如果 `cut-over` 过程的各个环节执行失败会发生什么？

其实除了安全，什么都不会发生。

- 如果 c10 的 create `\_b\_del` 失败, gh-ost 程序退出。
- 如果 c10 的加锁语句失败, gh-ost 程序退出, 因为表还未被锁定, dml 请求可以正常进行。
- 如果 c10 在 c20 执行 rename 之前出现异常
 - A. c10 持有的锁被释放, 查询 c1-c9, c11-c19 的请求可以立即在 b 执行。
 - B. 因为 `\_b\_del` 表存在, c20 的 rename table b to `\_b\_del` 会失败。
 - C. 整个操作都失败了, 但没有什么可怕的事情发生, 有些查询被阻止了一段时间, 我们需要重试。
- 如果 c10 在 c20 执行 rename 被阻塞时失败退出, 与上述类似, 锁释放, 则 c20 执行 rename 操作因为 —b_old 表存在而失败, 所有请求恢复正常。
- 如果 c20 异常失败, gh-ost 会捕获不到 rename, 会话 c10 继续运行, 释放 lock, 所有请求恢复正常。
- 如果 c10 和 c20 都失败了, 没问题: lock 被清除, rename 锁被清除。c1-c9, c11-c19, c21-c29 可以在 b 上正常执行。

整个过程对应用程序的影响

应用程序对表的写操作被阻止, 直到交换影子表成功或直到操作失败。如果成功, 则应用程序继续在新表上进行操作。如果切换失败, 应用程序继续继续在原表上进行操作。

对复制的影响

slave 因为 binlog 文件中不会复制 lock 语句, 只能应用 rename 语句进行原子操作, 对复制无损。

7 处理收尾工作

最后一部分操作其实和具体参数有一定关系。最重要必不可少的是

关闭 binlogsyncer 连接

至于中间表, 其实和参数有关 `--initially-drop-ghost-table --initially-drop-old-table`

3 小结

纵观 gh-ost 的执行过程, 查看源码算法设计, 尤其是 cut-over 设计思路之精妙, 原子操作, 任何异常都不会对业务有严重影响。欢迎已经使用过的朋友分享各自遇到的问题, 也欢迎还未使用过该工具的朋友大胆尝试。

参考文章

<https://www.cnblogs.com/mysql-dba/p/9901589.html>



MySQL 跨实例 copy 大表解决方案

作者：任坤

1 背景

某天晚上 20:00 左右开发人员找到我，要求把 pre-prod 环境上的某张表导入到 prod，第二天早上 07:00 上线要用。

该表有数亿条数据，压缩后 ibd 文件大约 25G 左右，表结构比较简单：

```
CREATE TABLE `t` (  
  `UNIQUE_KEY` varchar(32) NOT NULL,  
  `DESC` varchar(64) DEFAULT NULL ,  
  `NUM_ID` int(10) DEFAULT '0' ,  
  PRIMARY KEY (`UNIQUE_KEY`),  
  KEY `index_NumID` (`NUM_ID`) USING BTREE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8 ROW_FORMAT=COMPRESSED
```

MySQL 版本：pre-prod 和 prod 都采用 5.7.25，单向主从结构。

2 解决方案

最简单的方法是采用 mysqldump + source，但是该表数量比较多，之前测试的时候至少耗时 4h+，这次任务时间窗口比较短，如果中间执行失败再重试，可能会影响业务正式上线。采用 select into outfile + load infile 会快一点，但是该方案有个致命问题：该命令在主库会把所有数据当成单个事务执行，只有把数据全部成功插入后，才会将 binlog 复制到从库，这样会造成从库严重延迟，而且生成的单个 binlog 大小严重超标，在磁盘空间不足时可能会把磁盘占满。

经过比较，最终采用了可传输表空间方案，MySQL 5.6 借鉴 Oracle 引入该技术，允许在 2 个不同实例间快速的 copy innodb 大表。该方案规避了昂贵的 sql 解析和 B+tree 叶节点分裂，目标库可直接重用其他实例已有的 ibd 文件，只需同步一下数据字典，并对 ibd 文件页进行一下校验，即可完成数据同步操作。

具体操作步骤如下：

1. 目标库，创建表结构，然后执行 ALTER TABLE t DISCARD TABLESPACE，此时表 t 只剩下 frm 文件
2. 源库，开启 2 个会话
 - a) session1: 执行 FLUSH TABLES t FOR EXPORT，该命令会对 t 加锁，将 t 的脏数据从 buffer pool 同步到表文件，同时新生成 1 个文件 t.cfg，该文件存储了表的数据字典信息
 - b) session2: 保持 session1 打开状态，此时将 t.cfg 和 t.ibd 远程传输到目标库的数据目录，如果目标库是主从结构，需要分别传输到主从两个实例，传输完毕后修改属主为 mysql:mysql
 - c) 源库，session1 执行 unlock tables，解锁表 t，此时 t 恢复正常读写
 - d) 目标库，执行 ALTER TABLE t IMPORT TABLESPACE，如果是主从结构，只需要在主库执行即可

3 实测

针对该表，执行 `ALTER TABLE ... IMPORT TABLESPACE` 命令只需要 6 分钟完成，且 IO 消耗和主从延迟都被控制到合理范围。原本需要数个小时的操作，只需 10 多分钟完成（算上数据传输耗时）。如果线上有空表需要一次性加载大量数据，可以考虑先将数据导入到测试环境，然后通过可传输表空间技术同步到线上，可节约大量执行时间和服务器资源。

4 总结

可传输表空间，有如下使用限制：

- ☐ 源库和目标库版本一致
- ☐ 只适用于 innodb 引擎表
- ☐ 源库执行 `flush tables t for export` 时，该表会不可写



InnoDB 表空间加密

作者：秦沛

1 表空间加密概述

从 5.7.11 开始，InnoDB 支持对独立表空间进行静态数据加密。该加密是在引擎内部数据页级别的加密手段，在数据页写入文件系统时加密，从文件读到内存中时解密，目前广泛使用的是 YaSSL/OpenSSL 提供的 AES 加密算法，加密前后数据页大小不变，因此也称为透明加密。

它使用两层加密密钥架构，包括 master encryption key 和 tablespace key：

- ☐ master encryption key 用于加密解密 tablespace key。当对一个表空间加密时，一个 tablespace key 会被加密并存储在该表空间的头部
- ☐ tablespace key 用于加密表空间文件。当访问加密表空间时，
 - InnoDB 会先用 master encryption key 解密存储在表空间中的加密 tablespace key，得到明文 tablespace key 后，再用 tablespace key 解密数据
- ☐ tablespace key 解密后的明文信息基本不变（除非进行 `alter table test_1 encryption=NO`, `alter table test_1 encryption=YES`）。而 master key 可以随时改变（比如使用 `ALTER INSTANCE ROTATE INNODB MASTER KEY;`），这个称为 master key rotation。因为 tablespace key 的明文不会变，所以更新 master encryption key 之后只需要把 tablespace key 重新加密写入第一个页中即可。

1.1 应用场景

未配置表空间加密时，当发生类似拖库操作时，数据极可能会泄漏。

配置表空间加密时，如果没有加密时使用的 keyring（该文件由 `keyring_file_data` 参数设定），是读取不到加密表空间数据的。所以当发生类似拖库操作时，没有相关的 keyring 文件时，数据基本不会泄漏的。这就要求存储的 keyring 一定要严加保管，可以采取以下措施来保存 keyring：

- ☐ 避免将 keyring 文件与表空间文件放在一块
- ☐ keyring 文件所在目录的权限需要严格控制
- ☐ 使用 `keyring_file` 插件进行表空间加密时，keyring 文件是放在本地的，这个相对不够安全。但可以将其放到非本地中，需要时复制到相关目录（比如重启数据库、更新 master encryption key，不需要时删除该文件即可

1.2 加密插件

InnoDB 表空间加密依赖插件进行加密。企业版提供以下四种插件：

keyring_file, keyring_encrypted_file, keyring_okv, keyring_aws。

社区版目前只能使用 keyring_file 进行加密，本文仅介绍 keyring_file 插件：

- ☐ 企业版更加安全。因为对密钥也加密存储了，而社区版的密钥是放在本地存储的，相对不够安全
- ☐ 企业版支持多种后端存储密钥，对数据加密本身没区别

1.3 加密限制

1. AES 是唯一支持的加密算法。InnoDB 静态表空间加密使用 Electronic Codebook (ECB) 加密模式来加密 tablespace key，使用 Cipher Block Chaining (CBC) 加密模式加密数据文件
2. ENCRYPTION 使用该 COPY 命令而不是 INPLACE 命令进行表空间的加密
3. 仅支持对独立表空间加密。不支持加密其他表空间类型（如通用表空间和系统表空间）
4. 不能将表从加密的独立表空间移动或复制到不支持加密的表空间中
5. 表空间加密仅对表空间中的数据加密，不对 redo log, undo log, binary log 中的数据加密
6. 不允许修改已加密的表的存储引擎（修改为 innodb 存储引擎则没问题）

1.4 注意事项

1. ==在创建了第一个加密表空间、master encryption key 更新前后都必须立马备份密钥环文件。因为主加密密钥丢失后，加密表空间中的数据将无法恢复。所以加密表时，必须采取措施防止主加密密钥丢失，比如定时备份该文件#F44336 #F44336==
2. 从安全角度考虑，不建议将密钥环数据文件与表空间数据文件放在同一目录下
3. 如果数据库在正常操作期间退出或停止，一定要使用同样的加密配置来重启数据库，否则会导致以前加密的表空间无法访问
4. 当第一次对表空间加密时（无论是新表加密还是旧表加密），将生成第一个主加密密钥。但对于运行的数据库，移除 keyring 后，依旧可以创建、读写加密表空间（但在数据库重启或更新 master encryption key 后这些操作会失败）
5. 更新 master encryption key 前需确保 keyring 文件存在，如果不存在，则会更新失败，从而导致读写、创建加密表均失败
6. 仅当主从都配置了表空间加密，表空间加密操作才可能在主从均执行成功

2 加密表空间

以下的测试案例是在如下环境进行的：

- ☐ linux 版本：7.5
- ☐ mysql 版本：MySQL 5.7.21
- ☐ 加密插件：keyring_file

2.1 安装加密插件

1. 必须先安装并配置加密插件。在启动数据库时使用 `early-plugin-load` 选项来指定使用的加密插件，并使用 `keyring_file_data` 定义加密插件存放密钥环文件的路径
需要提前创建好相关目录并调整权限，不然可能会报错
2. 只能使用一个加密插件，不能同时使用多个加密插件
3. 一旦在 MySQL 实例中创建了加密表，后续重启该实例时，必须给 `early-plugin-load` 指定创建加密表时使用的加密插件。如果不这样做，则在启动服务器和 InnoDB 恢复期间会导致错误
4. 必须配置独立表空间：`innodb_file_per_table=1`

-- vim my.cnf, 在 [mysqld] 下添加以下参数

```
[mysqld]
early-plugin-load="keyring_file.so"
keyring_file_data=/opt/mysql/keyring/3306/keyring
innodb_file_per_table=1
```

-- 创建相关目录及修改密钥环文件所在目录的权限

```
mkdir -p /opt/mysql/keyring/3306/
chown -R actiontech-universe:actiontech /opt/mysql/keyring/
chown -R actiontech-mysql:actiontech-mysql /opt/mysql/keyring/3306/
```

-- 查看插件是否加载

```
SELECT  PLUGIN_NAME,  PLUGIN_STATUS  FROM    INFORMATION_SCHEMA.PLUGINS  WHERE
PLUGIN_NAME LIKE 'keyring_file';
```

2.2 配置表空间加密

以下以 `mydata.test_1` 表来进行测试

/*

1. 为新表加密
2. 插入数据至新表
3. 取消表加密
4. 开启表加密
5. 查看加密表

*/

```

mysql> CREATE TABLE mydata.test_1 (id INT primary key,age int) ENCRYPTION='Y';
Query OK, 0 rows affected (0.02 sec)
mysql> insert into test_1 select 9,9;
Query OK, 1 row affected (0.00 sec)
Records: 1 Duplicates: 0 Warnings: 0
mysql> ALTER TABLE mydata.test_1 ENCRYPTION='N';
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0
mysql> ALTER TABLE mydata.test_1 ENCRYPTION='Y';
Query OK, 0 rows affected (0.02 sec)
Records: 0 Duplicates: 0 Warnings: 0
mysql> show create table mydata.test_1;select * from mydata.test_1;
+-----+-----+
-----+
|          Table          |          Create          Table
|
+-----+-----+
-----+
| test_1 | CREATE TABLE `test_1` (
  `id` int(11) NOT NULL,
  `age` int(11) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 ENCRYPTION='Y' |
+-----+-----+
-----+
1 row in set (0.11 sec)

+----+-----+
| id | age |
+----+-----+
|  9 |   9 |
+----+-----+
1 row in set (0.00 sec)

/*

```

删除当前的 keyring，使用备份的 keyring 恢复到原路径并重启，此时再查看 mydata.test_1 会查看成功。因为 mydata.test_1 加密解密用的 keyring 一样

```

*/
[root@localhost ~]# rm -rf /opt/mysql/keyring/3306/keyring
[root@localhost ~]# mv /root/keyring /opt/mysql/keyring/3306/
[root@localhost ~]# systemctl restart mysqld_3306
[root@localhost ~]# mysql -h10.186.63.90 -uroot -p -P3306 -e"show create table
mydata.test_1;select * from mydata.test_1;"
Enter password:

```

```

+-----+-----+
--+
|          Table          |          Create          Table
|
+-----+-----+
-----+
| test_1 | CREATE TABLE `test_1` (
  `id` int(11) NOT NULL,
  `age` int(11) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 ENCRYPTION='Y' |
+-----+-----+
-----+
+-----+-----+
| id | age |
+-----+-----+
| 9 | 9 |
+-----+-----+

```

2.3 查看表空间被加密的表

通过 `ENCRYPTION` 选项加密表空间时，表的加密信息会存放到 `INFORMATION_SCHEMA.TABLES` 的 `CREATE_OPTIONS` 字段中。所以可以查询该表来判断表是否加密。

-- 查看加密的表：

```
SELECT TABLE_SCHEMA, TABLE_NAME, CREATE_OPTIONS FROM INFORMATION_SCHEMA.TABLES
WHERE CREATE_OPTIONS LIKE '%ENCRYPTION%';
```

-- 查看未加密的表：

```
select concat(TABLE_SCHEMA,".",TABLE_NAME) from INFORMATION_SCHEMA.TABLES where
(TABLE_SCHEMA,TABLE_NAME) not in (SELECT TABLE_SCHEMA,TABLE_NAME FROM
```

3 更新 master encryption key

应定期更换 master encryption key，或者怀疑 master encryption key 已经泄露时便需要更换了。

更新 master encryption key 只会改变 master encryption key 并重新加密 tablespace keys，不会解密或者重新加密表空间。

因为 tablespace_key 的明文不会变，更新 master_key 之后只需要把 tablespace_key 重新加密写入第一个页中即可。

master encryption key 的变更是一个原子的、实例级操作，每次变更 master encryption key 时，MySQL 实例中所有的 tablespace key 都会被重新加密并保存到各自的表空间头部。因为是原子操作，所以一旦开始更新，对所有 tablespace keys 的重新加密必须全部成功；

```
-- 手动更新 master encryption key, 成功后不影响加密表的使用
[root@localhost ~]# ll /opt/mysql/keyring/3306/keyring
-rw-r----- 1 mysql mysql 155 Apr 19 02:05 /opt/mysql/keyring/3306/keyring
[root@localhost ~]# mysql -h10.186.63.90 -uroot -p -P3306 -e"ALTER INSTANCE
ROTATE INNODB MASTER KEY;"
Enter password:
[root@localhost ~]# ll /opt/mysql/keyring/3306/keyring
-rw-r----- 1 mysql mysql 283 Apr 19 02:24 /opt/mysql/keyring/3306/keyring
[root@localhost ~]# mysql -h10.186.63.90 -uroot -p -P3306 -e"select * from
mydata.test_1;
Enter password:
+----+-----+
| id | age |
+----+-----+
| 9  | 9   |
+----+-----+
```

4 导入导出

为了支持 Export/Import 加密表，引入了 transfer_key，在 export 的时候随机生成一个 transfer_key，把现有的 tablespace_key 用 transfer_key 加密，并将两者同时写入 table_name.cfp 的文件中，注意这里 transfer_key 保存的是明文。Import 会读取 transfer_key 用来解密，然后执行正常的 import 操作即可，一旦 import 完成，table_name.cfp 文件会被立刻删除：

- 导出加密表空间时，innodb 除了会生成 .cfg 元数据文件之外，还会生成 .cfp 文件，用于加密 tablespace key 的 transfer key。加密的 tablespace key 和 transfer key 存储在 tablespace_name.cfp 文件
- 导入加密表时，需要同时导入 tablespace_name.cfp 和加密表空间才行，InnoDB 通过 transfer key 来解密 tablespace_name.cfp 文件中的 tablespace key

导入导出流程如下：

1. 目标库：CREATE TABLE mydata.test_1 (id INT primary key,age int) ENCRYPTION='Y'；，建立与源库同名、同结构的表。
2. 目标库：ALTER TABLE test_1 DISCARD TABLESPACE；，此时会删除 .ibd 文件
3. 源库：use test； FLUSH TABLES test_1 FOR EXPORT；，此时会产生 .cfg、.cfp 文件
4. 目标库：scp root@10.186.63.90:/opt/mysql/data/3306/mydata/test_1.{ibd,cfg,cfp} .
5. 目标库：chown actiontech-mysql:actiontech-mysql test_1*，复制文件后，需要修改用户组及权限。
6. 源库：unlock tables；，此时会删除 .cfg、.cfp 文件
7. 目标库：ALTER TABLE test_1 IMPORT TABLESPACE；，加载表 test_1

案例

源库: 10.186.63.90:3306

目标库: 10.186.63.91:3307

在目标库中建立与源库同名、同结构的

```
[root@localhost ~]# fg
mysql -h10.186.63.91 -uroot -p -P3307      (wd: ~)
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> create database mydata;
Query OK, 1 row affected (0.01 sec)

mysql> CREATE TABLE mydata.test_1 (id INT primary key,age int) ENCRYPTION='Y';
Query OK, 0 rows affected (0.02 sec)
```

目标库执行 DISCARD TABLESPACE

```
mysql> use mydata;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
Database changed
mysql> ALTER TABLE test_1 DISCARD TABLESPACE;
Query OK, 0 rows affected (0.02 sec)
```

源库执行 flush table ... fro export

```
[root@localhost ~]# fg
mysql -h10.186.63.90 -uroot -P3306 -p
mysql> use mydata;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
Database changed
mysql> show tables;
+-----+
| Tables_in_mydata |
+-----+
| test_1           |
+-----+
1 row in set (0.00 sec)
```

```
mysql> FLUSH TABLES test_1 FOR EXPORT;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql>
[1]+  Stopped                  mysql -h10.186.63.90 -uroot -P3306 -p
```

```
# 从源库拷贝文件至目标库
```

```
[root@localhost mydata]# scp root@10.186.63.90:/opt/mysql/data/3306/mydata/test_1.{ibd,cfg,cfp} .
root@10.186.63.90's password:
test_1.ibd
100% 96KB 41.0MB/s 00:00
root@10.186.63.90's password:
test_1.cfg
100% 400 343.7KB/s 00:00
root@10.186.63.90's password:
test_1.cfp
100% 100 132.7KB/s 00:00
```

```
[root@localhost mydata]# ll
total 120
-rw-r----- 1 actiontech-mysql actiontech-mysql 67 Sep 28 05:49 db.opt
-rw-r----- 1 root root 400 Sep 28 05:57 test_1.cfg
-rw-r----- 1 root root 100 Sep 28 05:57 test_1.cfp
-rw-r----- 1 actiontech-mysql actiontech-mysql 8584 Sep 28 05:50 test_1.frm
-rw-r----- 1 root root 98304 Sep 28 05:57 test_1.ibd
```

```
# 修改目标库上文件的权限
```

```
[root@localhost mydata]# chown actiontech-mysql:actiontech-mysql *
```

```
# 源库上执行 unlock tables
```

```
[root@localhost mydata]# fg
mysql -h10.186.63.90 -uroot -P3306 -p (wd: ~)
mysql> use mydata;
Database changed
mysql> unlock tables;
Query OK, 0 rows affected (0.00 sec)
```

```
# 目标库上执行 ALTER TABLE ... IMPORT TABLESPACE;
```

```
[root@localhost mydata]# fg
mysql -h10.186.63.91 -uroot -p -P3307 (wd: ~)
mysql> select * from mydata.test_1;
ERROR 1814 (HY000): Tablespace has been discarded for table 'test_1'

mysql> ALTER TABLE test_1 IMPORT TABLESPACE;
Query OK, 0 rows affected (0.05 sec)

mysql> select * from mydata.test_1;
Empty set (0.00 sec)
```

5 备份恢复

1) mysqlbackup 备份恢复

<https://dev.mysql.com/doc/mysql-enterprise-backup/4.1/en/meb-encrypted-innodb.html>

2) innobackupex 备份恢复

https://www.percona.com/doc/percona-xtrabackup/LATEST/advanced/encrypted_innodb_tablespace_backups.html#encrypted-innodb-tablespace-backups

mysqlbackup innobackupex 均可以对加密表空间进行加密，只不过需要注意版本：

- ☐ mysqlbackup 4.1.0 及更新的版本支持 MySQL 5.7.20 及更早版本的加密表空间备份；mysqlbackup 4.1.1 及更新版本则支持 MySQL 5.7 的加密表空间备份
- ☐ innobackupex 支持加密表空间备份恢复的最小版本没查到相关说明

这里以 innobackupex 2.4.5 为例进行加密表空间的备份恢复。

Innobackupex

- ☐ innobackupex 备份加密表空间的注意事项：innobackupex 仅支持使用 keyring_file、keyring_vault 插件加密表空间的备份
- ☐ innobackupex 不会复制密钥环文件到备份目录中，所以需要手动复制密钥环文件到配置文件指定的 keyring-file-data 路径
如果备份前后密钥环文件不同，则在还原时应该使用旧的密钥环文件

备份 10.186.63.90 中的数据至 10.186.63.91:3307

全量备份：加密表空间的全备的流程与常规的备份恢复基本一样，只是需要额外指定一个参数：--keyring-file-data

```
mkdir /data2/all_backup
```

```
/data/urman-agent/bin/innobackupex --defaults-file=/opt/mysql/etc/3306/my.cnf --user=root --password=test -P3306 --socket=/opt/mysql/data/3306/mysqld.sock --parallel=8 --keyring-file-data=/opt/mysql/keyring/3306/keyring --no-timestamp /data2/all_backup
```

```
# apply-log
```

```
/data/urman-agent/bin/innobackupex --apply-log --keyring-file-data=/opt/mysql/keyring/3306/keyring /data2/all_backup/
```

```
# 复制全备文件、keyring 文件至目标库
```

```
rm -rf /opt/mysql/data/3307/* && rm -rf /opt/mysql/keyring/3307/keyring && rm -rf /opt/mysql/log/redolog/3307/ib_logfile*
```

```
scp -r /data2/all_backup/ root@10.186.63.91:/data2/
```

```
scp /opt/mysql/keyring/3306/keyring root@10.186.63.91:/opt/mysql/keyring/3307
```



```
# copy back
/data/urman-agent//bin/innobackupex --defaults-file=/opt/mysql/etc/3307/my.cnf -
-copy-back --keyring-file-data=/opt/mysql/keyring/3307/keyring /data2/all_backup
/

# 修改权限
chown actiontech-mysql:actiontech-mysql /opt/mysql/keyring/3307/keyring
chown -R actiontech-mysql:actiontech-mysql /opt/mysql/data/3307/*
chown -R actiontech-mysql:actiontech-mysql /opt/mysql/log/redolog/*

# 启动
systemctl start mysqld_3307
```

参考文档

14.6.3.8 InnoDB Tablespace EncryptionInnoDB

<https://dev.mysql.com/doc/refman/5.7/en/innodb-tablespace-encryption.html#innodb-tablespace-encryption-about>

表空间加密—原理篇

<http://mysql.taobao.org/monthly/2018/04/01/>



slave_relay_log_info 表认知的一些展开

作者：胡呈清

slave_relay_log_info 表是这样的：

```
mysql> select * from mysql.slave_relay_log_info\G
***** 1. row *****
Number_of_lines: 7
Relay_log_name: ./mysql-relay.000015
Relay_log_pos: 621
Master_log_name: mysql-bin.000001
Master_log_pos: 2407
Sql_delay: 0
Number_of_workers: 16
Id: 1
Channel_name:
```

slave_relay_log_info 表存储 slave sql thread 的工作位置。

在从库启动的时候时，读取 slave_relay_log_info 表中存储的位置，并把值传给 “show slave status” 中的 Relay_Log_File、Relay_Log_Pos，下次 “start slave” 是从这个位置开始继续回放 relay log。

slave_relay_log_info 表存储的是持久化的状态、show slave status 输出的是内存中的状态：

- ☐ 两者输出的位置可能不一样
- ☐ stop slave 或者正常关闭 mysqld，都会将内存中的状态持久化到磁盘上（slave_relay_log_info 表中）
- ☐ 启动 mysqld 时会读取磁盘状态，初始化给内存状态
- ☐ start slave 时生效的是内存状态

slave io thread 按照 Master_Log_File、Read_Master_Log_Pos 位置读取主库的 binlog，并写入到本地 relay log（注意这两个位点信息保存在 slave_master_info 表中）；slave sql thread 按照 Relay_Log_Name、Relay_Log_Pos 位置进行 relay log 的回放。

但是同一个事务在从库 relay log 中的 position 和主库 binlog 中的 position 是不相等的，slave_relay_log_info 表通过 Master_log_name、Master_log_pos 这两个字段记录了 relay log 中事务对应主库 binlog 中的 position。

我们得知道如果 slave io thread 重复、遗漏的读取主库 binlog 写入到 relay log 中，sql thread 也会重复、遗漏地回放这些 relay log。也就是说从库的数据是否正确，io thread 的位置是否正确也非常重要。

在 MySQL 5.6 以前，复制位点信息只能存储在数据目录的 `master.info` 文件中，在回放事务后更新到文件中（默认每次回放 10000 个事务更新，受参数 `sync_relay_log_info` 控制）。即使每个事务都更新文件，意外宕机时也没法保证持久性一致性。

MySQL 5.6 开始，可以设置 `--relay-log-info-repository=TABLE`，将 `slave sql thread` 的工作位置存储在 `mysql.slave_relay_log_info` 表中，如果这个表是 InnoDB 这样的支持事务的引擎，则从库每回放一个事务时都会在这个事务里同时更新 `mysql.slave_relay_log_info` 表，使得 `sql thread` 的位置与数据保持一致。事实上在 5.6.0-5.6.5 的版本，`slave_relay_log_info` 表默认使用的是 MyISAM 引擎，之后的版本才改为 InnoDB，不过再考虑到 MySQL 5.6.10 才 GA，这个坑踩过的人应该不多。

更新机制

引用手册：

`sync_relay_log_info = 0`

If `relay_log_info_repository` is set to `FILE`, the MySQL server performs no synchronization of the `relay-log.info` file to disk; instead, the server relies on the operating system to flush its contents periodically as with any other file.

If `relay_log_info_repository` is set to `TABLE`, and the storage engine for that table is transactional, the table is updated after each transaction. (The `sync_relay_log_info` setting is effectively ignored in this case.)

If `relay_log_info_repository` is set to `TABLE`, and the storage engine for that table is not transactional, the table is never updated.

`sync_relay_log_info = N > 0`

If `relay_log_info_repository` is set to `FILE`, the slave synchronizes its `relay-log.info` file to disk (using `fdatsync()`) after every `N` transactions.

If `relay_log_info_repository` is set to `TABLE`, and the storage engine for that table is transactional, the table is updated after each transaction. (The `sync_relay_log_info` setting is effectively ignored in this case.)

If `relay_log_info_repository` is set to `TABLE`, and the storage engine for that table is not transactional, the table is updated after every `N` events.

一般的运维规范都会要求 `relay_log_info_repository=TABLE`，默认值 `sync_relay_log_info=10000` 此时会失效，变成每回放一个事务都会在这个事务里同时更新 `mysql.slave_relay_log_info` 表，保证持久性，以最终保证复制的数据一致。当然 InnoDB 的持久性需要 `innodb_flush_log_at_trx_commit=1` 来保证。

前面有一句话“也就是说从库的数据是否正确，`io thread` 的位置是否正确也非常重要”。简单来说 `io thread` 位置保存在 `slave_master_info` 表中，其实设置和 `relay_log_info_repository` 类似，不同的是它的持久化保障通常与性能冲突很大：

□ 必须设置 `master_info_repository = TABLE` 和 `sync_master_info=1`，刷盘的单位是 `binlog event` 而不是事务，写放大很严重，性能损耗大

所以通常 `sync_master_info` 使用默认值 10000，`io thread` 的位置无法保证持久化，也就没法保证正

确。MySQL 有另一个参数 `relay_log_recovery` 提供一种机制来保证 `mysqld` crash 后 `io thread` 位置的准确性，稍后进行介绍。

master_auto_position

`master_auto_position` 的作用是根据从库的 `Executed_Gtid_Set` 自动寻找主库上对应 `binlog` 位置，这是在 GTID 出现后的一个功能。

这里思考一个问题：开启 `master_auto_position` 后，`slave io thread` 能直接根据从库的 `Executed_Gtid_Set` 定位主库上 `binlog` 的位置吗？还需要 `slave_relay_log_info`、`slave_master_info` 表中记录的位点信息吗？

其实 `slave_relay_log_info`、`slave_master_info` 表依然发挥作用：

- ☐ 当第一次或者 `reset slave` 后，执行 `start slave`，`io thread` 将从库的 `Executed_Gtid_Set` 发往主库，获取到对应的 `File`、`Position`，之后更新到从库的 `slave_relay_log_info`、`slave_master_info` 表中
- ☐ 当 `slave_relay_log_info`、`slave_master_info` 表中存在位置信息后，此后无论是重启复制还是重启 `mysqld`，都是直接从这两个地方获取 `File`、`Position`，并从这里开始读取 `binlog` 和回放 `relay log`

注意：执行 “`reset slave`” 会删除从库上的 `relay log`，并且重置 `slave_relay_log_info` 表，即重置复制位置。如果 `master_auto_position=0`，下次启动复制时会从新开始获取并回放主库的 `binlog`，造成错误。

relay_log_recovery

当启用 `relay_log_recovery`，`mysqld` 启动时，`recovery` 过程会生成一个新的 `relay log`，并初始化 `slave sql thread` 的位置，表现为：

- ☐ `slave_relay_log_info` 表的 `Relay_Log_Name` 值更新为最新的日志名，`Relay_Log_Pos` 值更新为一个固定值 4（应该是头部固定信息占 4 个偏移量）
- ☐ 内存状态即 `show slave status` 输出中的 `Relay_Log_File`、`Relay_Log_Pos`，也更新成上面一样的

并且 `slave io thread` 的位置也会初始化，表现为：

- ☐ `slave_master_info` 表中的 `Master_log_name`、`Master_log_pos` 不会改变
- ☐ 内存状态即 `show slave status` 输出中的 `Master_Log_File`、`Read_Master_Log_Pos`，会更新为 `slave_relay_log_info` 表中 `Master_Log_Name`、`Master_Log_Pos`

`io thread` 这个位置的初始化思路就是：既然以前记录的位置不确定是否准确，那就直接不要了。`sql thread` 回放到哪，我就从哪开始重新拿主库的 `binlog`，这样准没错。一个事务在 `relay log` 中的 `position` 对应到主库 `binlog` 的 `position` 是这样来确定的：

- ☐ `slave_relay_log_info` 表中的 `Relay_log_name` 与 `Master_log_name`，`Relay_Log_Pos` 与 `Master_Log_Pos` 始终一一对应，代表同一个事务的位置。

所以，即使 `sync_master_info` 表的持久化无法保证，`relay_log_recovery` 也会将 `io thread` 重置到已经回放的那个位置。

`relay_log_recovery` 的另一个作用是防止 `relay log` 的损坏，因为默认 `relay log` 是不保证持久化的（也不推荐设置 `sync_relay_log=1`），当操作系统或者 `mysqld` crash 后，`sql thread` 可能会因为 `relay log` 的损坏、丢失导致错误。

一些结论

- 当开启 `GTID` 和 `master_auto_position`，并设置 `relay_log_recovery=1`，即使 `relay_log_info_repository` 设置为 `file`，操作系统或者 `mysqld` crash 后，`mysqld` 下次重启启动复制都能保证数据与主库一致。即使 `slave_relay_log_info` 表中记录的位置不是最新的，`sql thread` 可能会重复回放一部分事务，但是从库已经存在这些事务的 `GTID`，这部分重复的事务会被跳过。
- 当未开启 `GTID` 和 `master_auto_position`，必须要设置 `relay_log_info_repository=table`、`relay_log_recovery=1`，操作系统或者 `mysqld` crash 后，`mysqld` 下次重启启动复制才能保证数据与主库一致。



MySQL 慢查询记录原理和内容解析

作者：高鹏（八怪）

背景

本文并不准备说明如何开启记录慢查询，只是将一些重要的部分进行解析。

如何记录慢查询可以自行参考官方文档：

□ 5.4.5 The Slow Query Log

本文使用 Percona 版本来开启参数 `log_slow_verbosity`，得到了更详细的慢查询信息。通常情况下信息没有这么多，但是一定是包含关系，本文也会使用 Percona 的参数解释说明一下这个参数的含义。

1 慢查询中的时间

实际上慢查询中的时间就是时钟时间，是通过操作系统的命令获得的时间，如下是 Linux 中获取时间的方式：

```
while (gettimeofday(&t, NULL) != 0)
{
    newtime= (ulonglong)t.tv_sec * 1000000 + t.tv_usec;
    return newtime;
}
```

实际上就是通过 OS 的 API `gettimeofday` 函数获得的时间。

2 慢查询记录的依据

1. `long_query_time`：如果执行时间超过本参数设置记录慢查询。
2. `log_queries_not_using_indexes`：如果语句未使用索引记录慢查询。
3. `log_slow_admin_statements`：是否记录管理语句。（如 `ALTER TABLE`, `ANALYZE TABLE`, `CHECK TABLE`, `CREATE INDEX`, `DROP INDEX`, `OPTIMIZE TABLE`, and `REPAIR TABLE`。）

本文主要讨论 `long_query_time` 参数的含义。

3 `long_query_time` 参数的具体含义

如果我们将语句的执行时间定义为如下：

实际消耗时间 = 实际执行时间+锁等待消耗时间

那么 `long_query_time` 实际上界定的是实际执行时间，所以有些情况下虽然语句实际消耗的时间很长但

是因为锁等待时间较长而引起的，那么实际上这种语句也不会记录到慢查询。

我们看一下 `log_slow_applicable` 函数的代码片段：

```
res= cur_utime - thd->utime_after_lock;

if(res > thd->variables.long_query_time)
    thd->server_status|= SERVER_QUERY_WAS_SLOW;
else
    thd->server_status&= ~SERVER_QUERY_WAS_SLOW;
```

这里实际上清楚的说明了上面的观点，是不是慢查询就是通过这个函数进行的判断的，非常重要。我可以清晰的看到如下公式：

- res (实际执行时间) = cur_utime (实际消耗时间) - $thd->utime_after_lock$ (锁等待消耗时间)
- 实际上在慢查询中记录的正是
- `Query_time`: 实际消耗时间
- `Lock_time`: 锁等待消耗时间

但是是否是慢查询其评判标准却是实际执行时间及 `Query_time - Lock_time`

其中锁等待消耗时间 (`Lock_time`) 我现在已经知道的包括：

1. MySQL 层 MDL LOCK 等待消耗的时间。(Waiting for table metadata lock)
2. MySQL 层 MyISAM 表锁消耗的时间。(Waiting for table level lock)
3. InnoDB 层 行锁消耗的时间。

4 MySQL 是如何记录锁时间

我们可以看到在公式中 `utime_after_lock` (锁等待消耗时间 `Lock_time`) 的记录也就成了整个公式的关键，那么我们试着进行 debug。

4.1 MySQL 层 `utime_after_lock` 的记录方式

不管是 MDL LOCK 等待消耗的时间还是 MyISAM 表锁消耗的时间都是在 MySQL 层记录的，实际上它只是记录在函数 `mysql_lock_tables` 的末尾会调用的 `THD::set_time_after_lock` 进行的记录时间而已如下：

```
void set_time_after_lock()
{
    utime_after_lock= my_micro_time();
    MYSQL_SET_STATEMENT_LOCK_TIME(m_statement_psi, (utime_after_lock - start_utime));
}
```

那么这里可以解析为代码运行到 `mysql_lock_tables` 函数的末尾之前的所有的时间都记录到 `utime_after_lock` 时间中，实际上并不精确。但是 MDL LOCK 的获取和 MyISAM 表锁的获取都包含在里面。所以即便是 `select` 语句也可能会看到 `Lock_time` 并不为 0。

下面是部分栈帧：

```
#0  THD::set_time_after_lock (this=0x7fff28012820) at /root/mysql5.7.14/percona-server-5.7.14-7/sql/sql_class.h:3414
#1  0x0000000001760d6d in mysql_lock_tables (thd=0x7fff28012820, tables=0x7fff28c16b58, count=1, flags=0) at /root/mysql5.7.14/percona-server-5.7.14-7/sql/lock.cc:366
#2  0x000000000151dc1a in lock_tables (thd=0x7fff28012820, tables=0x7fff28c16b50, count=1, flags=0) at /root/mysql5.7.14/percona-server-5.7.14-7/sql/sql_base.cc:6700
#3  0x00000000017c4234 in Sql_cmd_delete::mysql_delete (this=0x7fff28c16b50, thd=0x7fff28012820, limit=18446744073709551615)
    at /root/mysql5.7.14/percona-server-5.7.14-7/sql/sql_delete.cc:136
```

4.2 InnoDB 层的行锁的 utime_after_lock 记录方式

InnoDB 引擎层调用通过 thd_set_lock_wait_time 调用 thd_storage_lock_wait 函数完成的，部分栈帧如下：

```
#0  thd_storage_lock_wait (thd=0x7fff2c000bc0, value=9503561) at /root/mysql5.7.14/percona-server-5.7.14-7/sql/sql_class.cc:798
#1  0x00000000019a4b2a in thd_set_lock_wait_time (thd=0x7fff2c000bc0, value=9503561)
    at /root/mysql5.7.14/percona-server-5.7.14-7/storage/innobase/handler/ha_innodb.cc:1784
#2  0x0000000001a4b50f in lock_wait_suspend_thread (thr=0x7fff2c088200) at /root/mysql5.7.14/percona-server-5.7.14-7/storage/innobase/lock/lock0wait.cc:363
#3  0x0000000001b0ec9b in row_mysql_handle_errors (new_err=0x7ffff0317d54, trx=0x7ffff2f2e5d0, thr=0x7fff2c088200, savept=0x0)
    at /root/mysql5.7.14/percona-server-5.7.14-7/storage/innobase/row/row0mysql.cc:772
#4  0x0000000001b4fe61 in row_search_mvcc (buf=0x7fff2c087640 "\377", mode=PAGE_CUR_G, prebuilt=0x7fff2c087ac0, match_mode=0, direction=0)
    at /root/mysql5.7.14/percona-server-5.7.14-7/storage/innobase/row/row0sel.cc:5940
```

函数本身还是很简单，自己看看就知道了，就是相加而已如下：

```
void thd_storage_lock_wait(THD *thd, longlong value)
{
    thd->utime_after_lock+= value;
}
```


5 Percona 中的 log_slow_verbosity 参数

这是 Percona 的解释：

Specifies how much information to include in your slow log. The value is a comma-delimited string, and can contain any combination of the following values:

- ☐ microtime: Log queries with microsecond precision (mandatory).
- ☐ query_plan: Log information about the query's execution plan (optional).
- ☐ innodb: Log InnoDB statistics (optional).
- ☐ minimal: Equivalent to enabling just microtime.
- ☐ standard: Equivalent to enabling microtime, innodb.
- ☐ full: Equivalent to all other values OR'ed together.

总之在 Percona 中可以修改这个参数获得更加详细的信息大概的格式如下：

```
# Time: 2018-05-30T09:30:12.039775Z
# User@Host: root[root] @ localhost [] Id: 10
# Schema: test Last_errno: 1317 Killed: 0
# Query_time: 19.254508 Lock_time: 0.001043 Rows_sent: 0 Rows_examined: 0 Rows_affected: 0
# Bytes_sent: 44 Tmp_tables: 0 Tmp_disk_tables: 0 Tmp_table_sizes: 0
# InnoDB_trx_id: 0
# QC_Hit: No Full_scan: No Full_join: No Tmp_table: No Tmp_table_on_disk: No
# Filesort: No Filesort_on_disk: No Merge_passes: 0
# InnoDB_IO_r_ops: 0 InnoDB_IO_r_bytes: 0 InnoDB_IO_r_wait: 0.000000
# InnoDB_rec_lock_wait: 0.000000 InnoDB_queue_wait: 0.000000
# InnoDB_pages_distinct: 0
SET timestamp=1527672612;
select count(*) from z1 limit 1;
```

6 输出的详细解释

本节将会进行详细的解释，全部的慢查询的输出都来自于函数 `File_query_log::write_slow`，有兴趣的同学可以自己看看，我这里也会给出输出的位置和含义，其中含义部分可能给出的是源码中的注释。

6.1 第一部分时间

Time: 2018-05-30T09:30:12.039775Z 对应的代码：

```
my_snprintf(buff, sizeof buff, "# Time: %s\n", my_timestamp);
```

其中 `my_timestamp` 取值来自于

```
thd->current_utime();
```

实际上就是：

```
while(gettimeofday(&t, NULL) != 0)
{}
    newtime= (ulonglong)t.tv_sec * 1000000+ t.tv_usec;
return newtime;
```

可以看到实际就是调用 `gettimeofday` 系统调用得到的系统当前时间。

注意：对于 5.6 来讲还有一句判断

```
if(current_time != last_time)
```

如果两次打印的时间秒钟一致则不会输出时间，只有通过后面介绍的

```
SET timestamp=1527753496;
```

来判断时间，5.7.14 没有看到这样的代码。

6.2 第二部分用户信息

User@Host: root[root] @ localhost [] Id: 10 对应的代码：

```
    buff_len= my_snprintf(buff, 32, "%5u", thd->thread_id());
if(my_b_printf(&log_file, "# User@Host: %s Id: %s\n", user_host, buff)
== (uint) -1)
goto err;
}
```

`user_host` 是一串字符串，参考代码：

```
size_t user_host_len= (strxnmov(user_host_buff, MAX_USER_HOST_SIZE,
                                sctx->priv_user().str
? sctx->priv_user().str : "",
"[", sctx_user.length ? sctx_user.str :
(thd->slave_thread ? "SQL_SLAVE": ""),
"] @ ",
                                sctx_host.length ? sctx_host.str : "", " [",
                                sctx_ip.length ? sctx_ip.str : "", "]",
NullS) - user_host_buff);
```

解释如下：

- `root: m_priv_user` - The user privilege we are using. May be "" for anonymous user.
- `[root]: m_user` - user of the client, set to NULL until the user has been read from the connection.
- `localhost: m_host` - host of the client.
- `[]:client IP m_ip` - client IP.
- `Id: 10 thd->thread_id()` 实际上就是 `show processlist` 出来的 `id`。

6.3 第三部分 schema 等信息

Schema: test Last_errno: 1317 Killed: 0 对应的代码:

```
"# Schema: %s Last_errno: %u Killed: %u\n"
(thd->db().str ? thd->db().str : ""),
thd->last_errno, (uint) thd->killed,
```

□ Schema:

m_db Name of the current (default) database. If there is the current (default) database, "db" contains its name. If there is no current (default) database, "db" is NULL and "db_length" is 0. In other words, "db", "db_length" must either be NULL, or contain a valid database name.

□ Last_errno:

Variable last_errno contains the last error/warning acquired during query execution.

□ Killed: 这里代表的是终止的错误码。源码中如下:

```
enum killedstate
{
    NOTKILLED=0,
    KILLBADDATA=1,
    KILLCONNECTION=ERSERVERSHUTDOWN,
    KILLQUERY=ERQUERYINTERRUPTED,
    KILLTIMEOUT=ERQUERYTIMEOUT,
    KILLEDNOVALUE /* means neither of the states */
};
```

在错误码中代表如下:

```
{ "ERSERVERSHUTDOWN", 1053, "Server shutdown in progress" },
{ "ERQUERYINTERRUPTED", 1317, "Query execution was interrupted" },
{ "ERQUERYTIMEOUT", 1886, "Query execution was interrupted,
maxstatement_time exceeded" },
```

6.4 第四部分执行信息

这部分可能是大家最关心的部分，很多信息也是默认输出都会输出的。

```
# Query_time: 19.254508 Lock_time: 0.001043 Rows_sent: 0 Rows_examined: 0 Rows
_affected: 0
# Bytes_sent: 44 Tmp_tables: 0 Tmp_disk_tables: 0 Tmp_table_sizes: 0
# InnoDB_trx_id: 0
```

对应代码:

```
my_b_printf(&log_file,
"# Schema: %s Last_errno: %u Killed: %u\n"
```

```
"# Query_time: %s Lock_time: %s Rows_sent: %llu"
" Rows_examined: %llu Rows_affected: %llu\n"
"# Bytes_sent: %lu",
(thd->db().str ? thd->db().str : ""),
    thd->last_errno, (uint) thd->killed,
    query_time_buff, lock_time_buff,
(ulonglong) thd->get_sent_row_count(),
(ulonglong) thd->get_examined_row_count(),
(thd->get_row_count_func() > 0)
? (ulonglong) thd->get_row_count_func() : 0,
(ulong) (thd->status_var.bytes_sent - thd->bytes_sent_old)
my_b_printf(&log_file,
" Tmp_tables: %lu Tmp_disk_tables: %lu "
"Tmp_table_sizes: %llu",
    thd->tmp_tables_used, thd->tmp_tables_disk_used,
    thd->tmp_tables_size)
snprintf(buf, 20, "%llX", thd->innodb_trx_id);及 thd->innodb_trx_id
```

Query_time: 语句执行的时间及实际消耗时间。

Lock_time: 包含 MDL lock 和 InnoDB row lock 和 MyISAM 表锁消耗时间的总和及锁等待消耗时间。前面已经进行了描述（实际上也并不是全是锁等待的时间只是锁等待包含在其中）。

我们来看看 Query_time 和 Lock_time 的源码来源，它们来自于 Query_logger::slow_log_write 函数如下：

```
query_utime= (current_utime > thd->start_utime) ?
(current_utime - thd->start_utime) : 0;
lock_utime= (thd->utime_after_lock > thd->start_utime) ?
(thd->utime_after_lock - thd->start_utime) : 0;
```

下面是数据 current_utime 的来源，

```
current_utime= thd->current_utime();
实际上就是：
while(gettimeofday(&t, NULL) != 0)
{
    newtime= (ulonglong)t.tv_sec * 1000000+ t.tv_usec;
return newtime;
获取当前时间而已
```

对于 thd->utime_after_lock 的获取我已经在前文进行了描述，不再解释。

- ☐ Rows_sent: 发送给 mysql 客户端的行数，下面是源码中的解释 Number of rows we actually sent to the client
- ☐ Rows_examined: InnoDB 引擎层扫描的行数，下面是源码中的解释。（备注栈帧 1）
- ☐ Number of rows read and/or evaluated for a statement. Used for slow log reporting. An e

xamined row is defined as a row that is read and/or evaluated according to a statement condition, including `increase_sort_index()`. Rows may be counted more than once, e.g., a statement including `ORDER BY` could possibly evaluate the row in `filesort()` before reading it for e.g. `update`.

- ☐ `Rows_affected`: 涉及到修改的话（比如 DML 语句）这是受影响的行数。
- ☐ for DML statements: to the number of affected rows;
- ☐ for DDL statements: to 0.
- ☐ `Bytes_sent`: 发送给客户端的实际数据的字节数，它来自于 `(ulong) (thd->status_var.bytes_sent - thd->bytes_sent_old)`
- ☐ `Tmp_tables`: 临时表的个数。
- ☐ `Tmp_disk_tables`: 磁盘临时表的个数。
- ☐ `Tmp_table_sizes`: 临时表的大小。

以上三个指标来自于：

```
thd->tmp_tables_used
thd->tmp_tables_disk_used
thd->tmp_tables_size
```

这三个指标增加的位置对应应在 `free_tmp_table` 函数中如下：

```
thd->tmp_tables_used++;
if(entry->file)
{
    thd->tmp_tables_size += entry->file->stats.data_file_length;
    if(entry->file->ht->db_type != DB_TYPE_HEAP)
        thd->tmp_tables_disk_used++;
}
```

- ☐ `InnoDB_trx_id`: 事物 ID，也就是 `trx->id`, `/*!< transaction id */`

6.5 第五部分优化器相关信息

```
# QC_Hit: No Full_scan: No Full_join: No Tmp_table: No Tmp_table_on_disk: No
# Filesort: No Filesort_on_disk: No Merge_passes: 0
```

这一行来自于如下代码：

```
my_b_printf(&log_file,
"# QC_Hit: %s Full_scan: %s Full_join: %s Tmp_table: %s "
"Tmp_table_on_disk: %s\n"
"# Filesort: %s Filesort_on_disk: %s Merge_passes: %lu\n",
((thd->query_plan_flags & QPLAN_QC) ? "Yes": "No"),
((thd->query_plan_flags & QPLAN_FULL_SCAN) ? "Yes": "No"),
((thd->query_plan_flags & QPLAN_FULL_JOIN) ? "Yes": "No"),
((thd->query_plan_flags & QPLAN_TMP_TABLE) ? "Yes": "No"),
```

```
((thd->query_plan_flags & QPLAN_TMP_DISK) ? "Yes": "No"),
((thd->query_plan_flags & QPLAN_FILESORT) ? "Yes": "No"),
((thd->query_plan_flags & QPLAN_FILESORT_DISK) ? "Yes": "No"),
```

这里注意一个处理的技巧，这里 `query_plan_flags` 中每一位都代表一个含义，这样存储既能存储足够的信息同时存储空间也很小，是 C/C++ 中常用的方式。

- ☐ `QC_Hit`: No: 是否 query cache 命中。
- ☐ `Full_scan`: 此处相当于 `Select_scan` 的含义，是否进行了全扫描包括 using index。
- ☐ `Full_join`: 此处相当于 `Select_full_join` 的含义，是否被驱动表使用到了索引，如果没有使用到索引则为 YES。

考虑如下的执行计划：

```
mysql> desc select*,sleep(1) from testuin a,testuin1 b where a.id1=b.id1;
+----+-----+-----+-----+-----+-----+-----+-----+-----+
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | r
ef | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE      | a      | NULL        | ALL  | NULL          | NULL | NULL    | NULL |
5 | 100.00 | NULL          |           |      |               |      |         |      |
| 1 | SIMPLE      | b      | NULL        | ALL  | NULL          | NULL | NULL    | NULL |
5 | 20.00 | Usingwhere; Using join buffer (BlockNestedLoop) |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
+----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows inset, 1 warning (0.00 sec)
```

如此输出如下：

```
# QC_Hit: No Full_scan: Yes Full_join: Yes
```

- ☐ `Tmp_table`: 是否使用了临时表，在函数 `create_tmp_table` 中设置。
- ☐ `Tmp_table_on_disk`: 是否使用了磁盘临时表，如果时候 innodb 引擎则在 `create_innodb_tmp_table` 函数中设置。
- ☐ `Filesort`: 是否进行了排序，在函数 `filesort` 中设置。
- ☐ `Filesort_on_disk`: 是否使用了磁盘排序，同样在函数 `filesort` 中设置，但是设置之前会进行是否需要磁盘排序文件的判断。
- ☐ `Merge_passes`: 进行多路归并排序，归并的次数。Variable `query_plan_fsort_passes` collects information about file sort passes acquired during query execution.

6.6 第六部分 InnoDB 相关信息

```
# InnoDB_IO_r_ops: 0 InnoDB_IO_r_bytes: 0 InnoDB_IO_r_wait: 0.000000
# InnoDB_rec_lock_wait: 0.000000 InnoDB_queue_wait: 0.000000
```

```
# InnoDB_pages_distinct: 0
```

这一行来自于如下代码:

```
char buf[3][20];
    snprintf(buf[0], 20, "%.6f", thd->innodb_io_reads_wait_timer / 1000000.0);
    snprintf(buf[1], 20, "%.6f", thd->innodb_lock_que_wait_timer / 1000000.0);
    snprintf(buf[2], 20, "%.6f", thd->innodb_innodb_que_wait_timer / 1000000.0);
if(my_b_printf(&log_file,
"# InnoDB_IO_r_ops: %lu InnoDB_IO_r_bytes: %llu "
"InnoDB_IO_r_wait: %s\n"
"# InnoDB_rec_lock_wait: %s InnoDB_queue_wait: %s\n"
"# InnoDB_pages_distinct: %lu\n",
                thd->innodb_io_reads, thd->innodb_io_read,
                buf[0], buf[1], buf[2], thd->innodb_page_access)
== (uint) -1)
```

- ☐ InnoDB_IO_r_ops: 物理 IO 读取次数。
- ☐ InnoDB_IO_r_bytes: 物理 IO 读取的总字节数。
- ☐ InnoDB_IO_r_wait: 物理 IO 读取等待的时间。innodb 使用 BUF_IO_READ 标记为物理 io 读取繁忙, 参考函数 buf_wait_for_read。
- ☐ InnoDB_rec_lock_wait: 等待行锁消耗的时间。在函数 que_thr_end_lock_wait 中设置。
- ☐ InnoDB_queue_wait: 等待进入 innodb 引擎消耗的时间, 在函数 srv_conc_enter_innodb_with_atomics 中设置。
- (参考 <http://blog.itpub.net/7728585/viewspace-2140446/>)
- ☐ InnoDB_pages_distinct: innodb 访问的页数, 包含物理和逻辑 IO, 在函数 buf_page_get_gen 的末尾通过 increment_page_get_statistics 函数设置。

6.7 第七部分 set timestamp

```
SET timestamp=1527753496;
```

这一句来自源码, 注意源码注释解释就是获取的服务器的当前的时间 (current_ftime)。

```
/*
    This info used to show up randomly, depending on whether the query
    checked the query start time or not. now we always write current
    timestamp to the slow log
*/
end= my_stpcpy(end, ",timestamp=");
end= int10_to_str((long) (current_ftime / 1000000), end, 10);
if(end!= buff)
{
    *end++=';';
    *end='\n';
if(my_b_write(&log_file, (uchar*) "SET ", 4) ||
    my_b_write(&log_file, (uchar*) buff + 1, (uint) (end-buff)))
```

```
goto err;
}
```

7 总结

本文通过查询源码解释了一些关于 MySQL 慢查询的相关知识，主要解释了慢查询是基于什么标准进行记录的，同时输出中各个指标的含义，当然这仅仅是我自己得出的结果，如果有不同意见可以一起讨论。

备注栈帧 1：本栈帧主要跟踪 Rows_examined 的变化及 join->examined_rows++ 的变化

```
(gdb) info b
NumTypeDispEnbAddressWhat
1 breakpoint keep y 0x000000000ebd5f3in main(int, char**) at /root/mysql5.7.14/percona-server-5.7.14-7/sql/main.cc:25
    breakpoint already hit 1 time
4 breakpoint keep y 0x000000000155b94fin do_select(JOIN*) at /root/mysql5.7.14/percona-server-5.7.14-7/sql/sql_executor.cc:872
    breakpoint already hit 5 times
5 breakpoint keep y 0x000000000155ca39in evaluate_join_record(JOIN*, QEP_TAB*) at /root/mysql5.7.14/percona-server-5.7.14-7/sql/sql_executor.cc:1473
    breakpoint already hit 20 times
6 breakpoint keep y 0x00000000019b4313in ha_innbase::index_first(uchar*)
    at /root/mysql5.7.14/percona-server-5.7.14-7/storage/innobase/handler/ha_innodb.cc:9547
    breakpoint already hit 4 times
7 breakpoint keep y 0x00000000019b45cdin ha_innbase::rnd_next(uchar*)
    at /root/mysql5.7.14/percona-server-5.7.14-7/storage/innobase/handler/ha_innodb.cc:9651
8 breakpoint keep y 0x00000000019b2ba6in ha_innbase::index_read(uchar*, uchar const*, uint, ha_rkey_function)
    at /root/mysql5.7.14/percona-server-5.7.14-7/storage/innobase/handler/ha_innodb.cc:9004
    breakpoint already hit 3 times
9 breakpoint keep y 0x00000000019b4233in ha_innbase::index_next(uchar*)
    at /root/mysql5.7.14/percona-server-5.7.14-7/storage/innobase/handler/ha_innodb.cc:9501
    breakpoint already hit 5 times
#0 ha_innbase::index_next (this=0x7fff2cbc6b40, buf=0x7fff2cbc7080 "\375\n") at /root/mysql5.7.14/percona-server-5.7.14-7/storage/innobase/handler/ha_innodb.cc:9501
#1 0x000000000f680d8 in handler::ha_index_next (this=0x7fff2cbc6b40, buf=0x7fff2cbc7080 "\375\n") at /root/mysql5.7.14/percona-server-5.7.14-7/sql/handler.cc:3269
#2 0x000000000155fa02 in join_read_next (info=0x7fff2c007750) at /root/mysql5.7.14/percona-server-5.7.14-7/sql/sql_executor.cc:2660
387
```



```
#3 0x000000000155c397 in sub_select (join=0x7fff2c007020, qep_tab=0x7fff2c007700,  
  end_of_records=false)  
  at /root/mysql5.7.14/percona-server-5.7.14-7/sql/sql_executor.cc:1274  
#4 0x000000000155bd06 in do_select (join=0x7fff2c007020) at /root/mysql5.7.14/percona-server-5.7.14-7/sql/sql_executor.cc:944
```



MySQL 的 join_buffer_size 在内连接上的应用

作者：杨涛涛

本文详细介绍了 MySQL 参数 join_buffer_size 在 INNER JOIN 场景的使用，OUTER JOIN 不包含。在讨论这个 BUFFER 之前，我们先了解下 MySQL 的 INNER JOIN 分类。

如果按照检索的性能方式来细分，那么无论是两表 INNER JOIN 还是多表 INNER JOIN，都大致可以分为以下几类：

1. JOIN KEY 有索引，主键
2. JOIN KEY 有索引，二级索引
3. JOIN KEY 无索引

今天主要针对第三种场景来分析，也就是就全表扫的场景。

回过头来看看什么是 join_buffer_size?

JOIN BUFFER 是 MySQL 用来缓存以上第二、第三这两类 JOIN 检索的一个 BUFFER 内存区域块。一般建议设置一个很小的 GLOBAL 值，完了在 SESSION 或者 QUERY 的基础上来做一个合适的调整。

比如，默认的值 512K，想要临时调整为 1G。那么，

```
mysql>set session join_buffer_size = 1024 * 1024 * 1024;
mysql>select * from ...;
mysql>set session join_buffer_size=default;
或者
mysql>select /* set_var(join_buffer_size=1G) */ * from ...;
```

接下来详细看下 JOIN BUFFER 的用法。

那么 MySQL 里针对 INNER JOIN 大致有以下几种算法，

1 Nested-Loop Join 翻译过来就是嵌套循环连接，简称 NLJ。

这种是 MySQL 里最简单、最容易理解的表关联算法。

比如，拿语句 `select * from p1 join p2 using(r1)` 来说，先从表 p1 里拿出来一条记录 ROW1，完了再用 ROW1 遍历表 p2 里的每一条记录，并且字段 r1 来做匹配是否相同，以便输出；再次循环刚才的过程，直到两表的记录数对比完成为止。

那看下实际 SQL 的执行计划，

```
mysql> explain format=json select * from p1 inner join p2 as b using(r1)\G
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "1003179606.87"
    },
    "nested_loop": [
      {
        "table": {
          "table_name": "b",
          "access_type": "ALL",
          "rows_examined_per_scan": 1000,
          "rows_produced_per_join": 1000,
          "filtered": "100.00",
          "cost_info": {
            "read_cost": "1.00",
            "eval_cost": "100.00",
            "prefix_cost": "101.00",
            "data_read_per_join": "15K"
          },
          "used_columns": [
            "id",
            "r1",
            "r2"
          ]
        },
        "table": {
          "table_name": "p1",
          "access_type": "ALL",
          "rows_examined_per_scan": 9979810,
          "rows_produced_per_join": 997981014,
          "filtered": "10.00",
          "cost_info": {
            "read_cost": "5198505.87",
            "eval_cost": "99798101.49",
            "prefix_cost": "1003179606.87",
            "data_read_per_join": "14G"
          },
          "used_columns": [
            "id",
            "r1",
            "r2"
          ]
        }
      }
    ]
  }
}
```

```

    ],
    "attached_condition": "(`ytt_new`.`p1`.`r1` = `ytt_new`.`b`.`r1`)"
  }
}
]
}
}
1 row in set, 1 warning (0.00 sec)

```

从上面的执行计划来看，表 p2 为第一张表（驱动表或者叫外表），第二张表为 p1，那 p2 需要遍历的记录数为 1000，同时 p1 需要遍历的记录数大概 1000W 条，那这条 SQL 要执行完成，就得对表 p1（内表）匹配 1000 次，对应的 read_cost 为 5198505.87。那如何才能减少表 p1 的匹配次数呢？那这个时候 JOIN BUFFER 就派上用场了

2 Block Nested-Loop Join ，块嵌套循环，简称 BNLJ

那 BNLJ 比 NLJ 来说，中间多了一块 BUFFER 来缓存外表的对应记录从而减少了外表的循环次数，也就减少了内表的匹配次数。还是那上面的例子来说，假设 join_buffer_size 刚好能容纳外表的对应 JOIN KEY 记录，那对表 p2 匹配次数就由 1000 次减少到 1 次，性能直接提升了 1000 倍。

我们看下用到 BNLJ 的执行计划，

```

mysql> explain format=json select * from p1 inner join p2 as b using(r1)\G
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "997986300.01"
    },
    "nested_loop": [
      {
        "table": {
          "table_name": "b",
          "access_type": "ALL",
          "rows_examined_per_scan": 1000,
          "rows_produced_per_join": 1000,
          "filtered": "100.00",
          "cost_info": {
            "read_cost": "1.00",
            "eval_cost": "100.00",
            "prefix_cost": "101.00",
            "data_read_per_join": "15K"
          },
          "used_columns": [

```

```

        "id",
        "r1",
        "r2"
    ]
}
},
{
    "table": {
        "table_name": "p1",
        "access_type": "ALL",
        "rows_examined_per_scan": 9979810,
        "rows_produced_per_join": 997981014,
        "filtered": "10.00",
        "using_join_buffer": "Block Nested Loop",
        "cost_info": {
            "read_cost": "5199.01",
            "eval_cost": "99798101.49",
            "prefix_cost": "997986300.01",
            "data_read_per_join": "14G"
        },
        "used_columns": [
            "id",
            "r1",
            "r2"
        ],
        "attached_condition": "(`ytt_new`.`p1`.`r1` = `ytt_new`.`b`.`r1`)"
    }
}
]
}
}
1 row in set, 1 warning (0.00 sec)

```

上面的执行计划有两点信息

第一：多了一条 "using_join_buffer": "Block Nested Loop"

第二：read_cost 这块由之前的 5198505.87 减少到 5199.01

3 最近 MySQL 8.0.18 发布，终于推出了新的 JOIN 算法 — HASH JOIN。

MySQL 的 HASH JOIN 也是用了 JOIN BUFFER 来做缓存，但是和 BNLJ 不同的是，它在 JOIN BUFFER 中以外表为基础建立一张哈希表，内表通过哈希算法来跟哈希表进行匹配，hash join 也就是进一步减少内表的匹配次数。当然官方并没有说明详细的算法描述，以上仅代表个人臆想。那还是针对以上的 SQL，我们来看下执行计划。

```
mysql> explain format=tree select * from p1 inner join p2 as b using(r1)\G
***** 1. row *****
EXPLAIN: -> Inner hash join (p1.r1 = b.r1) (cost=997986300.01 rows=997981015)
    -> Table scan on p1 (cost=105.00 rows=9979810)
        -> Hash
            -> Table scan on b (cost=101.00 rows=1000)
1 row in set (0.00 sec)
```

通过上面的执行计划看到，针对表 p2 建立一张哈希表，然后针对表 p1 来做哈希匹配。

目前仅仅支持简单查看是否用了 HASH JOIN，而没有其他更多的信息展示。

总结下，本文主要讨论 MySQL 的内表关联在没有任何索引的低效场景。其他的场景另外开篇。



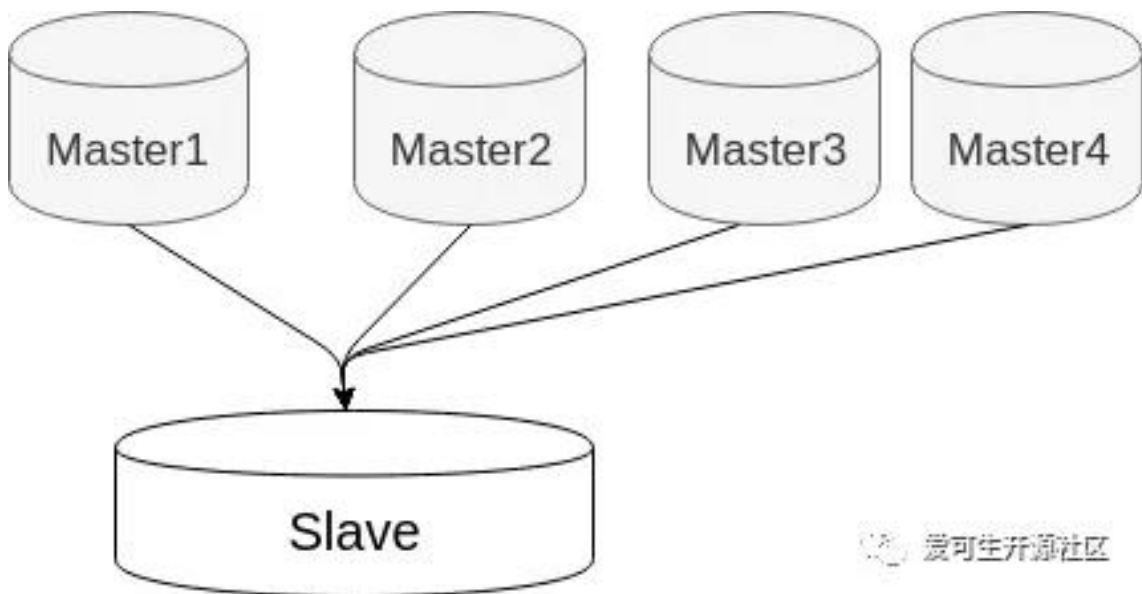
MySQL 多源复制场景分析

作者：杨涛涛

今天有客户问起：如何汇总多台 MySQL 数据到一台上？

我回答：可以尝试下 MySQL 的多源复制。

我们知道 MySQL 单主一从，单主多从，或者级联的主从架构我们都见的很多了。但是多主一从这种使用场景比较少，比如图 1：

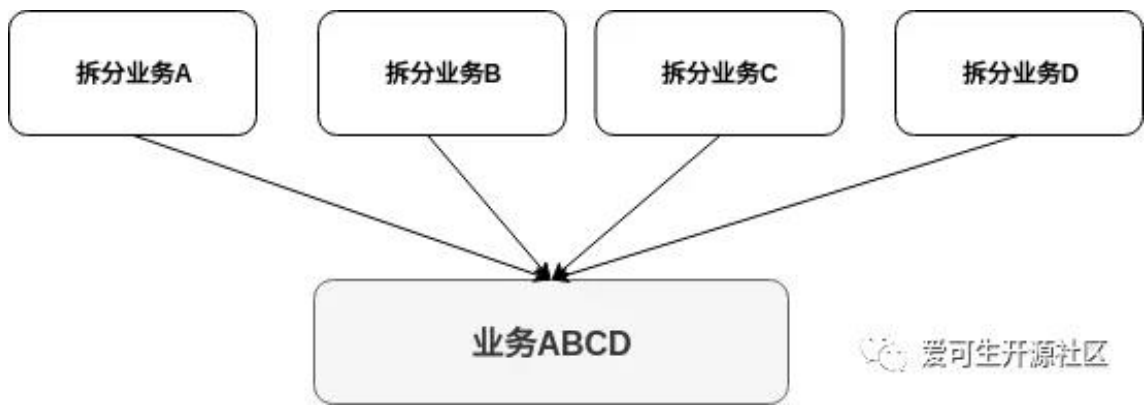


这种架构一般用在以下三类场景

1. 备份多台 Server 的数据到一台

如果按照数据切分方向来讲，那就是垂直切分。比如图 2，业务 A、B、C、D 是之前拆分好的业务，现在需要把这些拆分好的业务汇总起来备份，那这种需求也很适用于多源复制架构。

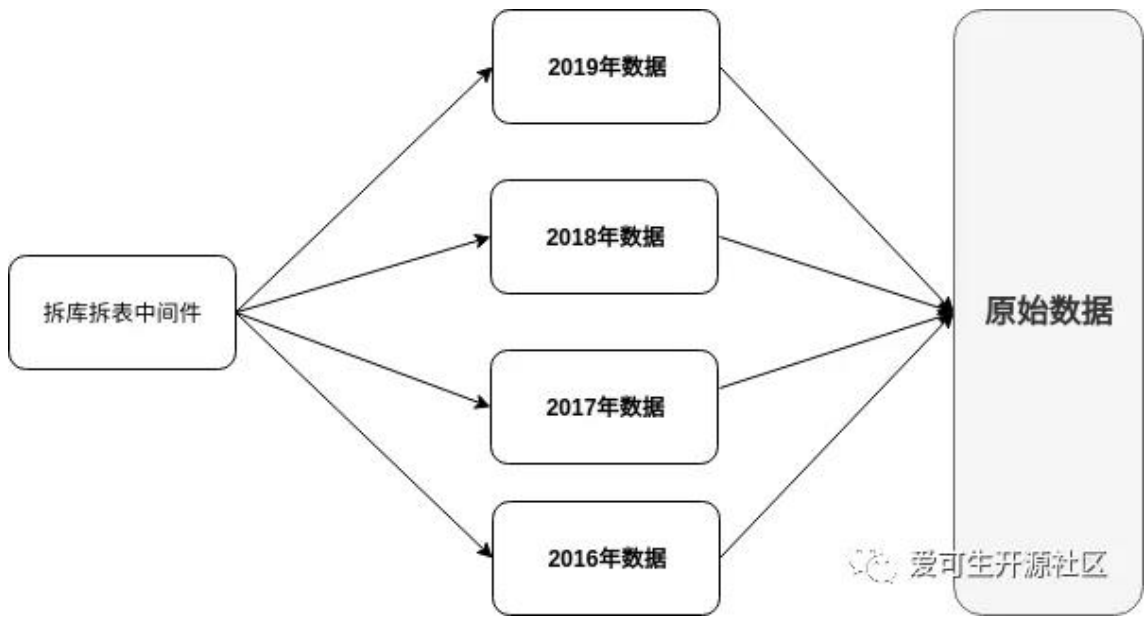
实现方法我大概描述下：业务 A、B、C、D 分别位于 4 台 Server，每台 Server 分别有一个数据库来隔离前端的业务数据，那这样，在从库就能把四台业务的数据全部汇总起来，而不需要做额外的操作。那没有多源复制之前，要实现这类需求，只能在汇总机器上搭建多个 MySQL 实例，那这样势必会涉及到跨库关联的问题，不但性能急剧下降，管理多个实例也没有单台来的容易。



2. 用来聚合前端多个 Server 的分片数据。

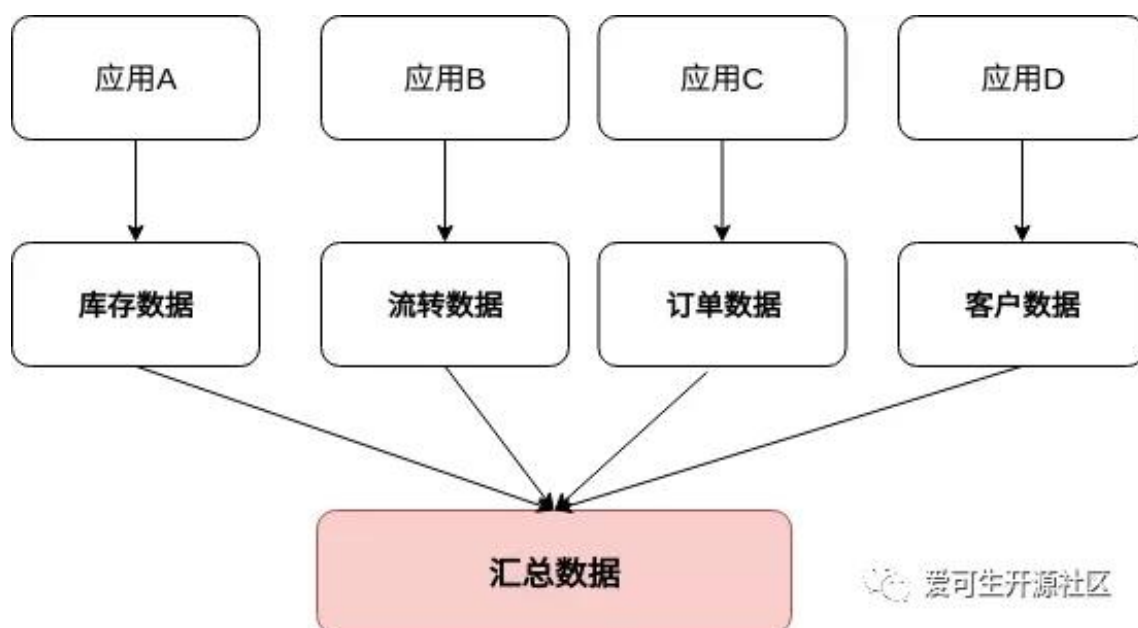
同样，按照数据切分方向来讲，属于水平切分。比如图 3，按照年份拆分好的数据，要做一个汇总数据展现，那这种架构也非常合适。

实现方法稍微复杂些：比如所有 Server 共享同一数据库和表，一般为了开发极端透明，前端配置有分库分表的中间件，比如爱可生的 DBLE。



3. 汇总并合并多个 Server 的数据

第三类和第一种场景类似。不一样的是不仅仅是数据需要汇总到目标端，还得合并这些数据，这就比第一种来的相对复杂些。比如图 4，那这样的需求，是不是也适合多源复制呢？答案是 YES。



那具体怎么做呢？

我举个例子，比如下面一张表 A，字段分表为 ID（主键）、F1、F2、F3...、F100。那按照这样的分法，前端 4 台 Server 的表分别为：

- ☐ A1 (ID, F1, F2, ..., F25)
- ☐ A2 (ID, F26, F27, ..., F50)
- ☐ A3 (ID, F51, F52, ..., F75)
- ☐ A4 (ID, F76, F77, ..., F100)

那上面几张表的数据如果要合并到表 A，可以建立一个 Event，定时的来给表 A 里插入数据。涉及到的核心 SQL 为：

```
insert ignore into A select A1.ID,F1,F2,...,F100 \
from A1 natural join A2 natural join A3 natural join A4;
```

那我们发现这个和第一个类似，只不过，所有的表最后到复制到了相同的数据库里。

总结下，我上面简单说明了 MySQL 多源复制的三种常用使用场景，希望对大家有所帮助。



mysql show processlist Time 为负数的思考

作者：高鹏（八怪）

1 问题来源

这是一个朋友问我的一个问题，问题如下，在 MTS 中 Worker 线程看到 Time 为负数是怎么回事，如下：

```
mysql> show processlist;
```

Id	User	Host	db	Command	Time	State	Info
16191183	system user		NULL	Connect	45712	Waiting for master to send event	NULL
16191184	system user		NULL	Connect	7	Slave has read all relay log; waiting for more updates	NULL
16191185	system user		NULL	Connect	-27	Waiting for an event from Coordinator	NULL
16191186	system user		NULL	Connect	-24	Waiting for an event from Coordinator	NULL
16191187	system user		NULL	Connect	-24	Waiting for an event from Coordinator	NULL
16191188	system user		NULL	Connect	141	Waiting for an event from Coordinator	NULL
16191189	system user		NULL	Connect	441	Waiting for an event from Coordinator	NULL
16191190	system user		NULL	Connect	10341	Waiting for an event from Coordinator	NULL
16191191	system user		NULL	Connect	10941	Waiting for an event from Coordinator	NULL
16191192	system user		NULL	Connect	28941	Waiting for an event from Coordinator	NULL
16217149	root	localhost	NULL	Query	0	starting	show processlist

2 关于 show processlist 中的 Time

实际上 show processlist 中的信息基本都来自函数 `mysqld_list_processes`，也就是说每次执行 show processlist 都需要执行这个函数来进行填充。对于 Time 值来讲它来自如下信息：

Percona:

```
time_t now= my_time(0);
protocol->store_long ((thd_info->start_time > now) ? 0 : (longlong) (now - thd_info->start_time));
```

官方版:

```
time_t now= my_time(0);
protocol->store_long ((longlong) (now - thd_info->start_time));
```

我们可以注意到在 Percona 的版本中对这个输出值做了优化，也就是如果出现负数的时候直接显示为 0，但是官方版中没有这样做，可能出现负数。

3 计算方式解读和测试

现在我们来看看这个简单的计算公式，实际上 now 很好理解就是服务器的当前时间，重点就在于 `thd_info->start_time` 的取值来自何处。实际这个时间来自于函数 `THD::set_time`，但是需要注意的是这个函数会进行重载，有下面三种方式：

重载 1

```
inline void set_time()
{
    start_ftime= ftime_after_lock= my_micro_time();
    if(user_time.tv_sec || user_time.tv_usec)
    {
        start_time= user_time;
    }
    else
        my_micro_time_to_timeval(start_ftime, &start_time);
    ...
}
```

重载 2

```
inline void set_time(const struct timeval *t)
{
    start_time= user_time= *t;
    start_ftime= ftime_after_lock= my_micro_time();
    ...
}
```

重载 3

```
void set_time(QUERY_START_TIME_INFO *time_info)
{
    start_time= time_info->start_time;
    start_ftime= time_info->start_ftime;
}
```

其实简单的说就是其中有一个 start_ftime，如果设置了 start_ftime 那么 start_time 将会指定为 start_ftime，并且在重载 1 中将会不会修改 start_time，这一点比较重要。

好了说了 3 种方式，我们来看看 Time 的计算有如下可能。

3.1 执行命令

如果主库执行常见的命令都会在命令发起的时候调用重载 1，设置 start_time 为命令开始执行的时间如下：

堆栈:

```
#0  THD::set_time (this=0x7ffe7c000c90) at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_class.h:3505
#1  0x0000000015c5fe8 in dispatch_command (thd=0x7ffe7c000c90, com_data=0x7ffffec03fd70, command=COM_QUERY)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_parse.cc:1247
```

可以看到这个函数没有实参, 因此 `start_time` 会设置为当前时间, 那 `Time` 的计算公式 `now - thdinfo->start_time` 就等于 (服务器当前时间 - 命令开始执行的时间)。

3.2 从库单 Sql 线程和 Worker 线程

其实不管单 Sql 线程还是 Worker 线程都是执行 Event 的, 这里的 `start_time` 将会被设置为 Event header 中 `timestamp` 的时间 (query event/dml event), 这个时间实际就是主库命令发起的时间。如下:

堆栈:

query event:

```
#0  THD::set_time (this=0x7ffe78000950, t=0x7ffe701ec430) at /root/mysqlall/percona-server-locks-detail-5.7.22/sql/sql_class.h:3526
#1  0x00000000018459ab in Query_log_event::do_apply_event (this=0x7ffe701ec310, rli=0x7ffe7003c050, query_arg=0x7ffe701d88a9 "BEGIN", q_len_arg=5)
    at /root/mysqlall/percona-server-locks-detail-5.7.22/sql/log_event.cc:4714
```

堆栈:

dml event:

```
#0  THD::set_time (this=0x7ffe78000950, t=0x7ffe701ed5b8) at /root/mysqlall/percona-server-locks-detail-5.7.22/sql/sql_class.h:3526
#1  0x000000000185aa6e in Rows_log_event::do_apply_event (this=0x7ffe701ed330, rli=0x7ffe7003c050)
    at /root/mysqlall/percona-server-locks-detail-5.7.22/sql/log_event.cc:11417
```

我们看到这里有一个实参的传入我们看一下代码如下:

```
thd->set_time(&(common_header->when))
```

实际上就是这一行, 这是我们前面说的重载 3, 这样设置后 `start_utime` 和 `start_time` 都将会设置, 即便调用重载 1 也不会更改, 那 `Time` 的计算方式 `now - thd_info->start_time` 就等于 (从库服务器当前时间 - Event header 中的时间), 但是要知道 Event header 中的时间实际也是来自于主库命令发起的时间。既然如此如果从库服务器的时间小于主库服务器的时间, 那么 `Time` 的结果可能是负数是可能出现的, 当然 Percona 版本做了优化负数将会显示为 0, 如果从库服务器的时间大于主库的时间那么可能看到 `Time` 比较大。

因为我的测试环境都是 Percona, 为了效果明显, 测试一下 Worker 线程 `Time` 很大的情况, 如下设

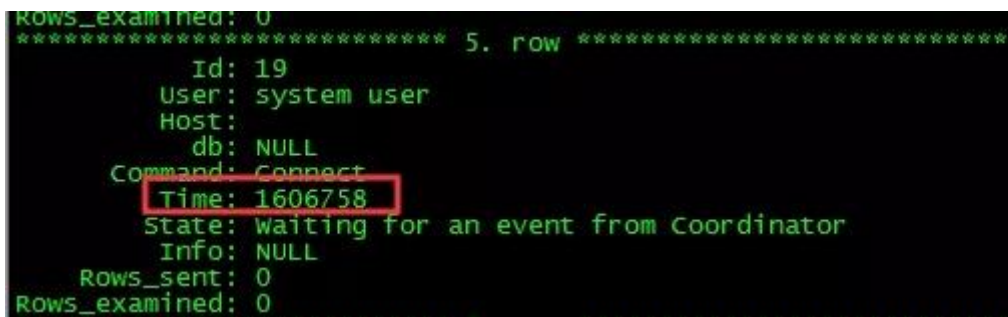
置：
主库：

```
[root@mysqltest2 test]# date
FriNov101:40:54 CST 2019
```

从库：

```
[root@gp1 log]# date
TueNov1915:58:37 CST 2019
```

主库随便做一个命令，然后观察如下：



```
ROWS_examined: 0
***** 5. row *****
      Id: 19
      User: system user
      Host:
      db: NULL
      Command: Connect
      Time: 1606758
      State: Waiting for an event from Coordinator
      Info: NULL
      Rows_sent: 0
      Rows_examined: 0
```

3.3 设置 timestamp

如果手动指定 timestamp 也会影响到 Time 的计算结果，因为 start_utime 和 start_time 都将会设置，如下：

```
mysql> set timestamp=1572540000
```

堆栈：

```
#0  THD::set_time (this=0x7ffe7c000c90, t=0x7fffec03db30) at /mysqldata/percona-server-locks-detail-5.7.22/sql/sql_class.h:3526
#1  0x000000000169e509 in update_timestamp (thd=0x7ffe7c000c90, var=0x7ffe7c006860) at /mysqldata/percona-server-locks-detail-5.7.22/sql/sys_vars.cc:4966
#2  0x00000000016b9a3d in Sys_var_session_special_double::session_update (this=0x2e68e20, thd=0x7ffe7c000c90, var=0x7ffe7c006860)
    at /mysqldata/percona-server-locks-detail-5.7.22/sql/sys_vars.h:1889
```

我们看到带入了实参，我们看看代码这一行如下：

```
thd->set_time(&tmp);
```

这就是重载 2 了，这样设置后 start_utime 和 start_time 都将会设置，即便调用重载 1 也不会更改，言外之意就是设置了 timestamp 后即便执行了其他的命令 Time 也不会更新。Time 的计算方式 $\text{now} - \text{thdinfo->starttime}$ 就等于（服务器当前时间 - 设置的 timestamp 时间），这样的话就可能出现 Time 出现异常，比如很大或者为负数（Percona 为 0）如下：

```

Rows_examined: 0
***** 2. row *****
      Id: 3
      User: root
      Host: localhost
      db: test
      Command: Sleep
      Time: 1000003308
      State:
      Info: NULL
      Rows_sent: 0
      Rows_examined: 0

```

3.4 空闲情况下

如果我们的会话空闲状态下那么 `now-thd_info->start_time` 公式中, `now` 会不断变大, 但是 `thd_info->start_time` 却不会改变, 因此 `Time` 会不断增大, 等待到下一次命令到来后才会更改。

4 延伸

这里我想在说明一下如果从库开启了 `log_slave_updates` 的情况下, 从库记录会记录来自主库的 Event, 但是这些 Event 的 timestamp 和 Query Event 的 exetime 如何取值呢?

```

#191101 1:47:29 server id 413340 end_log_pos 2129 CRC32 0x8b4e8f82 GTID last_comm
/*!50718 SET TRANSACTION ISOLATION LEVEL READ COMMITTED/*!*/;
SET @@SESSION.GTID_NEXT= 'cb7ea36e-670f-11e9-b483-5254008138e4:9'/*!*/;
# at 2129
#191101 1:47:29 server id 413340 end_log_pos 2192 CRC32 0xd6b5b81b Query thread_id
SET TIMESTAMP=1572544049/*!*/;
BEGIN
/*!*/;
# at 2192
#191101 1:47:29 server id 413340 end_log_pos 2242 CRC32 0xd92b9b34 Table_map: `test`
# at 2242
#191101 1:47:29 server id 413340 end_log_pos 2282 CRC32 0x0b71c6f4 write_rows: table
BINLOG '
MR67XR0cTgYAMgAAAMIIAAAAAGWAAAAAAAEABHRlc3QAB3Rlc3RkZWwAAQMAATSBk9k=
MR67XR6cTgYAKAAAAOoIAAAAAAGWAAAAAAAEAGAB//4KAAAA9MZxCw==
'/*!*/;
### INSERT INTO `test`.`testdel`
### SET
### @1=10 /* INT meta=0 nullable=1 is_null=0 */
# at 2282
#191101 1:47:29 server id 413340 end_log_pos 2313 CRC32 0xa422aa90 xid = 221
COMMIT/*!*/;
SET @@SESSION.GTID_NEXT= 'AUTOMATIC' /* added by mysqlbinlog *//*!*/;
DELIMITER ;
# End of log file
/*!50003 SET COMPLETION_TYPE=@OLD_COMPLETION_TYPE*/;
/*!50530 SET @@SESSION.PSEUDO_SLAVE_MODE=0*/;
[root@qpl log]# date
Tue Nov 19 16:14:29 CST 2019
[root@qpl log]#

```

Event 的 timestamp 的取值

其实上面我已经说了, 因为 `start_time` 将会被设置为 Event header 中 timestamp 的时间 (query event/dml event), 当记录 Event 的时候这个时间和主库基本一致, 如下:

很明显我们会发现这些 Event 的 timestamp 不是本地的时间, 而是主库的时间。

Query Event 的 exetime

我们先来看看这个时间的计算方式：

```
ulonglong micro_end_time= my_micro_time();//这里获取时间 query event
my_micro_time_to_timeval(micro_end_time, &end_time);

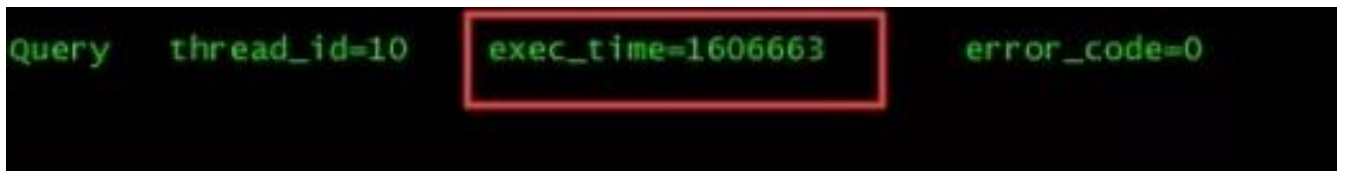
exec_time= end_time.tv_sec - thd_arg->start_time.tv_sec;//这里计算时间
```

相信对于 `thd_arg->start_time` 而言已经不再陌生，它就是主库命令发起的时间。我在我的《深入理解主从原理》系列中说过了，对于 Query Event 的 `exetime` 在 row 格式 binlog 下，DML 语句将会是第一行语句修改时间的时间，那么我们做如下定义（row 格式 DML 语句）：

- 主：主库第一行数据修改完成的服务器时间 - 主库本命令发起的时间
- 从：从库第一行数据修改完成的服务器时间 - 主库本命令发起的时间

他们的差值就是：（从库第一行数据修改完成的服务器时间 - 主库第一行数据修改完成的服务器时间）

同样如果我们从库的时间远远大于主库的时间，那么 `exetime` 也会出现异常如下：



5 最后

Time 是我们平时关注的一个指标，我们经常用它来表示我的语句执行了多久，但是如果出现异常的情况我们也应该能够明白为什么，这里我将它的计算方式做了一个不完全的解释，希望对大家有帮助。当然对于主从部分或者 Event 部分可以参考我的《深入理解主从原理》系列。



如何使用 dbdeployer 快速搭建 MySQL 测试环境

作者：余振兴

1 工具介绍

dbdeployer 是一款十分强大的数据库测试环境部署工具，可实现一键部署不同架构、不同版本数据库环境。

如：MySQL 主从复制、GTID 模式复制、MySQL 组复制（单主模式、多主模式等）完整的数据库类型支持及版本，可在安装完 dbdeployer 后使用 `dbdeployer admin capabilities` 命令进行查看，以下是当前已支持数据库及组件类型

- ☐ Oracle MySQL
- ☐ Percona MySQL
- ☐ MariaDB
- ☐ TiDB
- ☐ MySQL NDB Cluster
- ☐ Percona XtraDB Cluster
- ☐ mysql-shell

本文主要介绍基于 dbdeployer 工具快速搭建 MySQL 测试环境以及 dbdeployer 的基础使用。完整的功能特性以及使用方式可查看 dbdeployer 文档手册，dbdeployer 的文档手册十分详细的介绍了该工具特性及各类使用方式，可查看文末的相关链接。

2 工具安装

本文以 MacOS 下部署为例，Linux 平台安装部署方式基本类似（该工具不提供 Windows 版本），可访问 <https://github.com/datacharmer/dbdeployer/releases>

获取最新版 dbdeployer 下载链接，以下内容中 `shell>` 表示命令行提示符

下载当前最新版软件包

```
shell> wget https://github.com/datacharmer/dbdeployer/releases/download/v1.42.0/dbdeployer-1.42.0.osx.tar.gz
```

软件解压后实际只有一个单独的编译好的可执行文件

```
shell> tar xzvf dbdeployer-1.42.0.osx.tar.gz
```

将解压的软件拷贝至系统可执行目录下方便使用

```
shell> chmod +x dbdeployer-1.42.0.osx
```

```
shell> mv dbdeployer-1.42.0.osx /usr/local/bin/dbdeployer
```

验证是否安装成功


```
shell> dbdeployer --version
shell> dbdeployer -help
```

3 工具配置

该软件默认使用当前用户的 `$HOME/.dbdeployer/config.json` 作为配置文件。该文件默认未生成，可使用 `dbdeployer defaultsexportpath` 命令导出一份并进行修改。以下我只修改配置文件中的 `sandbox-binary` 参数，将该目录指定为自定义的 `$HOME/sandboxes/mysql_base`

```
## 创建 MySQL 软件包及解压后的软件目录
shell> mkdir -p ~/sandboxes/{mysql_package,mysql_base}

## 将默认配置中 sandbox-binary 参数修改为已创建的目录路径
## 该部分也可使用 dbdeployer defaults export
/Users/yuzhenxing/.dbdeployer/config.json 先将配置文件导出，再 vim 手工编辑修改
shell> dbdeployer defaults update sandbox-binary $HOME/sandboxes/mysql_base
## 查看已修改的配置信息
## 配置中包含各类 MySQL 的初始化信息，可根据实际情况灵活调整

shell> dbdeployer defaults show
# Configuration file: /Users/yuzhenxing/.dbdeployer/config.json
{
  "version": "1.39.0",
  "sandbox-home": "$HOME/sandboxes",
  "sandbox-binary": "$HOME/sandboxes/mysql_base",
  "use-sandbox-catalog": true,
  "log-sb-operations": false,
  "log-directory": "/Users/yuzhenxing/sandboxes/logs",
  "cookbook-directory": "recipes",
  "shell-path": "/bin/bash",
  "master-slave-base-port": 11000,
  "group-replication-base-port": 12000,
  "group-replication-sp-base-port": 13000,
  "fan-in-replication-base-port": 14000,
  "all-masters-replication-base-port": 15000,
  "multiple-base-port": 16000,
  "pxc-base-port": 18000,
  "ndb-base-port": 19000,
  "ndb-cluster-port": 20000,
  "group-port-delta": 125,
  "mysqlx-port-delta": 10000,
  "admin-port-delta": 11000,
  "master-name": "master",
  "master-abbr": "m",
  "node-prefix": "node",
  "slave-prefix": "slave",
```

```
"slave-abbr": "s",
"sandbox-prefix": "msb_",
"imported-sandbox-prefix": "imp_msb_",
"master-slave-prefix": "rsandbox_",
"group-prefix": "group_msb_",
"group-sp-prefix": "group_sp_msb_",
"multiple-prefix": "multi_msb_",
"fan-in-prefix": "fan_in_msb_",
"all-masters-prefix": "all_masters_msb_",
"reserved-ports": [
1186,
3306,
33060,
33062
],
"remote-repository": "https://raw.githubusercontent.com/datacharmer/mysql-
docker-minimal/master/dbdata",
"remote-index-file": "available.json",
"remote-completion-url":
"https://raw.githubusercontent.com/datacharmer/dbdeployer/master/docs/dbdeployer
_completion.sh",
"remote-tarball-url":
"https://raw.githubusercontent.com/datacharmer/dbdeployer/master/downloads/tarba
ll_list.json",
"pxc-prefix": "pxc_msb_",
"ndb-prefix": "ndb_msb_",
"timestamp": "Thu Nov 21 15:58:56 CST 2019"
}
```

4 基本使用

在使用之前我们需要先下载好对应操作系统的 MySQL 软件包，dbdeployer 软件提供了下载 MySQL 软件的管理命令，我们可通过 dbdeployer 工具下载，也可自行下载 MySQL 软件包，我们将下载的软件放置在之前已创建好的 `~/sandboxes/mysql_package` 目录下。以下以 MacOS 系统为例，Linux 类似。

4.1 使用 dbdeployer 下载 MySQL

查看 dbdeployer 工具支持下载的 MySQL 软件包

```
shell> dbdeployer downloads list
```

```
Available tarballs ()
```

	name	OS	version	flavor	size
e	minimal				

	tidb-master-darwin-amd64.tar.gz	Darwin	3.0.0	tidb	26 M

B

mysql-5.7.26-macos10.14-x86_64.tar.gz	Darwin5.7.26	mysql	337 MB
mysql-8.0.16-macos10.14-x86_64.tar.gz	Darwin8.0.16	mysql	153 M
mysql-8.0.15-macos10.14-x86_64.tar.gz	Darwin8.0.15	mysql	139 M
mysql-5.7.25-macos10.14-x86_64.tar.gz	Darwin5.7.25	mysql	337 MB
mysql-5.6.41-macos10.13-x86_64.tar.gz	Darwin5.6.41	mysql	176 MB
mysql-5.5.53-osx10.9-x86_64.tar.gz	Darwin5.5.53	mysql	114 MB
mysql-5.1.73-osx10.6-x86_64.tar.gz	Darwin5.1.73	mysql	82 MB
mysql-5.0.96-osx10.5-x86_64.tar.gz	Darwin5.0.96	mysql	61 MB
mysql-cluster-8.0.16-dmr-macos10.14-x86_64.tar.gz	Darwin8.0.16	ndb	252 MB
mysql-8.0.17-macos10.14-x86_64.tar.gz	Darwin8.0.17	mysql	155 MB
mysql-5.7.27-macos10.14-x86_64.tar.gz	Darwin5.7.27	mysql	337 MB
mysql-cluster-gpl-7.6.10-macos10.14-x86_64.tar.gz	Darwin7.6.10	ndb	482 MB
mysql-cluster-8.0.17-rc-macos10.14-x86_64.tar.gz	Darwin8.0.17	ndb	255 MB
mysql-cluster-gpl-7.6.11-macos10.14-x86_64.tar.gz	Darwin7.6.11	ndb	482 MB
mysql-shell-8.0.17-macos10.14-x86_64bit.tar.gz	Darwin8.0.17	mysql-shell	17 MB
mysql-5.7.28-macos10.14-x86_64.tar.gz	Darwin5.7.28	mysql	374 MB
mysql-8.0.18-macos10.14-x86_64.tar.gz	Darwin8.0.18	mysql	166 MB

下载并解压指定软件包

```
shell> cd sandboxes/mysql_package
```

```
shell> dbdeployer downloads get-unpack mysql-8.0.17-macos10.14-x86_64.tar.gz
```

4.2 自行下载 MySQL

访问 <https://downloads.mysql.com/archives/community/> 可下载各不同版本的 MySQL

下载并解压 MySQL 软件包

```
shell> cd sandboxes/mysql_package
```

```
shell> wget https://downloads.mysql.com/archives/get/file/mysql-8.0.17-macos10.14-x86_64.tar.gz
```

```
shell> dbdeployer unpack mysql-8.0.17-macos10.14-x86_64.tar.gz
```

4.3 快速部署实例

以下示例简单使用几条命令实现各种不同架构的 MySQL 部署，详细使用方式可查看文末 dbdeployer 文档链接。

部署一个单节点的 MySQL，开启 GTID 并指定字符集

GTID 和字符等参数也可在部署完成后在 MySQL 配置文件中指定

注意：部署的数据库默认自动运行，可以指定 --skip-start 参数只初始化但不启动

```
shell> dbdeployer deploy single 8.0.17 --gtid --my-cnf-options="character_set_server=utf8mb4"
```

部署一个主从复制 MySQL (默认初始化 3 个节点，一主两从)

```
shell> dbdeployer deploy replication 8.0.17 --repl-crash-safe --gtid --my-cnf-options="character_set_server=utf8mb4"
```

部署一个单主模式的 MGR (默认初始化 3 个节点)

```
shell> dbdeployer deploy --topology=group replication 8.0.17 --single-primary
```

```
## 部署一个多主模式的 MGR (默认初始化 3 个节点)
```

```
shell> dbdeployer deploy --topology=all-masters replication 8.0.17
```

4.4 实例组操作

一键部署完成后会在 `$HOME/sandboxes` 目录下生成各实例组对应的数据目录，该目录包含以下信息(部分信息)

- ☐ 一键启停该组所有实例的脚本
 - ☐ 一键登录数据库脚本
 - ☐ 一键重置该组所有实例的脚本（清除所有测试数据并重新初始化成全新的主从）
 - ☐ 主从实例的数据目录（主库为 master，从库分别为 node1、node2 依次递增）
- 各实例的配置文件
默认用户授权命令
单独启停实例命令
binlog、relaylog 解析命令

使用示例

```
## 查看该组所有实例状态
```

```
shell> cd ~/sandboxes/rsandbox_8_0_17
```

```
shell> ./status_all
```

```
REPLICATION  /Users/yuzhenxing/sandboxes/rsandbox_8_0_17
```

```
master : master on - port    20718(20718)
```

```
node1  : node1 on  - port    20719(20719)
```

```
node2  : node2 on  - port    20720(20720)
```

```
## 一键重启该组所有实例
```

```
shell> ./restart_al
```

```
stop /Users/yuzhenxing/sandboxes/rsandbox_8_0_17/master
```

```
stop /Users/yuzhenxing/sandboxes/rsandbox_8_0_17/node1
```

```
stop /Users/yuzhenxing/sandboxes/rsandbox_8_0_17/node2
```

```
executing 'start' on master
```

```
. sandbox server started
```

```
executing 'start' on slave 1
```

```
. sandbox server started
```

```
executing 'start' on slave 2
```

```
. sandbox server started
```

```
## 单独重启该组某一实例
```

```
shell> cd ~/sandboxes/rsandbox_8_0_17/master
```

```
shell> ./restart
```

```
## 登录指定实例
```

```
shell> ./use
```

5 dbdeployer 常用管理命令

以下是使用 dbdeployer 过程中总结的常用命令，详细使用方式可查看文末 dbdeployer 文档链接。

查看 dbdeployer 支持的各类数据库版本及版本特性（输出信息过长或过多，已省略输出结果）

```
shell> dbdeployer admin capabilities
shell> dbdeployer admin capabilities percona
shell> dbdeployer admin capabilities mysql
```

使用 dbdeployer 查看指定版本 MySQL 的基本信息

```
shell> dbdeployer downloads get-by-version 5.7 --newest --dry-run
Name:          mysql-5.7.28-macos10.14-x86_64.tar.gz
Short version: 5.7
Version:       5.7.28
Flavor:        mysql
OS:            Darwin
URL:           https://dev.mysql.com/get/Downloads/MySQL-5.7/mysql-5.7.28-
macos10.14-x86_64.tar.gz
Checksum:      MD5: 00c2cabb06d8573b5b52d3dd1576731e
Size:          374 MB
```

查看当前已安装的所有 MySQL 基本信息

包括架构类型、版本、端口等等（输出信息过长或过多，已省略输出结果）

```
shell> dbdeployer sandboxes --full-info
```

查看各组 MySQL 的运行情况

```
shell> dbdeployer global status
MULTIPLE /Users/yuzhenxing/sandboxes/all_masters_msb_8_0_17
node1 : node1 on - port 23818(23818)
node2 : node2 on - port 23819(23819)
node3 : node3 on - port 23820(23820)
MULTIPLE /Users/yuzhenxing/sandboxes/group_sp_msb_8_0_17
node1 : node1 off - (22718)
node2 : node2 off - (22719)
node3 : node3 off - (22720)
```

批量停止所有运行的 MySQL（输出信息过长或过多，已省略输出结果）

```
shell> dbdeployer global stop
```

删除已部署的 MySQL 实例

如正在运行则会先停止，后清除实例相关目录

```
shell> dbdeployer delete msb_8_0_17
```

可对实例加锁，防止误删除

```
## dbdeployer sandboxes --full-info 命令可查的到哪组实例已加锁
shell> dbdeployer admin lock group_sp_msb_8_0_17
shell> dbdeployer delete group_sp_msb_8_0_17
shell> dbdeployer admin unlock group_sp_msb_8_0_17
shell> dbdeployer sandboxes --full-info
```

其他更多的 dbdeployer 使用和管理命令，可执行 dbdeployer 或 dbdeployer --help 进行查看，更详尽的使用方式和使用示例可查看文末的 dbdeployer 官方手册链接。

相关链接

dbdeployer 官方手册

<https://github.com/datacharmer/dbdeployer>

MySQL 各历史版本下载链接

<https://downloads.mysql.com/archives/community/>

dbdeployer 的功能特性

<https://github.com/datacharmer/dbdeployer/blob/master/docs/features.md>



巧用 binlog Event 发现问题

作者：高鹏（八怪）

有了前面对 Event 的了解，我们就可以利用这些 Event 来完成一些工作了。我曾经在学习了这些常用的 Event 后，使用 C 语言写过一个解析 Event 的工具，我叫它 ‘infobin’，意思就是从 binary log 提取信息的意思。据我所知虽然这个工具在少数情况下会出现 BUG 但是还是有些朋友在用。我这里并不是要推广我的工具，而是要告诉大家这种思路。我是结合工作中遇到的一些问题来完成了这个工具的，主要功能包含如下：

- 分析 binary log 中是否有长期未提交的事务，长期不提交的事务将会引发更多的锁争用。
- 分析 binary log 中是否有大事务，大事务的提交可能会堵塞其它事务的提交。
- 分析 binary log 中每个表生成了多少 DML Event，这样就能知道哪个表的修改量最大。
- 分析 binary log 中 Event 的生成速度，这样就能知道哪个时间段生成的 Event 更多。

下面是这个工具的地址，供大家参考：

<https://github.com/gaopengcarl/infobin>

这个工具的帮助信息如下：

```
[root@gpl infobin]# ./infobin
USAGE ERROR!
[Author]: gaopeng [QQ]:22389860[blog]:http://blog.itpub.net/7728585/
--USAGE:./infobin binlogfile pieces bigtrxsize bigtrxtime [-t] [-force]
[binlogfile]:binlog file!
[piece]:how many piece will split,is a Highly balanced histogram,
find which time generate biggest binlog.(must:piece<2000000)
[bigtrxsize](bytes):larger than this size trx will view.(must:trx>256(bytes))
[bigtrxtime](sec):larger than this sec trx will view.(must:>0(sec))
[[-t]]:if[-t] no detail isprintout,the result will small
[[-force]]:force analyze if unkown error check!!
```

接下来我们具体来看看这几个功能大概是怎么实现的。

1 分析长期未提交的事务

前面我已经多次提到过对于一个手动提交的事务而言有如下特点，我们以 ‘Insert’ 语句为列：

1. GTID_LOG_EVENT 和 XID_EVENT 是命令 ‘COMMIT’ 发起的时间。
2. QUERY_EVENT 是第一个 ‘Insert’ 命令发起的时间。
3. MAP_EVENT/WRITE_ROWS_EVENT 是每个 ‘Insert’ 命令发起的时间。

那实际上我们就可以用（1）减去（2）就能得到第一个 ‘DML’ 命令发起到 ‘COMMIT’ 命令发起之间所消

耗的时间，再使用一个用户输入参数来自定义多久没有提交的事务叫做长期未提交的事务就可以了，我的工具中使用 bigtrxtime 作为这个输入。我们来用一个例子说明，我们做如下语句：

语句	时间
begin	T1
insert into testrr values(20);	11:25:22
insert into testrr values(30);	11:25:26
insert into testrr values(40);	11:25:28
commit;	11:25:30

我们来看看 Event 的顺序和时间如下：

Event	时间
GTID_LOG_EVENT	11:25:30
QUERY_EVENT	11:25:22
MAP_EVENT (第1个insert)	11:25:22
WRITE_ROWS_EVENT (第1个insert)	11:25:22
MAP_EVENT (第2个insert)	11:25:26
WRITE_ROWS_EVENT (第2个insert)	11:25:26
MAP_EVENT (第3个insert)	11:25:28
WRITE_ROWS_EVENT (第3个insert)	11:25:28
XID_EVENT	11:25:30

如果我们使用最后一个 XID_EVENT 的时间减去 QUERY_EVENT 的时间，那么这个事务从第一条语句开始到‘COMMIT’的时间就计算出来了。注意一点，实际上‘BEGIN’命令并没有记录到 Event 中，它只是做了一个标记让事务不会自动进入提交流程。

关于‘BEGIN’命令做了什么可以参考我的简书文章如下：

<https://www.jianshu.com/p/6de1e8071279>

2 分析大事务

这部分实现比较简单，我们只需要扫描每一个事务 GTID_LOG_EVENT 和 XID_EVENT 之间的所有 Event 将它们的总和计算下来，就可以得到每一个事务生成 Event 的大小（但是为了兼容最好计算 QUERY_EVENT 和 XID_EVENT 之间的 Event 总量）。再使用一个用户输入参数自定义多大的事务叫做大事务就可以了，我的工具中使用 bigtrxsize 作为这个输入参数。

如果参数 binlog_row_image 参数设置为 ‘FULL’，我们可以大概计算一下特定表的每行数据修改生成的日志占用的大小：

- ‘Insert’ 和 ‘Delete’：因为只有 before_image 或者 after_image，因此 100 字节一行数据加上一些额外的开销大约加上 10 字节也就是 110 字节一行。如果定位大事务为 100M 那么大约修改量为 100W 行数据。
- ‘Update’：因为包含 before_image 和 after_image，因此上面的计算的 110 字节还需要乘以 2。因此如果定位大事务为 100M 那么大约修改量为 50W 行数据。

我认为 20M 作为大事务的定义比较合适，当然这个根据自己的需求进行计算。

3 分析 binary log 中 Event 的生成速度

这个实现就很简单了，我们只需要把 binary log 按照输入参数进行分片，统计结束 Event 和开始 Event 的时间差值就能大概算出每个分片生成花费的时间，我们工具使用 piece 作为分片的传入参数。通过这个分析，我们可以大概知道哪一段时间 Event 生成量更大，也侧面反映了数据库的繁忙程度。

4 分析每个表生成了多少 DML Event

这个功能也非常实用，通过这个分析我们可以知道数据库中哪一个表的修改量最大。实现方式主要是通过扫描 binary log 中的 MAP_EVENT 和接下来的 DML Event，通过 table id 获取表名，然后将 DML Event 的大小归入这个表中，做一个链表，最后排序输出就好了。

但是前面我们说过 table id 即便在一个事务中也可能改变，这是我开始没有考虑到的，因此这个工具有一定的问题，但是大部分情况是可以正常运行的。

5 工具展示

下面我就来展示一下我说的这些功能，我做了如下操作：

```
mysql> flush binary logs;
Query OK, 0 rows affected (0.51 sec)

mysql> select count(*) from tti;
+-----+
| count(*) |
+-----+
| 98304 |
+-----+
1 row inset (0.06 sec)
```

```
mysql> deletetfrom tti;
Query OK, 98304 rows affected (2.47 sec)

mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into tti values(1,'gaopeng');
Query OK, 1 row affected (0.00 sec)

mysql> select sleep(20);
+-----+
| sleep(20) |
+-----+
| 0 |
+-----+

1 row inset (20.03 sec)

mysql> commit;
Query OK, 0 rows affected (0.22 sec)

mysql> insert into ttp values(10);
Query OK, 1 row affected (0.14 sec)
```

在示例中我切换了一个 binary log，同时做了 3 个事务：

- ☐ 删除了 tti 表数据一共 98304 行数据。
- ☐ 向 tti 表插入了一条数据，等待了 20 多秒提交。
- ☐ 向 ttp 表插入了一条数据。

我们使用工具来分析一下，下面是统计输出：

```
./infobin mysql-bin.0000053100000015-t > log.log
more log.log
...
-----Total now-----
Trx total[counts]:3
Event total[counts]:125
Max trx event size:8207(bytes) Pos:420[0X1A4]
Avg binlog size(/sec):9265.844(bytes) [9.049(kb)]
Avg binlog size(/min):555950.625(bytes) [542.921(kb)]
--Piece view:
(1)Time:1561442359-1561442383(24(s)) piece:296507(bytes) [289.558(kb)]
(2)Time:1561442383-1561442383(0(s)) piece:296507(bytes) [289.558(kb)]
(3)Time:1561442383-1561442455(72(s)) piece:296507(bytes) [289.558(kb)]
--Large than 500000(bytes) trx:
(1)Trx_size:888703(bytes) [867.874(kb)] trx_begin_p:299[0X12B]
```

```

trx_end_p:889002[0XD90AA]
Total large trx count size(kb):#867.874(kb)
--Large than 15(secs) trx:
(1)Trx_sec:31(sec) trx_begin_time:[2019062514:00:08(CST)]
trx_end_time:[2019062514:00:39(CST)] trx_begin_pos:889067
trx_end_pos:889267 query_exe_time:0
--EveryTable binlog size(bytes) and times:
Note:size unit is bytes
---(1)CurrentTable:test.tpp::
Insert:binlog size(40(Bytes)) times(1)
Update:binlog size(0(Bytes)) times(0)
Delete:binlog size(0(Bytes)) times(0)
Total:binlog size(40(Bytes)) times(1)
---(2)CurrentTable:test.tti::
Insert:binlog size(48(Bytes)) times(1)
Update:binlog size(0(Bytes)) times(0)
Delete:binlog size(888551(Bytes)) times(109)
Total:binlog size(888599(Bytes)) times(110)
---Total binlog dml event size:888639(Bytes) times(111)

```

我们发现我们做的操作都统计出来了:

- 包含一个大事务日志总量大于 500K, 大小为 800K 左右, 这是我的删除 tti 表中 98304 行数据造成的。

```

--Large than 500000(bytes) trx:
(1)Trx_size:888703(bytes) [867.874(kb)] trx_begin_p:299[0X12B]
trx_end_p:889002[0XD90AA]

```

- 包含一个长期未提交的事务, 时间为 31 秒, 这是我特意等待 20 多秒提交引起的。

```

--Large than 15(secs) trx:
(1)Trx_sec:31(sec) trx_begin_time:[2019062514:00:08(CST)]
trx_end_time:[2019062514:00:39(CST)] trx_begin_pos:889067
trx_end_pos:889267 query_exe_time:0

```

- 本 binary log 有两个表的修改记录 tti 和 tpp, 其中 tti 表有 ‘Delete’ 操作和 ‘Insert’ 操作, tpp 表只有 ‘Insert’ 操作, 并且包含了日志量的大小。

```

--EveryTable binlog size(bytes) and times:
Note:size unit is bytes
---(1)CurrentTable:test.tpp::
Insert:binlog size(40(Bytes)) times(1)
Update:binlog size(0(Bytes)) times(0)
Delete:binlog size(0(Bytes)) times(0)
Total:binlog size(40(Bytes)) times(1)
---(2)CurrentTable:test.tti::

```

```
Insert:binlog size(48(Bytes)) times(1)
Update:binlog size(0(Bytes)) times(0)
Delete:binlog size(888551(Bytes)) times(109)
Total:binlog size(888599(Bytes)) times(110)
---Total binlog dml event size:888639(Bytes) times(111)
```

好了到这里我想告诉你的就是，学习了 Event 过后就可以自己通过各种语言去试着解析 binary log，也许你还能写出更好的工具实现更多的功能。

当然也可以通过 mysqlbinlog 进行解析后，然后通过 shell/python 去统计，但是这个工具的速度要远远快于这种方式。

DBLE 系列

本系列由 DBLE 的开发团队、测试团队和社区志愿者们共同完成，本系列从公众号发文中精选 21 篇。文章内容从 DBLE 的入门部署到源码解读，分片算法到性能调优。在 DBLE 公开课和微课程的学习之余，做非常好的补充。

微信扫一扫阅读原文



开源分布式中间件 DBLE 快速入门指南

作者：张沈波

1 环境准备

DBLE 使用 Java 开发，所以启动 DBLE 需要先在机器上安装 Java 1.8 或以上版本，并且确保 JAVA_HOME 参数被正确的设置：

这里通过 yum 源的方式安装 openjdk，同学们可以自行 google jdk 的几百种安装方式，这里不再赘述：

```
bash# yum install java-1.8.0-openjdk
```

确认 Java 环境已配置完成：

```
bash# java -version
openjdk version "1.8.0_191"
OpenJDK Runtime Environment (build 1.8.0_191-b12)
OpenJDK 64-Bit Server VM (build 25.191-b12, mixed mode)
```

2 安装 DBLE

DBLE 的安装其实只要解压下载的目录就可以了，非常简单。

1. 通过此连接下载最新安装包：<https://github.com/actiontech/dble/releases>
2. 解压并安装 DBLE 到指定文件夹中

```
mkdir -p $working_dir
cd $working_dir
tar -xvf actiontech-dble-$version.tar.gz
cd $working_dir/dble/conf
```

安装完成后，目录如下：

目录	说明
bin	DBLE命令：启动、重启、停止等
conf	DBLE配置信息,本文重点关注
lib	DBLE引用的jar包
logs	日志文件，包括DBLE启动的日志和运行的日志

3 配置 DBLE

DBLE 的配置文件都在 conf 目录里面，这里介绍几个常用的文件：

文件	说明
server.xml	DBLE server相关参数定义，包括dble性能，定时任务，端口，用户配置等；本文主要涉及到访问用户的配置
schema.xml	DBLE具体分片定义，规定table和schema以及dataNode之间的关系，指定每个表格使用哪种类型的分片方法，定义每个dataNode的连接信息等
rule.xml	DBLE实际用到的分片算法的配置

3.1 应用场景一：数据拆分

后端 MySQL 节点

DBLE 的架构其实很好理解，DBLE 是代理中间件，DBLE 后面就是物理数据库。对于使用者来说，访问的都是 DBLE，不会接触到后端的数据库。

我们先演示简单的数据拆分的功能。物理部署结构如下表：

服务	IP:Port	说明
DBLE	172.16.3.1:9066	DBLE实例，连接数据库时，连接此IP:Port
MySQL A	172.16.3.1:14014	物理数据库实例A，真正存储数据的数据库
MySQL B	172.16.3.1:14015	物理数据库实例B，真正存储数据的数据库

备注：为了演示简单，这里将实例都部署在了一台机器上并用不同端口做区分，同学们也可以用三

台机器来做环境搭建。

在 MySQL A 和 MySQL B 中创建库表 testdb.users 来方便后续的验证，表结构如下：

```
CREATE TABLE `users` (
  `id` int(11) NOT NULL,
  `user` varchar(20) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1
```

server.xml

server.xml 里可以配置跟 DBLE 自身相关的许多参数，这里重点只关注下面这段访问用户相关的配置，其他默认即可；

1. 第一段 “< system >” 为 DBLE 的服务端口（默认 8066）和管理端口（默认 9066）的配置
管理端口只能接受 DBLE 的管理命令，这里不做展开
服务端端口即 DBLE 的业务访问端口，可以接受 SQL 语句
2. 第二段 “< user >” 配置管理理用户，默认为 man1 密码为 654321
即可以通过 `mysql -P9066 -h 127.0.0.1 -u man1 -p654321` 来下发管理命令
3. 第三段 “< user >” 配置业务用户，配置了一个账号 test 密码 password 针对数据库 testdb，读写权限都有。没有针对表做任何特殊的权限，故把表配置做了注释
即可以通过 `mysql -P8066 -h 127.0.0.1 -utest -ppassword` 下发 SQL 语句

```
...
<system>
...
<!-- property name="serverPort">8066</property> -->
<!--<property name="managerPort">9066</property> -->
...
</system>
<user name="man1">
<property name="password">654321</property>
<property name="manager">true</property>
<!-- manager user can't set schema-->
</user>

<user name="test">
<property name="password">password</property>
<property name="schemas">testdb</property>

<!-- table's DML privileges INSERT/UPDATE/SELECT/DELETE -->
<!--
```

```

        <privileges check="false">
            <schema name="TESTDB" dml="0110" >
                <table name="tb01" dml="0000"></table>
                <table name="tb02" dml="1111"></table>
            </schema>
        </privileges>
    -->
</user>
...

```

参数	说明
user	用户配置节点
name	登录的用户名，也就是连接DBLE的用户名
password	登录的密码，也就是连接DBLE的密码
schemas	数据库名，这里会和schema.xml中的配置关联，多个用逗号分开，例如需要这个用户需要管理两个数据库db1,db2，则配置db1,db2
privileges	配置用户针对表的增删改查的权限，具体见官方文档，这里不做展开

schema.xml

schema.xml 是最主要的配置项，我们将 users 用户表按照取模的方式平均拆分到了 MySQL A 和 MySQL B 两个数据数据库实例上。详细请看配置文件：

```

<?xml version="1.0"?>
<!DOCTYPE dble:schema SYSTEM "schema.dtd">
<dble:schema xmlns:dble="http://dble.cloud/">

    <schema name="testdb">
        <table name="users" primaryKey="ID" dataNode="dn1,dn2" rule="sharding-by-mod2" />
    </schema>

    <!-- 分片配置 -->
    <dataNode name="dn1" dataHost="Group1" database="testdb"/>
    <dataNode name="dn2" dataHost="Group2" database="testdb"/>
    <!-- 物理数据库配置 -->
    <dataHost name="Group1" maxCon="1000" minCon="10" balance="0" switchType="1" slaveThreshold="100">
        <heartbeat>show slave status</heartbeat>
        <writeHost host="MySQLA" url="172.16.3.1:14014" user="test"

```



```

password="password"/>
</dataHost>

<dataHost name="Group2" maxCon="1000" minCon="10" balance="0" switchType="1"
slaveThreshold="100">
<heartbeat>show slave status</heartbeat>
<writeHost          host="MySQLA"          url="172.16.3.1:14015"          user="test"
password="password"/>
</dataHost>
</dblschema>

```

参数说明:

1. schema 逻辑数据库信息，此数据库为逻辑数据库，name 与 server.xml 中 schema 对应；
2. dataNode 分片信息，此为分片节点的定义；分片名字和 schema 的 dataNode 对应；分片与下面的 dataHost 物理实例进行关联；
3. dataHost 物理实例组信息，dataHost 下可以挂载同组的读写物理实例节点，实现高可用或者读写分离；

每个节点的重点属性逐一说明:

1. schema 属性说明:
 - name 逻辑数据库名，与 server.xml 中的 schema 对应；
 - table 子属性说明:
 - name 表名，物理数据库中表名
 - dataNode 表存储到哪些节点，多个节点用逗号分隔
 - primaryKey 主键，用于主键缓存和自增识别，不作主键约束
 - autoIncrement 是否自增
 - rule 分片规则名，具体规则下文 rule 详细介绍
2. dataNode 属性说明:
 - name 节点名，与 table 中 dataNode 对应
 - datahost 物理实例组名，与 datahost 中 name 对应
 - database 物理数据库中数据库名；
3. dataHost 属性说明:
 - name 物理数据库名，与 dataNode 中 dataHost 对应
 - balance 均衡负载的方式
 - switchtype 写节点的高可用切换方式；等于 1 时，心跳不健康发生切换
 - heartbeat 心跳检测语句，注意语句结尾的分号要加
 - writehost 写物理实例 子属性说明 :
 - host 物理实例名
 - url 物理库 IP+Port
 - user 物理库用户
 - password 物理库密码

rule.xml

主要关注 rule 属性，rule 属性的内容来源于 rule.xml 这个文件，DBLE 支持多种分表分库的规则，基本能满足你所需要的要求。

table 中的 rule 属性对应的就是 rule.xml 文件中 tableRule 的 name，具体有哪些拆分算法实现，建议还是看文档。我这里选择的 sharding-by-mod2，是 hash 算法的特例，就是将数据平均拆分。因为我后端是两台物理库，所以 rule.xml 中 hashmod2 对应的 partitionCount 为 2，配置如下：

```
<tableRule name="sharding-by-mod2">
<rule>
<columns>id</columns>
<algorithm>hashmod2</algorithm>
</rule>
</tableRule>
<function name="hashmod2" class="Hash">
<property name="partitionCount">2</property>
<property name="partitionLength">1</property>
</function>
```

验证配置生效

启动 DBLE

```
## 进入 DBLE 安装目录，执行 start 命令
./bin/dble start
```

DBLE 启动会自动加载配置，需确认进程是否正常启动，如启动失败，建议按照日志报错排查问题，正确启动日志如下：

```
STATUS | wrapper | 2019/01/21 17:31:43 | --> Wrapper Started as Daemon
STATUS | wrapper | 2019/01/21 17:31:43 | Launching a JVM...
INFO | jvm 1 | 2019/01/21 17:31:43 | OpenJDK 64-Bit Server VM warning:
ignoring option MaxPermSize=64M; support was removed in 8.0
INFO | jvm 1 | 2019/01/21 17:31:44 | Wrapper (Version 3.2.3)
http://wrapper.tanukisoftware.org
INFO | jvm 1 | 2019/01/21 17:31:44 | Copyright 1999-2006 Tanuki Software,
Inc. All Rights Reserved.
INFO | jvm 1 | 2019/01/21 17:31:44 |
INFO | jvm 1 | 2019/01/21 17:31:45 | Server startup successfully. see logs
in logs/dble.log
```

通过 DBLE 流量入口 8066 登陆数据库，插入两条用户记录，并获取 DBLE 侧的查询记录。

```
mysql -P8066 -h 127.0.0.1 -utest -ppassword
mysql> insert into users(id,user) values(1,"zhangsan");
Query OK, 1 row affected (0.09 sec)

mysql> insert into users(id,user) values(2,"lisi");
Query OK, 1 row affected (0.09 sec)
```

```
mysql> explain select * from users;
+-----+-----+-----+
| DATA_NODE | TYPE      | SQL/REF          |
+-----+-----+-----+
| dn1        | BASE SQL  | select * from users |
| dn2        | BASE SQL  | select * from users |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> select * from users;
+----+-----+
| id | user      |
+----+-----+
| 2  | lisi      |
| 1  | zhangsan  |
+----+-----+
2 rows in set (0.01 sec)
```

获取 MySQL A 和 MySQL B 的记录

```
# mysql -P14014 -h 127.0.0.1 -utest -ppassword
mysql> select * from users;
+----+-----+
| id | user      |
+----+-----+
| 2  | lisi      |
+----+-----+
1 rows in set (0.01 sec)
```

```
# mysql -P14015 -h 127.0.0.1 -utest -ppassword
mysql> select * from users;
+----+-----+
| id | user      |
+----+-----+
| 1  | zhangsan  |
+----+-----+
1 rows in set (0.01 sec)
```

从上面的验证流程，往 DBLE 插入的数据，会按照取模的方式下发到真实的物理库，来实现数据库的自动分片；同时通过 DBLE 下发的查询会被 DBLE 自动下发给实际的物理库，合并返回给客户端，可以通过 explain 执行计划观察到下发的实际下发给物理库的 SQL 语句。

3.2 应用场景二：读写分离

DBLE 除了做数据的分片功能外，也支持读写分离功能；开启读写分离功能后，可以将主实例上的读压力负载给原本 stand by 的从实例，从而扩展整个集群的吞吐能力；

后端 MySQL 节点

我们再通过示例，演示 DBLE 的读写分离的功能。物理部署结构如下表：

服务	IP:Port	说明
DBLE	172.16.3.1:9066	DBLE实例，连接数据库时，连接此IP:Port
MySQL A	172.16.3.1:14014	物理数据库实例A， master实例
MySQL B	172.16.3.1:14015	物理数据库实例B， slave实例

备注：为了演示简单，这里将实例都部署在了一台机器上并用不同端口做区分，同学们也可以用三台机器来做环境搭建。

此场景中，我们将 MySQL A 和 MySQL B 搭建成主从复制关系，同时我们只变更 schema.xml 的配置来完成读写分离的架构；

schema.xml

```
<?xml version="1.0"?>
<!DOCTYPE dble:schema SYSTEM "schema.dtd">
<dble:schema xmlns:dble="http://dble.cloud/">

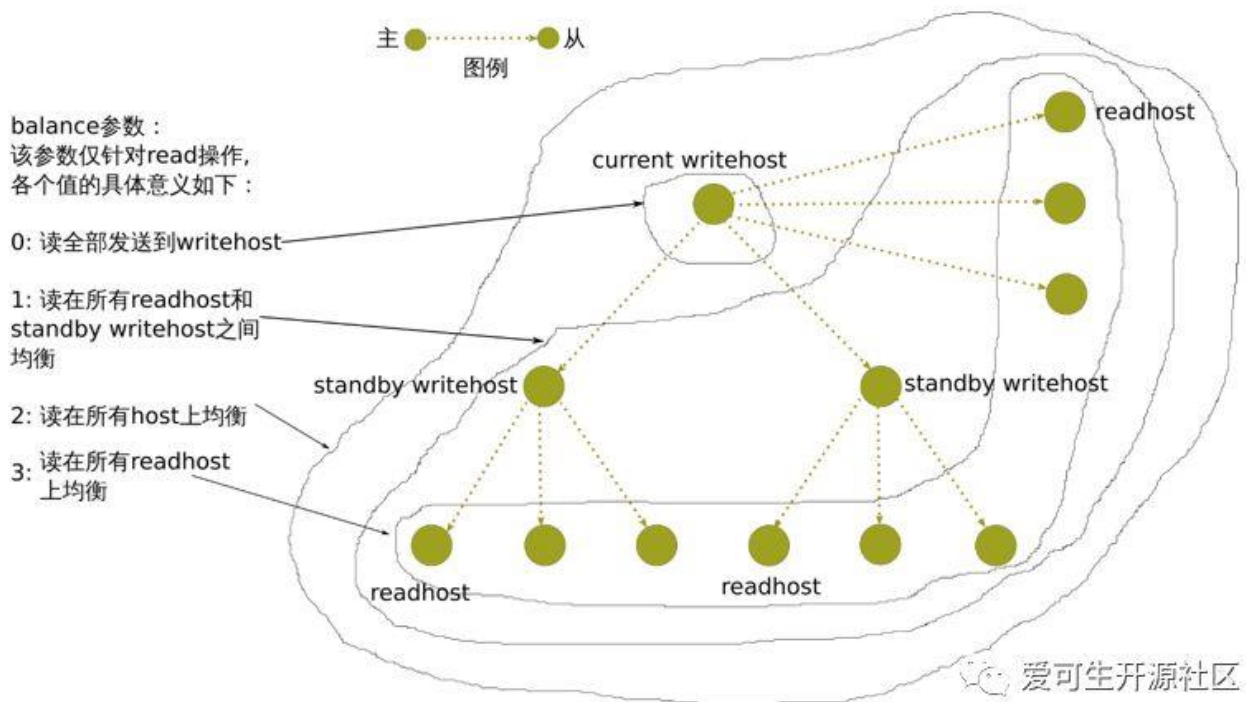
  <schema name="testdb" dataNode="dn1">
</schema>

  <!-- 分片配置 -->
  <dataNode name="dn1" dataHost="Group1" database="testdb"/>

  <!-- 物理数据库配置 -->
  <dataHost name="Group1" maxCon="1000" minCon="10" balance="3" switchType="1"
slaveThreshold="100">

    <heartbeat>show slave status</heartbeat>
    <writeHost host="MySQLA" url="172.16.3.1:14014" user="test" password="password">
    <readHost host="MySQLB" url="172.16.3.1:14015" user="test" password="password"/>
    </writeHost>
  </dataHost>
</dble:schema>
```

DBLE 通过 balance 参数来控制读写分离的负载策略，写节点是否参与均衡与 datahost 的 balance 属性有关，本案例中我们将值调整为 balance="3"，并定义了 writeHost 和 readHost。balance 的定义具体见下图：



验证配置生效

通过 DBLE 管理入口 9066 登陆数据库，注意这里我们通过管理入口的 `show @@datasource` 来验证读写分离的状态的正确性；

session 1:

```
## session1: 登陆DBLE的管理端，查看读写分离的节点状态
```

```
mysql -P9066 -h 172.16.3.1 -uman1 -p654321
```

```
mysql> show @@datasource;
```

NAME	HOST	PORT	W/R	ACTIVE	IDLE	SIZE	EXECUTE	READ_LOAD	WRITE_LOAD
MySQLA	172.16.3.1	19388	W	11	11	1000	11	0	0
MySQLB	172.16.3.1	19389	R	1	4	1000	3	0	0

2 rows in set (0.00 sec)

爱可生开源社区

session2:

```
## session2: 登录DBLE流量端，下发select语句5次
```

```
mysql -P8066 -h 172.16.3.1 -utest -ppassword
```

```
mysql> select * from users;
```

session 1:

```
## session1:重新返回DBLE的管理端，查看READ_LOAD字段计数器的变化
```

```
mysql> show @@datasource;
```

NAME	HOST	PORT	W/R	ACTIVE	IDLE	SIZE	EXECUTE	READ_LOAD	WRITE_LOAD
MySQLA	172.16.3.1	19388	W	11	11	1000	11	0	0
MySQLB	172.16.3.1	19389	R	1	4	1000	8	5	0

2 rows in set (0.00 sec)

 爱可生开源社区

从 `show @@datasource;` 这个管理命令上我们能够观测到 `READ_LOAD` 在 `slave` 节点上计数器增加了 5 次，也就是说读流量顺利的下发到了 `slave` 节点；当然大家也可以通过打开 MySQL 的 `general log` 来观测读写分离的情况。

4 总结

本文通过两个场景来讲解 DBLE 的快速入门，希望通过简单的示例来给大家梳理 DBLE 的基本概念，帮助大家快速熟悉和使用 DBLE 这个中间件；更高阶的使用方法和细节建议大家参考官方文档。



DBLE schema.xml 配置解析

作者：张沈波

1 DBLE 的主要配置文件

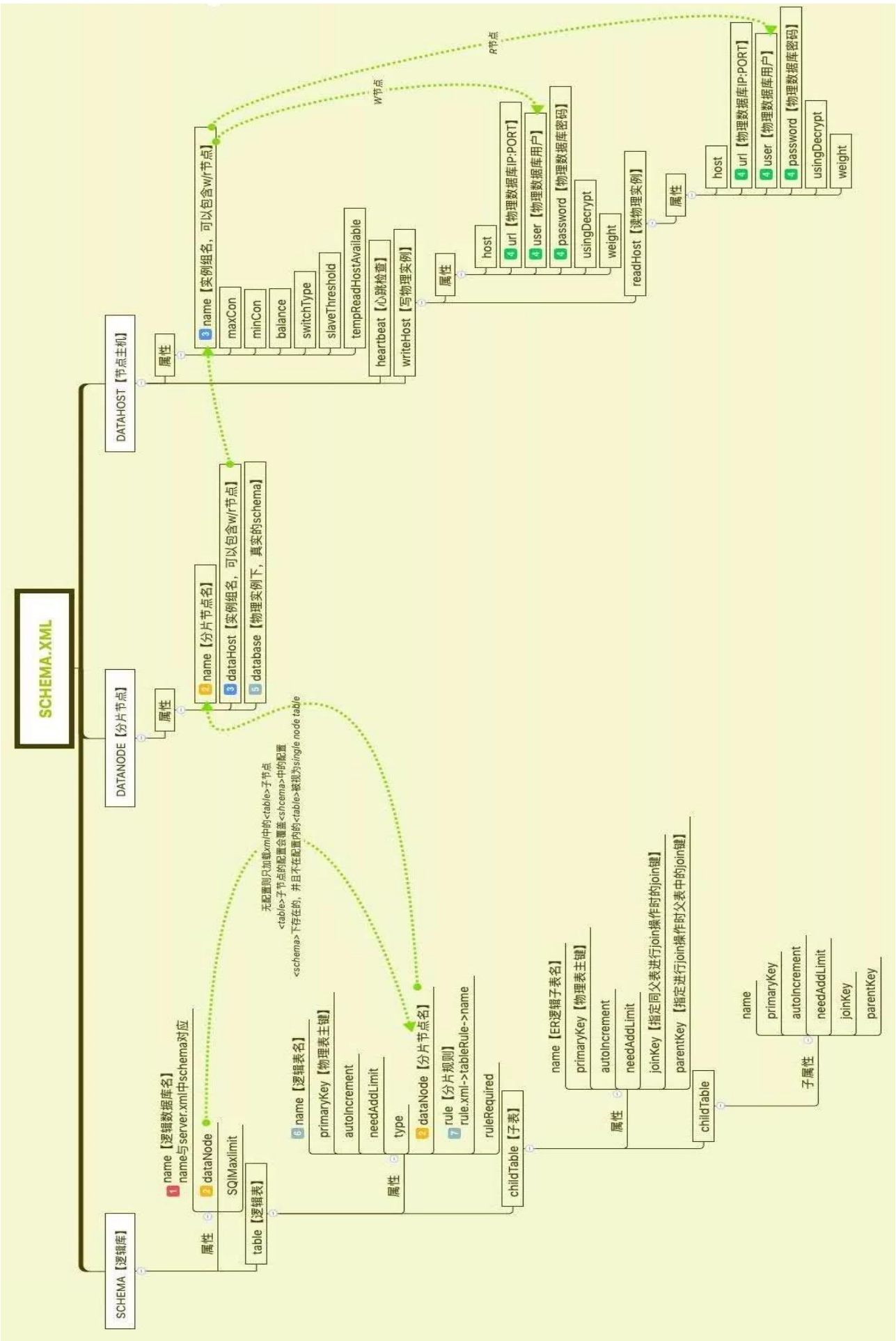
DBLE 的配置文件都在 conf 目录里面，常用的几个配置文件如下：

文件	说明
server.xml	DBLE server 相关参数定义，包括 DBLE 性能，定时任务，端口，用户配置等；本文主要涉及到访问用户的配置
schema.xml	DBLE 具体分片定义，规定 table 和 schema 以及 dataNode 之间的关系，指定每个表格使用哪种类型的分片方法，定义每个 dataNode 的连接信息等
rule.xml	DBLE 实际用到的分片算法的配置

2 schema.xml 配置解析

其中 schema.xml 是日常配置分片的时候最常用到的配置文件，我们通过思维导图的方式给大家整理了 DBLE 的 schema.xml 的配置：

schema.xml 文件主要由三个节点组成：SCHEMA 【逻辑库】、DATANODE 【分片节点】、DATAHOST 【节点主机】组成。



3 schema.xml 举例

下面举个 DBLE 的 schema 配置文件例子，并根据这个案例对逻辑数据库到物理数据库做了图解。

schema 配置文件：

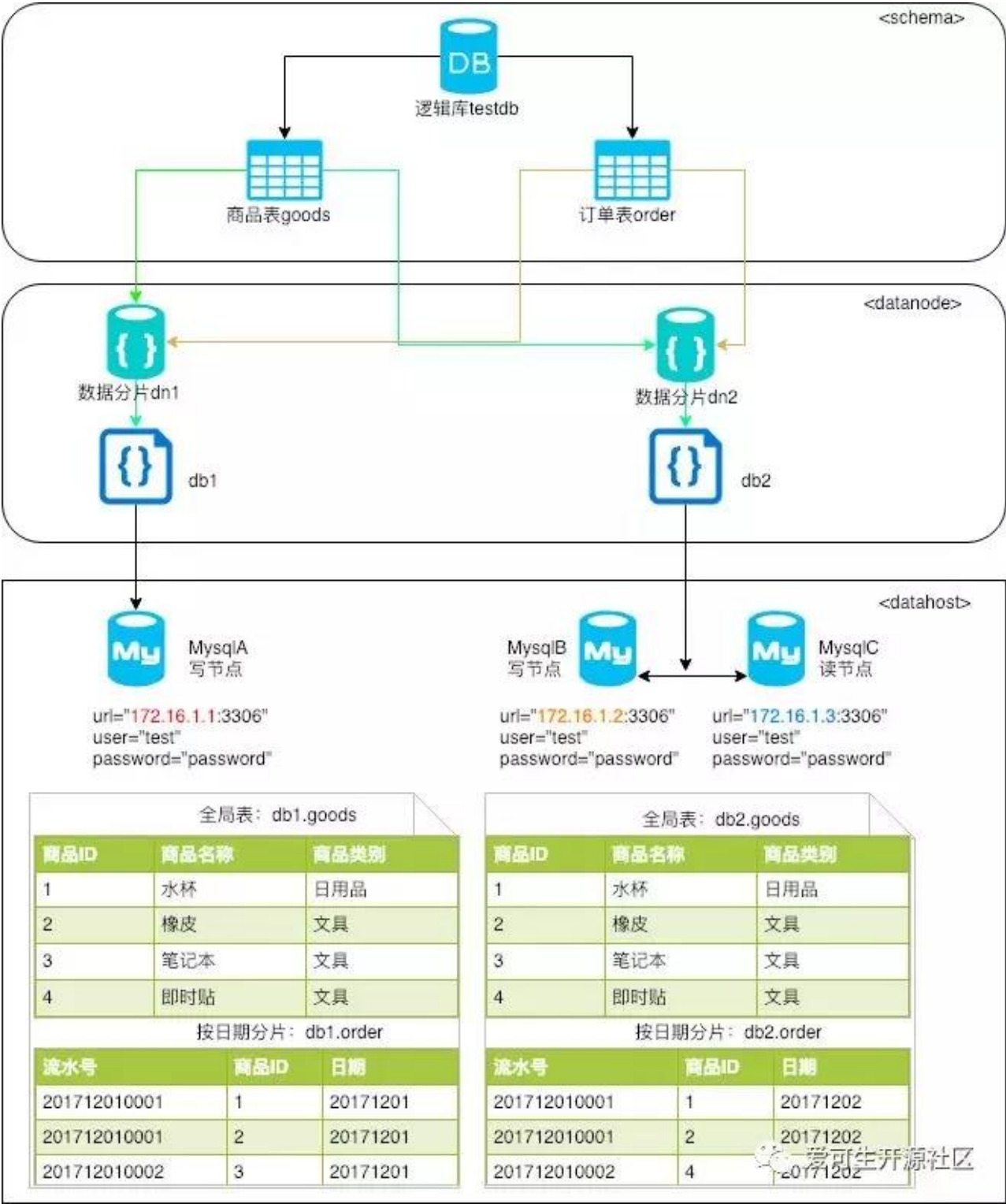
```
<?xml version="1.0"?>
<!DOCTYPE dble:system SYSTEM "schema.dtd">
<dble:system xmlns:dble="http://dble.cloud/">

    <schema name="testdb">
        <table name="order" primaryKey="ID" type="global" dataNode="dn1,dn2" />
        <table name="goods" primaryKey="ID" dataNode="dn1,dn2" rule="sharding-
by-date" />
    </schema>

    <!-- 分片配置 -->
    <dataNode name="dn1" dataHost="dh1" database="db1"/>
    <dataNode name="dn2" dataHost="dh2" database="db2"/>

    <!-- 物理数据库配置 -->
    <dataHost name="dh1" maxCon="1000" minCon="10" balance="0" switchType="1"
slaveThreshold="100">
        <heartbeat>show slave status</heartbeat>
        <writeHost host="MySQLA" url="172.16.1.1:3306" user="test"
password="password"/>
    </dataHost>
    <dataHost name="dh2" maxCon="1000" minCon="10" balance="0" switchType="1"
slaveThreshold="100">
        <heartbeat>show slave status</heartbeat>
        <writeHost host="MySQLB" url="172.16.1.2:3306" user="test"
password="password">
            <readHost host="MySQLC" url="172.16.1.3:3306" user="test"
password="password"/>
        </writeHost>
    </dataHost>
</dble:system>
```

4 图解 schema.xml



5.2.5 总结

schema.xml 是 DBLE 中间件如何配置分片最重要一个配置文件；如能熟悉掌握其中的逻辑概念，就可以对 DBLE 熟练配置；更高阶和详尽的用法，建议大家查阅官网的官方文档。

function(配置拆分算法，目前支持的拆分算法如下七种)

也叫固定分片hash算法，类似于十进制的求模运算，适用场景：
1. 拆分字段恒为整形，类似于对该字段取模

Hash
partitionCount(指定分区的区间数)
partitionLength(指定各区间长度)
与Hash类似，唯一的区别是该算法拆分字段只适用于字符类型。

StringHash
partitionCount(指定分区的区间数)
partitionLength(指定各区间长度)
hashSlice(指定参与hash值计算的key的子串，字符串从0开始索引计数。)
适用场景：拆分字段限定于少数固定值，比如，有些业务需要按照省份或区县来做保存，而这些值是有限的

Enum
mapFile partition.txt
type=0 int1=node0
type 1=0 string1=node0
string2=node1
defaultNode(指定默认节点号。默认值为-1，即没有默认节点。)
type 0 (表示整形)
非0 (非整形)
适用场景：固定时间周期轮转，比如：财务月表，及其他需要定期按月或天归档的表，如历史表等。
dateFormat (指定日期的格式)
sBeginDate(指定日期的开始时间。)
sEndDate(指定日期的结束时间。可以不配置或配置为空(")。)
sPartitionDay(指定分区的间隔，单位是天。)
defaultNode(指定默认节点号。默认值为-1，不指定默认节点。)

适用场景：
1、只适用于整形
2、明确知道拆分字段的范围（也可配置默认数据节点，找不到范围时的下发分片节点）
NumberRange
mapFile(指定配置文件名) partition.txt start1-end1=node1
start2-end2=node2
defaultNode(指定默认节点号,默认值为-1，即没有默认节点。
类似于NumberRange，但在进行分片查找时先对分片字段值进行patternValue运算，然后将得到的模数进行等同于numberRange分区算法的分区分找，取模之后的数据落在某一节点则数据落在该节点，适用于整型和可以将拆分字段转换为整型的字段。
PatternRange
mapFile partition.txt start1-end1=node1
start1-end2=node2
patternValue (指定模值，默认为1024)
defaultNode (指定默认节点号。默认值为-1，不指定默认节点。)
对拆分字段做尽可能hash计算并平均分布。
jumpstringhash (跳增字符串算法)
partitionCount(分片数量)
hashSlice(分片截取长度)

tableRule

rule
columns(拆分字段，只能单字段)
algorithm (只能是function声明过的元素)
name

schema.xml
table
rule
DATAHOST (节点主机)
DATANODE (分片节点)



DBLE rule.xml 配置解析

作者：余朝飞

1 DBLE 的主要配置文件

上一篇《DBLE schema.xml 配置解析》详细介绍了 DBLE 关于 schema.xml 的配置，本篇文章将继续为大家讲解一下 DBLE 中 rule.xml 文件的配置。

DBLE 的配置文件都在 conf 目录里面，常用的几个配置文件如下：

文件	说明
server.xml	DBLE server相关参数定义，包括dble性能，定时任务，端口，用户配置等；本文主要涉及到访问用户的配置
schema.xml	DBLE具体分片定义，规定table和schema以及dataNode之间的关系，指定每个表格使用哪种类型的分片方法，定义每个dataNode的连接信息等
rule.xml	DBLE实际用到的分片算法的配置

2 rule.xml 配置解析

其中 rule.xml 是日常配置分片算法的时候最常用到的配置文件，我们通过思维导图的方式给大家整理了 DBLE 的 rule.xml 的配置，需要注意的是思维导图不能代替看文档，导图只能起着概括归纳的作用，详细的细节还请参考官方文档。

3 rule.xml 举例

从分片的数据在各个数据节点分布来看，分片可分为连续分片和离散分片,连续分片就是将一定范围内的数据全部分布在某一 DataNode，离散分布则是通过 hash 取模等方法将数据打散较为均匀地分布在各个 DataNode。

分片	连续分片	离散分片
优点	并发访问能力有限，扩容迁移代价小	并发访问能力增强 范围条件查询性能提升
缺点	存在数据热点的可能性，并发访问能力受限 于单一或少量DataNode	数据扩容比较困难，需要对整体数据做重新分布。
举例	date,numberrange	hash,stringhash, patternrange

注：hash，patternrange 分片方式如果配置`分片区间`足够宽的话也是可以当做连续分片的。
以下我以 PatternRange 算法为例，讲解一下如何使用该算法，比如当前有一张表 task_log 已经有 1000 万的数据，这张表又应为需要和其他表进行关联查询，单表太大单值关联时异常缓慢，因此我们需要对这张表做拆分，将这张表分别放在三个分片上：dn1, dn2, dn3。

schema.xml 中的配置如下:

```
<table name="task_log" dataNode="dn1,dn2,dn3" rule="three_node_range" />
```

rule.xml 中配置如下:

```
<?xml version="1.0"?>
<!DOCTYPE dble:rule SYSTEM "rule.dtd">
<dble:rule xmlns:dble="http://dble.cloud/">
  <tableRule name="three_node_range">
    <rule>
      <columns>id</columns>
      <algorithm>three_node_range</algorithm>
    </rule>
  </tableRule>
  <function name="three_node_range" class="PatternRange">
    <property name="mapFile">partition.txt</property>
    <property name="patternValue">1024</property>
    <property name="defaultNode">0</property>
  </function>
</dble:rule>
```

mapfile partition.txt 定义如下:

```
[root@localhost ~]# cat partition.txt
0-255=0
256-511=1
512-1024=2
```

查找路由时, 将 id 字段与 patternValue 取模, 即计算 $M = id \% \text{patternValue}$

1. 如果 M 在 0-255 之间, 在数据落在 dn1 分片
 2. 如果 M 在 256-511 之间, 在数据落在 dn2 分片
 3. 如果 M 在 512-1024 之间, 则数据落在 dn3 分片
 4. 如果都匹配不上, 则落在默认节点 defaultNode dn1 分片 (理论上在这个例子中是不可能匹配不上的)
- 关于每一种拆分算法的详细介绍请参考官方文档介绍。

4 总结

rule.xml 定义实际用到的拆分算法, 熟悉各种拆分区算法的详细配置及其适用场景, 方便我们在众多数据拆分场景选择并配置合适的拆分规则, 同时这也是试用分库分表中间件的第一步。

将表的详细拆分算法写在配置中, 这是一种很“傻”的方式, 但是这也是万不得已的一种选择, 如果不通过配置文件的方式告诉中间件这些信息, 那么中间件就无从得知底层具体的数据分布情况, 也就达不到最终想要的目的。



DBLE server.xml 配置解析

作者：余朝飞

1 DBLE 的主要配置文件

本章前两篇文章分别介绍 DBLE 中 rule.xml 和 schema.xml 的配置，今天继续介绍 DBLE 中 Server.xml 文件的配置，希望通过本篇的介绍，能对 server.xml 文件的整体结构和内容有较为清晰的认识，方便大家快速入门。

DBLE 的配置文件都在 conf 目录里面，常用的几个配置文件如下：

文件	说明
server.xml	DBLE server相关参数定义，包括dbld性能，定时任务，端口，用户配置等
schema.xml	DBLE具体分片定义，规定table和schema以及dataNode之间的关系，指定每个表格使用哪种类型的分片方法，定义每个dataNode的连接信息等
rule.xml	DBLE实际用到的分片算法的配置

DBLE 是一个 JAVA 分库分表中间件，既然是 JAVA 应用肯定会有 JVM 相关的配置，在 DBLE 中我们选择 wrapper 来作为管理 DBLE 进程的工具，配置文件在 conf/wrapper.conf 中，关于 JVM 的详细介绍在 wrapp.conf。

wrapp.conf:

https://actiontech.github.io/dble-docs-cn/1.config_file/1.4_wrapper.conf.html

2 server.xml 配置解析

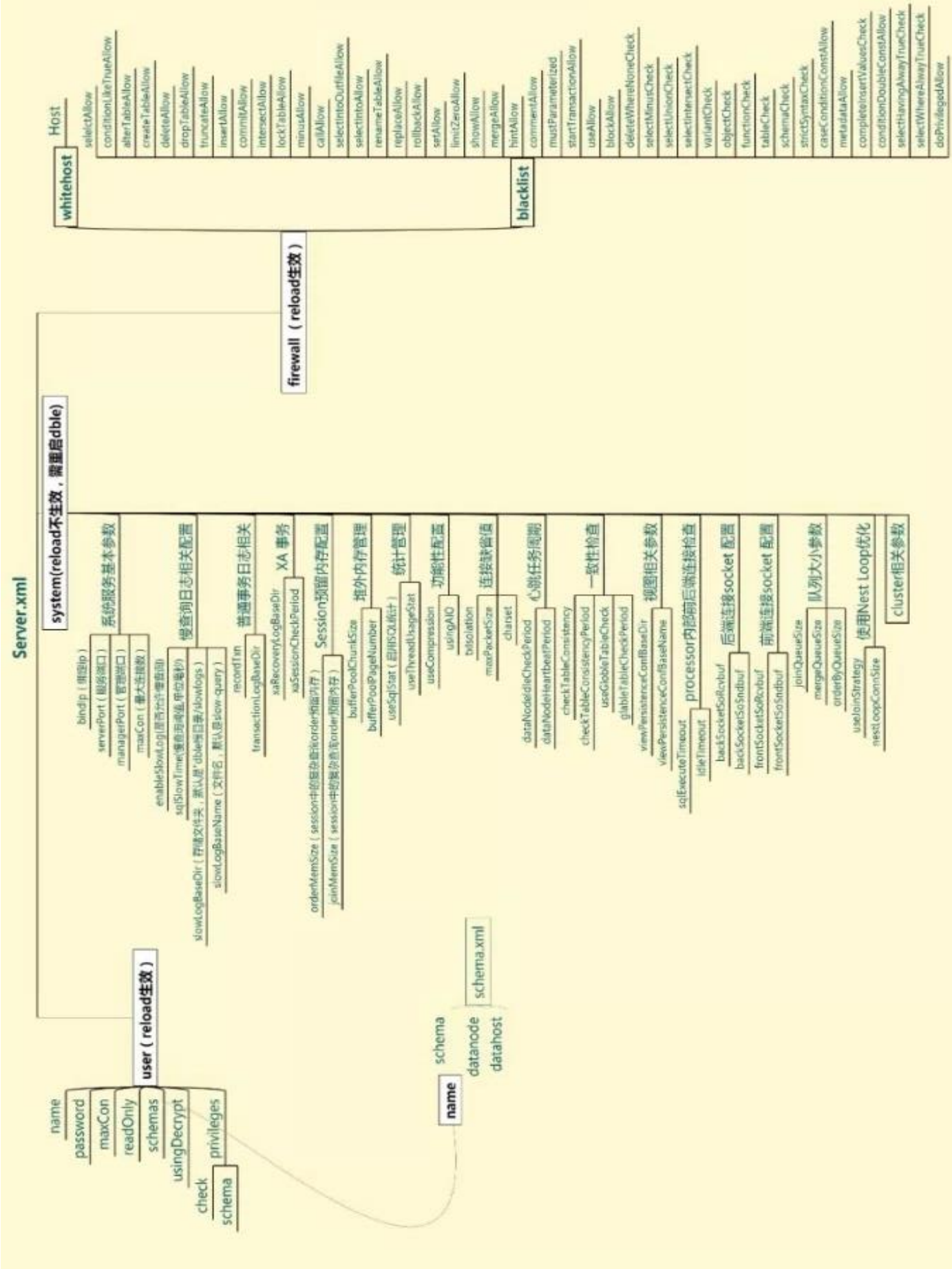
server.xml 是 DBLE 的重要配置文件之一，整体配置可以分为三段，<system>段，<user>段和<firewall>段，分别定义了 DBLE 软件的相关配置，用户配置和针对用户的权限控制功能。

```
<dbld:server>
  <system></system>
  <user></user>
  <firewall>
    <whitehost></whitehost>
  </firewall>
</dbld:server>
```

关于配置文件，如果在 DBLE 运行途中修改了配置，要想配置动态生效，只需要使用 reload @@config 命令，但是需要提醒的是 reload @@config 对 Server.xml 中的<system>段中的内容无法生效，即如果修改了 server.xml 的 system 的配置，需要重新启动 DBLE 使其生效。更多管理命令请参考：

https://actiontech.github.io/dble-docs-cn/2.Function/2.01_manager_cmd.html

其中个别参数配置只是粗略地将最重要的参数罗列出来，详细的参数还请参考官方文档。



3 system 配置

DBLE 通过 Server.xml 配置文件来定义相关管理行为，如监听端口，SQL 统计和控制连接行为等功能，DBLE 除了支持分库分表功能外，还实现了类似 MySQL 的慢查询日志功能，并且支持使用 pt-query-digest 这样的工具进行慢查询 SQL 分析，极大地方便了 DBA 的 SQL 性能分析与问题 SQL 定位，以下简单列出了一些最重要的也是最基础的功能配置。

property	作用
serverPort	业务用户连接端口
managerPort	管理用户连接端口
charset	字符集
maxCon	控制最大连接数
processors	NIO前端处理器的数量，默认java虚拟机核数
enableSlowLog	是否允许慢查询，默认不开启
sqlSlowTime	慢查询阈值，类似MySQL中的long_query_time，单位是毫秒

4 用户配置

用户配置在<user>段进行配置，因为在 schema.xml 中允许多个 schema 的存在，因此业务用户也是允许多个用户同时存在，并且还可以给这些用户进行更小粒度的权限划分。

以实际场景来举例，比如当前配置了两个逻辑库 adv 和 motor, 分别是汽车和广告业务，这两个库直接没有任何的关联，因此需要分别配置两个用户来使用这两个 schema，一个是 adv_user, 另一个是 motor_user, 这两个用户登录上去 DBLE 能看到只有自己的 schema，其他 schema 不可见，两个用户的配置如下：

```
<user name="adv_user">
  <property name="password">adv_user</property>
  <property name="schemas">adv</property>
  <property name="readOnly">>false</property>
</user>
<user name="motor_user">
  <property name="password">motor_user</property>
  <property name="schemas">motor</property>
  <property name="readOnly">>false</property>
</user>
```

但是你可能会想万一这两个库之中的表有关联查询呢？对应场景是：我们需要有个用户叫 adv_motor_user, 它对 motor 和 adv 库都需要有权限访问。别担心，DBLE 提供了对单个用户可以访问多个 schema 的配置方式，我们可以在 schemas 中指定多个 schema，之间用逗号分隔，配置生效后使用这个用户就能登录看到两个 schema。

```
<user name="adv_motor_user">
```



```

<property name="password">adv_motor_user</property>
<property name="schemas">motor,adv</property>
<!--多个逻辑库之间使用逗号分隔，这些逻辑库必须在 schema.xml 中定义-->
<property name="readOnly">false</property>
<!--用来做只读用户-->
</user>

```

你可能还会想，这样的权限粒度依然不够细，我们需要更细粒度的权限控制，比如需要 adv_user 的权限仅限于增删改查权限，即需要将权限细分到 DML 语句，DBLE 仍然提供这样的配置，我们可以继续增加 privileges 的配置，如下图示：

```

<user name="adv_user">
  <property name="password">adv_user</property>
  <property name="schemas">adv</property>
  <property name="readOnly">false</property>
  <privileges check="true">
    <schema name="adv" dml="1110" >
      <!-- 默认库中所有逻辑表的继承权限 -->
      <table name="tb01" dml="1111"></table>
      <!-- 单独指定表权限 -->
    </schema>
  </privileges>
</user>

```

privileges 的 check 参数作用于是否对用户权限进行检查，默认是不检查，DML 的权限是分别是 INSERT UPDATE SELECT DELETE 四种权限，用 4 个数字 0 或 1 的组合来表示是否开启，1 表示开启，0 表示关闭。

在上图中，adv_user 对 adv 库中所有表的默认权限是'1110'，即只有 insert，update 和 select 权限。即如果没有像 tb01 那样详细地列出来的情况，所有表的权限继承权限 1110，上面例子中，ta01 作为特殊指定的逻辑表的权限，adv_user 对该表的权限是 1111。

我们可以看到 DBLE 可以将权限划分到 DML，并且是可以精确到某一逻辑表级别，对于只需要 DML 权限的场景是足够了，但是这种权限控制还是很粗略，比如如果能让 adv_user 除了有 dml 权限之外，还需要有 drop table 权限，这时候在用 DBLE 的 privileges 权限控制就显得无能为力了。

因此可行的建议是：

1. 控制连接后端 MySQL 的用户的权限，将 DBLE 侧的权限完全放开，换句话说只严格限制 schema.xml 的 writehost 中配置的连接用户的权限。
2. 使用后面要讲的黑名单提供的权限控制。

5 黑白名单配置

针对上的权限粒度略显粗略的限制（事实上大多数场景下 DML 的权限也已经足够了），DBLE 提供黑白名单

的功能，白名单就是只允许白名单的连接，而黑名单则是详细的针对已经通过白名单的连接的权限控制，黑名单更类似于 SQL 审计，黑名单配置以 firewall 分隔开来，典型配置如下：

```
<firewall>
  <whitehost>
    <host host="127.0.0.1" user="adv_user"/>
    <host host="127.0.0.1" user="admin"/>
  </whitehost>
  <blacklist check="true">
<property name="selectAllow">false</property>  <!-- 允许 select-->
<property name="insertAllow">false</property>  <!-- 允许 insert-->
<property name="updateAllow">false</property>
.....
  </blacklist>
</firewall>
```

对于黑白名单，需要注意：

1. **DBLE 针对来源 IP 和用户名进行限制，放行白名单连接，对于不在白名单列之中的连接，统统拒绝而无法登陆。**白名单中的来源 ip, 只能指定固定 IP，暂不支持 MySQL “%” 类似的 ip 通配符。想象一种场景，需要像 MySQL 一样指定一个 ip 范围能允许连接 DBLE，这时只能一行一行增加允许 ip 了，此处略显笨拙。
2. **如果配置了黑名单，则再根据黑名单中的权限 property 来进一步限制白名单中的用户权限。**例如是否允许查询，是否允许 INSERT，是否允许 UPDATE。可以看出 DBLE 中黑名单更像是对用户的权限审计，DBLE 在黑名单中将上面只能粗糙地限制到 DML 权限的用户配置做了较多较细的扩展，这样权限粒度更小，从实际场景来说，这更符合生产需要，由此我们可以针对性地去掉一些危险的 SQL。

6 总结

本文简单介绍了 server.xml 中的三个重要的配置段落，分别是 DBLE 的系统配置，用户配置以及黑白名单功能，针对用户配置则介绍了实际应用场景下的配置以及对应的 DML 权限配置，并详细介绍了 DBLE 黑白名单配置实践。



深度分析 MyCat 与 DBLE 的对比性能调优

作者：阎虎青

问题起因：

用 benchmarksql_for_mysql 对原生 MyCat-1.6.1 和 DBLE-2.17.07 版做性能测试对比，发现 DBLE 性能只到原生版 MyCat 的 70% 左右。

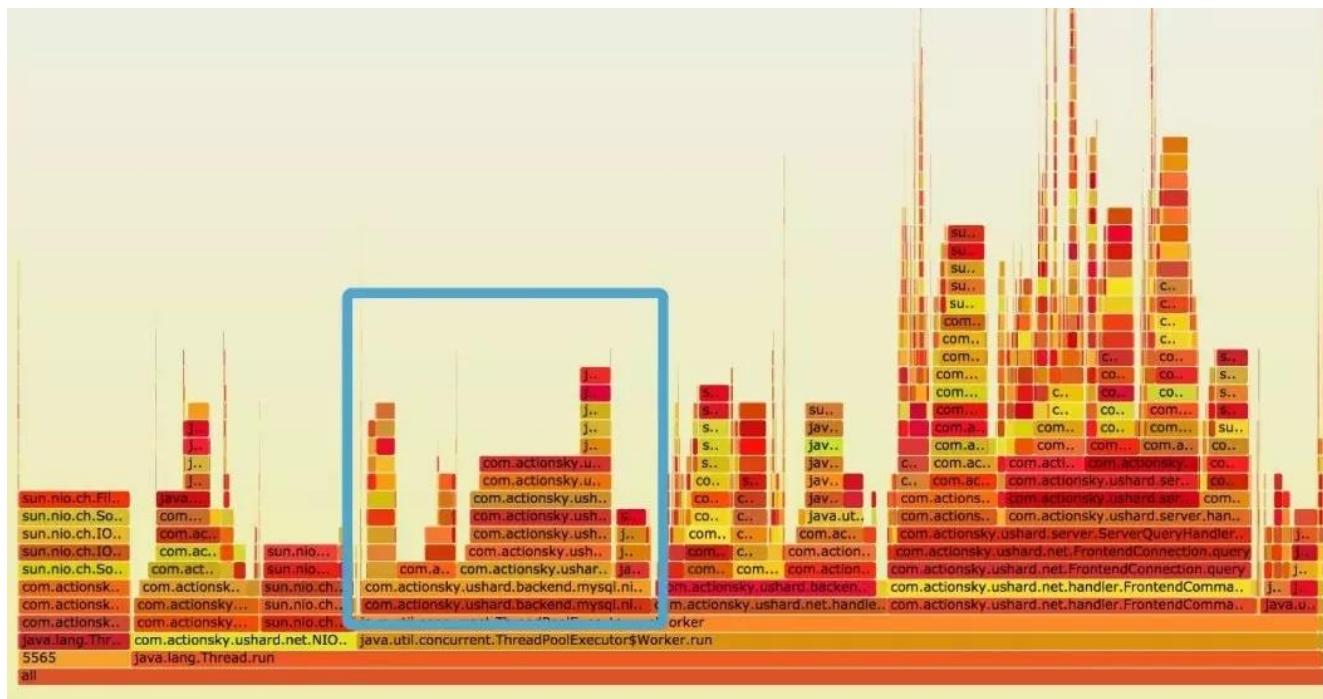
问题分析过程：

分析过程主要有以下内容：包括现象，收集数据，分析猜测原因，验证猜测的方式来进行。

1 分析瓶颈

1.1 先对两者进行一个 CPU 占用堆栈分析

通过对 CPU 火焰图的比较，发现 DBLE 用在纯排序上的 CPU 占用在 15% 以上，而 MyCat 在排序上没有看到明显的 CPU 占用。（复盘时的思考：这里有明显的可疑之处，应当及早观察两者是否公平）



1.2 首先猜测可能的原因

1. 由于 MyCat 对以下这条用例实现有 bug：具体方式是直接原句下发 SQL 到节点，收到各个节点的结果后直接做加法；而 DBLE 则是改写为 `select distinct s_i_id` 收集全部结果集，然后在中间件做去重和统计的工作。所以两者在这个 case 上的对比是不公平的。

```
//第二段查询代码
if (stockGetCountStock2 == null) {
    stockGetCountStock2 = conn
        .prepareStatement("SELECT COUNT(DISTINCT (s_i_id)) AS stock_count"
            + " FROM stock"
            + " WHERE s_w_id = ?"
            + " AND s_quantity < ?"
            + " AND s_i_id IN (" + in_list + ")");
```

2. 排序本身算法选择的问题。

1.3 对猜测原因的验证

1. 去除有 bug 的 case，并未看到性能有提升，而且考虑这条用例在所有用例出现的概率只有 4%，涉及到的数据也不多，所以应该不是性能问题的主因。
2. 去除有排序的 case，看到两者性能接近，确定是排序的问题。

2 猜测原因

2.1 猜测一：源码实现原因

猜测：梳理 DBLE 源码排序逻辑的实现细节，是多路归并的排序，理论上是最优选择。

实际上具体的实现上有可优化的空间。如下图，N 个数的 K 路排序的初始化值理论最优复杂度是 $O(N)$ ，而这里变成了 $O(N \times \log K \times 2)$

```
private int cmp(RowDataPacket o1, RowDataPacket o2, int index) {
    HandlerTool.initFields(sourceFields, o1.fieldValues);
    List<byte[]> bo1 = HandlerTool.getItemListBytes(cmpItems);
    HandlerTool.initFields(sourceFields, o2.fieldValues);
    List<byte[]> bo2 = HandlerTool.getItemListBytes(cmpItems);
    boolean isAsc = ascsc.get(index);
    Field field = cmpFields.get(index);
    byte[] b1 = bo1.get(index);
    byte[] b2 = bo2.get(index);
    int rs;
    if (isAsc) {
        rs = field.compare(b1, b2);
    } else {
        rs = field.compare(b2, b1);
    }
    if (rs != 0 || cmpFields.size() == (index + 1)) {
        return rs;
    } else {
        return cmp(o1, o2, index: index + 1);
    }
}
```

验证：为了快速将排序的因素排除，将 cmp 函数直接返回 1 再次做测试。结果提升了 10% 的性能，所以虽然 cmp 是有性能问题，但导致性能如此大还有其他原因。（复盘：新版本已优化此处 10% 的性能差异）

2.2 猜测二：回到排序 SQL

查看 B-SQL 源码，有 3 个排序 SQL，其中有 2 个排序 SQL 的排序列不在 select 项中，这本来应该引发 MyCat 的 bug 的，但我们在返回集和抓包中都没有发现。再仔细阅读源码，原来 B-SQL 通过 hard code 的方式使得压力永远跑不到这两个代码路径上，这样我们又排除了 2 个干扰因素，问题集中到剩下的那个排序上了。

将排序除去，64 数据量，64 并发，DBLE 的性能是 MyCat 的 96%。**证明确实和排序有关！**

3 分析多并发压力排序性能的原因

3.1 猜测排序算法在特殊场景下的适用性

猜测：由于 MyCat 排序采用的是 timsort，时间复杂度的可能最优是 $O(n)$ 。而 DBLE 的多路归并排序在 B-SQL 这个场景下时间复杂度最差情况是 $O(n*(k-1))$ 。猜测 timsort 排序在 B-SQL 多并发场景下可能会优于多路归并。

验证：用 B-SQL 压测并统计函数调用次数。

结论：在 B-SQL 场景下

1. 两者平均每个排序调用的 cmp 函数的次数并没有发生明显的异化。
2. 每个排序 cmp 次数虽然没有大的差异，但总的调用次数却相差很大，DBLE 大约是 MyCat 的 5 倍。

4 分析 DBLE 排序时 cmp 函数次数调用多的原因

问题集中在了为什么 DBLE 会有更多次的比较函数调用。接下来我们验证压力下发的 SQL 是否与 cmp 函数调用相符，是否下发的 SQL 就不公平？

4.1 收集数据

1. 用抓包的方式分别抓取 B-SQL 发给 MyCat 和 DBLE 的包，结果发现 DBLE 的所有 SQL 中排序这条 SQL 的发生次数是 MyCat 的 10 倍左右。
2. 再次用 yourkit 查看调用次数和 CPU 分布验证，发现调用次数确实符合抓包的结论，CPU 分布也是 DBLE 分了大量的时间用于排序，而 MyCat 对排序的分配几乎可以忽略。这也与最一开始的火焰图结论一样。
3. 用 wireshark 分析 DBLE 抓包结果，发现某些连接执行一段时间之后大量的重复出现排序+delete 的 query 请求直到压力结束，举例如下图。


```

146 Request Query { DELETE FROM new_order WHERE no_d_id = 1 AND no_w_id = 29 AND no_o_id = 557 }
156 Request Query { SELECT no_o_id FROM new_order WHERE no_d_id = 1 AND no_w_id = 29 ORDER BY no_o_id AS
100 Request Query { select @@session.tx_read_onl }
146 Request Query { DELETE FROM new_order WHERE no_d_id = 1 AND no_w_id = 29 AND no_o_id = 557 }
156 Request Query { SELECT no_o_id FROM new_order WHERE no_d_id = 1 AND no_w_id = 29 ORDER BY no_o_id AS
100 Request Query { select @@session.tx_read_onl }
146 Request Query { DELETE FROM new_order WHERE no_d_id = 1 AND no_w_id = 29 AND no_o_id = 557 }
100 [TCP Previous segment not captured] Request Query { select @@session.tx_read_onl }
146 Request Query { DELETE FROM new_order WHERE no_d_id = 1 AND no_w_id = 29 AND no_o_id = 557 }
156 Request Query { SELECT no_o_id FROM new_order WHERE no_d_id = 1 AND no_w_id = 29 ORDER BY no_o_id AS
100 Request Query { select @@session.tx_read_onl }
146 Request Query { DELETE FROM new_order WHERE no_d_id = 1 AND no_w_id = 29 AND no_o_id = 557 }
156 Request Query { SELECT no_o_id FROM new_order WHERE no_d_id = 1 AND no_w_id = 29 ORDER BY no_o_id AS
100 Request Query { select @@session.tx_read_onl }
146 Request Query { DELETE FROM new_order WHERE no_d_id = 1 AND no_w_id = 29 AND no_o_id = 557 }
156 Request Query { SELECT no o id FROM new order WHERE no d id = 1 AND no w id = 29 ORDER BY no o id AS

```

4.2 分析原因

分析 B-SQL 源码这里发现只有 delete 的数据为 0 才会引发死循环。

```

do {
    no_o_id = -1;
    if (delivGetOrderId == null) {
        delivGetOrderId = conn
            .prepareStatement("SELECT no_o_id FROM new_order WHERE no_d_id = ?"
                + " AND no_w_id = ?"
                + " ORDER BY no_o_id ASC");
    }

    delivGetOrderId.setInt(1, d_id);
    delivGetOrderId.setInt(2, w_id);

    rs = delivGetOrderId.executeQuery();
    if (rs.next()) {
        no_o_id = rs.getInt("no_o_id");
    }

    orderIDs[(int) d_id - 1] = no_o_id;
    rs.close();
    rs = null;
    delivGetOrderId.close();
    delivGetOrderId = null;

    newOrderRemoved = false;
    if (no_o_id != -1) {
        new_order.no_o_id = no_o_id;

        if (delivDeleteNewOrder == null) {
            delivDeleteNewOrder = conn
                .prepareStatement("DELETE FROM new_order"
                    + " WHERE no_d_id = ?"
                    + " AND no_w_id = ?"
                    + " AND no_o_id = ?");
        }

        delivDeleteNewOrder.setInt(1, d_id);
        delivDeleteNewOrder.setInt(2, w_id);
        delivDeleteNewOrder.setInt(3, no_o_id);

        result = delivDeleteNewOrder.executeUpdate();
        if (result > 0) {
            newOrderRemoved = true;
        }
        delivDeleteNewOrder.close();
        delivDeleteNewOrder = null;
    }
} while (no_o_id != -1 && !newOrderRemoved
    && !stopRunningSignal);

```

4.3 验证测试

在引发死循环的原因找到之前，先修改代码验证测试。无论 result 是否是 0 都设置 newOrderRemoved=true 使得 B-SQL 跳出死循环。

验证测试，DBLE 性能终于符合预期，变为 MyCat 的 105%。

至此，B-SQL 有排序引发 DBLE 性能下降的原因找到了，某种场景下 B-SQL 对 DBLE 执行 delete，影响行数为 0，导致此时会有死循环，发送了大量排序请求，严重降低了 DBLE 性能，并且并发压力越大越容易出现，但也有一定几率不会触发。

5 分析哪种场景下 delete 行数为 0

5.1 隔离级别测试

因为对隔离级别并不熟悉，花了很长时间才想到原因，在 MySQL 上做了一个实验。

前提条件-隔离级别为 RR:

```
root@localhost [(none)]>show variables like 'tx_isolation';
+-----+-----+
| Variable_name | Value                |
+-----+-----+
| tx_isolation  | REPEATABLE-READ     |
+-----+-----+
1 row in set (0.01 sec)
```

SESSION 1:

```
root@localhost [mycat_test]>set autocommit =0;
Query OK, 0 rows affected (0.00 sec)

root@localhost [mycat_test]>select * from sharding_four_node limit 10;
+----+-----+-----+
| id | c_flag | c_decimal |
+----+-----+-----+
| 103 | 103    | 103.0000 |
| 107 | 107    | 107.0000 |
| 111 | 111    | 111.0000 |
| 115 | 115    | 115.0000 |
| 119 | 119    | 119.0000 |
| 123 | 123    | 123.0000 |
| 127 | 127    | 127.0000 |
| 131 | 131    | 131.0000 |
| 135 | 135    | 135.0000 |
| 139 | 139    | 139.0000 |
+----+-----+-----+
10 rows in set (0.00 sec)
```

SESSION 2:

```
root@localhost [mycat_test]>delete from sharding_four_node where id =103;
Query OK, 1 row affected (0.00 sec)
```

```
root@localhost [mycat_test]>select * from sharding_four_node limit 10;
+----+-----+-----+
| id | c_flag | c_decimal |
+----+-----+-----+
| 103 | 103    | 103.0000 |
| 107 | 107    | 107.0000 |
| 111 | 111    | 111.0000 |
| 115 | 115    | 115.0000 |
| 119 | 119    | 119.0000 |
| 123 | 123    | 123.0000 |
| 127 | 127    | 127.0000 |
| 131 | 131    | 131.0000 |
| 135 | 135    | 135.0000 |
| 139 | 139    | 139.0000 |
+----+-----+-----+
10 rows in set (0.00 sec)

root@localhost [mycat_test]>delete from sharding_four_node where id =103;
Query OK, 0 rows affected (0.00 sec)
```

也就是说，在并发情况下确实有可能有死循环出现。

5.2 分析为什么只有 DBLE 上有这个问题存在 MyCat 上却没有

原因是 DBLE 和 MyCat 的默认隔离级别都是 REPEATED_READ，但 MyCat 的实现有 bug，除非客户端显式使用 set 语句，MyCat 后端连接使用的隔离级别都是下属结点上的默认隔离级别；而 DBLE 会在获取后端连接后同步上下文，使得 session 级别的隔离级别和 DBLE 配置相同。而后端的四个结点中除了 1 台是 REPEATED_READ，其他三个结点都是 READ_COMMITTED。这样同样的并发条件，DBLE 100% 会触发，而 MyCat 只有 25% 的概率触发。

5.3 验证测试

将 DBLE 上的配置添加 `<property name="txIsolation">2</property>`（默认是 3）与默认做对比。性能比是 1:0.75。符合期望，性能原因全部找到。

6 吐槽

最后吐槽一下 B-SQL，找了官方的 B-SQL 4.1 版和 5.0 版，4.1 版并未对此情况做任何改进，仍有可能陷入死循环影响测试。而 5.0 的对应代码处有这么一段注释，不知道 PGSQL 是否这里真的会触发异常，但 MySQL 并不会触发异常，仍有可能陷入死循环。

```
/*
 * Failed to delete the NEW_ORDER row. This is not
 * an error since for concurrency reasons we did
 * not select FOR UPDATE above. It is possible that
 * another, concurrent DELIVERY_BG transaction just
 * deleted this row and is working on it now. We
 * simply got back and try to get the next one.
 * This logic only works in READ_COMMITTED isolation
 * level and will cause SQLExceptions in anything
 * higher than that.
```

7 性能原因回顾

1. cmp 函数时候初始化值的问题，影响部分性能，非主要性能瓶颈，新版本已改进。
2. 同时触发了 MyCat 和 B-SQL 的两个 bug，导致测试的性能数据负负得正：
 - ☐ Mycat bug：配置的隔离级别不生效问题
 - ☐ B-SQL bug：RR 隔离级别下，delete 死循环问题

需要将 MySQL 结点都改为 READ_COMMITTED，再将配置改为：`<property name="txIsolation">2</property>`，避开上述的 bug。

8 收获

1. 测试环境的搭建无论是配置参数还是各个节点的状态都要同步，保证公平。
2. 性能分析工具的使用。
3. 性能测试可能一次的结果具有偶然性，需要多次验证。
4. 当有矛盾的结论时候，可能就快接近问题的真相了，需要持续关注。



基于分布式中间件的 SQL 改造指南

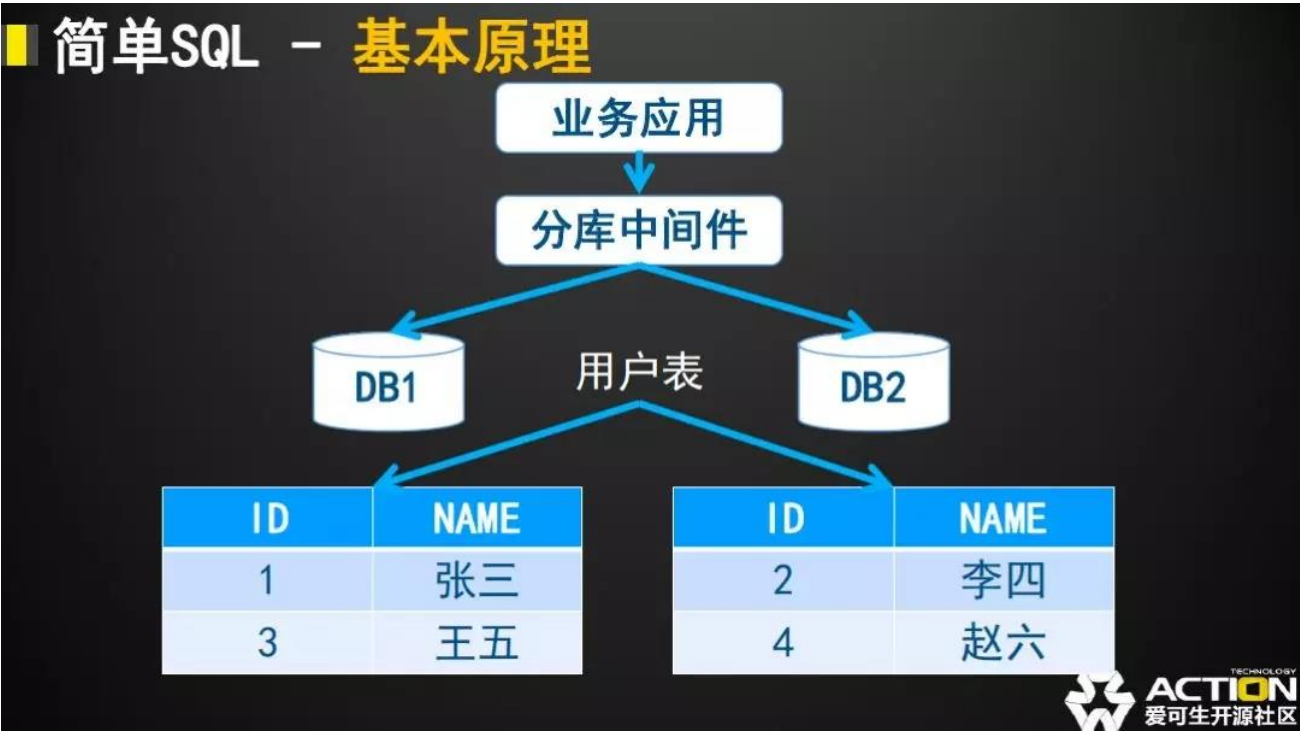
作者：孙正方

1 中间件中的简单 SQL

1.1 简单 SQL 的分类

在一般 SQL 中，一个单表查询的 SQL 往往是最简单的，而在中间件中也没什么区别，中间件中将一个对于单表的字段查询作为最简单的 SQL 进行处理。

1.2 中间件对于简单 SQL 的处理方式

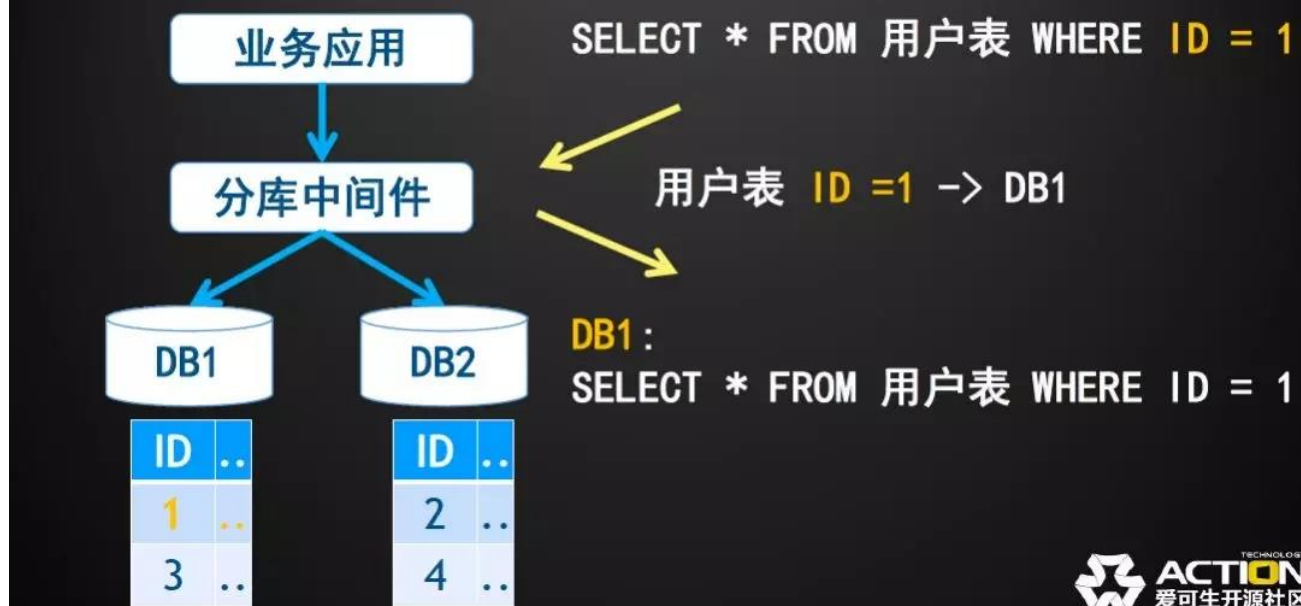


中间件会按照在配置文件中既定的数据拆分的设置，对于“用户表”这个表格的数据进行拆分，将每条通过中间件插入的数据按照规则进行分布，图上给的例子是按照“用户表”的 ID 奇偶性进行了数据的拆分。

有了合理的数据分布之后就是中间件是如何在需要的时候取用数据，在中间件中，对于基础 SQL，也就是单表的数据查询分为一下两个情况：

1. **带有分片信息的数据查询：**下图展示了一个例子，在单表查询的过程中，如果存在影响数据分布的查询条件(在此例子中为用户的 ID=1)

■ 简单SQL - 基本原理



在查询的简单 SQL 附带有分片条件时，中间件根据 SQL 语义解析的结果，根据这个分片条件对于数据可能存在的 DB 进行计算，最终把 SQL 转发到对应的 DB 中进行执行，查询出最终的正确结果。

2. 不带有分片信息的数据查询（广播）：当查询的 SQL 不带有分片条件时，中间件只好把 SQL 内容发送到表格涉及到的所有 DB 中，并从每个 DB 中返回数据并整合。

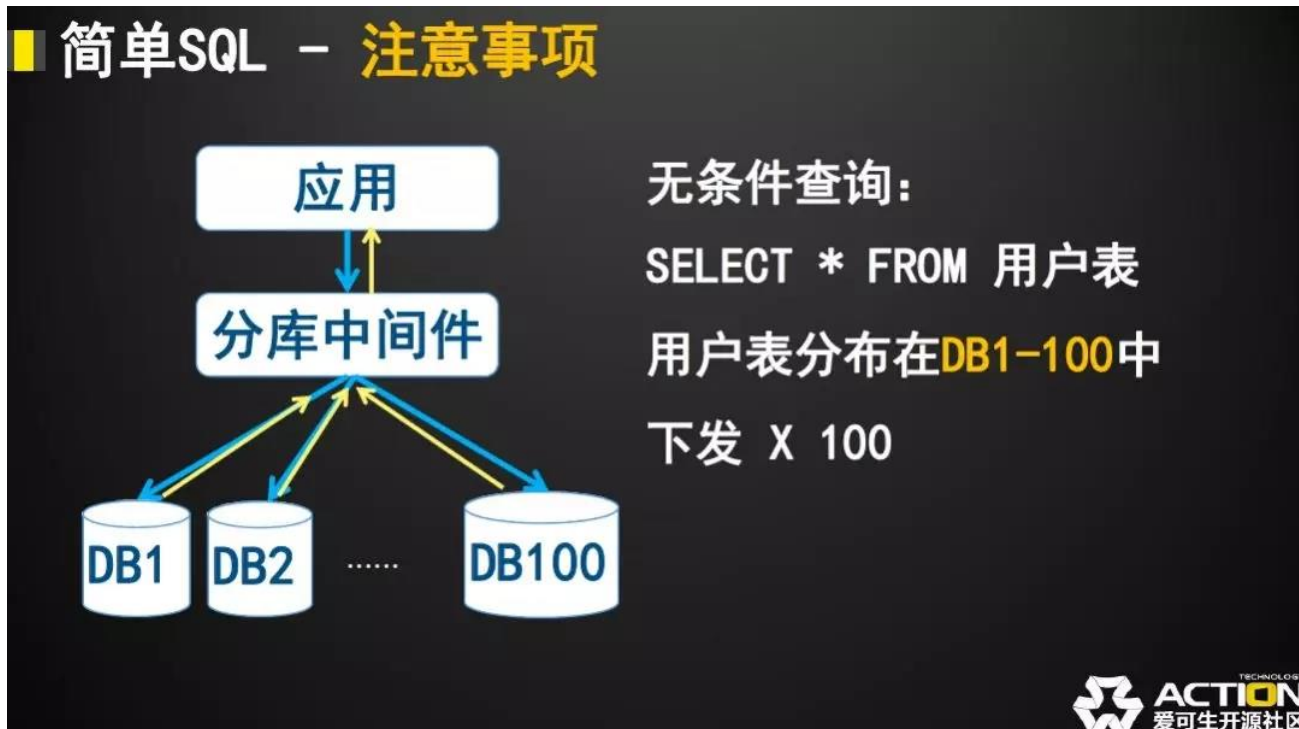
■ 简单SQL - 广播



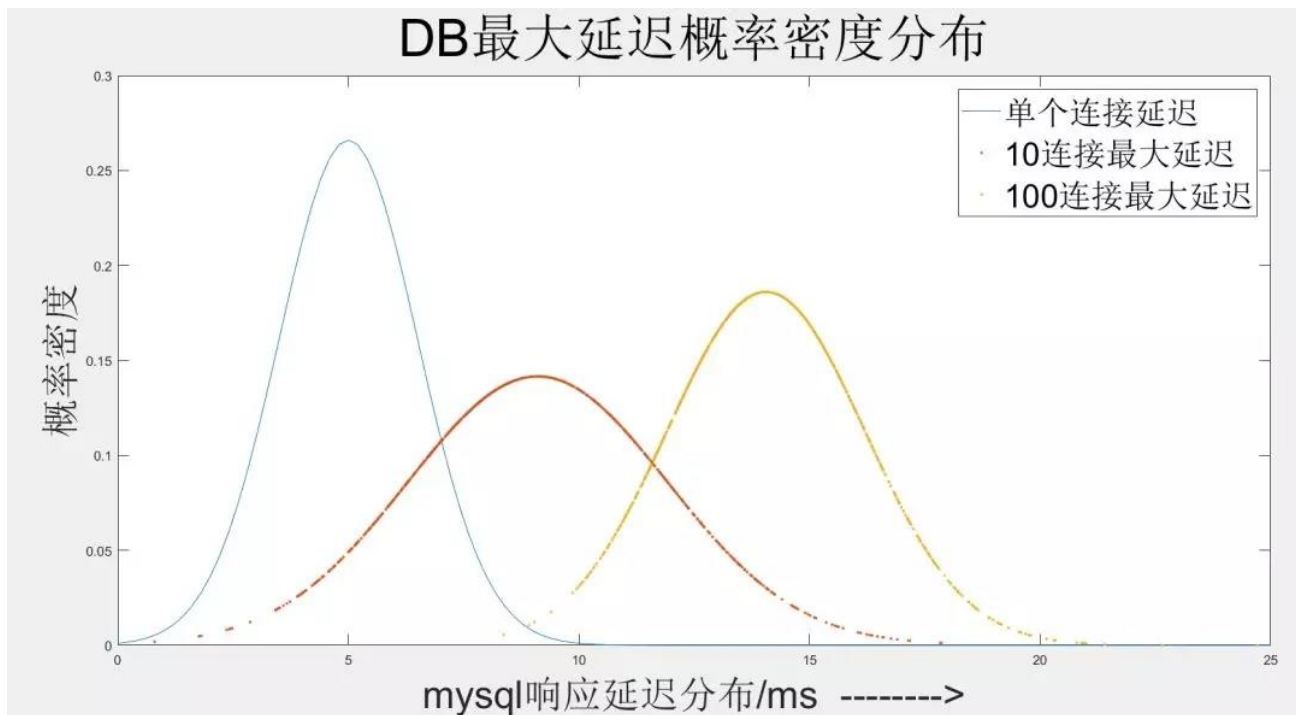
在这个过程中就存在一个现象，即“这一个查询需要等待最慢的那个 DB 查询结束”。

1.3 简单 SQL 的注意事项

上文中有提及广播类型的简单 SQL 存在一个特征“需要等待最慢的那个 DB 查询结束”，当分片数量持续上升的时候，如下图所示的场景，需要等待的连接数量会持续变多。



可以通过概率的方法分析下当连接数量增长的时候会发生什么变化：



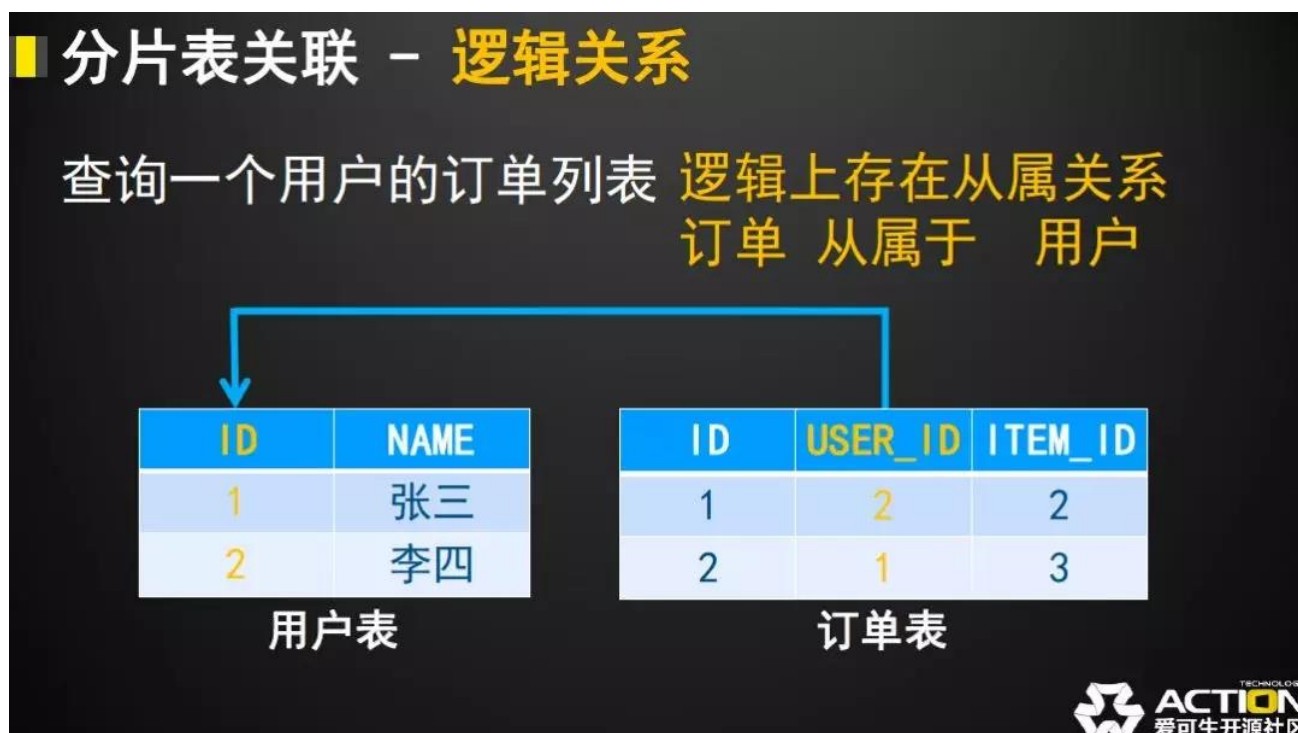
通过 Matlab 对于中间件广播查询的场景进行了如下的模拟，可以发现在随着需要等待连接数量的上升，整体的中间件响应速度会有一个成倍的下降，从单个连接平均 5ms 的查询延迟一直上升到 100 连接时候的接近 15ms 的整体延迟。

针对以上的场景，提出在中间件中进行单表查询的时候，尽量降低单个 SQL 涉及到的后端 DB 数量，具体方法是在 SQL 中给表格添加分片列的限定条件，就像 `select * from 用户表 where ID = 1` 中的这个条件 `ID = 1`。

2 中间件中的 ER 标格 JOIN

2.1 中间件中 ER 关系的定义

单表查询不能覆盖应用对于 SQL 的需求，中间件中就引入了 ER 关系的概念。ER 关系指的是两个表格在逻辑上就拥有的从属关系，如下图中所示的用户表和订单表：

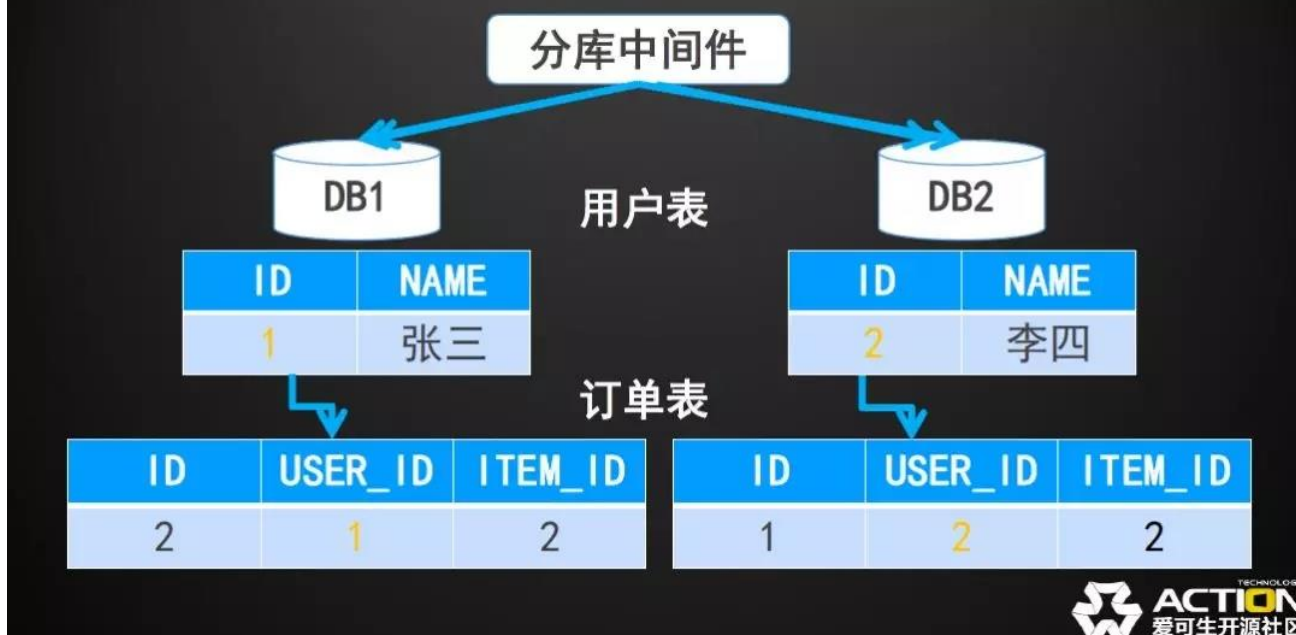


当两个表或者更多的表格存在这样逻辑上的从属关系的时候，在中间件中把这些关系定义为 ER 关系，并且当一个查询语句有且仅有存在 ER 关系的表格关联组成的时候，这个 SQL 被称为“ER 表格 JOIN”。

2.2 中间件对于 ER 表格 JOIN 的处理方式

当两个表格能够被定义为 ER 关系的时候，中间件能够按照之间的关联关系组织数据的存储，最后的存储方式如下图所示：

■ 分片表关联 - 存储分布



可以看到大致的结果就是子表（订单表）的数据会被存放在父表（用户表）对应数据的 DB 节点。

当出现这种数据分布的时候，就可以在 DB1 中查询到上例中张三的用户和他的所有数据，所以中间件就能直接把 SQL 语句下发到对应节点进行执行。

2.3 ER 关联 SQL 的注意事项

可以看到 ER 的这种查询方式是有一定的局限的，需要把查询的表格声明为 ER 关系，并且 ER 关系还是单向的，就是一个表只能是某一个表的子表，一个分片表不能拥有两个父表。

其次就是由于这个 ER 关联的查询处理是和上文的简单查询在本质上是一样的，就是查询 SQL 下发到 DB 中来执行，所以在注意事项上也是一致的，尽量在使用的时候提供能够确定分片的限定条件。

3 中间件中的跨库表格 JOIN

3.1 跨库表格 JOIN 的定义

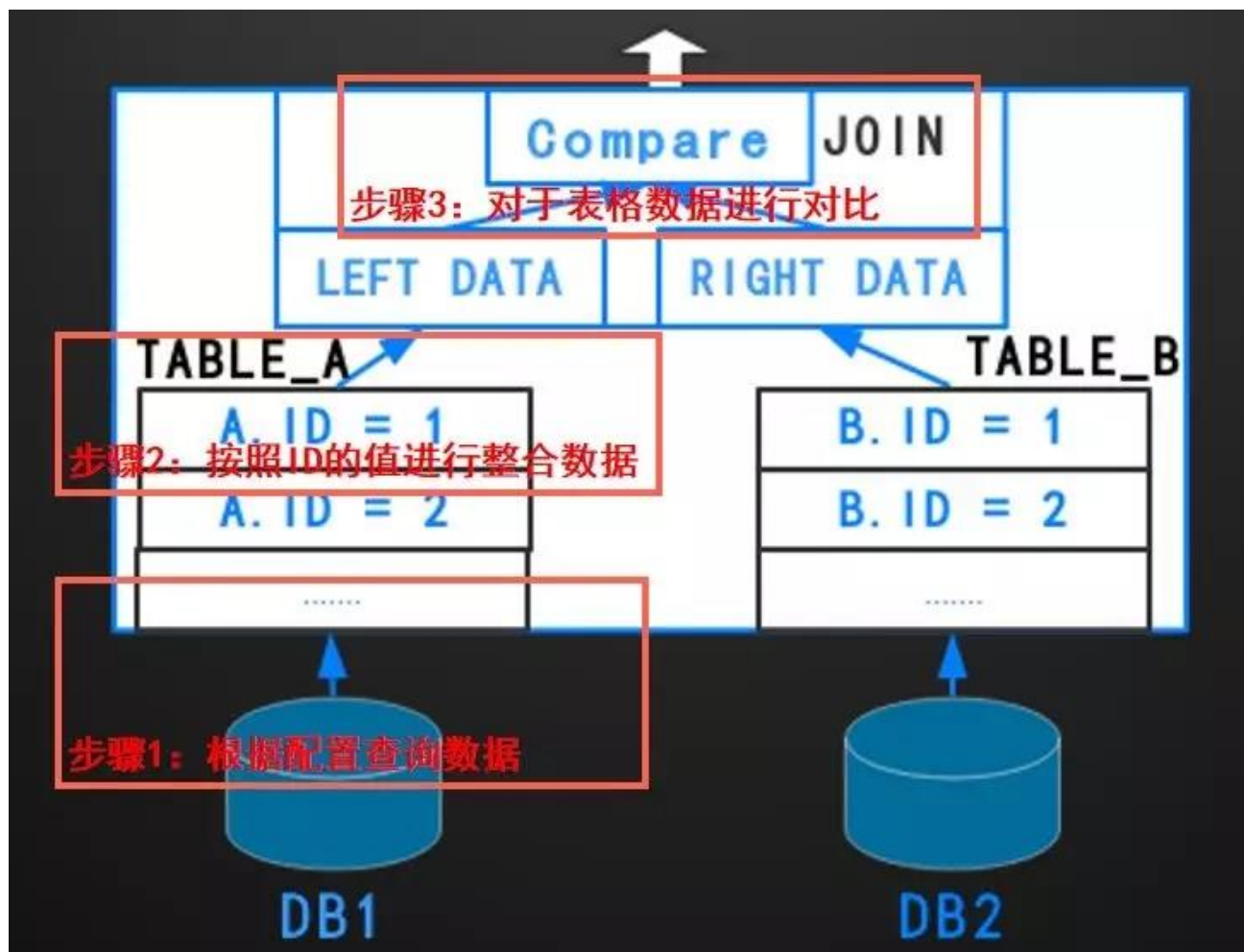
事实上上文所有的查询方法限制都是非常大的，但是往往应用的 SQL 无法满足于被限制在这么局限的范围内，那中间件中就有另一种机制来实现更广泛意义上的 SQL 执行，跨库表格关联查询。

当我的 SQL 内容无法被定义进入上文的两个“简单 SQL”或者是“ER SQL”的时候，中间件就会使用这种跨库表格 JOIN 的方式来处理这个问题，当然并非所有的中间件都有实现对应的功能，在这里仅讨论实现的部分中间件的实现逻辑。

3.2 中间件中跨库表格 JOIN 的执行

以下是在中间件中执行跨库表查询的这么一个大致执行逻辑图，图中可以看到整体的执行逻辑是从每个

DB 中分别取 TABLEA 和 TABLEB 的数据，最终在中间件中进行数据的整理组织 and 对比：



具体的跨库查询的执行过程可以被归纳为以下的几个步骤：

1. 根据配置文件中对于 TABLEA 和 TABLEB 的描述将表格的数据分别从 DB 中进行查询
2. 对于查询得到的表格数据进行整理，分别按照 ID 的值（关联列的值）进行排序和归档
3. 对于归档完成的数据进行 ID 值的对比，计算关联结果

在这个过程中会产生几个方面的资源消耗：

1. 组织数据产生的临时内存存储消耗
2. 对比数据产生的 CPU 消耗
3. 查询每一个存在 DB 中的表数据产生的连接数以及网络消耗

另一个方面就是中间件由于不能存储真实的数据，所以无法像 MySQL 优化器一样提前计算不同表格的聚合代价，这么一来，中间件就直接按照 SQL 中原有的表格聚合顺序进行数据的聚合和整理，这可能带来以下的 SQL 执行的计划。

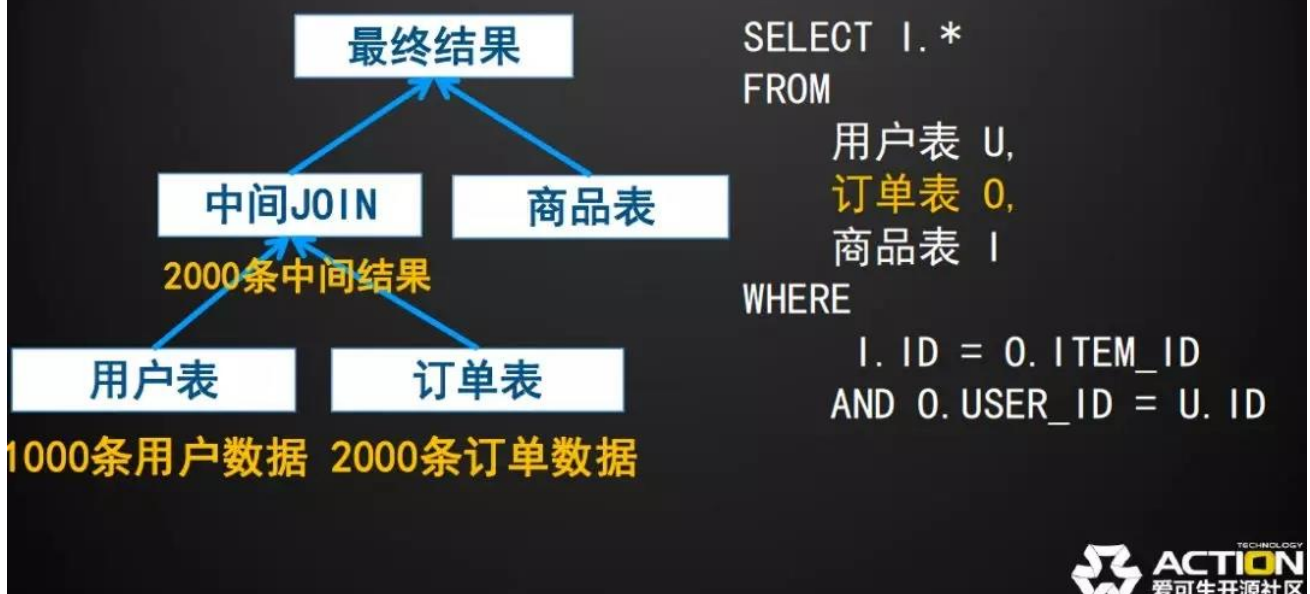
跨库表关联 - 亲和性



案例中用户表由于表格顺序的问题先和没有直接关系的商品表进行了表关联, 这样一来这里的中间结果就出现了一个笛卡尔积, 也就是无条件无字段的关联结果, 其结果为双边数据的乘积: $1000 \times 1000 = 1000000$

如果 SQL 中的表格顺序进行进一步的优化, 则可以在同样的 SQL 中获得下图中的执行顺序。

跨库表关联 - 亲和性



这次我们优化完成之后的查询中间结果将由用户表和有关系的订单表优先聚合完成, 因为他们之间的关系, 最多存在 2000 条中间结果, 相比与上例中的 1000000 条来说, 整个 SQL 执行效率上得到了很大的优化。

3.3 跨库 JOIN 的注意事项

针对以上的几种消耗类型，可以通过以下的几个手段进行降低消耗的量以获取最好的执行效果：

1. 添加足够的限定条件
2. 使用重复率低的列进行关联
3. 通过手动调整 SQL 中表格顺序的手段来进行执行计划的调整

4 通篇总结

从 DB 存储到真正成熟的分布式 DB，分布式中间件的架构是整个过程中的重要一环，在分布式架构的过程中，对 SQL 的规范和使用存在一些特别要求，应用在向这方面进行改造和适应的过程中需要更多的考虑到中间件架构带来的限制及相关注意事项。整体归纳可分为以下内容：

1. **中间件中尽量使用分片条件进行数据的查询**，不论整个 SQL 是属于简单 SQL，ER 查询或者是复杂查询。
2. **在复杂查询的状态下，优化方向为中间件中处理的数据尽量少**，此目标可以通过给表格添加限定条件，选择更加重复率低的关联列以及调整 SQL 中的表格关联顺序来达成。



开源分布式中间件 DBLE 容器化快速安装部署

作者：孙正方

1 关于本节

1. 如何快速使用 DBLE 的 docker-compose 文件来启动一个 DBLE 的 quick start。
2. 一个按照自定义的配置和 SQL 脚本来启动 DBLE quick start 的用例。

2 安装依赖

1. 安装 docker
2. 安装 docker-compose
3. 安装 MySQL 连接工具，用于进行连接测试观察结果

3 安装过程

从 DBLE 项目中下载最新的 docker-compose.yml 文件：

<https://raw.githubusercontent.com/actiontech/dble/master/docker-images/docker-compose.yml>

使用 docker-compose up 命令直接启动 dble-server，compose 配置文件会从 dockerhub 拉取镜像并最终启动 DBLE。下载 docker-compose.yml，进入对应目录：

```
docker-compose up -d
```

注意：一个创建的 3 个容器：其中两个为 MySQL 容器，它们将端口 3306 映射到 Docker 宿主机的端口 33061 和 33062；dble 容器中端口 8066 和 9066 映射到 Docker 主机上的相同端口；

4 连接并使用

使用准备好的 MySQL 连接工具连接主机的 8066 或者 9066 端口，在 docker 的默认配置中：

1. 8066 端口(服务端口能够执行 SQL 语句)的用户为 root/123456
2. 9066 端口(管理端口能够执行管理语句)的用户为 man1/654321

```
#connect dble server port
mysql -P8066 -u root -p123456 -h 127.0.0.1
#connect dble manager port
mysql -P9066 -u man1 -p123456 -h 127.0.0.1
#connect mysql1
mysql -P33061 -u root -p123456 -h 127.0.0.1
#connect mysql2
mysql -P33062 -u root -p123456 -h 127.0.0.1
```

请在 `dbtle-server` 容器中查阅 `/opt/dbtle/conf/schema.xml` 文件。

5 环境清理

使用完成或者进行环境重建的时候使用 `docker-compose stop/down` 来进行环境的清理：

```
docker-compose stop/down
```

6 使用自定义配置启动 DBLE

在这里介绍下如何使用自定义的 DBLE 本地配置来启动 `docker-compose` 中的 DBLE。首先，简单描述下默认 `docker-compose.yml` 执行的过程，`dbtle-compose` 挨个启动容器，并且在最终启动 `dbtle-server` 的时候调用。

存在于镜像 `actiontech/dbtle:latest` 目录下的 `/opt/dbtle/bin/wait-for-it.sh` 脚本等待指定的（默认为 MySQL1 容器的 MySQL 服务端点）TCP 端口启动；

待到指定 TCP 端口启动之后调用初始化脚本 `/opt/dbtle/bin/docker_init_start.sh` 对于 `dbtle-server` 这个容器进行初始化（替换配置文件，启动 DBLE，执行 SQL 文件）。

下面是默认的 `docker_init_start.sh` 脚本：

```
#!/bin/sh
echo "dbtle init&start in docker"
sh /opt/dbtle/bin/dbtle start
sh /opt/dbtle/bin/wait-for-it.sh 127.0.0.1:8066
mysql -P9066 -u man1 -h 127.0.0.1 -p654321 -e "create database @@dataNode
='dn1,dn2,dn3,dn4'"
mysql -P8066 -u root -h 127.0.0.1 -p123456 -e "source /opt/dbtle/conf/testdb.sql"
testdb
echo "dbtle init finish"
/bin/bash
```

可以轻易的看出脚本的内容非常简单，启动 `dbtle`→等待 DBLE 服务启动→执行 DBLE 管理命令 `create database` 在后端 MySQL 数据库中创建 `database`→执行初始化 SQL 脚本在 DBLE 中创建表和插入初始化数据。

所以当需要使用非默认的情况进行启动 `dbtle-server` 容器时需要一以下几个基本步骤：

1. 需要按照以上的模块化步骤编写一个自己的初始化 `sh` 文件，然后照着需要的步骤对于 DBLE 进行启动和初始化。
2. 在 `dbtle-server` 的启动配置中添加 `volumes` 项，将 linux 本地的配置文件和初始化文件存放目录挂载至 `dbtle-server` 容器内部（注意 DBLE 本地放在 `/opt/dbtle` 下）。
3. 修改 `dbtle-server` 的启动命令，将初始化脚本修改为自定义的容器内部的初始化脚本。

7 举例

docker-compose.yml 修改如下部分:

```

dble-server:
  image: actiontech/dble:latest
  container_name: dble-server
  hostname: dble-server
  privileged: true
  stdin_open: true
  tty: true
  volumes:
    - .:/opt/init/
  command: ["/opt/dble/bin/wait-for-it.sh", "backend-mysql1:3306", "--", "/opt/init/customized_script.sh"]
  ports:
    - "8066:8066"
    - "9066:9066"
  depends_on:
    - "mysql1"
    - "mysql2"
  networks:
    net:
      ipv4_address: 172.18.0.5

```

对应本地目录./存在以下文件:

schema.xml rule.xml server.xml init.sql customized_script.sh

脚本 customized_script.sh 中的内容为:

```

#!/bin/sh
echo "dble init&start in docker"
cp /opt/init/server.xml /opt/dble/conf/
cp /opt/init/schema.xml /opt/dble/conf/
cp /opt/init/rule.xml /opt/dble/conf/
sh /opt/dble/bin/dble start
sh /opt/dble/bin/wait-for-it.sh 127.0.0.1:8066
mysql -P9066 -u man1 -h 127.0.0.1 -p654321 -e "create database @@dataNode
='dn1,dn2,dn3,dn4'"
mysql -P8066 -u root -h 127.0.0.1 -p123456 -e "source /opt/init/init.sql" testdb
echo "dble init finish"
/bin/bash

```

以上是快速使用 DBLE 的 docker-compose 文件来启动一个 DBLE 的 quick start 的描述说明及具体举例。



DBLE 如何实现 Prepared Statement 协议

作者：鲍凤其

DBLE 是一款企业级的开源分布式中间件，江湖人送外号 “MyCat Plus”。Prepared Statement 协议是 MySQL 5.1 版本新加入的功能。MyCat 从 1.6 版本实现了 Prepared Statement 协议，但 MyCat 存在一些至今仍未修复的 Bug。本文从两名 DBLE 用户提交的 Bug 开始说起，详细解读 DBLE 是如何实现 Prepared Statement 协议的。

1 事发当天

2019 年 4 月 12 日下午，GitHub 得到举报，两名 DBLE 用户各发现了一个极为凶残的 Bug。DBLE 社区片儿警马上赶到案发现场进行取证并对 Bug 们开始展开调查。举报信息如下：

BugOne (<https://github.com/actiontech/dble/issues/1122>)

error message:

```
Dble has an error message 'unknown pstmtId when executing' when the client set useServerPrepStmts=true #1122
```

dble version:

```
dble-9.9.9.9-884fc6b612d64cc22101226536f8fd1d24580857-20190221182143
```

BugTwo (<https://github.com/actiontech/dble/issues/1124>)

error message:

```
use PreparedStatement with JDBC and MySQL J Connector will get wrong result when useCursorFetch=true #1124
```

dble version:

```
dble 2.18.10.5 and before (not yet test on later version)
```

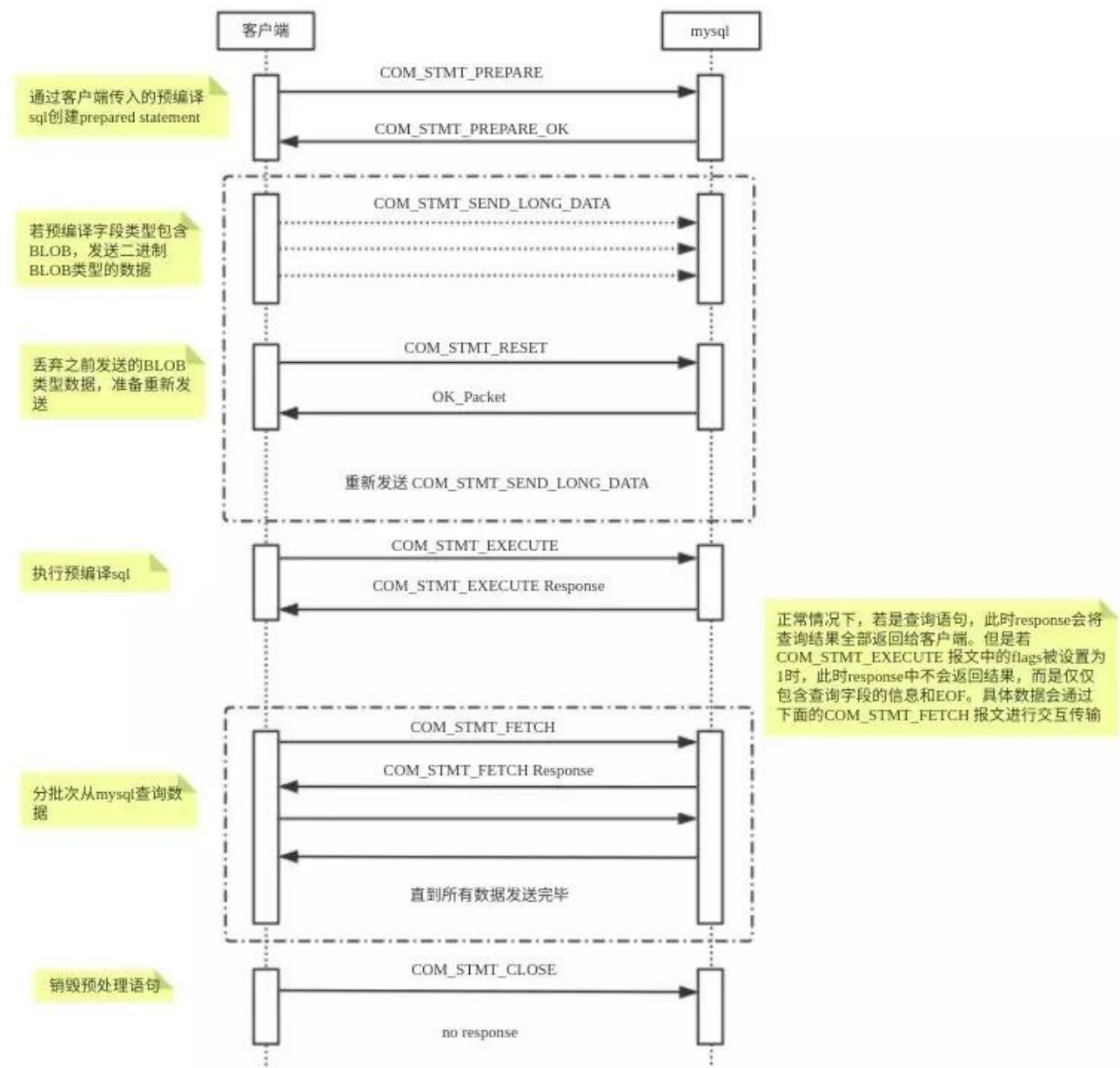
2 信息分析

MySQL 官方的介绍

MySQL 5.1 版本开始为服务器端预处理语句提供支持。此支持利用了高效的客户端/服务器二进制协议。将带有占位符的预准备语句用于参数值具有以下好处：

1. 每次执行时解析语句的开销更少。通常，数据库应用程序处理大量几乎相同的语句，只更改子句中的文字或变量值，例如 WHERE 查询和删除，SET 更新和 VALUES 插入。
2. 防止 SQL 注入攻击。参数值可以包含未转义的 SQL 引号和分隔符。

MySQL Prepared Statement 的二进制协议交互过程如图：



行为	命令	响应	说明
创建预处理	COM_STMT_PREPARE	COM_STMT_PREPARE_OK / ERR_Packet	发送预编译 SQL
发送长数据	COM_STMT_SEND_LONG_DATA	无	数据类型一般为 BLOB
重发长数据	COM_STMT_RESET	OK_Packet / ERR_Packet	只能在重置长数据发送时使用
执行发送	COM_STMT_EXECUTE	OK_Packet / ERR_Packet	发送数据并执行
分批次执行	COM_STMT_FETCH	A Multi-Resultset	分批次发送返回一组多结果集
销毁预处理	COM_STMT_CLOSE	无	销毁服务端缓存的预编译 SQL

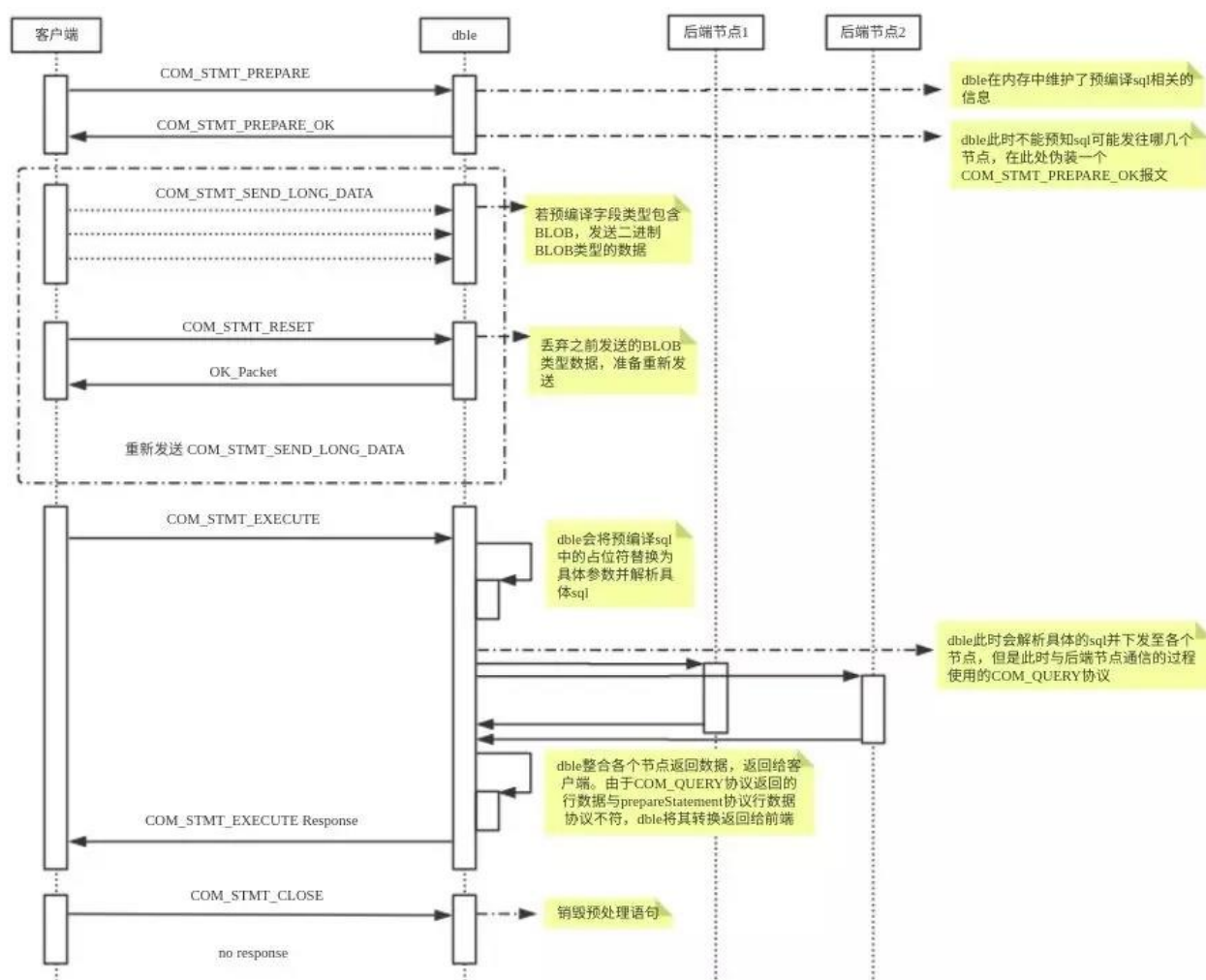
注意：

1. COM_STMT_SEND_LONG_DATA 必须在 COM_STMT_EXECUTE 前发送。
2. COM_STMT_FETCH 必须在 COM_STMT_EXECUTE 后发送。
3. COM_STMT_RESET 是专为重置 COM_STMT_SEND_LONG_DATA，不能单独使用。

通过一图一表，我们对 MySQL Prepared Statement 协议交互中的各种行为做了一个回顾。下面让我们看看 DBLE 是如何实现的。

3 DBLE 对 prepare statement 协议的实现

对于客户端的预编译 请求，DBLE 缓存了 SQL 并模拟返回报文，对于服务端改用 COM_QUERY 命令执行，并将各节点返回数据整合转换格式返回客户端。



说明：

1. 在 `COMSTMTPREPARE` 阶段，DBLE 此时接收的 SQL 不完整，不能确定下发节点，但 MySQL Prepared Statement 协议要求此处返回一个 response 报文，因此 DBLE 会伪装 `COMSTMTPREPAREOK` 报文返回。若有些 MySQL 驱动（如 JDBC）想从 response 报文中获取信息，这些信息会不准确。

2. 在 `COMSTMTEXECUTE` 阶段，DBLE 会根据客户端传输的参数将预编译 SQL 替换为具体的 SQL，并使用 `COMQUERY` 命令下发至后端节点，因为 `COMQUERY` 不再使用二进制协议传输，因此 DBLE 需要对后端返回的数据进行转换后再返回客户端。

3. DBLE 不支持 `COMSTMT_FETCH` 命令。

好了，当我们了解完 MySQL 和 DBLE 对 Prepared Statement 协议实现过程后，我们再回过头来看那两个 Bug 到底是怎么来的。

#1122 Bug 分析

经过分析，在同一连接中，当发起 COMSTMTCLOSE 销毁当前 prepare statement 后，紧接着又创建一个 prepare statement。这两个操作是在 DBLE 中是异步进行，存在线程安全的问题。知道问题的根源之后，解决方案是 DBLE 将两个操作变成同步操作，避免线程安全的问题。

#1124 Bug 分析

使用 JDBC 时，在 URL 中设置 useCursorFetch=true 的参数，希望开启 MySQL Server Side 游标功能。但是要使用 MySQL Server Side 游标需要满足下面条件：

1. 必须是 SELECT 语句
2. 设置了 fetchSize > 0
3. 设置了 useCursorFetch = true
4. 数据集类型为 ResultSet.TYPE_FORWARDONLY
5. 数据集并发设置为 ResultSet.CONCUR_READ_ONLY
6. Server versions 5.0.5 or newer

这是因为 JDBC 在代码层面做了如下限制：

```
// we only create cursor-backed result sets if
// a) The query is a SELECT
// b) The server supports it
// c) We know it is forward-only (note this doesn't preclude updatable result sets)
// d) The user has set a fetch size
if (this.resultFields != null && this.useCursorFetch && getResultSetType() ==
ResultSet.TYPE_FORWARD_ONLY
    && getResultSetConcurrency() == ResultSet.CONCUR_READ_ONLY && getFetchSize() > 0) {
    packet.writeByte(OPEN_CURSOR_FLAG);
} else {
    packet.writeByte((byte) 0); // placeholder for flags
}
```

那如何开启游标功能呢？以下是 JDBC 部分功能在进行预处理是开启游标的示例：

```
public static void testPrepareStmt() {
    Connection conn = null;
    PreparedStatement stmt = null;
    try {
        Class.forName("com.mysql.jdbc.Driver");
        conn = DriverManager.getConnection("jdbc:mysql://127.0.0.1:8066/poc?useCu
rsorFetch=true", "root", "123456");
        stmt = conn.prepareStatement("select long_col_1, long_col_2 from problemTa
ble where to_days(create_time) <= to_days(now()) and id = ?");
        stmt.setFetchSize(10);
        stmt.setInt(1, 2);
    }
```

```
ResultSet rs = stmt.executeQuery();
int count = 0;
while(rs.next()){
    ++count;
    System.out.println("##### row " + count + " #####");
    System.out.println("long_col_1 : " + rs.getString(1));
    System.out.println("long_col_2 : " + rs.getString(2));
    System.out.println();
}
} catch (SQLException e) {
    e.printStackTrace();
} catch (ClassNotFoundException ce) {
    ce.printStackTrace();
} finally {
    try {
        if (stmt != null) {
            stmt.close();
        }
        if (conn != null) {
            conn.close();
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}
```

4 尾声

#1122 和 #1124 两个 Bug 从被定位到抓获认罪只用了 5 天，随后 DBLE 社区发布 DBLE 2.19.03.0 版本，将真相大白于天下。

我们始终相信：真相只有一个！至此 DBLE 又踩平了一个 MyCat 的坑。



DBLE 自定义拆分算法

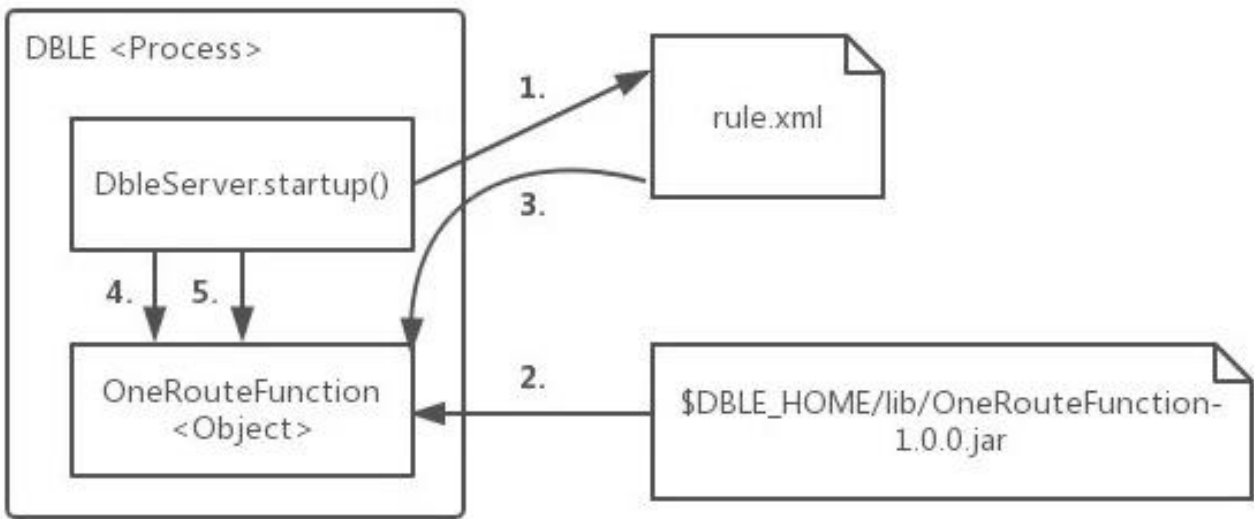
作者：钟悦

DBLE 默认支持数十种数据拆分算法，基本能满足大部分的社区用户的使用需求；为了满足更广的业务场景，DBLE 还支持更加灵活的自定义拆分算法；本文对面向有类似需求的 DBLE 开发者，提供了一个如何开发和部署自定义的拆分规则的一个指引。

1 工作原理

1.1 函数的加载

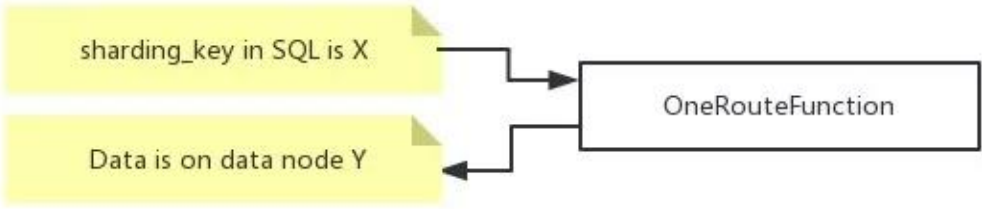
路由函数的加载发生在 DBLE 启动或重载时。



1. DBLE 读取 rule.xml 时，根据用户配置的标签的 class 属性
2. DBLE 通过 Java 的反射机制，从 \$DBLE_HOME/lib 的 jar 包中，找到对应的 jar（里的 class 文件），加载同名的类并创建对象
3. DBLE 会逐个扫描中的标签，并根据 name 属性来调用路由函数的对应 setter，以此完成赋值过程——例如，如果用户配置了 2，那么 DBLE 就会尝试找路由函数中叫做 setPartitionCount() 的方法，并将字符串“2”传给它
4. DBLE 调用路由函数的 selfCheck() 方法，执行函数编写者制定的检查动作，例如检查赋值得到的变量值是否有问题
5. DBLE 调用路由函数的 init() 方法，执行函数编写者制定的准备动作，例如创建后面要用到的一些中间变量

1.2 路由计算

路由函数接受用户 SQL 中的分片字段的值，计算出这个值对应的数据记录应该在哪个编号的数据分片（逻辑分片）上，DBLE 从而知道把这个 SQL 准确发到这些分片上。



1.3 参数查询

用户通过管理端口（默认 9066），通过 `SHOW @@ALGORITHM WHERE SCHEMA=? AND TABLE=?` 来查询表上的路由算法时，DBLE 调用路由算法的 `getAllProperties()` 方法，直接从内存中获取路由信息的配置。

```
mysql> show @@algorithm where schema=testdb and table=seqtest;
```

KEY	VALUE
TYPE	SHARDING TABLE
COLUMN	ID
CLASS	com.actiontech.dble.route.function.PartitionByLong
partitionCount	2
partitionLength	1

5 rows in set (0.05 sec)

2 开发和部署

2.1 开发

开发时，理论上只需要引入 `AbstractPartitionAlgorithm` 抽象类和 `RuleAlgorithm` 接口及它们的依赖类就可以了。但实际上 `AbstractPartitionAlgorithm` 抽象类依赖了 `TableConfig` 类，由此开启了环游世界的依赖之旅。因此，现实的操作还是引用整个 DBLE 项目的源代码会比较直接方便。

开发一个新的路由函数时，必须给这个路由函数的开发新建项目，然后再引用 DBLE 项目（项目引用项目的方式）。而不应该直接打开 DBLE 的项目，然后在 DBLE 的项目里面直接新建源代码来直接开发（内嵌开发方式）。通过遵循这个做法，会有以下好处：

1. 路由函数可以独立打包，直接去看路由函数的 jar 包版本就能够确认函数版本；而把路由函数嵌到 DBLE 里的话，就容易出现 DBLE 版本一样，但不清楚里面的函数是什么版本的窘况
2. 路由函数的递进可以更加自由，如果 DBLE 的 `AbstractPartitionAlgorithm` 抽象类和 `RuleAlgorithm` 接口没有变动，同一版本的路由函数可以延续使用好几个版本的 DBLE，而不需要每次 DBLE 释放新版就得去重编译
3. 可以让路由函数中的受保护代码免受 DBLE 自身的开源协议影响

2.2 部署

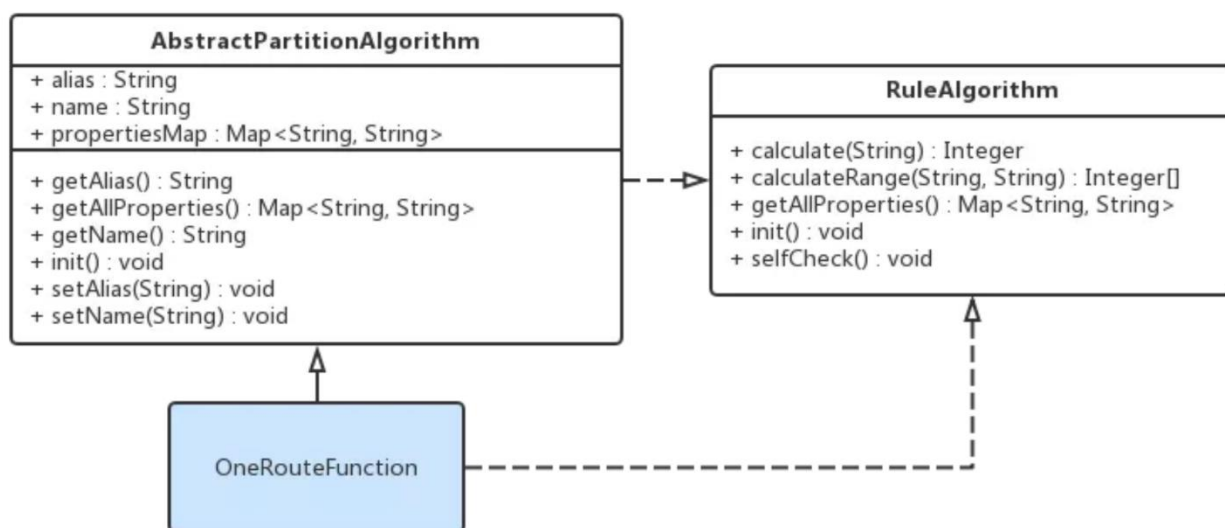
完成开发之后，成品打包成 jar 包进行发布，而不要直接发布 class 和依赖的 library（其他项目的 jar 包或 class 文件）。

让 DBLE 使用上新的路由函数的过程：

1. 将成品 jar 包放入 \$DBLE_HOME/lib 目录中
2. 调整 jar 包的所有者权限（chown）和文件权限（chmod），使之与其他 \$DBLE_HOME/lib 目录里的 jar 包一样
请注意：2.18.12.0 及以后版本建议放在 algorithm 目录下，用于区分原生 lib 和自定义 lib，便于管理
3. 按照原来的思路配置 rule.xml，但需要注意标签的 class 属性必须要填写新的路由函数类的完全限定名（Fully Qualified Name），例如 net.john.DBLE.route.functions.NewFunction
4. 配置逻辑表之类的必要信息，重启 DBLE 后，自动生效。

3 接口规范

每个路由函数本质上就是一个继承了 AbstractPartitionAlgorithm 抽象类，并且实现了 RuleAlgorithm 接口的一个类。下面以内置的 com.actiontech.DBLE.route.function.PartitionByLong 为例，介绍实现一个路由函数类所需要做的最小工作（必要工作）。



3.1 配置项 setters

在 rule.xml 中，我们需要配置 partitionCount 和 partitionLength 两个配置项。

```
<function name="hashmod" class="com">
  <property name="partitionCount">4</property>
  <property name="partitionLength">1</property>
</function>
```

为了让 DBLE 在函数加载过程中，能够认出这里的 `partitionCount`（值为 4）和 `partitionLength`（值为 1），因此 `PartitionByLong` 类中，就必须有属性设置方法（setter）`setPartitionCount()` 和 `setPartitionLength()`。而因为 `rule.xml` 是个文本型的 XML 文件，所以这些函数的传入参数就只能是一个 `String`，数据类型转换和预处理的动作就由这些 `setter` 来处理了。

```
public void setPartitionCount(String partitionCount) {
    this.count = toIntArray(partitionCount);
    /* 参考本文的 getAllProperties() 的说明 */
    propertiesMap.put("partitionCount", partitionCount);
}

public void setPartitionLength(String partitionLength) {
    this.length = toIntArray(partitionLength);
    /* 参考本文的 getAllProperties() 的说明 */
    propertiesMap.put("partitionLength", partitionLength);
}
```

3.2 selfCheck()

在函数加载过程中，完成了配置项赋值之后，DBLE 会调用这个路由函数对象的 `selfCheck()` 方法，让这个对象自我检查刚才读进来的配置项的值，放在一起是不是有问题。如果有问题的话，路由函数编写者在这时候，可以通过抛出 `RuntimeException` 来进行报错，并终止 DBLE 使用这个函数，当然，由于 `RuntimeException` 的霸道，DBLE 自己也会因此而报错退出。

由于 `selfCheck()` 是 `RuleAlgorithm` 接口的要求，而且 `AbstractPartitionAlgorithm` 抽象类没又实现它，对于想偷懒或者没有必要进行这个检查的人来说，还是需要自行定义一个空的同名方法来实现它。

```
@Override
public void selfCheck() {
}
```

3.3 init()

在函数加载过程的最后，DBLE 调用这个路由函数对象的 `init()` 方法，让这个对象完成一些内部的初始化工作。

在我们的例子 `PartitionByLong` 里，通过 `init()` 方法准备了 `PartitionUtil` 对象，其中有一个哈希值的范围与逻辑分片号对应的数组，这样在后面的路由计算时就能通过查数组来加速得到结果。

3.4 calculate()和 calculateRange()

DBLE 执行用户 SQL 时，根据用户 SQL 的不同，调用 `calculate()` 或 `calculateRange()` 来确定用户的 SQL 应该发到哪个数据分片上去。

从 IPO（Input-Process-Output）来分析，`calculate()` 和 `calculateRange()` 的工作原理是一样的：

1. Input：用户 SQL 中的分片字段值

2. Output: 用户 SQL 应该要发往的数据分片的编号
3. Process: Input 与 Output 转换的计算过程, 由函数开发者编写

calculate() 和 calculateRange() 的使用场景不同, 导致它们存在着一些微小的差异。

函数名	调用场景	Input	Output
calculate()	用户SQL里分片字段的值是单值的情况, 例如 ... WHERE sharding_key = 1	1个 String	1个 Integer
calculateRange()	用户SQL里分片字段的值是连续范围, 例如 ... WHERE sharding_key BETWEEN 1 AND 5	2个 String	Integer 数组

```

@Override
public Integer calculate(String columnValue) {
    try {
        if (columnValue == null || columnValue.equalsIgnoreCase("NULL")) {
            return 0;
        }
        long key = Long.parseLong(columnValue);
        return calculate(key);
    } catch (NumberFormatException e) {
        throw new IllegalArgumentException("columnValue:" + columnValue + " Please eliminate any quote and non number within it.", e);
    }
}

@Override
public Integer[] calculateRange(String beginValue, String endValue) {
    long begin = 0;
    long end = 0;
    try {
        begin = Long.parseLong(beginValue);
        end = Long.parseLong(endValue);
    } catch (NumberFormatException e) {
        return new Integer[0];
    }
    int partitionLength = partitionUtil.getPartitionLength();
    if (end - begin >= partitionLength || begin > end) { //TODO: optimize begin > end
        return new Integer[0];
    }
    Integer beginNode = calculate(begin);
    Integer endNode = calculate(end);

```

```
    if (endNode > beginNode || (endNode.equals(beginNode) && partitionUtil.isSingle
Node(begin, end))) {
        int len = endNode - beginNode + 1;
        Integer[] re = new Integer[len];

        for (int i = 0; i < len; i++) {
            re[i] = beginNode + i;
        }
        return re;
    } else {
        int split = partitionUtil.getSegmentLength() - beginNode;
        int len = split + endNode + 1;
        if (endNode.equals(beginNode)) {
            //remove duplicate
            len--;
        }
        Integer[] re = new Integer[len];
        for (int i = 0; i < split; i++) {
            re[i] = beginNode + i;
        }
        for (int i = split; i < len; i++) {
            re[i] = i - split;
        }
        return re;
    }
}
```

3.5 getAllProperties()

当用户找 DBLE 要路由函数的配置信息时，DBLE 通过访问路由函数的 `getAllProperties()` 来获得一个<配置项，配置值>的哈希表，然后将里面的内容逐项返回给用户。

`getAllProperties()` 是 `RuleAlgorithm` 接口所规定要实现的，但为了简化编写新的路由函数的工作，在 `AbstractPartitionAlgorithm` 抽象类里，定义了 `propertiesMap` 这个私有变量，并且把“将 `propertiesMap` 交出去”作为了实现了 `getAllProperties()` 方法的默认实现。

一般来说，这个默认的实现能满足需求，而新路由函数编写者只需要在配置项 `setters` 处理用户配置时，将<配置项，配置值>给 `put()` 进 `propertiesMap` 里就好了。

```
@Override
public Map<String, String> getAllProperties() {
    return propertiesMap;
}
```

4 内置路由函数的缩写与类名对照表

DBLE 内置的路由函数都位于 `com.actiontech.DBLE.route.function` 命名空间。但实际配置 `rule.xml` 的时候，却不用写那么长的完全限定名，这其实都是 `XMLRuleLoader` 类做了转换，因此实现了简写。下面就是 7 个内置函数的类名和它们的简写。

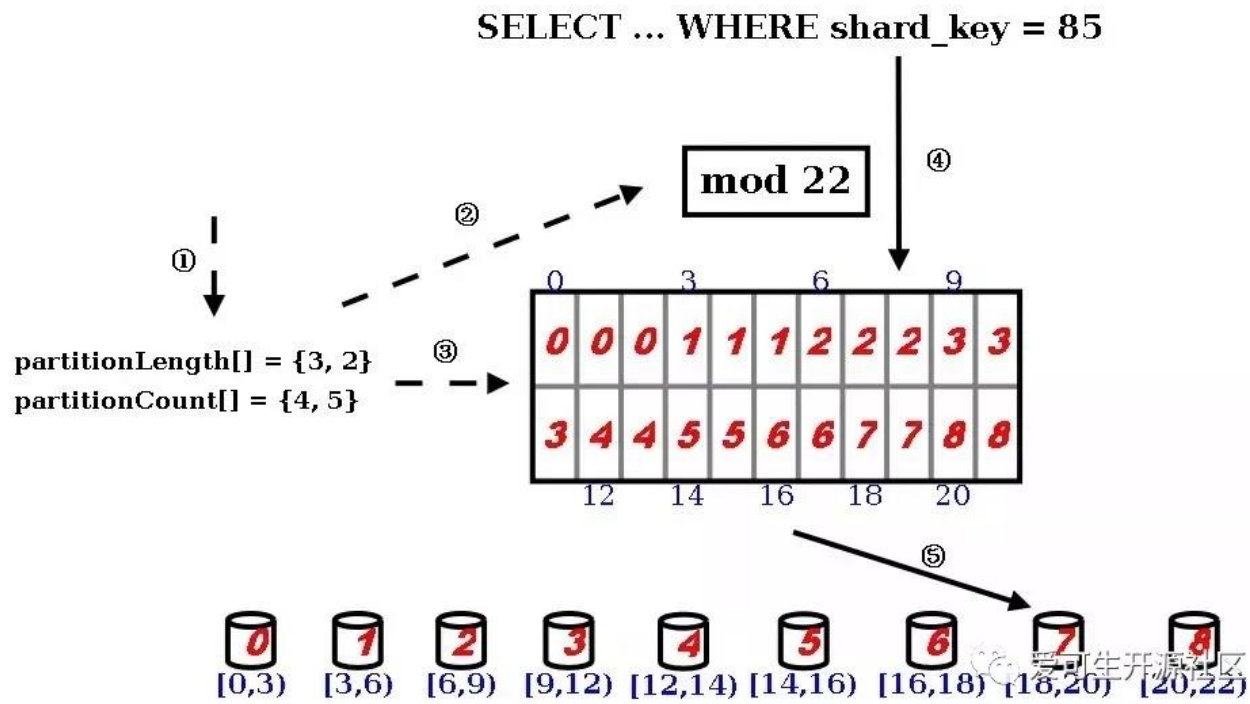
简写名	完整类名
date	PartitionByDate
enum	PartitionByFileMap
hash	PartitionByLong
jumpstringhash	PartitionByJumpConsistentHash
numberrange	AutoPartitionByLong
patternrange	PartitionByPattern
stringhash	PartitionByString



DBLE 与 MyCat 分片算法对比：hash 分片

作者：钟悦

这是先求模得到逻辑分片号，再根据逻辑分片号直接映射到物理分片的一种散列算法。



1. 用户需要在 rule.xml 中定义 partitionLength[] 和 partitionCount[] 两个数组
2. 在 DBLE 的启动阶段，点乘这两个数组得到模数，也是逻辑分片的数量
3. 并且根据两个数组的叉乘，得到各个逻辑分片到物理分片的映射表（物理分片数量是 partitionCount[] 数组的元素值之和）
4. 在 DBLE 的运行过程中，用户访问使用这个算法的表时，WHERE 子句中的分片索引值会被提取出来进行求模，得到逻辑分片号
5. 再根据逻辑分片号，查映射表，直接得到物理分片号

1 与 MyCat 的类似分片算法对比

中间件	DBLE	MyCat
分片算法种类	hash 分区算法	固定分片 hash 算法 / 取模
区别	分片数最大支持 2880	count 和 length 两个向量的点乘积恒等于 1024

1. artitionLength 为 1, partitionCount 为 N，就是按照 N 取模的拆分算法
2. partitionLength 与 partitionCount 点乘值在 [1,2880] 之间

2 开发注意点

【分片索引】1. 必须是整型数字或整型数字的字符串（可以为负数）

【分片索引】2. 最大物理分片配置方法是，让 partitionCount 数组的和等于 2880

```
<property name="partitionLength">1</property>
<property name="partitionCount">2880</property>
```

或

```
<property name="partitionLength">1,1</property>
<property name="partitionCount">1440,1440</property>
```

【分片索引】3. 最小物理分片配置方法是，让 partitionCount 数组的和等于 1

```
<property name="partitionLength">2880</property>
<property name="partitionCount">1</property>
```

【分片索引】4. partitionLength 和 partitionCount 被当做两个逗号分隔的一维数组，它们之间的点乘必须在 [1, 2880] 范围内

【分片索引】5. partitionLength 和 partitionCount 的配置对顺序敏感

```
<property name="partitionLength">512,256</property>
<property name="partitionCount">1,2</property>
```

和

```
<property name="partitionLength">256,512</property>
<property name="partitionCount">2,1</property>
```

是不同的分片结果

【数据分布】1. 与分片索引值相关而与 INSERT 先后无相关性，所以在直接使用时无法保证数据分布均匀，但如果分片索引本身连续递增（交易流水号等），则可以期待数据分布较为平均，但副作用会导致范围语句变成跨分片查询

例如：

```
SELECT ... WHERE shard_key BETWEEN 1 AND 100
```

3 运维注意点

【扩容】1. 预先过量分片，并且不改变 partitionCount 和 partitionLength 点乘结果，可以避免数据再平衡，只需进行涉及数据的迁移

【扩容】2. 若需要改变 partitionCount 和 partitionLength 点乘结果，需要数据再平衡

【缩容】1. 预先过量分片，并且不改变 partitionCount 和 partitionLength 点乘结果，可以避免数据再平衡，只需进行涉及数据的迁移

【缩容】2. 若需要改变 partitionCount 和 partitionLength 点乘结果，需要数据再平衡

4 配置注意点

【配置项】1. 在 rule.xml 中，可配置项为 `<property name="partitionLength">` 和 `<property name="partitionCount">`

【配置项】2. 在 rule.xml 中配置 `<propertyname="partitionLength">` 标签
内容形式为：`<物理分片持有的虚拟分片数>[,<物理分片持有的虚拟分片数>,...<物理分片持有的虚拟分片数>]`

物理分片持有的虚拟分片数必须是整型，物理分片持有的虚拟分片数从左到右与同顺序的物理分片数对应，partitionLength 和 partitionCount 的点乘结果必须在 [1, 2880] 范围内

【配置项】3. 在 rule.xml 中配置 `<propertyname="partitionCount">` 标签
内容形式为：`<物理分片数>[,<物理分片数>,...<物理分片数>]`

其中物理分片数必须是整型，物理分片数按从左到右的顺序，与同顺序的物理分片持有的虚拟分片数对应。物理分片的编号从左到右连续递进，partitionLength 和 partitionCount 的点乘结果必须在 [1, 2880] 范围内

【配置项】4. partitionLength 和 partitionCount 的语义是：持有 partitionLength[i] 个虚拟分片的物理分片有 partitionCount[i] 个

例如：

```
<property name="partitionLength">512,256</property>
<property name="partitionCount">1,2</property>
```

语义是持有 512 个 逻辑分片的物理分片有 1 个，紧随其后，持有 256 个逻辑分片的物理分片有 2 个

【配置项】5. partitionLength 和 partitionCount 都对书写顺序敏感

例如：

```
<property name="partitionLength">512,256</property>
<property name="partitionCount">1,2</property>
```

分片结果是第一个物理分片持有头 512 个逻辑分片，第二个物理分片持有紧接着的 256 个逻辑分片，第三个物理分片持有最后 256 个逻辑分片，相对的

```
<property name="partitionLength">256,512</property>
<property name="partitionCount">2,1</property>
```

分片结果则是第一个物理分片持有头 256 个逻辑分片，第二个物理分片持有紧接着的 256 个逻辑分片，第三个物理分片持有最后 512 个逻辑分片

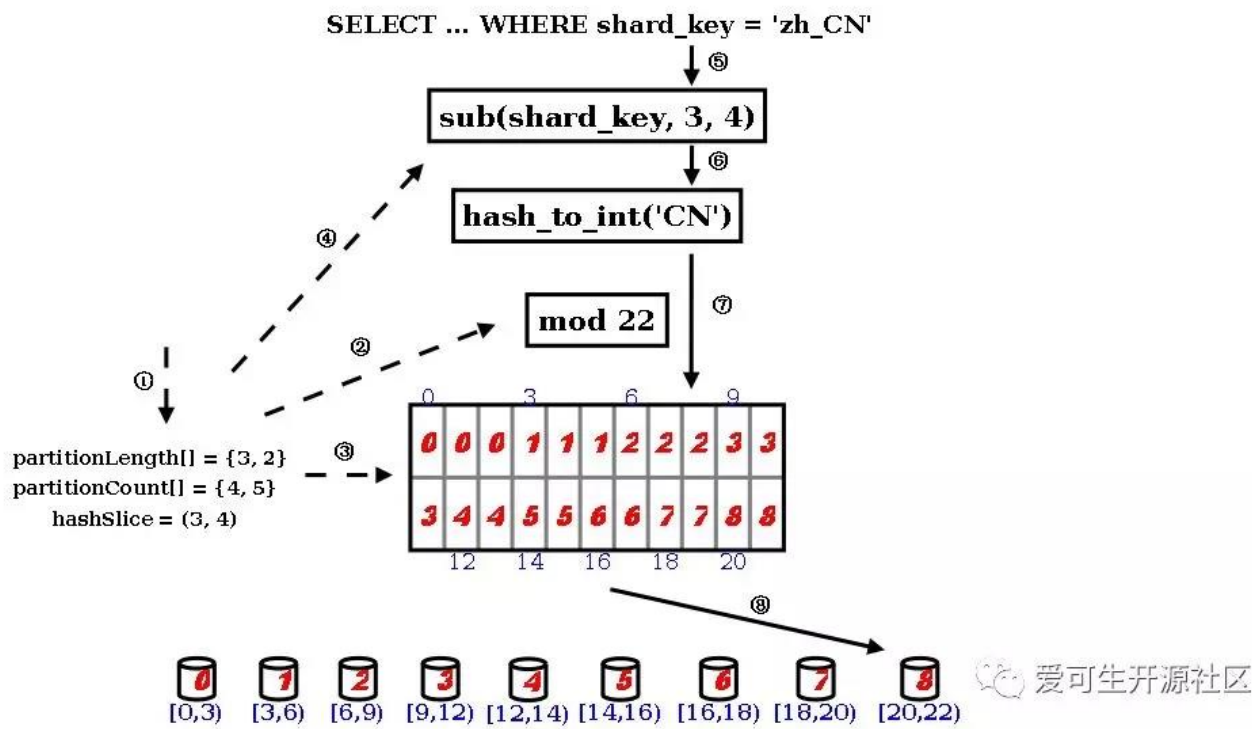
【配置项】6. partitionLength[] 的元素全部为 1 时，这时候 partitionCount 数组和等于 partitionLength 和 partitionCount 的点乘，物理分片和逻辑分片就会一一对应，该分片算法等效于直接取余
469



DBLE 与 MyCat 分片算法对比：stringhash 分片

作者：钟悦

当分片索引不是纯整型的字符串时，只接受整型的内置 hash 算法是无法使用的。为此，stringhash 按照用户定义的起点和终点去截取分片索引字段中的部分字符，根据当中每个字符的二进制 unicode 值换算出一个长整型数值，然后就直接调用内置 hash 算法求解分片路由：先求模得到逻辑分片号，再根据逻辑分片号直接映射到物理分片。



1. 用户需要在 rule.xml 中定义 partitionLength[] 和 partitionCount[] 和 hashSlice 二元组。
2. 在 DBLE 的启动阶段，点乘两个数组得到模数，也是逻辑分片的数量
3. 并且根据两个数组的叉乘，得到各个逻辑分片到物理分片的映射表（物理分片数量由 partitionCount[] 数组的元素值之和）
4. 此外根据 hashSlice 二元组，约定把分片索引值中的第 4 字符到第 5 字符（字符串以 0 开始编号，编号 3 到编号 4 等于第 4 字符到第 5 字符）字符串用于 “字符串->整型” 的转换
5. 在 DBLE 的运行过程中，用户访问使用这个算法的表时，WHERE 子句中的分片索引值会被提取出来，取当中的第 4 个字符到第 5 字符，送入下一步
6. 设置一个初始值为 0 的累计值，逐个取字符，把累计值乘以 31，再把这个字符的 unicode 值当成长整型加入到累计值中，如此类推直至处理完截取出来的所有字符，此时的累计值就能够代表用户的分片索引值，完成了 “字符串->整型” 的转换
7. 对上一步的累计值进行求模，得到逻辑分片号
8. 再根据逻辑分片号，查映射表，直接得到物理分片号

1 与 MyCat 的类似分片算法对比

中间件	DBLE	MyCat
分片算法种类	stringhash 分区算法	截取数字 hash 解析
区别	分片数最大支持 2880	count 和 length 两个向量的点乘积恒等于 1024

 爱可生开源社区

两种算法在 string 转化为 int 之后，和 hash 分区算法相同，区别也继承了 hash 算法的区别。

2 开发注意点

【分片索引】1. 必须是字符串

【分片索引】2. 最大物理分片配置方法是，让 partitionCount[] 数组和等于 2880

```
<property name="partitionLength">1</property>
<property name="partitionCount">2880</property>
```

或

```
<property name="partitionLength">1,1</property>
<property name="partitionCount">1440,1440</property>
```

【分片索引】3. 最小物理分片配置方法是，让 partitionCount[] 数组和等于 1

```
<property name="partitionLength">2880</property>
<property name="partitionCount">1</property>
```

【分片索引】4. partitionLength 和 partitionCount 被当做两个逗号分隔的一维数组，它们之间的点乘必须在 [1, 2880] 范围内

【分片索引】5. partitionLength 和 partitionCount 的配置对顺序敏感

```
<property name="partitionLength">512,256</property>
<property name="partitionCount">1,2</property>
```

和

```
<property name="partitionLength">256,512</property>
<property name="partitionCount">2,1</property>
```

是不同的分片结果

【分片索引】6. 分片索引字段长度小于用户指定的截取长度时，截取长度会安全减少到符合分片索引字段长度

【数据分布】1. 分片索引字段截取越长则越有利于数据均匀分布

【数据分布】2. 分片索引字段的内容重复率越低则越有利于数据均匀分布

3 运维注意点

【扩容】1. 预先过量分片，并且不改变 partitionCount 和 partitionLength 点乘结果，也不改变截取设置 hashSlice 时，可以避免数据再平衡，只需进行涉及数据的迁移

【扩容】2. 若需要改变 partitionCount 和 partitionLength 点乘结果或改变截取设置 hashSlice 时，需要数据再平衡

【缩容】1. 预先过量分片，并且不改变 partitionCount 和 partitionLength 点乘结果，也不改变截取设置 hashSlice 时，可以避免数据再平衡，只需进行涉及数据的迁移

【缩容】2. 若需要改变 partitionCount 和 partitionLength 点乘结果或改变截取设置 hashSlice 时，需要数据再平衡

4 配置注意点

【配置项】1, 在 rule.xml 中，可配置项为 <property name="partitionLength">、<property name="partitionCount"> 和 <property name="hashSlice">`

【配置项】2, 在 rule.xml 中配置 <property name="partitionLength">标签

内容形式为：<物理分片持有的虚拟分片数>[, <物理分片持有的虚拟分片数>, ... <物理分片持有的虚拟分片数>]

物理分片持有的虚拟分片数必须是整型，物理分片持有的虚拟分片数从左到右与同顺序的物理分片数对应，partitionLength 和 partitionCount 的点乘结果必须在 [1, 2880] 范围内

【配置项】3, 在 rule.xml 中配置 <property name="partitionCount"> 标签

内容形式为：<物理分片数>[, <物理分片数>, ... <物理分片数>]

其中物理分片数必须是整型，物理分片数按从左到右的顺序与同顺序的物理分片持有的虚拟分片数对应，物理分片的编号从左到右连续递进，partitionLength 和 partitionCount 的点乘结果必须在 [1, 2880] 范围内

【配置项】4, partitionLength 和 partitionCount 的语义是：持有 partitionLength[i] 个虚拟分片的物理分片有 partitionCount[i] 个

```
<property name="partitionLength">512,256</property>
<property name="partitionCount">1,2</property>
```

语义是持有 512 个逻辑分片的物理分片有 1 个，紧随其后，持有 256 个逻辑分片的物理分片有 2 个

【配置项】5, partitionLength 和 partitionCount 都对书写顺序敏感，

例如

```
<property name="partitionLength">512,256</property>
<property name="partitionCount">1,2</property>
```

分片结果是第一个物理分片持有头 512 个逻辑分片，第二个物理分片持有紧接着的 256 个逻辑分片，第三个物理分片持有最后 256 个逻辑分片，相对的

```
<property name="partitionLength">256,512</property>
<property name="partitionCount">2,1</property>
```

分片结果则是第一个物理分片持有头 256 个逻辑分片，第二个物理分片持有紧接着的 256 个逻辑分片，第三个物理分片持有最后 512 个逻辑分片

【配置项】6，partitionLength[] 的元素全部为 1 时，这时候 partitionCount 数组和等于 partitionLength 和 partitionCount 的点乘，物理分片和逻辑分片就会一一对应，该分片算法等效于直接取余

【配置项】7，在 rule.xml 中配置 <property name="hashSlice"> 标签，从分片索引字段的第几个字符开始截取到第几个字符：

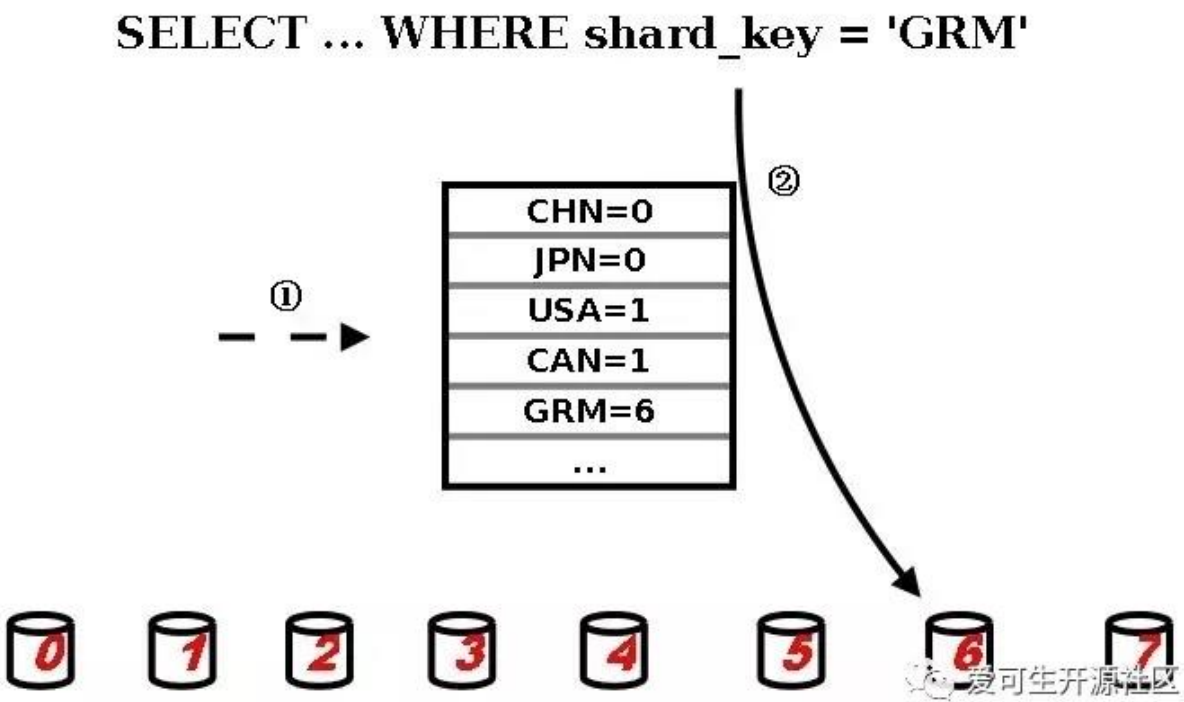
- ☐ 若希望从首字符开始截取 k 个字符（k 为正整数），配置的内容形式可以为“0 : k”、“k”或“ : k”；
- ☐ 若希望从末字符开始截取 k 个字符（k 为正整数），则配置的内容形式可以为“-k : 0”、“-k”或“-k : ”；
- ☐ 若希望从头第 m 个字符起算截取 n 个字符（m 和 n 都是正整数），则先计算出 $i = m - 1$ 和 $j = i + n - 1$ ，配置的内容形式为“i : j”；
- ☐ 若希望从尾第 m 个字符起算截取从尾算起的 n 个字符（m 和 n 都是正整数），则先计算出 $i = -m + n - 1$ ，配置的内容形式可以为“-m : i”；
- ☐ 若希望不截取，则配置的内容形式可以为“0 : 0”、“0 : ”、“ : 0”或“ : ”



DBLE 与 MyCat 分片算法对比：enum 分片

作者：钟悦

根据用户定义的枚举值与分片节点映射文件，直接定位目标分片。



- 1. 用户在 rule.xml 中配置枚举值文件路径和分片索引是字符串还是数字，DBLE 在启动时会把枚举值文件加载到内存中，形成一个映射表
- 2. 在 DBLE 的运行过程中，用户访问使用这个算法的表时，WHERE 子句中的分片索引值会被提取出来，直接查映射表得到分片编号

1 与 MyCat 的类似分片算法对比

中间件	DBLE	MyCat
分片算法种类	enum 分区算法	分片枚举

两种中间件的枚举分片算法使用上无差别。

2 开发注意点

【分片索引】1. 整型数字（可以为负数）或字符串（（不含=和换行符）

【分片索引】2. 枚举值之间不能重复，否则会导致分片策略加载出错

【分片索引】3. 不同枚举值可以映射到同一个分片上

```
Mr=0
Mrs=1
Miss=1
Ms=1
123=0
```

3 运维注意点

【扩容】1. 增加枚举值无需数据再平衡

【扩容】2. 增加一个枚举值的分片数量数时，需要对局部数据进行迁移

【缩容】1. 减少枚举值需要数据再平衡

【缩容】2. 减少一个枚举值的分片数量数时，需要对局部数据进行迁移

4 配置注意点

【配置项】1. 在 rule.xml 中，可配置项为 <property name="defaultNode">、<property name="mapFile"> 和 <property name="type">

【配置项】2. 在 rule.xml 中配置 <property name="defaultNode"> 标签，非必须配置项，不配置该项的话，用户的分片索引值没落在 mapFile 定义的范围时，DBLE 会报错；若需要配置，必须为非负整数，用户的分片索引值没落在 mapFile 定义的范围时，DBLE 会路由至这个值的 MySQL 分片

【配置项】3. 在 rule.xml 中配置 <property name="mapFile"> 标签，范围映射文件的路径：若在映射文件在 DBLE_HOME/conf 或其中，则可以使用相对路径的形式配置，例如，映射文件是 DBLE_HOME/conf/map/table_map.txt 时，配置值就可以简写为 map/table_map.txt；映射文件在 DBLE_HOME/conf 目录以外时，需要使用绝对路径，但这种做法需要考虑用户权限等问题，因此不建议把映射文件放在 DBLE_HOME/conf 外。

【配置项】4. 编辑 mapFile 所配置的文件

记录格式为：<枚举值>=<分片编号>

枚举值可以是整型数字，或任意字符（除了=和换行符），分片编号必须是非负整型数字，记录之间以换行分隔，一行仅能有一条记录，枚举值不能够是“DEFAULT_NODE”这个字符串，允许以“//”和“#”在行首来注释该行

【配置项】5. 在 rule.xml 中配置 <property name="type"> 标签；type 必须为整型；取值为 0 时，mapFile 的<枚举值>必须为整型；取值为非 0 时，mapFile 的<枚举值>可以是任意字符（除了=和换行符）

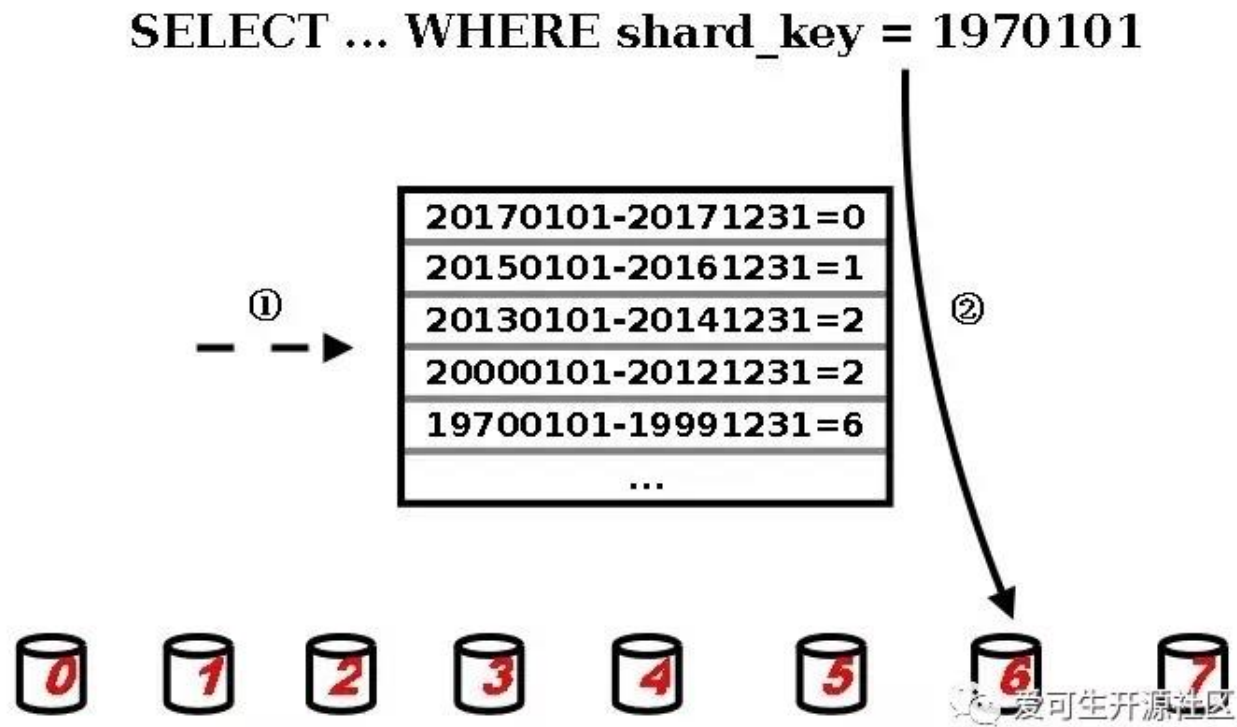
微信扫一扫阅读原文



DBLE 与 MyCat 分片算法对比：numberrange 分片

作者：钟悦

根据用户定义的范围与分片节点映射文件，直接定位目标分片。



1. 用户在 rule.xml 中配置范围定义文件路径，文件中定义的范围被加载到内存中，形成一个映射表
2. 在 DBLE 的运行过程中，用户访问使用这个算法的表时，WHERE 子句中的分片索引值会被提取出来，直接查映射表得到分片编号

1 与 MyCat 的类似分片算法对比

中间件	DBLE	MyCat
分片算法种类	numberrange 分区算法	范围约定
区别	写具体的数值	可用K或M等数量替换付

K=1000 , M=10000

2 开发注意点

- 【分片索引】1. 整型数字（可以为负数），取值范围是长整型
- 【分片索引】2. 范围包含其起点和终点，例如，范围 1-100 包含 1 和 100（即 [1, 100]）
- 【分片索引】3. 如果范围与范围之间存在重叠（例如 1-100 和 100-200 重叠于 100），不会引起异常，会命中在范围配置文件 mapFile 中最先出现的那一个并按其执行
- 【分片索引】4. 不同范围可以映射到同一个分片上 $0 - 99 = 0$ ； $100 - 199 = 0$
- 【数据分布】1. 无法保证均匀
- 【数据分布】2. 总分数目等于范围配置文件 mapFile 中各范围持有的分片数量之和

3 运维注意点

- 【扩容】1. 原有范围（含默认节点）的数据太热，需要拆分成多个新的更小的范围来扩展到不同 MySQL 节点时，需要对局部数据进行迁移
- 【缩容】1. 原有的几个范围的数据太冷，需要合并到同一个 MySQL 节点来节省资源时，需要对局部数据进行迁移

4 配置注意点

- 【配置项】1. 在 rule.xml 中，可配置项为 `<property name="defaultNode">` 和 `<property name="mapFile">`
- 【配置项】2. 在 rule.xml 中配置 `<property name="defaultNode">` 标签，非必须配置项，不配置该项的话，用户的分片索引值没落在 mapFile 定义的范围时，DBLE 会报错；若需要配置，必须为非负整数，用户的分片索引值没落在 mapFile 定义的范围时，DBLE 会路由至这个值的 MySQL 分片
- 【配置项】3. 在 rule.xml 中配置 `<property name="mapFile">` 标签，范围映射文件的路径：若在映射文件在 DBLE_HOME/conf 或其中，则可以使用相对路径的形式配置，例如，映射文件是 DBLE_HOME/conf/map/table_map.txt 时，配置值就可以简写为 map/table_map.txt；映射文件在 DBLE_HOME/conf 目录以外时，需要使用绝对路径，但这种做法需要考虑用户权限等问题，因此不建议把映射文件放在 DBLE_HOME/conf 外。
- 【配置项】4. 编辑 mapFile 所配置的文件

记录格式为：<范围最小值> - <范围最大值> = <分片编号>

范围最小值和范围最大值必须是整型数字，取值范围为 Java 的长整型范围内，分片编号必须是非负整型数字，记录之间以换行分隔，一行仅能有一条记录，允许以 “//” 和 “#” 在行首来注释该行

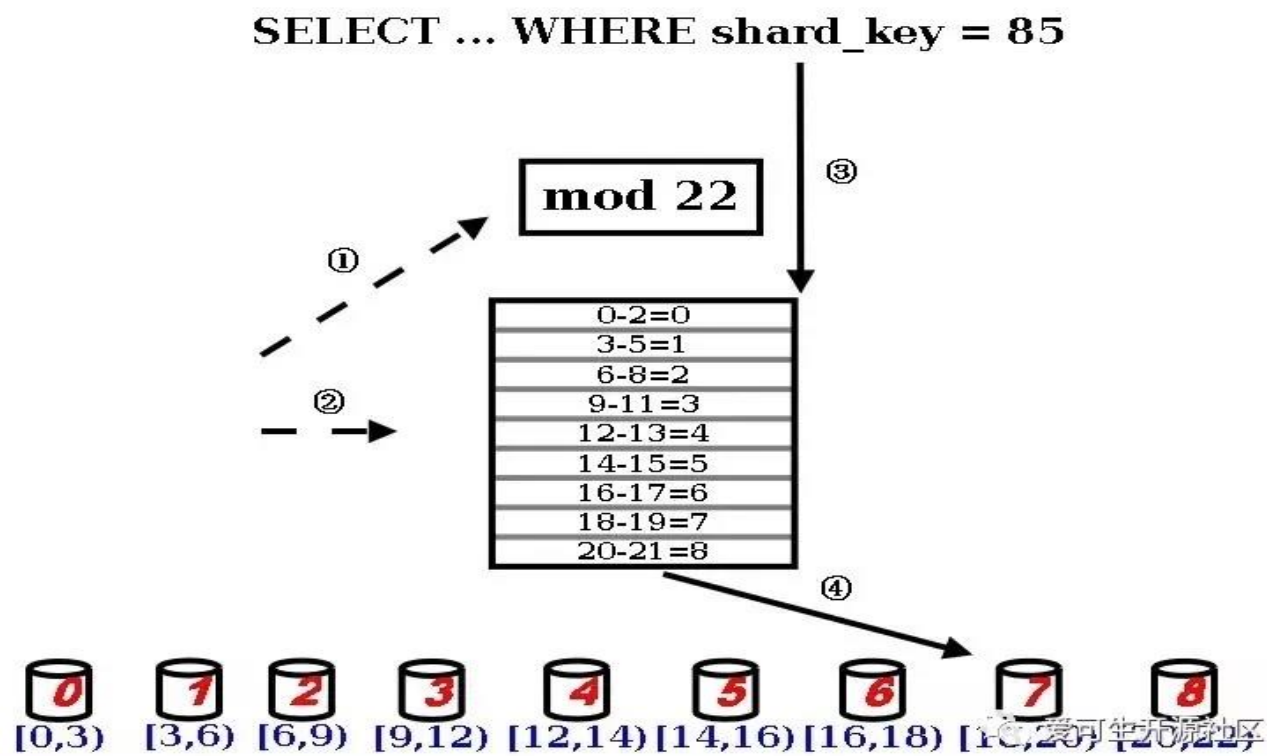
- 【配置项】5. 读取 mapFile 时，DBLE 不会对其中的范围记录查重或排序，也不会检查范围最小值和范围最大值相互之间谁更大
- 【配置项】6. mapFile 中的记录的先后顺序非常重要，目前的最佳实践是人手确保范围与范围之间没有重叠，而且按照数据查询频率，从高频到低频来顺序填写 mapFile



DBLE 与 MyCat 分片算法对比：patternrange 分片

作者：钟悦

与 hash 算法的最终效果一样，这个算法也是先求模得到逻辑分片号，再根据逻辑分片号直接映射到物理分片的一种散列算法。



1. 用户需要在 rule.xml 中给出 patternValue 来定义逻辑分片数量
2. 在 DBLE 的启动阶段，读取用户在 rule.xml 中给出的 mapFile，得到逻辑分片到物理分片的映射表
3. 在 DBLE 的运行过程中，用户访问使用这个算法的表时，WHERE 子句中的分片索引值会被提取出来进行求模，得到逻辑分片号
4. 再根据逻辑分片号，查映射表，直接得到物理分片号

1 与 MyCat 的类似分片算法对比

中间件	DBLE	MyCat
分片算法种类	patternrange 分区算法	取模范围约束

两种中间件的取模范围分片算法使用上无差别

2 开发注意点

【分片索引】1. 必须是整型数字或整型数字的字符串（可以为负数）

【分片索引】2. 最大物理分片配置方法是，在 mapFile 文件中，为每一个逻辑分片指定单独的物理分片

例如：

```
0=0
1=1
...
```

【分片索引】3. 最小物理分片配置方法是，在 mapFile 文件中，为所有逻辑分片指定同一个物理分片

例如：

```
0-<逻辑分片数量>=0
```

【数据分布】1. 与分片索引值相关而与 INSERT 先后无相关性，所以在直接使用时无法保证数据分布均匀，但如果分片索引本身连续递增（交易流水号等），则可以期待数据分布较为平均，但副作用会导致范围语句

例如

```
SELECT ... WHERE shard_key BETWEEN 1 AND 100
```

变成跨分片查询

5.14.3 运营注意点

【扩容】1. 预先过量分片，并且不改变 patternValue，可以避免数据再平衡，只需进行涉及数据的迁移

【扩容】2. 若需要改变 patternValue，需要数据再平衡

【缩容】1. 预先过量分片，并且不改变 patternValue，可以避免数据再平衡，只需进行涉及数据的迁移

【缩容】2. 若需要改变 patternValue，需要数据再平衡

5.14.4 配置注意点

【配置项】1. 在 rule.xml 中，可配置项为 <property name="patternValue"> 和 <property name="mapFile"> 和 <property name="defaultNode">

【配置项】2. 在 rule.xml 中配置 <property name="defaultNode"> 标签，非必须配置项，不配置该项的话，用户的分片索引值没落在 mapFile 定义的范围时，DBLE 会报错；若需要配置，必须为非负整数，用户的分片索引值没落在 mapFile 定义的范围时，DBLE 会路由至这个值的 MySQL 分片

【配置项】3. 在 rule.xml 中配置 `<property name="mapFile">` 标签，范围映射文件的路径：若在映射文件在 DBLE_HOME/conf 或其中，则可以使用相对路径的形式配置，例如，映射文件是 DBLE_HOME/conf/map/table_map.txt 时，配置值就可以简写为 map/table_map.txt；映射文件在 DBLE_HOME/conf 目录以外时，需要使用绝对路径，但这种做法需要考虑用户权限等问题，因此不建议把映射文件放在 DBLE_HOME/conf 外。

【配置项】4. 编辑 mapFile 所配置的文件

记录格式为：`<逻辑分片范围的最小值>-<逻辑分片范围的最大值>=<物理分片编号>`

逻辑分片范围的最小值和逻辑分片范围的最大值必须是整型数字，取值范围为 Java 的长整型范围内，物理分片编号必须是非负整型数字，记录之间以换行分隔，一行仅能有一条记录，允许以 “//” 和 “#” 在行首来注释该行

【配置项】5. 读取 mapFile 时，DBLE 不会对其中的范围记录查重，也不会检查范围最小值和范围最大值相互之间谁更大

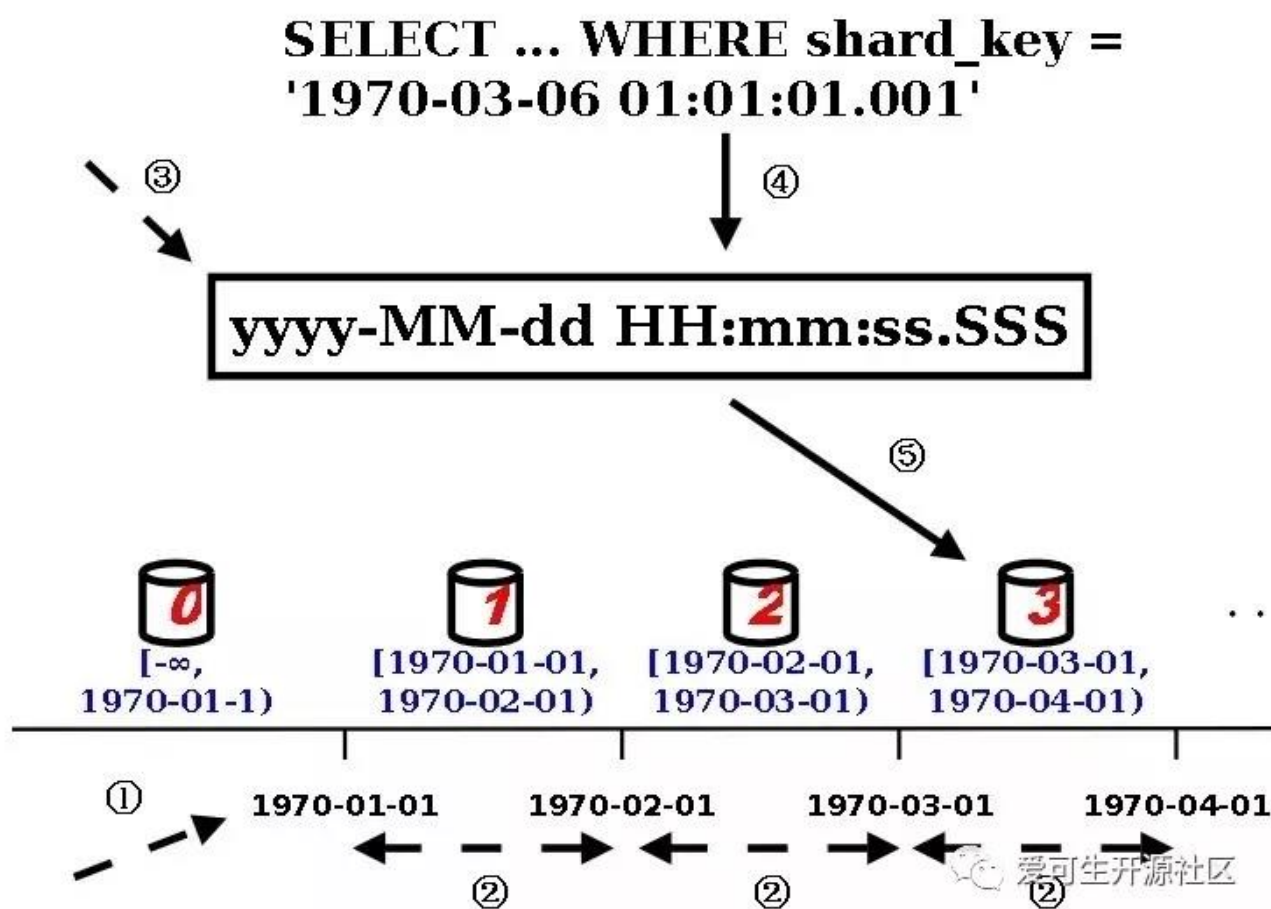
【配置项】6. mapFile 中逻辑分片范围的最小值非常重要，DBLE 读取 mapFile 时会对范围进行基于逻辑分片范围的最小值的插入排序，目前的最佳实践是人手确保范围与范围之间没有重叠



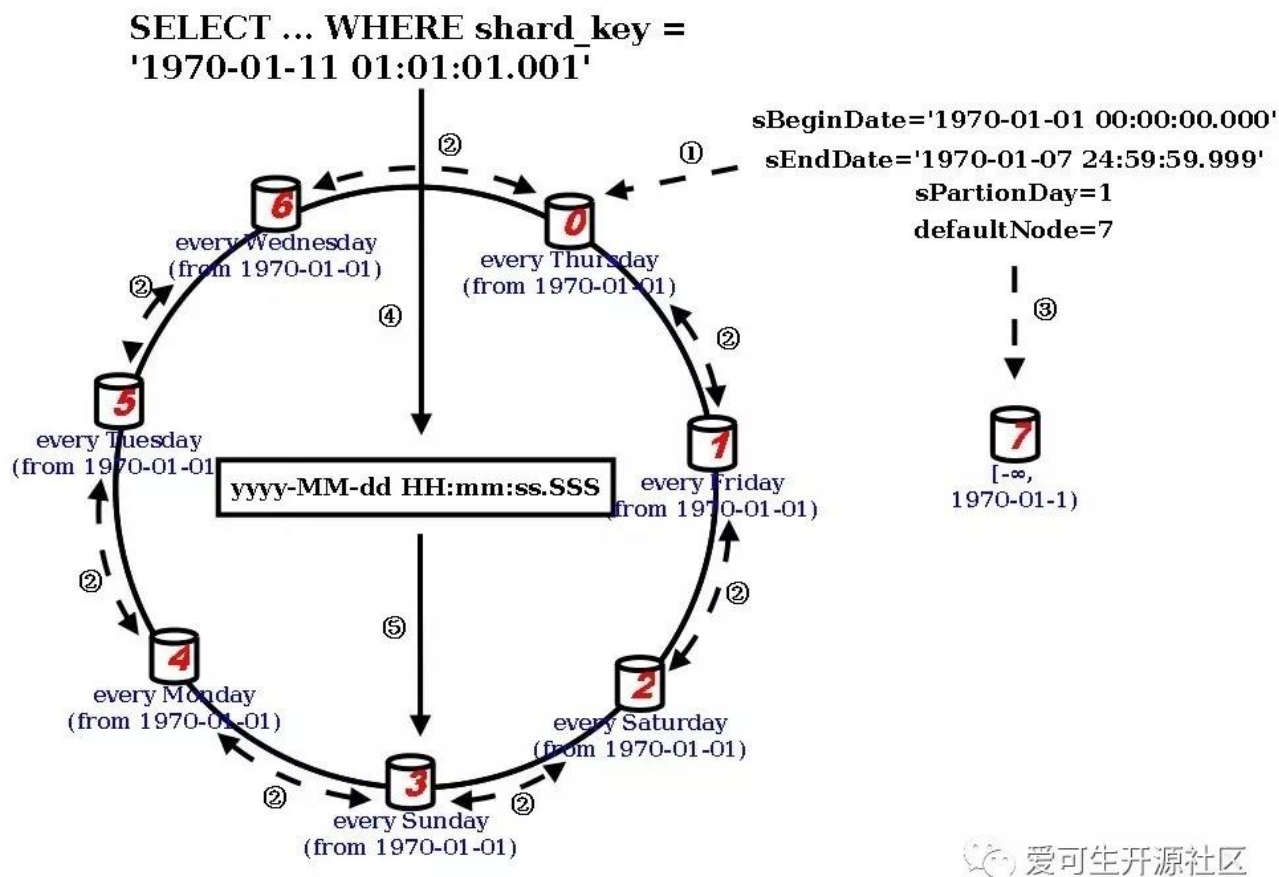
DBLE 与 MyCat 分片算法对比：date 分片

作者：钟悦

以每 24 小时作为一份时间（而非自然日），根据用户的配置有两种工作模式：带状模式中，用户仅定义开始日期时，从开始日期（含）开始，每份时间 1 个分片地无限增加下去；环状模式中，用户定义了开始日期和结束日期时，以结束日期（含）和开始日期（含）之间的时间份数作为分片总数（分片数量固定），以类似取模的方式路由到这些分片里。



1. DBLE 启动时，读取用户在 rule.xml 配置的 sBeginDate 来确定起始时间
2. 读取用户在 rule.xml 配置的 sPartitionDay 来确定每个 MySQL 分片承载多少天内的数据
3. 读取用户在 rule.xml 配置的 dateFormat 来确定分片索引的日期格式
4. 在 DBLE 的运行过程中，用户访问使用这个算法的表时，WHERE 子句中的分片索引值（字符串），会被提取出来尝试转换成 Java 内部的时间类型
5. 然后求分片索引值与起始时间的差，除以 MySQL 分片承载的天数，确定所属分片



1. DBLE 启动时，读取用户在 rule.xml 配置的起始时间 sBeginDate、终止时间 sEndDate 和每个 MySQL 分片承载多少天数据 sPartitionDay
2. 根据用户设置，建立起以 sBeginDate 开始，每 sPartitionDay 天一个分片，直到 sEndDate 为止的一个环，把分片串联串联起来
3. 读取用户在 rule.xml 配置的 defaultNode
4. 在 DBLE 的运行过程中，用户访问使用这个算法的表时，WHERE 子句中的分片索引值（字符串），会被提取出来尝试转换成 Java 内部的日期类型
5. 然后求分片索引值与起始日期的差：如果分片索引值不早于 sBeginDate（哪怕晚于 sEndDate），就以 MySQL 分片承载的天数为模数，对分片索引值求模得到所属分片；如果分片索引值早于 sBeginDate，就会被放到 defaultNode 分片上

1 与 MyCat 的类似分片算法对比

中间件	DBLE	MyCat
分片算法种类	date 分区算法	按日期 (天) 分片

两种中间件的取模范围分片算法使用上无差别

2 开发注意点

【分片索引】1. 必须是字符串, 而且 `java.text.SimpleDateFormat` 能基于用户指定的 `dateFormat` 来转换成 `java.util.Date`

【分片索引】2. 提供带状模式和环状模式两种模式

【分片索引】3. 带状模式以 `sBeginDate` (含) 起, 以 86400000 毫秒 (24 小时整) 为一份, 每 `sPartitionDay` 份为一个分片, 理论上分片数量可以无限增长, 但是出现 `sBeginDate` 之前的数据而且没有设定 `defaultNode` 的话, 会路由失败 (如果有 `defaultNode`, 则路由至 `defaultNode`)

【分片索引】4. 环状模式以 86400000 毫秒 (24 小时整) 为一份, 每 `sPartitionDay` 份为一个分片, 以 `sBeginDate` (含) 到 `sEndDate` (含) 的时间长度除以单个分片长度得到恒定的分片数量, 但是出现 `sBeginDate` 之前的数据而且没有设定 `defaultNode` 的话, 会路由失败 (如果有 `defaultNode`, 则路由至 `defaultNode`)

【分片索引】5. 无论哪种模式, 分片索引字段的格式化字符串 `dateFormat` 由用户指定

【分片索引】6. 无论哪种模式, 划分不是以日历时间为准, 无法对应自然月和自然年, 且会受闰秒问题影响

3 运维注意点

【扩容】1. 带状模式中, 随着 `sBeginDate` 之后的数据出现, 分片数量的增加无需再平衡

【扩容】2. 带状模式没有自动增添分片的能力, 需要运维手工提前增加分片; 如果路由策略计算出的分片并不存在时, 会导致失败

【扩容】3. 环状模式中, 如果新旧 `[sBeginDate, sEndDate]` 之间有重叠, 需要进行部分数据迁移; 如果新旧 `[sBeginDate, sEndDate]` 之间没有重叠, 需要数据再平衡

4 配置注意点

【配置项】1. 在 `rule.xml` 中, 可配置项为 `<propertyname="sBeginDate">`、`<propertyname="sPartitionDay">`、`<propertyname="dateFormat">`、`<propertyname="sEndDate">` 和 `<propertyname="defaultNode">`

【配置项】2. 在 `rule.xml` 中配置 `<propertyname="dateFormat">`, 符合 `java.text.SimpleDateFormat` 规范的字符串, 用于告知 DBLE 如何解析 `sBeginDate` 和 `sEndDate`

【配置项】3. 在 `rule.xml` 中配置 `<propertyname="sBeginDate">`, 必须是符合 `dateFormat` 的日期字符串

【配置项】4. 在 `rule.xml` 中配置 `<propertyname="sEndDate">`, 必须是符合 `dateFormat` 的日期字符串; 配置了该项使用的是环状模式, 若没有配置该项则使用的是带状模式

【配置项】5. 在 `rule.xml` 中配置 `<propertyname="sPartitionDay">`, 非负整数, 该分片策略以 8640000 毫秒 (24 小时整) 作为一份, 而 `sPartitionDay` 告诉 DBLE 把每多少份放在同一个分片

【配置项】6. 在 `rule.xml` 中配置 `<propertyname="defaultNode">` 标签, 非必须配置项, 不配置该项的话, 用户的分片索引值没落在 `mapFile` 定义



DBLE 与 MyCat 分片算法系列总结

作者：管长龙

我们已经通过之前的六篇文章，介绍了 DBLE 和 MyCat 常见的六种分片算法，那么我们来做个总结吧！

1 DBLE 与 MyCat 对应分片算法名和异同

DBLE 分片算法	MyCat分片算法	DBLE	MyCat
hash	固定分片 hash 取模	分片数最大支持 2880	count 和 length 两个向量的点乘积恒等于 1024
stringhash	截取数字 hash 解析	分片数最大支持 2880	count 和 length 两个向量的点乘积恒等于 1024
enum	分片枚举	无	无
numberrange	范围约定	写具体的数值	可用 K 或 M 等数量替换符
patternrange	取模范围约束	无	无
date	按日期 (天) 分片	无	无

2 第七种 DBLE 分片算法：jumpStringHash

除了以上六种常见的分片算法之外，DBLE 还独有一种分片算法：跳跃字符串算法。
具体配置如下：

```
#rule.xml
<function name="jumphash"
    class="jumpStringHash">
    <property name="partitionCount">2</property>
    <property name="hashSlice">0:2</property>
</function>
```

partitionCont：分片数量

hashSlice：分片截取长度

该算法来自于 Google 的一篇文章《A Fast, Minimal Memory, Consistent Hash Algorithm》其核心思想是通过概率分布的方法将一个 hash 值在每个节点分布的概率变成 $1/n$ ，并且可以通过更简便的方法可以计算得出，而且分布也更加均匀。

注意事项：分片字段值为 NULL 时，数据恒落在 0 号节点之上；当真实存在于 mysql 的字段值为 not null 的时候，报错 “Sharding column can't be null when the table in MySQL column is not null”

至此，我们 DBLE 中支持的七种分片算法就介绍完了，相信读者朋友对 DBLE 的使用也慢慢熟悉。



DBLE 和 Mycat 跨分片查询结果不一致案例分析

作者：杨严豪

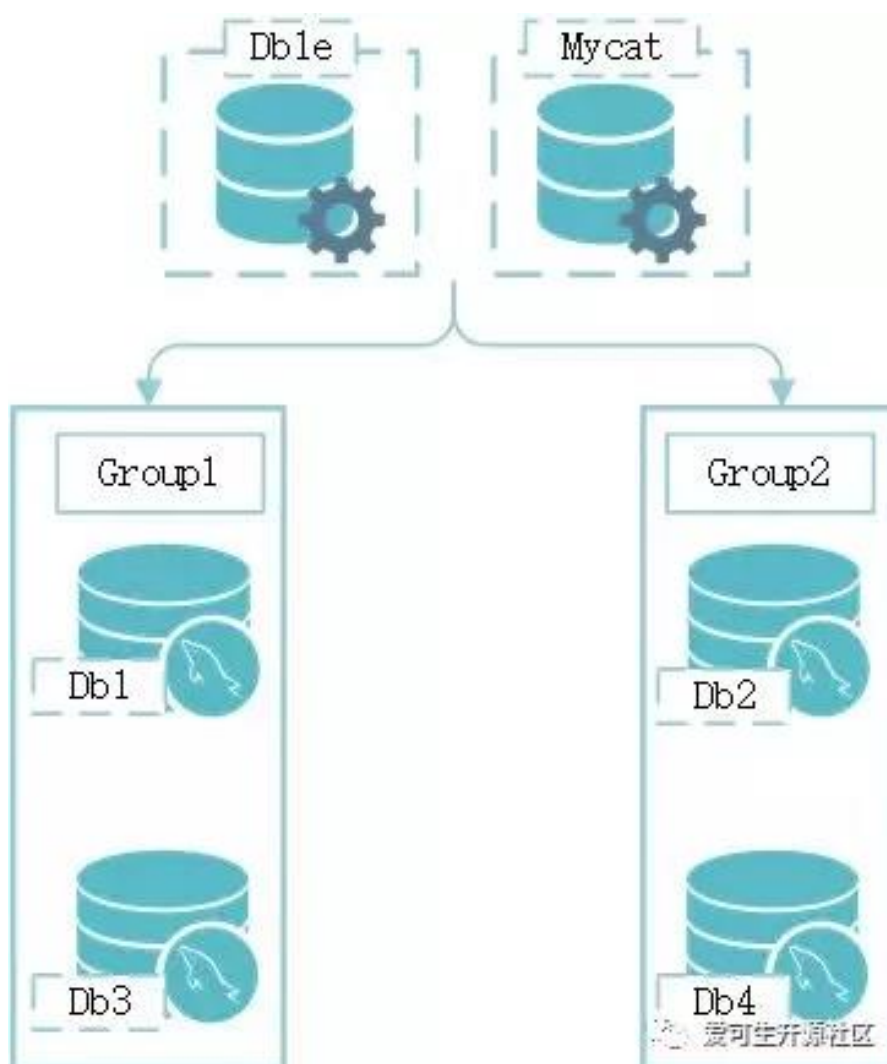
某一零售业后端使用了分布式中间件+ MySQL 数据库作为后端存储。但是因为历史问题存在两种分布式中间件，分别是 Mycat 和 DBLE，共用一组后端 MySQL 实例。分片规则以及后端数据完全一致。最近碰到了一个比较有意思的场景，财务结算单来往明细和业务来往单据的关联查询。一条跨节点 join 查询在 DBLE、Mycat 的查询得到的结果不一致。究竟谁对谁错？

1 环境准备

在虚拟机搭建类似架构，模拟场景，比较 Mycat-DBLE 在跨节点 join 上的异同点。

1.1 测试架构

测试环境架构比较简单，DBLE 与 Mycat 共用数据库。



1.2 测试软件版本

软件名称	软件版本	端口	管理端口
db1e	db1e-2.18.10.1-cb392c3-20181106093917	3309	3310
mycat	mycat-1.6-RELEASE-20161028204710	8066	9066
MySQL	5.7.21-log	3306	无

1.3 表结构

结算单来往明细表

```
CREATE TABLE `t_bl_detail` (  
  `unit_num` int(11) DEFAULT NULL,  
  `tenantid` int(11) DEFAULT NULL,  
  `detail_num` int(11) DEFAULT NULL,  
  `balance_date` datetime DEFAULT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

业务来往单据表

```
CREATE TABLE `t_bl_super_detail` (  
  `unit_num` int(11) DEFAULT NULL,  
  `sup_id` int(11) DEFAULT NULL,  
  `tenantid` int(11) DEFAULT NULL,  
  `bl_unit_num` int(11) DEFAULT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

1.4 分片规则配置

配置表 t_bl_detail、t_bl_super_detail，使用取模算法，数据分布在 db1-db4 四个 database 中。

DBLE - schema 配置

```
<schema name="testdb" sqlMaxLimit="100" dataNode="dn01">  
  <table name="t_bl_detail" rule="mod4Series" dataNode="dn01,dn02,dn03,dn04">  
    </table>  
  <table name="t_bl_super_detail" rule="mod4Series" dataNode="dn01,dn02,dn03,  
dn04"></table>  
</schema>  
  
<dataNode name="dn01" dataHost="group1" database="db1"></dataNode>  
<dataNode name="dn02" dataHost="group2" database="db2"></dataNode>  
<dataNode name="dn03" dataHost="group1" database="db3"></dataNode>  
<dataNode name="dn04" dataHost="group2" database="db4"></dataNode>
```

```

<dataHost name="group1" maxCon="100" minCon="5" balance="0" switchType="-1">
  <heartbeat>select user()</heartbeat>
  <writeHost host="mysql-w9zhkr" url="1.1.1.26:3306" user="ushard" password="
VRDkwsUlq++O3HeamsDcuW/2K22si3RIhcTpAdjalbkiinNAeUUDWk11ttls1Z3PXY5TxGJToeK4LsJE
3+k0Wg==" id="mysql-w9zhkr" usingDecrypt="1"></writeHost>
</dataHost>
<dataHost name="group2" maxCon="100" minCon="5" balance="0" switchType="-1">
  <heartbeat>select user()</heartbeat>
  <writeHost host="mysql-ci2va0" url="1.1.1.27:3306" user="ushard" password="
O/iCi0F+9ts5YR78ZcQnKzpb5GqG7xSUzJcO44yZ3BhNkT5E7aiZakmz1tbKIQylngq20vu5Z9egXBUC
3GOKow==" id="mysql-ci2va0" usingDecrypt="1"></writeHost>
</dataHost>

```

Mycat - schema 配置

```

<schema name="testdb" sqlMaxLimit="100" dataNode="dn01">
  <table name="t_bl_detail" rule="mod4Series" dataNode="dn01,dn02,dn03,dn04">
</table>
  <table name="t_bl_super_detail" rule="mod4Series" dataNode="dn01,dn02,dn03,
dn04"></table>
</schema>
<dataNode name="dn01" dataHost="group1" database="db1"></dataNode>
<dataNode name="dn02" dataHost="group2" database="db2"></dataNode>
<dataNode name="dn03" dataHost="group1" database="db3"></dataNode>
<dataNode name="dn04" dataHost="group2" database="db4"></dataNode>
<dataHost name="group1" maxCon="100" minCon="5" balance="0" switchType="-1">
  <heartbeat>select user()</heartbeat>
  <writeHost host="mysql-w9zhkr" url="1.1.1.26:3306" user="ushard" password="
VRDkwsUlq++O3HeamsDcuW/2K22si3RIhcTpAdjalbkiinNAeUUDWk11ttls1Z3PXY5TxGJToeK4LsJE
3+k0Wg=="
id="mysql-w9zhkr" usingDecrypt="1"></writeHost>
</dataHost>
<dataHost name="group2" maxCon="100" minCon="5" balance="0" switchType="-1">
  <heartbeat>select user()</heartbeat>
  <writeHost host="mysql-ci2va0" url="1.1.1.27:3306" user="ushard" password="
O/iCi0F+9ts5YR78ZcQnKzpb5GqG7xSUzJcO44yZ3BhNkT5E7aiZakmz1tbKIQylngq20vu5Z9egXBUC
3GOKow==" id="mysql-ci2va0" usingDecrypt="1"></writeHost>
</dataHost>

```

DBLE - rule 配置

```

<tableRule name="mod4Series">
  <rule>
    <columns>unit_num</columns>
    <algorithm>mod4DB</algorithm>
  </rule></tableRule><function name="mod4DB" class="Hash">

```

```
<property name="partitionCount">4</property>
<property name="partitionLength">1</property></function>
```

Mycat - rule 配置

```
<tableRule name="mod4Series">
  <rule>
    <columns>id</columns>
    <algorithm>mod4DB</algorithm>
  </rule></tableRule><function
class="io.mycat.route.function.PartitionByMod">
  <property name="count">4</property></function>
```

name="mod4DB"

2 对比开始

2.1 准备测试数据

登录任意一台中间件写入写入测试数据：

```
insert into t_bl_detail values(1,3,123443,'2019-01-01 00:00:00');
insert into t_bl_detail values(2,3,3423524,'2019-01-01 00:00:00');
insert into t_bl_detail values(3,3,245245,'2019-01-01 00:00:00');
insert into t_bl_detail values(4,4,356356,'2019-01-01 00:00:00');
insert into t_bl_super_detail values(1,10342,3,2);
insert into t_bl_super_detail values(2,12355,3,2);
insert into t_bl_super_detail values(3,62542,3,3);
insert into t_bl_super_detail values(4,74235,4,1);
```

```
mysql> select * from t_bl_detail;
+-----+-----+-----+-----+
| unit_num | tenantid | detail_num | balance_date |
+-----+-----+-----+-----+
| 4 | 4 | 356356 | 2019-01-01 00:00:00 |
| 1 | 3 | 123443 | 2019-01-01 00:00:00 |
| 2 | 3 | 3423524 | 2019-01-01 00:00:00 |
| 3 | 3 | 245245 | 2019-01-01 00:00:00 |
+-----+-----+-----+-----+
4 rows in set (0.08 sec)

mysql> select * from t_bl_super_detail;
+-----+-----+-----+-----+
| unit_num | sup_id | tenantid | bl_unit_num |
+-----+-----+-----+-----+
| 4 | 74235 | 4 | 1 |
| 1 | 10342 | 3 | 2 |
| 2 | 12355 | 3 | 2 |
| 3 | 62542 | 3 | 3 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

2.2 执行跨节点 join 查询

```
select m.unit_num,m.balance_date from t_bl_detail m join t_bl_super_detail n
where n.tenantid = 3 and m.unit_num = n.bl_unit_num;
```

在通过中间件之前，现在 MySQL 中执行一遍看下结果，作为预期结果供后续案例使用。

```
mysql> select version();
+-----+
| version() |
+-----+
| 5.7.21-log |
+-----+
1 row in set (0.00 sec)

mysql> select m.unit_num, m.balance_date from t_bl_detail m join t_bl_super_detail n where
n.tenantid = 3 and m.unit_num = n.bl_unit_num;
+-----+-----+
| unit_num | balance_date |
+-----+-----+
| 2 | 2019-01-01 00:00:00 |
| 2 | 2019-01-01 00:00:00 |
| 3 | 2019-01-01 00:00:00 |
+-----+-----+
3 rows in set (0.00 sec)
```

 爱可生开源社区

分别通过 DBLE、Mycat 执行跨节点 join 语句。

DBLE 执行跨节点 join 语句

```
mysql> select version();
+-----+
| VERSION() |
+-----+
| 5.7.21-dble-2.18.10.3-e570f91-20190118145110 |
+-----+
1 row in set (0.00 sec)

mysql> select m.unit_num, m.balance_date from t_bl_detail m join t_bl_super_detail n where
n.tenantid = 3 and m.unit_num = n.bl_unit_num;
+-----+-----+
| unit_num | balance_date |
+-----+-----+
| 2 | 2019-01-01 00:00:00 |
| 2 | 2019-01-01 00:00:00 |
| 3 | 2019-01-01 00:00:00 |
+-----+-----+
3 rows in set (0.00 sec)
```

 爱可生开源社区

Mycat 执行跨节点 join 语句

```
mysql> select version();
+-----+
| VERSION() |
+-----+
| 5.7.21-mycat-1.6-RELEASE-20161028204710 |
+-----+
1 row in set (0.00 sec)

mysql> select m.unit_num, m.balance_date from t_bl_detail m join t_bl_super_detail n where
n.tenantid = 3 and m.unit_num = n.bl_unit_num;
+-----+-----+
| unit_num | balance_date |
+-----+-----+
| 2 | 2019-01-01 00:00:00 |
| 3 | 2019-01-01 00:00:00 |
+-----+-----+
2 rows in set (0.00 sec)
```

可看到相同的查询语句，DBLE 执行结果符合预期，Mycat 执行结果缺失。数据差异在于 DBLE 查询结果相较于 Mycat 多了跨节点的结果。虽然 Mycat 执行跨节点 join 不报错，但是查询结果却和预期不一致。

2.3 执行计划

只从结果上判断并没有办法知道是什么原因导致了 Mycat 结果缺失，查看查询计划，比较两者差异。

通过 DBLE 的执行计划可看出，DBLE 内部分别对结算明细表、业务单据表做了各自的数据查询，将查询结果在中间层做了 merge。最后获得跨节点 join 的结果。

```
mysql> explain select m.unit_num, m.balance_date from t_bl_detail m join t_bl_super_detail n where n.tenantid = 3 and m.unit_num = n.bl_unit_num;
+-----+-----+-----+
| DATA_NODE | TYPE | SQL/REF |
+-----+-----+-----+
| dn01_0 | BASE SQL | select 'm'.unit_num,'m'.balance_date from 't_bl_detail' 'm' ORDER BY 'm'.unit_num ASC |
| dn02_0 | BASE SQL | select 'm'.unit_num,'m'.balance_date from 't_bl_detail' 'm' ORDER BY 'm'.unit_num ASC |
| dn03_0 | BASE SQL | select 'm'.unit_num,'m'.balance_date from 't_bl_detail' 'm' ORDER BY 'm'.unit_num ASC |
| dn04_0 | BASE SQL | select 'm'.unit_num,'m'.balance_date from 't_bl_detail' 'm' ORDER BY 'm'.unit_num ASC |
| merge_1 | MERGE | dn01_0; dn02_0; dn03_0; dn04_0 |
| shuffle_field_1 | SHUFFLE_FIELD | merge_1 |
| dn01_1 | BASE SQL | select 'n'.bl_unit_num from 't_bl_super_detail' 'n' where n.tenantid = 3 ORDER BY 'n'.bl_unit_num ASC |
| dn02_1 | BASE SQL | select 'n'.bl_unit_num from 't_bl_super_detail' 'n' where n.tenantid = 3 ORDER BY 'n'.bl_unit_num ASC |
| dn03_1 | BASE SQL | select 'n'.bl_unit_num from 't_bl_super_detail' 'n' where n.tenantid = 3 ORDER BY 'n'.bl_unit_num ASC |
| dn04_1 | BASE SQL | select 'n'.bl_unit_num from 't_bl_super_detail' 'n' where n.tenantid = 3 ORDER BY 'n'.bl_unit_num ASC |
| merge_2 | MERGE | dn01_1; dn02_1; dn03_1; dn04_1 |
| shuffle_field_3 | SHUFFLE_FIELD | merge_2 |
| join_1 | JOIN | shuffle_field_1; shuffle_field_3 |
| shuffle_field_2 | SHUFFLE_FIELD | join_1 |
+-----+-----+-----+
14 rows in set (0.00 sec)
```

Mycat 对于跨节点 join 的处理则相对暴力，直接将查询语句下发到各个节点，最后将结果进行汇总，如果表连接涉及到跨节点。则跨节点的数据无法进行 join。

```
mysql> explain select m.unit_num, m.balance_date from t_bl_detail m join t_bl_super_detail n where n.tenantid = 3 and m.unit_num = n.bl_unit_num;
+-----+-----+
| DATA_NODE | SQL |
+-----+-----+
| dn01 | select m.unit_num, m.balance_date from t_bl_detail m join t_bl_super_detail n where n.tenantid = 3 and m.unit_num = n.bl_unit_num |
| dn02 | select m.unit_num, m.balance_date from t_bl_detail m join t_bl_super_detail n where n.tenantid = 3 and m.unit_num = n.bl_unit_num |
| dn03 | select m.unit_num, m.balance_date from t_bl_detail m join t_bl_super_detail n where n.tenantid = 3 and m.unit_num = n.bl_unit_num |
| dn04 | select m.unit_num, m.balance_date from t_bl_detail m join t_bl_super_detail n where n.tenantid = 3 and m.unit_num = n.bl_unit_num |
+-----+-----+
4 rows in set (0.00 sec)
```


3 总结

Mycat 是一款非常优秀的分布式中间件，但是在某些细节方面处理的不尽人意。在跨节点关联查询场景下，Mycat 采取的策略是直接将语句透传到各个节点上，将获取到的结果整合后返回，得到的结果集和预期结果有出入，缺失了跨节点关联的数据。

DBLE 处理跨节点的关联查询是先获取到关联需要的数据，提取到中间件进行融合，得到关联查询的结果并返回。得到的结果集符合预期，与 MySQL 执行结果一致。可见 DBLE 在跨节点关联查询方面做了优化，能够提供准确的查询结果。



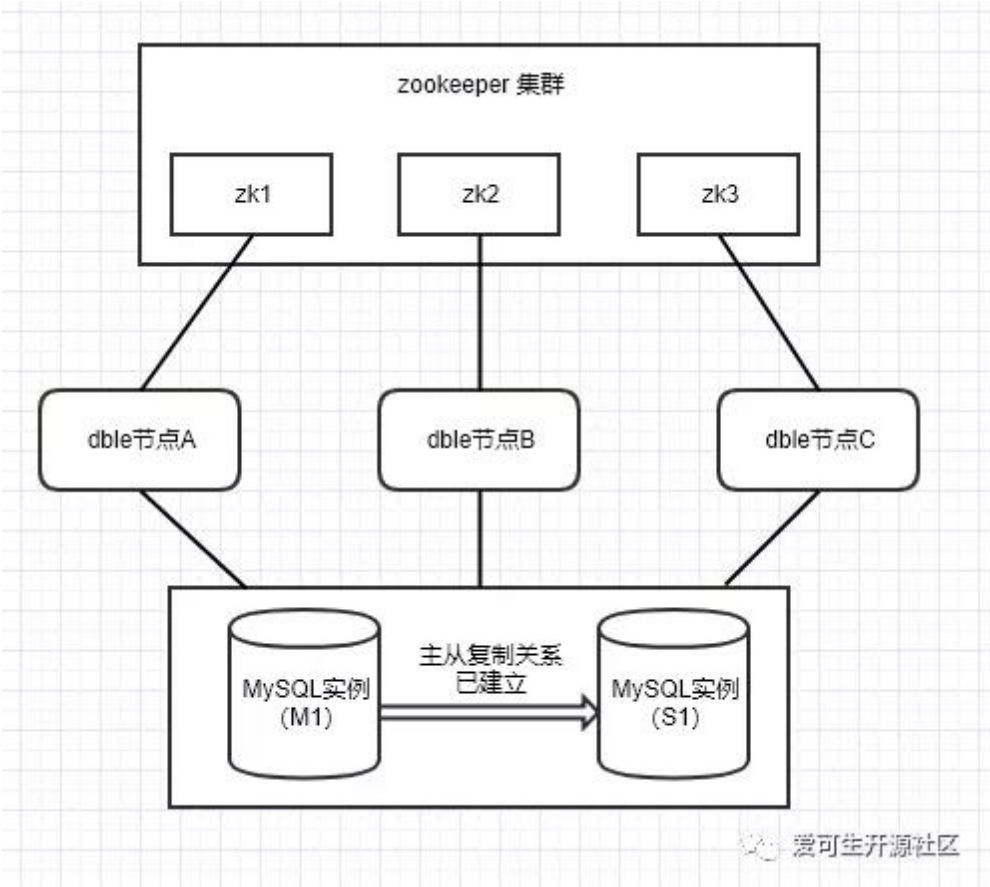
ZooKeeper 快速部署 DBLE 集群环境

作者：叶晓莉

ZooKeeper - 分布式协调技术 / DBLE - 分布式数据库中间件，使用二者搭建集群，能够为 MySQL 集群提供稳定的分布式协调环境，进一步实现服务高可用。

说明：ZooKeeper（以下简称 ZK）可以单例管理 DBLE 集群，此文我们使用 ZK 的集群模式（至少 3 台）管理 DBLE 集群。利用 ZK 集群实现 DBLE 集群中各节点的状态和元数据的一致性，当集群中有节点故障时，可使用其他节点继续提供服务。通过实验，验证此架构的可行性。接下来介绍具体步骤。

1 环境架构



环境版本信息：
系统 | Linux : CentOS Linux release 7.5.1804 (Core)
中间件 | DBLE: 2.19.05.0
数据库 | MySQL : 5.7.13
协调器 | ZooKeeper: 3.4.12

2 部署依赖（重要）

1. 安装 JDK 版本 1.8 或以上，确保 JAVA_HOME 配置正确
2. 启动 MySQL 实例，创建好用户，并对该用户授权远程登陆权限

3 安装 DBLE（3 台）

1. 下载安装包

```
wget https://github.com/actiontech/dble/releases/download/2.19.05.0%2Ftag/actiontech-dble-2.19.05.0.tar.gz
```

2. 解压安装包

```
tar zxvf actiontech-dble-2.19.05.0.tar.gz
```

3. 配置 DBLE 后端节点，修改 dble/conf/ 目录下 schema.xml 文件，配置 writeHost, readHost 为自己的 MySQL 实例，如下：

```
<writeHost host="hostM1" url="ip1:3306" user="your_user" password="your_psw">
    <readHost host="hostS1" url="ip2:3306" user="your_user" password="your_psw"
/>
</writeHost>
```

4 部署 ZooKeeper 集群

1. 下载并解压安装包，官网链接：

```
http://archive.apache.org/dist/zookeeper/
```

2. 正确配置 ZOOKEEPER_HOME
3. 配置 zoo.cfg 将 zookeeper-3.4.12/conf 目录下的 zoo_sample.cfg 重命名为 zoo.cfg，修改如下：

```
tickTime=2000
initLimit=10
syncLimit=5
#zookeeper 客户端的端口
clientPort=2181
#zookeeper 的 data 目录路径
dataDir=/opt/zookeeper/data
#zookeeper 的日志目录的路径
dataLogDir=/opt/zookeeper/logs
#id 为 ServerID，用来标识服务器在集群中的序号，server_ip 为服务器所在的 ip 地址
server.id=zk1_server_ip:2888:3888
server.id=zk2_server_ip:2888:3888
server.id=zk3_server_ip:2888:3888
```

4. 其他两台 ZK 做相同配置
5. 创建 myid 文件，在 zookeeper-3.4.12/data 目录下创建文件 myid，该文件只有一行内容，即对应于每台服务器的 Server ID
6. 启动集群，在每台机器上启动 ZK 服务

```
cd zookeeper-3.4.12/bin && ./zkServer.sh start
```

7. 验证 ZK 集群部署成功，查看每台服务器在集群中的角色，其中一台为 Leader，其他两台为 Follower

```
cd zookeeper-3.4.12/bin && ./zkServer.sh status
```

5 部署 DBLE 集群

1. 在 dble/conf 目录下编辑文件 myid.properties 如下：

```
cluster=zk
#clinet info
ipAddress=zk_server_ip:zk_port
#cluster namespace, please use the same one in one cluster
clusterId=cluster-1
#it must be different for every node in cluster
myid=1
```

说明：ipAddress 可以配置集群中 1 个或多个或所有 ZK 的 ip 和 port，配置多个 ip 时需要用 “,” 隔开，举例：Address=172.100.9.1:2181,172.100.9.2:2181

详情可参考文档：

https://github.com/actiontech/dble-docs-cn/blob/master/1.config_file/1.08_myid.properties.md

2. 其他两台 DBLE 做相同配置，特别指出：每台 DBLE 的 myid 值必须不同
3. 启动所有 DBLE

```
cd dble/bin && ./dbled start
```

4. 通过 ZK 客户端验证 DBLE 集群部署成功，在任何一台装有 ZK 的机器上登 ZK 的客户端，查看所有 DBLE 都在线，DBLE 集群部署成功！

```
cd zookeeper-3.4.12/bin && ./zkCli.sh
[zk: localhost:2181(CONNECTED) 0] ls /dble/cluster-1/online
[001, 3, server_2]
```

6 简单测试

场景 1: reload @@config_all 命令

目的: 验证 reload @@config_all 命令能把配置同步到集群中所有节点

步骤一: 修改 DBLE-A 的 schema.xml, 添加一个全局表, 如下:

```
<table name="test_global" dataNode="dn1,dn2" type="global"/>
```

步骤二: 连接 DBLE-A 管理端口, 执行管理命令: reload @@config_all;

步骤三: 查看 DBLE-B 和 DBLE-C 的 schema.xml, 观察到 test_global 已经更新到集群中其他节点

结论: DBLE 集群中主节点更新的配置, 在执行 reload @@config_all 之后会下推到其他节点。

场景 2: 视图同步

目的: 验证通过 DBLE 创建的视图只保存在 ZK 上, 并在集群所有节点同步, 不存在 MySQL 中

步骤一: 连接 DBLE-A 业务端口, 创建视图, 如下:

```
mysql> create view view_test as select * from test_global;
Query OK, 0 rows affected (0.10 sec)
mysql> show tables;
+-----+
| Tables in schemal |
+-----+
| test_global      |
| view_test        |
+-----+
2 rows in set (0.00 sec)
```

步骤二: 登陆 ZK 客户端查看, 创建的视图已存在 ZK view 目录下

```
[zk: localhost:2181(CONNECTED) 5] ls /dbale/cluster-1/view
[schemal:view_test]
```

步骤三: 连接 DBLE-B 业务端口, 观察到视图存在于该节点

```
mysql> show tables;
+-----+
| Tables in schemal |
+-----+
| test_global      |
| view_test        |
+-----+
2 rows in set (0.00 sec)
```

步骤四: 直连 MySQL, 观察到创建的视图并没有存在 MySQL 中

```
mysql> show tables;
+-----+
| Tables in schema1 |
+-----+
| test_global      |
+-----+
1 rows in set (0.00 sec)
```

结论：通过 DBLE 节点创建的视图存储在 ZK 中，并在集群中所有节点同步，不存在 MySQL 中。

场景 3：当集群中有节点掉线

目的：验证当集群中有节点故障时，不影响集群向外提供服务

步骤一：手动停掉 DBLE-A

步骤二：连接 DBLE-C 业务端口，修改 test_global 表结构

```
mysql> alter table test_global add column name varchar(20);
Query OK, 0 rows affected (0.15 sec)
```

步骤三：连接 DBLE-B 业务端口，查看 test_global 表结构

```
mysql> desc test_global;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int(11)       | YES  |     | NULL    |       |
| name  | varchar(20)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

结论：DBLE 集群中，当其他节点掉线或故障时，不影响其他节点提供服务！

需要了解集群同步状态的操作和细节，请查看开源文档：

https://actiontech.github.io/dble-docs-cn/2.Function/2.08_cluster.html

微信扫一扫阅读原文



DBLE 沿用 jumpstringhash 移除 Mycat 一致性 hash 原因解析

作者：孙正方

1 背景介绍

MyCat 对于字符串类型为分片字段的数据，有三种分片模式，分别是：模值 hash（求模法），jumpstringhash（跳跃法），一致性 hash（环割法）。DBLE 对于 hash 算法选取方面，除了继承 MyCat 的模值 hash，并没有延续使用 MyCat 的一致性 hash，而是推荐使用性能更佳、均衡性更好的“jumpstringhash”算法。下面对于环割法（一致性 hash）及跳跃法（jumpstringhash）的原理、特性及优缺点进行简单的介绍。

2 环割法（一致性 hash）

环割法的原理如下：

- 1. 初始化的时候生成分片数量 $X \times$ 环割数量 N 的固定方式编号的字符串，例如 SHARD-1-NODE-1，并计算所有 $X \times N$ 个字符串的所有 hash 值。
- 2. 将所有计算出来的 hash 值放到一个排序的 Map 中，并将其中的所有元素进行排序。
- 3. 输入字符串的时候计算输入字符串的 hash 值，查看 hash 值介于哪两个元素之间，取小于 hash 值的那个元素对应的分片为数据的分片。

特点	缺点
随机性强	初始化耗时长，内存消耗较高，需要进行大量数据计算，可分片消耗高

3 跳跃法（jumpstringhash）

跳跃法的原理如下：

- 1. 公式：

$$\frac{1}{x} - (1 - \frac{1}{x+1}) \times (1 - \frac{1}{x+2}) \times \dots \times (1 - \frac{1}{n-1}) \times (1 - \frac{1}{n}) \times \frac{1}{n}$$

将数据落在每一个节点的概率进行平均分配。

- 2. 对于输入的字符串进行计算 hash 值，通过判断每次产生的伪随机值是否小于当前判定的节点 $1/x$ ，最终取捕获节点编号最大的作为数据的落点。
- 3. 在实际使用中使用倒数的方法从最大节点值进行反向判断，一旦当产生的伪随机值大于 x 则判定此

节点 x 作为数据的落点。

特点

内存消耗小，均衡性高，计算量相对较小

爱可生开源社区

4 数据比较

下面将通过测试对环割法和跳跃法的性能及均衡性进行对比，说明 DBLE 为何使用跳跃法代替了环割法。

数据源：现场数据 350595 条

测试经过

1. 通过各自的测试方法执行对于测试数据的分片任务。
2. 测试方法：记录分片结果的方差；记录从开始分片至分片结束时间；记录分片结果与平均数的最大差值。
3. 由于在求模法 PartitionByString 的方法中要求分片的数量是 1024 的因数，所以测试过程只能使用 2 的指数形式进行测试，并在 PartitionByString 方法进行测试的时候不对于 MAC 地址进行截断，取全量长度进行测试。

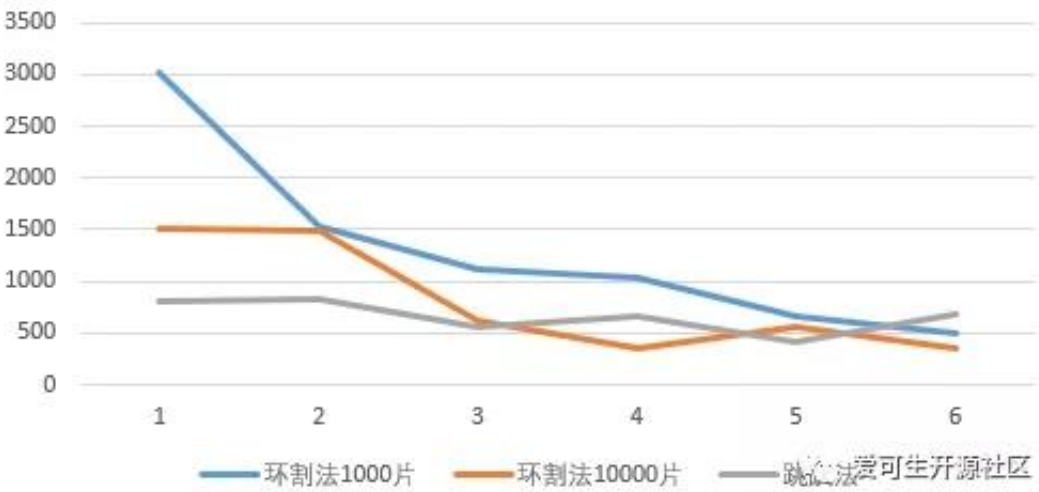
测试结果

1. 使用不同方法时，方差随分片数量的情况变化表

分片数量\方差	环割法1000片	环割法10000片	跳跃法
4	4484780	812418	315703
8	601593	545599	315587
16	453694	131816	67018
32	213856	74360	86125
64	69313	46618	37939
128	24329	26415	25429

爱可生开源社区

节点分配数据量与平均数据量的最大偏差和分片数量间的关系



summary

- 在同样分片数据量的情况下，两种方法的方差都会随着分片数量的增加而减少。
- 在分片数量足够多的情况下，两种方法并没有太大的区别。
- 在节点数量相对较少的情况下，跳跃法的均衡性最好。
- 使用环割法时，增加环切的数量能够提高分片的均衡性，但是均衡性提高的幅度会随着分片数量的增加而减少。

2. 使用不同方法时，消耗的时间随着分片数量的情况变化表

分片数量\耗时	环割法1000片	环割法10000片	跳跃法
4	0.419	0.532	0.069
8	0.41	0.425	0.067
16	0.512	0.545	0.06
32	0.55	0.525	0.07
64	0.683	0.59	0.087
128	0.559	0.659	0.094



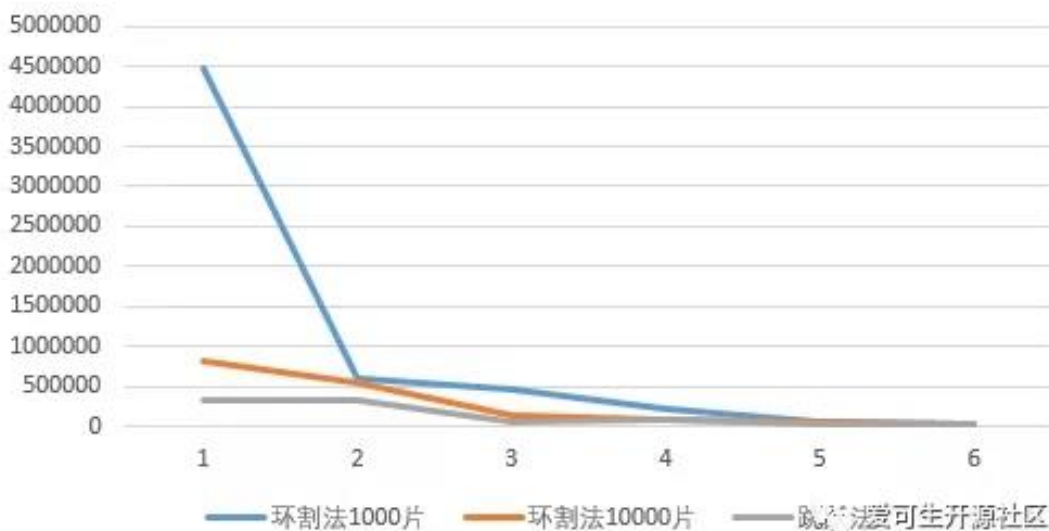
summary

- 环割法相比于跳跃法有较大的性能差距，大约相差一个数量级。
- 跳跃法的时间消耗会随着分片数量的增加而小幅度的上升。
- 环割法的时间消耗随着环割数量的增加而小幅度的增加。

3. 使用不同方法时，节点分配数据量和平均数据量的最大偏差和分片数量的情况变化表

分片数量\最大平均数偏离	环割法1000片	环割法10000片	跳跃法
4	3027	1513	802
8	1530	1488	822
16	1121	622	556
32	1044	358	672
64	673	560	424
128	503	348	312

方差随分片数量变化图



summary

1. 两种方法的最大偏差都会随着分片数量的上升而下降，最后当分片数量足够多的时候，两种方法并没有明显的区别。
2. 在分片数量少的时候，跳跃法的均衡性最好，环割法相对最差。
3. 使用环割法时，增加环切的数量能够提高分片的均衡性，但是均衡性提高的幅度会随着分片数量的增加而减少。

综上所述, 从一致性 hash 的多种测试结果来看，都没有很好的性能表现。因此，DBLE 在 hash 算法选取方面，使用 jumpstringhash 代替了一致性 hash。

如果您是从 Mycat 迁移过来的，使用了一致性 hash，可以通过自定义拆分算法来实现。

自定义拆分算法相关链接：

https://github.com/actiontech/dble-docs-cn/blob/master/1.config_file/1.09_dble_route_function_spec.md

开源产品

开源式分布式中间件 DBLE

破壳日:2017.10.24

技能:数据水平拆分、读写分离、分布式事务支持、多分片算法、全局 ID、IP/SQL 黑白名单

特长:MySQL 语法兼容、复杂查询优化、低改造成本、成熟稳定、成熟技术栈



开源数据传输组件 DTLE

破壳日:2018.10.24

技能:MySQL 全量增量复制迁移, 双向复制、数据库汇聚、库表转换、数据订阅

特长:多场景多网络适配、云间迁移利器、跨数据中心数据同步



开源分布式事务框架 TXLE

破壳日:2019.10.24

技能:微服务分布式事务框架, 提供数据最终一致性、全局事务重试、超时、降级、异常快照

特长:兼容主流 (SpringCloud、Dubbo 等) 框架、提供多种一致性保障手段



爱可生开源社区:

<https://opensource.actionsky.com>



关注微信公众号
获取最新的技术分享



社区吉祥物:雍正